เอกสารคำสอน
กระบวนวิชา 204355 การโปรแกรมเชิงแข่งขัน

Competitive Programming

โดย จักริน ชวชาติ

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์
มหาวิทยาลัยเชียงใหม่

คำนำ

เอกสารฉบับนี้จัดทำขึ้นเพื่อรวบรวมเอกสารต่างๆ ที่ใช้ประกอบการเรียนการสอนในกระบวนวิชา 204355 การโปรแกรมเชิงแข่งขัน ซึ่งเปิดสอนที่ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยเชียงใหม่ ตั้งแต่ปี พ.ศ. 2560 เป็นต้นมา ทั้งนี้ในแต่ละปี เนื้อหาที่ใช้ในการสอนอาจมีการปรับเพิ่มหรือลดให้เหมาะสมกับบริบทและสถานการณ์ในช่วงเวลานั้น ๆ จนกระทั่งปัจจุบันจึงได้รวบรวมและเรียบเรียงเนื้อหาทั้งหมดให้เป็นเอกสารฉบับเดียว เพื่อความสะดวกในการศึกษาและใช้อ้างอิง

เนื้อหาในช่วงเริ่มต้นอ้างอิงจากหนังสือ Competitive Programming โดย Steven Halim และ Felix Halim [3] และต่อมาได้เพิ่มเติมจากหนังสือ Competitive Programmer's Handbook โดย Antti Laaksonen[5] ซึ่งเป็นแหล่งอ้างอิงหลักสองเล่ม จากนั้นจึงมีการเสริมเนื้อหาในประเด็นอื่น ๆ ตามความเหมาะสม สำหรับโจทย์ที่ใช้ประกอบการเรียนการสอนนั้น มีทั้งโจทย์ที่ผู้สอนออกแบบขึ้นเอง และโจทย์ที่ดัดแปลงหรือแปลมาจากแพลตฟอร์มออนไลน์ต่าง ๆ เช่น UVa Online Judge[20] เป็นต้น

ผู้เรียบเรียงหวังว่าเอกสารคำสอนฉบับนี้จะเป็นประโยชน์ในการใช้ประกอบการเรียนการสอนในกระบวนวิชา 204355 การโปรแกรมเชิงแข่งขัน เพื่อช่วยพัฒนาทักษะและความสามารถของนักศึกษาภายในภาควิชา อีกทั้งยังสามารถใช้เป็นเอกสารอ่านเสริมสำหรับการเตรียมตัวเข้าร่วมกิจกรรมที่เกี่ยวกับการโปรแกรมเชิงแข่งขันหรือค่ายโอลิมปิกวิชาการ รวมถึงผู้สนใจทั่วไป ได้อีกด้วย

ท้ายที่สุดนี้ หากเอกสารฉบับนี้สามารถช่วยให้ผู้อ่านเรียนรู้ได้สะดวกขึ้น และนำไปประยุกต์ใช้ได้จริง ผู้เรียบเรียงยินดีเป็นอย่างยิ่ง และหวังว่าจะเป็นประโยชน์ไม่มากก็น้อยแก่ทุกท่าน

จักริน ชวชาติ

Contents

คำนำ 2

1 บทนำเกี่ยวกับการเขียนโปรแกรมเชิงแข่งขัน 10

1 บทนำสู่การโปรแกรมเชิงแข่งขัน 12
- 1.1 การโปรแกรมเชิงแข่งขันคืออะไร . . . 12
- 1.2 แพลตฟอร์มการแข่งขัน . . . 12
- 1.3 ภาษาโปรแกรมและการตั้งค่า . . . 14
- 1.4 การรับค่าและแสดงผล . . . 15
- 1.5 การทำงานกับตัวเลข . . . 17
- 1.6 เทคนิคย่อโค้ด . . . 20
- 1.7 รูปแบบการรับค่าเพิ่มเติม . . . 21
- 1.8 โจทย์แนะนำ . . . 24
- 1.9 สรุปบทที่ 1 . . . 25

2 ความซับซ้อนของเวลา 26
- 2.1 ความซับซ้อนของเวลาคืออะไร . . . 26
- 2.2 สัญกรณ์ Big-O . . . 28
- 2.3 สัญกรณ์ Omega และ Theta . . . 31
- 2.4 การวิเคราะห์ความซับซ้อนพื้นฐาน . . . 32
- 2.5 การวิเคราะห์ขั้นตอนวิธีแบบเรียกซ้ำ . . . 36
- 2.6 การประมาณค่าจากข้อจำกัดโจทย์ . . . 39
- 2.7 ตารางอ้างอิงเลือกขั้นตอนวิธีตามข้อจำกัด . . . 42
- 2.8 ตัวอย่างรวม . . . 44
- 2.9 การวิเคราะห์ Amortized . . . 48
- 2.10 ความซับซ้อนของหน่วยความจำ . . . 51
- 2.11 โจทย์แนะนำ . . . 53
- 2.12 สรุปบทที่ 2 . . . 54

3 การเรียงลำดับและการค้นหาแบบทวิภาค 57
- 3.1 การเรียงลำดับด้วยไลบรารีมาตรฐาน C++ . . . 57
- 3.2 ตัวเปรียบเทียบ (Comparison Operators) . . . 61
- 3.3 ขั้นตอนวิธีการเรียงลำดับพื้นฐาน . . . 66
- 3.4 ขั้นตอนวิธีการเรียงลำดับขั้นสูง . . . 69

3.5 การค้นหาแบบทวิภาค (Binary Search) . . . 73
3.6 การประยุกต์ Binary Search ขั้นสูง . . . 79
3.7 โจทย์แนะนำ . . . 88
3.8 สรุปบทที่ 3 . . . 90

4 แบ่งแยกและเอาชนะ (Divide and Conquer) 92
4.1 หลักการของ Divide and Conquer . . . 92
4.2 Merge Sort . . . 93
4.3 Quick Sort . . . 97
4.4 การนับ Inversion ด้วย Merge Sort . . . 99
4.5 การเข้ารหัส Bitmap (Bitmap Encoder) . . . 100
4.6 Master Theorem . . . 103
4.7 Closest Pair of Points . . . 105
4.8 การยกกำลังแบบเร็ว (Fast Exponentiation) . . . 108
4.9 โจทย์แนะนำ . . . 111
4.10 สรุปบทที่ 4 . . . 111

2 โครงสร้างข้อมูลขั้นสูง 113

5 โครงสร้างข้อมูล 115
5.1 แนะนำโครงสร้างข้อมูล . . . 115
5.2 แถวลำดับและ Vector . . . 116
5.3 String และ Iterator . . . 121
5.4 Bitset . . . 124
5.5 Stack และ Queue . . . 126
5.6 Set และ Multiset . . . 131
5.7 Map . . . 136
5.8 Heap และ Priority Queue . . . 139
5.9 Linked List และ Hash Table . . . 143
5.10 Bitmask เป็นเซตเบา (Lightweight Boolean Set) . . . 146
5.11 Segment Tree . . . 149
5.12 Fenwick Tree (Binary Indexed Tree) . . . 153
5.13 โจทย์แนะนำ . . . 156
5.14 สรุปบทที่ 5 . . . 156

6 การสอบถามแบบช่วง (Range Queries) 160
6.1 บทนำสู่ Range Queries . . . 160
6.2 Static Array Queries . . . 161

6.3 Binary Indexed Tree (Fenwick Tree) . . . 167
6.4 Segment Tree . . . 170
6.5 Lazy Propagation . . . 174
6.6 เทคนิคเพิ่มเติม . . . 177
6.7 โจทย์แนะนำ . . . 178
6.8 สรุปบทที่ 6 . . . 179

7 Complete Search 180
7.1 แนะนำ Complete Search . . . 180
7.2 การสร้างชุดย่อย (Generating Subsets) . . . 183
7.3 การสร้างการเรียงสับเปลี่ยน (Generating Permutations) . . . 188
7.4 Backtracking . . . 190
7.5 การตัดแต่งกิ่ง (Pruning) . . . 194
7.6 Meet in the Middle . . . 197
7.7 Generating vs. Filtering และ Pre-Computation . . . 201
7.8 โจทย์แนะนำ . . . 206
7.9 สรุปบทที่ 7 . . . 207

3 ชุดปัญหาด้านคณิตศาสตร์ 209

8 คณิตศาสตร์พื้นฐาน 211
8.1 สัญลักษณ์และสัญกรณ์คณิตศาสตร์ . . . 211
8.2 ผลบวกและผลคูณ (Summation and Product) . . . 212
8.3 การหารลงตัว (Divisibility) . . . 215
8.4 อนุกรม (Series) . . . 215
8.5 ทฤษฎีเซตและตรรกะ . . . 217
8.6 ฟังก์ชันพื้นฐาน . . . 219
8.7 จำนวนฟีโบนักชี (Fibonacci Numbers) . . . 220
8.8 ลอการิทึม (Logarithm) . . . 221
8.9 ตัวหารร่วมมากและตัวคูณร่วมน้อย (GCD and LCM) . . . 223
8.10 กลยุทธ์การแก้ปัญหาทางคณิตศาสตร์ . . . 225
8.11 จำนวนเฉพาะและตะแกรงเอราทอสเธเนส (Prime Numbers and Sieve of Eratosthenes) 226
8.12 การแยกตัวประกอบจำนวนเฉพาะ (Prime Factorization) . . . 229
8.13 ฟังก์ชันทฤษฎีจำนวน (Number Theory Functions) . . . 232
8.14 การยกกำลังแบบมอดูลาร์ (Modular Exponentiation) . . . 235
8.15 ขั้นตอนวิธีของยูคลิดแบบขยาย (Extended Euclidean Algorithm) . . . 238
8.16 คอมบินาทอริกซ์ (Combinatorics) . . . 241
8.17 โจทย์แนะนำ . . . 250

8.18 สรุปบทที่ 8 250

4 ชุดปัญหาด้านการโปรแกรมแบบพลวัต 252

9 Dynamic Programming 254

9.1 แนะนำ Dynamic Programming 254
9.2 Overlapping Subproblems 255
9.3 Memoization (Top-Down) 257
9.4 Tabulation (Bottom-Up) 259
9.5 Optimal Substructure 261
9.6 4 ขั้นตอนแก้ปัญหา DP 262
9.7 Max 1D Range Sum (Kadane's Algorithm) 266
9.8 Coin Change 268
9.9 0-1 Knapsack 272
9.10 Traveling Salesman Problem (TSP) 276
9.11 Partition Problem 280
9.12 Edit Distance 283
9.13 Longest Common Subsequence (LCS) 286
9.14 Longest Increasing Subsequence (LIS) 290
9.15 Dynamic Programming บน DAG (DP on DAG) 295
9.16 โจทย์แนะนำ 300
9.17 สรุปบทที่ 9 300

5 ชุดปัญหาด้านเกมเชิงการจัด 303

10 ขั้นตอนวิธีแบบโลภ 305

10.1 แนะนำขั้นตอนวิธีแบบโลภ 305
10.2 การเลือกกิจกรรม (Activity Selection) 307
10.3 ปัญหาการจัดตาราง (Scheduling Problems) 310
10.4 ฮัฟฟ์แมนเข้ารหัส (Huffman Coding) 313
10.5 ปัญหา Fractional Knapsack 317
10.6 การเลือกช่วงเวลา/เส้นเชื่อม (Interval Problems) 319
10.7 Greedy on Graphs 323
10.8 การพิสูจน์ความถูกต้อง (Proving Greedy Correctness) 325
10.9 ตัวอย่างโจทย์เพิ่มเติม 328
10.10 โจทย์แนะนำ 332
10.11 สรุปบทที่ 10 334

6 ชุดปัญหาด้านกราฟ 336

11 ทฤษฎีกราฟ 338
11.1 แนะนำกราฟ . . . 338
11.2 การแทนกราฟ . . . 341
11.3 Depth-First Search (DFS) . . . 346
11.4 Breadth-First Search (BFS) . . . 352
11.5 องค์ประกอบเชื่อมโยง . . . 356
11.6 Flood Fill . . . 358
11.7 การเรียงลำดับทอพอโลยี . . . 360
11.8 โจทย์แนะนำ . . . 364
11.9 สรุปบทที่ 11 . . . 364

12 เส้นทางสั้นสุด 366
12.1 แนะนำปัญหา Shortest Path . . . 366
12.2 Bellman-Ford Algorithm . . . 367
12.3 Dijkstra's Algorithm . . . 370
12.4 Floyd-Warshall Algorithm . . . 372
12.5 Shortest Path บน DAG . . . 375
12.6 การประยุกต์ Floyd-Warshall เพิ่มเติม . . . 379
12.7 เปรียบเทียบขั้นตอนวิธี Shortest Path . . . 385
12.8 โจทย์แนะนำ . . . 386
12.9 สรุปบทที่ 12 . . . 387

13 Union-Find และต้นไม้แผ่คลุมต่ำสุด 388
13.1 Union-Find (Disjoint Set Union) . . . 388
13.2 Minimum Spanning Tree (MST) . . . 392
13.3 Kruskal's Algorithm . . . 393
13.4 Prim's Algorithm . . . 396
13.5 เปรียบเทียบ Kruskal's กับ Prim's . . . 398
13.6 โจทย์แนะนำ . . . 399
13.7 สรุปบทที่ 13 . . . 399

14 หัวข้อกราฟเพิ่มเติม 401
14.1 การตรวจสอบกราฟทวิภาคี (Bipartite Check) . . . 401
14.2 จุดตัดและสะพาน (Articulation Points & Bridges) . . . 403
14.3 Strongly Connected Components (SCC) . . . 406
14.4 Kosaraju's Algorithm — ขั้นตอนวิธี SCC ทางเลือก . . . 410
14.5 ปัญหา 2-SAT . . . 413

14.6 โจทย์แนะนำ . . . 418
14.7 สรุปบทที่ 14 . . . 419

7 ชุดปัญหาด้านเครือข่ายการไหล 420

15 การไหลสูงสุด 422
15.1 แนะนำเครือข่ายการไหล . . . 422
15.2 วิธีแบบโลภและข้อจำกัด . . . 423
15.3 กราฟคงเหลือ (Residual Graph) . . . 425
15.4 เส้นทางเสริม (Augmenting Path) . . . 426
15.5 ขั้นตอนวิธี Ford-Fulkerson . . . 427
15.6 ขั้นตอนวิธี Edmonds-Karp . . . 428
15.7 ขั้นตอนวิธี Dinic . . . 430
15.8 ทฤษฎีบท Max-Flow Min-Cut . . . 435
15.9 การประยุกต์ใช้ Maximum Flow . . . 437
15.10 Hopcroft-Karp Algorithm — MCBM ที่เร็วกว่า . . . 439
15.11 Min-Cost Max-Flow . . . 445
15.12 Flow Graph Modeling . . . 448
15.13 โจทย์แนะนำ . . . 450
15.14 สรุปบทที่ 15 . . . 451

8 ชุดปัญหาด้านสายอักขระ 453

16 ขั้นตอนวิธีสายอักขระ 455
16.1 แนะนำสายอักขระใน C++ . . . 455
16.2 การค้นหารูปแบบ (Pattern Matching) . . . 459
16.3 ขั้นตอนวิธี KMP (Knuth-Morris-Pratt) . . . 460
16.4 ขั้นตอนวิธี Rabin-Karp . . . 464
16.5 ฟังก์ชัน Z (Z-Function) . . . 467
16.6 แถวลำดับคำต่อท้าย (Suffix Array) . . . 470
16.7 ปัญหาสายอักขระที่พบบ่อย . . . 476
16.8 โจทย์แนะนำ . . . 481
16.9 สรุปบทที่ 16 . . . 482

9 ชุดปัญหาด้านเรขาคณิต 487

17 เรขาคณิตเชิงคำนวณ (Computational Geometry) 489

17.1 บทนำสู่เรขาคณิตเชิงคำนวณ . . . 489
17.2 จุด (Point) และเวกเตอร์ (Vector) . . . 491
17.3 เส้นตรง (Line) และส่วนของเส้นตรง (Line Segment) . . . 495
17.4 ผลคูณเชิงเวกเตอร์ (Cross Product) และ CCW Test . . . 498
17.5 วัตถุ 2 มิติ: วงกลม (Circle) . . . 501
17.6 วัตถุ 2 มิติ: สามเหลี่ยม (Triangle) . . . 503
17.7 วัตถุ 2 มิติ: สี่เหลี่ยม (Quadrilateral) . . . 506
17.8 รูปหลายเหลี่ยม (Polygon) . . . 506
17.9 เปลือกนูน (Convex Hull) . . . 510
17.10 โจทย์แนะนำ . . . 513
17.11 สรุปบทที่ 17 . . . 513

อภิธานศัพท์ 515

ทั่วไป . . . 515
โครงสร้างข้อมูล . . . 516
ขั้นตอนวิธี . . . 517
คณิตศาสตร์ . . . 518
กราฟ . . . 519
สายอักขระ . . . 521
เรขาคณิต . . . 522

บรรณานุกรม 524

ดัชนี 526

1

บทนำเกี่ยวกับการเขียนโปรแกรมเชิงแข่งขัน

ส่วนแรกนี้กล่าวถึงพื้นฐานที่จำเป็นสำหรับผู้สนใจการโปรแกรมเชิงแข่งขันทุกคน เริ่มจากแนวคิดของการโปรแกรมเชิงแข่งขัน รูปแบบของการแข่งขัน เครื่องมือที่ใช้ รวมถึงกระบวนการวิเคราะห์ปัญหาและเขียนโปรแกรมภายใต้ข้อจำกัดของเวลาและทรัพยากร นอกจากนี้ยังครอบคลุมเนื้อหาพื้นฐานที่จำเป็นต่อการเรียนรู้ในส่วนถัดไป ได้แก่ ความซับซ้อนทางเวลา (Time Complexity) สำหรับวิเคราะห์ประสิทธิภาพขั้นตอนวิธี การเรียงลำดับและการค้นหาแบบทวิภาค (Sorting and Binary Search) และกลยุทธ์แบ่งแยกและเอาชนะ (Divide and Conquer) ซึ่งเป็นทักษะพื้นฐานที่จำเป็นต้องเชี่ยวชาญก่อนจะก้าวต่อไปสู่หัวข้อขั้นสูง

Chapter 1

บทนำสู่การโปรแกรมเชิงแข่งขัน

การโปรแกรมเชิงแข่งขัน (Competitive Programming) คือกีฬาที่ใช้ทักษะทางปัญญาที่ผู้แข่งขันต้องใช้ความรู้ด้านคณิตศาสตร์ การออกแบบขั้นตอนวิธี และทักษะการเขียนโปรแกรม เพื่อแก้โจทย์ปัญหาที่มีข้อจำกัดด้านเวลาและทรัพยากรอย่างเข้มข้น ในบทนี้ เราจะทำความรู้จักกับโลกของการแข่งขัน ตั้งแต่ความหมาย แพลตฟอร์มที่ใช้ฝึกฝน ไปจนถึงการตั้งค่าเครื่องมือที่จำเป็นสำหรับการแข่งขัน

1.1 การโปรแกรมเชิงแข่งขันคืออะไร

การโปรแกรมเชิงแข่งขันเป็นการผสมผสานระหว่างสองทักษะหลัก ได้แก่ **การออกแบบขั้นตอนวิธี** (Design of Algorithms) ซึ่งหมายถึงความสามารถในการคิดวิเคราะห์โจทย์ปัญหา เลือกใช้วิธีการแก้ปัญหาที่เหมาะสม และออกแบบขั้นตอนวิธีที่มีประสิทธิภาพ กับ **การเขียนโปรแกรมแสดงผล** (Implementation of Algorithms) ซึ่งหมายถึงความสามารถในการแปลงแนวคิดให้เป็นโค้ดที่ถูกต้อง และทำงานได้ภายในเวลาที่กำหนด[3, 5]

โดยปกติในการแข่งขัน ผู้แข่งขันจะได้รับโจทย์ปัญหาหลายข้อ แต่ละข้อจะระบุ **ข้อมูลนำเข้า** (Input) และ **ข้อมูลส่งออก** (Output) อย่างชัดเจน พร้อมทั้งข้อจำกัดด้านเวลา (Time Limit) และด้านหน่วยความจำ (Memory Limit) หน้าที่ของผู้แข่งคือเขียนโปรแกรมที่รับข้อมูลนำเข้า ประมวลผล และแสดงผลลัพธ์ที่ถูกต้องภายในข้อจำกัดเหล่านั้น

สิ่งที่แยกการโปรแกรมเชิงแข่งขันออกจากการเขียนโปรแกรมทั่วไปคือ **ความเร็วและความถูกต้อง** ในการแข่งขัน โปรแกรมที่ให้ผลลัพธ์ถูกต้องแต่ช้าเกินไปจะไม่ได้คะแนนหรือได้คะแนนเพียงบางส่วน เช่นเดียวกับโปรแกรมที่เร็วแต่ให้ผลผิด ดังนั้นผู้แข่งขันจึงต้องหาสมดุลระหว่างประสิทธิภาพและความถูกต้องของขั้นตอนวิธี นอกจากนี้ การแข่งขันยังฝึกทักษะสำคัญอื่นๆ เช่น การอ่านโจทย์อย่างละเอียด การจัดการเวลา และการทำงานเป็นทีม (ในรายการแบบทีม)

นิยาม

การโปรแกรมเชิงแข่งขัน = **การออกแบบขั้นตอนวิธี** + **การเขียนโปรแกรมแสดงผล**
เป้าหมาย: แก้โจทย์ปัญหาให้ **ถูกต้อง** และ **รวดเร็ว** ภายในข้อจำกัดที่กำหนด

1.2 แพลตฟอร์มการแข่งขัน

มีแพลตฟอร์มออนไลน์มากมายที่จัดการแข่งขันหรือให้ฝึกฝนการโปรแกรมเชิงแข่งขัน แต่ละแพลตฟอร์มมีจุดเด่นและระดับความยากแตกต่างกัน ผู้เริ่มต้นควรเลือกแพลตฟอร์มที่เหมาะสมกับระดับทักษะของตนเอง

1.2.1 การแข่งขันระดับนานาชาติ

ICPC (International Collegiate Programming Contest) เป็นการ แข่งขัน โปรแกรม เชิง แข่งขัน ระดับ มหาวิทยาลัยที่เก่าแก่และมีชื่อเสียงที่สุดในโลก แข่งขันแบบทีมละ 3 คน ใช้เวลา 5 ชั่วโมง แก้โจทย์ประมาณ 10–13 ข้อ ทีมที่แก้โจทย์ได้มากที่สุดและใช้เวลาน้อยที่สุดจะเป็นผู้ชนะ การแข่งขันจะแบ่งเป็นรอบภายในประเทศ (Regional) และรอบโลก (World Finals)[13]

IOI (International Olympiad in Informatics)[14] เป็นการแข่งขันโอลิมปิกวิชาการด้านคอมพิวเตอร์สำหรับนักเรียนมัธยมศึกษา แข่งขันแบบเดี่ยว 2 วัน วันละ 3 โจทย์ ใช้เวลา 5 ชั่วโมงต่อวัน ประเทศไทยจะคัดเลือกผู้แทนผ่านการแข่งขัน **TOI (Thai Olympiad in Informatics)** ซึ่งจัดโดยมูลนิธิส่งเสริมโอลิมปิกวิชาการและพัฒนามาตรฐานวิทยาศาสตร์ศึกษา (สอวน.) หลังจากนั้นจะไปเก็บตัวแล้วคัดตัวผู้แทนระดับประเทศกับสถาบันส่งเสริมการสอนวิทยาศาสตร์และเทคโนโลยี (สสวท.) ดังนั้นสำหรับนักเรียนที่สนใจควรเริ่มฝึกฝนตั้งแต่ชั้นมัธยมต้น

1.2.2 แพลตฟอร์มฝึกฝนออนไลน์

ตารางที่ 1.1 แพลตฟอร์มฝึกฝนการโปรแกรมเชิงแข่งขันยอดนิยม

แพลตฟอร์ม	คำอธิบาย	ระดับ
Codeforces[10]	จัดแข่งขันสัปดาห์ละหลายครั้ง เรตติ้งแบ่งระดับ จาก Gray ถึง Legendary Grandmaster	ทุกระดับ
AtCoder[8]	แพลตฟอร์ม ญี่ปุ่น โจทย์ คุณภาพ สูง เน้นความคิดสร้างสรรค์	ปานกลาง–สูง
CodeChef[9]	จัด แข่ง 3 ระดับ ความ ยาก (Long Challenge, Cook-Off, Lunchtime)	ทุกระดับ
Topcoder[19]	แพลตฟอร์มเก่าแก่ มีระบบ SRM (Single Round Match)	ปานกลาง–สูง
LeetCode[16]	เน้น โจทย์ สัมภาษณ์ งาน มี โจทย์ มากกว่า 2,500 ข้อ	เริ่มต้น–ปานกลาง
Project Euler[18]	โจทย์ คณิตศาสตร์ และ การ เขียน โปรแกรม แก้ได้ด้วยภาษาใดก็ได้	ปานกลาง–สูง
Programming.in.th[17]	โจทย์ภาษาไทย	ทุกระดับ

คำแนะนำสำหรับผู้เริ่มต้น

เริ่มจาก Codeforces ระดับ Div.2 หรือ LeetCode Easy ก่อน เมื่อมั่นใจแล้วค่อยขยับไป AtCoder[8] Beginner Contest หรือ Codeforces Div.1 สิ่งสำคัญคือ **ฝึกฝนอย่างสม่ำเสมอ** มากกว่าฝึกหนักเป็นครั้งคราว

1.3 ภาษาโปรแกรมและการตั้งค่า

ในการโปรแกรมเชิงแข่งขัน การเลือกภาษาโปรแกรมมีความสำคัญอย่างยิ่ง เนื่องจากแต่ละภาษามีความเร็วในการทำงาน ไลบรารีมาตรฐาน และความสะดวกในการเขียนโค้ดที่แตกต่างกัน

1.3.1 ทำไม C++ จึงเป็นภาษายอดนิยม

C++ เป็นภาษาที่นักแข่งขันส่วนใหญ่เลือกใช้ เหตุผลหลักคือ:

1. **ความเร็วในการทำงาน** — C++ เป็นภาษาคอมไพล์ที่ทำงานเร็วมาก ทำให้สามารถใช้ขั้นตอนวิธีที่ซับซ้อนได้โดยไม่ต้องกังวลเรื่อง Time Limit
2. **STL (Standard Template Library)** — ไลบรารีมาตรฐานของ C++ มีโครงสร้างข้อมูลและขั้นตอนวิธีพร้อมใช้ เช่น `vector`, `set`, `map`, `sort`, `lower_bound` ซึ่งช่วยประหยัดเวลาในการแข่งขันได้มาก
3. **การควบคุมระดับต่ำ** — C++ ให้การควบคุมหน่วยความจำโดยตรง ทำให้เขียนโค้ดที่มีประสิทธิภาพสูงได้

ตารางที่ 1.2 เปรียบเทียบภาษาโปรแกรมในการแข่งขัน

ภาษา	ความเร็ว	ความสะดวก	ไลบรารี	ความนิยม
C++	สูงมาก	ปานกลาง	STL แข็งแกร่ง	สูงสุด
C	สูงมาก	ต่ำ	น้อย	ต่ำ
Java	ปานกลาง	ปานกลาง	ดี	ปานกลาง
Python	ต่ำ	สูงมาก	ดีมาก	ปานกลาง

1.3.2 เทมเพลตโค้ด C++ มาตรฐาน

ในการแข่งขัน ผู้แข่งขันมักใช้เทมเพลตโค้ด (Template) ที่มีส่วนประกอบพื้นฐานครบถ้วน เพื่อประหยัดเวลาในการเขียนส่วนที่ซ้ำๆ กันทุกข้อ ต่อไปนี้คือเทมเพลตที่แนะนำ:

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // Write your code here
    return 0;
}
```

ตัวอย่างโค้ดที่ 1.1: เทมเพลตโค้ด C++ สำหรับการแข่งขัน

คำอธิบายเทมเพลต

- `#include <bits/stdc++.h>` — รวมไลบรารีทั้งหมดของ C++ STL ในบรรทัดเดียว (ใช้ได้เฉพาะกับ GCC ในการแข่งขันเท่านั้น ไม่แนะนำให้ใช้นอกการแข่งขัน)
- `using namespace std;` — ทำให้ไม่ต้องพิมพ์ `std::` นำหน้าทุกครั้ง
- `ios::sync_with_stdio(false)` และ `cin.tie(nullptr)` — เร่งความเร็วการรับค่า/แสดงผล (อ่านรายละเอียดเพิ่มเติมในหัวข้อ 1.4)

1.3.3 คำสั่งคอมไพล์

ในการแข่งขันส่วนใหญ่ ระบบจะคอมไพล์โค้ด C++ ด้วยคำสั่งดังนี้:

```
g++ -std=c++14 -O2 -Wall -o solution solution.cpp
```

ตัวอย่างโค้ดที่ 1.2: คำสั่งคอมไพล์ C++ มาตรฐาน

- `-std=c++14` — ใช้มาตรฐาน C++14 (หรือ C++17 หรือเลขเวอร์ชันอื่นในบางการแข่งขัน)
- `-O2` — เปิดการเพิ่มประสิทธิภาพระดับ 2 ทำให้โค้ดทำงานเร็วขึ้นอย่างมาก
- `-Wall` — แสดงคำเตือนทั้งหมด ช่วยตรวจจับข้อผิดพลาดเบื้องต้น

1.4 การรับค่าและแสดงผล

การรับค่า (Input) และแสดงผล (Output) เป็นทักษะพื้นฐานที่สำคัญที่สุดอย่างหนึ่งในการแข่งขัน หากอ่านค่าไม่ถูกต้องหรือแสดงผลผิดรูปแบบ โปรแกรมจะตอบผิดทันที แม้ขั้นตอนวิธีจะถูกต้องก็ตาม

1.4.1 วิธีรับค่าและแสดงผลพื้นฐาน

C++ มีวิธีรับค่า/แสดงผล 2 ชุดหลัก:

```cpp
// Method 1: cin / cout (C++ style)
int a;
cin >> a;
cout << a << "\n";
// Method 2: scanf / printf (C style)
int b;
scanf("%d", &b);
printf("%d\n", b);
```

ตัวอย่างโค้ดที่ 1.3: การรับค่าและแสดงผลด้วย cin/cout และ scanf/printf

ข้อแตกต่างสำคัญคือ `cin/cout` ใช้ง่ายกว่าแต่ช้ากว่า `scanf/printf` อย่างไรก็ตาม เราสามารถเร่งความเร็ว `cin/cout` ได้ด้วยคำสั่ง:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

ตัวอย่างโค้ดที่ 1.4: เร่งความเร็ว cin/cout

คำสั่ง `ios::sync_with_stdio(false)` ยกเลิกการซิงค์ระหว่าง C++ stream กับ C stream ทำให้ `cin/cout` ทำงานเร็วขึ้นอย่างมาก ส่วน `cin.tie(nullptr)` ยกเลิกการผูก (tie) ระหว่าง `cin` กับ `cout` ช่วยเร่งความเร็วเพิ่มเติม

ข้อควรระวัง

เมื่อใช้ `ios::sync_with_stdio(false)` แล้ว **ห้าม** ใช้ `cin/cout` ร่วมกับ `scanf/printf` ในโปรแกรมเดียวกัน เพราะอาจทำให้ลำดับการรับค่า/แสดงผลสลับกันได้

1.4.2 \n vs endl

ใน C++ สามารถขึ้นบรรทัดใหม่ได้ 2 วิธี:

```
cout << "Hello" << "\n";    // Method 1: '\n' -- fast
cout << "Hello" << endl;    // Method 2: endl -- slower
```

ตัวอย่างโค้ดที่ 1.5: เปรียบเทียบขึ้นบรรทัดใหม่

`endl` จะ flush buffer ทุกครั้ง ซึ่งทำให้ช้าลงอย่างมากเมื่อแสดงผลจำนวนมาก ในการแข่งขันควรใช้ `"\n"` แทน

1.4.3 การรับค่าข้อความทั้งบรรทัด

การใช้ `cin >> s` จะอ่านเฉพาะคำเดียว (หยุดที่ช่องว่าง) หากต้องการอ่านข้อความทั้งบรรทัด ต้องใช้ `getline`:

```
string line;
getline(cin, line);    // Read entire line
```

ตัวอย่างโค้ดที่ 1.6: การใช้ getline อ่านข้อความทั้งบรรทัด

ข้อควรระวัง: ผสม cin» กับ getline

หลังจากใช้ `cin >> x` แล้วตามด้วย `getline` จะมีอักขระขึ้นบรรทัดใหม่ (`\n`) เหลืออยู่ใน buffer ทำให้ `getline` อ่านบรรทัดว่างแทนที่จะอ่านข้อความจริง วิธีแก้คือใช้ `cin.ignore()` ก่อน `getline`:

```cpp
int n;
cin >> n;
cin.ignore();              // Skip remaining '\n'
string line;
getline(cin, line);        // Now reads the actual line
```

ตัวอย่างโค้ดที่ 1.7: Fix cin and getline

1.4.4 การใช้ istringstream แยกค่า

เมื่อรับข้อความทั้งบรรทัดแล้วต้องการแยกค่าแต่ละตัว สามารถใช้ `istringstream` ได้:

```cpp
#include <sstream>
string line = "10 20 30";
istringstream iss(line);
int a, b, c;
iss >> a >> b >> c;     // a=10, b=20, c=30
```

ตัวอย่างโค้ดที่ 1.8: ใช้ istringstream แยกค่าจากข้อความ

1.4.5 การอ่าน/เขียนไฟล์ด้วย freopen

ในการแข่งขันบางรายการ ข้อมูลนำเข้าและส่งออกจะอยู่ในรูปไฟล์ เราสามารถเปลี่ยน `cin`/`cout` ให้อ่าน/เขียนไฟล์โดยตรงด้วย `freopen`:

```cpp
freopen("input.txt", "r", stdin);      // Read from file
freopen("output.txt", "w", stdout);    // Write to file
```

ตัวอย่างโค้ดที่ 1.9: ใช้ freopen อ่าน/เขียนไฟล์

หลังจากเรียก `freopen` แล้ว `cin` จะอ่านจากไฟล์แทนแป้นพิมพ์ และ `cout` จะเขียนลงไฟล์แทนหน้าจอ วิธีนี้สะดวกมากเพราะไม่ต้องเปลี่ยนโค้ดส่วนอื่นเลย

1.5 การทำงานกับตัวเลข

การทำงานกับตัวเลขเป็นหัวใจสำคัญของการแข่งขัน ความเข้าใจเรื่องประเภทข้อมูล ขอบเขตค่า และข้อควรระวังต่างๆ จะช่วยหลีกเลี่ยงข้อผิดพลาดที่พบบ่อย

1.5.1 ประเภทข้อมูลจำนวนเต็ม

ตารางที่ 1.3 ประเภทข้อมูลจำนวนเต็มใน C++

ประเภท	ขนาด (บิต)	ค่าต่ำสุด	ค่าสูงสุด
`int`	32	-2^{31}	$2^{31}-1 \approx 2 \times 10^9$
`unsigned int`	32	0	$2^{32}-1 \approx 4 \times 10^9$
`long long`	64	-2^{63}	$2^{63}-1 \approx 9 \times 10^{18}$
`unsigned long long`	64	0	$2^{64}-1 \approx 1.8 \times 10^{19}$

ข้อควรระวัง Overflow

`int` เก็บค่าได้สูงสุดประมาณ 2×10^9 เท่านั้น หากโจทย์มีค่าเกินกว่านี้ **ต้อง** ใช้ `long long` ตัวอย่าง เช่น ถ้าต้องคำนวณ $n!$ เมื่อ $n \geq 13$ ค่าจะเกินขอบเขตของ `int` ทันที

เมื่อกำหนดค่าคงที่ประเภท `long long` ควรเติม `LL` ต่อท้ายเสมอ เพื่อป้องกันการคูณเกินขอบเขตก่อนจะเก็บลงตัวแปร:

```
1 long long a = 1000000000LL * 1000000000LL;   // Correct: 10^18
2 long long b = 1000000000 * 1000000000;       // Wrong! overflow before storing
```

ตัวอย่างโค้ดที่ 1.10: ความสำคัญของ LL suffix

ในบรรทัดที่ 2 ตัวเลข `1000000000` เป็น `int` เมื่อคูณกัน ผลลัพธ์จะเกินขอบเขต `int` ก่อนที่จะถูกแปลงเป็น `long long` ทำให้ค่าที่ได้ไม่ถูกต้อง

1.5.2 เลขคณิตมอดูลาร์ (Modular Arithmetic)

ในการแข่งขันจำนวนมาก โจทย์จะขอให้ตอบค่า "mod m" ซึ่งมักเป็น $m = 10^9 + 7$ หรือ $m = 10^9 + 9$ เลขคณิตมอดูลาร์มีคุณสมบัติสำคัญดังนี้:

$$(a + b) \bmod m = \big((a \bmod m) + (b \bmod m)\big) \bmod m \tag{1.1}$$

$$(a - b) \bmod m = \big((a \bmod m) - (b \bmod m) + m\big) \bmod m \tag{1.2}$$

$$(a \times b) \bmod m = \big((a \bmod m) \times (b \bmod m)\big) \bmod m \tag{1.3}$$

ข้อสังเกตสมการ (1.2)

สำหรับการลบ ต้องบวก m ก่อน mod เพื่อป้องกันค่าติดลบ เพราะใน C++ ผลลัพธ์ของ `%` กับจำนวนลบอาจเป็นลบได้

การหารไม่กระจายเหนือ mod

สังเกตว่า $(a/b) \bmod m \neq ((a \bmod m)/(b \bmod m)) \bmod m$ การหารในเลขคณิตมอดูลาร์ต้องใช้ **ตัวผกผันมอดูลาร์** (Modular Inverse) ซึ่งจะกล่าวถึงในบทที่ 8

ตัวอย่าง: คำนวณ n! mod m

```cpp
long long factorial_mod(int n, long long m) {
    long long ans = 1;
    for (int i = 2; i <= n; i++) {
        ans = (ans * i) % m;   // use modular property
    }
    return ans;
}
```

1.5.3 จุดทศนิยม (Floating Point)

การทำงานกับจุดทศนิยมในคอมพิวเตอร์มีข้อจำกัดเรื่องความแม่นยำ (Precision) ต้องระมัดระวังเป็นพิเศษ:

```cpp
const double EPS = 1e-9;
// Check if a equals b (within epsilon)
if (fabs(a - b) < EPS) {
    // a and b are considered equal
}
// Check if a is less than b
if (a < b - EPS) {
    // a is strictly less than b
}
```

ตัวอย่างโค้ดที่ 1.11: การเปรียบเทียบจุดทศนิยมด้วย epsilon

ข้อผิดพลาดที่พบบ่อย

1. **อย่าเปรียบเทียบจุดทศนิยมด้วย ==** — ใช้ `fabs(a-b) < EPS` แทนเสมอ
2. **ระวังการสะสมความคลาดเคลื่อน** — การบวก/ลบจุดทศนิยมหลายครั้งจะสะสมความคลาดเคลื่อน
3. **ใช้ `double` แทน `float`** — `double` มีความแม่นยำสูงกว่า (15–16 หลัก กับ 6–7 หลัก)
4. **การแสดงผล** — ใช้ `printf("%.9f", x)` หรือ `cout << fixed << setprecision(9) << x` เพื่อแสดงทศนิยมให้ครบ

1.6 เทคนิคย่อโค้ด

ในการแข่งขัน การพิมพ์โค้ดให้เร็วและสั้นเป็นข้อได้เปรียบ โดยเฉพาะเมื่อเวลาจำกัด C++ มีเทคนิคหลายอย่างที่ช่วยย่อโค้ดได้ ทั้งนี้แล้วแต่ความชอบความถนัดของแต่ละคน

1.6.1 typedef — ย่อชื่อชนิด

typedef ใช้สร้างชื่อย่อสำหรับชนิดข้อมูลที่ยาว ทำให้พิมพ์สั้นลงและอ่านง่ายขึ้น:

```
typedef long long ll;
typedef vector<int> vi;
typedef pair<int,int> ii;
// Now you can write:
ll a = 1000000000LL;          // instead of long long a = ...
vi v = {1, 2, 3};             // instead of vector<int> v = ...
ii p = {1, 2};                // instead of pair<int,int> p = ...
```

ตัวอย่างโค้ดที่ 1.12: ใช้ typedef ย่อชื่อประเภท

1.6.2 มาโคร (Macros)

มาโครใช้ #define เพื่อสร้างตัวย่อสำหรับคำสั่งหรือรูปแบบที่ใช้บ่อย:

```
#define F first
#define S second
#define PB push_back
#define MP make_pair
#define REP(i,a,b) for (int i = a; i <= b; i++)
// Usage examples:
vector<ii> v;
v.PB(MP(3, 1));                  // instead of v.push_back(make_pair(3, 1))
cout << v[0].F << endl;          // instead of v[0].first
REP(i, 1, 10) cout << i;         // instead of for(int i=1; i<=10; i++)
```

ตัวอย่างโค้ดที่ 1.13: มาโครที่ใช้บ่อยในการแข่งขัน

ข้อจำกัดของมาโคร

มาโครเป็นการแทนที่ข้อความ (Text Substitution) ก่อนคอมไพล์ ไม่ใช่ตัวแปรจริง ดังนั้น:

1. ไม่มีการตรวจสอบชนิด (Type Checking)
2. อาจเกิดผลข้างเคียงที่ไม่คาดคิดได้ เช่น #define SQ(x) x*x แล้วใช้ SQ(1+2) จะได้ 1+2*1+2 = 5 แทนที่จะเป็น 9

3. ควรใช้อย่างระมัดระวัง และเฉพาะในบริบทการแข่งขันเท่านั้น

1.6.3 เทมเพลตโค้ดเต็มรูปแบบ

รวมเทคนิคย่อโค้ดทั้งหมดเป็นเทมเพลตที่พร้อมใช้งาน:

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<int> vi;
typedef pair<int,int> ii;
#define F first
#define S second
#define PB push_back
#define MP make_pair
#define REP(i,a,b) for (int i = a; i <= b; i++)
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // Write your code here
    return 0;
}
```

ตัวอย่างโค้ดที่ 1.14: เทมเพลตโค้ด C++ เต็มรูปแบบสำหรับการแข่งขัน

เทมเพลตนี้ครอบคลุมทุกสิ่งที่จำเป็นหลายอย่างสำหรับการแข่งขัน: การรวมไลบรารี การเร่งความเร็ว I/O การย่อชื่อประเภท และมาโครที่ใช้บ่อย นักแข่งมักบันทึกเทมเพลตนี้ไว้และวางลงเป็นจุดเริ่มต้นของทุกข้อ

1.7 รูปแบบการรับค่าเพิ่มเติม

ในหัวข้อก่อนหน้า เราได้เรียนรู้วิธีรับค่าและแสดงผลพื้นฐานไปแล้ว อย่างไรก็ตาม ในการแข่งขันจริง โจทย์แต่ละข้อมีรูปแบบการรับค่าที่แตกต่างกัน ความเข้าใจในรูปแบบเหล่านี้จะช่วยให้เขียนโค้ดได้รวดเร็วและไม่พลาด โดยเฉพาะเมื่อเวลาในการแข่งขันมีจำกัด หากอ่านรูปแบบผิด โปรแกรมจะตอบผิดทันทีแม้ขั้นตอนวิธีถูกต้อง

1.7.1 รูปแบบที่ 1: ระบุจำนวนเทสต์เคส (T Test Cases)

รูปแบบที่พบบ่อยที่สุดคือการที่บรรทัดแรกบอกจำนวนเทสต์เคส T แล้วตามด้วยข้อมูล T ชุด โครงสร้างโจทย์มักเป็นดังนี้: บรรทัดแรกระบุ T แล้วแต่ละเทสต์เคสจะมีข้อมูลนำเข้าของตัวเอง ผลลัพธ์ของแต่ละเทสต์เคสอาจแสดงในบรรทัดแยกกันหรืออยู่ในบรรทัดเดียวกันก็ได้ ขึ้นอยู่กับโจทย์

```cpp
int T;
cin >> T;
while (T--) {
    int a, b;
    cin >> a >> b;
    cout << a + b << "\n";
}
```

ตัวอย่างโค้ดที่ 1.15: รูปแบบ T Test Cases

ข้อสังเกต

รูปแบบนี้พบได้บ่อยใน Codeforces, CodeChef และ UVA ข้อสำคัญคือต้องอ่าน T ก่อนเสมอ แล้วใช้ลูป `while(T--)` หรือ `for(int t=0; t<T; t++)` เพื่อประมวลผลแต่ละเทสต์เคส ระวังอย่าลืมรีเซ็ตตัวแปรภายในลูปทุกรอบ

1.7.2 รูปแบบที่ 2: หยุดเมื่อถึงค่าที่เฝ้ารอ (Sentinel)

บางโจทย์ไม่ได้บอกจำนวนเทสต์เคสล่วงหน้า แต่ให้อ่านข้อมูลไปเรื่อยๆ จนกว่าจะพบค่าที่เฝ้ารอ (Sentinel Value) ที่ระบุว่าจบแล้ว เช่น อ่านคู่จำนวนเต็มไปเรื่อยๆ จนกว่าจะเจอ `0 0`

```cpp
int a, b;
while (cin >> a >> b) {
    if (a == 0 && b == 0) break;
    cout << a + b << "\n";
}
```

ตัวอย่างโค้ดที่ 1.16: รูปแบบ Sentinel

ตำแหน่งของเงื่อนไขตรวจสอบ

ต้องตรวจสอบค่าเฝ้า **หลังจาก** อ่านค่าเข้ามาแล้ว ไม่ใช่ก่อนอ่าน นอกจากนี้ ในบางโจทย์ค่าเฝ้าอาจเป็น -1 หรือค่าอื่นที่ไม่ใช่ `0` ต้องอ่านโจทย์ให้ละเอียด

1.7.3 รูปแบบที่ 3: อ่านจนกว่าจะหมดไฟล์ (EOF-Terminated)

รูปแบบนี้คล้ายกับรูปแบบ Sentinel แต่ไม่มีค่าเฝ้าบอกจุดสิ้นสุด โปรแกรมต้องอ่านข้อมูลจนกว่าจะหมดไฟล์ (End of File — EOF) พบบ่อยใน UVA Online Judge

```cpp
int a, b;
while (cin >> a >> b) {
    cout << a + b << "\n";
}
```

ตัวอย่างโค้ดที่ 1.17: รูปแบบ EOF-Terminated ด้วย cin

```
int a, b;
while (scanf("%d %d", &a, &b) != EOF) {
    printf("%d\n", a + b);
}
```

ตัวอย่างโค้ดที่ 1.18: รูปแบบ EOF-Terminated ด้วย scanf

สังเกตว่า `cin >> a >> b` จะคืนค่า `false` เมื่ออ่านถึงจุด EOF ส่วน `scanf` จะคืนค่า EOF เมื่ออ่านหมดไฟล์ ทั้งสองวิธีใช้ได้ แต่ต้องอย่าลืม **ห้าม** ใช้ `cin` ร่วมกับ `scanf` เมื่อเปิด `ios::sync_with_stdio(false)` แล้ว

1.7.4 รูปแบบที่ 4: การแสดงผล ”Case N: answer”

หลายโจทย์โดยเฉพาะในรูปแบบ Google Code Jam และ UVA กำหนดให้แสดงผลในรูปแบบ `Case N: answer` โดย N เป็นหมายเลขเทสต์เคสเริ่มจาก 1

```
int T;
cin >> T;
for (int t = 1; t <= T; t++) {
    int a, b;
    cin >> a >> b;
    cout << "Case " << t << ": " << a + b << "\n";
}
```

ตัวอย่างโค้ดที่ 1.19: รูปแบบ ”Case N: answer”

ระวังบรรทัดว่างระหว่างเคส

บางโจทย์กำหนดให้เว้นบรรทัดว่างระหว่างเทสต์เคส ซึ่งต้องระวังเป็นพิเศษ:

- **เว้นบรรทัดระหว่างเคส** — ใช้ `if (t > 1) cout << "\n";` ก่อนแสดงผลของเคสปัจจุบัน
- **ไม่เว้นบรรทัดหลังเคสสุดท้าย** — ต้องตรวจสอบว่าเคสปัจจุบันเป็นเคสสุดท้ายหรือไม่
- **อ่านโจทย์ทุกครั้ง** — บางโจทย์บอก ”blank line between cases” บางโจทย์ไม่บอก การเว้นผิดจะทำให้ได้ Wrong Answer

ตัวอย่าง: โจทย์หาผลบวกของจำนวนเต็ม N จำนวน — ทุกรูปแบบ

โจทย์เดียวกันอาจนำเสนอได้ 3 รูปแบบ:
รูปแบบ T: บรรทัดแรก = T ตามด้วย T ชุด แต่ละชุดมี N แล้วตามด้วย N จำนวน

รูปแบบ Sentinel: อ่าน N ไปเรื่อยๆ เมื่อ N=0 หยุด แต่ละชุดมี N จำนวน
รูปแบบ EOF: อ่าน N แล้วตามด้วย N จำนวน ทำซ้ำจนหมดไฟล์
สิ่งสำคัญคือต้องอ่านโจทย์ให้ชัดเจนว่าเป็นรูปแบบใด ก่อนจะเริ่มเขียนโค้ด

1.8 โจทย์แนะนำ

เนื่องจากบทนี้เป็นบทนำ โจทย์แนะนำจึงเน้นทักษะพื้นฐาน เช่น การรับค่าและแสดงผลในรูปแบบต่างๆ การจัดการเลขจำนวนเต็ม และการใช้เลขคณิตมอดูลาร์เบื้องต้น โจทย์ทั้งหมดคัดมาจากแพลตฟอร์มยอดนิยมที่เข้าถึงได้ฟรี โดยเรียงจากง่ายไปยาก ผู้เรียนควรพยายามแก้ด้วยตนเองก่อนอ่านเฉลย และใช้โจทย์เหล่านี้เป็นการฝึกฝนทักษะการอ่านโจทย์และการเลือกรูปแบบ I/O ที่เหมาะสม

ตารางที่ 1.4 โจทย์แนะนำสำหรับบทที่ 1

โจทย์	แหล่ง	ระดับ	เทคนิคที่ใช้	หมายเหตุ
A+B Problem	CSES	ง่าย	การรับค่าและบวกเลขพื้นฐาน	โจทย์แรกสำหรับทดสอบการส่งคำตอบ
Watermelon	Codeforces 4A	ง่าย	เลขคู่/คี่, การคิดเงื่อนไขง่ายๆ	ฝึกอ่านโจทย์และจัดการ edge case
Way Too Long Words	Codeforces 71A	ง่าย	การจัดการสายอักขระ, ความยาวคำ	ฝึกใช้ `string` และการตัดสินใจแบบมีเงื่อนไข
Two Sum	CSES	ปานกลาง	การรับ N จำนวนและค้นหาคู่	ฝึกใช้แถวลำดับและลูปซ้อน หรือใช้ `map`
Bit++	Codeforces 282A	ง่าย	การอ่านสายอักขระและนับตัวแปร	ฝึกใช้ลูปและเงื่อนไขสายอักขระ
Repetitions	CSES	ปานกลาง	การนับรันยาวสุดในสายอักขระ	ฝึกการไล่สายอักขระและเก็บสถานะ
Raising Bacteria	Codeforces 579A	ปานกลาง	การนับบิตที่เป็น 1 (popcount)	ฝึกคิดเชิงบิตเบื้องต้น สัมพันธ์กับการยกกำลังสอง

คำแนะนำในการฝึก

- **เริ่มจากง่าย** — แก้โจทย์ CSES[12] และ Codeforces ระดับ 800–1000 ก่อน เพื่อสร้างความคุ้นเคยกับระบบตัดสินและรูปแบบ I/O
- **ฝึกอ่านโจทย์** — ก่อนเขียนโค้ด ให้อ่านโจทย์ทั้งหมดและระบุรูปแบบ I/O (T test cases,

sentinel, EOF, หรือ Case N:)

- **ใช้เทมเพลต** — ใช้เทมเพลตโค้ดมาตรฐานที่ได้จากบทนี้เป็นฐาน แล้วปรับเพิ่มเติมตามโจทย์
- **ตรวจสอบ edge case** — โจทย์แรกๆ มักมี edge case เล็กน้อย เช่น N=0, N=1, หรือค่าติดลบ ควรทดสอบด้วยตนเองก่อนส่ง

1.9 สรุปบทที่ 1

ในบทนี้ เราได้เรียนรู้พื้นฐานของการโปรแกรมเชิงแข่งขัน ตั้งแต่ความหมายและแพลตฟอร์มการแข่งขัน ไปจนถึงการตั้งค่าสภาพแวดล้อมการพัฒนา การรับค่าและแสดงผล การทำงานกับตัวเลข และเทคนิคย่อโค้ดที่จำเป็นสำหรับการแข่งขัน

สรุปสิ่งสำคัญของบทที่ 1

1. **C++ เป็นภาษายอดนิยม** ในการแข่งขันเพราะเร็วและมี STL ที่ทรงพลัง
2. **เทมเพลตโค้ดมาตรฐาน** ช่วยให้เริ่มเขียนโจทย์ได้เร็วขึ้น ควรจดจำและปรับแต่งให้เหมาะกับตนเอง
3. **การรับค่าและแสดงผล** เป็นทักษะพื้นฐานที่ต้องฝึกจนคล่อง โดยเฉพาะรูปแบบ T Test Cases, Sentinel, EOF-Terminated และ Case N: answer
4. **การใช้** `freopen` ช่วยให้ไม่ต้องพิมพ์ข้อมูลนำเข้าซ้ำๆ ขณะทดสอบ
5. **ประเภทข้อมูลจำนวนเต็ม** ต้องเลือกให้เหมาะกับขนาดข้อมูล โดยเฉพาะ `long long` สำหรับค่าที่เกิน 2×10^9
6. **เลขคณิตมอดูลาร์** พบบ่อยมากในโจทย์ ต้องระวัง overflow และค่าติดลบหลัง mod
7. **typedef และมาโคร** ช่วยย่อโค้ดให้สั้นลง แต่ต้องใช้อย่างระมัดระวัง

Chapter 2

ความซับซ้อนของเวลา

ในบทก่อนหน้า เราได้เรียนรู้พื้นฐานการเขียนโปรแกรมสำหรับการแข่งขันไปแล้ว แต่การเขียนโปรแกรมที่ “ใช้ได้” ไม่เพียงพอ — โปรแกรมต้องทำงานได้ **เร็วพอ** ภายในข้อจำกัดเวลาที่โจทย์กำหนด บทนี้จะนำเสนอเครื่องมือที่สำคัญที่สุดในการวิเคราะห์ประสิทธิภาพของขั้นตอนวิธี นั่นคือ **ความซับซ้อนของเวลา** (Time Complexity) เราจะเรียนรู้วิธีวิเคราะห์โค้ด จัดประเภทขั้นตอนวิธีตามความเร็ว และใช้ข้อจำกัดของโจทย์ในการเลือกขั้นตอนวิธีที่เหมาะสม[2]

2.1 ความซับซ้อนของเวลาคืออะไร

เมื่อเราเขียนโปรแกรมแก้โจทย์ปัญหาหนึ่งๆ มักมีหลายวิธีที่สามารถแก้ได้ แต่ละวิธีอาจใช้เวลาในการทำงานที่แตกต่างกัน ยิ่งข้อมูลนำเข้ามีขนาดใหญ่ ความแตกต่างนี้ยิ่งชัดเจน การวิเคราะห์ความซับซ้อนของเวลาคือกระบวนการประเมินว่าขั้นตอนวิธีหนึ่งใช้เวลาในการทำงานเพิ่มขึ้นเร็วแค่ไหนเมื่อขนาดของข้อมูลนำเข้าเพิ่มขึ้น

2.1.1 ทำไมต้องวิเคราะห์ขั้นตอนวิธี

หากเรามีวิธีเดียวในการแก้ปัญหา การวิเคราะห์เวลาในการทำงานก็อาจจะไม่จำเป็น แต่จริงๆ แล้วในการแก้ปัญหาส่วนใหญ่นั้นมีหลายวิธีในการแก้ ดังนั้นเราในการที่เราจะเลือกว่าจะใช้วิธีการไหน เราก็ต้องวิเคราะห์ขั้นตอนวิธีดูก่อน ลองนึกภาพว่าเราต้องหาผลรวมของจำนวนเต็มตั้งแต่ 1 ถึง n เราสามารถเขียนได้ 2 วิธี:

```cpp
long long sum = 0;
for (int i = 1; i <= n; i++) {
    sum += i;
}
```

ตัวอย่างโค้ดที่ 2.1: วิธีที่ 1: วนลูปบวกทีละตัว

```cpp
long long sum = (long long)n * (n + 1) / 2;
```

ตัวอย่างโค้ดที่ 2.2: วิธีที่ 2: ใช้สูตรคณิตศาสตร์

วิธีที่ 1 ต้องบวกทั้งหมด n ครั้ง ส่วนวิธีที่ 2 ใช้การคำนวณแค่ครั้งเดียวไม่ว่า n จะมากแค่ไหน เมื่อ $n = 10^9$ วิธีที่ 1 ต้องทำงาน 10^9 ครั้ง ซึ่งช้าเกินไปสำหรับการแข่งขัน ในขณะที่วิธีที่ 2 ตอบได้ทันที นี่คือเหตุผลว่าทำไมเราต้องวิเคราะห์ขั้นตอนวิธี — เพื่อเลือกวิธีที่มีประสิทธิภาพก่อนเสียเวลาเขียนโค้ดที่ช้าเกินไป

2.1.2 กรณีที่เลวร้ายที่สุด กรณีเฉลี่ย และกรณีที่ดีที่สุด

เมื่อวิเคราะห์เวลาของขั้นตอนวิธี เรามักพิจารณา 3 กรณี:

- **กรณีที่เลวร้ายที่สุด (Worst Case)** — เวลาที่ขั้นตอนวิธีใช้มากที่สุดสำหรับข้อมูลนำเข้าขนาด n นี่คือกรณีที่เราให้ความสำคัญมากที่สุดในการแข่งขัน เพราะเราต้องการรับประกันว่าโปรแกรมจะทำงานทันเวลาในทุกกรณี
- **กรณีเฉลี่ย (Average Case)** — เวลาเฉลี่ยที่ขั้นตอนวิธีใช้สำหรับข้อมูลนำเข้าขนาด n ทั้งหมด การคำนวณกรณีเฉลี่ยมักซับซ้อนกว่าและต้องสมมติว่าข้อมูลนำเข้ามีการกระจายแบบใดแบบหนึ่ง
- **กรณีที่ดีที่สุด (Best Case)** — เวลาที่ขั้นตอนวิธีใช้น้อยที่สุดสำหรับข้อมูลนำเข้าขนาด n กรณีนี้มักไม่ค่อยมีประโยชน์ในทางปฏิบัติ เพราะไม่ได้บอกอะไรมากนักเกี่ยวกับประสิทธิภาพโดยรวม

ตัวอย่าง: Linear Search

พิจารณาการค้นหาแบบเส้นตรง (Linear Search) ที่ตรวจสอบสมาชิกในแถวลำดับทีละตัวจนเจอค่าที่ต้องการ:

- **Best Case:** $\mathcal{O}(1)$ — เจอที่ตำแหน่งแรก
- **Average Case:** $\mathcal{O}(n)$ — เจอที่ตำแหน่งกลางๆ โดยเฉลี่ย
- **Worst Case:** $\mathcal{O}(n)$ — ไม่เจอเลย หรือเจอที่ตำแหน่งสุดท้าย

ในการแข่งขัน เราสนใจ Worst Case คือ $\mathcal{O}(n)$ เพื่อรับประกันว่าโปรแกรมจะทันเวลาเสมอ

2.1.3 การนับ “การดำเนินการพื้นฐาน”

แทนที่จะวัดเวลาเป็นวินาที (ซึ่งขึ้นกับฮาร์ดแวร์) เรานับจำนวน **การดำเนินการพื้นฐาน** (Basic Operations) ที่ขั้นตอนวิธีทำ การดำเนินการพื้นฐานได้แก่:

- การกำหนดค่า เช่น `x = 5`
- การดำเนินการทางคณิตศาสตร์ เช่น `a + b`, `x * y`
- การเปรียบเทียบ เช่น `a < b`, `x == y`
- การเข้าถึงสมาชิกในแถวลำดับ เช่น `arr[i]`

การดำเนินการเหล่านี้แต่ละอย่างเราจะสมมติว่าใช้เวลาคงที่ (Constant Time) บนคอมพิวเตอร์สมัยใหม่ ดังนั้นจำนวนการดำเนินการพื้นฐานจึงเป็นตัววัดที่ดีของเวลาที่ขั้นตอนวิธีใช้

2.1.4 ทำไมถึงใช้ Asymptotic Analysis แทนการนับจำนวนแน่นอน

สมมติว่าขั้นตอนวิธี A ใช้ $3n^2 + 5n + 100$ การดำเนินการ และขั้นตอนวิธี B ใช้ $2n^2 + 8n + 50$ การดำเนินการ เมื่อ n มีค่าเล็กๆ เช่น $n = 10$ ขั้นตอนวิธี A ใช้ $300 + 50 + 100 = 450$ ครั้ง ส่วน B ใช้ $200 + 80 + 50 = 330$ ครั้ง ดูเหมือนขั้นตอนวิธี B จะเร็วกว่า แต่เมื่อ $n = 10000$ ขั้นตอนวิธี A ใช้ประมาณ 3×10^8 ครั้ง ส่วน B ใช้ประมาณ 2×10^8 ครั้ง — อย่างไรก็ตามทั้งสองขั้นตอนวิธีอยู่ในระดับ 10^8 ครั้งเหมือนกัน กล่าวคือ เมื่อ n มีค่ามากๆ พจน์ n^2 จะ “กลืน” พจน์ที่เล็กกว่าทั้งหมด

หลักการของ **Asymptotic Analysis** คือ เราสนใจอัตราการเติบโต (Growth Rate) ของเวลาเมื่อ $n \to \infty$ ไม่ใช่ค่าแน่นอน ทำให้เราไม่ต้องนับค่าคงที่ที่ซับซ้อน และสามารถเปรียบเทียบขั้นตอนวิธีได้อย่างมีประสิทธิภาพ

หลักการสำคัญ

เมื่อ n มีขนาดใหญ่ พจน์ที่โตเร็วที่สุดจะกลืนพจน์อื่นทั้งหมด ดังนั้นเราจึงสนใจเฉพาะพจน์ที่โตเร็วที่สุด และตัดค่าคงที่ออก เช่น $3n^2 + 5n + 100$ จะกลายเป็น $\mathcal{O}(n^2)$

2.2 สัญกรณ์ Big-O

สัญกรณ์ Big-O เป็นเครื่องมือหลักที่ใช้บอกความซับซ้อนของเวลาของขั้นตอนวิธี[2] โดยให้ขอบเขตบน (Upper Bound) ของอัตราการเติบโต กล่าวคือ มันบอกว่าเวลาที่ขั้นตอนวิธีใช้จะ “ไม่เกิน” ระดับใด

2.2.1 นิยามอย่างเป็นทางการ

นิยาม 2.1 (สัญกรณ์ Big-O). กำหนดให้ $f(n)$ และ $g(n)$ เป็นฟังก์ชันบนจำนวนเต็มบวก เรากล่าวว่า $f(n) = \mathcal{O}(g(n))$ ก็ต่อเมื่อมีค่าคงที่ $c > 0$ และ $n_0 \geq 1$ ที่ทำให้

$$f(n) \leq c \cdot g(n) \qquad \text{สำหรับทุก } n \geq n_0$$

กล่าวอีกนัยหนึ่ง Big-O บอกว่า สำหรับ n ที่มากพอ ฟังก์ชัน $f(n)$ จะไม่เกิน $g(n)$ คูณด้วยค่าคงที่บางค่า ยกตัวอย่างเช่น $3n^2 + 5n + 100 = \mathcal{O}(n^2)$ เพราะเมื่อเลือก $c = 8$ และ $n_0 = 5$ จะได้ว่า $3n^2 + 5n + 100 \leq 8n^2$ สำหรับทุก $n \geq 5$

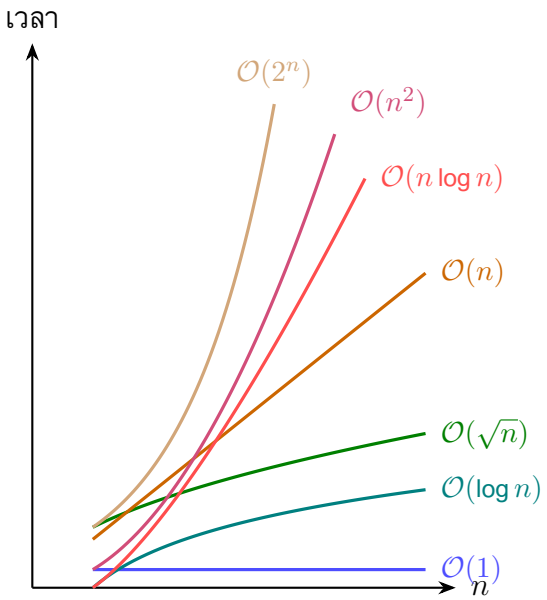
2.2.2 ความเข้าใจเชิงสังหรณ์ด้วยกราฟ

ลองจินตนาการกราฟของ $f(n)$ และ $c \cdot g(n)$ บนระนาบเดียวกัน สัญกรณ์ $f(n) = \mathcal{O}(g(n))$ หมายความว่า หลังจากจุด n_0 เป็นต้นไป เส้นกราฟของ $f(n)$ จะอยู่ **ใต้** เส้นกราฟของ $c \cdot g(n)$ เสมอ ค่าคงที่ c ช่วย “ยก” เส้นกราฟ $g(n)$ ให้สูงพอที่จะครอบคลุม $f(n)$ ได้ ส่วน n_0 บอกว่าต้องดูเฉพาะ n ที่มากพอเท่านั้น ส่วน n ที่น้อยๆ นั้นไม่สำคัญ

ตัวอย่าง: พิสูจน์ 3n+2

เราต้องหา c และ n_0 ที่ทำให้ $3n + 2 \leq cn$ สำหรับทุก $n \geq n_0$
เลือก $c = 4$: $3n + 2 \leq 4n$ เมื่อ $n \geq 2$
ดังนั้น $3n + 2 = \mathcal{O}(n)$ โดย $c = 4$ และ $n_0 = 2$

รูปที่ 2.1 แสดงกราฟเปรียบเทียบอัตราการเติบโตของฟังก์ชันความซับซ้อนต่างๆ ซึ่งช่วยให้เห็นภาพว่าทำไม $\mathcal{O}(n^2)$ ถึงช้ากว่า $\mathcal{O}(n \log n)$ มากเพียงใดเมื่อ n มีขนาดใหญ่



รูปที่ 2.1 กราฟเปรียบเทียบอัตราการเติบโตของฟังก์ชันความซับซ้อนต่างๆ (มีการปรับค่า c เพื่อให้เห็นความแตกต่างชัดเจน)

2.2.3 วิธีหา Big-O จากโค้ด

วิธีง่ายๆ ในการหา Big-O จากโค้ดคือ:

1. หาส่วนที่ทำงานช้าที่สุด (Bottleneck) ของโปรแกรม
2. นับจำนวนครั้งที่ลูปชั้นในสุดทำงาน
3. ละทิ้งค่าคงที่และพจน์ที่เล็กกว่า

```
// Part 1: O(1)
int a = 5;
// Part 2: O(n)
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
```

```cpp
// Part 3: O(n^2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // ...
    }
}
// Total complexity = O(n^2) (choose the largest term)
```

ตัวอย่างโค้ดที่ 2.3: ตัวอย่าง: หา Big-O จากโค้ด

เมื่อโปรแกรมมีหลายส่วนที่ทำงานตามลำดับ ความซับซ้อนรวมคือพจน์ที่ใหญ่ที่สุด เพราะเมื่อ n มากๆ พจน์ที่ใหญ่ที่สุดจะครอบงำพจน์อื่นทั้งหมด

2.2.4 คลาส Big-O ที่พบบ่อย

ตารางต่อไปแสดงคลาสความซับซ้อนที่พบบ่อย เรียงจากเร็วสุดไปช้าสุด พร้อมตัวอย่างขั้นตอนวิธีที่มีความซับซ้อนดังกล่าว

ตารางที่ 2.1 คลาสความซับซ้อน Big-O ที่พบบ่อย

ความซับซ้อน	ชื่อ	ตัวอย่างขั้นตอนวิธี
$\mathcal{O}(1)$	คงที่ (Constant)	เข้าถึงสมาชิกแถวลำดับ `arr[i]`
$\mathcal{O}(\log n)$	ลอการิทึม (Logarithmic)	Binary Search
$\mathcal{O}(\sqrt{n})$	รากที่สอง (Square Root)	ตรวจสอบจำนวนเฉพาะเบื้องต้น
$\mathcal{O}(n)$	เชิงเส้น (Linear)	Linear Search, หาผลรวมแถวลำดับ
$\mathcal{O}(n \log n)$	Linearithmic	Merge Sort, Quick Sort (เฉลี่ย)
$\mathcal{O}(n^2)$	กำลังสอง (Quadratic)	Bubble Sort, Selection Sort
$\mathcal{O}(n^3)$	กำลังสาม (Cubic)	Floyd-Warshall
$\mathcal{O}(2^n)$	เอกซ์โพเนนเชียล (Exponential)	Subset enumeration
$\mathcal{O}(n!)$	แฟกทอเรียล (Factorial)	Permutation enumeration

ลำดับความเร็ว

ลำดับจากเร็วไปช้า: $\mathcal{O}(1) < \mathcal{O}(\log n) < \mathcal{O}(\sqrt{n}) < \mathcal{O}(n) < \mathcal{O}(n \log n) < \mathcal{O}(n^2) < \mathcal{O}(n^3) < \mathcal{O}(2^n) < \mathcal{O}(n!)$

ข้อสังเกต: $\mathcal{O}(n \log n)$ อยู่ระหว่าง $\mathcal{O}(n)$ กับ $\mathcal{O}(n^2)$ — เป็นจุดเปลี่ยนสำคัญที่แยกขั้นตอนวิธีที่ “ยังใช้ได้” จากขั้นตอนวิธีที่ “ช้าเกินไป” สำหรับข้อมูลขนาดใหญ่

2.3 สัญกรณ์ Omega และ Theta

นอกจากสัญกรณ์ Big-O ซึ่งให้ขอบเขตบนแล้ว ยังมีสัญกรณ์อีก 2 ชนิดที่สำคัญ ได้แก่ Big-Omega (Ω) ซึ่งให้ขอบเขตล่าง และ Big-Theta (Θ) ซึ่งให้ขอบเขตแน่นหนา สัญกรณ์เหล่านี้ช่วยให้เราอธิบายพฤติกรรมของขั้นตอนวิธีได้แม่นยำขึ้น

2.3.1 สัญกรณ์ Big-Omega (Ω)

นิยาม 2.2 (สัญกรณ์ Big-Omega). เรากล่าวว่า $f(n) = \Omega(g(n))$ ก็ต่อเมื่อมีค่าคงที่ $c > 0$ และ $n_0 \geq 1$ ที่ทำให้

$$f(n) \geq c \cdot g(n) \quad \text{สำหรับทุก } n \geq n_0$$

Big-Omega บอกว่าขั้นตอนวิธีใช้เวลา **อย่างน้อย** ในระดับ $g(n)$ กล่าวคือ $g(n)$ เป็นขอบเขตล่าง (Lower Bound) ของเวลาทำงาน ตัวอย่างเช่น $3n^2 + 5n = \Omega(n^2)$ เพราะ $3n^2 + 5n \geq n^2$ สำหรับทุก $n \geq 1$

2.3.2 สัญกรณ์ Big-Theta (Θ)

นิยาม 2.3 (สัญกรณ์ Big-Theta). เรากล่าวว่า $f(n) = \Theta(g(n))$ ก็ต่อเมื่อ $f(n) = \mathcal{O}(g(n))$ และ $f(n) = \Omega(g(n))$ พร้อมกัน

กล่าวอีกนัยหนึ่ง มีค่าคงที่ $c_1, c_2 > 0$ และ $n_0 \geq 1$ ที่ทำให้

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{สำหรับทุก } n \geq n_0$$

Big-Theta บอกว่าขั้นตอนวิธีใช้เวลา “พอๆ” กับ $g(n)$ กล่าวคือ $g(n)$ เป็นขอบเขตแน่นหนา (Tight Bound) ทั้งบนและล่าง เมื่อเราพูดว่า “ขั้นตอนวิธีนี้มีความซับซ้อน $\Theta(n^2)$” หมายความว่าเวลาทำงานเติบโตเป็น n^2 อย่างแน่นอน ไม่ใช่แค่ “ไม่เกิน” n^2

2.3.3 ความสัมพันธ์ระหว่าง $\mathcal{O}$, Ω, Θ

สามสัญกรณ์นี้มีความสัมพันธ์กันดังนี้:

- $f(n) = \Theta(g(n))$ ก็ต่อเมื่อ $f(n) = \mathcal{O}(g(n))$ **และ** $f(n) = \Omega(g(n))$
- ถ้า $f(n) = \Theta(g(n))$ แล้ว $f(n) = \mathcal{O}(g(n))$ เสมอ (แต่ไม่จริงในทางกลับกัน)
- ถ้า $f(n) = \Theta(g(n))$ แล้ว $f(n) = \Omega(g(n))$ เสมอ (แต่ไม่จริงในทางกลับกัน)

ตัวอย่าง: สัญกรณ์ทั้งสามสำหรับ f(n)

- สมมติให้ $f(n) = 3n^2 + 5n + 7$
- $f(n) = \mathcal{O}(n^2)$ เพราะ $3n^2 + 5n + 7 \leq 4n^2$ เมื่อ $n \geq 7$

- $f(n) = \Omega(n^2)$ เพราะ $3n^2 + 5n + 7 \geq n^2$ เมื่อ $n \geq 1$
- $f(n) = \Theta(n^2)$ เพราะเป็นไปตามทั้ง $\mathcal{O}(n^2)$ และ $\Omega(n^2)$

สังเกตว่าเราสามารถพูดได้ว่า $f(n) = \mathcal{O}(n^3)$ เพราะ $3n^2 + 5n + 7 \leq n^3$ เมื่อ n มากพอ แต่นี่เป็นขอบเขตที่ "หลวม" (Loose Bound) เพราะ $f(n)$ โตช้ากว่า n^3 อย่างมาก

2.3.4 ทำไม Big-O จึงใช้บ่อยที่สุดในการแข่งขัน

ในการโปรแกรมเชิงแข่งขัน เราใช้ Big-O เป็นหลักเพราะเหตุผลดังนี้:

1. **ความปลอดภัย** — Big-O ให้ขอบเขตบน หมายความว่าถ้าเราบอกว่าขั้นตอนวิธีเป็น $\mathcal{O}(n^2)$ โปรแกรมจะไม่ช้ากว่า n^2 แน่นอน ทำให้เรามั่นใจได้ว่าจะทันเวลา
2. **ง่ายต่อการคำนวณ** — การหา Big-O ทำได้ง่ายกว่าการหา Theta หรือ Omega เพราะเพียงแค่หาขอบเขตบน
3. **เพียงพอต่อการตัดสินใจ** — ในการแข่งขัน เราต้องการรู้ว่าขั้นตอนวิธี "ช้าเกินไปหรือไม่" ซึ่ง Big-O ตอบคำถามนี้ได้โดยตรง

ข้อควรระวัง

แม้ว่าเรามักพูดกันว่า "ขั้นตอนวิธีนี้เป็น $\mathcal{O}(n \log n)$" แต่ในความเป็นจริงสิ่งที่เราหมายถึงมักเป็น $\Theta(n \log n)$ กล่าวคือ ขั้นตอนวิธีนั้นใช้เวลาพอๆ กับ $n \log n$ ไม่ใช่แค่ "ไม่เกิน" ในหนังสือเล่มนี้ เราจะใช้ Big-O ตามธรรมเนียมของการแข่งขัน แต่ผู้อ่านควรเข้าใจความแตกต่าง

2.4 การวิเคราะห์ความซับซ้อนพื้นฐาน

ในหัวข้อนี้ เราจะเรียนรู้กฎพื้นฐานสำหรับการวิเคราะห์ความซับซ้อนของโค้ด รวมถึงการวิเคราะห์ลูปอย่างง่าย ซึ่งเป็นโครงสร้างที่พบมากที่สุดในการแข่งขัน

2.4.1 กฎพื้นฐาน

ทฤษฎีบท 2.1 (กฎการดำเนินการตามลำดับ (Sequence Rule))**.** ถ้าบล็อกของโค้ดประกอบด้วยส่วน A ที่ใช้เวลา $\mathcal{O}(f(n))$ ตามด้วยส่วน B ที่ใช้เวลา $\mathcal{O}(g(n))$ แล้วเวลารวมเป็น $\mathcal{O}(f(n) + g(n)) = \mathcal{O}(\max(f(n), g(n)))$

กฎนี้หมายความว่าเมื่อโค้ดทำงานตามลำดับ เราเลือกพจน์ที่ใหญ่ที่สุดมาเป็นความซับซ้อนรวม เช่น ถ้าส่วนแรกเป็น $\mathcal{O}(n)$ และส่วนที่สองเป็น $\mathcal{O}(n^2)$ ความซับซ้อนรวมคือ $\mathcal{O}(n^2)$

ทฤษฎีบท 2.2 (กฎการเลือก (Selection Rule))**.** ถ้าโค้ดมีเงื่อนไข `if-else` โดยกรณี A ใช้เวลา $\mathcal{O}(f(n))$ และกรณี B ใช้เวลา $\mathcal{O}(g(n))$ แล้วเวลาในกรณีเลวร้ายที่สุดเป็น $\mathcal{O}(\max(f(n), g(n)))$

เราเลือกกรณีที่ช้ากว่าเพราะเราวิเคราะห์กรณีที่เลวร้ายที่สุด ตัวอย่างเช่น:

```
if (n > 1000) {
    // Condition A: O(n^2)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            // ...
} else {
    // Condition B: O(n)
    for (int i = 0; i < n; i++)
        // ...
}
// Complexity: O(n^2)
```

ตัวอย่างโค้ดที่ 2.4: ตัวอย่างกฎการเลือก

2.4.2 การวิเคราะห์ลูป

ลูปเดี่ยว (Single Loop): ลูปที่วน n ครั้ง และแต่ละรอบใช้เวลาคงที่ จะมีความซับซ้อน $\mathcal{O}(n)$

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];    // O(1) per iteration
}
// Total: O(n)
```

ตัวอย่างโค้ดที่ 2.5: ลูปเดี่ยว: O(n)

หรืออีกตัวอย่างที่วนลูปลดลง แต่ก็ยังเป็น $\mathcal{O}(n)$

```
for (int i = n; i > 0; i--) {
    // ...
}
```

ตัวอย่างโค้ดที่ 2.6: ลูปนับถอยหลังจาก n ถึง 1

ลูปซ้อนที่การเพิ่มขึ้นหรือลดลงไม่ใช่ทีละหนึ่ง: บางครั้งจำนวนรอบของลูปไม่ได้ตรงไปตรงมาเหมือนตัวอย่างข้างต้น ลองดูตัวอย่างต่อไปนี้:

```
for (int i = 0; i < n; i += 2) {
    // ...
}
// n/2 iterations => still O(n)
```

ตัวอย่างโค้ดที่ 2.7: ลูปที่วนเพียงครึ่งหนึ่ง

เมื่อโครงสร้างลูปมีความซับซ้อน แนะนำว่าให้ลองแทนค่า พบว่าลูปนี้จะวนเพียง $n/2$ รอบ แต่ $n/2$ ก็ยังเป็น $\Theta(n)$ เพราะเราละทิ้งค่าคงที่ อีกตัวอย่าง กรณี i เพิ่มขึ้นเป็น 2 เท่าทุกรอบ (Logarithmic Loop):

```
for (int i = 1; i < n; i *= 2) {
    // O(1)
}
```

ตัวอย่างโค้ดที่ 2.8: ลูปแบบลอการิทึม: O(log n)

ลูปนี้วนเมื่อ $i = 1, 2, 4, 8, \ldots, 2^k$ โดยที่ $2^k < n$ ดังนั้นจำนวนรอบคือ $k = \lfloor \log_2 n \rfloor + 1$ ซึ่งเป็น $\mathcal{O}(\log n)$

เทคนิค: ลูป `i *= 2`

เมื่อใดก็ตามที่ตัวแปรลูปคูณด้วยค่าคงที่ (เช่น 2) ทุกรอบ จำนวนรอบจะเป็น $\mathcal{O}(\log n)$ เช่นเดียวกัน:

- `j *= 3` $\Rightarrow \mathcal{O}(\log_3 n) = \mathcal{O}(\log n)$
- `j /= 2` $\Rightarrow \mathcal{O}(\log n)$

เราไม่ระบุฐานของลอการิทึม เพราะ $\log_a n = \frac{\log_b n}{\log_b a}$ ซึ่งต่างกันแค่ค่าคงที่

ลูปซ้อน 2 ชั้น (Nested Loop): ลูปนอกวน n ครั้ง และแต่ละรอบมีลูปในที่วน n ครั้ง จะมีความซับซ้อน $\mathcal{O}(n^2)$

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // O(1) per iteration
    }
}
// Total: n * n = O(n^2)
```

ตัวอย่างโค้ดที่ 2.9: ลูปซ้อน 2 ชั้น: O(n^2)

ลูปซ้อนอิสระต่อกัน เมื่อลูปในไม่ขึ้นกับตัวแปรของลูปนอก ทั้งสองลูปเป็นอิสระต่อกัน:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        // O(1)
    }
}
// Total: n * m = O(nm)
```

ตัวอย่างโค้ดที่ 2.10: ลูปซ้อนอิสระ: O(n · m)

กรณีนี้ง่ายที่สุด — จำนวนรอบรวมคือผลคูณของจำนวนรอบลูปนอกและลูปใน ถ้า $m = n$ ก็จะเป็น $\mathcal{O}(n^2)$

ลูปซ้อนที่มีการพึ่งพา (Dependent Nested Loops) เมื่อขอบเขตของลูปในขึ้นกับตัวแปรของลูปนอก จะเกิดรูปแบบสามเหลี่ยม (Triangular Pattern):

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        // O(1)
    }
}
```

ตัวอย่างโค้ดที่ 2.11: ลูปซ้อนแบบสามเหลี่ยม: O(n^2)

จำนวนรอบรวมคือ:

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2)$$

แม้ว่าจำนวนรอบจะเป็น $\frac{n(n+1)}{2}$ ซึ่งเท่ากับประมาณ $\frac{1}{2}n^2$ แต่เราละทิ้งค่าคงที่ $\frac{1}{2}$ ดังนั้นความซับซ้อนยังคงเป็น $\mathcal{O}(n^2)$

รูปแบบลูปที่ซับซ้อนขึ้น รูปแบบที่ 1: i จาก 1 ถึง n, j จาก 1 ถึง i และ k จาก 1 ถึง j:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        for (int k = 1; k <= j; k++) {
            // O(1)
        }
    }
}
```

ตัวอย่างโค้ดที่ 2.12: ลูปซ้อน 3 ชั้นแบบสามเหลี่ยม

จำนวนรอบรวมคือ:

$$\sum_{i=1}^{n}\sum_{j=1}^{i} j = \sum_{i=1}^{n} \frac{i(i+1)}{2} = \frac{n(n+1)(n+2)}{6} = \Theta(n^3)$$

รูปแบบลูปที่ซับซ้อนขึ้น รูปแบบที่ 2: ผสมลูปเชิงเส้นกับลูปลอการิทึม:

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j *= 2) {
        // O(1)
    }
}
// Total: n * log n = O(n log n)
```

ตัวอย่างโค้ดที่ 2.13: ลูปผสม: O(n log n)

รูปแบบลูปที่ซับซ้อนขึ้น รูปแบบที่ 3: ลูปซ้อนที่ขอบเขตลดลงเป็นครึ่งหนึ่ง:

```cpp
for (int i = n; i >= 1; i /= 2) {
    for (int j = 0; j < i; j++) {
        // O(1)
    }
}
```

ตัวอย่างโค้ดที่ 2.14: ลูปซ้อน แต่ขนาดลูปในลดครึ่งทุกครั้ง (รวม O(n))

จำนวนรอบรวมคือ $n + \frac{n}{2} + \frac{n}{4} + \cdots + 1 \leq 2n = \mathcal{O}(n)$ (อนุกรมเรขาคณิต)

2.4.3 ตารางสรุปรูปแบบลูปที่พบบ่อย

การวิเคราะห์ลูปรูปแบบต่างๆ ที่พบบ่อยแสดงได้ดังตารางที่ 2.2

ตารางที่ 2.2 สรุปรูปแบบลูปและความซับซ้อน

รูปแบบลูป	ความซับซ้อน
`for(i=0; i<n; i++)`	$\mathcal{O}(n)$
`for(i=0; i<n; i+=2)`	$\mathcal{O}(n)$
`for(j=1; j<n; j*=2)`	$\mathcal{O}(\log n)$
`for(i=0; i<n; i++) for(j=0; j<m; j++)`	$\mathcal{O}(nm)$
`for(i=0; i<n; i++) for(j=0; j<n; j++)`	$\mathcal{O}(n^2)$
`for(i=1; i<=n; i++) for(j=1; j<=i; j++)`	$\mathcal{O}(n^2)$
`for(i=0; i<n; i++) for(j=1; j<n; j*=2)`	$\mathcal{O}(n \log n)$
`for(i=n; i>=1; i/=2) for(j=0; j<i; j++)`	$\mathcal{O}(n)$

ข้อควรระวัง: อย่าสรุปเร็วเกินไป

การวิเคราะห์ลูปซ้อนไม่ใช่แค่นับจำนวนชั้นของลูปเสมอไป ลูปซ้อน 2 ชั้นอาจไม่ได้เป็น $\mathcal{O}(n^2)$ เสมอ และลูปซ้อน 3 ชั้นอาจไม่ได้เป็น $\mathcal{O}(n^3)$ ต้องพิจารณา **จำนวนรอบรวม** ตัวอย่างเช่น ลูปซ้อน 2 ชั้น ที่ลูปในเพิ่มแบบลอการิทึมให้ $\mathcal{O}(n \log n)$ ไม่ใช่ $\mathcal{O}(n^2)$

2.5 การวิเคราะห์ขั้นตอนวิธีแบบเรียกซ้ำ

ขั้นตอนวิธีแบบเรียกซ้ำ (Recursive Algorithms) เป็นอีกรูปแบบหนึ่งที่พบได้บ่อยในการแข่งขัน เช่น Binary Search, Merge Sort, Quick Sort และ Dynamic Programming วิธีวิเคราะห์ความซับซ้อนของขั้นตอนวิธีเหล่านี้แตกต่างจากลูปธรรมดา เราต้องใช้ **สมการเวียนเกิด** (Recurrence Relation) ในการอธิบายเวลาทำงาน

2.5.1 สมการเวียนเกิด (Recurrence Relation)

สมการเรียกซ้ำอธิบายเวลาทำงาน $T(n)$ ของขั้นตอนวิธีแบบเรียกซ้ำในรูปของเวลาทำงานของปัญหาย่อยที่เล็กกว่า ยกตัวอย่างเช่น ถ้าขั้นตอนวิธีเรียกตัวเอง 2 ครั้งบนปัญหาขนาด $n/2$ และใช้เวลา $\mathcal{O}(n)$ ในการรวมผล สมการเรียกซ้ำจะเป็น:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

2.5.2 การแก้สมการเรียกซ้ำอย่างง่าย

กรณีที่ 1: $T(n) = T(n-1) + \mathcal{O}(1)$

สมการนี้เกิดจากขั้นตอนวิธีที่ลดขนาดปัญหาทีละ 1 และทำงานคงที่ในแต่ละขั้น เช่น Linear Search แบบเรียกซ้ำ:

$$T(n) = T(n-1) + c = T(n-2) + 2c = \cdots = T(0) + nc = \mathcal{O}(n)$$

```
int linearSearch(int arr[], int n, int target) {
    if (n == 0) return -1;
    if (arr[n-1] == target) return n-1;
    return linearSearch(arr, n-1, target);
}
// T(n) = T(n-1) + O(1) = O(n)
```

ตัวอย่างโค้ดที่ 2.15: ตัวอย่าง: Linear Search แบบเรียกซ้ำ

กรณีที่ 2: $T(n) = T(n/2) + \mathcal{O}(1)$

สมการนี้เกิดจากขั้นตอนวิธีที่ลดขนาดปัญหาลงครึ่งหนึ่งในแต่ละขั้น เช่น Binary Search:

$$T(n) = T\left(\frac{n}{2}\right) + c = T\left(\frac{n}{4}\right) + 2c = \cdots = T(1) + c\log_2 n = \mathcal{O}(\log n)$$

```
int binarySearch(int arr[], int lo, int hi, int target) {
    if (lo > hi) return -1;
    int mid = (lo + hi) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target)
        return binarySearch(arr, mid+1, hi, target);
    else
        return binarySearch(arr, lo, mid-1, target);
}
// T(n) = T(n/2) + O(1) = O(log n)
```

ตัวอย่างโค้ดที่ 2.16: ตัวอย่าง: Binary Search

กรณีที่ 3: $T(n) = 2T(n/2) + \mathcal{O}(n)$

สมการนี้เกิดจากขั้นตอนวิธีที่แบ่งปัญหาเป็น 2 ส่วนเท่าๆ กัน และใช้เวลาเชิงเส้นในการรวมผล เช่น Merge Sort:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn = \mathcal{O}(n \log n)$$

```
void mergeSort(int arr[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(arr, lo, mid);        // T(n/2)
    mergeSort(arr, mid+1, hi);      // T(n/2)
    merge(arr, lo, mid, hi);        // O(n)
}
// T(n) = 2T(n/2) + O(n) = O(n log n)
```

ตัวอย่างโค้ดที่ 2.17: ตัวอย่าง: Merge Sort (โครงสร้างหลัก)

2.5.3 วิธีต้นไม้เรียกซ้ำ (Recursion Tree Method)

วิธีต้นไม้เรียกซ้ำเป็นเทคนิคที่ช่วยให้เห็นภาพและเข้าใจสมการเรียกซ้ำได้ดีขึ้น ลองพิจารณา $T(n) = 2T(n/2) + cn$:

- **ระดับที่ 0:** มี 1 ปัญหาขนาด n ใช้เวลา cn
- **ระดับที่ 1:** มี 2 ปัญหาขนาด $n/2$ ใช้เวลา $2 \cdot c \cdot \frac{n}{2} = cn$
- **ระดับที่ 2:** มี 4 ปัญหาขนาด $n/4$ ใช้เวลา $4 \cdot c \cdot \frac{n}{4} = cn$
- **ระดับที่ k:** มี 2^k ปัญหาขนาด $n/2^k$ ใช้เวลา cn
- **ระดับสุดท้าย:** เมื่อ $n/2^k = 1$ คือ $k = \log_2 n$ มี n ปัญหาขนาด 1

เวลารวม $= cn + cn + cn + \cdots + cn$ (ทั้งหมด $\log_2 n + 1$ ระดับ) $= cn(\log_2 n + 1) = \mathcal{O}(n \log n)$

เทคนิค: ต้นไม้เรียกซ้ำ

สำหรับสมการเรียกซ้ำ $T(n) = aT(n/b) + f(n)$ ให้คิดภาพ “ต้นไม้” ที่แต่ละระดับมีการแตกกิ่ง a กิ่ง และขนาดปัญหาลดลง b เท่า เวลารวมคือผลรวมของเวลาทุกระดับ ถ้าเวลาแต่ละระดับเท่ากัน (เช่น cn ทุกระดับ) ก็คูณด้วยจำนวนระดับ (ความสูงของต้นไม้)

2.5.4 Master Theorem (เวอร์ชันย่อ)

Master Theorem เป็นเครื่องมือที่ทรงพลังสำหรับแก้สมการเรียกซ้ำในรูปแบบทั่วไป:

ทฤษฎีบท 2.3 (Master Theorem). กำหนดสมการเรียกซ้ำ

$$T(n) = aT\left(\frac{n}{b}\right) + \mathcal{O}(n^d)$$

โดยที่ $a \geq 1$, $b > 1$, และ $d \geq 0$ แล้ว:

1. ถ้า $d < \log_b a$ แล้ว $T(n) = \mathcal{O}(n^{\log_b a})$
2. ถ้า $d = \log_b a$ แล้ว $T(n) = \mathcal{O}(n^d \log n) = \mathcal{O}(n^{\log_b a} \log n)$
3. ถ้า $d > \log_b a$ แล้ว $T(n) = \mathcal{O}(n^d)$

ตัวอย่างการใช้ Master Theorem

ตัวอย่าง 1: $T(n) = 2T(n/2) + \mathcal{O}(n)$
$a = 2, b = 2, d = 1$; $\log_b a = \log_2 2 = 1 = d \Rightarrow$ กรณี 2: $T(n) = \mathcal{O}(n \log n)$

ตัวอย่าง 2: $T(n) = T(n/2) + \mathcal{O}(1)$
$a = 1, b = 2, d = 0$; $\log_b a = \log_2 1 = 0 = d \Rightarrow$ กรณี 2: $T(n) = \mathcal{O}(\log n)$

ตัวอย่าง 3: $T(n) = 3T(n/2) + \mathcal{O}(n)$
$a = 3, b = 2, d = 1$; $\log_b a = \log_2 3 \approx 1.585 > d \Rightarrow$ กรณี 1: $T(n) = \mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$

ตัวอย่าง 4: $T(n) = 2T(n/2) + \mathcal{O}(n^2)$
$a = 2, b = 2, d = 2$; $\log_b a = 1 < d \Rightarrow$ กรณี 3: $T(n) = \mathcal{O}(n^2)$

2.6 การประมาณค่าจากข้อจำกัดโจทย์

หนึ่งในทักษะที่สำคัญที่สุดในการโปรแกรมเชิงแข่งขันคือ **การอ่านข้อจำกัด (Constraints) ของโจทย์แล้วรู้ทันทีว่าควรใช้ขั้นตอนวิธีประเภทใด** ทักษะนี้ช่วยประหยัดเวลาได้มาก เพราะไม่ต้องลองผิดลองถูก และช่วยกรองขั้นตอนวิธีที่ช้าเกินไปออกตั้งแต่แรก[3]

2.6.1 หลักการสำคัญ: 10^8 การดำเนินการต่อวินาที

ในการแข่งขันส่วนใหญ่ คอมพิวเตอร์สามารถทำการดำเนินการพื้นฐานได้ประมาณ 10^8 ถึง 10^9 ครั้งต่อวินาที เมื่อ Time Limit ปกติอยู่ที่ประมาณ 1–2 วินาที เราสามารถสรุปได้ว่า:

กฎเกณฑ์ง่าย ๆ

เวลาจำกัด $\approx$ 1 วินาที $\Rightarrow$ โปรแกรมควรทำการดำเนินการไม่เกินประมาณ 10^8 ครั้ง

ด้วยกฎเกณฑ์นี้ เมื่อเรารู้ข้อจำกัด N ของโจทย์ เราสามารถคำนวณได้ว่าความซับซ้อนสูงสุดที่ยังทันเวลาคือเท่าไร

2.6.2 วิธีใช้ตารางเลือกขั้นตอนวิธี

เมื่ออ่านโจทย์แล้ว ให้ทำตามขั้นตอนดังนี้:

1. **อ่านข้อจำกัด** — หาค่า N สูงสุดจากโจทย์ (มักอยู่ในส่วน “Constraints” หรือ “ข้อจำกัด”)
2. **ดูตาราง** — จากข้อจำกัด N หาความซับซ้อนสูงสุดที่ยอมรับได้
3. **เลือกขั้นตอนวิธี** — เลือกขั้นตอนวิธีที่มีความซับซ้อนอยู่ในขอบเขตที่ยอมรับได้
4. **ตรวจสอบ** — ก่อนเขียนโค้ด คำนวณคร่าวๆ ว่าจำนวนการดำเนินการไม่เกิน 10^8

ตารางที่ 2.3 ข้อจำกัด N เทียบกับความซับซ้อนที่ยอมรับได้ (Time Limit $\approx$ 1 วินาที)

ข้อจำกัด N	ความซับซ้อนที่ใช้ได้	ตัวอย่างขั้นตอนวิธี
$N \leq 10$	$\mathcal{O}(n!)$ หรือ $\mathcal{O}(2^n)$	ลองทุก Permutation, Subset
$N \leq 20$	$\mathcal{O}(2^n)$	Bitmask DP, Subset enumeration
$N \leq 500$	$\mathcal{O}(n^3)$	Floyd-Warshall, Matrix Chain
$N \leq 5000$	$\mathcal{O}(n^2)$	DP 2 มิติ, Bubble Sort
$N \leq 10^5$	$\mathcal{O}(n\sqrt{n})$ หรือ $\mathcal{O}(n \log^2 n)$	MO’s Algorithm, Divide & Conquer on Tree
$N \leq 10^6$	$\mathcal{O}(n \log n)$	Merge Sort, Segment Tree
$N \leq 10^7$	$\mathcal{O}(n)$	Sieve of Eratosthenes, Prefix Sum
$N \leq 10^8$	$\mathcal{O}(n)$ (โค้ดเรียบง่าย)	Linear Scan
$N \leq 10^{18}$	$\mathcal{O}(\log n)$ หรือ $\mathcal{O}(1)$	Binary Exponentiation, Math

ตัวอย่าง: การใช้ตารางเลือกขั้นตอนวิธี

โจทย์: เรียงลำดับแถวลำดับขนาด N ($N \leq 10^5$)
จากตาราง: $N \leq 10^5 \Rightarrow$ ต้องการ $\mathcal{O}(n \log n)$ หรือเร็วกว่า

- Bubble Sort ($\mathcal{O}(n^2)$): 10^{10} การดำเนินการ $\Rightarrow$ ช้าเกินไป $\times$
- Merge Sort ($\mathcal{O}(n \log n)$): $10^5 \times 17 \approx 1.7 \times 10^6 \Rightarrow$ ทันเวลา ✓
- `sort()` ของ STL ($\mathcal{O}(n \log n)$): เช่นเดียวกัน ✓

2.6.3 ข้อควรระวัง

ปัจจัยแฝง (Hidden Factors)

บางครั้งความซับซ้อนมีปัจจัยแฝงที่ทำให้ช้ากว่าที่คิด:

- **ปัจจัยคงที่ใหญ่** — ขั้นตอนวิธีที่เป็น $\mathcal{O}(n)$ แต่มีค่าคงที่ใหญ่ (เช่น ต้องเข้าถึงหน่วยความจำแบบกระจาย) อาจช้ากว่าขั้นตอนวิธี $\mathcal{O}(n \log n)$ ที่มีค่าคงที่เล็ก
- **หลายตัวแปร** — บางโจทย์มีข้อจำกัดหลายตัว เช่น $N \leq 10^5$ และ $Q \leq 10^5$ อาจต้องพิจารณาทั้งคู่ เช่น $\mathcal{O}((N+Q)\log N)$ แทนที่จะดูแค่ N
- **Time Limit แตกต่าง** — บางโจทย์ให้เวลา 2–5 วินาที ทำให้ยอมรับขั้นตอนวิธีที่ช้ากว่าได้

ปัจจัยคงที่ (Constant Factors)

แม้ว่าเราจะละทิ้งค่าคงที่ในสัญกรณ์ Big-O แต่ในทางปฏิบัติ ปัจจัยคงที่มีผลอย่างมาก:

- $\mathcal{O}(n)$ ที่มีค่าคงที่ 100 อาจช้ากว่า $\mathcal{O}(n \log n)$ ที่มีค่าคงที่ 1 เมื่อ $n < 10^{30}$
- การเข้าถึงหน่วยความจำแบบสุ่ม (Random Access) ช้ากว่าการเข้าถึงแบบต่อเนื่อง (Sequential) อย่างมากบนฮาร์ดแวร์สมัยใหม่เนื่องจาก Cache
- โค้ดที่ซับซ้อนมากอาจช้ากว่าโค้ดที่เรียบง่ายแม้จะมีความซับซ้อนเท่ากันใน Big-O

ดังนั้น เมื่อ Big-O บอกว่าขั้นตอนวิธี "น่าจะทันเวลา" แต่ใกล้เคียงขอบเขต ควรเขียนโค้ดให้มีประสิทธิภาพสูงสุดและระวังปัจจัยคงที่

ตัวอย่าง: การประมาณเวลาจากโจทย์จริง

โจทย์: ให้แถวลำดับ A ขนาด N ($1 \leq N \leq 200000$) และคิวรี Q ข้อ ($1 \leq Q \leq 200000$) แต่ละคิวรีถามว่ามีคู่ (i, j) ที่ $A_i + A_j = K$ หรือไม่
วิเคราะห์:

- วิธี $\mathcal{O}(n^2 Q)$: $200000^2 \times 200000 = 8 \times 10^{15} \Rightarrow$ ช้าเกินไป $\times$
- วิธี $\mathcal{O}(nQ)$: $200000 \times 200000 = 4 \times 10^{10} \Rightarrow$ ช้าเกินไป $\times$
- วิธี $\mathcal{O}(Q \log n)$: $200000 \times 18 \approx 3.6 \times 10^6 \Rightarrow$ ทันเวลา ✓
  - เรียงแถวลำดับ $\mathcal{O}(n \log n)$ จากนั้นตอบแต่ละคิวรีด้วย Two Pointers หรือ Binary Search $\mathcal{O}(\log n)$
- วิธี $\mathcal{O}(n + Q)$: ใช้ Hash Set ตอบแต่ละคิวรี $\mathcal{O}(1)$ (เฉลี่ย) $\Rightarrow$ เร็วที่สุด ✓

ข้อสังเกต: ต้องพิจารณาทั้ง N และ Q ร่วมกัน ไม่ใช่แค่ N อย่างเดียว

สรุป: ขั้นตอนการเลือกขั้นตอนวิธีจากข้อจำกัด

1. อ่านข้อจำกัดทั้งหมดของโจทย์ (ทุกตัวแปร ไม่ใช่แค่ N)
2. ประมาณจำนวนการดำเนินการจากความซับซ้อนที่คิดไว้
3. เปรียบเทียบกับ 10^8 (สำหรับ Time Limit 1 วินาที)
4. ถ้าเกิน $\Rightarrow$ ต้องหาขั้นตอนวิธีที่เร็วกว่า
5. ถ้าใกล้เคียง $\Rightarrow$ ระวังปัจจัยคงที่ เขียนโค้ดให้มีประสิทธิภาพ
6. ถ้าเหลือเฟือ $\Rightarrow$ สบายใจได้ แต่ระวังข้อผิดพลาดอื่นๆ

2.7 ตารางอ้างอิงเลือกขั้นตอนวิธีตามข้อจำกัด

ในหัวข้อก่อนหน้า เราได้เรียนรู้หลักการประมาณค่าจากข้อจำกัดของโจทย์และดูตารางคร่าวๆ ไปแล้ว ในหัวข้อนี้ เราจะขยายเป็นตารางอ้างอิงที่ครบถ้วนยิ่งขึ้น พร้อมตัวอย่างขั้นตอนวิธีที่ใช้ได้จริงสำหรับแต่ละช่วงของ n เพื่อให้ผู้อ่านสามารถใช้เป็น “เครื่องมือตัดสินใจอย่างรวดเร็ว” เมื่อแข่งขันจริง

ตารางที่ 2.4 ตารางอ้างอิงเลือกขั้นตอนวิธีตามข้อจำกัด

ข้อจำกัด	ความซับซ้อน	ตัวอย่างขั้นตอนวิธี / เทคนิค
$n \leq 10$	$\mathcal{O}(n!)$, $\mathcal{O}(2^n \cdot n)$	Brute Force, Permutation, Traveling Salesman (TSP) แบบลองทุกวิธี, Subset enumeration ทุกชุดย่อย
$n \leq 20$	$\mathcal{O}(2^n)$, $\mathcal{O}(2^n \cdot n)$	Bitmask DP, Meet in the Middle, Subset Sum แบบลองทุกชุด
$n \leq 500$	$\mathcal{O}(n^3)$	Floyd-Warshall (All-Pairs Shortest Path), Matrix Chain Multiplication, DP ที่มี state 3 มิติ
$n \leq 5000$	$\mathcal{O}(n^2)$	DP 2 มิติ, Bubble Sort / Selection Sort, การนับคู่อันดับผกผัน (Inversion Count) แบบง่าย
$n \leq 10^5$	$\mathcal{O}(n \log n)$	Merge Sort, Quick Sort, Segment Tree, Binary Search, BFS/DFS + sorting, Coordinate Compression
$n \leq 10^5$	$\mathcal{O}(n\sqrt{n})$	Mo's Algorithm, Square Root Decomposition
$n \leq 10^6$	$\mathcal{O}(n \log n)$, $\mathcal{O}(n)$	Sieve of Eratosthenes, Prefix Sum, Linear DP, Counting Sort
$n \leq 10^7$	$\mathcal{O}(n)$	Sieve of Eratosthenes (เวอร์ชันปรับปรุง), Prefix Sum, Two Pointers, Sliding Window
$n \leq 10^9$	$\mathcal{O}(\sqrt{n})$, $\mathcal{O}(\log n)$	ตรวจสอบจำนวนเฉพาะ, หาตัวหาร, Binary Search บนคำตอบ, Binary Exponentiation
$n \leq 10^{18}$	$\mathcal{O}(\log n)$, $\mathcal{O}(1)$	Binary Exponentiation (a^n mod m), Matrix Exponentiation, GCD (Euclidean Algorithm), ทฤษฎีจำนวน (Fermat's Little Theorem)

2.7.1 คำแนะนำในการใช้ตาราง

เมื่อเห็นข้อจำกัดของโจทย์ ให้ทำตามขั้นตอนดังนี้:

1. **ระบุค่า n สูงสุด** จากโจทย์ — บางครั้งโจทย์มีหลายตัวแปร เช่น n และ q (จำนวนคำถาม) ให้พิจารณาตัวแปรที่มีผลต่อลูปหลักของขั้นตอนวิธี
2. **ดูช่วงของ n ในตาราง** — เลือกช่วงที่ n สูงสุดตกอยู่

3. **เลือกขั้นตอนวิธีที่มีความซับซ้อนตรงกัน** — ถ้า $n = 10^5$ ขั้นตอนวิธี $\mathcal{O}(n \log n)$ มักเป็นตัวเลือกแรกที่ควรพิจารณา

4. **ตรวจสอบค่าคงที่แฝง** — ถ้าขั้นตอนวิธีมีลูปซ้อนหลายชั้นในแต่ละรอบ ค่าคงที่แฝงอาจทำให้ช้ากว่าที่คาด

เทคนิค: ถ้าไม่แน่ใจ ให้เริ่มจาก $\mathcal{O}(n \log n)$

สำหรับ $n \leq 10^5$ ถ้าไม่แน่ใจว่าจะใช้ขั้นตอนวิธีอะไร ให้ลองคิดดูก่อนว่ามีวิธีแก้แบบ $\mathcal{O}(n \log n)$ หรือไม่ เพราะ $\mathcal{O}(n \log n)$ เป็นความซับซ้อนที่พบได้บ่อยที่สุดสำหรับช่วง n นี้ โดยเฉพาะขั้นตอนวิธีที่ใช้ Sorting, Binary Search, หรือ Tree Structure

2.7.2 กรณีพิเศษ: โจทย์ที่มีหลายตัวแปร

บางโจทย์มีหลายตัวแปร เช่น n จำนวนสมาชิก และ q จำนวนคำถาม ในกรณีนี้ เราต้องพิจารณาทั้งคู่:

ตัวอย่าง: โจทย์ที่มี n และ q

โจทย์กำหนด: $n \leq 10^5$, $q \leq 10^5$
ทางเลือกที่ 1: ตอบแต่ละคำถามใน $\mathcal{O}(n)$ รวม $\mathcal{O}(nq)$ = $\mathcal{O}(10^{10}) \Rightarrow$ **ช้าเกินไป**
ทางเลือกที่ 2: เตรียมข้อมูลล่วงหน้า (Preprocessing) ใน $\mathcal{O}(n \log n)$ แล้วตอบแต่ละคำถามใน $\mathcal{O}(\log n)$ รวม $\mathcal{O}(n \log n + q \log n) \approx \mathcal{O}(2 \times 10^5 \times 17) \approx 3.4 \times 10^6 \Rightarrow$ **ผ่าน**
ทางเลือกที่ 3: Preprocessing ใน $\mathcal{O}(n)$ แล้วตอบแต่ละคำถามใน $\mathcal{O}(1)$ รวม $\mathcal{O}(n + q) \approx 2 \times 10^5$ $\Rightarrow$ **ผ่านแน่นอน**
สรุป: เมื่อเห็น $n, q \leq 10^5$ ให้มองหาขั้นตอนวิธีที่ตอบคำถามได้ใน $\mathcal{O}(\log n)$ หรือ $\mathcal{O}(1)$ ต่อคำถาม

2.8 ตัวอย่างรวม

ในหัวข้อนี้ เราจะมาลองวิเคราะห์โจทย์ ตั้งแต่การอ่านโจทย์ วิเคราะห์ข้อจำกัด เลือกขั้นตอนวิธี จนถึงเขียนโค้ด เพื่อให้ผู้อ่านเห็นภาพรวมของกระบวนการทั้งหมด

2.8.1 ตัวอย่างที่ 1: โจทย์จัดเรียงแถวลำดับ

โจทย์: Array Sorting

กำหนดแถวลำดับ A ขนาด n จงจัดเรียงแถวลำดับจากน้อยไปมาก
ข้อจำกัด: $1 \leq n \leq 10^5$, $1 \leq A_i \leq 10^9$, Time Limit = 1 วินาที

ขั้นที่ 1: วิเคราะห์ข้อจำกัด

$n \leq 10^5$ และ Time Limit = 1 วินาที ดังนั้นเราต้องการขั้นตอนวิธีที่ทำงานไม่เกินประมาณ 10^8 ครั้ง ทำให้ความซับซ้อนที่ยอมรับได้คือ:

- $\mathcal{O}(n)$ — ยอดเยี่ยม แต่ไม่มีขั้นตอนวิธีจัดเรียงแบบเปรียบเทียบที่เป็น $\mathcal{O}(n)$
- $\mathcal{O}(n \log n)$ — เหมาะสม เพราะ $10^5 \times 17 \approx 1.7 \times 10^6 \ll 10^8$
- $\mathcal{O}(n^2)$ — ไม่ได้ เพราะ $(10^5)^2 = 10^{10} \gg 10^8$

ขั้นที่ 2: เลือกขั้นตอนวิธี

จากการวิเคราะห์ เราต้องใช้ขั้นตอนวิธีจัดเรียงที่เป็น $\mathcal{O}(n \log n)$ เช่น Merge Sort, Quick Sort (เฉลี่ย), หรือ Heap Sort ในการแข่งขัน เรามักใช้ `std::sort` ของ C++ ซึ่งใช้ Introsort (ผสม Quick Sort, Heap Sort และ Insertion Sort) ที่รับประกัน $\mathcal{O}(n \log n)$ ในกรณีเลวร้ายที่สุด

ขั้นที่ 3: เขียนโค้ด

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    } // std::sort has O(n log n) complexity
    sort(a.begin(), a.end());
    for (int i = 0; i < n; i++) {
        cout << a[i] << " \n"[i == n-1];
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 2.18: โจทย์จัดเรียงแถวลำดับ: ใช้ std::sort

ขั้นที่ 4: ตรวจสอบ

เมื่อ $n = 10^5$: `sort` ทำงานประมาณ $10^5 \times \log_2(10^5) \approx 1.7 \times 10^6$ ครั้ง ซึ่งเร็วกว่า 10^8 มาก โปรแกรมควรจะทำงานเสร็จภายในหนึ่งวินาที

2.8.2 ตัวอย่างที่ 2: โจทย์กราฟ — หาระยะทางสั้นสุด

โจทย์: Shortest Path on Grid

กำหนดตารางขนาด $n \times n$ แต่ละช่องมีค่าเป็น 0 หรือ 1 (0 = ผ่านได้, 1 = มีอุปสรรค) จงหาเส้นทางสั้นที่สุดจากมุมซ้ายบน $(1, 1)$ ไปมุมขวาล่าง (n, n) เคลื่อนที่ได้เฉพาะขึ้น ลง ซ้าย ขวา

ข้อจำกัด: $1 \leq n \leq 1000$, Time Limit = 1 วินาที

ขั้นที่ 1: วิเคราะห์ข้อจำกัด

จำนวนช่องทั้งหมดคือ $n^2 \leq 10^6$ ดังนั้นเราต้องการขั้นตอนวิธีที่ทำงานไม่เกินประมาณ 10^8 ครั้งบน 10^6 ช่อง:

- $\mathcal{O}(n^2)$ — ยอมรับได้ เพราะ 10^6 การดำเนินการทั้งหมด (ถ้าแต่ละช่องใช้ $\mathcal{O}(1)$)
- $\mathcal{O}(n^2 \log n)$ — ยังพอได้ เพราะ $10^6 \times 10 \approx 10^7 \ll 10^8$
- $\mathcal{O}(n^3)$ — ไม่ได้ เพราะ $10^9 > 10^8$

ขั้นที่ 2: เลือกขั้นตอนวิธี

ปัญหาหาเส้นทางสั้นสุดบนกราฟที่ทุกเส้นเชื่อมมีน้ำหนักเท่ากัน (Unweighted Graph) สามารถแก้ได้ด้วย BFS (Breadth-First Search) ซึ่งมีความซับซ้อน $\mathcal{O}(V + E)$ โดย $V = n^2$ และ $E \leq 4n^2$ (แต่ละช่องมีเส้นเชื่อมได้มากสุด 4 เส้น) ดังนั้น BFS มีความซับซ้อน $\mathcal{O}(n^2)$ ซึ่งผ่านเกณฑ์

ขั้นที่ 3: เขียนโค้ด

```cpp
#include <bits/stdc++.h>
using namespace std;
int dr[] = {0, 0, 1, -1};
int dc[] = {1, -1, 0, 0};
int main() {
    int n;
    cin >> n;
    vector<vector<int>> grid(n, vector<int>(n));
    vector<vector<int>> dist(n, vector<int>(n, -1));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> grid[i][j]; // BFS: O(n^2)
    queue<pair<int,int>> q;
    if (grid[0][0] == 0) {
        q.push({0, 0});
        dist[0][0] = 1;  // Count starting point too
    }
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x + dr[d], ny = y + dc[d];
            if (nx < 0 || nx >= n || ny < 0 || ny >= n) continue;
            if (grid[nx][ny] == 1 || dist[nx][ny] != -1) continue;
            dist[nx][ny] = dist[x][y] + 1;
            q.push({nx, ny});
        }
    }
    cout << dist[n-1][n-1] << "\n";
    return 0;
```

```
}// V = n^2, E <= 4*n^2 => O(V+E) = O(n^2)
```

ตัวอย่างโค้ดที่ 2.19: โจทย์ Shortest Path on Grid: BFS

ขั้นที่ 4: ตรวจสอบ

เมื่อ $n = 1000$: BFS ทำงาน $\mathcal{O}(n^2) = 10^6$ ครั้ง ซึ่งเร็วมาก โปรแกรมควรทำงานเสร็จภายในเวลาไม่กี่มิลลิวินาที

2.8.3 ตัวอย่างที่ 3: โจทย์ทฤษฎีจำนวน — ตรวจสอบจำนวนเฉพาะ

โจทย์: Primality Test

กำหนดจำนวนเต็มบวก n จงตรวจสอบว่า n เป็นจำนวนเฉพาะ (Prime Number) หรือไม่
ข้อจำกัด: $1 \leq n \leq 10^{12}$, Time Limit = 2 วินาที

ขั้นที่ 1: วิเคราะห์ข้อจำกัด

$n \leq 10^{12}$ เป็นค่าที่ใหญ่มาก! เรามีเวลา 2×10^8 ครั้ง ดังนั้น:

- $\mathcal{O}(n)$ — ไม่ได้ เพราะ $10^{12} \gg 10^8$
- $\mathcal{O}(n \log n)$ — ไม่ได้ เหมือนกัน
- $\mathcal{O}(\sqrt{n})$ — ได้ เพราะ $\sqrt{10^{12}} = 10^6 \ll 10^8$
- $\mathcal{O}(\log n)$ — ได้ แต่ต้องใช้ probabilistic test (เช่น Miller-Rabin)

ขั้นที่ 2: เลือกขั้นตอนวิธี

สำหรับ $n \leq 10^{12}$ เราสามารถตรวจสอบจำนวนเฉพาะได้โดยทดสอบการหารด้วยจำนวนเต็ม $2, 3, \ldots, \lfloor\sqrt{n}\rfloor$ ซึ่งมีความซับซ้อน $\mathcal{O}(\sqrt{n})$ วิธีนี้เรียบง่ายและเพียงพอสำหรับ $n \leq 10^{12}$

ขั้นที่ 3: เขียนโค้ด

```
#include <bits/stdc++.h>
using namespace std;
bool isPrime(long long n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}
int main() {
    long long n;
    cin >> n;
```

```
    cout << (isPrime(n) ? "YES" : "NO") << "\n";
    return 0;
}// When n = 10^12 => sqrt(n) = 10^6 => O(10^6)
```

ตัวอย่างโค้ดที่ 2.20: ตรวจสอบจำนวนเฉพาะ: O($\sqrt{n}$)

ขั้นที่ 4: ตรวจสอบ

เมื่อ $n = 10^{12}$: `isPrime` ทำงาน $\mathcal{O}(\sqrt{n}) = 10^6$ ครั้ง ซึ่งเร็วกว่า 10^8 มาก โปรแกรมจะทำงานเสร็จภายในหนึ่งวินาที

สังเกต: ข้อจำกัดบอกทิศทางขั้นตอนวิธี

จาก 3 ตัวอย่างข้างต้น เห็นได้ว่าข้อจำกัดของโจทย์เป็น “แนวทาง” ที่บอกขั้นตอนวิธีที่น่าจะเป็นไปได้:

- $n \leq 10^5 \Rightarrow$ มองหา $\mathcal{O}(n \log n)$ — มักเกี่ยวกับ Sorting, Binary Search, Tree
- $n \leq 10^3 \Rightarrow$ มองหา $\mathcal{O}(n^2)$ — มักเกี่ยวกับ DP 2 มิติ, ลูปซ้อน
- $n \leq 10^{12} \Rightarrow$ มองหา $\mathcal{O}(\sqrt{n})$ หรือ $\mathcal{O}(\log n)$ — มักเกี่ยวกับทฤษฎีจำนวน, Binary Search บนคำตอบ

การฝึก “อ่านข้อจำกัดแล้วรู้ทันที” เป็นทักษะที่พัฒนาได้ด้วยการฝึกทำโจทย์หลายๆ ข้อ อย่างไรก็ตาม นี่เป็นเพียงการคาดเดาเพื่อช่วยในการตัดสินใจ

2.9 การวิเคราะห์ Amortized

ในบางครั้ง ขั้นตอนวิธีอาจมีการดำเนินการบางอย่างที่ “แพง” มากในบางครั้ง แต่โดยรวมแล้วเฉลี่ยต่อการดำเนินการหนึ่งครั้งกลับ “ถูก” การวิเคราะห์ Amortized (Amortized Analysis) คือเครื่องมือที่ช่วยอธิบายปรากฏการณ์นี้[2] ซึ่งมีประโยชน์มากในการทำความเข้าใจโครงสร้างข้อมูลหลายชนิดใน C++ STL

2.9.1 แนวคิดพื้นฐาน

นิยาม 2.4 (การวิเคราะห์ Amortized)**.** การวิเคราะห์ Amortized เป็นวิธีวิเคราะห์ความซับซ้อนของลำดับการดำเนินการ โดยพิจารณา **เวลาเฉลี่ยต่อการดำเนินการหนึ่งครั้ง** ในลำดับที่ยาวที่สุด แม้ว่าการดำเนินการบางครั้งอาจช้ามาก แต่เมื่อรวมกับการดำเนินการที่เร็วอื่นๆ แล้ว เฉลี่ยออกมาจะยังคงถูก

สิ่งสำคัญที่ต้องแยกแยะ:

- **Worst Case** — เวลาที่ช้าที่สุดของการดำเนินการ *หนึ่งครั้ง*
- **Amortized** — เวลาเฉลี่ยต่อการดำเนินการ *หนึ่งครั้ง* เมื่อพิจารณาลำดับการดำเนินการทั้งหมด
- **Average Case** — เวลาเฉลี่ยที่คาดหวังจากการกระจายของข้อมูลนำเข้า (แตกต่างจาก Amortized ที่ไม่ต้องสมมติการกระจายใดๆ)

2.9.2 ตัวอย่าง: Dynamic Array (`vector`)

ตัวอย่างคลาสสิกของการวิเคราะห์ Amortized คือการเพิ่มสมาชิกลงใน Dynamic Array (`std::vector` ใน C++):

```
vector<int> v;
for (int i = 0; i < n; i++) {
    v.push_back(i);
}
```

ตัวอย่างโค้ดที่ 2.21: การเพิ่มสมาชิกลงใน vector

เมื่อ `vector` มีพื้นที่เต็ม การเรียก `push_back` จะต้อง:

1. จองพื้นที่ใหม่ที่ใหญ่ขึ้น 2 เท่า
2. คัดลอกสมาชิกเก่าทั้งหมดไปยังพื้นที่ใหม่
3. คืนพื้นที่เก่า

การคัดลอกนี้ใช้เวลา $\mathcal{O}(n)$ ซึ่ง “แพง” มาก แต่มันเกิดขึ้นไม่บ่อย — ทุกครั้งที่ขนาดเป็นกำลังสอง $(1, 2, 4, 8, 16, \ldots)$ ลองคำนวณจำนวนครั้งรวม:

- เมื่อขนาดเพิ่มจาก 1 เป็น 2: คัดลอก 1 ครั้ง
- เมื่อขนาดเพิ่มจาก 2 เป็น 4: คัดลอก 2 ครั้ง
- เมื่อขนาดเพิ่มจาก 4 เป็น 8: คัดลอก 4 ครั้ง
- $\ldots$
- เมื่อขนาดเพิ่มจาก $n/2$ เป็น n: คัดลอก $n/2$ ครั้ง

จำนวนครั้งรวม $= 1 + 2 + 4 + \cdots + n/2 \leq n - 1 = \mathcal{O}(n)$
ดังนั้นสำหรับ n ครั้งของ `push_back` เวลารวมคือ $\mathcal{O}(n)$ สำหรับการคัดลอกบวกกับ $\mathcal{O}(n)$ สำหรับการแทรกตัวเลข รวมเป็น $\mathcal{O}(n)$ เฉลี่ยต่อครั้งคือ $\mathcal{O}(n)/n = \mathcal{O}(1)$

ทฤษฎีบท 2.4 (Amortized Complexity ของ `vector::push_back`). การเรียก `push_back` บน `std::vector` มีความซับซ้อนแบบ amortized เป็น $\mathcal{O}(1)$ แม้ว่าบางครั้งจะต้องคัดลอกสมาชิกทั้งหมด (ซึ่งใช้เวลา $\mathcal{O}(n)$) แต่เมื่อพิจารณาลำดับของการเรียก n ครั้ง เวลาเฉลี่ยต่อครั้งเป็น $\mathcal{O}(1)$

เทคนิค: จองพื้นที่ล่วงหน้า

ถ้าเรารู้จำนวนสมาชิกล่วงหน้า สามารถใช้ `v.reserve(n)` เพื่อจองพื้นที่พอตั้งแต่แรก จะไม่ต้องคัดลอกเลย:

```
vector<int> v;
v.reserve(n);  // Pre-allocate space for n elements
```

```cpp
for (int i = 0; i < n; i++) {
    v.push_back(i);   // Truly O(1) each time (no reallocation)
}
```

2.9.3 ทำไมบางการดำเนินการที่ดูแพงจึงถูกเฉลี่ย

หลักการสำคัญของการวิเคราะห์ Amortized คือ: **การดำเนินการที่แพงเกิดขึ้นน้อยครั้ง และการดำเนินการที่ถูกเกิดขึ้นบ่อยครั้ง** เมื่อรวมกันแล้ว ค่าเฉลี่ยจึงต่ำ

ตัวอย่างอื่นๆ ที่พบในการแข่งขัน:

- **Disjoint Set Union (Union-Find)** — การเรียก `find` แต่ละครั้งอาจต้องเดินขึ้นไปบนต้นไม้ แต่เมื่อใช้ Path Compression ร่วมกับ Union by Rank การเรียก `find` จะมีค่า amortized เป็น $\mathcal{O}(\alpha(n))$ (ฟังก์ชันอัคเคอร์แมนผกผัน ซึ่งเกือบเป็น $\mathcal{O}(1)$ สำหรับ n ที่เป็นไปได้ในทางปฏิบัติ)
- **HashMap** — การแทรกค่าบางครั้งต้อง rehash ทั้งตาราง (แพง) แต่โดยรวมแล้ว amortized $\mathcal{O}(1)$ ต่อการแทรก
- **Stack** — การดึงสมาชิกออกหลายๆ ครั้ง แต่ละครั้งเป็น $\mathcal{O}(1)$ แม้ว่าจะมีการดึงหลายร้อยครั้ง รวมก็ยังเป็น $\mathcal{O}(n)$

ตัวอย่าง: วิเคราะห์ Multi-stack

พิจารณาโค้ดที่ใช้ `vector` เก็บสมาชิก และมีการ `push_back` และ `pop_back` สลับกัน:

```cpp
vector<int> v;
for (int i = 0; i < n; i++) {
    v.push_back(i);   // amortized O(1)
}
for (int i = 0; i < n; i++) {
    v.pop_back();     // Always O(1)
}
```

`push_back`: amortized $\mathcal{O}(1)$ (อาจช้าบางครั้งจากการคัดลอก)
`pop_back`: $\mathcal{O}(1)$ เสมอ (แค่ลดขนาด ไม่ต้องคัดลอก)
รวม: $\mathcal{O}(n)$ สำหรับ $2n$ การดำเนินการ

ข้อควรระวัง: Amortized $\neq$ Worst Case

อย่าสับสนระหว่าง amortized กับ worst case! แม้ `push_back` จะมีค่า amortized เป็น $\mathcal{O}(1)$ แต่การเรียกครั้งเดียวอาจใช้เวลา $\mathcal{O}(n)$ ในกรณีส่วนใหญ่ของการแข่งขัน amortized ก็เพียงพอ เพราะเราทำการดำเนินการหลายๆ ครั้งติดกัน แต่ถ้าโจทย์ต้องการรับประกันว่าการดำเนินการ *ทุกครั้ง* เร็ว ต้องใช้ worst case analysis แทน

2.10 ความซับซ้อนของหน่วยความจำ

นอกจากความซับซ้อนของเวลาแล้ว อีกมิติหนึ่งที่สำคัญในการแข่งขันคือ **ความซับซ้อนของหน่วยความจำ** (Space Complexity) โจทย์ส่วนใหญ่กำหนด Memory Limit ด้วย และโปรแกรมที่ใช้หน่วยความจำเกินจะถูกตัดสินว่า Memory Limit Exceeded (MLE)

2.10.1 นิยามของความซับซ้อนของหน่วยความจำ

นิยาม 2.5 (ความซับซ้อนของหน่วยความจำ)**.** ความซับซ้อนของหน่วยความจำของขั้นตอนวิธี คือปริมาณหน่วยความจำ **เพิ่มเติม** ที่ขั้นตอนวิธีใช้ นอกเหนือจากข้อมูลนำเข้า เมื่อวัดในรูปของขนาดข้อมูลนำเข้า n

เช่นเดียวกับความซับซ้อนของเวลา เราใช้สัญกรณ์ Big-O ในการบอกความซับซ้อนของหน่วยความจำ เช่น ขั้นตอนวิธีที่ใช้แถวลำดับเพิ่มเติมขนาด n จะมีความซับซ้อนของหน่วยความจำเป็น $\mathcal{O}(n)$

2.10.2 คลาสความซับซ้อนของหน่วยความจำที่พบบ่อย

ตารางที่ 2.5 คลาสความซับซ้อนของหน่วยความจำที่พบบ่อย

ความซับซ้อน	ชื่อ	ตัวอย่าง
$\mathcal{O}(1)$	คงที่ (Constant)	สลับค่า 2 ตัวแปร, หาค่ามากสุดในแถวลำดับ
$\mathcal{O}(n)$	เชิงเส้น (Linear)	แถวลำดับขนาด n, DP 1 มิติ, Prefix Sum
$\mathcal{O}(n \log n)$	Linearithmic	Segment tree, Sparse Table
$\mathcal{O}(n^2)$	กำลังสอง (Quadratic)	เมทริกซ์ $n \times n$, DP 2 มิติ
$\mathcal{O}(\sqrt{n})$	รากที่สอง	Block decomposition

2.10.3 ข้อจำกัดหน่วยความจำในการแข่งขัน

ในการแข่งขันส่วนใหญ่ Memory Limit มักอยู่ที่ 256 MB หรือ 512 MB เราต้องคำนวณว่าแถวลำดับหรือโครงสร้างข้อมูลของเราใช้หน่วยความจำเท่าไร

ตารางที่ 2.6 การใช้หน่วยความจำของแถวลำดับขนาดต่าง ๆ

ชนิดข้อมูล	ขนาดแถวลำดับ	หน่วยความจำที่ใช้
`int` (4 bytes)	10^6	≈ 4 MB
`long long` (8 bytes)	10^6	≈ 8 MB
`int` (4 bytes)	10^7	≈ 40 MB
`long long` (8 bytes)	10^7	≈ 80 MB
`int` (4 bytes)	10^8	≈ 400 MB
`long long` (8 bytes)	10^8	≈ 800 MB
`bool` (1 byte)	10^8	≈ 100 MB
`vector<bool>`	10^8	≈ 12.5 MB

วิธีคำนวณหน่วยความจำ

วิธีคำนวณง่ายๆ: `int arr[1000000]` ใช้ $10^6 \times 4 = 4{,}000{,}000$ bytes ≈ 4 MB

สูตร: หน่วยความจำ (MB) $= \dfrac{\text{จำนวนสมาชิก} \times \text{ขนาดต่อสมาชิก (bytes)}}{10^6}$

ค่าคงที่สำคัญ: `int` $= 4$ bytes, `long long` $= 8$ bytes, `double` $= 8$ bytes, `bool` ≥ 1 byte

2.10.4 การแลกเปลี่ยนระหว่างเวลาและหน่วยความจำ (Time–Space Trade-off)

บ่อยครั้งที่เราสามารถลดเวลาทำงานได้โดยใช้หน่วยความจำมากขึ้น หรือประหยัดหน่วยความจำได้โดยยอมทำงานช้าลง:

ตัวอย่าง: การจดจำด้วย Memoization

สมมติเราต้องคำนวณฟังก์ชัน Fibonacci $f(n)$ ที่มีการเรียกซ้ำหลายครั้ง:

วิธีที่ 1: ไม่จดจำ (ประหยัดหน่วยความจำ)

```
// Time: O(2^n), Space: O(n)
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

ตัวอย่างโค้ดที่ 2.22: ไม่ใช้ Memoization

วิธีที่ 2: ใช้ Memoization (เร็วกว่า แต่ใช้หน่วยความจำมากกว่า)

```
// Time: O(n), Space: O(n)
int memo[MAXN];
bool computed[MAXN];
int fib(int n) {
```

```
    if (n <= 1) return n;
    if (computed[n]) return memo[n];
    computed[n] = true;
    return memo[n] = fib(n-1) + fib(n-2);
}
```

ตัวอย่างโค้ดที่ 2.23: ใช้ Memoization

วิธีที่ 2 ใช้หน่วยความจำ $\mathcal{O}(n)$ เพิ่มขึ้น แต่ลดเวลาจาก $\mathcal{O}(2^n)$ เป็น $\mathcal{O}(n)$ — ซึ่งเป็นการแลกที่คุ้มค่ามาก!

ตัวอย่างอื่นๆ ของ Time–Space Trade-off:

- **Hash Table vs Linear Search** — Hash Table ใช้หน่วยความจำมากกว่า แต่ค้นหาได้เร็วกว่า ($\mathcal{O}(1)$ amortized vs $\mathcal{O}(n)$)
- **Prefix Sum Array** — ใช้หน่วยความจำ $\mathcal{O}(n)$ เพิ่มเติม แต่ลดเวลาหาผลรวมช่วงจาก $\mathcal{O}(n)$ เป็น $\mathcal{O}(1)$
- **Sparse Table** — ใช้หน่วยความจำ $\mathcal{O}(n \log n)$ เพื่อตอบคำถาม Range Minimum Query ใน $\mathcal{O}(1)$

ข้อควรระวัง: MLE

ในการแข่งขัน ข้อผิดพลาดที่พบบ่อยคือประกาศแถวลำดับขนาดใหญ่เกินไปโดยไม่จำเป็น ตัวอย่างเช่น:

- `int arr[100000][100000]` ใช้ $10^{10} \times 4 = 40$ GB — เกิน Memory Limit อย่างมาก!
- `long long dp[5000][5000]` ใช้ $25 \times 10^6 \times 8 = 200$ MB — อาจเกิน Memory Limit 256 MB เมื่อรวมหน่วยความจำอื่น
- ควรตรวจสอบขนาดแถวลำดับทุกครั้งก่อนส่งโจทย์

2.11 โจทย์แนะนำ

บทนี้เน้นทักษะการวิเคราะห์ความซับซ้อนของเวลาและพื้นที่ โจทย์แนะนำในบทนี้จึงไม่ใช่โจทย์ที่ต้องการขั้นตอนวิธีซับซ้อน แต่เป็นโจทย์ที่ฝึกให้ผู้เรียนรู้จัก **อ่านข้อจำกัด** และ **เลือกความซับซ้อนที่เหมาะสม** ก่อนเริ่มเขียนโค้ด รวมถึงโจทย์ที่ต้องวิเคราะห์ลูปซ้อนและสมการเรียกซ้ำ โจทย์เรียงจากง่ายไปยากตามลำดับทักษะที่ใช้

ตารางที่ 2.7 โจทย์แนะนำสำหรับบทที่ 3

โจทย์	แหล่ง	ระดับ	เทคนิคที่ใช้	หมายเหตุ
Twins	Codeforces 160A	ง่าย	การเรียงลำดับ + การนับ	ฝึก วิ เคราะห์ ว่า ลูปเดียว $\mathcal{O}(n)$ พอเพียง
Dragons	Codeforces 230A	ง่าย	Sort + Greedy	ฝึก สังเกต ว่า ข้อจำกัด เล็ก ทำให้ ใช้ $\mathcal{O}(n \log n)$ ได้
Insomnia cure	Codeforces 148A	ง่าย	ลูปซ้อนง่ายๆ $\mathcal{O}(nd)$	ฝึกวิเคราะห์ข้อจำกัด $\mathcal{O}(n \cdot d)$ เมื่อ $d \leq 5$
Pashmak and Graph	Codeforces 459E	ปานกลาง	DP + Sort	ฝึก วิเคราะห์ ว่า $\mathcal{O}(n^2)$ ผ่าน เมื่อ $n \leq 10^5$ ผิด
Physical Education	Codeforces 53E	ปานกลาง	Bitmask DP	ฝึก อ่าน ข้อ จำกัด $n \leq 18$ ทำให้ ได้ $\mathcal{O}(2^n \cdot n^2)$
Lucky Sum	Codeforces 121A	ปานกลาง	การ ประมาณ ค่า จาก ข้อจำกัด	ฝึกเลือกความซับซ้อน จาก $n \leq 10^9$
Little Elephant and Cards	Codeforces 204B	ยาก	Sort + การนับ	ฝึก วิเคราะห์ $\mathcal{O}(n \log n)$ กับ $n \leq 10^5$

คำแนะนำในการฝึก

- **อ่านข้อจำกัดก่อนทุกครั้ง** — ก่อนคิดขั้นตอนวิธี ให้ดูค่า n สูงสุดก่อนเสมอ แล้วเปิดตารางที่ 2.3 เพื่อดูความซับซ้อนที่ยอมรับได้
- **คำนวณจำนวนการดำเนินการ** — คูณ n กับสัมประสิทธิ์ของความซับซ้อน แล้วเทียบกับเกณฑ์ 10^8 ต่อวินาที ถ้าเกินแสดงว่าต้องหาขั้นตอนวิธีที่เร็วกว่า
- **ฝึกวิเคราะห์ลูปซ้อน** — เมื่อเจอลูปซ้อน ให้ฝึกบอกความซับซ้อนภายใน 10 วินาที โดยสังเกตว่าลูปในพึ่งพาตัวแปรของลูปนอกหรือไม่
- **ระวัง overflow ในการวิเคราะห์** — บางครั้งความซับซ้อน $\mathcal{O}(n^2)$ ดูผ่านได้ แต่ค่าที่เก็บอาจเกิน `long long` ทำให้คำตอบผิดโดยไม่รู้ตัว

2.12 สรุปบทที่ 2

บทนี้เราได้เรียนรู้เครื่องมือและเทคนิคสำคัญในการวิเคราะห์ความซับซ้อนของขั้นตอนวิธี ซึ่งเป็นพื้นฐานที่จำเป็นสำหรับการโปรแกรมเชิงแข่งขัน ในหัวข้อต่อไปนี้จะสรุปประเด็นสำคัญทั้งหมด

2.12.1 ประเด็นสำคัญ

1. **ความซับซ้อนของเวลา** วัดอัตราการเติบโตของเวลาทำงานเมื่อขนาดข้อมูลเพิ่มขึ้น ไม่ใช่เวลาจริงเป็นวินาที

2. **สัญกรณ์ Big-O** ให้ขอบเขตบนของอัตราการเติบโต และเป็นสัญกรณ์ที่ใช้บ่อยที่สุดในการแข่งขัน

3. **สัญกรณ์ Ω และ Θ** ให้ขอบเขตล่างและขอบเขตแน่นหนาตามลำดับ

4. **การวิเคราะห์ลูป** — นับจำนวนรอบรวมของลูปชั้นในสุด อย่าสรุปเร็วเกินไปจากจำนวนชั้นของลูป

5. **สมการเรียกซ้ำ** ใช้วิเคราะห์ขั้นตอนวิธีแบบเรียกซ้ำ และ Master Theorem ช่วยแก้สมการเรียกซ้ำได้อย่างรวดเร็ว

6. **การประมาณจากข้อจำกัด** — กฎ 10^8 การดำเนินการต่อวินาทีช่วยเลือกขั้นตอนวิธีที่เหมาะสม

7. **Amortized Analysis** — บางการดำเนินการที่ดูแพงอาจถูกเฉลี่ยเมื่อพิจารณาลำดับทั้งหมด

8. **ความซับซ้อนของหน่วยความจำ** — ต้องคำนึงถึงทั้งเวลาและหน่วยความจำ และเข้าใจ Time–Space Trade-off

2.12.2 ตารางอ้างอิงด่วน: คลาสความซับซ้อน

ตารางที่ 2.8 สรุปคลาสความซับซ้อนทั้งหมด

ความซับซ้อน	ชื่อ	ตัวอย่างที่สำคัญ
$\mathcal{O}(1)$	คงที่	เข้าถึงแถวลำดับ, Hash lookup
$\mathcal{O}(\log n)$	ลอการิทึม	Binary Search, Binary Exponentiation
$\mathcal{O}(\sqrt{n})$	รากที่สอง	แยกตัวประกอบ, Two pointers
$\mathcal{O}(n)$	เชิงเส้น	Linear Search, Prefix sum
$\mathcal{O}(n \log n)$	Linearithmic	Sort, Segment tree
$\mathcal{O}(n\sqrt{n})$	Square root	Mo's algorithm, Block decomposition
$\mathcal{O}(n^2)$	กำลังสอง	Bubble Sort, DP 2 มิติแบบง่าย
$\mathcal{O}(n^3)$	กำลังสาม	Floyd-Warshall, Matrix chain
$\mathcal{O}(2^n)$	เอกซ์โพเนนเชียล	Bitmask DP, Subset enumeration
$\mathcal{O}(n!)$	แฟกทอเรียล	Permutation enumeration

2.12.3 ข้อผิดพลาดที่พบบ่อย

ข้อผิดพลาดยอดนิยมในการวิเคราะห์ความซับซ้อน

1. **นับจำนวนชั้นลูปแล้วสรุปเร็วเกินไป** — ลูปซ้อน 2 ชั้นไม่ได้เป็น $\mathcal{O}(n^2)$ เสมอ ต้องดูจำนวนรอบรวมจริง
2. **ลืมพิจารณาค่าคงที่ที่ซ่อนอยู่** — $\mathcal{O}(n \log n)$ ของขั้นตอนวิธีที่มีค่าคงที่สูงอาจช้ากว่า $\mathcal{O}(n^2)$ สำหรับ n เล็กๆ
3. **ไม่ดูข้อจำกัดของโจทย์** — เขียนขั้นตอนวิธีโดยไม่เช็คว่าผ่าน Time Limit หรือไม่
4. **สับสน Worst Case กับ Amortized** — คิดว่า `push_back` เป็น $\mathcal{O}(1)$ เสมอ ทั้งที่ worst case เป็น $\mathcal{O}(n)$
5. **ประกาศแถวลำดับใหญ่เกินไป** — ไม่คำนวณหน่วยความจำจนเกิน Memory Limit
6. **ใช้ `int` แทน `long long`** — เมื่อค่าเกิน 2×10^9 จะเกิด Overflow ทำให้คำตอบผิด

2.12.4 คำแนะนำในการฝึกฝน

เคล็ดลับการฝึกฝน

1. **ฝึกวิเคราะห์โค้ดทุกครั้ง** — เมื่อเห็นโค้ด ให้ฝึกบอกความซับซ้อนภายใน 10 วินาที
2. **อ่านข้อจำกัดก่อนคิดขั้นตอนวิธี** — ฝึกนิสัยดู $n \leq ?$ ก่อนเสมอ แล้วเปิดตารางที่ 2.3
3. **ฝึกกับโจทย์จริง** — ลองแก้โจทย์หลายๆ ข้อ และสังเกตว่าข้อจำกัดบอกอะไรเราเกี่ยวกับขั้นตอนวิธีที่ต้องใช้
4. **เปรียบเทียบขั้นตอนวิธี** — เมื่อแก้โจทย์ได้แล้ว ลองหาขั้นตอนวิธีที่เร็วกว่า และเปรียบเทียบความซับซ้อน
5. **เข้าใจ Time–Space Trade-off** — ฝึกคิดว่าจะใช้หน่วยความจำเพิ่มเพื่อลดเวลาได้อย่างไร

Chapter 3

การเรียงลำดับและการค้นหาแบบทวิภาค

การเรียงลำดับ (Sorting) เป็นหนึ่งในปัญหาพื้นฐานที่สำคัญที่สุดในวิทยาการคอมพิวเตอร์ และมีบทบาทสำคัญอย่างยิ่งในการโปรแกรมเชิงแข่งขัน โจทย์จำนวนมากต้องใช้ข้อมูลที่ถูกเรียงลำดับเสียก่อนจึงจะแก้ได้ ไม่ว่าจะเป็นการค้นหาแบบทวิภาค (Binary Search) การนับจำนวนค่าที่ซ้ำกัน หรือการหาค่ามัธยฐาน ในบทนี้ เราจะเรียนรู้วิธีใช้ฟังก์ชันเรียงลำดับจากไลบรารีมาตรฐานของ C++ ทำความเข้าใจขั้นตอนวิธีการเรียงลำดับพื้นฐานและขั้นสูง รวมถึงเจาะลึกการค้นหาแบบทวิภาคและการประยุกต์ใช้ขั้นสูง ซึ่งเป็นเทคนิคที่พบได้บ่อยมากในการแข่งขัน[2, 6]

3.1 การเรียงลำดับด้วยไลบรารีมาตรฐาน C++

ในการโปรแกรมเชิงแข่งขัน การใช้ฟังก์ชันเรียงลำดับจากไลบรารีมาตรฐานของ C++ เป็นสิ่งที่ทำได้เร็วที่สุดและปลอดภัยที่สุด ฟังก์ชัน `sort()` ใน `<algorithm>` ถูกออกแบบมาให้มีประสิทธิภาพสูง ทำงานในเวลา $\mathcal{O}(n \log n)$ และผ่านการทดสอบมาอย่างดี นักแข่งขันทุกคนควรใช้ฟังก์ชันนี้เป็นหลักและไม่ควรเสียเวลาเขียนขั้นตอนวิธีการเรียงลำดับด้วยตัวเอง เว้นแต่โจทย์จะมีข้อกำหนดพิเศษบางประการ

3.1.1 ทำไมไม่ควรเขียน sort เอง

หลายคนอาจสงสัยว่าทำไมไม่ควรเขียนขั้นตอนวิธีการเรียงลำดับด้วยตัวเอง โดยเฉพาะผู้ที่เพิ่งเริ่มเรียนรู้การโปรแกรมเชิงแข่งขัน อาจรู้สึกว่าการเขียนเองจะช่วยให้เข้าใจขั้นตอนวิธีได้ดีกว่า ซึ่งเป็นความจริงในแง่ของการเรียนรู้ แต่ในการแข่งขันจริง มีเหตุผลสำคัญ 2 ประการที่ทำให้เราควรใช้ `sort()` จากไลบรารี:

1. **ประหยัดเวลา** — ในการแข่งขัน เวลาเป็นสิ่งที่มีค่าที่สุด การใช้ `sort()` จากไลบรารีช่วยประหยัดเวลาเขียนโค้ดได้หลายนาที ซึ่งสามารถนำไปใช้แก้โจทย์ข้ออื่นแทนได้ ในขณะที่การเขียนขั้นตอนวิธีเองต้องใช้เวลาทั้งเขียนและแก้จุดบกพร่อง ซึ่งมักใช้เวลานานกว่าที่คิด

2. **ถูกต้องและมีประสิทธิภาพ** — `sort()` ของ C++ ถูกพัฒนาและทดสอบโดยผู้เชี่ยวชาญมานานหลายสิบปี มีประสิทธิภาพ $\mathcal{O}(n \log n)$ ในทุกกรณี และไม่มีจุดบกพร่อง ในขณะที่โค้ดที่เขียนเองมีโอกาสผิดพลาดสูง โดยเฉพาะในเงื่อนไขขอบเขต (Boundary Conditions) เช่น แถวลำดับว่าง แถวลำดับที่มีสมาชิกเพียง 1 ตัว หรือแถวลำดับที่มีสมาชิกทุกตัวเท่ากัน

ตัวอย่าง: เรียงลำดับตัวเลข

ให้แถวลำดับ $a = [948, 178, 428, 191, 988, 492, 559, 939, 123, 842, 751]$ หลังจากเรียงลำดับ

จากน้อยไปมาก จะได้:

$$a = [123, 178, 191, 428, 492, 559, 751, 842, 939, 948, 988]$$

การใช้ `sort()` ทำได้ด้วยโค้ดเพียงบรรทัดเดียว ในขณะที่การเขียนขั้นตอนวิธีเองต้องใช้โค้ดหลายสิบบรรทัดและมีโอกาสผิดมากกว่า

3.1.2 ฟังก์ชัน sort() กับ vector

การใช้ `sort()` กับ `vector` เป็นวิธีที่พบบ่อยที่สุดในการแข่งขัน เนื่องจาก `vector` เป็นโครงสร้างข้อมูลที่ใช้งานสะดวกและยืดหยุ่น สามารถเพิ่มและลบสมาชิกได้ตามต้องการ และสอดคล้องกับรูปแบบ iterator ของ STL ทุกประการ การใช้ `sort()` กับ `vector` จึงเป็นทักษะพื้นฐานที่นักแข่งขันทุกคนต้องมี

เรียงจากน้อยไปมาก (Ascending):

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> v = {948, 178, 428, 191, 988, 492, 559, 939, 123, 842};
    sort(v.begin(), v.end());
    // v = {123, 178, 191, 428, 492, 559, 842, 939, 948, 988}
    for (int x : v) cout << x << " ";
    return 0;
}
```

ตัวอย่างโค้ดที่ 3.1: เรียง vector จากน้อยไปมาก

เรียงจากมากไปน้อย (Descending):

```cpp
vector<int> v = {948, 178, 428, 191, 988, 492, 559, 939, 123, 842};
sort(v.rbegin(), v.rend());
// v = {988, 948, 939, 842, 559, 492, 428, 191, 178, 123}
```

ตัวอย่างโค้ดที่ 3.2: เรียง vector จากมากไปน้อย

อีกวิธีหนึ่งในการเรียงจากมากไปน้อยคือใช้ `greater<int>()` เป็นตัวเปรียบเทียบ:

```cpp
sort(v.begin(), v.end(), greater<int>());
```

ตัวอย่างโค้ดที่ 3.3: เรียงจากมากไปน้อยด้วย greater

ทั้ง สอง วิธี ให้ ผลลัพธ์ เหมือน กัน แต่ การ ใช้ `rbegin()`/`rend()` มัก สั้น กว่า และ เขียน สะดวก กว่า อย่างไร ก็ตาม การ ใช้ `greater<int>()` มี ข้อดี คือ สามารถ ปรับ เปลี่ยน เป็น `greater<string>()` หรือ `greater<pair<int,int>>()` ได้ตามประเภทข้อมูลที่ต้องการเรียง

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];
    sort(v.begin(), v.end());
    for (int i = 0; i < n; i++) {
        cout << v[i];
        if (i < n - 1) cout << " ";
    }
    cout << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 3.4: โปรแกรมตัวอย่าง: เรียงลำดับเต็มรูปแบบ

3.1.3 sort() กับ array ธรรมดา

นอกจาก `vector` แล้ว `sort()` ยังใช้กับแถวลำดับแบบ C (C-style Array) ได้เช่นกัน โดยใช้พอยน์เตอร์แทน iterator ในการระบุช่วงที่ต้องการเรียง วิธีนี้มีประโยชน์เมื่อโจทย์กำหนดขนาดแถวลำดับล่วงหน้า หรือเมื่อต้องการประสิทธิภาพที่ดีกว่า `vector` เล็กน้อย (แม้ว่าความแตกต่างมักจะไม่มากนักในทางปฏิบัติ)

```cpp
int a[] = {948, 178, 428, 191, 988, 492, 559, 939, 123, 842};
int n = 10;
sort(a, a + n);
// a = {123, 178, 191, 428, 492, 559, 842, 939, 948, 988}
```

ตัวอย่างโค้ดที่ 3.5: เรียง array ธรรมดา

Half-open Interval

`sort(a, a+n)` ใช้ช่วง **ครึ่งเปิด** (Half-open Interval) $[a, a+n)$ กล่าวคือ เรียงสมาชิกตั้งแต่ `a[0]` ถึง `a[n-1]` ไม่รวม `a[n]` รูปแบบนี้เป็นมาตรฐานของ STL ทุกฟังก์ชัน เช่น `sort(v.begin(), v.end())` ก็เรียงตั้งแต่สมาชิกตัวแรกถึงตัวสุดท้ายเช่นกัน แนวคิด Half-open Interval ลดข้อผิดพลาดจาก off-by-one error ได้ดี เพราะจำนวนสมาชิกในช่วง $[\text{begin}, \text{end})$ คือ end $-$ begin เสมอ

หากต้องการเรียงเฉพาะช่วงใดช่วงหนึ่งของแถวลำดับ สามารถระบุได้โดยตรง:

```cpp
// Sort only a[2] to a[6] (excluding a[7])
sort(a + 2, a + 7);
```

ตัวอย่างโค้ดที่ 3.6: เรียงเฉพาะช่วงของ array

ความสามารถนี้มีประโยชน์ในบางโจทย์ที่ต้องเรียงเฉพาะส่วนใดส่วนหนึ่งของข้อมูล เช่น เรียงเฉพาะครึ่งหลังของแถวลำดับ หรือเรียงเฉพาะสมาชิกในช่วงที่กำหนดโดยตำแหน่งเริ่มต้นและสิ้นสุด

3.1.4 sort() กับ string

สายอักขระใน C++ สามารถเรียงลำดับได้เช่นเดียวกับคอนเทนเนอร์อื่นๆ การเรียงลำดับสายอักขระจะเรียงตามลำดับอักขระ ASCII กล่าวคือ ตัวพิมพ์ใหญ่จะมาก่อนตัวพิมพ์เล็ก (เพราะรหัส ASCII ของ `'A'` คือ 65 ส่วน `'a'` คือ 97) และตัวเลขจะมาก่อนตัวอักษร การเข้าใจลำดับ ASCII มีความสำคัญเพราะมันส่งผลต่อผลลัพธ์ของการเรียงลำดับโดยตรง

```
string s = "monkey";
sort(s.begin(), s.end());
// s = "ekmnoy"
cout << s << "\n";
```

ตัวอย่างโค้ดที่ 3.7: เรียง string ตามอักขระ

การเรียงสายอักขระตามอักขระเป็นการจัดเรียงอักขระแต่ละตัวภายในสายอักขระ ไม่ใช่การเรียงสายอักขระหลายๆ อันตามลำดับพจนานุกรม หากต้องการเรียงสายอักขระหลายอัน ต้องใช้ `vector<string>` แทน:

```
vector<string> names = {"banana", "apple", "cherry", "date"};
sort(names.begin(), names.end());
// names = {"apple", "banana", "cherry", "date"}
```

ตัวอย่างโค้ดที่ 3.8: เรียง vector ของ string ตามพจนานุกรม

ระวังตัวพิมพ์ใหญ่-เล็ก

เมื่อเรียงสายอักขระโดยใช้ `sort()` ตัวพิมพ์ใหญ่จะมาก่อนตัวพิมพ์เล็กเสมอ เช่น `"Zebra"` จะมาก่อน `"apple"` เพราะ `'Z'` มีรหัส ASCII น้อยกว่า `'a'` หากต้องการเรียงแบบไม่สนใจตัวพิมพ์ ต้องเขียนตัวเปรียบเทียบเองโดยใช้ `tolower()` หรือ `toupper()` แปลงก่อนเปรียบเทียบ

ตัวอย่าง: เรียงแล้วหาค่ามัธยฐาน

การหาค่ามัธยฐาน (Median) เป็นตัวอย่างคลาสสิกของการใช้ sorting ในการแข่งขัน หลังจากเรียงลำดับแล้ว:

- ถ้า n เป็นจำนวนคี่ มัธยฐานคือ $v[n/2]$
- ถ้า n เป็นจำนวนคู่ มัธยฐานคือ $\frac{v[n/2-1]+v[n/2]}{2}$

```
#include <bits/stdc++.h>
using namespace std;
int main() {
```

```cpp
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];
    sort(v.begin(), v.end());
    if (n % 2 == 1) {
        cout << v[n / 2] << "\n";
    } else {
        cout << (v[n / 2 - 1] + v[n / 2]) / 2.0 << "\n";
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 3.9: หาค่ามัธยฐาน

สังเกตว่าเราต้องเรียงก่อนจึงจะหามัธยฐานได้ ซึ่งใช้เวลา $\mathcal{O}(n \log n)$ ในขั้นเรียง และ $\mathcal{O}(1)$ ในขั้นหาค่ามัธยฐาน

3.2 ตัวเปรียบเทียบ (Comparison Operators)

การเรียงลำดับโดยพื้นฐานแล้วต้องมีวิธีเปรียบเทียบว่าสมาชิกสองตัวใดควรมาก่อน ฟังก์ชัน `sort()` ใช้ **ตัวเปรียบเทียบ** (Comparator) ในการตัดสินใจ ถ้าไม่ระบุตัวเปรียบเทียบ `sort()` จะใช้ `operator<` ของประเภทข้อมูลนั้นโดยอัตโนมัติ ความเข้าใจในตัวเปรียบเทียบจึงมีความสำคัญอย่างยิ่งเมื่อต้องเรียงข้อมูลที่ซับซ้อนกว่าจำนวนเต็มธรรมดา เช่น struct หรือข้อมูลที่ต้องเรียงตามเงื่อนไขพิเศษ

3.2.1 ตัวเปรียบเทียบมาตรฐานของ C++

C++ มีตัวเปรียบเทียบมาตรฐานสำหรับประเภทข้อมูลพื้นฐานอยู่แล้ว ทำให้เราสามารถเรียงข้อมูลเหล่านี้ได้ทันทีโดยไม่ต้องเขียนอะไรเพิ่มเติม:

- **จำนวนเต็มและจุดทศนิยม** — เปรียบเทียบตามค่า (เช่น $3 < 5$, $2.7 < 3.14$)
- **สายอักขระ** — เปรียบเทียบตามลำดับพจนานุกรม (Lexicographic Order) โดยเปรียบเทียบทีละอักขระจากซ้ายไปขวา เช่น `"apple"` $<$ `"banana"` และ `"abc"` $<$ `"abd"`

ลำดับพจนานุกรม (Lexicographic Order)

ลำดับพจนานุกรมเปรียบเทียบอักขระทีละตัวจากซ้ายไปขวา ตัวแรกที่แตกต่างกันจะตัดสินลำดับ หากสายอักขระหนึ่งเป็นคำนำหน้าของอีกสายอักขระหนึ่ง สายอักขระที่สั้นกว่าจะมาก่อน เช่น `"ab"` $<$

"abc" ลำดับนี้เหมือนกับการเปิดพจนานุกรมหาคำ — คำที่สะกดด้วยอักษรข้างหน้าจะอยู่ก่อนเสมอ

3.2.2 การเรียง pair

pair<T1, T2> เป็นโครงสร้างข้อมูลที่เก็บค่า 2 ค่า C++ มี operator< มาตรฐานสำหรับ pair อยู่แล้ว โดยเปรียบเทียบ first ก่อน ถ้า first เท่ากันจึงเปรียบเทียบ second:

```
vector<pair<int,int>> v = {{3,1}, {1,4}, {1,2}, {2,5}};
sort(v.begin(), v.end());
// v = {{1,2}, {1,4}, {2,5}, {3,1}}
```

ตัวอย่างโค้ดที่ 3.10: เรียง vector ของ pair

จากตัวอย่าง เมื่อ first เท่ากัน (เช่น {1,4} และ {1,2}) จะเปรียบเทียบ second ทำให้ {1,2} มาก่อน {1,4} พฤติกรรมนี้มีประโยชน์มากในการแข่งขัน เช่น การเรียงพิกัด (x, y) การเรียงคู่ (คะแนน, หมายเลข) หรือการเรียงเหตุการณ์ตามเวลา (เวลา, ประเภท) โดยที่เราไม่ต้องเขียนตัวเปรียบเทียบเองเลย

3.2.3 การเรียง tuple

tuple<T1, T2, T3> เป็นโครงสร้างข้อมูลที่เก็บค่าหลายค่า โดยเปรียบเทียบจากสมาชิกตัวแรกไปถึงตัวสุดท้ายตามลำดับ กล่าวคือ เปรียบเทียบสมาชิกตัวแรกก่อน ถ้าเท่ากันเปรียบเทียบตัวที่สอง ถ้ายังเท่ากันจึงเปรียบเทียบตัวที่สาม ไปเรื่อยๆ จนกว่าจะพบตำแหน่งที่แตกต่าง:

```
vector<tuple<int,int,int>> v = {{1,2,3}, {1,2,1}, {2,1,1}, {1,1,5}};
sort(v.begin(), v.end());
// v = {{1,1,5}, {1,2,1}, {1,2,3}, {2,1,1}}
```

ตัวอย่างโค้ดที่ 3.11: เรียง vector ของ tuple

จะเห็นว่าการเปรียบเทียบทำไปทีละสมาชิก: {1,1,5} มาก่อน {1,2,1} เพราะสมาชิกตัวแรกเท่ากัน แต่สมาชิกตัวที่สอง $1 < 2$ ในทำนองเดียวกัน {1,2,1} มาก่อน {1,2,3} เพราะสมาชิกสองตัวแรกเท่ากันแต่สมาชิกตัวที่สาม $1 < 3$ พฤติกรรมนี้คล้ายกับ pair แต่ขยายไปยังจำนวนสมาชิกที่มากกว่า 2

3.2.4 นิยาม operator< ใน struct

เมื่อเราสร้าง struct ขึ้นมาเอง C++ จะ **ไม่มี** operator< มาตรฐานให้ ดังนั้นหากต้องการเรียงลำดับด้วย sort() จะต้องนิยาม operator< ภายใน struct เอง การนิยาม operator< ช่วยให้เรากำหนดได้ว่าจะเปรียบเทียบสมาชิกของ struct ตามเงื่อนไขใด ซึ่งมีความยืดหยุ่นสูงมาก

```
struct Point {
    int x, y;
    bool operator<(const Point& other) const {
        if (x != other.x) return x < other.x;
        return y < other.y;
```

```
    }
};
vector<Point> pts = {{3,1}, {1,4}, {1,2}, {2,5}};
sort(pts.begin(), pts.end());
// pts = {{1,2}, {1,4}, {2,5}, {3,1}}
```

ตัวอย่างโค้ดที่ 3.12: นิยาม operator< ใน struct

จากตัวอย่าง เรานิยามให้ `Point` เรียงตาม x ก่อน ถ้า x เท่ากันจึงเรียงตาม y ซึ่งเหมือนกับพฤติกรรมของ `pair` แต่เราสามารถกำหนดเงื่อนไขที่ซับซ้อนกว่านั้นได้ เช่น เรียงตามระยะทางจากจุดกำเนิด หรือเรียงตามผลรวมของ x และ y

อย่าลืม const

การนิยาม `operator<` ต้องใส่ `const` ทั้งสองที่:

- `const Point& other` — บอกว่าไม่แก้ไข `other`
- `const` ท้ายเมธอด — บอกว่าไม่แก้ไขอ็อบเจกต์ตัวเอง (`*this`)

หากลืม `const` แม้แต่ตัวเดียว โค้ดจะคอมไพล์ไม่ผ่านเมื่อใช้กับ `sort()` เพราะ `sort()` ต้องการตัวเปรียบเทียบที่ไม่แก้ไขสมาชิกของแถวลำดับ ข้อผิดพลาดนี้พบได้บ่อยมากในหมู่ผู้เริ่มต้น

3.2.5 ฟังก์ชันเปรียบเทียบภายนอก (callback)

นอกจากการนิยาม `operator<` ใน `struct` แล้ว เรายังสามารถเขียนฟังก์ชันเปรียบเทียบแยกต่างหากแล้วส่งเป็นพารามิเตอร์ที่สามของ `sort()` ได้ วิธีนี้มีข้อดีคือสามารถกำหนดเงื่อนไขการเรียงได้หลายแบบโดยไม่ต้องแก้ `struct` ต้นทาง ทำให้โค้ดยืดหยุ่นมากขึ้น นอกจากนี้ ยังสามารถใช้ Lambda Expression (C++11) เขียนตัวเปรียบเทียบแบบอินไลน์ได้อีกด้วย

```
struct Student {
    string name;
    int score;
};
bool comp(const Student& a, const Student& b) {
    // Sort score descending; if equal, sort by name
    if (a.score != b.score) return a.score > b.score;
    return a.name < b.name;
}
// Usage:
sort(students.begin(), students.end(), comp);
```

ตัวอย่างโค้ดที่ 3.13: ฟังก์ชันเปรียบเทียบภายนอก

ฟังก์ชันเปรียบเทียบต้องรับพารามิเตอร์ 2 ตัวและคืนค่า `bool` โดยคืน `true` เมื่อ `a` ควรมาก่อน `b` ในลำดับที่ต้องการ ในตัวอย่างข้างต้น เราต้องการให้คะแนนมากอยู่ก่อน จึงใช้ `a.score > b.score` (มากไปน้อย) และเมื่อคะแนนเท่ากัน ให้เรียงชื่อตามลำดับพจนานุกรม จึงใช้ `a.name < b.name` (น้อยไปมาก)

กฎของตัวเปรียบเทียบ

ฟังก์ชันเปรียบเทียบต้องเป็น **Strict Weak Ordering** กล่าวคือ:

1. **ไม่สะท้อน (Irreflexive):** `comp(a, a)` ต้องคืน `false` เสมอ
2. **ไม่สมมาตร (Asymmetric):** ถ้า `comp(a, b)` เป็น `true` แล้ว `comp(b, a)` ต้องเป็น `false`
3. **ถ่ายทอด (Transitive):** ถ้า `comp(a, b)` และ `comp(b, c)` แล้ว `comp(a, c)`

หากละเมิดกฎเหล่านี้ (เช่น เขียน `<=` แทน `<`) โปรแกรมอาจทำงานผิดพลาดหรือค้างได้ ข้อผิดพลาดที่พบบ่อยที่สุดคือการใช้ `<=` แทน `<` ซึ่งทำให้ `comp(a,a)` คืน `true` และละเมิดกฎ Irreflexive

3.2.6 ตัวอย่าง: เรียง string ตามความยาว

ปัญหาที่พบบ่อยคือการเรียงสายอักขระโดยใช้เงื่อนไขที่ซับซ้อนกว่าลำดับพจนานุกรม เช่น เรียงตามความยาวก่อน ถ้าความยาวเท่ากันจึงเรียงตามลำดับพจนานุกรม ปัญหานี้ไม่สามารถใช้ `sort()` แบบปกติได้ เพราะการเรียงตามพจนานุกรมไม่ได้เรียงตามความยาว (เช่น `"z"` มาก่อน `"aa"` ในลำดับพจนานุกรม แต่ `"z"` สั้นกว่า)

```cpp
#include <bits/stdc++.h>
using namespace std;
bool comp(const string& a, const string& b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<string> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];
    sort(v.begin(), v.end(), comp);
    for (const string& s : v) cout << s << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 3.14: โปรแกรม: เรียง string ตามความยาว

จากโค้ดข้างต้น หากอินพุตเป็น `"banana", "a", "cherry", "apple", "hi"` ผลลัพธ์จะเป็น `"a", "hi", "apple", "banana", "cherry"` เพราะเรียงตามความยาวก่อน และสายอักขระที่มีความยาวเท่ากันจะเรียงตามลำดับพจนานุกรม สังเกตว่า `"hi"` (2 ตัว) มาก่อน `"apple"` (5 ตัว) เพราะสั้นกว่า และ `"apple"` (5 ตัว) มาก่อน `"banana"` (6 ตัว) เพราะสั้นกว่า ส่วน `"a"` (1 ตัว) สั้นที่สุดจึงมาก่อนสายอักขระอื่นทั้งหมด

ตัวอย่าง: เรียงคะแนนนักเรียนด้วย pair

ให้คะแนนนักเรียน 5 คน ต้องเรียงจากคะแนนมากไปน้อย ถ้าคะแนนเท่ากันให้คนที่มีหมายเลขน้อยกว่ามาก่อน

วิธีการ: เก็บข้อมูลในรูปแบบ `pair<int,int>` โดย `first` = คะแนน (คูณ -1 เพื่อกลับทิศทาง) และ `second` = หมายเลข

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<int,int>> v(n);   // (negative score, id)
    for (int i = 0; i < n; i++) {
        int score;
        cin >> score;
        v[i] = {-score, i + 1};   // Multiply by -1 to sort descending
    }
    sort(v.begin(), v.end());
    for (auto& p : v) {
        cout << p.second << " " << -p.first << "\n";
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 3.15: เรียงคะแนนด้วย pair

เทคนิคการคูณ -1 เป็นเทคนิคที่ใช้บ่อยในการแข่งขัน เพราะ `sort()` เรียงจากน้อยไปมากโดยปกติ แต่หากเราคูณคะแนนด้วย -1 แล้วเรียงจากน้อยไปมาก ผลลัพธ์จะเทียบเท่ากับการเรียงคะแนนจากมากไปน้อย

3.3 ขั้นตอนวิธีการเรียงลำดับพื้นฐาน

แม้ว่าในการแข่งขันเราจะใช้ `sort()` จากไลบรารีเป็นหลัก แต่การทำความเข้าใจขั้นตอนวิธีการเรียงลำดับพื้นฐานมีความสำคัญ เพราะช่วยให้เข้าใจหลักการทำงานของการเรียงลำดับ และในบางสถานการณ์ขั้นตอนวิธีเหล่านี้อาจมีประโยชน์ เช่น Insertion Sort ที่ทำงานเร็วมากเมื่อข้อมูลเกือบเรียงแล้ว นอกจากนี้ การเข้าใจขั้นตอนวิธีเหล่านี้ยังเป็นพื้นฐานสำหรับการเรียนรู้ขั้นตอนวิธีขั้นสูงในหัวข้อถัดไป และเป็นคำถามที่พบบ่อยในการสอบสัมภาษณ์ด้านวิทยาการคอมพิวเตอร์

3.3.1 Bubble Sort

Bubble Sort เป็นขั้นตอนวิธีการเรียงลำดับที่เรียบง่ายที่สุด หลักการทำงานคือเปรียบเทียบสมาชิกที่อยู่ติดกันสองตัว แล้วสลับตำแหน่งหากสมาชิกทางซ้ายมากกว่าสมาชิกทางขวา ทำซ้ำจนกว่าข้อมูลจะเรียงเสร็จ ชื่อ “Bubble” มาจากลักษณะที่สมาชิกที่มีค่ามากจะค่อย ๆ ลอยขึ้นไปด้านขวาเหมือนฟองอากาศ ในแต่ละรอบของลูปนอก สมาชิกที่มีค่ามากที่สุดในส่วนที่ยังไม่เรียงจะถูกดันไปอยู่ตำแหน่งสุดท้ายของส่วนนั้น

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;    // If no swap this round = already sorted
    }
}
```

ตัวอย่างโค้ดที่ 3.16: Bubble Sort

สังเกตว่าลูปในวนเพียง $n-1-i$ ครั้ง เพราะหลังจากรอบที่ i แล้ว i สมาชิกสุดท้ายจะอยู่ในตำแหน่งที่ถูกต้องแล้ว ไม่จำเป็นต้องตรวจสอบซ้ำ ตัวแปร `swapped` ช่วยตรวจจับกรณีที่ข้อมูลเรียงแล้ว ทำให้กรณีที่ดีที่สุดเป็น $\mathcal{O}(n)$ แทนที่จะเป็น $\mathcal{O}(n^2)$

ความซับซ้อนของ Bubble Sort คือ $\mathcal{O}(n^2)$ ในกรณีเฉลี่ยและกรณีเลวร้ายที่สุด เพราะในกรณีทั่วไปต้องวนลูปซ้อน 2 ชั้น จำนวนการเปรียบเทียบรวมคือประมาณ $\frac{n(n-1)}{2}$ ครั้ง

ทำไมต้องเรียน Bubble Sort

แม้ Bubble Sort ไม่เหมาะสำหรับใช้ในการแข่งขัน (ช้าเกินไปเมื่อ n มาก) แต่มันเป็นขั้นตอนวิธีที่ดีสำหรับการทำความเข้าใจแนวคิดพื้นฐานของการเรียงลำดับ นั่นคือการ **เปรียบเทียบ** และ **สลับตำแหน่ง** อย่างไรก็ตาม อย่าใช้ Bubble Sort ในการแข่งขันจริง!

3.3.2 Selection Sort

Selection Sort ทำงานโดยหาสมาชิกที่น้อยที่สุดในส่วนที่ยังไม่เรียง แล้วนำมาวางไว้ที่ตำแหน่งถัดจากส่วนที่เรียงแล้ว หลักการคือแบ่งแถวลำดับออกเป็นสองส่วน: ส่วนที่เรียงแล้ว (ด้านซ้าย) และส่วนที่ยังไม่เรียง (ด้านขวา) แต่ละรอบจะเลือก (Select) ค่าน้อยสุดจากส่วนที่ยังไม่เรียงมาสลับกับตำแหน่งแรกของส่วนนั้น เมื่อผ่านไป i รอบ สมาชิก i ตัวแรกจะอยู่ในตำแหน่งที่ถูกต้องแล้ว

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIdx]) {
                minIdx = j;
            }
        }
        swap(a[i], a[minIdx]);
    }
}
```

ตัวอย่างโค้ดที่ 3.17: Selection Sort

ความซับซ้อนของ Selection Sort คือ $\mathcal{O}(n^2)$ ในทุกกรณี เพราะต้องค้นหาค่าน้อยสุดในแต่ละรอบเสมอ ไม่ว่าข้อมูลจะเรียงอยู่แล้วหรือไม่ อย่างไรก็ตาม Selection Sort มีข้อดีคือจำนวนการสลับตำแหน่ง (Swap) น้อยมาก เพียง $n-1$ ครั้ง (หรือ $\mathcal{O}(n)$) ซึ่งมีประโยชน์เมื่อต้นทุนของการสลับสูง เช่น เมื่อสมาชิกของแถวลำดับเป็นอ็อบเจกต์ขนาดใหญ่ที่การย้ายข้อมูลมีค่าใช้จ่ายสูง

3.3.3 Insertion Sort

Insertion Sort ทำงานคล้ายกับวิธีเรียงไพ่ — หยิบสมาชิกทีละตัวจากส่วนที่ยังไม่เรียง แล้วแทรก (Insert) ในตำแหน่งที่ถูกต้องในส่วนที่เรียงแล้ว การแทรกทำโดยเลื่อนสมาชิกที่ใหญ่กว่าออกไปทางขวาทีละตัว จนกว่าจะเจอตำแหน่งที่เหมาะสม จากนั้นจึงวางค่าที่หยิบมาลงในตำแหน่งนั้น วิธีนี้ทำให้ส่วนที่เรียงแล้วเติบโตทีละหนึ่งสมาชิกในแต่ละรอบ

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
```

```
11 }
```

ตัวอย่างโค้ดที่ 3.18: Insertion Sort

ความซับซ้อนของ Insertion Sort แตกต่างกันไปตามลักษณะของข้อมูล:

- **กรณีเลวร้ายที่สุด:** $\mathcal{O}(n^2)$ — เมื่อข้อมูลเรียงย้อนกลับ (จากมากไปน้อย) ทุกสมาชิกที่หยิบมาต้องเลื่อนสมาชิกทั้งหมดที่อยู่ด้านซ้าย
- **กรณีที่ดีที่สุด:** $\mathcal{O}(n)$ — เมื่อข้อมูลเรียงแล้ว (ลูป `while` ไม่ต้องเลื่อนเลย เพราะ `a[j] > key` เป็นเท็จทันที)
- **กรณีเฉลี่ย:** $\mathcal{O}(n^2)$ — โดยเฉลี่ยต้องเลื่อนประมาณครึ่งหนึ่งของสมาชิกที่อยู่ด้านซ้าย

Insertion Sort กับข้อมูลที่เกือบเรียงแล้ว

Insertion Sort มีประโยชน์มากเมื่อข้อมูล “เกือบเรียงแล้ว” (Nearly Sorted) เช่น เมื่อเพิ่มสมาชิกใหม่เข้าไปในแถวลำดับที่เรียงแล้ว หรือเมื่อมีการสลับสมาชิกเพียงไม่กี่ตัว ในกรณีเหล่านี้ Insertion Sort ทำงานใกล้เคียง $\mathcal{O}(n)$ ซึ่งเร็วกว่าขั้นตอนวิธี $\mathcal{O}(n \log n)$ บางตัวเสียอีก เพราะค่าคงที่ (Constant Factor) ที่ต่ำกว่า นี่คือเหตุผลที่ Introsort (ขั้นตอนวิธีที่ `std::sort` ใช้) เปลี่ยนไปใช้ Insertion Sort เมื่อขนาดปัญหาย่อยเล็กมาก (ประมาณ 16 ตัว)

3.3.4 การเปรียบเทียบขั้นตอนวิธี O(n²)

ตารางต่อไปนี้เปรียบเทียบขั้นตอนวิธีการเรียงลำดับ $\mathcal{O}(n^2)$ ทั้งสามตัว:

ตารางที่ 3.1 เปรียบเทียบขั้นตอนวิธีการเรียงลำดับ $\mathcal{O}(n^2)$

ขั้นตอนวิธี	Best	Average	Worst	หมายเหตุ
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	เข้าใจง่าย แต่ช้าที่สุด
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	Swap น้อย ไม่เปลี่ยนตามข้อมูล
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	เร็วเมื่อเกือบเรียง

แต่ละขั้นตอนวิธีมีจุดเด่นที่แตกต่างกัน: Bubble Sort ง่ายที่สุดสำหรับการทำความเข้าใจ, Selection Sort มีจำนวนครั้งที่สลับตำแหน่งน้อยที่สุด และไม่ขึ้นกับลักษณะของข้อมูล, และ Insertion Sort เร็วที่สุดเมื่อข้อมูลเกือบเรียงแล้ว อย่างไรก็ตาม ทั้งสามตัวไม่เหมาะสำหรับข้อมูลขนาดใหญ่ เพราะเมื่อ $n = 10^5$ แล้ว $\mathcal{O}(n^2)$ จะช้าเกินไป (10^{10} การดำเนินการ ซึ่งใช้เวลาหลายสิบวินาที)

ตัวอย่าง: จำนวนการเปรียบเทียบใน Bubble Sort

สำหรับแถวลำดับขนาด $n = 5$ จำนวนการเปรียบเทียบของ Bubble Sort ในแต่ละรอบคือ:

- รอบที่ 1: เปรียบเทียบ 4 ครั้ง ($n - 1$)

- รอบที่ 2: เปรียบเทียบ 3 ครั้ง $(n-2)$
- รอบที่ 3: เปรียบเทียบ 2 ครั้ง $(n-3)$
- รอบที่ 4: เปรียบเทียบ 1 ครั้ง $(n-4)$

รวมทั้งหมด $= 4+3+2+1 = 10$ ครั้ง ซึ่งตรงกับ $\frac{n(n-1)}{2} = \frac{5\times4}{2} = 10$ สำหรับ $n=5$
สำหรับ $n = 10^5$ จะต้องเปรียบเทียบประมาณ $\frac{10^5\times99999}{2} \approx 5\times10^9$ ครั้ง ซึ่งช้าเกินไปสำหรับ Time Limit ปกติ (1–2 วินาที)

3.4 ขั้นตอนวิธีการเรียงลำดับขั้นสูง

ขั้นตอนวิธีการเรียงลำดับขั้นสูงใช้เทคนิคที่ซับซ้อนกว่าเพื่อให้ได้ความซับซ้อน $\mathcal{O}(n \log n)$ ซึ่งเร็วกว่า $\mathcal{O}(n^2)$ อย่างมากเมื่อข้อมูลมีขนาดใหญ่ ตัวอย่างเช่น เมื่อ $n = 10^6$ ขั้นตอนวิธี $\mathcal{O}(n^2)$ ต้องทำงานประมาณ 10^{12} ครั้ง ในขณะที่ $\mathcal{O}(n \log n)$ ทำเพียงประมาณ 2×10^7 ครั้ง ต่างกันหลายหมื่นเท่า ในหัวข้อนี้ เราจะทำความเข้าใจหลักการของขั้นตอนวิธีเหล่านี้ รวมถึงกรณีพิเศษอย่าง Counting Sort ที่สามารถเรียงได้เร็วกว่า $\mathcal{O}(n \log n)$ ในบางสถานการณ์

3.4.1 Merge Sort

Merge Sort เป็นขั้นตอนวิธีแบบแบ่งแยกและเอาชนะ (Divide and Conquer) ที่ทำงานโดย:

1. **แบ่ง** — แบ่งแถวลำดับเป็นสองส่วนเท่าๆ กัน
2. **เอาชนะ** — เรียงลำดับแต่ละส่วนแบบเรียกซ้ำ
3. **รวม** — ผสาน (Merge) สองส่วนที่เรียงแล้วเข้าด้วยกัน

จุดเด่นของ Merge Sort คือ **รับประกันความซับซ้อน $\mathcal{O}(n \log n)$ ในทุกกรณี** ไม่ว่าข้อมูลจะมีลักษณะใดก็ตาม นอกจากนี้ Merge Sort ยังเป็น **Stable Sort** กล่าวคือ สมาชิกที่มีค่าเท่ากันจะรักษาลำดับเดิมไว้ คุณสมบัตินี้มีประโยชน์เมื่อต้องเรียงตามหลายเงื่อนไข เช่น เรียงตามนามสกุลก่อนแล้วจึงเรียงตามชื่อโดยไม่ทำลายลำดับนามสกุล

```
void mergeSort(int a[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);        // Sort left half
    mergeSort(a, mid + 1, hi);    // Sort right half
    merge(a, lo, mid, hi);        // Merge two parts
}
```

ตัวอย่างโค้ดที่ 3.19: Merge Sort — โครงสร้างหลัก

ฟังก์ชัน `merge()` ผสานสองส่วนที่เรียงแล้วโดยใช้แถวลำดับชั่วคราว มีความซับซ้อน $\mathcal{O}(n)$ ต่อการเรียกครั้งหนึ่ง หลักการคือเปรียบเทียบสมาชิกตัวแรกของแต่ละส่วน แล้วเลือกตัวที่น้อยกว่ามาใส่ในผลลัพธ์ ทำซ้ำจนกว่าสมาชิกทั้งหมดจะถูกนำไปใส่แล้ว เนื่องจากมีการเรียกซ้ำ $\log_2 n$ ระดับ แต่ละระดับใช้เวลารวม $\mathcal{O}(n)$ ดังนั้นความซับซ้อนรวมจึงเป็น $\mathcal{O}(n \log n)$

สมการเรียกซ้ำของ Merge Sort คือ:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) = \mathcal{O}(n \log n)$$

ซึ่งสอดคล้องกับกรณีที่ 2 ของ Master Theorem ที่กล่าวถึงในบทที่ 2 ข้อเสียของ Merge Sort คือใช้หน่วยความจำเพิ่มเติม $\mathcal{O}(n)$ สำหรับแถวลำดับชั่วคราวในขั้นผสาน

```
void merge(int a[], int lo, int mid, int hi) {
    vector<int> tmp(hi - lo + 1);
    int i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else tmp[k++] = a[j++];
    }
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi)  tmp[k++] = a[j++];
    for (int i = lo; i <= hi; i++)
        a[i] = tmp[i - lo];
}
```

ตัวอย่างโค้ดที่ 3.20: Merge — ผสานสองส่วนที่เรียงแล้ว

ฟังก์ชัน `merge()` ข้างต้นผสานสองส่วนที่เรียงแล้ว คือ $a[lo..mid]$ และ $a[mid+1..hi]$ โดยใช้แถวลำดับชั่วคราว `tmp` ขนาดเท่ากับจำนวนสมาชิกที่ต้องผสาน การเปรียบเทียบ `a[i] <= a[j]` (ใช้ `<=` ไม่ใช่ `<`) ทำให้ Merge Sort เป็น Stable Sort เพราะเมื่อค่าเท่ากัน สมาชิกจากส่วนซ้ายจะถูกเลือกก่อน ซึ่งรักษาลำดับเดิมไว้

3.4.2 Quick Sort

Quick Sort เป็นขั้นตอนวิธีแบบแบ่งแยกและเอาชนะเช่นกัน แต่ใช้กลยุทธ์ที่แตกต่างออกไป:

1. **เลือก Pivot** — เลือกสมาชิกหนึ่งตัวเป็น Pivot
2. **แบ่ง (Partition)** — จัดเรียงให้สมาชิกที่น้อยกว่า Pivot อยู่ด้านซ้าย มากกว่าอยู่ด้านขวา
3. **เอาชนะ** — เรียงสองส่วนที่แบ่งแล้วแบบเรียกซ้ำ

ข้อแตกต่างสำคัญจาก Merge Sort คือ Quick Sort ทำงานหลักในขั้น **แบ่ง** (Partition) ไม่ใช่ขั้น **รวม** (Merge) หลังจากแบ่งแล้ว สองส่วนไม่ต้องผสานกันเพราะสมาชิกทุกตัวในส่วนซ้ายน้อยกว่าทุกตัวในส่วนขวาอยู่แล้ว

```
int partition(int a[], int lo, int hi) {
    int pivot = a[hi];   // Choose last element as Pivot
    int i = lo;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {
            swap(a[i], a[j]);
            i++;
        }
    }
    swap(a[i], a[hi]);
    return i;   // Position of Pivot after partitioning
}
void quickSort(int a[], int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}
```

ตัวอย่างโค้ดที่ 3.21: Quick Sort — Partition

ฟังก์ชัน `partition()` ย้ายสมาชิกที่น้อยกว่า Pivot ไปด้านซ้าย และสมาชิกที่มากกว่าไปด้านขวา แล้ววาง Pivot ในตำแหน่งที่ถูกต้อง (ตำแหน่ง p) หลังจากนั้น สมาชิกทุกตัวใน $a[lo..p-1]$ จะน้อยกว่า $a[p]$ และสมาชิกทุกตัวใน $a[p+1..hi]$ จะมากกว่าหรือเท่ากับ $a[p]$

ความซับซ้อนของ Quick Sort ขึ้นกับการเลือก Pivot:

- **กรณีเฉลี่ย:** $\mathcal{O}(n \log n)$ — เมื่อ Pivot แบ่งข้อมูลเป็นสองส่วนพอๆ กัน
- **กรณีเลวร้ายที่สุด:** $\mathcal{O}(n^2)$ — เมื่อ Pivot เลือกได้แย่ที่สุดทุกรอบ (เช่น ข้อมูลเรียงแล้วและเลือกสมาชิกตัวแรกเป็น Pivot เสมอ ทำให้แบ่งได้ส่วนเดียวทุกรอบ)

Quick Sort **ไม่เป็น Stable Sort** กล่าวคือ สมาชิกที่มีค่าเท่ากันอาจสลับลำดับได้ เนื่องจากขั้น Partition อาจย้ายสมาชิกข้าม Pivot อย่างไรก็ตาม Quick Sort มีค่าคงที่ที่ต่ำ (Small Constant Factor) ทำให้ในทางปฏิบัติมักทำงานเร็วกว่า Merge Sort ในกรณีเฉลี่ย

3.4.3 C++ Introsort

ฟังก์ชัน `std::sort()` ของ C++ ไม่ได้ใช้ Quick Sort เพียงอย่างเดียว แต่ใช้ขั้นตอนวิธีผสมที่เรียกว่า **Introsort** (Introspective Sort) ซึ่งเป็นการผสานข้อดีของ 3 ขั้นตอนวิธีเข้าด้วยกันอย่างชาญฉลาด:

1. **Quick Sort** — เริ่มต้นด้วย Quick Sort เพราะเร็วในกรณีทั่วไป (ค่าคงที่ต่ำ)
2. **Heap Sort** — หาก Quick Sort แบ่งไม่สมดุลเกินไป (ความลึกของการเรียกซ้ำเกิน $2 \log_2 n$) จะเปลี่ยนไปใช้ Heap Sort เพื่อรับประกัน $\mathcal{O}(n \log n)$ ในกรณีเลวร้าย

3. **Insertion Sort** — เมื่อขนาดปัญหาย่อยเล็กมาก (ประมาณ 16 ตัว) จะเปลี่ยนไปใช้ Insertion Sort เพราะเร็วกว่าสำหรับข้อมูลขนาดเล็ก (ค่าคงที่ต่ำกว่า)

การสลับไปมาระหว่างขั้นตอนวิธีทั้งสามนี้ทำให้ Introsort ได้ทั้งความเร็วในทางปฏิบัติและการรับประกันความซับซ้อน $\mathcal{O}(n \log n)$ ในทุกกรณี นี่คือเหตุผลที่ `std::sort()` มีประสิทธิภาพสูงมากในทุกสถานการณ์

อย่าเขียน sort เองในการแข่งขัน

`std::sort()` ใช้ Introsort ซึ่งได้รับการปรับแต่งอย่างดีแล้ว รับประกัน $\mathcal{O}(n \log n)$ ในทุกกรณี และทำงานเร็วในทางปฏิบัติ การเขียนขั้นตอนวิธีเรียงลำดับเองในการแข่งขันเป็นการเสียเวลาโดยใช่เหตุและมีโอกาสผิดสูง ให้ใช้ `sort()` เสมอ!

Introsort ไม่เป็น Stable Sort หากต้องการ Stable Sort ให้ใช้ `stable_sort()` แทน ซึ่งรับประกันว่าสมาชิกที่เท่ากันจะรักษาลำดับเดิมไว้:

```
stable_sort(v.begin(), v.end());
```

ตัวอย่างโค้ดที่ 3.22: ใช้ stable_sort

`stable_sort()` มีความซับซ้อน $\mathcal{O}(n \log^2 n)$ ในกรณีเลวร้ายที่สุด (เมื่อหน่วยความจำไม่พอ) และ $\mathcal{O}(n \log n)$ ในกรณีปกติ แม้จะช้ากว่า `sort()` เล็กน้อย แต่ก็ยังเร็วพอสำหรับการใช้งานในการแข่งขัน

3.4.4 Counting Sort (กรณีพิเศษ)

ขั้นตอนวิธีการเรียงลำดับทั้งหมดที่กล่าวมาข้างต้นเป็น **Comparison-based Sort** กล่าวคือ ทำงานโดยการเปรียบเทียบสมาชิกสองตัวเพื่อตัดสินลำดับ มีทฤษฎีบทยืนยันว่า Comparison-based Sort ไม่สามารถทำได้เร็วกว่า $\Omega(n \log n)$ ในกรณีเลวร้ายที่สุด แต่ Counting Sort ทำลายขอบเขตนี้ได้ เพราะไม่ได้ใช้การเปรียบเทียบระหว่างสมาชิกเลยแม้แต่ครั้งเดียว

Counting Sort ใช้ได้เมื่อค่าของสมาชิกอยู่ในช่วงที่จำกัด เช่น คะแนนสอบ 0–100 หรืออายุ 0–150 หลักการคือนับจำนวนของแต่ละค่าก่อน แล้วสร้างผลลัพธ์จากการนับนั้น โดยไม่ต้องเปรียบเทียบสมาชิกสองตัวเลย

```
void countingSort(int a[], int n) {
    int count[101] = {};   // Values from 0 to 100
    for (int i = 0; i < n; i++) {
        count[a[i]]++;
    }
    int idx = 0;
    for (int val = 0; val <= 100; val++) {
        while (count[val] > 0) {
            a[idx++] = val;
            count[val]--;
        }
    }
```

```
}
```

ตัวอย่างโค้ดที่ 3.23: Counting Sort — เรียงคะแนน 0–100

ความซับซ้อนของ Counting Sort คือ $\mathcal{O}(n+k)$ โดยที่ k คือช่วงของค่า (Range) ในตัวอย่างนี้ $k = 101$ เมื่อ k มีขนาดใกล้เคียง n หรือน้อยกว่า Counting Sort จะเร็วกว่า $\mathcal{O}(n \log n)$ อย่างมาก เพราะ $n + k < n \log n$ สำหรับ n ที่ไม่เล็กเกินไป

Counting Sort ทำลายขอบเขต $\Omega(n \log n)$

Counting Sort ไม่ใช่ Comparison-based Sort จึงไม่ถูกจำกัดด้วยขอบเขตล่าง $\Omega(n \log n)$ อย่างไรก็ตาม Counting Sort ใช้ได้เฉพาะเมื่อ k ไม่ใหญ่เกินไป หาก $k = 10^9$ แล้วจะใช้หน่วยความจำมากเกินไป ทำให้ไม่สามารถใช้ได้ในทางปฏิบัติ ดังนั้น Counting Sort จึงเหมาะสำหรับกรณีที่ค่าอยู่ในช่วงแคบเท่านั้น

ตัวอย่าง: นับจำนวนค่าที่ซ้ำด้วย Counting Sort

ให้คะแนนสอบของนักเรียน $n = 20$ คน คะแนนอยู่ในช่วง 0–100 ต้องการนับว่ามีนักเรียนกี่คนที่ได้คะแนนแต่ละช่วง

```
int count[101] = {};   // Initialize all slots to 0
for (int i = 0; i < n; i++) {
    count[score[i]]++;
}
// Now count[s] = number of people with score s
// Count people who passed (score >= 50):
int pass = 0;
for (int s = 50; s <= 100; s++) {
    pass += count[s];
}
cout << pass << "\n";
```

ตัวอย่างโค้ดที่ 3.24: นับคะแนนด้วย Counting Sort

วิธีนี้เร็วกว่าการเรียงด้วย `sort()` แล้วนับ เพราะใช้เวลาเพียง $\mathcal{O}(n+k)$ แทนที่จะเป็น $\mathcal{O}(n \log n)$

3.5 การค้นหาแบบทวิภาค (Binary Search)

การค้นหาแบบทวิภาค (Binary Search) เป็นขั้นตอนวิธีที่ทรงพลังที่สุดอย่างหนึ่งในการโปรแกรมเชิงแข่งขัน หลักการคือลดช่วงการค้นหาลงครึ่งหนึ่งในแต่ละขั้น ทำให้สามารถค้นหาในข้อมูลขนาด n ได้ภายในเวลาเพียง $\mathcal{O}(\log n)$ ตัวอย่างเช่น ในแถวลำดับที่มีสมาชิก 10^9 ตัว Binary Search ต้องตรวจสอบเพียงประมาณ 30 ตัวเท่านั้น อย่างไรก็ตาม ข้อจำกัดสำคัญคือข้อมูลต้อง **เรียงลำดับแล้ว** จึงจะใช้ Binary Search ได้ ถ้าข้อมูลไม่เรียง ต้องเรียงก่อน (ใช้เวลา $\mathcal{O}(n \log n)$) แล้วจึงค้นหา[2, 3]

3.5.1 Linear Search เป็นจุดเริ่มต้น

วิธีค้นหาพื้นฐานที่สุดคือ Linear Search — ตรวจสอบสมาชิกทีละตัวตั้งแต่ต้นจนกว่าจะเจอค่าที่ต้องการ:

```
int linearSearch(int a[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (a[i] == x) return i;
    }
    return -1;  // Not found
}
```

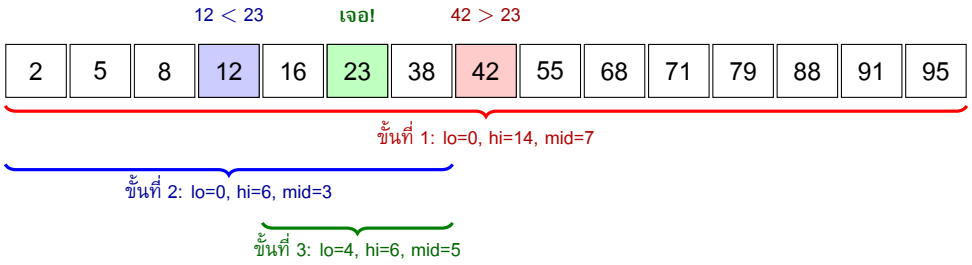
ตัวอย่างโค้ดที่ 3.25: Linear Search

Linear Search มีความซับซ้อน $\mathcal{O}(n)$ หากแถวลำดับไม่มีการเรียงลำดับ นี่คือวิธีที่ดีที่สุดที่เราทำได้ เพราะเราต้องตรวจสอบสมาชิกทุกตัวจึงจะแน่ใจได้ว่าค่าที่ค้นหาไม่อยู่ในแถวลำดับ แต่หากแถวลำดับถูกเรียงลำดับแล้ว เราสามารถทำได้ดีกว่านี้มาก โดยใช้ประโยชน์จากการที่สมาชิกเรียงลำดับอยู่ เพื่อตัดทอนช่วงการค้นหาลงอย่างรวดเร็ว

3.5.2 Binary Search วิธีที่ 1: ลดช่วงครึ่งหนึ่ง

ลองนึกถึงการค้นหาคำในพจนานุกรม — เราไม่ได้เปิดดูทีละหน้าตั้งแต่ต้น แต่เปิดไปที่กลางๆ แล้วดูว่าคำที่ต้องการอยู่ก่อนหรือหลังจุดนั้น แล้วลดช่วงการค้นหาลงครึ่งหนึ่ง Binary Search ใช้หลักการเดียวกัน:

รูปที่ 3.1 แสดงตัวอย่างการทำงานของ Binary Search บนแถวลำดับที่เรียงแล้ว สังเกตว่าในแต่ละขั้นช่วงการค้นหาจะลดลงครึ่งหนึ่ง



รูปที่ 3.1 ตัวอย่าง Binary Search: ค้นหา $x = 23$ ในแถวลำดับที่เรียงแล้ว — ใช้เพียง 3 ขั้นจาก 15 ตัว

```
int binarySearch(int a[], int n, int x) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == x) return mid;
        else if (a[mid] < x) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;  // Not found
```

```
10 }
```

ตัวอย่างโค้ดที่ 3.26: Binary Search วิธีลดช่วง

การทำงานของขั้นตอนวิธี:

1. เริ่มจากช่วง $[lo, hi]$ ที่ครอบคลุมทั้งแถวลำดับ
2. ตรวจสอบสมาชิกตรงกลาง `a[mid]`
3. ถ้า `a[mid] == x` พบค่าที่ต้องการ คืนตำแหน่ง
4. ถ้า `a[mid] < x` แปลว่า x ต้องอยู่ในช่วงครึ่งขวา ย้าย `lo` ไป `mid + 1`
5. ถ้า `a[mid] > x` แปลว่า x ต้องอยู่ในช่วงครึ่งซ้าย ย้าย `hi` ไป `mid - 1`
6. ทำซ้ำจนกว่า `lo > hi` (ไม่เจอ)

ในแต่ละขั้น ช่วงการค้นหาจะลดลงครึ่งหนึ่ง ดังนั้นจำนวนขั้นสูงสุดคือ $\lceil \log_2 n \rceil$ ทำให้ความซับซ้อนเป็น $\mathcal{O}(\log n)$

นิยาม 3.1 (เงื่อนไขสำหรับ Binary Search)**.** Binary Search ใช้ได้เมื่อข้อมูลมีสมบัติว่า: ถ้า `a[i]` $\leq x$ แล้ว `a[j]` $\leq x$ สำหรับทุก $j < i$ กล่าวอีกนัยหนึ่ง ข้อมูลต้องเรียงลำดับจากน้อยไปมาก สมบัตินี้ทำให้เราสามารถตัดทอนช่วงการค้นหาได้ เพราะถ้า `a[mid]` $< x$ เรารู้ว่าสมาชิกทุกตัวที่อยู่ก่อน `mid` ก็ต้องน้อยกว่า x เช่นกัน จึงไม่จำเป็นต้องตรวจสอบ

ตัวอย่าง: ค้นหา 42 ในแถวลำดับที่เรียงแล้ว

แถวลำดับ: $[2, 5, 8, 12, 16, 23, 38, 42, 56, 72, 91]$

- ขั้นที่ 1: $lo = 0, hi = 10, mid = 5$; $a[5] = 23 < 42 \Rightarrow lo = 6$
- ขั้นที่ 2: $lo = 6, hi = 10, mid = 8$; $a[8] = 56 > 42 \Rightarrow hi = 7$
- ขั้นที่ 3: $lo = 6, hi = 7, mid = 6$; $a[6] = 38 < 42 \Rightarrow lo = 7$
- ขั้นที่ 4: $lo = 7, hi = 7, mid = 7$; $a[7] = 42 = 42 \Rightarrow$ พบที่ตำแหน่ง 7!

ใช้เพียง 4 ขั้น ในขณะที่ Linear Search ต้องตรวจสอบ 8 ตัว สำหรับแถวลำดับขนาด $n = 10^6$ Linear Search ต้องตรวจสอบถึง 10^6 ตัว แต่ Binary Search ตรวจสอบเพียงประมาณ 20 ตัวเท่านั้น

Overflow ในการหาค่ากลาง

เมื่อใช้ Binary Search กับค่าที่มีขนาดใหญ่ นิพจน์ `(lo + hi) / 2` อาจทำให้เกิด Overflow ได้ เพราะ `lo + hi` อาจเกินขอบเขตของ `int` ตัวอย่างเช่น ถ้า `lo = 2000000000` และ `hi = 2000000001` แล้ว `lo + hi = 4000000001` ซึ่งเกินค่าสูงสุดของ `int` ($\approx 2.1 \times 10^9$)
วิธีแก้: ใช้ `lo + (hi - lo) / 2` แทน ซึ่งให้ผลลัพธ์เดียวกันแต่ไม่เกิด Overflow เพราะ `hi -`

`lo` มีค่าไม่มาก หรือใช้ `long long` สำหรับ `lo` และ `hi` ตั้งแต่ต้น

3.5.3 Binary Search วิธีที่ 2: กระโดดแล้วชะลอ

วิธีที่ 2 ของ Binary Search ใช้แนวคิดที่แตกต่างออกไป — เริ่มจากกระโดดขนาดใหญ่ (ครึ่งหนึ่งของแถวลำดับ) แล้วลดขนาดกระโดดลงครึ่งหนึ่งในแต่ละขั้น วิธีนี้มีประโยชน์มากเมื่อต้องการหาตำแหน่งที่เงื่อนไขบางอย่างเปลี่ยนจากเท็จเป็นจริง ซึ่งเป็นรูปแบบที่พบบ่อยมากในการประยุกต์ Binary Search ขั้นสูง

```cpp
// Find last position where a[i] <= x
int search(int a[], int n, int x) {
    int k = 0;
    for (int b = n / 2; b >= 1; b /= 2) {
        while (k + b < n && a[k + b] <= x) {
            k += b;
        }
    }
    return k;   // k is the last position where a[k] <= x
}
```

ตัวอย่างโค้ดที่ 3.27: Binary Search วิธีกระโดดแล้วชะลอ

หลักการทำงาน:

1. เริ่มจากตำแหน่ง $k = 0$ และขนาดกระโดด $b = n/2$
2. ในแต่ละขั้น ลองกระโดดไป $k + b$ ถ้ายังเป็นไปได้ ($a[k + b] \leq x$) ให้กระโดด
3. ลดขนาดกระโดดลงครึ่งหนึ่ง ($b = b/2$)
4. ทำซ้ำจนกว่า $b = 0$

ความซับซ้อนยังคงเป็น $\mathcal{O}(\log n)$ เพราะขนาดกระโดดลดลงครึ่งหนึ่งทุกขั้น และลูป `while` ภายในทำงานไม่เกิน 2 ครั้งต่อขั้น (เพราะเราเพิ่งตรวจสอบแล้วว่า $a[k + b]$ เป็นไปได้ ดังนั้น $a[k + 2b]$ จะไม่เป็นไปได้เพราะขั้นก่อนหน้าจะกระโดดไปแล้ว)

เปรียบเทียบสองวิธี

วิธีที่ 1 (ลดช่วง) เข้าใจง่ายกว่าและเหมาะสำหรับการค้นหาค่าเฉพาะเจาะจง ส่วนวิธีที่ 2 (กระโดดแล้วชะลอ) มีประโยชน์มากกว่าเมื่อต้องการหา “จุดเปลี่ยน” — ตำแหน่งที่เงื่อนไขเปลี่ยนจากเท็จเป็นจริง ซึ่งเป็นรูปแบบที่พบบ่อยมากในการประยุกต์ Binary Search ขั้นสูง ในทางปฏิบัติ ทั้งสองวิธีให้ความซับซ้อนเท่ากันคือ $\mathcal{O}(\log n)$

3.5.4 ฟังก์ชัน STL: lower_bound, upper_bound, equal_range

C++ STL มีฟังก์ชันสำหรับ Binary Search ที่พร้อมใช้งาน ซึ่งมีประโยชน์มากในการแข่งขัน เพราะไม่ต้องเขียน Binary Search เอง ลดโอกาสเกิดข้อผิดพลาดจากการเขียนลูปผิด:

lower_bound คืน iterator ชี้ไปยังสมาชิกตัวแรกที่ **ไม่น้อยกว่า** ($\geq$) x:

```
vector<int> v = {1, 2, 4, 4, 4, 7, 9, 9, 10};
// Find first position where value >= 4
auto it = lower_bound(v.begin(), v.end(), 4);
// it points to v[2] = 4 (first element >= 4)
int pos = it - v.begin();   // pos = 2
```

ตัวอย่างโค้ดที่ 3.28: ใช้ lower_bound

upper_bound คืน iterator ชี้ไปยังสมาชิกตัวแรกที่ **มากกว่า** ($>$) x:

```
// Find first position where value > 4
auto it = upper_bound(v.begin(), v.end(), 4);
// it points to v[5] = 7 (first element > 4)
int pos = it - v.begin();   // pos = 5
```

ตัวอย่างโค้ดที่ 3.29: ใช้ upper_bound

ต้องเรียงก่อน!

lower_bound และ upper_bound ทำงานบนช่วงที่ **เรียงลำดับแล้ว** หากใช้กับข้อมูลที่ไม่เรียง ผลลัพธ์จะผิดพลาดโดยไม่มีการเตือนใดๆ จากคอมไพเลอร์ ต้องเรียงก่อนเสมอ!

การตรวจสอบว่าค่า x มีอยู่ในแถวลำดับหรือไม่:

```
bool exists = binarySearch(v.begin(), v.end(), x);
```

ตัวอย่างโค้ดที่ 3.30: ตรวจสอบว่า x มีในแถวลำดับหรือไม่

ฟังก์ชัน `binary_search()` คืน `true` ถ้า x มีในแถวลำดับ และ `false` ถ้าไม่มี อย่างไรก็ตาม ฟังก์ชันนี้ไม่ได้คืนตำแหน่ง หากต้องการตำแหน่งด้วย ให้ใช้ `lower_bound` แทน:

```
auto it = lower_bound(v.begin(), v.end(), x);
if (it != v.end() && *it == x) {
        // Found x at position (it - v.begin())
    int pos = it - v.begin();
}
```

ตัวอย่างโค้ดที่ 3.31: ตรวจสอบและหาตำแหน่งด้วย lower_bound

วิธีนี้มีประโยชน์มากกว่า `binary_search()` เพราะได้ทั้งคำตอบว่ามีอยู่หรือไม่และตำแหน่ง ในการแข่งขัน ส่วนใหญ่เราต้องการรู้ตำแหน่งด้วย จึงมักใช้ `lower_bound` แทน `binary_search`

equal_range คืนคู่ iterator ของ `lower_bound` และ `upper_bound` พร้อมกัน:

```cpp
auto range = equal_range(v.begin(), v.end(), 4);
// range.first  = lower_bound (points to v[2])
// range.second = upper_bound (points to v[5])
```

ตัวอย่างโค้ดที่ 3.32: ใช้ equal_range

3.5.5 การนับจำนวนค่าที่ซ้ำ

การนับจำนวนครั้งที่ค่า x ปรากฏในแถวลำดับที่เรียงแล้ว เป็นปัญหาที่พบบ่อยในการแข่งขัน สามารถทำได้ 2 วิธี:

วิธีที่ 1: ใช้ `upper_bound` $-$ `lower_bound`:

```cpp
int count = upper_bound(v.begin(), v.end(), x)
          - lower_bound(v.begin(), v.end(), x);
```

ตัวอย่างโค้ดที่ 3.33: นับจำนวนค่าซ้ำด้วย lower/upper_bound

หลักการ: `lower_bound` ชี้ไปตำแหน่งแรกที่ $\geq x$ และ `upper_bound` ชี้ไปตำแหน่งแรกที่ $> x$ ผลต่างของสอง iterator นี้จึงเท่ากับจำนวนสมาชิกที่มีค่าเท่ากับ x พอดี กล่าวคือ ช่วง $[\text{lower_bound}, \text{upper_bound})$ ครอบคลุมสมาชิกที่มีค่าเท่ากับ x ทุกตัว

วิธีที่ 2: ใช้ `equal_range`:

```cpp
auto range = equal_range(v.begin(), v.end(), x);
int count = range.second - range.first;
```

ตัวอย่างโค้ดที่ 3.34: นับจำนวนค่าซ้ำด้วย equal_range

ทั้งสองวิธีให้ผลลัพธ์เหมือนกัน แต่วิธีที่ 2 เรียกฟังก์ชันเพียงครั้งเดียว จึงมีประสิทธิภาพดีกว่าเล็กน้อย ความซับซ้อนของทั้งสองวิธีคือ $\mathcal{O}(\log n)$ ซึ่งเร็วกว่าการนับด้วย Linear Search ($\mathcal{O}(n)$) อย่างมาก

ตัวอย่าง: ค้นหาขอบเขตบนด้วย upper_bound

ให้แถวลำดับที่เรียงแล้ว $v = \{1, 2, 4, 4, 4, 7, 9, 9, 10\}$ ต้องการหาจำนวนสมาชิกที่มีค่าน้อยกว่าหรือเท่ากับ 4

เราสามารถใช้ `upper_bound` เพื่อหาตำแหน่งแรกที่มากกว่า 4 แล้วลบด้วย `v.begin()` เพื่อหาจำนวนสมาชิกที่ ≤ 4:

```cpp
auto it = upper_bound(v.begin(), v.end(), 4);
int count = it - v.begin();   // count = 5
// v[0]=1, v[1]=2, v[2]=4, v[3]=4, v[4]=4  all 5 elements <= 4
```

ตัวอย่างโค้ดที่ 3.35: หาจำนวนสมาชิกที่ <= x

ในทำนองเดียวกัน หากต้องการหาจำนวนสมาชิกที่น้อยกว่า 4 อย่างเคร่งครัด ให้ใช้ `lower_bound`:

```cpp
auto it = lower_bound(v.begin(), v.end(), 4);
```

```
int count = it - v.begin();   // count = 2
// v[0]=1, v[1]=2  both 2 elements < 4
```

ตัวอย่างโค้ดที่ 3.36: หาจำนวนสมาชิกที่ < x

3.6 การประยุกต์ Binary Search ขั้นสูง

การค้นหาแบบทวิภาคไม่ได้ใช้เพียงแค่การหาค่าในแถวลำดับเท่านั้น แต่ยังสามารถประยุกต์ใช้ได้หลากหลายมาก โดยเฉพาะในการโปรแกรมเชิงแข่งขัน หัวใจสำคัญของการประยุกต์ Binary Search คือการมองหา **จุดเปลี่ยน** — ตำแหน่งที่ฟังก์ชันหรือเงื่อนไขเปลี่ยนจากค่าหนึ่งไปอีกค่าหนึ่ง เทคนิคเหล่านี้อาจดูยากในตอนแรก แต่เมื่อเข้าใจหลักการแล้ว จะพบว่ามันประยุกต์ใช้ได้กับโจทย์หลากหลายมาก และเป็นเทคนิคที่แทบจะขาดไม่ได้สำหรับนักแข่งขันระดับกลางขึ้นไป

3.6.1 หาค่า k น้อยสุดที่ทำให้ฟังก์ชันเป็นจริง

สมมติว่ามีฟังก์ชัน `ok(x)` ที่มีสมบัติว่า:

- `ok(x)` เป็น `false` เมื่อ $x < k$
- `ok(x)` เป็น `true` เมื่อ $x \geq k$

เราต้องการหาค่า k ที่น้อยที่สุดที่ทำให้ `ok(x)` เป็น `true` เทคนิคคือหาค่า x ที่มากที่สุดที่ `ok(x)` เป็น `false` แล้ว $k = x + 1$:

```
// Find max x where ok(x) is false
// Answer is x + 1
int x = -1;   // Initial value: haven't found any x where ok() is false
for (int b = z; b >= 1; b /= 2) {
    while (!ok(x + b)) {
        x += b;
    }
}
int k = x + 1;
```

ตัวอย่างโค้ดที่ 3.37: หาค่า k น้อยสุดที่ ok(k) เป็นจริง

โค้ดนี้ใช้วิธีกระโดดแล้วชะลอ: เริ่มจากกระโดดขนาด z (ค่าสูงสุดที่เป็นไปได้) แล้วลดลงครึ่งหนึ่งทุกขั้น ในแต่ละขั้น ถ้ายังไม่ถึงจุดเปลี่ยน (`ok(x+b)` เป็น `false`) ให้กระโดดต่อ เมื่อจบแล้ว x จะเป็นค่ามากสุดที่ `ok(x)` เป็น `false` และ $k = x + 1$ คือค่าตอบ

ความซับซ้อน: $\mathcal{O}(\log z \times \text{cost of ok}())$ ซึ่งรวดเร็วมากหาก `ok()` ทำงานในเวลาพอสมควร

ตัวอย่าง: หาค่าราคาที่ถูกที่สุดที่ซื้อของได้

สมมติมีสินค้า n ชิ้น แต่ละชิ้นมีราคา a_i เรามีงบประมาณ B บาท อยากรู้ว่าสามารถซื้อสินค้าได้มากที่สุดกี่ชิ้น ถ้าเลือกซื้อ k ชิ้นที่ถูกที่สุด

```
// ok(k): can we buy k cheapest items?
bool ok(int k) {
    long long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += a[i];   // a is already sorted
    }
    return sum <= B;
}
```

เมื่อเรียง a จากน้อยไปมากแล้ว `ok(k)` จะเป็น `true` สำหรับ k น้อยๆ และเปลี่ยนเป็น `false` เมื่อ k มากเกินไป เราสามารถใช้ Binary Search หาค่า k มากสุดที่ `ok(k)` เป็น `true` ได้ โดยใช้รูปแบบ “หาค่า $k+1$ น้อยสุดที่ `ok(k+1)` เป็น `false`” นั่นเอง

3.6.2 หาค่าสูงสุดของฟังก์ชัน Unimodal

ฟังก์ชัน Unimodal คือฟังก์ชันที่เพิ่มขึ้นก่อนแล้วลดลง (เหมือนเขายอดเดียว) สำหรับฟังก์ชันประเภทนี้ เราสามารถใช้ Binary Search เพื่อหาจุดสูงสุดได้ หลักการคือ: ถ้า $f(x) < f(x+1)$ แสดงว่ายังไม่ถึงจุดสูงสุด (กำลังขึ้นเขา) ถ้า $f(x) > f(x+1)$ แสดงว่าผ่านจุดสูงสุดมาแล้ว (กำลังลงเขา) เราต้องการหาค่า x ที่มากที่สุดที่ $f(x) < f(x+1)$ จุดสูงสุดจะอยู่ที่ $x+1$

```
// Find x that maximizes f(x)
// Assume f increases first, then decreases
int x = lo;   // Initial value
for (int b = (hi - lo) / 2; b >= 1; b /= 2) {
    while (x + b <= hi && f(x + b) > f(x)) {
        x += b;
    }
}
// f(x) is the maximum value
```

ตัวอย่างโค้ดที่ 3.38: หาจุดสูงสุดของฟังก์ชัน Unimodal

ข้อจำกัด: ค่าต่อเนื่องไม่เท่ากัน

วิธีนี้ใช้ได้ **เฉพาะเมื่อ** ไม่มีค่าต่อเนื่องที่เท่ากัน (No Plateau) กล่าวคือ ถ้ามีช่วงที่ $f(x) = f(x+1)$ แล้ววิธีนี้จะหาจุดสูงสุดไม่ได้ เพราะไม่รู้ว่าช่วงที่เท่ากันนั้นอยู่ก่อนหรือหลังจุดสูงสุด ในกรณีนี้ ต้องใช้ Ternary Search หรือวิธีอื่นแทน

3.6.3 Binary Search on Answer

Binary Search on Answer เป็นเทคนิคที่ทรงพลังมากในการแข่งขัน และเป็นหนึ่งในเทคนิคที่พบบ่อยที่สุดในโจทย์ระดับกลางถึงยาก หลักการคือแทนที่จะหาคำตอบโดยตรง เรา “ทาย” คำตอบแล้วตรวจสอบว่าทายนั้นเป็นไปได้หรือไม่ ด้วย Binary Search เราสามารถลดจำนวนครั้งที่ต้องทายจาก 10^9 ครั้งเหลือเพียง $\log_2(10^9) \approx 30$ ครั้ง ทำให้สามารถแก้โจทย์ที่ดูเหมือนจะต้องลองทุกค่าได้ภายในเวลาที่กำหนด

รูปแบบโจทย์ที่พบบ่อย:

- “หาค่า **น้อยสุด** ของ **ค่ามากสุด**” — เช่น หาระยะห่างน้อยสุดระหว่างจุดที่ทำให้ครอบคลุมทุกจุด
- “หาค่า **มากสุด** ของ **ค่าน้อยสุด**” — เช่น หาระยะห่างมากสุดระหว่างจุดที่ใกล้กันที่สุด
- “หาเวลา **น้อยสุด** ที่ทำงานเสร็จ” — เช่น หาเวลาน้อยสุดที่คน k คนทำงาน n ชิ้นเสร็จ

```cpp
// Template: find minimum x where check(x) is true
long long lo = 0, hi = 1e18;
while (lo < hi) {
    long long mid = (lo + hi) / 2;
    if (check(mid)) {
        hi = mid;          // mid is possible, try smaller value
    } else {
        lo = mid + 1;      // mid doesn't work, need to increase
    }
}
// lo is the answer
```

ตัวอย่างโค้ดที่ 3.39: เทมเพลต Binary Search on Answer

สังเกตว่าเราใช้ `long long` สำหรับค่าตอบ เพราะคำตอบอาจมีค่ามากกว่า 2^{31} และเราใช้ `lo + (hi - lo) / 2` หรือ `(lo + hi) / 2` (ปลอดภัยเมื่อใช้ `long long`) สำหรับการหาค่ากลาง การใช้ `while (lo < hi)` แทน `while (lo <= hi)` ทำให้ลูปจบเมื่อ `lo == hi` ซึ่งก็คือคำตอบ

ตัวอย่าง: Aggressive Cows

โจทย์: มีจุด n จุดบนเส้นตรง (เรียงแล้ว) และวัว c ตัว ต้องวางวัวแต่ละตัวที่จุดใดจุดหนึ่ง โดยต้องการให้ระยะห่างน้อยสุดระหว่างวัว 2 ตัวที่ใกล้กันที่สุด **มีค่ามากที่สุด** จะวางวัวอย่างไร?
วิธีคิด: Binary Search บนระยะห่าง d

- `check(d)`: วางวัวได้หรือไม่ โดยที่วัวทุกคู่ห่างกันอย่างน้อย d
- ถ้า `check(d)` เป็น `true` แสดงว่าระยะ d เป็นไปได้ ลองเพิ่ม d
- ถ้า `check(d)` เป็น `false` แสดงว่าระยะ d ใหญ่เกินไป ต้องลด d

```cpp
bool check(long long d) {
    int count = 1;              // Place first cow at first position
```

```
    int last = pos[0];
    for (int i = 1; i < n; i++) {
        if (pos[i] - last >= d) {
            count++;
            last = pos[i];
        }
    }
    return count >= c;          // Placed all cows or not
}
```

ฟังก์ชัน `check(d)` ทำงานโดยวางวัวตัวแรกที่จุดแรก แล้ววางวัวตัวถัดไปที่จุดแรกที่ห่างจากวัวตัวก่อนอย่างน้อย d ทำซ้ำจนกว่าจะผ่านทุกจุด ถ้าวางวัวได้ครบ c ตัว แสดงว่าระยะ d เป็นไปได้ ความซับซ้อนรวม: $\mathcal{O}(n \log(\text{range}))$ ซึ่งรวดเร็วมาก

ตัวอย่าง: Binary Search on Answer — เวลาน้อยสุดที่ทำงานเสร็จ

โจทย์: มีเครื่องจักร n เครื่อง แต่ละเครื่องสามารถผลิตชิ้นงานได้ทุก ๆ t_i วัน (เครื่องจักรเครื่องที่ i ใช้เวลา t_i วันต่อ 1 ชิ้น) ต้องการผลิตชิ้นงานรวม k ชิ้น แต่ละเครื่องทำงานได้ทีละ 1 ชิ้น หาเวลาน้อยสุดที่ผลิตชิ้นงาน k ชิ้นเสร็จ

วิธีคิด: Binary Search บนเวลา d

```
bool check(long long d) {
    long long total = 0;
    for (int i = 0; i < n; i++) {
        total += d / t[i];   // Machine producing 1 item per t[i] days
    }                        // Can produce d/t[i] items in d days
    return total >= k;
}
```

ตัวอย่างโค้ดที่ 3.40: ตรวจสอบว่าเวลา d วันเพียงพอหรือไม่

ถ้า `check(d)` เป็น `true` แสดงว่า d วันเพียงพอ ลองลดเวลาลง ถ้า `check(d)` เป็น `false` แสดงว่า d วันไม่พอ ต้องเพิ่มเวลา

3.6.4 ตัวอย่าง Binary Search on Answer เพิ่มเติม

หลังจากที่เราได้เรียนรู้แนวคิดของ Binary Search on Answer ไปแล้ว เรามาดูตัวอย่างเพิ่มเติมจากโจทย์ข้ออื่นๆ ที่ใช้รูปแบบเดียวกัน ได้แก่ การหาค่าน้อยสุดของค่ามากสุด (Minimize the Maximum) โจทย์ทั้งสองข้อที่จะนำเสนอมีรูปแบบคล้ายกันมาก — ให้ข้อมูลที่ต้องแบ่งออกเป็นกลุ่ม แล้วหาค่ามากสุดของผลรวมในแต่ละกลุ่มที่น้อยที่สุดเท่าที่เป็นไปได้ นี่คือรูปแบบโจทย์ที่พบบ่อยมากในการแข่งขัน และเป็นรูปแบบที่ Binary Search on Answer ตอบได้ดีที่สุด

รูปแบบโจทย์ Minimize the Maximum

รูปแบบโจทย์ “หาค่าน้อยสุดของค่ามากสุด” เป็นหนึ่งในรูปแบบที่พบบ่อยที่สุดของ Binary Search on Answer หลักการคือ:

1. กำหนดค่า X ที่เป็น “ขีดจำกัด” ของสิ่งที่เราต้องการลดให้น้อยที่สุด
2. เขียนฟังก์ชัน `check(X)` ที่ตรวจสอบว่าเราสามารถแบ่งข้อมูลได้โดยที่ค่ามากสุดในแต่ละกลุ่มไม่เกิน X หรือไม่
3. ใช้ Binary Search หาค่า X น้อยสุดที่ `check(X)` เป็น `true`

สมบัติสำคัญ (Monotonic Predicate) คือ: ถ้า `check(X)` เป็น `true` แล้ว `check(X+1)` จะเป็น `true` ด้วยเสมอ (เพราะขีดจำกัดที่มากขึ้นย่อมทำให้แบ่งได้ง่ายขึ้น) ทำให้เราสามารถใช้ Binary Search ได้

UVa 714 — Copying Books

โจทย์: มีหนังสือ m เล่ม เล่มที่ i มีจำนวนหน้า p_i หน้า ต้องการจ้างคัดลอก (Scriber) k คนให้คัดลอกหนังสือทั้งหมด โดยหนังสือจะถูกแบ่งเป็น k ช่วงต่อเนื่อง (หนังสือเล่มที่ $1, 2, \ldots, p_1$ ให้คนที่ 1, เล่มถัดไปให้คนที่ 2, ...) คัดลอกคนละช่วง และต้องการให้จำนวนหน้ามากสุดที่คัดลอกคนใดคนหนึ่งต้องทำ **น้อยที่สุด** หาค่าดังกล่าว[20]

ตัวอย่าง: หนังสือ 9 เล่ม หน้า $= [100, 200, 300, 400, 500, 600, 700, 800, 900]$ จ้างคัดลอก 3 คน พิจารณาวิธีแบ่ง: $[100{+}200{+}300{+}400{+}500{+}600]$, $[700{+}800]$, $[900]$ ได้ผลรวมมากสุด $= 2100$ แต่หากแบ่งเป็น $[100{+}200{+}300{+}400{+}500]$, $[600{+}700]$, $[800{+}900]$ จะได้ผลรวมมากสุด $= 1700$ ซึ่งน้อยกว่า และเป็นคำตอบที่เหมาะสมที่สุด

Monotonic Predicate: กำหนดค่า X แล้วถามว่า “เราสามารถแบ่งหนังสือทั้งหมดให้คัดลอก k คน โดยที่คนใดๆ คัดลอกไม่เกิน X หน้า ได้หรือไม่?” สังเกตว่า:

- ถ้า X น้อยเกินไป จะแบ่งไม่ได้ (หนังสือเล่มใดเล่มหนึ่งอาจมีหน้าเกิน X หรือต้องใช้คัดลอกมากกว่า k คน)
- ถ้า X มากพอ จะแบ่งได้เสมอ
- ถ้า `check(X)` เป็น `true` แล้ว `check(X+1)` ก็เป็น `true` เช่นกัน เพราะขีดจำกัดที่มากขึ้นย่อมทำให้แบ่งได้ง่ายขึ้นหรือเท่าเดิม

ด้วยสมบัติ Monotonic นี้ เราสามารถใช้ Binary Search หาค่า X น้อยสุดที่ `check(X)` เป็น `true` ได้

วิธีการเขียนฟังก์ชัน `check(X)`: วนจากหนังสือเล่มแรกไปเล่มสุดท้าย สะสมจำนวนหน้า ถ้าผลรวมเกิน X ให้เริ่มช่วงใหม่ (เพิ่มจำนวนคัดลอกที่ต้องใช้) ถ้าจำนวนคัดลอกที่ต้องใช้ไม่เกิน k แสดงว่า X เป็นไปได้

```
#include <bits/stdc++.h>
using namespace std;
int m, k;
```

```cpp
long long pages[505];
// Check: can we assign all books to at most k scribers
// such that no scriber copies more than X pages?
bool check(long long X) {
    int scriber_count = 1;    // Start with 1 scriber
    long long current_sum = 0;
    for (int i = 0; i < m; i++) {
        if (pages[i] > X) return false;   // Single book exceeds limit
        if (current_sum + pages[i] > X) {
            scriber_count++;      // Need a new scriber
            current_sum = pages[i];
        } else {
            current_sum += pages[i];
        }
    }
    return scriber_count <= k;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--) {
        cin >> m >> k;
        long long lo = 0, hi = 0;
        for (int i = 0; i < m; i++) {
            cin >> pages[i];
            lo = max(lo, pages[i]);   // Lower bound: largest single book
            hi += pages[i];           // Upper bound: sum of all pages
        }
        // Binary search for minimum X where check(X) is true
        while (lo < hi) {
            long long mid = (lo + hi) / 2;
            if (check(mid)) {
                hi = mid;         // mid works, try smaller
            } else {
                lo = mid + 1;     // mid too small, increase
            }
        }
        cout << lo << "\n";       // lo is the answer
    }
```

```
45      return 0;
46  }
```

ตัวอย่างโค้ดที่ 3.41: UVa 714 — Copying Books: Binary Search on Answer

การกำหนดขอบเขตล่างของ Binary Search

ในโจทย์ Copying Books ขอบเขตล่าง `lo` ไม่ควรเริ่มจาก 0 แต่ควรเริ่มจากค่ามากสุดของหนังสือแต่ละเล่ม เพราะไม่ว่าจะแบ่งอย่างไร คัดลอกคนใดคนหนึ่งต้องรับหนังสือเล่มที่หนาที่สุดอยู่ดี ดังนั้นคำตอบไม่มีทางน้อยกว่าจำนวนหน้าของหนังสือเล่มที่หนาที่สุด การกำหนดขอบเขตให้แน่นขึ้นช่วยลดจำนวนรอบของ Binary Search และที่สำคัญกว่านั้นคือป้องกันกรณีที่ `check()` ทำงานผิดพลาดเมื่อ X น้อยกว่าหนังสือเล่มใดเล่มหนึ่ง

ระวังขอบเขตบนและ Overflow

ขอบเขตบน `hi` คือผลรวมของหน้าทั้งหมด ซึ่งอาจมีค่ามากถึง $m \times 10^7 \approx 5 \times 10^9$ สำหรับโจทย์นี้ ดังนั้นต้องใช้ `long long` สำหรับ `lo`, `hi`, `mid` และตัวแปรสะสมผลรวมภายใน `check()` การใช้ `int` จะทำให้เกิด Integer Overflow และได้คำตอบผิด ข้อผิดพลาดนี้พบได้บ่อยมากในการแข่งขัน!

ความซับซ้อนของขั้นตอนวิธีนี้คือ $\mathcal{O}(m \log(\text{sum}))$ โดย `sum` คือผลรวมของหน้าทั้งหมด ซึ่งอยู่ในระดับ 10^9 ทำให้ $\log(\text{sum}) \approx 30$ รอบ แต่ละรอบของ `check()` ใช้เวลา $\mathcal{O}(m)$ ดังนั้นรวมแล้วเร็วมาก

ตัวอย่างการทำงาน: Copying Books

สมมติหนังสือ 9 เล่ม หน้า $= [100, 200, 300, 400, 500, 600, 700, 800, 900]$ คัดลอก 3 คน
ขอบเขต: `lo` $= 900$ (หนังสือหนาที่สุด), `hi` $= 4500$ (ผลรวมทั้งหมด)
Binary Search:

- $mid = 2700$: `check(2700)` = แบ่งได้ $[100{+}200{+}300{+}400{+}500{+}600{+}700]$, $[800]$, $[900] \rightarrow$ ใช้ 3 คน $\leq k \rightarrow$ `true`, `hi` $= 2700$
- $mid = 1800$: `check(1800)` = แบ่งได้ $[100{+}200{+}300{+}400{+}500]$, $[600{+}700]$, $[800{+}900] \rightarrow$ ใช้ 3 คน $\leq k \rightarrow$ `true`, `hi` $= 1800$
- $mid = 1350$: `check(1350)` = แบ่งได้ $[100{+}200{+}300{+}400]$, $[500{+}600]$, $[700]$,... ใช้มากกว่า 3 คน $\rightarrow$ `false`, `lo` $= 1351$
- ... ทำต่อจนได้คำตอบ $= 1700$

คำตอบสุดท้ายคือ 1700 ซึ่งเป็นจำนวนหน้ามากสุดที่คัดลอกคนใดคนหนึ่งต้องทำ และน้อยที่สุดเท่าที่เป็นไปได้

UVa 11413 — Fill the Containers

โจทย์: มีภาชนะบรรจุนม n ใบเรียงอยู่ตามลำดับ ภาชนะใบที่ i บรรจุนมได้ m_i หน่วย ต้องการบรรจุนมทั้งหมดลงในคอนเทนเนอร์ (Container) c ใบ โดยนมจะถูกแบ่งเป็น c ช่วงต่อเนื่อง (เช่นเดียวกับ Copying Books) แต่ละคอนเทนเนอร์รับนมได้ไม่เกินความจุของคอนเทนเนอร์ ต้องการหาความจุคอนเทนเนอร์ **น้อยสุด** ที่บรรจุนมทั้งหมดได้[20]

ตัวอย่าง: นม $= [5, 3, 8, 1, 4]$ คอนเทนเนอร์ 2 ใบ จงพิจารณาวิธีแบ่งทั้งหมด: $[5{+}3{+}8{+}1] = 17$, $[4] = 4 \rightarrow$ มากสุด 17; $[5{+}3{+}8] = 16$, $[1{+}4] = 5 \rightarrow$ มากสุด 16; $[5{+}3] = 8$, $[8{+}1{+}4] = 13 \rightarrow$ มากสุด 13; $[5] = 5$, $[3{+}8{+}1{+}4] = 16 \rightarrow$ มากสุด 16 — คำตอบที่ถูกต้องคือ **13** จากการแบ่ง $[5, 3]$ และ $[8, 1, 4]$ ซึ่งทำให้ค่ามากสุดน้อยที่สุด

Monotonic Predicate: กำหนดค่า X แล้วถามว่า “เราสามารถบรรจุนมทั้งหมดลงในคอนเทนเนอร์ c ใบ โดยแต่ละใบไม่เกิน X หน่วย ได้หรือไม่?” เช่นเดียวกับ Copying Books สมบัติ Monotonic ก็ใช้ได้: ถ้า `check(X)` เป็น `true` แล้ว `check(X+1)` ก็เป็น `true` เช่นกัน

โครงสร้างของฟังก์ชัน `check(X)` ก็คล้ายกับ Copying Books มาก: วนจากภาชนะใบแรกไปใบสุดท้าย สะสมปริมาณนม ถ้าเกิน X ให้เปิดคอนเทนเนอร์ใบใหม่ ถ้าจำนวนคอนเทนเนอร์ที่ใช้ไม่เกิน c แสดงว่า X เป็นไปได้

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, c;
int milk[1005];
// Check: can we fill all milk into at most c containers
// such that no container holds more than X units?
bool check(long long X) {
    int container_count = 1;    // Start with 1 container
    long long current_sum = 0;
    for (int i = 0; i < n; i++) {
        if (milk[i] > X) return false;   // Single vessel exceeds capacity
        if (current_sum + milk[i] > X) {
            container_count++;    // Need a new container
            current_sum = milk[i];
        } else {
            current_sum += milk[i];
        }
    }
    return container_count <= c;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    while (cin >> n >> c) {
        long long lo = 0, hi = 0;
```

```
        for (int i = 0; i < n; i++) {
            cin >> milk[i];
            lo = max(lo, (long long)milk[i]);   // Lower bound: largest vessel
            hi += milk[i];                      // Upper bound: total milk
        }
        // Binary search for minimum container capacity
        while (lo < hi) {
            long long mid = (lo + hi) / 2;
            if (check(mid)) {
                hi = mid;          // mid works, try smaller capacity
            } else {
                lo = mid + 1;      // mid too small, increase capacity
            }
        }
        cout << lo << "\n";        // lo is the answer
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 3.42: UVa 11413 — Fill the Containers: Binary Search on Answer

ตัวอย่างการทำงาน: Fill the Containers

สมมตินม $= [5, 3, 8, 1, 4]$ คอนเทนเนอร์ 2 ใบ
ขอบเขต: `lo` $= 8$ (นมมากสุด), `hi` $= 21$ (ผลรวมทั้งหมด)
Binary Search:

- $mid = 14$: `check(14)` $\rightarrow [5{+}3] = 8$, $[8{+}1{+}4] = 13 \rightarrow$ 2 ใบ $\leq c \rightarrow$ `true`, `hi` $= 14$
- $mid = 11$: `check(11)` $\rightarrow [5{+}3] = 8$, $[8{+}1] = 9$, $[4] = 4 \rightarrow$ 3 ใบ $> c \rightarrow$ `false`, `lo` $= 12$
- $mid = 13$: `check(13)` $\rightarrow [5{+}3] = 8$, $[8{+}1{+}4] = 13 \rightarrow$ 2 ใบ $\leq c \rightarrow$ `true`, `hi` $= 13$
- $mid = 12$: `check(12)` $\rightarrow [5{+}3] = 8$, $[8{+}1] = 9$, $[4] = 4 \rightarrow$ 3 ใบ $> c \rightarrow$ `false`, `lo` $= 13$

เมื่อ `lo == hi == 13` ลูปจบ คำตอบคือ 13

สังเกตว่าโค้ดของทั้งสองข้อแทบจะเหมือนกันทุกประการ — ต่างกันเพียงชื่อตัวแปรเท่านั้น นี่เป็นเพราะทั้งสองข้อเป็นรูปแบบเดียวกัน: แบ่งลำดับต่อเนื่องเป็นกลุ่ม โดยหาค่ามากสุดของผลรวมในแต่ละกลุ่มให้น้อยที่สุด

รูปแบบที่น่าสนใจ

เมื่อเจอโจทย์ที่พูดถึง “แบ่งข้อมูลต่อเนื่องเป็นกลุ่ม โดยหาค่า ...ของแต่ละกลุ่มให้น้อยที่สุด/มากที่สุด” ให้นึกถึง Binary Search on Answer ทันที รูปแบบนี้ปรากฏในโจทย์หลายร้อยข้อบน UVa[20], Codeforces[10], และ ICPC[13] การจดจำรูปแบบ (Pattern Recognition) เป็นทักษะสำคัญที่ช่วยให้แก้โจทย์ได้เร็วขึ้นมาก
ขั้นตอนคิดสำหรับรูปแบบนี้เสมอ:

1. กำหนด “ขีดจำกัด” X ที่ต้องการหาค่าดีที่สุด
2. เขียน `check(X)` แบบ Greedy: สะสมค่าไปเรื่อยๆ ถ้าเกิน X ให้เริ่มกลุ่มใหม่
3. กำหนดขอบเขต: `lo` $=$ ค่ามากสุดของสมาชิกตัวเดียว, `hi` $=$ ผลรวมทั้งหมด
4. Binary Search หาค่า X ที่ดีที่สุด

3.7 โจทย์แนะนำ

บทนี้ครอบคลุมสองทักษะหลักคือ การเรียงลำดับด้วย `sort()` และการค้นหาแบบทวิภาค (Binary Search) รวมถึงเทคนิคขั้นสูงอย่าง Binary Search on Answer โจทย์แนะนำด้านล่างจัดเป็นสองกลุ่ม กลุ่มแรกเน้นการใช้ `sort()` และตัวเปรียบเทียบ กลุ่มที่สองเน้น Binary Search ตั้งแต่ระดับ `lower_bound` ธรรมดาไปจนถึง Binary Search on Answer ขอแนะนำให้เริ่มจากกลุ่ม Sorting ก่อน จากนั้นจึงไป Binary Search ธรรมดา และสุดท้ายคือ Binary Search on Answer ซึ่งเป็นเทคนิคขั้นสูงที่ต้องฝึกฝนมากขึ้น

ตารางที่ 3.2 โจทย์แนะนำสำหรับบทที่ 3(Sorting)

โจทย์	แหล่ง	ระดับ	เทคนิคที่ใช้	หมายเหตุ
Twins	Codeforces[10] 160A	ง่าย	`sort()` + prefix sum	ฝึกเรียงแล้วหาคำตอบจากด้านหนึ่ง
Sort the Array	Codeforces 451B	ง่าย	`sort()` เปรียบเทียบกับของเดิม	ฝึกตรวจสอบว่า subarray เรียงกลางเป็นช่วงเดียวหรือไม่
Two-gram	Codeforces 977B	ง่าย	นับคู่อักขระด้วย `map`	ฝึกใช้ `map<string,int>` นับ substring
Write and Erase	AtCoder ABC 073 C	ปานกลาง	`sort()` + นับซ้ำ	ฝึก sort แล้วนับ element ที่ซ้ำกัน
Sorting Methods	CSES	ปานกลาง	เลือก algorithm ตามเงื่อนไข	ฝึก เข้าใจ คุณสมบัติแต่ละ sort (in-place, stable)

ตารางที่ 3.3 โจทย์แนะนำสำหรับบทที่ 3(Binary Search)

โจทย์	แหล่ง	ระดับ	เทคนิคที่ใช้	หมายเหตุ
Maximum Increase	Codeforces 702A	ง่าย	ลูปเดียว $\mathcal{O}(n)$	ฝึกอ่านข้อจำกัดและเลือกความซับซ้อน
Snuke Festival	AtCoder ABC 077 C	ปานกลาง	`lower_bound` สองแถวลำดับ	ฝึกใช้ `lower_bound` กับข้อมูลที่เรียงแล้ว
Books	CSES	ปานกลาง	Binary Search / Two Pointers	ฝึกเลือกไฟล์ที่อ่านติดต่อกันได้ภายในเวลา
Hamburgers	Codeforces 371C	ปานกลาง	Binary Search on Answer	ฝึกคิดฟังก์ชัน check สำหรับจำนวน hamburger สูงสุด
Factory Machines	CSES	ปานกลาง	Binary Search on Answer	ฝึกหาเวลาน้อยสุดที่ผลิตสินค้าครบตามเป้า
Array Division	CSES	ยาก	Binary Search on Answer	ฝึกแบ่ง subarray เป็น K ช่วง โดยผลรวมสูงสุด $\leq$ mid
Widespread	AtCoder ABC 063 D	ยาก	Binary Search on Answer	ฝึกฟังก์ชัน check ที่ซับซ้อนขึ้น (เกี่ยวข้องกับการระเบิด)
Aggressive Cows	SPOJ AGGRCOW	ยาก	Binary Search on Answer	ฝึกหาระยะห่างน้อยสุดระหว่าง $\geq K$ ตัว

คำแนะนำในการฝึก

- **เรียงก่อนค้น** — โจทย์ Binary Search เกือบทั้งหมดต้องเรียงข้อมูลก่อน ฝึกนิสัยเรียงก่อนใช้ `lower_bound`, `upper_bound`, `binary_search`
- **เริ่มจาก `lower_bound`** — ก่อนเขียน Binary Search เอง ให้ลองใช้ STL `lower_bound` / `upper_bound` ก่อน มักจะสั้นและผิดน้อยกว่า
- **ฝึก Binary Search on Answer อย่างน้อย 3–5 ข้อ** — เป็นเทคนิคที่ต้องฝึกฝนจนคุ้นเคยกับรูปแบบ โดยเฉพาะการออกแบบฟังก์ชัน `check(x)` ที่ตอบคำถาม “ถ้าคำตอบ $\leq x$ จะทำได้หรือไม่”
- **ระวัง overflow ใน mid** — ใช้ `lo + (hi - lo) / 2` แทน `(lo + hi) / 2` เมื่อ `lo`, `hi` มีค่ามาก เพื่อหลีกเลี่ยง overflow

3.8 สรุปบทที่ 3

บทนี้กล่าวถึงการเรียงลำดับและการค้นหาแบบทวิภาคซึ่งเป็นหัวข้อพื้นฐานที่สำคัญอย่างยิ่งในการโปรแกรมเชิงแข่งขัน โจทย์จำนวนมากต้องใช้ทั้งสองเทคนิคร่วมกัน — เรียงลำดับข้อมูลก่อน แล้วใช้ Binary Search ในการค้นหาหรือตอบคำถาม ในหัวข้อนี้ เราจะสรุปสิ่งที่เรียนรู้และข้อผิดพลาดที่ควรระวัง

3.8.1 ตารางสรุปขั้นตอนวิธี Sorting

ตารางที่ 3.4 สรุปขั้นตอนวิธีการเรียงลำดับ

ขั้นตอนวิธี	Best	Average	Worst	Space	Stable	หมายเหตุ
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	ใช่	เรียนรู้เท่านั้น
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	ไม่	Swap น้อย
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	ใช่	เร็วเมื่อเกือบเรียง
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	ใช่	รับประกันเสมอ
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	ไม่	เร็วในทางปฏิบัติ
Introsort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$	ไม่	`std::sort`
Counting Sort	$\mathcal{O}(n+k)$	$\mathcal{O}(n+k)$	$\mathcal{O}(n+k)$	$\mathcal{O}(k)$	ใช่	ค่าจำกัดช่วง k

3.8.2 ตารางสรุป Binary Search

ตารางที่ 3.5 สรุปฟังก์ชัน Binary Search ใน C++

ฟังก์ชัน	เวลา	คำอธิบาย
`binary_search`	$\mathcal{O}(\log n)$	ตรวจสอบว่า x มีอยู่หรือไม่ (คืน `bool`)
`lower_bound`	$\mathcal{O}(\log n)$	iterator ชี้สมาชิกตัวแรก $\geq x$
`upper_bound`	$\mathcal{O}(\log n)$	iterator ชี้สมาชิกตัวแรก $> x$
`equal_range`	$\mathcal{O}(\log n)$	คืนคู่ (lower_bound, upper_bound)
Binary Search on Answer	$\mathcal{O}(\log z \times \text{check})$	หาคำตอบที่เป็นไปได้

3.8.3 ข้อผิดพลาดที่พบบ่อย

ข้อผิดพลาดที่พบบ่อยในบทที่ 3

1. **ลืม เรียง ก่อน ใช้ Binary Search** — `lower_bound`, `upper_bound` และ `binary_search` ต้อง ใช้ กับ ข้อมูล ที่ เรียง แล้ว เท่านั้น หาก ใช้ กับ ข้อมูล ที่ ไม่ เรียง ผลลัพธ์จะผิดโดยไม่มีข้อความเตือนใดๆ จากคอมไพเลอร์

2. **ตัวเปรียบเทียบใช้ <= แทน <** — ฟังก์ชันเปรียบเทียบต้องเป็น Strict Weak Ordering การใช้ <= ทำให้ `comp(a,a)` คืน `true` ซึ่งละเมิดเงื่อนไข Irreflexive อาจทำให้ `sort()` ทำงานผิดพลาดหรือค้างได้

3. **Overflow ในการหาค่ากลาง** — `(lo + hi) / 2` อาจเกินขอบเขต `int` เมื่อ `lo` และ `hi` มีค่ามาก วิธีแก้คือใช้ `lo + (hi - lo) / 2` แทน หรือใช้ `long long` ตั้งแต่ต้น

4. **Binary Search วนไม่รู้จบ** — ต้องระวังเงื่อนไขหยุดของลูป โดยเฉพาะเมื่อใช้ `while (lo < hi)` ต้องแน่ใจว่า `lo` หรือ `hi` เปลี่ยนแปลงในทุกรอบ มิฉะนั้นลูปจะวนไม่รู้จบ

5. **ลืม `const` ใน `operator<`** — หากลืม `const` ไม่ว่าจะที่พารามิเตอร์หรือท้ายเมธอด โค้ดจะคอมไพล์ไม่ผ่านเมื่อใช้กับ `sort()`

6. **ใช้ Counting Sort กับค่าที่ช่วงใหญ่เกินไป** — Counting Sort ใช้หน่วยความจำ $\mathcal{O}(k)$ หาก k มีขนาดใหญ่เกินไป (เช่น $k = 10^9$) จะใช้หน่วยความจำเกินข้อจำกัดของโจทย์

Chapter 4

แบ่งแยกและเอาชนะ (Divide and Conquer)

แบ่งแยกและเอาชนะ (Divide and Conquer) เป็นหนึ่งในรูปแบบการแก้ปัญหาที่ทรงพลังและพบได้บ่อยที่สุดในวิทยาการคอมพิวเตอร์ หลักการพื้นฐานคือการแบ่งปัญหาขนาดใหญ่ให้เป็นปัญหาย่อยที่เล็กลง แก้ปัญหาย่อยเหล่านั้นแยกกัน (โดยทั่วไปใช้การเรียกตัวเองซ้ำ หรือ Recursion) แล้วจึงรวมคำตอบย่อยมาเป็นคำตอบของปัญหาเดิม กลยุทธ์นี้ปรากฏในขั้นตอนวิธีสำคัญมากมาย เช่น Merge Sort, Quick Sort, Binary Search และการแก้ปัญหาเรขาคณิตเชิงคำนวณ ในบทนี้ เราจะศึกษาหลักการของ Divide and Conquer อย่างเป็นระบบ เรียนรู้ขั้นตอนวิธีคลาสสิกที่ใช้หลักการนี้ และฝึกปฏิบัติผ่านโจทย์ตัวอย่างที่พบได้ในการแข่งขัน[2]

4.1 หลักการของ Divide and Conquer

Divide and Conquer ประกอบด้วย 3 ขั้นตอนหลัก ดังนี้:

1. **แบ่ง (Divide):** แบ่งปัญหาเดิมที่มีขนาด n ออกเป็นปัญหาย่อย (Subproblems) ขนาดเล็กลง โดยทั่วไปมักแบ่งเป็นปัญหาย่อย a ปัญหา แต่ละปัญหามีขนาดประมาณ n/b เมื่อ a และ b เป็นค่าคงที่ ตัวอย่างเช่น แบ่งแถวลำดับครึ่งหนึ่ง ($b = 2$) แล้วแก้ปัญหาย่อย 2 ปัญหา ($a = 2$)

2. **เอาชนะ (Conquer):** แก้ปัญหาย่อยแต่ละปัญหาโดยการเรียกตัวเองซ้ำ (Recursive Call) ถ้าปัญหาย่อยมีขนาดเล็กพอที่จะแก้ได้ทันที ก็คืนคำตอบโดยตรงโดยไม่ต้องเรียกซ้ำ (Base Case)

3. **รวม (Combine):** รวมคำตอบย่อยจากปัญหาย่อยทั้งหมดมาเป็นคำตอบของปัญหาเดิม ขั้นตอนนี้อาจง่ายมาก (เช่น Quick Sort ที่ไม่ต้องทำอะไรเลย) หรือซับซ้อน (เช่น Merge Sort ที่ต้องรวมแถวลำดับสองส่วน)

รูปแบบทั่วไปของขั้นตอนวิธี Divide and Conquer เขียนเป็น pseudocode ได้ดังนี้:

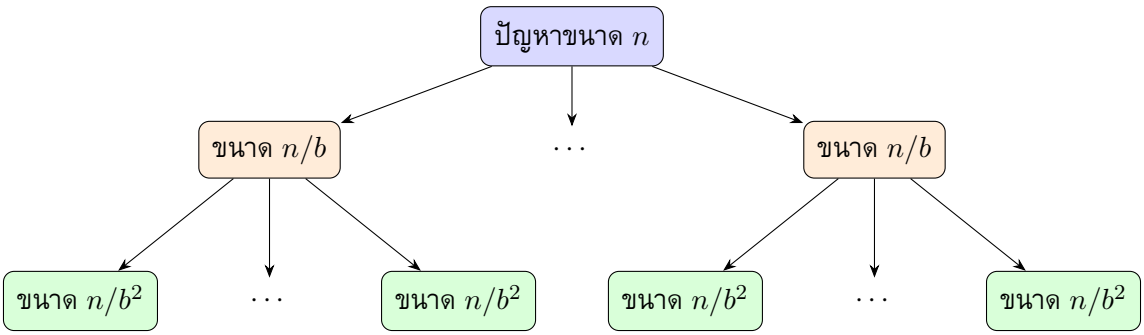
```
DivideAndConquer(P) {
    if (baseCase(P))
        return solution(P);   // Base case
    Divide P into subproblems P1, P2, ..., Pk
    return Combine(
        DivideAndConquer(P1),
        DivideAndConquer(P2),
        ...,
        DivideAndConquer(Pk)
    );
}
```

ตัวอย่างโค้ดที่ 4.1: รูปแบบทั่วไปของ Divide and Conquer

Base Case สำคัญมาก!

ทุกขั้นตอนวิธีแบบ Recursive ต้องมี **Base Case** หรือเงื่อนไขหยุด มิฉะนั้นจะเกิด Infinite Recursion ทำให้โปรแกรมค้างหรือ Stack Overflow ใน Divide and Conquer โดยทั่วไป Base Case คือเมื่อปัญหามีขนาดเล็กพอที่จะแก้ได้ทันที เช่น แถวลำดับมีสมาชิก 0 หรือ 1 ตัว

รูปที่ 4.1 แสดงภาพรวมของกระบวนการ Divide and Conquer โดยปัญหาขนาด n จะถูกแบ่งเป็นปัญหาย่อยขนาด n/b แต่ละปัญหาถูกแก้แยกกัน แล้วคำตอบย่อยจะถูกรวมกลับมาเป็นคำตอบของปัญหาเดิม



รูปที่ 4.1 ภาพรวมกระบวนการ Divide and Conquer: ปัญหาขนาด n ถูกแบ่งเป็นปัญหาย่อยขนาดเล็กลงเรื่อยๆ

ตัวอย่างที่เห็นได้ชัดที่สุดของ Divide and Conquer คือ **Binary Search** (การค้นหาแบบทวิภาค) ซึ่งเราได้เรียนไปแล้วในบทที่ 4 ในการค้นหาแบบทวิภาค เราแบ่งช่วงค้นหาเป็นครึ่งซ้ายและครึ่งขวา ตรวจสอบตรงกลางว่าเป็นค่าที่ต้องการหรือไม่ แล้วเลือกครึ่งที่เป็นไปได้เพียงครึ่งเดียวค้นหาต่อ การพิจารณาปัญหาที่มีขนาดลดลงครึ่งหนึ่งในแต่ละขั้นทำให้ได้ความซับซ้อน $\mathcal{O}(\log n)$ อย่างไรก็ตาม บางคนจัด Binary Search ว่าเป็น *Decrease and Conquer* มากกว่า เพราะแบ่งแล้วใช้เพียงปัญหาย่อยเดียว ไม่ได้ใช้ทั้งสองปัญหาย่อย

4.2 Merge Sort

Merge Sort เป็นขั้นตอนวิธีการเรียงลำดับที่ใช้หลักการ Divide and Conquer อย่างชัดเจน และเป็นหนึ่งในขั้นตอนวิธีที่มีประสิทธิภาพสูงสุดในแง่ของความซับซ้อนเชิงเวลา เนื่องจากทำงานในเวลา $\mathcal{O}(n \log n)$ ในทุกกรณี ไม่ว่าข้อมูลจะมีลักษณะใด[2, 6]

4.2.1 หลักการทำงาน

Merge Sort ทำงานตามขั้นตอนของ Divide and Conquer ดังนี้:

1. **แบ่ง (Divide):** แบ่งแถวลำดับออกเป็น 2 ส่วนเท่าๆ กัน ตรงจุดกึ่งกลาง

2. **เอาชนะ (Conquer):** เรียงลำดับส่วนซ้ายและส่วนขวาแยกกันโดยการเรียก Merge Sort ซ้ำ (Recursive) จนกระทั่งเหลือสมาชิกเพียง 0 หรือ 1 ตัว (ซึ่งถือว่าเรียงแล้วโดยปริยาย)

3. **รวม (Combine):** รวมแถวลำดับที่เรียงแล้ว 2 ส่วนกลับมาเป็นแถวลำดับเดียวที่เรียงแล้ว ขั้นตอนนี้เรียกว่า **Merge**

4.2.2 ขั้นตอน Merge

ก่อนจะเข้าใจ Merge Sort ทั้งหมด เราต้องเข้าใจขั้นตอน Merge ก่อน สมมติว่าเรามีแถวลำดับสองส่วนที่เรียงแล้ว คือ A (ส่วนซ้าย) และ B (ส่วนขวา) และต้องการรวมเป็นแถวลำดับเดียวที่เรียงแล้ว วิธีการคือเปรียบเทียบสมาชิกตัวหน้าสุดของแต่ละแถวลำดับ ตัวไหนน้อยกว่าก็เอามาใส่แถวลำดับผลลัพธ์ก่อน ทำเช่นนี้จนกว่าสมาชิกทุกตัวจะถูกนำไปใส่หมด

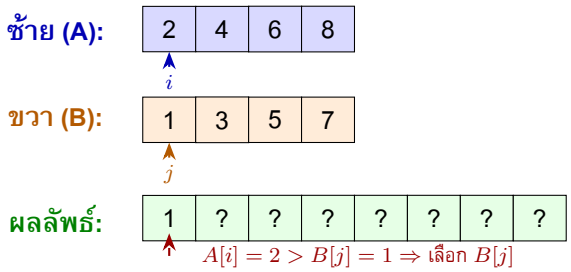
```
void merge(int a[], int lo, int mid, int hi) {
    // Create temporary array
    vector<int> temp;
    int i = lo, j = mid + 1;
    while (i <= mid && j <= hi) {
        if (a[i] <= a[j])
            temp.push_back(a[i++]);
        else
            temp.push_back(a[j++]);
    }
    // Copy remaining elements from left part
    while (i <= mid)
        temp.push_back(a[i++]);
    // Copy remaining elements from right part
    while (j <= hi)
        temp.push_back(a[j++]);
    // Copy back to original array
    for (int k = 0; k < (int)temp.size(); k++)
        a[lo + k] = temp[k];
}
```

ตัวอย่างโค้ดที่ 4.2: ฟังก์ชัน Merge

รูปที่ 4.2 แสดงขั้นตอน Merge ด้วยสองตัวชี้ (Two-Pointer) เปรียบเทียบสมาชิกตัวหน้าของส่วนซ้ายและส่วนขวา ตัวไหนน้อยกว่าก็นำมาใส่แถวลำดับผลลัพธ์ก่อน

สังเกตว่าขั้นตอน Merge ใช้เวลา $\mathcal{O}(n)$ เพราะสมาชิกทุกตัวถูกพิจารณาครบทั้งหมดเพียงครั้งเดียว และต้องใช้พื้นที่เพิ่มเติม $\mathcal{O}(n)$ สำหรับแถวลำดับชั่วคราว

4.2.3 โค้ด Merge Sort



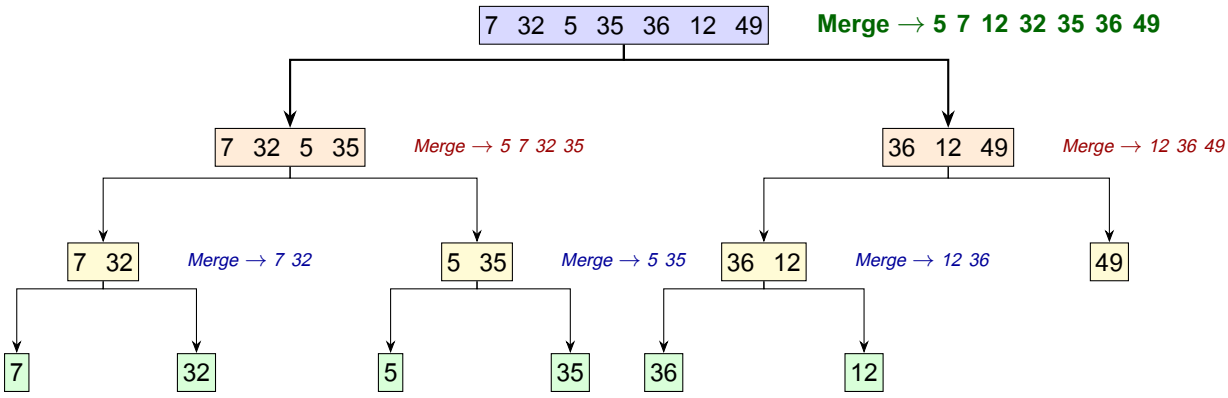
รูปที่ 4.2 ขั้นตอน Merge: เปรียบเทียบสมาชิกตัวหน้าของส่วนซ้ายและขวา ตัวไหนน้อยกว่านำมาใส่ผลลัพธ์ก่อน

```
void mergeSort(int a[], int lo, int hi) {
    if (lo >= hi) return;   // Base case: 0 or 1 element
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);          // Sort left part
    mergeSort(a, mid + 1, hi);      // Sort right part
    merge(a, lo, mid, hi);          // Merge two parts
}
```

ตัวอย่างโค้ดที่ 4.3: Merge Sort

เนื่องจาก Merge Sort เป็น Recursive มันจะเรียกส่วนซ้ายให้เสร็จก่อน แล้วจึงเรียกส่วนขวา และสุดท้ายจึง Merge รวม ลำดับการทำงานจึงเป็น: เจาะลึกส่วนซ้าย ซ้าย ซ้าย จนเหลือ 1 ตัว → เรียกส่วนขวา → Merge → กลับขึ้นมาชั้นบน → เรียกส่วนขวาของชั้นนั้น → Merge → ไล่ขึ้นไปเรื่อยๆ จนครบ

รูปที่ 4.3 แสดงขั้นตอนการทำงานของ Merge Sort แบบภาพ โดยเริ่มจากการแบ่งแถวลำดับครึ่งซ้าย-ขวาซ้ำๆ จนเหลือ 1 ตัว แล้วค่อยๆ Merge รวมกลับขึ้นไป



รูปที่ 4.3 ขั้นตอนการทำงานของ Merge Sort: แบ่งครึ่งซ้ำๆ จนเหลือ 1 ตัว แล้ว Merge รวมกลับ

4.2.4 ตัวอย่างการทำงานของ Merge Sort

สมมติเราต้องการเรียงแถวลำดับ $[7, 32, 5, 35, 36, 12, 49]$ ขั้นตอนจะเป็นดังนี้:

1. แบ่งเป็นสองส่วน: $[7, 32, 5, 35]$ และ $[36, 12, 49]$
2. ส่วนซ้าย $[7, 32, 5, 35]$ แบ่งเป็น $[7, 32]$ และ $[5, 35]$
3. $[7, 32]$ แบ่งเป็น $[7]$ และ $[32]$ $\rightarrow$ Merge ได้ $[7, 32]$
4. $[5, 35]$ แบ่งเป็น $[5]$ และ $[35]$ $\rightarrow$ Merge ได้ $[5, 35]$
5. Merge $[7, 32]$ และ $[5, 35]$ ได้ $[5, 7, 32, 35]$
6. ส่วนขวา $[36, 12, 49]$ แบ่งเป็น $[36, 12]$ และ $[49]$
7. $[36, 12]$ แบ่งเป็น $[36]$ และ $[12]$ $\rightarrow$ Merge ได้ $[12, 36]$
8. Merge $[12, 36]$ และ $[49]$ ได้ $[12, 36, 49]$
9. สุดท้าย Merge $[5, 7, 32, 35]$ และ $[12, 36, 49]$ ได้ $[5, 7, 12, 32, 35, 36, 49]$

4.2.5 การวิเคราะห์ความซับซ้อน

เวลาที่ Merge Sort ใช้สามารถเขียนเป็น Recurrence Relation ได้ดังนี้:

$$T(n) = \begin{cases} 1 & \text{ถ้า } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{ถ้า } n > 1 \end{cases}$$

คำอธิบาย: เราแบ่งปัญหาเป็น 2 ปัญหาย่อยขนาด $n/2$ แต่ละอันใช้เวลา $T(n/2)$ และขั้นตอน Merge ใช้เวลา $\mathcal{O}(n)$ การแก้ Recurrence นี้ (โดยใช้ Master Theorem หรือ Recursion Tree) จะได้ $T(n) = \mathcal{O}(n \log n)$

ตารางที่ 4.1 สรุปคุณสมบัติของ Merge Sort

คุณสมบัติ	รายละเอียด
ความซับซ้อนเชิงเวลา (ทุกกรณี)	$\mathcal{O}(n \log n)$
ความซับซ้อนเชิงพื้นที่	$\mathcal{O}(n)$ (แถวลำดับชั่วคราว)
เสถียรภาพ (Stable)	ใช่ — สมาชิกที่มีค่าเท่ากันจะรักษาลำดับเดิม
ข้อดี	ประสิทธิภาพคงที่ $\mathcal{O}(n \log n)$ ในทุกกรณี
ข้อเสีย	ต้องใช้หน่วยความจำเพิ่ม $\mathcal{O}(n)$

Merge Sort ในการแข่งขัน

แม้ Merge Sort จะมีประสิทธิภาพคงที่ $\mathcal{O}(n \log n)$ ในทุกกรณี แต่ในการแข่งขันจริง ควรใช้ `sort()` จากไลบรารีมาตรฐานของ C++ เป็นหลัก เพราะเร็วกว่าในทางปฏิบัติ (ค่าคงที่ต่ำกว่า) และเขียนได้ในบรรทัดเดียว อย่างไรก็ตาม การเข้าใจ Merge Sort มีความสำคัญเพราะเป็นพื้นฐานของการนับ Inversion (ดูหัวข้อถัดไป) และเป็นแนวคิดที่ใช้ในหลายขั้นตอนวิธีขั้นสูง

4.3 Quick Sort

Quick Sort เป็นขั้นตอนวิธีการเรียงลำดับอีกตัวหนึ่งที่ใช้หลักการ Divide and Conquer แต่ต่างจาก Merge Sort ตรงที่ขั้นตอน “แบ่ง” ของ Quick Sort ทำงานหนักกว่า ส่วนขั้นตอน “รวม” แทบไม่ต้องทำอะไรเลย[2, 6]

4.3.1 หลักการทำงาน

Quick Sort ทำงานตามขั้นตอนดังนี้:

1. **แบ่ง (Partition):** เลือกสมาชิกตัวหนึ่งจากแถวลำดับเป็น **Pivot** (สมมติให้เป็น p) แล้วจัดเรียงสมาชิกทั้งหมดให้:
   - สมาชิกที่น้อยกว่า p อยู่ทางซ้าย
   - Pivot p อยู่ตรงกลาง (ตำแหน่งที่ถูกต้องในแถวลำดับที่เรียงแล้ว)
   - สมาชิกที่มากกว่าหรือเท่ากับ p อยู่ทางขวา
2. **เอาชนะ (Conquer):** เรียก Quick Sort ซ้ำกับส่วนซ้ายและส่วนขวาแยกกัน
3. **รวม (Combine):** ไม่ต้องทำอะไร เพราะหลังจาก Partition แล้ว Pivot อยู่ตำแหน่งที่ถูกต้องแล้ว และส่วนซ้าย-ขวาก็ถูกเรียงโดย Recursive Call

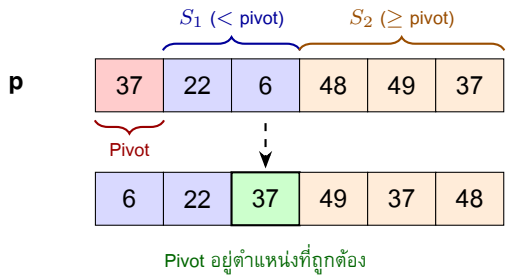
4.3.2 ฟังก์ชัน Partition

หัวใจสำคัญของ Quick Sort คือฟังก์ชัน **Partition** ซึ่งทำหน้าที่จัดเรียงสมาชิกตาม Pivot วิธีที่นิยมใช้คือ **Lomuto Partition Scheme** โดยเลือกสมาชิกตัวแรกของแถวลำดับเป็น Pivot

```
int partition(int a[], int lo, int hi) {
    int pivot = a[lo];    // Choose first element as pivot
    int m = lo;           // m is the last boundary of group < pivot
    for (int k = lo + 1; k <= hi; k++) {
        if (a[k] < pivot) {
            m++;
            swap(a[m], a[k]);
        }
    }
    swap(a[lo], a[m]);    // Move pivot to correct position
    return m;             // Return pivot position
}
```

ตัวอย่างโค้ดที่ 4.4: ฟังก์ชัน Partition (Lomuto Scheme)

รูปที่ 4.4 แสดงโครงสร้างของแถวลำดับระหว่างทำ Partition โดยมีกลุ่ม S_1 (สมาชิกที่น้อยกว่า Pivot), กลุ่ม S_2 (สมาชิกที่มากกว่าหรือเท่ากับ Pivot) และส่วน Unknown ที่ยังไม่ได้ตรวจสอบ



รูปที่ 4.4 ขั้นตอน Partition: แบ่งแถวลำดับเป็นกลุ่ม S_1 (น้อยกว่า Pivot) และ S_2 (มากกว่าหรือเท่า Pivot) แล้วสลับ Pivot ไปตำแหน่งที่ถูกต้อง

ตัวอย่างการทำงานของ Partition

สมมติแถวลำดับ $= [37, 48, 22, 49, 37, 6]$ เลือก pivot $= 37$

- เริ่มต้น: $m = 0$ กลุ่ม S_1 (น้อยกว่า pivot) ยังว่าง กลุ่ม S_2 (มากกว่าหรือเท่า pivot) ยังว่าง
- $a[1] = 48 \geq 37 \rightarrow$ เข้ากลุ่ม S_2: $S_1 = \{\}$, $S_2 = \{48\}$
- $a[2] = 22 < 37 \rightarrow$ เข้ากลุ่ม S_1: สลับ $a[1]$ กับ $a[2]$, $S_1 = \{22\}$, $S_2 = \{48\}$
- $a[3] = 49 \geq 37 \rightarrow$ เข้ากลุ่ม S_2: $S_1 = \{22\}$, $S_2 = \{48, 49\}$
- $a[4] = 37 \geq 37 \rightarrow$ เข้ากลุ่ม S_2: $S_1 = \{22\}$, $S_2 = \{48, 49, 37\}$
- $a[5] = 6 < 37 \rightarrow$ เข้ากลุ่ม S_1: สลับ $a[2]$ กับ $a[5]$, $S_1 = \{22, 6\}$, $S_2 = \{49, 37, 48\}$
- สุดท้าย: สลับ pivot ($a[0]$) กับ $a[m]$: ได้ $[6, 22, 37, 49, 37, 48]$
- Pivot 37 อยู่ที่ตำแหน่ง 2 (ถูกต้อง!) ส่วนซ้าย $[6, 22]$ น้อยกว่า 37 ส่วนขวา $[49, 37, 48]$ มากกว่าหรือเท่ากับ 37

4.3.3 โค้ด Quick Sort

```
void quickSort(int a[], int lo, int hi) {
    if (lo < hi) {
        int m = partition(a, lo, hi);
        quickSort(a, lo, m - 1);    // Sort left part
        quickSort(a, m + 1, hi);    // Sort right part
    }
}
```

ตัวอย่างโค้ดที่ 4.5: Quick Sort

4.3.4 การวิเคราะห์ความซับซ้อน

- **กรณีที่ดีที่สุด / กรณีเฉลี่ย:** $\mathcal{O}(n \log n)$ — เมื่อ Partition แบ่งแถวลำดับออกเป็นสองส่วนที่ใกล้เคียงกันทุกครั้ง ความซับซ้อนจะเหมือน Merge Sort
- **กรณีที่เลวร้ายที่สุด:** $\mathcal{O}(n^2)$ — เกิดขึ้นเมื่อ Pivot ที่เลือกมาเป็นสมาชิกที่มากที่สุดหรือน้อยที่สุดทุกครั้ง ทำให้แบ่งได้ส่วนว่างกับส่วนที่มีขนาด $n-1$ เช่น แถวลำดับที่เรียงแล้วและเลือกตัวแรกเป็น Pivot เสมอ

วิธีหลีกเลี่ยงกรณีเลวร้ายของ Quick Sort

ในทางปฏิบัติ การเลือก Pivot แบบสุ่ม (Randomized Pivot) หรือเลือก Median-of-Three (ค่ามัธยฐานจาก 3 ตัว: ตัวแรก ตัวกลาง ตัวสุดท้าย) ช่วยลดโอกาสเจอกรณีเลวร้ายได้อย่างมีนัยสำคัญ ทำให้ Quick Sort ทำงานในเวลา $\mathcal{O}(n \log n)$ ในทางปฏิบัติ นี่คือเหตุผลที่ `std::sort` ของ C++ ใช้ Introsort ซึ่งผสม Quick Sort เข้ากับ Heap Sort เพื่อรับประกัน $\mathcal{O}(n \log n)$ ในทุกกรณี

ตารางที่ 4.2 เปรียบเทียบ Merge Sort และ Quick Sort

คุณสมบัติ	Merge Sort	Quick Sort	หมายเหตุ
เวลา (เฉลี่ย)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	เท่ากัน
เวลา (เลวร้าย)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	Merge Sort ดีกว่า
พื้นที่เพิ่ม	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	Quick Sort ประหยัดกว่า
Stable	ใช่	ไม่ใช่	Merge Sort รักษาลำดับเดิม
In-place	ไม่ใช่	ใช่ (เกือบ)	Quick Sort ใช้หน่วยความจำน้อย

4.4 การนับ Inversion ด้วย Merge Sort

Inversion คือคู่อันดับ (i, j) ที่ $i < j$ แต่ $a[i] > a[j]$ กล่าวคือ สมาชิกที่อยู่ข้างหน้ามีค่ามากกว่าสมาชิกที่อยู่ข้างหลัง จำนวน Inversion บ่งบอกถึงความ “ไม่เรียง” ของแถวลำดับ — แถวลำดับที่เรียงแล้วมี Inversion เท่ากับ 0 ส่วนแถวลำดับที่เรียงย้อนกลับมี Inversion มากที่สุดคือ $\frac{n(n-1)}{2}$

ปัญหาการนับ Inversion เป็นโจทย์คลาสสิกที่สามารถแก้ได้อย่างมีประสิทธิภาพด้วยการดัดแปลง Merge Sort[2] เหตุผลก็คือ ในขั้นตอน Merge เมื่อเราเปรียบเทียบสมาชิกของส่วนซ้ายกับส่วนขวา ถ้าสมาชิกของส่วนขวาน้อยกว่า แสดงว่าสมาชิกนั้นสร้าง Inversion กับสมาชิกที่เหลืออยู่ทั้งหมดในส่วนซ้าย

```
long long mergeCount(int a[], int lo, int mid, int hi) {
    vector<int> temp;
    long long cnt = 0;
    int i = lo, j = mid + 1;
    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) {
```

```
            temp.push_back(a[i++]);
        } else {
            // a[j] is less than all a[i]..a[mid]
            // So there are (mid - i + 1) inversions
            cnt += (mid - i + 1);
            temp.push_back(a[j++]);
        }
    }
    while (i <= mid) temp.push_back(a[i++]);
    while (j <= hi) temp.push_back(a[j++]);
    for (int k = 0; k < (int)temp.size(); k++)
        a[lo + k] = temp[k];
    return cnt;
}
long long countInversion(int a[], int lo, int hi) {
    if (lo >= hi) return 0;
    int mid = (lo + hi) / 2;
    long long cnt = 0;
    cnt += countInversion(a, lo, mid);
    cnt += countInversion(a, mid + 1, hi);
    cnt += mergeCount(a, lo, mid, hi);
    return cnt;
}
```

ตัวอย่างโค้ดที่ 4.6: นับ Inversion ด้วย Merge Sort

ตัวอย่าง: นับ Inversion

ให้แถวลำดับ $a = [2, 4, 1, 3, 5]$ หาจำนวน Inversion
Inversion ทั้งหมดคือ: $(2, 1), (4, 1), (4, 3)$ รวม 3 คู่
เมื่อรันขั้นตอนวิธีข้างต้นจะได้คำตอบ 3 ซึ่งตรงกัน

สังเกตว่าจำนวน Inversion อาจมีค่ามากถึง $\frac{n(n-1)}{2}$ ซึ่งสำหรับ $n = 10^5$ จะมีค่าประมาณ 5×10^9 ดังนั้นต้องใช้ `long long` เก็บค่า

4.5 การเข้ารหัส Bitmap (Bitmap Encoder)

ในหัวข้อนี้ เราจะนำหลักการ Divide and Conquer มาใช้แก้ปัญหา **Bitmap Encoder** ซึ่งเป็นโจทย์ที่แสดงให้เห็นว่า Divide and Conquer สามารถใช้กับข้อมูลสองมิติได้ ไม่ใช่แค่แถวลำดับหนึ่งมิติ

4.5.1 โจทย์

มีตารางขนาด $N \times M$ ที่ภายในมีเพียง 0 หรือ 1 เท่านั้น ต้องการเข้ารหัสตารางนี้ด้วยกฎดังนี้:

- ถ้าทุกช่องในตารางมีค่าเหมือนกันหมด (เป็น 1 หมดหรือ 0 หมด) จะเข้ารหัสเป็น `1` หรือ `0` ตามค่านั้น
- ถ้าไม่ใช่ (มีทั้ง 0 และ 1 ปนกัน) จะเข้ารหัสด้วยอักษร `D` แล้วแบ่งตารางเป็น 4 ส่วน: บนซ้าย บนขวา ล่างซ้าย ล่างขวา แล้วเข้ารหัสแต่ละส่วนตามเงื่อนไขเดียวกันนี้แบบ Recursive

4.5.2 ตัวอย่าง

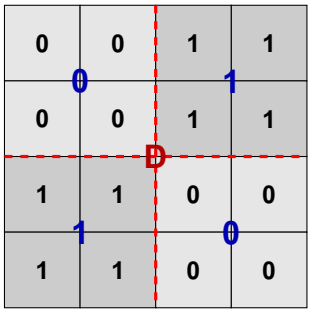
สมมติตารางขนาด 4×4:

$$\begin{matrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{matrix}$$

การเข้ารหัส:

1. ตารางทั้งหมดมีทั้ง 0 และ 1 $\rightarrow$ เข้ารหัส `D` แล้วแบ่งเป็น 4 ส่วน
2. บนซ้าย: 0 หมด $\rightarrow$ `0`
3. บนขวา: 1 หมด $\rightarrow$ `1`
4. ล่างซ้าย: 1 หมด $\rightarrow$ `1`
5. ล่างขวา: 0 หมด $\rightarrow$ `0`

รูปที่ 4.5 แสดงภาพประกอบการแบ่งตาราง Bitmap Encoder ออกเป็น 4 ส่วน



ผลลัพธ์: `D0110`

รูปที่ 4.5 การเข้ารหัส Bitmap: ตาราง 4×4 ถูกแบ่งเป็น 4 ส่วน แต่ละส่วนมีค่าเหมือนกันหมด จึงเข้ารหัสได้เป็น `D0110`

ผลลัพธ์: `D0110`

4.5.3 แนวคิดการแก้ปัญหา

ปัญหานี้เป็น Divide and Conquer อย่างชัดเจน:

1. **ตรวจสอบ:** ตรวจสอบทั้งตารางว่าทุกช่องเหมือนกันหรือไม่ ถ้าใช่ คืน `0` หรือ `1` ถ้าไม่ใช่ ใส่ `D` แล้วแบ่ง
2. **แบ่ง:** แบ่งตารางเป็น 4 ส่วนโดยใช้มุมบนซ้าย (i_1, j_1) และมุมล่างขวา (i_2, j_2) กำหนดจุดแบ่ง:

$$mi = \frac{i_2 - i_1 + 1}{2}, \quad mj = \frac{j_2 - j_1 + 1}{2}$$

เรียก Recursive สำหรับ 4 ส่วน:

- บนซ้าย: `encode`$(i_1,\ i_1 + mi - 1,\ j_1,\ j_1 + mj - 1)$
- บนขวา: `encode`$(i_1,\ i_1 + mi - 1,\ j_1 + mj,\ j_2)$
- ล่างซ้าย: `encode`$(i_1 + mi,\ i_2,\ j_1,\ j_1 + mj - 1)$
- ล่างขวา: `encode`$(i_1 + mi,\ i_2,\ j_1 + mj,\ j_2)$

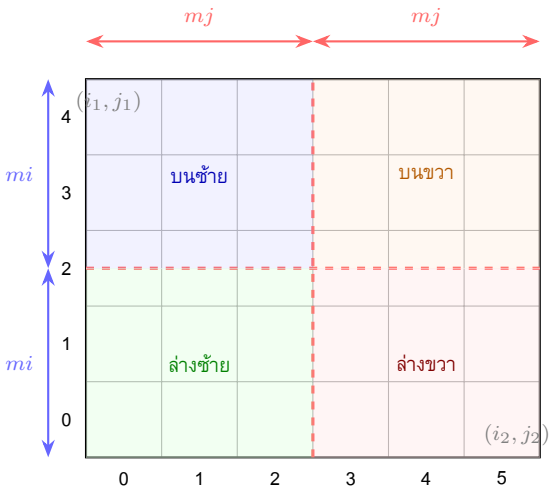
4.5.4 โค้ด Bitmap Encoder

```cpp
#include <bits/stdc++.h>
using namespace std;
int table[105][105];
string encode(int i1, int i2, int j1, int j2) {
    // Check if all cells have the same value
    bool same = true;
    int val = table[i1][j1];
    for (int i = i1; i <= i2 && same; i++)
        for (int j = j1; j <= j2 && same; j++)
            if (table[i][j] != val)
                same = false;
    if (same)
        return string(1, '0' + val);
    // Divide into 4 quadrants
    int mi = (i2 - i1 + 1) / 2;
    int mj = (j2 - j1 + 1) / 2;
    string ans = "D";
    ans += encode(i1, i1 + mi - 1, j1, j1 + mj - 1);         // Top-left
    ans += encode(i1, i1 + mi - 1, j1 + mj, j2);             // Top-right
    ans += encode(i1 + mi, i2, j1, j1 + mj - 1);             // Bottom-left
    ans += encode(i1 + mi, i2, j1 + mj, j2);                 // Bottom-right
```

```cpp
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cin >> table[i][j];
    cout << encode(0, n - 1, 0, m - 1) << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 4.7: Bitmap Encoder

รูปที่ 4.6 แสดงการแบ่งตารางออกเป็น 4 ส่วนด้วยพิกัดมุมบนซ้ายและมุมล่างขวา



รูปที่ 4.6 การแบ่งตาราง Bitmap เป็น 4 ส่วนด้วยจุดแบ่ง mi และ mj: บนซ้าย, บนขวา, ล่างซ้าย, ล่างขวา

ระวังการแบ่งส่วน

เมื่อแบ่งตารางเป็น 4 ส่วน ต้องระวังเรื่องขอบเขตของแต่ละส่วนให้ถูกต้อง โดยเฉพาะเมื่อขนาดตารางเป็นเลขคี่ นอกจากนี้ ต้องแน่ใจว่าทุกช่องถูกครอบคลุมครบ ไม่มีช่องซ้ำหรือหาย ในตัวอย่างข้างต้นเรากำหนดให้ขนาดตารางเป็น $2^k \times 2^k$ เสมอ เพื่อให้แบ่งได้พอดี

4.6 Master Theorem

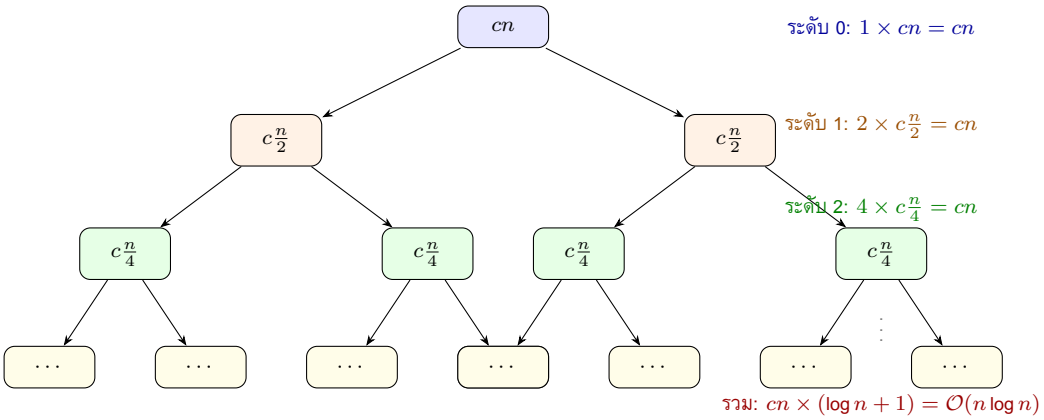
Master Theorem เป็นเครื่องมือสำคัญสำหรับวิเคราะห์ความซับซ้อนของขั้นตอนวิธี Divide and Conquer โดยให้สูตรสำเร็จรูปสำหรับ Recurrence Relation ในรูปแบบ:[2]

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + \mathcal{O}(n^d)$$

โดยที่:

- $a \geq 1$ คือจำนวนปัญหาย่อย
- $b > 1$ คืออัตราส่วนการลดขนาดปัญหา
- $d \geq 0$ คือเลขชี้กำลังของเวลาที่ใช้ในขั้นตอน Divide และ Combine

รูปที่ 4.7 แสดงต้นไม้เรียกซ้ำ (Recursion Tree) สำหรับ $T(n) = 2T(n/2)+\mathcal{O}(n)$ ซึ่งเป็น Recurrence ของ Merge Sort สังเกตว่าแต่ละระดับใช้เวลารวมเป็น cn เสมอ และมี $\log_2 n$ ระดับ ทำให้เวลารวมเป็น $\mathcal{O}(n \log n)$



รูปที่ 4.7 ต้นไม้เรียกซ้ำสำหรับ $T(n) = 2T(n/2) + \mathcal{O}(n)$: แต่ละระดับใช้เวลา cn รวม $\log n + 1$ ระดับ

ผลลัพธ์ของ Master Theorem แบ่งเป็น 3 กรณี:

ตารางที่ 4.3 Master Theorem — 3 กรณี

เงื่อนไข	ผลลัพธ์	ความหมาย
$a < b^d$	$T(n) = \mathcal{O}(n^d)$	ขั้นตอน Combine หนักกว่า Subproblems
$a = b^d$	$T(n) = \mathcal{O}(n^d \log n)$	สมดุลระหว่าง Combine กับ Subproblems
$a > b^d$	$T(n) = \mathcal{O}(n^{\log_b a})$	Subproblems หนักกว่าขั้นตอน Combine

4.6.1 ตัวอย่างการใช้ Master Theorem

ตัวอย่างที่ 1: Merge Sort

Merge Sort: $T(n) = 2T(n/2) + \mathcal{O}(n)$
วิเคราะห์: $a = 2, b = 2, d = 1 \rightarrow a = b^d$ $(2 = 2^1) \rightarrow$ กรณี 2
ผลลัพธ์: $T(n) = \mathcal{O}(n \log n)$

ตัวอย่างที่ 2: Binary Search

Binary Search: $T(n) = 1 \cdot T(n/2) + \mathcal{O}(1)$
วิเคราะห์: $a = 1, b = 2, d = 0 \rightarrow a = b^d$ $(1 = 2^0) \rightarrow$ กรณี 2
ผลลัพธ์: $T(n) = \mathcal{O}(n^0 \log n) = \mathcal{O}(\log n)$

ตัวอย่างที่ 3: Strassen's Matrix Multiplication

$T(n) = 7T(n/2) + \mathcal{O}(n^2)$
วิเคราะห์: $a = 7, b = 2, d = 2 \rightarrow a > b^d$ $(7 > 4) \rightarrow$ กรณี 3
ผลลัพธ์: $T(n) = \mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.81})$ ซึ่งดีกว่าการคูณเมทริกซ์แบบธรรมดา $\mathcal{O}(n^3)$

ตัวอย่างที่ 4: Bitmap Encoder

Bitmap Encoder: $T(n) = 4T(n/2) + \mathcal{O}(n^2)$ (แบ่งเป็น 4 ส่วน ขนาดลดลงครึ่งหนึ่ง การตรวจสอบใช้เวลา $\mathcal{O}(n^2)$)
วิเคราะห์: $a = 4, b = 2, d = 2 \rightarrow a = b^d$ $(4 = 2^2) \rightarrow$ กรณี 2
ผลลัพธ์: $T(n) = \mathcal{O}(n^2 \log n)$

4.7 Closest Pair of Points

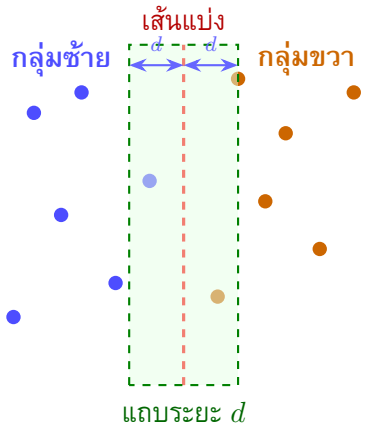
Closest Pair of Points เป็นโจทย์คลาสสิกอีกข้อหนึ่งของ Divide and Conquer ที่พบบ่อยในการแข่งขัน โจทย์กำหนดจุด n จุดบนระนาบสองมิติ และถามหาคู่จุดที่มีระยะห่างน้อยที่สุด[2, 7]

วิธีแก้แบบ Brute Force ตรวจสอบทุกคู่ใช้เวลา $\mathcal{O}(n^2)$ แต่ด้วย Divide and Conquer สามารถลดลงเป็น $\mathcal{O}(n \log n)$ ได้

รูปที่ 4.8 แสดงแนวคิดหลักของขั้นตอนวิธี Closest Pair โดยแบ่งจุดออกเป็นสองกลุ่มด้วยเส้นแนวตั้ง แล้วพิจารณาเฉพาะจุดที่อยู่ในแถบระยะ d จากเส้นแบ่ง

4.7.1 แนวคิด

1. **เตรียมข้อมูล:** เรียงจุดตามพิกัด x
2. **แบ่ง:** แบ่งจุดออกเป็นสองกลุ่มที่ขนาดใกล้เคียงกันโดยใช้เส้นแนวตั้งตรงกลาง



รูปที่ 4.8 แนวคิด Closest Pair: แบ่งจุดด้วยเส้นแนวตั้ง หาระยะน้อยสุดแต่ละฝั่ง แล้วตรวจสอบเฉพาะจุดในแถบระยะ d

3. **เอาชนะ:** หาระยะห่างน้อยสุดในกลุ่มซ้าย (d_L) และกลุ่มขวา (d_R) แยกกัน ให้ $d = \min(d_L, d_R)$

4. **รวม:** ตรวจสอบคู่จุดที่อยู่ข้ามเส้นแบ่ง — พิจารณาเฉพาะจุดที่อยู่ในแถบระยะ d จากเส้นแบ่ง และตรวจสอบเฉพาะจุดที่อยู่ในกรอบสี่เหลี่ยมขนาด $d \times 2d$ (พิสูจน์ได้ว่าในกรอบนี้มีจุดได้ไม่เกิน 6 จุด) ทำให้ขั้นตอนนี้ใช้เวลา $\mathcal{O}(n)$

4.7.2 โค้ด Closest Pair of Points

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Point {
    double x, y;
};
bool cmpX(const Point& a, const Point& b) { return a.x < b.x; }
bool cmpY(const Point& a, const Point& b) { return a.y < b.y; }
double dist(const Point& a, const Point& b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}
double bruteForce(vector<Point>& pts, int lo, int hi) {
    double mn = 1e18;
    for (int i = lo; i <= hi; i++)
        for (int j = i + 1; j <= hi; j++)
            mn = min(mn, dist(pts[i], pts[j]));
    return mn;
}
double closestUtil(vector<Point>& pts, int lo, int hi) {
    if (hi - lo + 1 <= 3)
        return bruteForce(pts, lo, hi);
```

```
    int mid = (lo + hi) / 2;
    double midX = pts[mid].x;
    double dL = closestUtil(pts, lo, mid);
    double dR = closestUtil(pts, mid + 1, hi);
    double d = min(dL, dR);
    // Collect points within distance d from dividing line
    vector<Point> strip;
    for (int i = lo; i <= hi; i++)
        if (abs(pts[i].x - midX) < d)
            strip.push_back(pts[i]);
    sort(strip.begin(), strip.end(), cmpY);
    // Check points across the dividing line
    for (int i = 0; i < (int)strip.size(); i++)
        for (int j = i + 1; j < (int)strip.size()
            && strip[j].y - strip[i].y < d; j++)
            d = min(d, dist(strip[i], strip[j]));
    return d;
}
double closest(vector<Point>& pts) {
    sort(pts.begin(), pts.end(), cmpX);
    return closestUtil(pts, 0, pts.size() - 1);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<Point> pts(n);
    for (int i = 0; i < n; i++)
        cin >> pts[i].x >> pts[i].y;
    cout << fixed << setprecision(6) << closest(pts) << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 4.8: Closest Pair of Points

ความซับซ้อนเชิงเวลาของขั้นตอนวิธีนี้คือ $\mathcal{O}(n \log n)$ สำหรับการเรียงลำดับเริ่มต้น และ $\mathcal{O}(n \log^2 n)$ สำหรับส่วน Recursive (เพราะต้องเรียง `strip` ในแต่ละระดับ) อย่างไรก็ตาม หากเราเตรียมจุดที่เรียงตาม y ไว้ล่วงหน้า สามารถลดลงเป็น $\mathcal{O}(n \log n)$ ได้

สำคัญ: ขั้นตอน Strip

กุญแจสำคัญที่ทำให้ Closest Pair ทำงานใน $\mathcal{O}(n \log n)$ คือการพิสูจน์ว่าในแถบระยะ d จากเส้นแบ่ง จุดที่อยู่ในกรอบสี่เหลี่ยมขนาด $d \times 2d$ มีได้ไม่เกิน 6 จุด (เพราะถ้ามีมากกว่านั้น จะขัดกับที่ d เป็นระยะน้อยสุดในแต่ละฝั่ง) ดังนั้นการวนซ้ำภายในจึงวนไม่เกิน 6 ครั้ง ทำให้ขั้นตอน Strip ใช้เวลา $\mathcal{O}(n)$

4.8 การยกกำลังแบบเร็ว (Fast Exponentiation)

Fast Exponentiation หรือที่เรียกว่า **Binary Exponentiation** เป็นขั้นตอนวิธีที่ใช้หลักการ Divide and Conquer ในการคำนวณ a^n (หรือ $a^n \bmod m$) อย่างมีประสิทธิภาพ แทนที่จะคูณ a ซ้ำๆ n ครั้ง เราสามารถลดจำนวนการคูณลงอย่างมากด้วยการแบ่งครึ่งเลขชี้กำลังในแต่ละขั้น[3, 11]

4.8.1 แนวคิด

ปัญหา: คำนวณ a^n โดยที่ a เป็นจำนวนเต็มและ n เป็นเลขชี้กำลังจำนวนเต็มไม่เป็นลบ

วิธี Naive: คูณ a ด้วยตัวมันเอง n ครั้ง ใช้เวลา $\mathcal{O}(n)$

วิธี Divide and Conquer: สังเกตว่าเราสามารถเขียน a^n ในรูปแบบที่ลดขนาดปัญหาลงครึ่งหนึ่งได้:

- ถ้า n เป็นเลขคู่: $a^n = (a^{n/2})^2$ — คำนวณ $a^{n/2}$ ก่อน แล้วจึงยกกำลังสอง
- ถ้า n เป็นเลขคี่: $a^n = a \times a^{n-1}$ — ลด n ลง 1 ทำให้กลายเป็นเลขคู่
- Base case: $a^0 = 1$

แต่ละขั้น n จะลดลงครึ่งหนึ่ง (หรือลดลง 1 แล้วลดครึ่ง) ทำให้จำนวนขั้นตอนทั้งหมดเป็น $\mathcal{O}(\log n)$ เท่านั้น!

4.8.2 การวิเคราะห์ความซับซ้อน

Recurrence Relation ของ Fast Exponentiation คือ:

$$T(n) = T(n/2) + \mathcal{O}(1) \implies T(n) = \mathcal{O}(\log n)$$

เปรียบเทียบกับวิธี Naive ที่ใช้เวลา $\mathcal{O}(n)$ ถ้า $n = 10^9$ วิธี Naive ต้องคูณพันล้านครั้ง แต่ Fast Exponentiation ใช้เพียงประมาณ 30 ขั้นตอน!

4.8.3 การพัฒนาแบบ Recursive

```
long long fastPow(long long a, long long n) {
    if (n == 0) return 1;              // Base case
    if (n % 2 == 0) {
        long long half = fastPow(a, n / 2);
```

```
        return half * half;
    }
    return a * fastPow(a, n - 1);       // n is odd
}
```

ตัวอย่างโค้ดที่ 4.9: Fast Exponentiation แบบ Recursive

4.8.4 การพัฒนาแบบ Iterative

แม้ว่าเวอร์ชัน Recursive จะเข้าใจง่าย แต่ในการแข่งขันควรใช้เวอร์ชัน Iterative มากกว่า เพราะหลีกเลี่ยงปัญหา Stack Overflow สำหรับเลขชี้กำลังขนาดใหญ่ และทำงานเร็วกว่าเล็กน้อย

หลักการของเวอร์ชัน Iterative คือมองเลขชี้กำลัง n ในรูปเลขฐานสอง ถ้าบิตที่ i ของ n เป็น 1 ให้คูณผลลัพธ์ด้วย a^{2^i}

```
long long fastPow(long long a, long long n) {
    long long ans = 1;
    while (n > 0) {
        if (n & 1)              // Current bit is 1
            ans *= a;
        a *= a;                 // Square the base
        n >>= 1;                // Shift to next bit
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 4.10: Fast Exponentiation แบบ Iterative

4.8.5 Modular Exponentiation

ในการแข่งขัน การใช้งานที่พบบ่อยที่สุดของ Fast Exponentiation คือการคำนวณ $a^n \bmod m$ ซึ่งเรียกว่า **Modular Exponentiation** โดยใช้สมบัติ:

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

นั่นคือ เราสามารถ mod ทุกครั้งที่คูณ เพื่อไม่ให้ค่ากลางโตเกินไป

```
long long modPow(long long a, long long n, long long m) {
    long long ans = 1;
    a %= m;                             // In case a >= m
    while (n > 0) {
        if (n & 1)
            ans = (ans * a) % m;
        a = (a * a) % m;
```

```
        n >>= 1;
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 4.11: Modular Exponentiation

ตัวอย่าง: Fast Exponentiation

คำนวณ 2^{10}:

- $2^{10} = (2^5)^2 = 32^2 = 1024$
- $2^{10} \bmod 1000 = 1024 \bmod 1000 = 24$

4.8.6 ตัวอย่างการ Trace: 3^{13}

มาดูการทำงานของเวอร์ชัน Iterative กับ 3^{13} โดย 13 ในฐานสองคือ 1101_2

ขั้น	n (ฐานสอง)	n & 1	result	base
เริ่มต้น	1101	—	1	3
ขั้น 1	1101	1	$1 \times 3 = 3$	$3^2 = 9$
ขั้น 2	110	0	3	$9^2 = 81$
ขั้น 3	11	1	$3 \times 81 = 243$	$81^2 = 6561$
ขั้น 4	1	1	$243 \times 6561 = 1594323$	6561^2

ผลลัพธ์: $3^{13} = 1594323$ ☒

สังเกตว่าเราคูณผลลัพธ์เฉพาะเมื่อบิตของ n เป็น 1 และ base จะยกกำลังสองทุกขั้น ทำให้ได้ base = $3, 3^2, 3^4, 3^8, \ldots$ ตามลำดับ

ใช้เวอร์ชัน Iterative ในการแข่งขัน

ในการแข่งขัน ควรใช้เวอร์ชัน Iterative เป็นหลัก เพราะหลีกเลี่ยงปัญหา Stack Overflow สำหรับเลขชี้กำลังขนาดใหญ่ และทำงานเร็วกว่าเวอร์ชัน Recursive เล็กน้อย เวอร์ชัน Recursive มีประโยชน์สำหรับทำความเข้าใจแนวคิดเบื้องต้น

Modular Exponentiation กับทฤษฎีจำนวน

Modular Exponentiation เป็นเครื่องมือพื้นฐานที่จำเป็นมากสำหรับโจทย์ทฤษฎีจำนวน เช่น การตรวจจำนวนเฉพาะ (Primality Testing), ขั้นตอนวิธี RSA, และการหา Inverse แบบ Modular นอกจากนี้ยังเป็นพื้นฐานของ **Matrix Exponentiation** ซึ่งใช้คำนวณ M^k สำหรับเมทริกซ์ เพื่อแก้ปัญหา Linear Recurrence ในเวลา $\mathcal{O}(k^3 \log n)$ เมื่อ k คือขนาดเมทริกซ์

ระวัง Overflow!

เมื่อคำนวณ $(a \times b) \bmod m$ โดยที่ a, b มีค่าถึง 10^9 ผลคูณ $a \times b$ อาจมีค่าถึง 10^{18} ซึ่ง `int` (32-bit) เก็บไม่ได้ ต้องใช้ `long long` (64-bit) สำหรับตัวแปรที่เก็บผลคูณก่อนจะ `mod` มิฉะนั้นจะเกิด Overflow ทำให้คำตอบผิด

4.9 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนเทคนิค Divide and Conquer:

โจทย์แนะนำสำหรับ Divide and Conquer

Merge Sort & Counting Inversion:

- CSES Inversion Counting — นับ Inversion
- SPOJ INVCNT — Inversion Count
- Codeforces 1608C — Market Manipulation (Inversion)[10]

Divide and Conquer ทั่วไป:

- CSES Nested Ranges Count — D&C + Coordinate Compression
- Codeforces 1430E — String Reversal (Inversion)[10]
- UVA 10245 — The Closest Pair Problem (Closest Pair)[20]
- Codeforces 61E — Enemy is Weak (3-element inversion)[10]

Fast Exponentiation:

- CSES Bit Strings — การยกกำลังแบบมอดูลาร์
- Codeforces 1182B — Plus from Picture (Matrix Exponentiation concept)[10]

4.10 สรุปบทที่ 4

Divide and Conquer เป็นรูปแบบการแก้ปัญหาที่ทรงพลัง โดยมีหลักการสำคัญ 3 ขั้นตอน: แบ่ง เอาชนะ และรวม ในบทนี้ เราได้ศึกษาขั้นตอนวิธีและโจทย์สำคัญดังนี้:

ตารางที่ 4.4 สรุปขั้นตอนวิธี Divide and Conquer ที่ศึกษา

ขั้นตอนวิธี	เวลา	พื้นที่	หมายเหตุ
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Stable, ประสิทธิภาพคงที่
Quick Sort	$\mathcal{O}(n \log n)^*$	$\mathcal{O}(\log n)$	In-place, *เลวร้าย $\mathcal{O}(n^2)$
Count Inversion	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	ดัดแปลงจาก Merge Sort
Bitmap Encoder	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^2)$	D&C สองมิติ (รวมขนาดของผลลัพธ์)
Closest Pair	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	เรขาคณิตเชิงคำนวณ

เทคนิคสำคัญในการแก้โจทย์ D&C

1. **ระบุ Base Case ให้ชัดเจน** — ปัญหาขนาดเล็กเท่าใดจึงจะแก้โดยตรง?
2. **คิดให้ออกว่าแบ่งอย่างไร** — ครึ่งซ้าย-ขวา? 4 ส่วน? หรือแบบอื่น?
3. **ขั้นตอนรวมคือหัวใจ** — หลายโจทย์ D&C ความยากอยู่ที่ขั้นตอน Combine มากกว่า Divide
4. **ใช้ Master Theorem ช่วยวิเคราะห์** — ช่วยยืนยันความซับซ้อนได้อย่างรวดเร็ว
5. **ระวังขอบเขต** — การแบ่งส่วนผิดพลาดเพียงเล็กน้อยอาจทำให้ได้คำตอบผิดได้

2

โครงสร้างข้อมูลขั้นสูง

ส่วนนี้นำเสนอโครงสร้างข้อมูลทั้งแบบพื้นฐานและขั้นสูงที่ใช้ในการแก้ปัญหา ตั้งแต่ Array, Stack, Queue, Set, Map ไปจนถึง Segment Tree และ Fenwick Tree รวมถึงเทคนิคการสอบถามแบบช่วง (Range Queries) ที่จำเป็นสำหรับการแก้ปัญหาที่มีการอัปเดตและสอบถามค่าในช่วง นอกจากนี้ยังครอบคลุมกลยุทธ์การค้นหาแบบครบถ้วน (Complete Search) ที่เป็นพื้นฐานสำคัญในการแก้โจทย์หลายประเภท โดยเฉพาะเมื่อขนาดข้อมูลนำเข้ามีข้อจำกัดไม่ใหญ่เกินไป

Chapter 5

โครงสร้างข้อมูล

ในการโปรแกรมเชิงแข่งขัน โครงสร้างข้อมูล (Data Structure) เป็นเครื่องมือพื้นฐานที่ขาดไม่ได้[2, 3] เช่นเดียวกับการเลือกเครื่องมือที่เหมาะสมในการก่อสร้าง การเลือกโครงสร้างข้อมูลที่ถูกต้องจะทำให้โปรแกรมทำงานได้รวดเร็วและถูกต้อง ในขณะที่การเลือกผิดอาจทำให้โปรแกรมช้าเกินไปหรือใช้หน่วยความจำเกินจำกัด ในบทนี้ เราจะศึกษาโครงสร้างข้อมูลที่ใช้บ่อยที่สุดในการแข่งขัน ตั้งแต่แถวลำดับและ Vector ไปจนถึง Set, Map, Priority Queue และอื่นๆ อีกมากมาย โดยเน้นที่การใช้งานผ่าน C++ STL ซึ่งเป็นไลบรารีมาตรฐานที่พร้อมใช้งานทันที

5.1 แนะนำโครงสร้างข้อมูล

โครงสร้างข้อมูลเป็นหัวข้อที่ครอบคลุมและสำคัญมากในวิทยาการคอมพิวเตอร์ ก่อนที่เราจะศึกษาโครงสร้างข้อมูลแต่ละชนิดอย่างละเอียด เรามาทำความเข้าใจพื้นฐานกันก่อนว่าโครงสร้างข้อมูลคืออะไร และทำไมการเลือกโครงสร้างข้อมูลที่เหมาะสมจึงมีความสำคัญอย่างยิ่งต่อการแก้โจทย์ในการแข่งขัน

5.1.1 โครงสร้างข้อมูลคืออะไร

โครงสร้างข้อมูลคือ **วิธีการจัดเก็บและจัดระเบียบข้อมูล**ในหน่วยความจำของคอมพิวเตอร์ เพื่อให้สามารถเข้าถึง และดำเนินการกับข้อมูลได้อย่างมีประสิทธิภาพ โครงสร้างข้อมูลแต่ละชนิดถูกออกแบบมาให้มีประสิทธิภาพกับการดำเนินการบางอย่าง แต่อาจทำงานได้ช้ากับการดำเนินการอื่นๆ

ยกตัวอย่างเช่น แถวลำดับ (Array) สามารถเข้าถึงสมาชิกด้วยดัชนีได้รวดเร็วในเวลา $\mathcal{O}(1)$ แต่การแทรกหรือลบสมาชิกตรงกลางแถวลำดับต้องใช้เวลา $\mathcal{O}(n)$ ในทางกลับกัน Linked List สามารถแทรกหรือลบสมาชิกได้เร็วในเวลา $\mathcal{O}(1)$ แต่การเข้าถึงสมาชิกด้วยดัชนีต้องใช้เวลา $\mathcal{O}(n)$

คำถามสำคัญ

เมื่อเลือกโครงสร้างข้อมูล ให้ถามตัวเองว่า: **โจทย์ต้องการการดำเนินการใดเป็นหลัก?** ถ้าต้องการเข้าถึงด้วยดัชนีบ่อยๆ ให้ใช้แถวลำดับ ถ้าต้องการค้นหา แทรก และลบบ่อยๆ ให้ใช้ Set หรือ Map การเลือกโครงสร้างข้อมูลที่เหมาะสมกับการดำเนินการที่โจทย์ต้องการคือกุญแจสำคัญ

5.1.2 การเลือกโครงสร้างข้อมูลให้เหมาะกับโจทย์

โครงสร้างข้อมูลที่แตกต่างกันมีจุดแข็งและจุดอ่อนที่แตกต่างกัน การเลือกโครงสร้างข้อมูลที่เหมาะสมขึ้นอยู่กับลักษณะของโจทย์ โดยเฉพาะการดำเนินการที่โจทย์ต้องการทำบ่อยๆ

ตัวอย่างเช่น สมมติว่าเราต้องเก็บข้อมูลแบบ key-value (คีย์–ค่า) เราสามารถเลือกได้ 2 วิธีหลัก:

1. **ใช้ array of struct:** เหมาะเมื่อ key เป็นจำนวนเต็มที่ต่อเนื่องและขนาดไม่ใหญ่เกินไป เข้าถึงได้ในเวลา $\mathcal{O}(1)$ แต่สิ้นเปลืองหน่วยความจำหาก key กระจายอยู่ในช่วงกว้าง

2. **ใช้ map:** เหมาะเมื่อ key เป็นประเภทใดก็ได้ (ไม่จำเป็นต้องเป็นจำนวนเต็มต่อเนื่อง) เข้าถึงได้ในเวลา $\mathcal{O}(\log n)$ ประหยัดหน่วยความจำเมื่อ key กระจายอยู่ในช่วงกว้าง

ดังนั้นก่อนจะเริ่มเขียนโค้ด ควรวิเคราะห์ก่อนว่าโจทย์ต้องการการดำเนินการอะไรบ้าง และแต่ละการดำเนินการต้องทำกี่ครั้ง แล้วจึงเลือกโครงสร้างข้อมูลที่รองรับการดำเนินการเหล่านั้นได้อย่างมีประสิทธิภาพ

5.1.3 ตารางสรุปภาพรวม

ตารางต่อไปนี้สรุปโครงสร้างข้อมูลหลักที่จะศึกษาในบทนี้ พร้อมการดำเนินการสำคัญและความซับซ้อนของเวลา

ตารางที่ 5.1 ภาพรวมโครงสร้างข้อมูลในบทที่ 5

โครงสร้าง	หัวข้อ	การดำเนินการสำคัญ
`vector`	Dynamic Array	เข้าถึงด้วยดัชนี $\mathcal{O}(1)$, เพิ่ม/ลบท้าย $\mathcal{O}(1)$
`string`	String	ต่อข้อความ $\mathcal{O}(n)$, `substr` $\mathcal{O}(x)$, `find` $\mathcal{O}(n)$
`bitset`	Bitset	เข้าถึงบิต $\mathcal{O}(1)$, ดำเนินการบิต $\mathcal{O}(n/64)$
`stack`	Stack	`push/pop/top` $\mathcal{O}(1)$
`queue`	Queue	`push/pop/front` $\mathcal{O}(1)$
`deque`	Deque	`push/pop` ทั้ง 2 ด้าน $\mathcal{O}(1)$
`set`	Balanced BST	`insert/erase/count` $\mathcal{O}(\log n)$, `lower_bound` $\mathcal{O}(\log n)$
`map`	Balanced BST	`insert/erase/count` $\mathcal{O}(\log n)$, เข้าถึงด้วย `[]` $\mathcal{O}(\log n)$
`priority_queue`	Heap	`push` $\mathcal{O}(\log n)$, `pop/top` $\mathcal{O}(\log n)$

5.2 แถวลำดับและ Vector

แถวลำดับเป็นโครงสร้างข้อมูลพื้นฐานที่สุดที่ใช้จัดเก็บข้อมูลหลายค่าในตัวแปรเดียว ใน C++ เราสามารถใช้แถวลำดับแบบคงที่ (Static Array) หรือแถวลำดับแบบไดนามิก (Dynamic Array) ซึ่งใน STL คือ `vector`[3] ทั้งสองชนิดมีข้อดีและข้อจำกัดที่แตกต่างกัน และการเลือกใช้ให้เหมาะสมจะช่วยให้เขียนโปรแกรมได้อย่างมีประสิทธิภาพ

5.2.1 Static Array

แถวลำดับแบบคงที่คือแถวลำดับที่มีขนาดคงที่ตั้งแต่ตอนคอมไพล์ ไม่สามารถเปลี่ยนขนาดได้ระหว่างโปรแกรมทำงาน ข้อดีคือเข้าถึงสมาชิกด้วยดัชนีได้รวดเร็วในเวลา $\mathcal{O}(1)$ และไม่มีค่าใช้จ่ายเพิ่มเติมจากการจัดการหน่วยความจำ

ในการแข่งขัน เรามักประกาศแถวลำดับด้วยขนาดสูงสุดที่โจทย์กำหนด **บวกเผื่อ** (Safety Margin) อีกเล็กน้อย เพื่อป้องกันปัญหาการเข้าถึงตำแหน่งที่เกินขอบเขต (Out of Bounds):

```cpp
// 1D array: declare with max size + margin
int arr[100005];          // If problem says n <= 100000
// 2D array
int mat[105][105];        // For matrix up to 100x100
// 3D array
int cube[25][25][25]; // For 3D data
// Access element by index: O(1)
arr[0] = 42;
int x = arr[0];
// Caution: index starts from 0
// Array of size N uses indices 0 to N-1 only
```

ตัวอย่างโค้ดที่ 5.1: การประกาศแถวลำดับแบบคงที่

ข้อจำกัดของ Static Array

- ขนาดต้องเป็นค่าคงที่ที่ตอนคอมไพล์ — ไม่สามารถกำหนดจากตัวแปรได้ (ในมาตรฐาน C++ เดิม)
- แถวลำดับที่ประกาศในฟังก์ชันจะมีค่าเริ่มต้นเป็นค่าขยะ (Garbage Value) — ต้องกำหนดค่าเริ่มต้นเอง
- แถวลำดับขนาดใหญ่เกินไปที่ประกาศในฟังก์ชันอาจทำให้เกิด Stack Overflow — ควรประกาศเป็น Global หรือใช้ `vector`

5.2.2 Dynamic Array (vector)

`vector` เป็นแถวลำดับแบบไดนามิก (Dynamic Array) ที่สามารถเปลี่ยนขนาดได้ระหว่างโปรแกรมทำงาน เป็นโครงสร้างข้อมูลที่นักแข่งขันใช้บ่อยที่สุด เพราะมีความยืดหยุ่นสูง `vector` จัดการหน่วยความจำให้อัตโนมัติ เมื่อเพิ่มสมาชิกจนเกินความจุปัจจุบัน มันจะจัดสรรพื้นที่ใหม่ที่ใหญ่ขึ้น (ปกติเป็น 2 เท่า) และย้ายข้อมูลเดิมไป ทำให้การเพิ่มสมาชิกท้ายแถวลำดับมีความซับซ้อนเฉลี่ย (Amortized) เป็น $\mathcal{O}(1)$

เมื่อไหร่ควรใช้ vector

ใช้ `vector` เมื่อ:

- ไม่ทราบขนาดของข้อมูลล่วงหน้าตอนคอมไพล์
- ต้องการเพิ่มหรือลบสมาชิกที่ท้ายแถวลำดับบ่อยๆ
- ต้องการใช้ฟังก์ชันอำนวยความสะดวก เช่น `sort`, `lower_bound` ผ่าน iterator

- ต้องการส่งแถวลำดับเป็นพารามิเตอร์ของฟังก์ชันโดยไม่ต้องส่งขนาดแยก

5.2.3 การประกาศและใช้งาน vector

vector สามารถประกาศได้หลายรูปแบบ ขึ้นอยู่กับลักษณะการใช้งาน:

```cpp
#include <vector>
using namespace std;
// Declare empty vector
vector<int> v1;
// Declare with initial values (C++11)
vector<int> v2 = {1, 2, 3, 4, 5};
// Declare vector of size 10 (default value 0)
vector<int> v3(10);
// Declare vector of size 10 with default value 5
vector<int> v4(10, 5);
// Declare 2D vector (5 rows x 10 columns, default 0)
vector<vector<int>> v5(5, vector<int>(10));
// Vector of strings
vector<string> names = {"Alice", "Bob", "Charlie"};
// Vector of pairs
vector<pair<int,int>> edges;
```

ตัวอย่างโค้ดที่ 5.2: การประกาศ vector ใน C++

ระวัง vector ว่าง

เมื่อประกาศ `vector<int> v1;` โดยไม่ระบุขนาด `v1` จะเป็น vector ว่างที่มีขนาด 0 การเข้าถึง `v1[0]` จะเป็นพฤติกรรมที่ไม่นิยาม (Undefined Behavior) ต้องเพิ่มสมาชิกก่อนจึงจะเข้าถึงได้

5.2.4 push_back, pop_back, back, front

ฟังก์ชันเหล่านี้เป็นวิธีหลักในการเพิ่ม ลบ และเข้าถึงสมาชิกที่ปลายทั้งสองของ vector:

```cpp
vector<int> v;
// push_back(): Add element at end -- O(1) amortized
v.push_back(10);    // v = {10}
v.push_back(20);    // v = {10, 20}
v.push_back(30);    // v = {10, 20, 30}
// back(): Return last element -- O(1)
int last = v.back();    // last = 30
// front(): Return first element -- O(1)
int first = v.front(); // first = 10
```

```
// pop_back(): Remove last element -- O(1)
v.pop_back();          // v = {10, 20}
// Access by index -- O(1)
int x = v[0];          // x = 10
int y = v[1];          // y = 20
// at(): Access by index + bounds checking
int z = v.at(0);       // z = 10 (throws exception if out of bounds)
```

ตัวอย่างโค้ดที่ 5.3: ใช้ push_back, pop_back, back, front กับ vector

เทคนิคในการแข่งขัน

ในการแข่งขัน ใช้ `v[i]` แทน `v.at(i)` เสมอ เพราะ `v.at(i)` มีค่าใช้จ่ายเพิ่มเติมจากการตรวจสอบขอบเขต ทำให้ช้ากว่าเล็กน้อย แม้ว่า `v[i]` จะไม่ตรวจสอบขอบเขตก็ตาม แต่ในการแข่งขันเรามักควบคุมดัชนีเองได้อยู่แล้ว

5.2.5 size, empty, clear, resize

ฟังก์ชันอำนวยความสะดวกเหล่านี้ช่วยให้จัดการ vector ได้สะดวกยิ่งขึ้น:

```
vector<int> v = {10, 20, 30};
// size(): Return current number of elements
int n = v.size();     // n = 3
// empty(): Check if vector is empty
bool is_empty = v.empty();   // is_empty = false
// clear(): Remove all elements
v.clear();            // v = {}  (size is 0)
// resize(): Change vector size
v.resize(5);          // v = {0, 0, 0, 0, 0}
v.resize(3);          // v = {0, 0, 0} (truncated last 2 elements)
v.resize(6, 99);      // v = {0, 0, 0, 99, 99, 99} (new elements are 99)
// Loop with range-based for (C++11)
vector<int> w = {1, 2, 3, 4, 5};
for (int x : w) {
    cout << x << " ";     // Prints: 1 2 3 4 5
}
```

ตัวอย่างโค้ดที่ 5.4: ฟังก์ชัน size, empty, clear, resize

ชนิดของ size()

`v.size()` คืน ค่า ประเภท `size_t` ซึ่ง เป็น จำนวนเต็ม ไม่มี เครื่องหมาย (Unsigned) ดัง นั้น `v.size() - 1` จะเกิด Underflow ถ้า v ว่าง ควรระวังโดยใช้ `v.empty()` ตรวจสอบก่อน หรือใช้

`(int)v.size() - 1` เพื่อแปลงเป็นจำนวนเต็มมีเครื่องหมาย

5.2.6 การใช้ lower_bound/binary_search กับ vector

เราสามารถใช้ Binary Search กับ vector ได้โดยใช้ฟังก์ชัน `lower_bound` และ `binary_search` จาก `<algorithm>` แต่ **ข้อควรระวังที่สำคัญคือ vector ต้องถูกเรียงลำดับแล้ว** จึงจะใช้ฟังก์ชันเหล่านี้ได้อย่างถูกต้อง:

```cpp
#include <algorithm>
vector<int> v = {5, 2, 8, 1, 9, 3};
// Must sort first!
sort(v.begin(), v.end());   // v = {1, 2, 3, 5, 8, 9}
// lower_bound: iterator to first element >= x
auto it = lower_bound(v.begin(), v.end(), 5);
if (it != v.end()) {
    cout << *it << "\n";            // Prints: 5
    cout << it - v.begin() << "\n"; // Prints: 3 (index)
}
// Find index of lower_bound directly
int idx = lower_bound(v.begin(), v.end(), 4) - v.begin();
// idx = 3 (first value >= 4 is 5 at index 3)
// binary_search: Check if value x exists in vector
bool found = binary_search(v.begin(), v.end(), 5);   // true
bool not_found = binary_search(v.begin(), v.end(), 7); // false
// upper_bound: iterator to first element > x
auto it2 = upper_bound(v.begin(), v.end(), 5);
// it2 points to 8 (first value greater than 5)
// Count elements equal to x
int count_x = upper_bound(v.begin(), v.end(), 5)
            - lower_bound(v.begin(), v.end(), 5);
```

ตัวอย่างโค้ดที่ 5.5: ใช้ lower_bound และ binary_search กับ vector

ตัวอย่าง: หาสมาชิกที่ใกล้ที่สุด

```cpp
// Find element in vector closest to value x
int closest(vector<int>& v, int x) {
    auto it = lower_bound(v.begin(), v.end(), x);
    if (it == v.begin()) return *it;
    if (it == v.end()) return *(it - 1);
    // Compare distance of two closest values
    if (x - *(it - 1) <= *it - x)
```

```
        return *(it - 1);
    else
        return *it;
}
```

5.3 String และ Iterator

String เป็นโครงสร้างข้อมูลที่ใช้จัดเก็บข้อความ ใน C++ `string` เป็น Dynamic Array ของอักขระ (Character) ที่มีฟังก์ชันอำนวยความสะดวกมากมาย ส่วน Iterator เป็นแนวคิดสำคัญที่ช่วยให้เราใช้ฟังก์ชันต่างๆ ของ STL กับโครงสร้างข้อมูลได้อย่างเป็นระบบ

5.3.1 String: การประกาศและใช้งาน

ใน C++ `string` สามารถประกาศและใช้งานได้หลายรูปแบบ:

```cpp
#include <string>
// Declare empty string
string s1;
// Declare with value
string s2 = "Hello";
// Declare from C-string
const char* cstr = "World";
string s3(cstr);      // s3 = "World"
// Declare string with repeated character
string s4(10, 'a');   // s4 = "aaaaaaaaaa"
// Declare string from substring
string s5 = s2.substr(0, 3);   // s5 = "Hel"
// String length
int len = s2.size();   // len = 5
int len2 = s2.length(); // len2 = 5 (same as size())
// Access character by index
char c = s2[0];        // c = 'H'
char d = s2[1];        // d = 'e'
```

ตัวอย่างโค้ดที่ 5.6: การประกาศและใช้งาน string

5.3.2 ปฏิบัติการ String

`string` มีฟังก์ชันสำหรับจัดการข้อความที่ใช้บ่อยในการแข่งขัน:

```cpp
string a = "Hello";
string b = "World";
// Concatenate strings with +
string c = a + " " + b;     // c = "Hello World"
// Compare strings with ==, <, >
if (a == "Hello") { /* true */ }
if (a < b) { /* lexicographic comparison */ }
// substr(k, x): Extract substring starting at position k, length x
string s = "ABCDEFGHIJ";
string sub1 = s.substr(2, 3);   // sub1 = "CDE" (starts at position 2, length 3)
string sub2 = s.substr(5);      // sub2 = "FGHIJ" (from position 5 to end)
// find(t): Find first position of substring t (returns string::npos if not found)
string text = "Hello World Hello";
size_t pos1 = text.find("World");    // pos1 = 6
size_t pos2 = text.find("xyz");      // pos2 = string::npos
size_t pos3 = text.find("Hello", 1); // pos3 = 12 (search starting from position 1)
// Add/remove characters
s.push_back('K');       // Add at end
s.pop_back();           // Remove at end
```

ตัวอย่างโค้ดที่ 5.7: ฟังก์ชันจัดการ string

substr อาจทำให้ช้า

`substr` สร้าง string ใหม่ ซึ่งใช้เวลา $\mathcal{O}(x)$ เมื่อ x คือความยาวของข้อความย่อย หากเรียก `substr` หลายครั้งในลูป อาจทำให้โปรแกรมช้าได้ ในบางกรณี อาจใช้การเข้าถึงด้วยดัชนีแทนการตัดข้อความย่อยได้

5.3.3 Iterator และ Range

Iterator เป็นวัตถุที่ชี้ไปยังสมาชิกของโครงสร้างข้อมูล ทำหน้าที่เหมือน “ตัวชี้” ที่สามารถเลื่อนไปมาได้ ฟังก์ชัน `begin()` คืน iterator ที่ชี้ไปยังสมาชิกตัวแรก ส่วน `end()` คืน iterator ที่ชี้ไป **ผ่าน** สมาชิกตัวสุดท้ายหนึ่งตำแหน่ง (Past-the-end) ช่วง `[begin(), end())` จึงเป็นช่วงครึ่งเปิด (Half-Open Range) ที่ครอบคลุมสมาชิกทั้งหมด

```cpp
vector<int> v = {10, 20, 30, 40, 50};
// Loop using iterator
for (auto it = v.begin(); it != v.end(); it++) {
    cout << *it << " ";     // Prints: 10 20 30 40 50
}
// range-based for (shorter, easier to write)
for (int x : v) {
```

```
    cout << x << " ";
}
// auto keyword shortens iterator type
auto it = v.begin();   // Instead of vector<int>::iterator it = v.begin();
cout << *it;           // 10 (dereference value with *)
it++;                  // Move to next element
cout << *it;           // 20
```

ตัวอย่างโค้ดที่ 5.8: ใช้ Iterator กับ vector

Iterator คือกุญแจสู่ STL

ฟังก์ชันหลายตัวของ STL รับ iterator เป็นพารามิเตอร์ เช่น `sort(v.begin(), v.end())`, `reverse(v.begin(), v.end())`, `lower_bound(v.begin(), v.end(), x)` เป็นต้น การเข้าใจ iterator จะช่วยให้ใช้งานฟังก์ชันเหล่านี้ได้อย่างคล่องแคล่ว

5.3.4 sort, reverse, random_shuffle ผ่าน iterator

เราสามารถใช้ฟังก์ชันจาก `<algorithm>` ผ่าน iterator เพื่อจัดการข้อมูลใน vector และแถวลำดับ:

```
#include <algorithm>
// === vector ===
vector<int> v = {5, 2, 8, 1, 9, 3};
// sort: ascending order
sort(v.begin(), v.end());          // v = {1, 2, 3, 5, 8, 9}
// sort: descending order
sort(v.begin(), v.end(), greater<int>()); // v = {9, 8, 5, 3, 2, 1}
// Partial sort (positions 1 to 4)
sort(v.begin() + 1, v.begin() + 5);
// reverse: reverse order
reverse(v.begin(), v.end());       // Reverse all elements
// === C-style array ===
int arr[] = {5, 2, 8, 1, 9, 3};
int n = 6;
// Sort array: use pointer as iterator
sort(arr, arr + n);                // arr = {1, 2, 3, 5, 8, 9}
// Reverse array
reverse(arr, arr + n);             // arr = {9, 8, 5, 3, 2, 1}
// Partial sort
sort(arr + 1, arr + 4);            // Sort positions 1 to 3
```

ตัวอย่างโค้ดที่ 5.9: sort, reverse ผ่าน iterator

ตัวอย่าง: เรียงลำดับด้วยเงื่อนไขกำหนดเอง

```cpp
// Sort pair by second first, then by first
vector<pair<int,int>> v = {{3,1}, {1,2}, {2,1}};
sort(v.begin(), v.end(), [](const pair<int,int>& a,
                            const pair<int,int>& b) {
    if (a.second != b.second) return a.second < b.second;
    return a.first < b.first;
});
// v = {{2,1}, {3,1}, {1,2}}
```

5.4 Bitset

Bitset เป็นโครงสร้างข้อมูลที่จัดเก็บข้อมูลแบบบิต โดยแต่ละตำแหน่งเก็บค่าได้เพียง 0 หรือ 1 เท่านั้น แต่ใช้หน่วยความจำเพียง 1 บิตต่อตำแหน่ง ซึ่งประหยัดกว่าการใช้ `int` (32 บิต) หรือ `bool` (8 บิต) อย่างมาก Bitset มีประโยชน์อย่างยิ่งในโจทย์ที่ต้องจัดการกับข้อมูลไบนารีจำนวนมาก

5.4.1 Bitset คืออะไร

`bitset` คือแถวลำดับที่แต่ละสมาชิกมีค่าเป็น 0 หรือ 1 โดยใช้หน่วยความจำเพียง 1 บิตต่อสมาชิก เปรียบเทียบกับ `bool` ที่ใช้ 8 บิต (1 ไบต์) ต่อสมาชิก และ `int` ที่ใช้ 32 บิตต่อสมาชิก ดังนั้น `bitset<1000>` ใช้หน่วยความจำเพียงประมาณ 125 ไบต์ ในขณะที่ `bool[1000]` ใช้ 1,000 ไบต์ และ `int[1000]` ใช้ 4,000 ไบต์

ข้อสำคัญ: ขนาดของ `bitset` ต้องเป็นค่าคงที่ที่ตอนคอมไพล์ (เช่นเดียวกับแถวลำดับแบบคงที่) ไม่สามารถกำหนดจากตัวแปรได้

5.4.2 การใช้งาน Bitset

```cpp
#include <bitset>
// Declare bitset of 10 bits (default all 0)
bitset<10> s;
// Set bit values
s[1] = 1;      // Set bit 1 to 1
s[3] = 1;      // Set bit 3 to 1
s[5] = 1;      // Set bit 5 to 1
// s = 0000010101 (bits 1, 3, 5 are 1)
// Create bitset from string
bitset<10> b(string("0010011010"));
// Bits 1, 3, 4, 7 are 1
// Create bitset from integer
```

```cpp
bitset<8> c(42);   // 42 = 00101010 in binary
// count(): Count number of 1 bits
int ones = s.count();   // ones = 3
// Check individual bit positions
if (s[3]) { /* bit 3 is 1 */ }
// test(): Check bit (like [] but with bounds checking)
if (s.test(5)) { /* bit 5 is 1 */ }
// set() and reset()
s.set(7);         // Set bit 7 to 1
s.reset(3);       // Set bit 3 to 0
s.set();          // Set all bits to 1
s.reset();        // Set all bits to 0
s.flip();         // Flip all bits (0->1, 1->0)
```

ตัวอย่างโค้ดที่ 5.10: การใช้งาน bitset

5.4.3 การดำเนินการบิต

ข้อดีอย่างหนึ่งของ `bitset` คือรองรับการดำเนินการบิต (Bitwise Operations) โดยตรง ซึ่งทำงานได้รวดเร็วมากเพราะดำเนินการกับหลายบิตพร้อมกัน (ปกติ 64 บิตต่อการดำเนินการ):

```cpp
bitset<8> a(string("10101010"));   // 170
bitset<8> b(string("11001100"));   // 204
// AND (&): result bit is 1 when both bits are 1
bitset<8> and_result = a & b;       // 10001000 (136)
// OR (|): result bit is 1 when at least one bit is 1
bitset<8> or_result = a | b;        // 11101110 (238)
// XOR (^): result bit is 1 when the two bits differ
bitset<8> xor_result = a ^ b;       // 01100110 (102)
// NOT (~): Flip all bits
bitset<8> not_result = ~a;          // 01010101 (85)
// Left shift (<<) and Right shift (>>)
bitset<8> left = a << 2;            // 10101000 (shifted left 2 positions)
bitset<8> right = a >> 1;           // 01010101 (shifted right 1 position)
```

ตัวอย่างโค้ดที่ 5.11: การดำเนินการบิตกับ bitset

ใช้ bitset เพิ่มประสิทธิภาพขั้นตอนวิธี

การดำเนินการบิตกับ `bitset<N>` ใช้เวลา $\mathcal{O}(N/64)$ ซึ่งเร็วกว่าการวนลูปทีละบิต ($\mathcal{O}(N)$) ถึง 64 เท่า! เทคนิคนี้มีประโยชน์มากในโจทย์ที่ต้องจัดการกับข้อมูลไบนารีจำนวนมาก เช่น Subset Sum หรือ BFS บนกราฟขนาดใหญ่

5.4.4 ตัวอย่าง: การประหยัดหน่วยความจำ

เปรียบเทียบการใช้หน่วยความจำระหว่างแถวลำดับ `int` กับ `bitset`:

ตารางที่ 5.2 เปรียบเทียบหน่วยความจำ: int array vs bitset

โครงสร้าง	หน่วยความจำ (บิต/สมาชิก)	ขนาดรวม ($n = 10^6$)
`int arr[n]`	32	4 MB
`bool arr[n]`	8	1 MB
`bitset<n>`	1	$\approx$ 125 KB

จะเห็นว่า `bitset` ประหยัดหน่วยความจำได้มากกว่า `int` ถึง 32 เท่า และมากกว่า `bool` ถึง 8 เท่า ในโจทย์ที่มีข้อจำกัดหน่วยความจำ การใช้ `bitset` อาจเป็นความแตกต่างระหว่างผ่านกับไม่ผ่าน

เมื่อไหร่ควรใช้ bitset

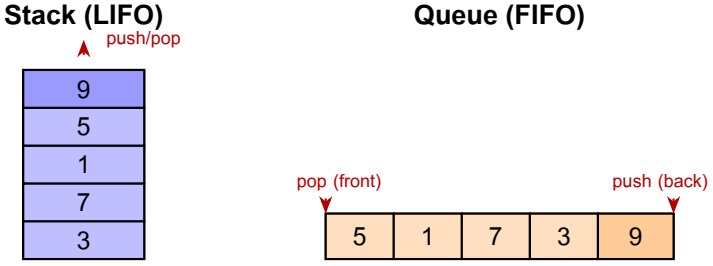
ใช้ `bitset` เมื่อ:

- ข้อมูลแต่ละตำแหน่งมีค่าเพียง 0 หรือ 1 เท่านั้น
- ต้องการดำเนินการบิตกับข้อมูลจำนวนมากอย่างรวดเร็ว
- มีข้อจำกัดด้านหน่วยความจำที่เข้มงวด
- ต้องการทำ Subset Sum หรือปัญหาที่คล้ายกันด้วยเทคนิค Bitmask DP

5.5 Stack และ Queue

Stack และ Queue เป็นโครงสร้างข้อมูลที่จัดเก็บข้อมูลแบบเรียงลำดับ แต่ต่างกันที่ลำดับการนำข้อมูลออก Stack ใช้หลักการ LIFO (Last In, First Out) ส่วน Queue ใช้หลักการ FIFO (First In, First Out) ทั้งสองมีประโยชน์อย่างมากในการแข่งขัน และเป็นฐานสำคัญของขั้นตอนวิธีหลายตัว

รูปที่ 5.1 เปรียบเทียบหลักการทำงานของ Stack (LIFO) และ Queue (FIFO)



รูปที่ 5.1 เปรียบเทียบ Stack (LIFO): เพิ่ม/ลบที่ด้านบนเท่านั้น กับ Queue (FIFO): เพิ่มที่ด้านหลัง ลบที่ด้านหน้า

5.5.1 Stack (LIFO): push, pop, top

Stack เป็นโครงสร้างข้อมูลที่ทำงานตามหลักการ **Last In, First Out (LIFO)** — ข้อมูลที่ใส่เข้าไปล่าสุดจะถูกนำออกก่อน เปรียบเหมือนกองจานที่เราวางจานใหม่ทับจานเก่า และหยิบจานบนสุดออกก่อนเสมอ

```cpp
#include <stack>
stack<int> s;
// push(): Add data to top -- O(1)
s.push(10);
s.push(20);
s.push(30);     // stack: [10, 20, 30] (30 is on top)
// top(): View top data (doesn't remove) -- O(1)
int x = s.top();   // x = 30
// pop(): Remove top data -- O(1)
s.pop();           // stack: [10, 20]
// empty(): Check if stack is empty
bool is_empty = s.empty();   // false
// size(): Number of elements
int n = s.size();            // n = 2
```

ตัวอย่างโค้ดที่ 5.12: การใช้งาน stack

ระวัง: ห้ามเรียก top() หรือ pop() เมื่อ stack ว่าง

การเรียก `top()` หรือ `pop()` บน stack ว่างเป็นพฤติกรรมที่ไม่นิยาม (Undefined Behavior) ควรตรวจสอบด้วย `!s.empty()` ก่อนเสมอ

5.5.2 การประยุกต์ Stack: ตรวจวงเล็บ, แปลงนิพจน์

Stack มีประโยชน์มากในโจทย์ที่เกี่ยวข้องกับการจับคู่ การตรวจสอบโครงสร้าง และการแปลงนิพจน์ ตัวอย่างที่พบบ่อยได้แก่:

1. ตรวจสอบวงเล็บให้ถูกต้อง:

```cpp
bool isBalanced(const string& expr) {
    stack<char> st;
    for (char c : expr) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char top = st.top();
            st.pop();
            if (c == ')' && top != '(') return false;
```

```
            if (c == ']' && top != '[') return false;
            if (c == '}' && top != '{') return false;
        }
    }
    return st.empty();
}
// isBalanced("({[]})") = true
// isBalanced("([)]") = false
// isBalanced("(()") = false
```

ตัวอย่างโค้ดที่ 5.13: ตรวจสอบวงเล็บด้วย stack

2. คำนวณนิพจน์ Postfix:

นิพจน์ Postfix (Reverse Polish Notation) คือนิพจน์ที่ตัวดำเนินการอยู่หลังตัวถูกดำเนินการ เช่น 3 4 + 5 * หมายถึง $(3+4)\times 5 = 35$ การคำนวณนิพจน์ Postfix ใช้ Stack ได้อย่างเป็นธรรมชาติ:

```
int evalPostfix(const string& expr) {
    stack<int> st;
    istringstream iss(expr);
    string token;
    while (iss >> token) {
        if (token == "+" || token == "-" ||
            token == "*" || token == "/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (token == "+") st.push(a + b);
            if (token == "-") st.push(a - b);
            if (token == "*") st.push(a * b);
            if (token == "/") st.push(a / b);
        } else {
            st.push(stoi(token));
        }
    }
    return st.top();
}
```

ตัวอย่างโค้ดที่ 5.14: คำนวณนิพจน์ Postfix ด้วย stack

5.5.3 Queue (FIFO): push, pop, front

Queue เป็นโครงสร้างข้อมูลที่ทำงานตามหลักการ **First In, First Out (FIFO)** — ข้อมูลที่ใส่เข้าไปก่อนจะถูกนำออกก่อน เปรียบเหมือนคิวซื้อตั๋วที่คนที่มาก่อนได้รับบริการก่อน

```
#include <queue>
```

```cpp
queue<int> q;
// push(): Add data to back -- O(1)
q.push(10);
q.push(20);
q.push(30);     // queue: [10, 20, 30] (10 is at front)
// front(): View front data (doesn't remove) -- O(1)
int x = q.front();   // x = 10
// back(): View back data -- O(1)
int y = q.back();    // y = 30
// pop(): Remove front data -- O(1)
q.pop();             // queue: [20, 30]
// empty() and size()
bool is_empty = q.empty();   // false
int n = q.size();            // n = 2
```

ตัวอย่างโค้ดที่ 5.15: การใช้งาน queue

สังเกต: push/pop ทำงานตรงข้ามกับ stack

ใน Queue `push()` เพิ่มที่ด้านหลัง และ `pop()` ลบที่ด้านหน้า ซึ่งตรงข้ามกับ Stack ที่ `push()` และ `pop()` ทำงานที่ด้านเดียวกัน (ด้านบน) ระวังสับสน!

5.5.4 Deque: push/pop ทั้งสองด้าน

Deque (Double-Ended Queue) เป็นโครงสร้างข้อมูลที่ขยายความสามารถของ Queue โดยสามารถเพิ่มและลบสมาชิกได้ทั้งด้านหน้าและด้านหลัง ด้วยเวลา $\mathcal{O}(1)$ เฉลี่ย Deque เหมาะสำหรับโจทย์ที่ต้องจัดการข้อมูลทั้งสองด้านของโครงสร้าง

```cpp
#include <deque>
deque<int> dq;
// Add element at back
dq.push_back(10);    // dq: [10]
dq.push_back(20);    // dq: [10, 20]
// Add element at front
dq.push_front(5);    // dq: [5, 10, 20]
dq.push_front(3);    // dq: [3, 5, 10, 20]
// Remove element at back
dq.pop_back();       // dq: [3, 5, 10]
// Remove element at front
dq.pop_front();      // dq: [5, 10]
// Access by index -- O(1)
int x = dq[0];       // x = 5
```

```
// View data at front and back
int f = dq.front(); // f = 5
int b = dq.back();  // b = 10
```

ตัวอย่างโค้ดที่ 5.16: การใช้งาน deque

เปรียบเทียบ deque กับ vector

deque รองรับการเพิ่ม/ลบทั้งสองด้านด้วย $\mathcal{O}(1)$ เหมือน list แต่ยังรองรับการเข้าถึงด้วยดัชนี $\mathcal{O}(1)$ เหมือน vector อย่างไรก็ตาม deque ช้ากว่า vector เล็กน้อยในการเข้าถึงสมาชิก เนื่องจากโครงสร้างหน่วยความจำที่ซับซ้อนกว่า

5.5.5 การประยุกต์ Queue: BFS, Sliding Window

Queue มีบทบาทสำคัญในหลายขั้นตอนวิธี:

1. **BFS (Breadth-First Search):** ขั้นตอนวิธีค้นหาแบบกว้างใช้ Queue เป็นโครงสร้างข้อมูลหลัก (จะกล่าวรายละเอียดในบทที่เกี่ยวกับกราฟ)

2. **Sliding Window Minimum:** หาค่าต่ำสุดในหน้าต่างเลื่อนขนาด k โดยใช้ deque:

```
// Find minimum in every window of size k of the array
vector<int> slidingWindowMin(vector<int>& arr, int k) {
    deque<int> dq;  // Store indices (values sorted ascending)
    vector<int> ans;
    for (int i = 0; i < (int)arr.size(); i++) {
        // Remove elements outside window
        while (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }
        // Remove elements greater than arr[i] (can't be minimum)
        while (!dq.empty() && arr[dq.back()] >= arr[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        if (i >= k - 1) {
            ans.push_back(arr[dq.front()]);
        }
    }
    return ans;
}
// arr = {1, 3, -1, -3, 5, 3, 6, 7}, k = 3
// Result: {-1, -3, -3, -3, 3, 3}
```

ตัวอย่างโค้ดที่ 5.17: Sliding Window Minimum ด้วย deque

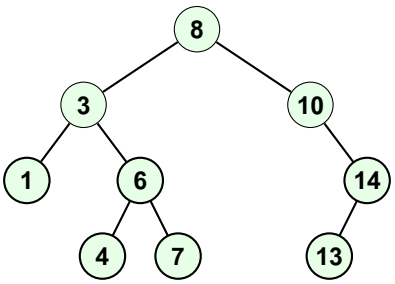
Monotonic Deque

เทคนิค Sliding Window Minimum ใช้ Deque แบบ Monotonic (ค่าเรียงลำดับ) หลักการคือ: สมาชิกที่ “ไม่มีโอกาส” เป็นค่าต่ำสุดจะถูกลบทิ้งทันที ทำให้สมาชิกที่เหลือใน Deque มีลำดับที่เพิ่มขึ้นเสมอ เทคนิคนี้ใช้เวลา $\mathcal{O}(n)$ รวม (แต่ละสมาชิกถูกเพิ่มและลบจาก Deque อย่างมาก 1 ครั้ง)

5.6 Set และ Multiset

Set เป็นโครงสร้างข้อมูลที่เก็บข้อมูลแบบ **ไม่ซ้ำ** และ **เรียงลำดับ** โดยอัตโนมัติ ภายใน Set ใช้ต้นไม้ค้นหาแบบทวิภาคสมดุล (Balanced Binary Search Tree) ทำให้การดำเนินการส่วนใหญ่ใช้เวลา $\mathcal{O}(\log n)$ เท่านั้น Set มีประโยชน์มากในการแข่งขันสำหรับการตรวจสอบการมีอยู่ของข้อมูล การหาค่าที่ใกล้ที่สุด และการดึงข้อมูลตามลำดับ

รูปที่ 5.2 แสดงตัวอย่างต้นไม้ค้นหาแบบทวิภาค (BST) ซึ่งเป็นโครงสร้างภายในของ `set` ใน C++ สมาชิกที่น้อยกว่าอยู่ซ้าย สมาชิกที่มากกว่าอยู่ขวา ทำให้การค้นหา การแทรก และการลบใช้เวลา $\mathcal{O}(\log n)$



รูปที่ 5.2 ตัวอย่าง BST: สมาชิกที่น้อยกว่ารากอยู่ซ้าย มากกว่ารากอยู่ขวา ทำให้ค้นหาได้ใน $\mathcal{O}(\log n)$

5.6.1 Set: insert, count, erase

```cpp
#include <set>
set<int> s;
// insert(): Add element -- O(log n)
s.insert(3);
s.insert(1);
s.insert(5);
s.insert(2);
s.insert(4);
// s = {1, 2, 3, 4, 5} (auto-sorted!)
// count(): Count elements equal to given value -- O(log n)
// Since set has no duplicates, ans is 0 or 1 only
cout << s.count(3);  // 1 (exists)
cout << s.count(7);  // 0 (not found)
// erase(): Remove element -- O(log n)
```

```
s.erase(3);              // s = {1, 2, 4, 5}
// Cannot access by index like s[0] -- not allowed!
// Must use iterator only
// size() and empty()
int n = s.size();        // n = 4
bool is_empty = s.empty(); // false
```

ตัวอย่างโค้ดที่ 5.18: การใช้งาน set

5.6.2 คุณสมบัติไม่ซ้ำ

คุณสมบัติสำคัญของ `set` คือไม่อนุญาตให้มีสมาชิกที่ซ้ำกัน หากพยายามแทรกค่าที่มีอยู่แล้ว การแทรกจะไม่มีผล:

```
set<int> s;
s.insert(5);
s.insert(5);    // Nothing happens -- 5 already exists
s.insert(5);    // Still nothing happens
cout << s.count(5);  // 1 (not 3)
cout << s.size();    // 1 (not 3)
```

ตัวอย่างโค้ดที่ 5.19: คุณสมบัติไม่ซ้ำของ set

5.6.3 Multiset: อนุญาตให้ซ้ำ

`multiset` เหมือน `set` ทุกประการ ยกเว้นว่า **อนุญาตให้มีสมาชิกซ้ำกันได้** ดังนั้น `count()` อาจคืนค่ามากกว่า 1 ได้:

```
#include <set>
multiset<int> ms;
ms.insert(5);
ms.insert(3);
ms.insert(5);
ms.insert(5);
ms.insert(3);
// count() returns number of elements equal to given value
cout << ms.count(5);  // 3
cout << ms.count(3);  // 2
cout << ms.size();    // 5
// Warning: erase(x) removes all elements equal to x!
ms.erase(5);          // Remove all 3 instances of 5
cout << ms.count(5);  // 0
// To remove only 1 instance: use erase(find(x))
```

```
multiset<int> ms2 = {5, 5, 5};
ms2.erase(ms2.find(5));  // Remove only 1 instance
cout << ms2.count(5);    // 2
```

ตัวอย่างโค้ดที่ 5.20: การใช้งาน multiset

ระวัง: erase ใน multiset

การใช้ `ms.erase(x)` จะลบสมาชิกที่เท่ากับ x **ทั้งหมด** หากต้องการลบเพียง 1 ตัว ให้ใช้ `ms.erase(ms.find(x))` แทน นอกจากนี้ ต้องตรวจสอบว่า `ms.find(x) != ms.end()` ก่อนเรียก `erase` เพื่อป้องกันการลบ iterator ที่ไม่ถูกต้อง

5.6.4 Iterator ของ Set

เนื่องจาก `set` ไม่สามารถเข้าถึงด้วยดัชนีได้ เราต้องใช้ iterator เพื่อวนลูปหรือเข้าถึงสมาชิก:

```
set<int> s = {5, 2, 8, 1, 9, 3};
// Loop to display elements (auto-sorted ascending)
for (auto it = s.begin(); it != s.end(); it++) {
    cout << *it << " ";    // Prints: 1 2 3 5 8 9
}
// Using range-based for (shorter)
for (int x : s) {
    cout << x << " ";      // Prints: 1 2 3 5 8 9
}
// Find maximum value (last element)
int max_val = *s.rbegin();    // 9
// Find minimum value (first element)
int min_val = *s.begin();     // 1
// Move iterator
auto it = s.begin();
it++;           // Move to next element
it--;           // Move to previous element
```

ตัวอย่างโค้ดที่ 5.21: ใช้ iterator กับ set

5.6.5 lower_bound/upper_bound ใน Set

หนึ่งในฟังก์ชันที่ทรงพลังที่สุดของ `set` คือ `lower_bound` และ `upper_bound` ซึ่งทำงานในเวลา $\mathcal{O}(\log n)$:

```
set<int> s = {1, 3, 5, 7, 9};
// lower_bound(x): iterator to first element >= x
auto it1 = s.lower_bound(5);    // Points to 5
```

```cpp
auto it2 = s.lower_bound(6);      // Points to 7 (first element >= 6)
// upper_bound(x): iterator to first element > x
auto it3 = s.upper_bound(5);      // Points to 7 (first element > 5)
auto it4 = s.upper_bound(9);      // s.end() (no value > 9)
// If not found, lower_bound returns end()
auto it5 = s.lower_bound(10);     // s.end()
// Return value from iterator
if (it1 != s.end()) {
    cout << *it1 << "\n";   // Prints: 5
}
```

ตัวอย่างโค้ดที่ 5.22: lower_bound และ upper_bound ใน set

ห้ามใช้ std::lower_bound กับ set

ห้ามใช้ `lower_bound(s.begin(), s.end(), x)` กับ `set` เพราะเวอร์ชันนี้ใช้เวลา $\mathcal{O}(n)$ (วนลูปทีละตัว) ให้ใช้ `s.lower_bound(x)` ของ set เอง ซึ่งใช้เวลา $\mathcal{O}(\log n)$ เท่านั้น สังเกตว่า `unordered_set` ไม่มีฟังก์ชัน `lower_bound` เพราะข้อมูลไม่ได้เรียงลำดับ

5.6.6 การหาค่าที่ใกล้ที่สุด

เทคนิคการหาสมาชิกใน set ที่ใกล้ค่า x ที่สุด เป็นโจทย์คลาสสิกที่พบบ่อยในการแข่งขัน:

```cpp
// Return element in set closest to x
// Assume set is not empty
int closest(set<int>& s, int x) {
    auto it = s.lower_bound(x);
    // Case 1: all elements are less than x
    if (it == s.end()) {
        return *(--it);   // Return the last element
    }
    // Case 2: first element >= x equals x itself
    if (*it == x) {
        return x;
    }
    // Case 3: x is between two elements
    if (it == s.begin()) {
        return *it;   // No previous element
    }
    int after = *it;
    int before = *(--it);
    // Compare distances
```

```
    if (x - before <= after - x) {
        return before;
    } else {
        return after;
    }
}
```

ตัวอย่างโค้ดที่ 5.23: หาสมาชิกใน set ที่ใกล้ค่า x ที่สุด

ตัวอย่าง: หาค่าใกล้ที่สุด

สมมติ `s = {2, 5, 8, 12, 20}`

- `closest(s, 6)` $\Rightarrow$ 5 (ระยะห่าง 1 จาก 5, 2 จาก 8)
- `closest(s, 9)` $\Rightarrow$ 8 (ระยะห่าง 1 จาก 8, 3 จาก 12)
- `closest(s, 1)` $\Rightarrow$ 2 (ค่าต่ำสุดใน set)
- `closest(s, 25)` $\Rightarrow$ 20 (ค่าสูงสุดใน set)

5.6.7 unordered_set

`unordered_set` เป็นอีกทางเลือกหนึ่งสำหรับเก็บข้อมูลแบบไม่ซ้ำ แต่ใช้ Hash Table เป็นโครงสร้างภายในแทน Balanced BST ทำให้การดำเนินการพื้นฐานมีความซับซ้อนเฉลี่ย $\mathcal{O}(1)$:

```
#include <unordered_set>
unordered_set<int> us;
us.insert(3);     // O(1) average
us.insert(1);
us.insert(5);
cout << us.count(3);    // 1 -- O(1) average
cout << us.count(7);    // 0
us.erase(3);            // O(1) average
// Limitation: data is not sorted
// Loop output may be in any order
for (int x : us) {
    cout << x << " ";   // May print: 5 1 or 1 5
}
// No lower_bound and upper_bound!
// us.lower_bound(3);  // This function doesn't exist
```

ตัวอย่างโค้ดที่ 5.24: การใช้งาน unordered_set

ตารางที่ 5.3 เปรียบเทียบ set กับ unordered_set

คุณสมบัติ	set	unordered_set
โครงสร้างภายใน	Balanced BST	Hash Table
insert/erase/count	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$ เฉลี่ย
ข้อมูลเรียงลำดับ	ใช่	ไม่ใช่
lower_bound/upper_bound	มี	ไม่มี
กรณีเลวร้ายที่สุด	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

เลือก set หรือ unordered_set?

สำหรับการแข่งขัน `set` มักเพียงพอแล้ว เพราะ $\mathcal{O}(\log n)$ ยังเร็วมากสำหรับ $n \leq 10^6$ (เพียงประมาณ 20 การเปรียบเทียบ) ใช้ `unordered_set` เมื่อ:

- ต้องการความเร็วสูงสุดและไม่ต้องการ `lower_bound`
- ต้องการเพิ่ม/ลบ/ค้นหาจำนวนมาก (เช่น 10^7 ครั้งขึ้นไป)

5.7 Map

Map เป็นโครงสร้างข้อมูลที่เก็บข้อมูลเป็นคู่ key–value (คีย์–ค่า) โดย key ทำหน้าที่เหมือนดัชนีในแถวลำดับ แต่สามารถเป็นประเภทใดก็ได้ (ไม่จำเป็นต้องเป็นจำนวนเต็มต่อเนื่อง) Map เปรียบเหมือน “แถวลำดับทั่วไป” ที่ดัชนีไม่จำเป็นต้องเป็น 0, 1, 2, …แต่สามารถเป็น string, pair, หรือประเภทอื่นได้

5.7.1 Map: แถวลำดับทั่วไป (key→value)

Map มี 2 การใช้งานหลักใน C++:

- `map` — ใช้ Balanced BST เป็นโครงสร้างภายใน การดำเนินการใช้เวลา $\mathcal{O}(\log n)$ และ key จะเรียงลำดับ
- `unordered_map` — ใช้ Hash Table เป็นโครงสร้างภายใน การดำเนินการเฉลี่ยใช้เวลา $\mathcal{O}(1)$ แต่ key ไม่เรียงลำดับ

Map มีประโยชน์มากในการแข่งขัน เช่น การนับความถี่ของข้อมูล การเก็บ mapping ระหว่างสองค่า และการใช้เป็น “แถวลำดับ” ที่มีดัชนีเป็น string หรือค่าอื่นๆ ที่ไม่ใช่จำนวนเต็มต่อเนื่อง

5.7.2 การประกาศและใช้งาน

```cpp
#include <map>
// Declare map: string key, int value
map<string, int> m;
```

```cpp
// Add/modify data with []
m["monkey"] = 4;
m["banana"] = 3;
m["harpsichord"] = 9;
// Access data with [] -- O(log n)
cout << m["banana"];    // 3
// Modify value
m["banana"] = 5;
cout << m["banana"];    // 5
// Map of other types
map<int, int> cnt;              // key: int, value: int
map<string, string> dict;       // key: string, value: string
map<int, vector<int>> graph;    // Value can be a vector
```

ตัวอย่างโค้ดที่ 5.25: การประกาศและใช้งาน map

5.7.3 ค่าเริ่มต้นและการตรวจสอบ key

ข้อควรระวังสำคัญของ `map`: การเข้าถึง key ที่ไม่มีอยู่ด้วย `[]` จะ **แทรกค่าเริ่มต้น** (Default Value) เข้าไปโดยอัตโนมัติ สำหรับ `int` ค่าเริ่มต้นคือ 0 สำหรับ `string` คือ string ว่าง:

```cpp
map<string, int> m;
m["banana"] = 3;
// Access non-existent key: inserts new key with value 0
cout << m["apple"];    // 0 (and inserts "apple" into the map!)
cout << m.size();      // 2 (increased from 1 to 2)
// How to check if key exists (without inserting new entry)
if (m.count("banana")) {
    cout << "has banana" << "\n";
}
// Use find() to search for key
auto it = m.find("banana");
if (it != m.end()) {
    cout << "banana = " << it->second << "\n";
}
// Remove key
m.erase("apple");
```

ตัวอย่างโค้ดที่ 5.26: ค่าเริ่มต้นใน map และการตรวจสอบ key

ระวัง: [

แทรกค่าเริ่มต้นอัตโนมัติ] ห้ามใช้ `m[key]` เพื่อตรวจสอบว่า key มีอยู่หรือไม่ เพราะจะแทรก key ใหม่เข้าไปโดยไม่ตั้งใจ ใช้ `m.count(key)` หรือ `m.find(key)` แทน

5.7.4 การวนลูปแสดง key-value

```cpp
map<string, int> m;
m["banana"] = 3;
m["apple"] = 5;
m["cherry"] = 7;
// Loop with range-based for
for (auto x : m) {
    cout << x.first << " -> " << x.second << "\n";
}
// Result (auto-sorted by key):
// apple -> 5
// banana -> 3
// cherry -> 7
// Or use structured binding (C++17)
for (auto& [key, value] : m) {
    cout << key << " -> " << value << "\n";
}
// Reverse loop (descending order)
for (auto it = m.rbegin(); it != m.rend(); it++) {
    cout << it->first << " -> " << it->second << "\n";
}
```

ตัวอย่างโค้ดที่ 5.27: วนลูปแสดง key-value ใน map

Map เรียงลำดับตาม key

ใน map ข้อมูลจะถูกเรียงลำดับตาม key อัตโนมัติ เมื่อวนลูปด้วย iterator จะได้สมาชิกตามลำดับจากน้อยไปมากของ key คุณสมบัตินี้มีประโยชน์เมื่อต้องการประมวลผลข้อมูลตามลำดับของ key โดยไม่ต้องเรียงแยกต่างหาก

5.7.5 unordered_map

unordered_map ใช้ Hash Table เป็นโครงสร้างภายใน ทำให้การดำเนินการพื้นฐานมีความซับซ้อนเฉลี่ย $\mathcal{O}(1)$ เร็วกว่า map แต่ key จะไม่เรียงลำดับ:

```cpp
#include <unordered_map>
```

```cpp
unordered_map<string, int> um;
um["hello"] = 1;      // O(1) average
um["world"] = 2;
cout << um.count("hello");   // 1 -- O(1) average
cout << um["world"];         // 2 -- O(1) average
// Loop: order is not fixed
for (auto& [key, value] : um) {
    cout << key << " " << value << "\n";
}
// No lower_bound/upper_bound
// Key order is not guaranteed
```

ตัวอย่างโค้ดที่ 5.28: การใช้งาน unordered_map

ตารางที่ 5.4 เปรียบเทียบ map กับ unordered_map

คุณสมบัติ	map	unordered_map
โครงสร้างภายใน	Balanced BST	Hash Table
insert/erase/count/[]	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$ เฉลี่ย
Key เรียงลำดับ	ใช่	ไม่ใช่
lower_bound/upper_bound	มี	ไม่มี
กรณีเลวร้ายที่สุด	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Key ที่ใช้ได้	รองรับ <	รองรับ hash

เลือก map หรือ unordered_map?

สำหรับการแข่งขัน map มักเพียงพอและปลอดภัยกว่า เพราะ:

- ไม่มีกรณีเลวร้าย $\mathcal{O}(n)$ จาก Hash Collision
- รองรับ key ทุกประเภทที่มี operator<
- มี lower_bound และ upper_bound ให้ใช้

ใช้ unordered_map เมื่อต้องการความเร็วสูงสุดและไม่ต้องการฟังก์ชันเหล่านี้

5.8 Heap และ Priority Queue

Priority Queue เป็นโครงสร้างข้อมูลที่รองรับการดึงสมาชิกที่มี “ลำดับความสำคัญ” สูงสุดออกมาก่อน โดยภายในใช้โครงสร้างแบบ Binary Heap Priority Queue มีประโยชน์มากในการแข่งขัน โดยเฉพาะในขั้นตอนวิธี Dijkstra และ Prim ที่จะกล่าวถึงในบทถัดไป

5.8.1 Binary Heap: โครงสร้างและสมบัติ

Binary Heap เป็นต้นไม้ทวิภาคแบบ Complete Binary Tree ที่จัดเก็บในแถวลำดับ 1 มิติ โดยมีสมบัติสำคัญดังนี้:

- **Max Heap:** ค่าของทุกโหนด $\geq$ ค่าของลูกทั้งสอง $\Rightarrow$ ราก (root) เก็บค่ามากที่สุด
- **Min Heap:** ค่าของทุกโหนด $\leq$ ค่าของลูกทั้งสอง $\Rightarrow$ ราก (root) เก็บค่าน้อยที่สุด

เนื่องจากเป็น Complete Binary Tree ทุกระดับจะเต็มสมบูรณ์ยกเว้นระดับสุดท้าย ที่จะถูกเติมจากซ้ายไปขวา ทำให้สามารถจัดเก็บในแถวลำดับได้โดยไม่เสียพื้นที่

5.8.2 สูตรดัชนี (parent, left, right)

การจัดเก็บ Binary Heap ในแถวลำดับ (ดัชนีเริ่มจาก 1) มีสูตรคำนวณดัชนีดังนี้:

ตารางที่ 5.5 สูตรดัชนีของ Binary Heap

ความสัมพันธ์	สูตรดัชนี
โหนดพ่อของ i	$\lfloor i/2 \rfloor$
ลูกซ้ายของ i	$2i$
ลูกขวาของ i	$2i+1$

สูตรเทียบเท่าด้วย Bit Manipulation (เร็วกว่า):

- โหนดพ่อ: `i >> 1` (เลื่อนบิตขวา 1 ตำแหน่ง = หาร 2)
- ลูกซ้าย: `i << 1` (เลื่อนบิตซ้าย 1 ตำแหน่ง = คูณ 2)
- ลูกขวา: `(i << 1) | 1` หรือ `(i << 1) + 1`

ตัวอย่าง: Max Heap

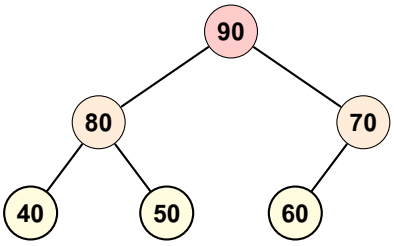
สมมติมี Max Heap เก็บค่า {90, 80, 70, 40, 50, 60} ในแถวลำดับ:

ดัชนี	1	2	3	4	5	6
ค่า	90	80	70	40	50	60

- ราก (ดัชนี 1) = 90 (ค่ามากที่สุด)
- ลูกซ้ายของ 90 (ดัชนี 2) = 80, ลูกขวา (ดัชนี 3) = 70
- ลูกซ้ายของ 80 (ดัชนี 4) = 40, ลูกขวา (ดัชนี 5) = 50
- ลูกซ้ายของ 70 (ดัชนี 6) = 60

ตรวจสอบสมบัติ Max Heap: $90 \geq 80 \geq 40$, $90 \geq 80 \geq 50$, $90 \geq 70 \geq 60$ ✓

รูปที่ 5.3 แสดงโครงสร้างต้นไม้ของ Max Heap ตัวอย่างข้างต้น ควบคู่กับการจัดเก็บในแถวลำดับ



ดัชนี	1	2	3	4	5	6
ค่า	90	80	70	40	50	60

รูปที่ 5.3 Max Heap: โครงสร้างต้นไม้ (บน) และการจัดเก็บในแถวลำดับ (ล่าง) สังเกตว่าค่าของพ่อ $\geq$ ค่าของลูกเสมอ

5.8.3 Max Heap vs Min Heap

ทั้ง Max Heap และ Min Heap รองรับการดำเนินการสำคัญ 2 อย่าง:

- **Insert (เพิ่มสมาชิก):** เพิ่มสมาชิกใหม่ที่ท้ายแถวลำดับ แล้ว “ดันขึ้น” (Sift Up / Bubble Up) เพื่อรักษาสมบัติ Heap — ใช้เวลา $\mathcal{O}(\log n)$
- **Extract (ดึงสมาชิกสูงสุด/ต่ำสุด):** สลับรากกับสมาชิกตัวสุดท้าย ลบสมาชิกตัวสุดท้าย แล้ว “ดันลง” (Sift Down / Bubble Down) เพื่อรักษาสมบัติ Heap — ใช้เวลา $\mathcal{O}(\log n)$

Heap ไม่รองรับการค้นหา

Binary Heap ไม่รองรับการค้นหาสมาชิกที่มีค่าเท่ากับ x ได้อย่างมีประสิทธิภาพ (ต้องใช้เวลา $\mathcal{O}(n)$) หากต้องการค้นหา ควรใช้ `set` หรือ `map` แทน

5.8.4 Priority Queue ใน C++

ใน C++ `priority_queue` เป็น adapter ที่ห่อหุ้ม Binary Heap โดยค่าเริ่มต้นเป็น **Max Heap**:

```cpp
#include <queue>
// Default: Max Heap (largest value on top)
priority_queue<int> pq;
// push(): Add element -- O(log n)
pq.push(5);
pq.push(1);
pq.push(10);
pq.push(3);
```

```
pq.push(7);
// top(): View largest element -- O(1)
cout << pq.top();    // 10
// pop(): Remove largest element -- O(log n)
pq.pop();            // Remove 10
cout << pq.top();    // 7
// size() and empty()
cout << pq.size();   // 4
cout << pq.empty();  // 0 (false)
```

ตัวอย่างโค้ดที่ 5.29: การใช้งาน priority_queue (Max Heap)

5.8.5 Min Priority Queue

หากต้องการ Min Heap (ค่าน้อยสุดอยู่บนสุด) ต้องระบุพารามิเตอร์เพิ่มเติม:

```
// Min Heap: smallest value on top
priority_queue<int, vector<int>, greater<int>> min_pq;
min_pq.push(5);
min_pq.push(1);
min_pq.push(10);
min_pq.push(3);
cout << min_pq.top();    // 1 (smallest value)
min_pq.pop();
cout << min_pq.top();    // 3
// Priority queue of pair (compares first first)
priority_queue<pair<int,int>> pq_pair;
pq_pair.push({3, 100});
pq_pair.push({1, 200});
pq_pair.push({5, 50});
cout << pq_pair.top().first;    // 5
// Min priority queue of pair
priority_queue<pair<int,int>,
               vector<pair<int,int>>,
               greater<pair<int,int>>> min_pq_pair;
```

ตัวอย่างโค้ดที่ 5.30: Min Priority Queue

เทคนิค: ใช้ค่าตรงข้ามแทน Min Heap

อีกวิธีที่ง่ายกว่าในการสร้าง Min Heap คือเก็บค่าตรงข้าม (Negative) แล้วใช้ Max Heap ปกติ:

```
priority_queue<int> pq;   // Max Heap
```

```
pq.push(-5);     // Instead of push(5)
pq.push(-1);     // Instead of push(1)
cout << -pq.top();   // 1 (smallest = opposite of largest)
```

วิธีนี้เขียนสั้นกว่าและไม่ต้องจำ syntax ยาวๆ ของ Min Heap

5.8.6 การประยุกต์: Dijkstra, Prim

Priority Queue เป็นโครงสร้างข้อมูลหลักในขั้นตอนวิธีหลายตัว:

1. **Dijkstra's Shortest Path:** ใช้ Min Priority Queue เพื่อดึงโหนดที่มีระยะทางสั้นที่สุดออกมาก่อนเสมอ ทำให้ขั้นตอนวิธีทำงานได้ถูกต้องและมีประสิทธิภาพ
2. **Prim's MST:** ใช้ Min Priority Queue เพื่อเลือกเส้นเชื่อมที่มีน้ำหนักน้อยที่สุดในแต่ละขั้น

รายละเอียดของขั้นตอนวิธีทั้งสองจะกล่าวถึงในบทที่เกี่ยวกับกราฟ

5.9 Linked List และ Hash Table

ในหัวข้อนี้ เราจะพูดถึง Linked List ซึ่งเป็นโครงสร้างข้อมูลที่ใช้ pointer เชื่อมโยงโหนดเข้าด้วยกัน และ Hash Table ซึ่งเป็นโครงสร้างพื้นฐานเบื้องหลัง `unordered_map` และ `unordered_set` แม้ว่าในการแข่งขันเรามักไม่ค่อยใช้ Linked List โดยตรง แต่ความเข้าใจในโครงสร้างเหล่านี้จะช่วยให้เลือกใช้โครงสร้างข้อมูลได้อย่างเหมาะสม

5.9.1 Linked List (list): ข้อดี-ข้อเสีย

Linked List เป็นโครงสร้างข้อมูลที่จัดเก็บข้อมูลเป็นโหนด (Node) ที่เชื่อมโยงกันด้วย pointer แต่ละโหนดเก็บข้อมูลและ pointer ชี้ไปยังโหนดถัดไป ใน C++ STL `list` เป็น Doubly Linked List ที่สามารถเดินได้ทั้งสองทิศทาง

```
#include <list>
list<int> lst;
// Add at front and back -- O(1)
lst.push_back(10);
lst.push_back(20);
lst.push_front(5);
// lst: [5, 10, 20]
// Insert before iterator position -- O(1)
auto it = lst.begin();
it++;
lst.insert(it, 7);     // lst: [5, 7, 10, 20]
// Remove at iterator position -- O(1)
lst.erase(it);          // lst: [5, 7, 20]
```

```
14  // Access by index: not supported! Must use iterator
15  // lst[0];  // ERROR!
```

ตัวอย่างโค้ดที่ 5.31: การใช้งาน list

5.9.2 เมื่อไหร่ควรใช้ list แทน vector

`list` มีข้อดีเหนือ `vector` ในบางกรณี แต่ส่วนใหญ่ `vector` เป็นตัวเลือกที่ดีกว่า:

ตารางที่ 5.6 เปรียบเทียบ list กับ vector

การดำเนินการ	`vector`	`list`
เข้าถึงด้วยดัชนี	$\mathcal{O}(1)$	$\mathcal{O}(n)$
เพิ่ม/ลบที่ท้าย	$\mathcal{O}(1)$ เฉลี่ย	$\mathcal{O}(1)$
เพิ่ม/ลบที่หน้า	$\mathcal{O}(n)$	$\mathcal{O}(1)$
แทรก/ลบที่กลาง	$\mathcal{O}(n)$	$\mathcal{O}(1)$ ด้วย iterator
Iterator stability	ไม่เสถียร	เสถียร
การใช้หน่วยความจำ	ต่ำ (contiguous)	สูง (ต่อโหนด)

จะเห็นว่า `list` มีข้อดีเฉพาะเมื่อต้องการ:

- แทรกหรือลบสมาชิกที่กลางโครงสร้างบ่อยๆ (และมี iterator ชี้อยู่แล้ว)
- ความเสถียรของ iterator (iterator ไม่เสียเมื่อแทรก/ลบสมาชิกอื่น)

ส่วนใหญ่ vector ดีกว่า list

ในการแข่งขัน `vector` เป็นตัวเลือกที่ดีกว่า `list` เกือบทุกกรณี เพราะ:

1. `vector` ใช้หน่วยความจำน้อยกว่า (Cache-friendly เนื่องจากข้อมูลอยู่ติดกัน)
2. `vector` เข้าถึงด้วยดัชนีได้ ($\mathcal{O}(1)$) ซึ่งใช้บ่อยในการแข่งขัน
3. การเรียงลำดับ `vector` เร็วกว่าเรียง `list` อย่างมาก
4. แม้การแทรกกลางจะเป็น $\mathcal{O}(n)$ แต่ด้วย Cache Effect ทำให้ `vector` มักเร็วกว่าในทางปฏิบัติ สำหรับ n ขนาดเล็กถึงปานกลาง

ใช้ `list` เฉพาะเมื่อมีเหตุผลชัดเจน เช่น ต้องการ iterator stability

5.9.3 Hash Table: แนวคิด

Hash Table เป็นโครงสร้างข้อมูลที่อยู่เบื้องหลัง `unordered_map` และ `unordered_set` หลักการทำงานคือใช้ **Hash Function** แปลง key ให้เป็นดัชนีของแถวลำดับ (Bucket) แล้วเก็บข้อมูลไว้ที่ตำแหน่งนั้น

ขั้นตอนการทำงาน:

1. รับ key k เข้ามา
2. คำนวณดัชนี: $h = \text{hash}(k) \bmod B$ เมื่อ B คือจำนวน bucket
3. เก็บ/ค้นหาข้อมูลที่ bucket หมายเลข h

เมื่อ key สองค่าแตกต่างกันแต่ hash ได้ดัชนีเดียวกัน เรียกว่า **Collision** วิธีจัดการ Collision ที่นิยมได้แก่:

- **Chaining:** แต่ละ bucket เก็บ Linked List ของข้อมูลที่ hash มาที่ bucket นั้น
- **Open Addressing:** หา bucket ว่างถัดไปเมื่อเกิด Collision

ความซับซ้อนของเวลา:

- กรณีเฉลี่ย: $\mathcal{O}(1)$ สำหรับ insert, erase, find
- กรณีเลวร้าย: $\mathcal{O}(n)$ (เมื่อ key ทุกตัว hash ไป bucket เดียวกัน)

5.9.4 เปรียบเทียบ BST vs Hash Table

ตาราง ต่อ ไป นี้ สรุป ความ แตก ต่าง ระหว่าง Balanced BST (ใช้ใน `map`/`set`) กับ Hash Table (ใช้ใน `unordered_map`/`unordered_set`):

ตารางที่ 5.7 เปรียบเทียบ BST กับ Hash Table

คุณสมบัติ	**Balanced BST** (`map`)	**Hash Table** (`unordered_map`)
Insert/Erase/Find	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$ เฉลี่ย, $\mathcal{O}(n)$ เลวร้าย
ข้อมูลเรียงลำดับ	ใช่	ไม่ใช่
lower_bound/upper_bound	มี	ไม่มี
ลำดับการวนลูป	เรียงตาม key	ไม่แน่นอน
หน่วยความจำ	ปานกลาง	อาจมากกว่า (bucket overhead)
Key ที่ใช้ได้	รองรับ <	ต้องมี hash function

สำหรับการแข่งขัน: map มักเพียงพอ

ในการแข่งขัน `map` มักเพียงพอและปลอดภัยกว่า `unordered_map` เพราะ:

- $\mathcal{O}(\log n)$ ยังเร็วมาก (≤ 30 การเปรียบเทียบสำหรับ $n \leq 10^9$)
- ไม่มีกรณีเลวร้าย $\mathcal{O}(n)$ จาก Hash Collision
- มี `lower_bound`, `upper_bound` และข้อมูลเรียงลำดับให้

ใช้ `unordered_map` เฉพาะเมื่อต้องการความเร็วสูงสุดและไม่ต้องการฟังก์ชันที่ต้องใช้การเรียงลำดับ

5.10 Bitmask เป็นเซตเบา (Lightweight Boolean Set)

ในหัวข้อก่อนหน้านี้ เราได้เรียนรู้เรื่อง `bitset` ซึ่งเป็นโครงสร้างข้อมูลสำหรับจัดเก็บบิตจำนวนมากที่ขนาดคงที่ แต่ในการแข่งขันจริง เรามักต้องจัดการกับเซตขนาดเล็ก (ไม่เกิน 60 ตัว) ที่แต่ละสมาชิกมีสถานะเพียง “เลือก” หรือ “ไม่เลือก” เท่านั้น ในกรณีเช่นนี้ เราไม่จำเป็นต้องใช้ `bitset` หรือ `vector<bool>` เลย เพราะ**จำนวนเต็ม (integer) ตัวเดียวก็เพียงพอ**ที่จะแทนเซตได้ เทคนิคนี้เรียกว่า **Bitmask**

5.10.1 Bitmask คืออะไร

Bitmask คือการใช้จำนวนเต็ม (integer) แทนเซตของค่าบูลีน โดยบิตที่ i ของจำนวนเต็มบอกว่าสมาชิกตัวที่ i อยู่ในเซตหรือไม่ บิต 1 หมายถึง “มี” และบิต 0 หมายถึง “ไม่มี”

ตัวอย่างเช่น ถ้า $S = 13 = 1101_{(2)}$ แสดงว่าเซตนี้มีสมาชิกที่ตำแหน่ง 0, 2, 3 (นับจากบิตขวาสุด) นั่นคือ $S = \{0, 2, 3\}$

ข้อดีหลักของ Bitmask คือ:

- ใช้หน่วยความจำน้อยมาก — เซตขนาด $n \leq 20$ ใช้เพียง 1 ตัวแปร `int`
- การดำเนินการทำได้รวดเร็วด้วย Bitwise Operator ในเวลา $\mathcal{O}(1)$
- เหมาะสำหรับใช้เป็นสถานะใน Dynamic Programming (DP+Bitmask)

5.10.2 การดำเนินการพื้นฐานของ Bitmask

การดำเนินการทั้งหมดใช้ Bitwise Operator โดยตรง ทำได้ในเวลา $\mathcal{O}(1)$:

```
int S = 0;                  // Empty set (all bits are 0)
// Set bit i (add element i to the set)
S |= (1 << i);
// Clear bit i (remove element i from the set)
S &= ~(1 << i);
// Toggle bit i (flip element i in the set)
S ^= (1 << i);
// Check bit i (is element i in the set?)
bool inSet = (S >> i) & 1;
// Alternative: bool inSet = S & (1 << i);
// Example usage
S = 0;                      // S = {} (empty)
S |= (1 << 2);              // S = {2}   (binary: 100)
S |= (1 << 0);              // S = {0,2}  (binary: 101)
S |= (1 << 3);              // S = {0,2,3}(binary: 1101 = 13)
bool has2 = (S >> 2) & 1;   // true (bit 2 is set)
bool has1 = (S >> 1) & 1;   // false (bit 1 is not set)
```

```
S &= ~(1 << 2);          // S = {0,3}  (removed element 2)
S ^= (1 << 1);           // S = {0,1,3}(toggled element 1)
```

ตัวอย่างโค้ดที่ 5.32: การดำเนินการพื้นฐานของ Bitmask

ข้อจำกัดของ Bitmask

จำนวนสมาชิกที่ Bitmask รองรับขึ้นอยู่กับจำนวนบิตของชนิดข้อมูล:

- `int` (32 บิต): รองรับเซตได้สูงสุด 31 สมาชิก (บิตที่ 0–30)
- `long long` (64 บิต): รองรับเซตได้สูงสุด 63 สมาชิก (บิตที่ 0–62)

ถ้าเซตมีขนาดใหญ่กว่านี้ ต้องใช้ `bitset` หรือโครงสร้างข้อมูลอื่นแทน

5.10.3 เทคนิค Bitmask ขั้นสูง

มีเทคนิคหลายอย่างที่ใช้ประโยชน์จากคุณสมบัติของเลขฐานสองได้อย่างชาญฉลาด:

```
// S & (S-1): Clear the lowest set bit
// Example: S = 10110, S-1 = 10101, S & (S-1) = 10100
int S = 22; // 10110
int cleared = S & (S - 1); // 10100 = 20
// S | (S+1): Fill the lowest zero bit
// Example: S = 10101, S+1 = 10110, S | (S+1) = 10111
int S2 = 21; // 10101
int filled = S2 | (S2 + 1); // 10111 = 23
// S & (S+1): Clear all trailing 1s
// Example: S = 10011, S+1 = 10100, S & (S+1) = 10000
int S3 = 19; // 10011
int clearedTrailing = S3 & (S3 + 1); // 10000 = 16
// Count number of set bits
int popcount = __builtin_popcount(S);       // For int
int popcount_ll = __builtin_popcountll(S); // For long long
// Get lowest set bit value
int lowestBit = S & (-S);
// Check if S is a power of 2
bool isPow2 = (S & (S - 1)) == 0 && S > 0;
// Iterate over all set bits
int T = S;
while (T) {
    int bit = T & (-T);          // Get lowest set bit
    int i = __builtin_ctz(T); // Get index of lowest set bit
    // ... process bit i ...
```

```
    T &= (T - 1);                    // Clear lowest set bit
}
```

ตัวอย่างโค้ดที่ 5.33: เทคนิค Bitmask ขั้นสูง

builtin functions ของ GCC

GCC มีฟังก์ชันในตัวที่มีประโยชน์มากสำหรับ Bitmask:

- `__builtin_popcount(x)` — นับจำนวนบิตที่เป็น 1 (สำหรับ `int`)
- `__builtin_popcountll(x)` — นับจำนวนบิตที่เป็น 1 (สำหรับ `long long`)
- `__builtin_ctz(x)` — ตำแหน่งบิต 1 ต่ำสุด (Count Trailing Zeros)
- `__builtin_clz(x)` — จำนวนเลขศูนย์นำหน้า (Count Leading Zeros)

ฟังก์ชันเหล่านี้ทำงานได้เร็วมาก มักใช้เวลาเพียง 1–2 รอบสัญญาณนาฬิกา

5.10.4 การวนซ้ำผ่าน Subset ทั้งหมด

การวนซ้ำผ่าน subset ทั้งหมดของเซตขนาด n เป็นเทคนิคที่ใช้บ่อยมากในการแข่งขัน:

```
// Enumerate all subsets of a set of size n
// There are 2^n subsets total
int n = 4;
for (int mask = 0; mask < (1 << n); mask++) {
    // mask represents a subset
    // bit i is 1 means element i is in the subset
    cout << "Subset: ";
    for (int i = 0; i < n; i++) {
        if ((mask >> i) & 1) {
            cout << i << " ";
        }
    }
    cout << "\n";
}
// Output: {}, {0}, {1}, {0,1}, {2}, {0,2}, ...
// Total: 2^4 = 16 subsets
// Enumerate all subsets of a given bitmask S
int S = 13; // {0, 2, 3}
int sub = S;
do {
    // Process subset 'sub'
    cout << sub << "\n";
```

```
    if (sub == 0) break;
    sub = (sub - 1) & S;
} while (sub != S);
// Iterates: 13, 9, 5, 4, 1, 0
// i.e., {0,2,3}, {0,3}, {0,2}, {2}, {0}, {}
```

ตัวอย่างโค้ดที่ 5.34: วนซ้ำผ่าน subset ทั้งหมด

ตัวอย่าง: นับจำนวนวิธีเลือกสมาชิกจากเซต

```
// Count ways to choose k elements from n elements
// Using bitmask to enumerate and count
int n = 20, k = 3;
long long count = 0;
for (int mask = 0; mask < (1 << n); mask++) {
    if (__builtin_popcount(mask) == k) {
        count++;
    }
}
// Result: C(20, 3) = 1140
// Note: This approach is O(2^n * n)
// For larger n, use combinatorics formula C(n,k) directly
```

Bitmask เชื่อมโยงกับบทถัดไป

Bitmask เป็นเครื่องมือพื้นฐานที่สำคัญมากสำหรับเทคนิคขั้นสูงในบทถัดไป:

- **บทที่ 6 (Backtracking):** ใช้ Bitmask ติดตามสมาชิกที่เลือกแล้ว เช่น N-Queens, Hamiltonian Path
- **บทที่ 7 (Dynamic Programming):** ใช้ Bitmask เป็นสถานะของ DP (DP+Bitmask) เช่น Traveling Salesman Problem (TSP) ซึ่งสถานะคือ `dp[mask][i]` แทนค่าต่ำสุดเมื่อเยือนเซต `mask` และจบที่เมือง i

ควรฝึกใช้ Bitmask ให้คล่อง เพราะจะได้ใช้อีกมากในบทถัดไป!

5.11 Segment Tree

ในหัวข้อก่อนหน้านี้ เราได้ศึกษาโครงสร้างข้อมูลจาก STL ที่รองรับการดำเนินการพื้นฐาน เช่น การเพิ่ม ลบ ค้นหา แต่ในหลายโจทย์ เราต้องการ **สอบถามข้อมูลบนช่วง (Range Query)** เช่น หาผลบวกของข้อมูลในช่วง $[l, r]$ หรือหาค่าต่ำสุดในช่วงดังกล่าว และยังต้อง **แก้ไขข้อมูล**ได้อีกด้วย

ถ้าใช้แถวลำดับธรรมดา การสอบถามช่วงใช้เวลา $\mathcal{O}(n)$ แต่ถ้าใช้ Prefix Sum การสอบถามใช้เวลา $\mathcal{O}(1)$

แต่การแก้ไขข้อมูลใช้เวลา $\mathcal{O}(n)$ Segment Tree เป็นโครงสร้างข้อมูลที่แก้ปัญหานี้โดยรองรับ **ทั้งสองการดำเนินการในเวลา** $\mathcal{O}(\log n)$[3, 11]

5.11.1 โครงสร้างของ Segment Tree

Segment Tree เป็นต้นไม้ทวิภาค (Binary Tree) ที่แต่ละ node เก็บผลลัพธ์ของข้อมูลบนช่วง (Interval) หนึ่ง:

- **Root node** เก็บผลลัพธ์ของข้อมูลทั้งหมด (ช่วง $[0, n-1]$)
- **Leaf node** แต่ละอันเก็บค่าของข้อมูลตัวเดียว
- **Internal node** เก็บผลลัพธ์ที่รวมจากลูกซ้ายและลูกขวา

การดำเนินการที่รองรับขึ้นอยู่กับว่าเรา “รวม” ผลลัพธ์จากลูกอย่างไร:

- **Range Sum Query (RSQ):** ลูกซ้าย + ลูกขวา
- **Range Minimum Query (RMQ):** min(ลูกซ้าย, ลูกขวา)
- **Range Maximum Query:** max(ลูกซ้าย, ลูกขวา)

ความซับซ้อนของเวลา:

- **สร้าง (Build):** $\mathcal{O}(n)$
- **สอบถามช่วง (Query):** $\mathcal{O}(\log n)$
- **แก้ไขจุด (Point Update):** $\mathcal{O}(\log n)$

5.11.2 การ implement Segment Tree แบบแถวลำดับ

วิธีที่นิยมที่สุดในการ implement Segment Tree คือใช้แถวลำดับขนาด $4n$ (เพียงพอสำหรับต้นไม้สูง $\lceil \log_2 n \rceil + 1$ ระดับ) โดย:

- Node ที่ตำแหน่ง p มีลูกซ้ายที่ $2p$ และลูกขวาที่ $2p+1$
- Root node อยู่ที่ตำแหน่ง 1

ตัวอย่างต่อไปนี้เป็น Segment Tree สำหรับ **Range Sum Query** พร้อม Point Update:

```
#include <vector>
using namespace std;
class SegTree {
private:
    int n;
    vector<int> tree; // tree[1] is root, size = 4*n
    // Build tree from array a[]
```

```
    void build(const vector<int>& a, int p, int lo, int hi) {
        if (lo == hi) {                // Leaf node
            tree[p] = a[lo];
            return;
        }
        int mid = (lo + hi) / 2;
        build(a, 2*p, lo, mid);        // Build left child
        build(a, 2*p+1, mid+1, hi);    // Build right child
        tree[p] = tree[2*p] + tree[2*p+1]; // Combine
    }
    // Range sum query on [l, r]
    int query(int p, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return 0;       // Outside range
        if (l <= lo && hi <= r) return tree[p]; // Fully inside range
        int mid = (lo + hi) / 2;
        return query(2*p, lo, mid, l, r)
            + query(2*p+1, mid+1, hi, l, r); // Combine both sides
    }
    // Point update: set a[idx] = val
    void update(int p, int lo, int hi, int idx, int val) {
        if (lo == hi) {                // Leaf node
            tree[p] = val;
            return;
        }
        int mid = (lo + hi) / 2;
        if (idx <= mid)
            update(2*p, lo, mid, idx, val);
        else
            update(2*p+1, mid+1, hi, idx, val);
        tree[p] = tree[2*p] + tree[2*p+1]; // Recombine
    }
public:
    SegTree(const vector<int>& a) {
        n = a.size();
        tree.assign(4*n, 0);
        build(a, 1, 0, n-1);
    }
    int query(int l, int r) {
        return query(1, 0, n-1, l, r);
    }
    void update(int idx, int val) {
```

```
        update(1, 0, n-1, idx, val);
    }
};
```

ตัวอย่างโค้ดที่ 5.35: Segment Tree สำหรับ Range Sum Query

5.11.3 ตัวอย่างการใช้งาน Segment Tree

```
int main() {
    vector<int> a = {1, 3, 5, 7, 9, 11};
    // Indices:  0 1 2 3 4 5
    SegTree st(a);
    // Range sum query
    cout << st.query(1, 3) << "\n";  // 3+5+7 = 15
    cout << st.query(0, 5) << "\n";  // 1+3+5+7+9+11 = 36
    cout << st.query(2, 2) << "\n";  // 5 (single element)
    // Point update
    st.update(1, 10);  // a[1] = 10 (was 3)
    cout << st.query(1, 3) << "\n";  // 10+5+7 = 22
    st.update(4, 0);  // a[4] = 0 (was 9)
    cout << st.query(0, 5) << "\n";  // 1+10+5+7+0+11 = 34
    return 0;
}
```

ตัวอย่างโค้ดที่ 5.36: ตัวอย่างการใช้ Segment Tree

5.11.4 เมื่อไหร่ควรใช้ Segment Tree

Segment Tree เป็นโครงสร้างข้อมูลที่ทรงพลัง แต่ไม่ควรใช้ในทุกกรณี ควรพิจารณาดังนี้:

- ถ้า **ไม่ต้องแก้ไขข้อมูล** และต้องการ RSQ เท่านั้น $\rightarrow$ ใช้ **Prefix Sum** (สอบถาม $\mathcal{O}(1)$ สร้าง $\mathcal{O}(n)$)
- ถ้า **ไม่ต้องแก้ไขข้อมูล** และต้องการ RMQ เท่านั้น $\rightarrow$ ใช้ **Sparse Table** (สอบถาม $\mathcal{O}(1)$ สร้าง $\mathcal{O}(n \log n)$)
- ถ้า **ต้องแก้ไขข้อมูล** และต้องการ RSQ/RMQ $\rightarrow$ ใช้ **Segment Tree** หรือ **Fenwick Tree**
- ถ้าต้องการ **Range Update** (แก้ไขทั้งช่วง) $\rightarrow$ ใช้ Segment Tree + Lazy Propagation

Segment Tree vs Fenwick Tree

ทั้ง Segment Tree และ Fenwick Tree รองรับ RSQ พร้อม Point Update ในเวลา $\mathcal{O}(\log n)$ แต่มีข้อแตกต่าง:

- **Fenwick Tree:** โค้ดสั้นกว่ามาก (ประมาณ 10 บรรทัด) เหมาะสำหรับ RSQ อย่างเดียว
- **Segment Tree:** โค้ดยาวกว่า แต่ยืดหยุ่นกว่า — รองรับ RMQ, Range Update (Lazy Propagation), และสอบถามที่ซับซ้อนกว่าได้
- **คำแนะนำ:** ถ้าโจทย์ต้องการเพียง RSQ ให้ใช้ Fenwick Tree (เร็วและโค้ดสั้น) แต่ถ้าต้องการ RMQ หรือการดำเนินการที่ซับซ้อนกว่า ให้ใช้ Segment Tree

5.12 Fenwick Tree (Binary Indexed Tree)

Fenwick Tree หรืออีกชื่อคือ Binary Indexed Tree (BIT) เป็นโครงสร้างข้อมูลที่ทำหน้าที่เดียวกับ Segment Tree สำหรับ **Range Sum Query (RSQ)** พร้อม Point Update แต่มีข้อดีคือ **โค้ดสั้นกว่ามาก** และค่าคงที่ (Constant Factor) ต่ำกว่า ทำให้ทำงานได้เร็วกว่าในทางปฏิบัติ[3, 11]

5.12.1 แนวคิดหลักของ Fenwick Tree

แนวคิดสำคัญของ Fenwick Tree คือ: **แต่ละดัชนี i ในแถวลำดับจะรับผิดชอบช่วงของข้อมูลที่มีความยาวเท่ากับ Lowest Set Bit ของ i**

ตัวอย่างเช่น:

- ดัชนี $i = 6 = 110_{(2)}$ มี Lowest Set Bit ที่ตำแหน่ง 1 (ค่า $2^1 = 2$) ดังนั้น `tree[6]` รับผิดชอบช่วง $[5, 6]$ (2 ตัว)
- ดัชนี $i = 8 = 1000_{(2)}$ มี Lowest Set Bit ที่ตำแหน่ง 3 (ค่า $2^3 = 8$) ดังนั้น `tree[8]` รับผิดชอบช่วง $[1, 8]$ (8 ตัว)
- ดัชนี $i = 5 = 101_{(2)}$ มี Lowest Set Bit ที่ตำแหน่ง 0 (ค่า $2^0 = 1$) ดังนั้น `tree[5]` รับผิดชอบเพียงตัวเดียว คือ $[5, 5]$

เราสามารถหาค่า Lowest Set Bit ได้อย่างง่ายดายด้วยสูตร:

$$\text{lsOne}(i) = i \,\&\, (-i)$$

ตัวอย่าง: $\text{lsOne}(6) = 6 \,\&\, (-6) = 110_{(2)} \,\&\, 010_{(2)} = 2$

5.12.2 การ implement Fenwick Tree

Fenwick Tree ใช้แถวลำดับขนาด $n + 1$ (ดัชนีเริ่มจาก 1) และมีเพียง 2 ฟังก์ชันหลัก:

```
#include <vector>
using namespace std;
class FenwickTree {
```

```
private:
    int n;
    vector<int> tree; // tree[1..n], index starts from 1
public:
    // Constructor: create Fenwick Tree of size n (all zeros)
    FenwickTree(int n) : n(n), tree(n + 1, 0) {}
    // Constructor: build from an array a[1..n]
    FenwickTree(const vector<int>& a) {
        n = a.size() - 1; // a is 1-indexed
        tree.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            tree[i] += a[i];
            int j = i + (i & (-i)); // Add to parent
            if (j <= n) tree[j] += tree[i];
        }
    }
    // Point update: add delta to a[i]
    void update(int i, int delta) {
        for (; i <= n; i += (i & (-i)))
            tree[i] += delta;
    }
    // Prefix sum query: sum of a[1..i]
    int prefixSum(int i) {
        int sum = 0;
        for (; i > 0; i -= (i & (-i)))
            sum += tree[i];
        return sum;
    }
    // Range sum query: sum of a[l..r] (1-indexed)
    int rsq(int l, int r) {
        return prefixSum(r) - prefixSum(l - 1);
    }
};
```

ตัวอย่างโค้ดที่ 5.37: Fenwick Tree สำหรับ Range Sum Query

5.12.3 ตัวอย่างการใช้งาน Fenwick Tree

```
int main() {
    // 1-indexed array: a[1..6] = {1, 3, 5, 7, 9, 11}
    vector<int> a = {0, 1, 3, 5, 7, 9, 11}; // a[0] unused
    FenwickTree ft(a);
```

```
    // Range sum query
    cout << ft.rsq(1, 3) << "\n";    // 1+3+5 = 9
    cout << ft.rsq(2, 5) << "\n";    // 3+5+7+9 = 24
    cout << ft.rsq(1, 6) << "\n";    // 1+3+5+7+9+11 = 36
    cout << ft.rsq(3, 3) << "\n";    // 5 (single element)
    // Point update: a[2] += 4 (a[2] changes from 3 to 7)
    ft.update(2, 4);
    cout << ft.rsq(1, 3) << "\n";    // 1+7+5 = 13
    // Point update: a[5] -= 5 (a[5] changes from 9 to 4)
    ft.update(5, -5);
    cout << ft.rsq(1, 6) << "\n";    // 1+7+5+7+4+11 = 35
    return 0;
}
```

ตัวอย่างโค้ดที่ 5.38: ตัวอย่างการใช้ Fenwick Tree

5.12.4 เปรียบเทียบ Segment Tree กับ Fenwick Tree

ตารางต่อไปนี้เปรียบเทียบ Segment Tree กับ Fenwick Tree อย่างครบถ้วน:

ตารางที่ 5.8 เปรียบเทียบ Segment Tree กับ Fenwick Tree

คุณสมบัติ	**Segment Tree**	**Fenwick Tree**
ขนาดแถวลำดับ	$4n$	$n+1$
โค้ดความยาว	ประมาณ 40–50 บรรทัด	ประมาณ 10–15 บรรทัด
RSQ + Point Update	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
RMQ	รองรับ	**ไม่รองรับ**
Range Update	รองรับ (Lazy Propagation)	จำกัด (บางกรณีได้)
ค่าคงที่ (Constant)	ใหญ่กว่า	เล็กกว่า (เร็วกว่า)
การขยายไป 2 มิติ	ได้	ได้

ควรใช้อันไหน?

- ถ้าโจทย์ต้องการ **RSQ + Point Update** อย่างเดียว → ใช้ **Fenwick Tree** เพราะโค้ดสั้น เขียนเร็ว และทำงานเร็วกว่า
- ถ้าโจทย์ต้องการ **RMQ** หรือการดำเนินการอื่นที่ไม่ใช่ผลบวก → ต้องใช้ **Segment Tree**
- ถ้าโจทย์ต้องการ **Range Update** (แก้ไขทั้งช่วง) → ใช้ **Segment Tree + Lazy Propagation**
- ในการแข่งขันจริง **Fenwick Tree** คือค่าเริ่มต้นสำหรับ RSQ เพราะเขียนได้ภายใน 2 นาที!

5.13 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนการใช้โครงสร้างข้อมูล:

ตารางที่ 5.9 โจทย์แนะนำสำหรับบทที่ 5

แพลตฟอร์ม	โจทย์	โครงสร้างข้อมูลที่ใช้
Codeforces[10]	1462A	vector
Codeforces	1547B	stack
Codeforces	1582C	map
Codeforces	1612B	set
Codeforces	1367C	queue
AtCoder	ABC217 D	set (lower_bound)
AtCoder	ABC228 E	bitset
Codeforces	1373D	priority_queue
AtCoder	ABC218 D	set
Codeforces	1691B	deque

คำแนะนำการฝึกฝน

1. **เริ่มจากโจทย์ง่าย:** ฝึกใช้ vector กับ string จนคล่องก่อน แล้วค่อยขยับไป set/map
2. **อ่าน constraints:** ก่อนเขียนโค้ด ให้ดูว่าโจทย์ต้องการ operations อะไร และมีข้อจำกัดอะไร แล้วเลือกโครงสร้างข้อมูลให้เหมาะสม
3. **ฝึก lower_bound:** ฟังก์ชัน `lower_bound` ปรากฏในโจทย์บ่อยมาก ควรฝึกจนคล่องทั้งกับ vector และ set
4. **ทำโจทย์หลากหลาย:** แต่ละโจทย์ใช้โครงสร้างข้อมูลต่างกัน ยิ่งทำหลายโจทย์ ยิ่งเข้าใจว่าเมื่อไหร่ควรใช้อะไร

5.14 สรุปบทที่ 5

ในบทนี้ เราได้ศึกษาโครงสร้างข้อมูลที่สำคัญที่สุดสำหรับการโปรแกรมเชิงแข่งขัน ตั้งแต่แถวลำดับและ Vector ไปจนถึง Set, Map, Priority Queue และอื่นๆ แต่ละโครงสร้างมีจุดแข็งและจุดอ่อนที่แตกต่างกัน การเลือกโครงสร้างข้อมูลที่เหมาะสมกับโจทย์เป็นทักษะสำคัญที่ต้องฝึกฝน

5.14.1 ตารางสรุปโครงสร้างข้อมูลทั้งหมด

ตารางที่ 5.10 สรุปโครงสร้างข้อมูลทั้งหมดในบทที่ 5

โครงสร้าง	Header	การดำเนินการสำคัญ	หมายเหตุ
vector	<vector>	push_back, pop_back, []	Dynamic array ยืดหยุ่น
string	<string>	+, substr, find	Dynamic array ของ char
bitset	<bitset>	[], count, &, |, ^	ประหยัดหน่วยความจำ
stack	<stack>	push, pop, top	LIFO
queue	<queue>	push, pop, front	FIFO
deque	<deque>	push_front/back, pop_front/back	ทั้งสองด้าน
set	<set>	insert, erase, count, lower_bound	ไม่ซ้ำ, เรียงลำดับ
multiset	<set>	insert, erase, count	ซ้ำได้, เรียงลำดับ
unordered_set	<unordered_set>	insert, erase, count	ไม่ซ้ำ, $\mathcal{O}(1)$ เฉลี่ย
map	<map>	[], count, lower_bound	key-value, เรียงตาม key
unordered_map	<unordered_map>	[], count	key-value, $\mathcal{O}(1)$ เฉลี่ย
priority_queue	<queue>	push, pop, top	Max/Min Heap
list	<list>	push_front/back, insert, erase	Doubly Linked List

5.14.2 ตารางเปรียบเทียบความซับซ้อน

ตารางที่ 5.11 เปรียบเทียบความซับซ้อนของการดำเนินการ

โครงสร้าง	Access	Search	Insert	Delete
`vector/string`	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)^{*}$ / $\mathcal{O}(n)$	$\mathcal{O}(n)$
`bitset`	$\mathcal{O}(1)$	—	—	—
`stack/queue`	$\mathcal{O}(1)^{**}$	—	$\mathcal{O}(1)$	$\mathcal{O}(1)$
`deque`	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
`set/map`	—	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
`unordered_set/map`	—	$\mathcal{O}(1)^{***}$	$\mathcal{O}(1)^{***}$	$\mathcal{O}(1)^{***}$
`priority_queue`	$\mathcal{O}(1)^{**}$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
`list`	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)^{****}$	$\mathcal{O}(1)^{****}$

*Amortized **เฉพาะ top/front/back ***เฉลี่ย, เลวร้าย $\mathcal{O}(n)$ ****ด้วย iterator

5.14.3 คู่มือเลือกใช้

การเลือกโครงสร้างข้อมูลที่เหมาะสมสามารถทำได้โดยพิจารณาจากการดำเนินการที่โจทย์ต้องการ:

1. **ต้องการเข้าถึงด้วยดัชนี:** ใช้ `vector` หรือ `deque`
2. **ต้องการเพิ่ม/ลบที่ท้าย:** ใช้ `vector`
3. **ต้องการเพิ่ม/ลบทั้งสองด้าน:** ใช้ `deque`
4. **ต้องการตรวจสอบการมีอยู่ของข้อมูล:** ใช้ `set` หรือ `unordered_set`
5. **ต้องการหาค่าที่ใกล้ที่สุด:** ใช้ `set` (มี `lower_bound`)
6. **ต้องการเก็บ key-value:** ใช้ `map` หรือ `unordered_map`
7. **ต้องการดึงค่ามากสุด/น้อยสุด:** ใช้ `priority_queue`
8. **ต้องการข้อมูลเรียงลำดับ:** ใช้ `set` หรือ `map`
9. **ต้องการจัดการข้อมูลไบนารี:** ใช้ `bitset`
10. **ต้องการ LIFO/FIFO:** ใช้ `stack/queue`

เคล็ดลับการเลือกโครงสร้างข้อมูล

1. **เริ่มจาก vector:** ถ้าไม่แน่ใจ ใช้ `vector` ก่อน เพราะมันยืดหยุ่นและเร็ว
2. **ดู operations ที่ทำบ่อย:** ถ้าต้องการ `lower_bound` ใช้ `set/map` ถ้าต้องการ $\mathcal{O}(1)$ ค้นหา

ใช้ `unordered_set/map`

3. **ดู constraints:** ถ้า $n \leq 10^5$ และต้องการค้นหา $\mathcal{O}(\log n)$ ก็เพียงพอ ถ้าต้องการ $\mathcal{O}(1)$ อาจต้องใช้ hash-based structure
4. **อย่า over-engineer:** ถ้าแถวลำดับง่ายๆ เพียงพอ ไม่ต้องใช้โครงสร้างที่ซับซ้อนกว่า

Chapter 6

การสอบถามแบบช่วง (Range Queries)

การสอบถามแบบช่วง (Range Query) เป็นหนึ่งในปัญหาพื้นฐานที่พบได้บ่อยที่สุดในการแข่งขันเขียนโปรแกรม โดยเราต้องการคำนวณค่าบางอย่างจากช่วงย่อย (subarray) ของแถวลำดับ เช่น ผลรวม ค่าน้อยสุด หรือค่ามากสุดในช่วง $[a, b]$ แม้วิธีการง่ายๆ อย่างการวนซ้ำตรวจสอบทุกค่าจะตอบคำถามได้ แต่เมื่อขนาดข้อมูลและจำนวนคำถามมากขึ้น เราจำเป็นต้องใช้โครงสร้างข้อมูลที่มีประสิทธิภาพมากกว่า ในบทนี้ เราจะศึกษาโครงสร้างข้อมูลหลายชนิดที่ช่วยให้เราตอบคำถามแบบช่วงได้อย่างรวดเร็ว ตั้งแต่ Prefix Sum Array สำหรับแถวลำดับคงที่ ไปจนถึง Segment Tree และ Binary Indexed Tree สำหรับแถวลำดับที่มีการเปลี่ยนแปลงค่าระหว่างการสอบถาม[3, 11]

6.1 บทนำสู่ Range Queries

กำหนดแถวลำดับ $a[0 \ldots n-1]$ การสอบถามแบบช่วงที่พบบ่อยที่สุดมี 3 ประเภท ได้แก่:

- sumq(a, b): คำนวณผลรวมของสมาชิกในช่วง $[a, b]$ กล่าวคือ $a[a] + a[a+1] + \cdots + a[b]$
- minq(a, b): หาค่าน้อยสุดในช่วง $[a, b]$ กล่าวคือ min$(a[a], a[a+1], \ldots, a[b])$
- maxq(a, b): หาค่ามากสุดในช่วง $[a, b]$ กล่าวคือ max$(a[a], a[a+1], \ldots, a[b])$

เพื่อให้เห็นภาพ พิจารณาแถวลำดับต่อไปนี้:

1	8	7	1	2	9	2	6
0	1	2	3	4	5	6	7

หากเราสอบถามช่วง $[3, 6]$ จะได้ว่า:

- sumq$(3, 6) = 1 + 2 + 9 + 2 = 14$
- minq$(3, 6) = 1$
- maxq$(3, 6) = 9$

วิธีการง่ายที่สุดในการตอบคำถามเหล่านี้คือการวนซ้ำตรวจสอบทุกค่าในช่วงที่สนใจ ตัวอย่างเช่น ฟังก์ชันสำหรับหาผลรวม:

```cpp
int sum(int a, int b) {
    int s = 0;
    for (int i = a; i <= b; i++) {
        s = s + arr[i];
    }
    return s;
}
```

ตัวอย่างโค้ดที่ 6.1: วิธีการง่ายๆ สำหรับ Sum Query

ฟังก์ชันนี้ทำงานในเวลา $\mathcal{O}(n)$ ต่อการสอบถามหนึ่งครั้ง ดังนั้นหากมี q คำถาม จะใช้เวลารวม $\mathcal{O}(qn)$ ซึ่งช้ามากเมื่อ n และ q มีขนาดใหญ่ (เช่น $n = q = 10^5$) เราจึงต้องหาวิธีที่มีประสิทธิภาพกว่านี้

ปัญหา Range Queries สามารถแบ่งออกเป็น 2 ประเภทหลัก ขึ้นอยู่กับว่าแถวลำดับมีการเปลี่ยนแปลงค่าระหว่างการสอบถามหรือไม่:

1. **Static Array Queries:** ค่าในแถวลำดับไม่เปลี่ยนแปลง เราสามารถประมวลผลล่วงหน้า (preprocess) ได้ครั้งเดียว แล้วตอบคำถามได้อย่างรวดเร็ว

2. **Dynamic Array Queries:** ค่าในแถวลำดับเปลี่ยนแปลงได้ระหว่างการสอบถาม โครงสร้างข้อมูลต้องรองรับทั้งการอัปเดตและการสอบถามอย่างมีประสิทธิภาพ

ในบทนี้ เราจะศึกษาโครงสร้างข้อมูลสำหรับทั้งสองกรณี โดยเริ่มจากกรณี Static ก่อน แล้วจึงพิจารณากรณี Dynamic ต่อไป

6.2 Static Array Queries

เราจะเริ่มพิจารณากรณีที่แถวลำดับเป็น Static หรือค่าในแถวลำดับไม่เปลี่ยนแปลงระหว่างการสอบถาม ในกรณีนี้ เราสามารถสร้างโครงสร้างข้อมูลที่ไม่เปลี่ยนแปลง (คือสร้างแล้วไม่อัปเดตทีหลัง) เพื่อมาช่วยตอบคำถามของเราได้อย่างรวดเร็ว

6.2.1 Prefix Sum Array (Sum Queries)

สำหรับ Sum Queries บน Static Array เราสามารถใช้ **Prefix Sum Array** (หรืออาจเคยได้ยินชื่อว่า Quick Sum) เพื่อตอบคำถามได้ในเวลา $\mathcal{O}(1)$ ต่อคำถาม

นิยาม 6.1 (Prefix Sum Array)**.** กำหนดแถวลำดับ $a[0 \ldots n-1]$ Prefix Sum Array $p[0 \ldots n-1]$ นิยามว่า

$$p[k] = a[0] + a[1] + \cdots + a[k] = \sum_{i=0}^{k} a[i]$$

กล่าวคือ $p[k]$ เก็บผลรวมของสมาชิกตั้งแต่ช่องแรกจนถึงช่องที่ k

สามารถสร้าง Prefix Sum Array ได้ในเวลา $\mathcal{O}(n)$ ด้วยสูตร:

$$p[0] = a[0], \quad p[k] = p[k-1] + a[k] \text{ สำหรับ } k \geq 1$$

```cpp
vector<long long> buildPrefixSum(vector<int>& a) {
    int n = a.size();
    vector<long long> p(n);
    p[0] = a[0];
    for (int i = 1; i < n; i++)
        p[i] = p[i - 1] + a[i];
    return p;
}
```

ตัวอย่างโค้ดที่ 6.2: สร้าง Prefix Sum Array

ตัวอย่างเช่น จากแถวลำดับ:

	0	1	2	3	4	5	6	7
a:	1	8	7	1	2	9	2	6
p:	1	9	16	17	19	28	30	36

เมื่อมี Prefix Sum Array แล้ว เราสามารถคำนวณ sumq(a, b) ได้ในเวลา $\mathcal{O}(1)$ ด้วยสูตร:

$$\text{sumq}(a, b) = p[b] - p[a-1]$$

โดยกำหนดให้ $p[-1] = 0$ เพื่อจัดการกรณีที่ $a = 0$

```cpp
long long sumQuery(vector<long long>& p, int a, int b) {
    if (a == 0) return p[b];
    return p[b] - p[a - 1];
}
```

ตัวอย่างโค้ดที่ 6.3: Sum Query ด้วย Prefix Sum

จากตัวอย่าง sumq$(3, 6) = p[6] - p[2] = 30 - 16 = 14$ ซึ่งตรงกับการคำนวณโดยตรง

Prefix Sum กับจำนวนเต็มขนาดใหญ่

ผลรวมของสมาชิกในแถวลำดับอาจมีค่ามาก โดยเฉพาะเมื่อ n มีขนาดใหญ่ เช่น ถ้า $n = 10^5$ และสมาชิกแต่ละตัวมีค่าถึง 10^9 ผลรวมอาจสูงถึง 10^{14} ซึ่งเกินขอบเขตของ `int` ดังนั้นควรใช้ `long long` สำหรับ Prefix Sum Array เสมอ

6.2.2 Prefix Sum 2 มิติ

หลักการของ Prefix Sum สามารถขยายไปยัง 2 มิติได้ ซึ่งมีประโยชน์มากในการหาผลรวมของพื้นที่สี่เหลี่ยมใดๆ ในตาราง 2 มิติได้ในเวลา $\mathcal{O}(1)$

กำหนดตาราง $a[0 \ldots n-1][0 \ldots m-1]$ Prefix Sum 2 มิติ $p[i][j]$ นิยามว่า:

$$p[i][j] = \sum_{r=0}^{i} \sum_{c=0}^{j} a[r][c]$$

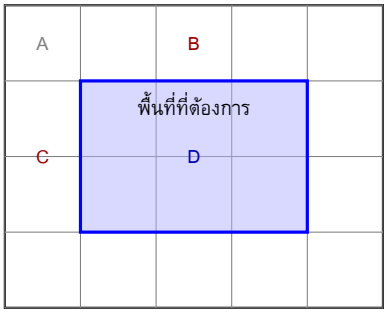
สามารถคำนวณได้จากสูตร:

$$p[i][j] = a[i][j] + p[i-1][j] + p[i][j-1] - p[i-1][j-1]$$

และผลรวมในพื้นที่สี่เหลี่ยมจากแถว r_1 ถึง r_2 และคอลัมน์ c_1 ถึง c_2 คือ:

$$\text{sum} = p[r_2][c_2] - p[r_1-1][c_2] - p[r_2][c_1-1] + p[r_1-1][c_1-1]$$

รูปที่ 6.1 แสดงหลักการของ Prefix Sum 2 มิติ โดยพื้นที่สี่เหลี่ยมที่ต้องการหาผลรวม สามารถคำนวณได้จากการบวกลบพื้นที่ย่อยที่เริ่มจากมุมบนซ้าย



$\text{sum} = D - B - C + A$

รูปที่ 6.1 หลักการ Prefix Sum 2 มิติ: ผลรวมในพื้นที่สี่เหลี่ยมใดๆ คำนวณได้จากการบวกลบพื้นที่ย่อย

6.2.3 Minimum Queries บน Static Array

Minimum Queries ยากกว่า Sum Queries เพราะไม่สามารถใช้การลบกลับได้เหมือน Prefix Sum อย่างไรก็ตาม เรายังสามารถประมวลผลล่วงหน้าเพื่อตอบคำถามได้อย่างมีประสิทธิภาพ ทั้งนี้ Maximum Queries สามารถแก้ได้ในลักษณะเดียวกับ Minimum Queries ดังนั้นในส่วนนี้จะกล่าวถึงเฉพาะ Minimum Queries

วิธีเก็บตารางโดยตรง ($\mathcal{O}(n^2)$ preprocess, $\mathcal{O}(1)$ query)

วิธีที่ตรงไปตรงมาที่สุดคือสร้างตารางขนาด $n \times n$ โดย `lookup[i][j]` เก็บดัชนีของค่าน้อยสุดในช่วง $[i, j]$

```
int memo[MAXN][MAXN];
void preprocess(int arr[], int n) {
```

```
    for (int i = 0; i < n; i++)
        memo[i][i] = i;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[memo[i][j - 1]] < arr[j])
                memo[i][j] = memo[i][j - 1];
            else
                memo[i][j] = j;
        }
    }
}
int minQuery(int arr[], int a, int b) {
    return arr[memo[a][b]];
}
```

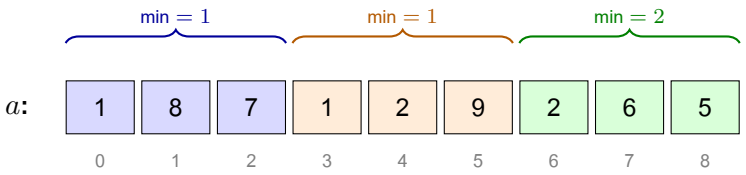
ตัวอย่างโค้ดที่ 6.4: สร้าง Lookup Table สำหรับ Minimum Query

วิธีนี้ใช้เวลา $\mathcal{O}(n^2)$ ในการสร้างตาราง แต่สอบถามได้ใน $\mathcal{O}(1)$ เนื่องจากใช้หน่วยความจำ $\mathcal{O}(n^2)$ จึงไม่เหมาะเมื่อ n มีขนาดใหญ่ (เช่น $n = 10^5$ จะต้องใช้หน่วยความจำ ≈ 40 GB)

Square Root Decomposition ($\mathcal{O}(n)$ preprocess, $\mathcal{O}(\sqrt{n})$ query)

Square Root Decomposition เป็นเทคนิคที่แบ่งแถวลำดับออกเป็นก้อน (block) ขนาด $\sqrt{n}$ แต่ละก้อน และเก็บค่าน้อยสุดของแต่ละก้อนไว้

รูปที่ 6.2 แสดงการแบ่งแถวลำดับออกเป็นก้อนด้วย Square Root Decomposition



รูปที่ 6.2 Square Root Decomposition: แบ่งแถวลำดับออกเป็นก้อนละ $\sqrt{n}$ ตัว และเก็บค่าน้อยสุดของแต่ละก้อน

เมื่อสอบถามช่วง $[L, R]$ เราจะ:

1. ตรวจสอบสมาชิกในก้อนแรกที่ทับช่วงเพียงบางส่วน (ไม่เกิน $\sqrt{n}$ ตัว) ด้วย Linear Search
2. ใช้ค่าน้อยสุดของก้อนกลางที่ทับช่วงทั้งก้อนโดยตรง (มีไม่เกิน $\sqrt{n}$ ก้อน)
3. ตรวจสอบสมาชิกในก้อนสุดท้ายที่ทับช่วงเพียงบางส่วน (ไม่เกิน $\sqrt{n}$ ตัว) ด้วย Linear Search

ดังนั้นเวลาในการสอบถามคือ $\mathcal{O}(\sqrt{n})$ และใช้พื้นที่ $\mathcal{O}(\sqrt{n})$ สำหรับเก็บค่าน้อยสุดของแต่ละก้อน

```cpp
int arr[MAXN], block[MAXSQR];
int blk_sz;
void preprocess(int input[], int n) {
    blk_sz = sqrt(n);
    int blk_idx = -1;
    for (int i = 0; i < n; i++) {
        arr[i] = input[i];
        if (i % blk_sz == 0) {
            blk_idx++;
            block[blk_idx] = input[i];
        }
        if (input[i] < block[blk_idx])
            block[blk_idx] = input[i];
    }
}
int query(int l, int r) {
    int m = arr[l];
    // Traverse first partial block
    while (l < r && l % blk_sz != 0) {
        if (arr[l] < m) m = arr[l];
        l++;
    }
    // Traverse fully overlapped blocks
    while (l + blk_sz <= r) {
        if (block[l / blk_sz] < m) m = block[l / blk_sz];
        l += blk_sz;
    }
    // Traverse last partial block
    while (l <= r) {
        if (arr[l] < m) m = arr[l];
        l++;
    }
    return m;
}
```

ตัวอย่างโค้ดที่ 6.5: Square Root Decomposition สำหรับ Minimum Query

Square Root Decomposition รองรับการอัปเดต

ข้อดีของ Square Root Decomposition คือรองรับการอัปเดตค่าได้ โดยเมื่อเปลี่ยนค่า `arr[k]` เราเพียงคำนวณค่าน้อยสุดของก้อนที่ k สังกัดใหม่ ซึ่งใช้เวลา $\mathcal{O}(\sqrt{n})$

Sparse Table ($\mathcal{O}(n \log n)$ preprocess, $\mathcal{O}(1)$ query)

Sparse Table เป็นโครงสร้างข้อมูลสำหรับ Minimum Query บน Static Array ที่มีประสิทธิภาพสูงมาก โดยใช้เวลา $\mathcal{O}(n \log n)$ ในการสร้าง และตอบคำถามได้ใน $\mathcal{O}(1)$[11, 3]

แนวคิดหลักคือการคำนวณค่าน้อยสุดของทุกช่วงที่มีความยาวเป็นกำลังสองไว้ล่วงหน้า กำหนดให้ `lookup[i][j]` เก็บดัชนีของค่าน้อยสุดในช่วงที่เริ่มจากตำแหน่ง i และมีความยาว 2^j (กล่าวคือช่วง $[i, i + 2^j - 1]$)

```
int memo[MAXN][LOG];
void preprocess(int arr[], int n) {
    // Base case: intervals of length 1
    for (int i = 0; i < n; i++)
        memo[i][0] = i;
    // Build for lengths 2, 4, 8, ...
    for (int j = 1; (1 << j) <= n; j++) {
        for (int i = 0; i + (1 << j) - 1 < n; i++) {
            if (arr[memo[i][j - 1]] <
                arr[memo[i + (1 << (j - 1))][j - 1]])
                memo[i][j] = memo[i][j - 1];
            else
                memo[i][j] = memo[i + (1 << (j - 1))][j - 1];
        }
    }
}
```

ตัวอย่างโค้ดที่ 6.6: สร้าง Sparse Table

จำนวนค่าที่ต้องคำนวณคือ $\mathcal{O}(n \log n)$ (สำหรับทุกตำแหน่งเริ่มต้น n ตำแหน่ง และทุกความยาว $\log n$ ค่า) การคำนวณแต่ละค่าใช้เวลา $\mathcal{O}(1)$ เพราะใช้ค่าจากขนาดที่เล็กกว่ามาคำนวณ (Dynamic Programming)

สำหรับการสอบถาม minq(a, b) เราจะเลือก k เป็นกำลังสองที่มากที่สุดที่ไม่เกินความยาวของช่วง (กล่าวคือ $k = \lfloor \log_2(b - a + 1) \rfloor$) แล้วใช้:

$$\text{minq}(a, b) = \min(\text{lookup}[a][k],\ \text{lookup}[b - 2^k + 1][k])$$

```
int query(int arr[], int a, int b) {
    int k = log2(b - a + 1);  // Largest power of 2 <= length
    if (arr[memo[a][k]] <=
        arr[memo[b - (1 << k) + 1][k]])
        return arr[memo[a][k]];
    else
        return arr[memo[b - (1 << k) + 1][k]];
}
```

ตัวอย่างโค้ดที่ 6.7: Minimum Query ด้วย Sparse Table

สังเกตว่าช่วงที่ได้จากสองส่วนนี้อาจซ้อนทับกัน แต่สำหรับการหาค่าน้อยสุด การซ้อนทับไม่กระทบผลลัพธ์ เพราะ min เป็นฟังก์ชันที่มีคุณสมบัติ idempotent (หมายความว่า $\min(x, x) = x$) นี่เป็นเหตุผลที่ Sparse Table เหมาะสำหรับ min/max queries แต่ไม่เหมาะสำหรับ sum queries ที่การซ้อนทับจะทำให้นับสมาชิกซ้ำ

ตัวอย่าง: Sparse Table Query

จากแถวลำดับ $a = [1, 8, 7, 1, 2, 9, 2, 6]$ สอบถาม $\text{minq}(1, 6)$:

- ความยาวช่วง $= 6 - 1 + 1 = 6$ ดังนั้น $k = \lfloor \log_2 6 \rfloor = 2$ หรือ $2^k = 4$
- ช่วงแรก: $[1, 4]$ จาก `lookup[1][2]` $\rightarrow$ ค่าน้อยสุดคือ 1
- ช่วงที่สอง: $[3, 6]$ จาก `lookup[3][2]` $\rightarrow$ ค่าน้อยสุดคือ 1
- ผลลัพธ์: $\min(1, 1) = 1$

ข้อจำกัดของ Sparse Table

Sparse Table เหมาะสำหรับ Static Array เท่านั้น เพราะหลังจากเปลี่ยนค่าในแถวลำดับ เราต้องสร้างตารางใหม่ทั้งหมด ซึ่งใช้เวลา $\mathcal{O}(n \log n)$ นอกจากนี้ Sparse Table ไม่รองรับ Sum Query โดยตรง เนื่องจากช่วงที่ซ้อนทับกันจะทำให้นับสมาชิกซ้ำ

6.3 Binary Indexed Tree (Fenwick Tree)

Binary Indexed Tree หรืออีกชื่อคือ **Fenwick Tree** เป็นโครงสร้างข้อมูลที่สามารถมองได้ว่าเป็น Prefix Sum Array ที่ค่าในแถวลำดับเปลี่ยนแปลงได้ มันรองรับการดำเนินการสองอย่างที่ใช้เวลา $\mathcal{O}(\log n)$ ได้แก่ การสอบถามผลรวมแบบช่วง และการอัปเดตค่าที่เก็บ[3, 11]

ข้อได้เปรียบหลักของ Binary Indexed Tree เทียบกับ Prefix Sum Array คือรองรับการอัปเดตค่าในแถวลำดับได้อย่างมีประสิทธิภาพ หากใช้ Prefix Sum Array การอัปเดตค่าแม้เพียงตำแหน่งเดียวจะต้องสร้าง Prefix Sum Array ใหม่ทั้งหมด ซึ่งใช้เวลา $\mathcal{O}(n)$ แต่ Binary Indexed Tree สามารถอัปเดตได้ในเวลาเพียง $\mathcal{O}(\log n)$

6.3.1 แนวคิดพื้นฐาน

แนวคิดหลักของ Binary Indexed Tree คือการแทนตัวเลขด้วยผลรวมของกำลังสอง เช่น:

$$15 = 2^3 + 2^2 + 2^1 + 2^0 = 8 + 4 + 2 + 1$$

ดังนั้นผลรวมในช่วง $[1, 15]$ สามารถแบ่งออกเป็น:

$$\text{sum}(1, 15) = \text{range}(1, 8) + \text{range}(9, 12) + \text{range}(13, 14) + \text{range}(15, 15)$$

กำหนดให้ $p(k)$ แทนค่าของกำลังสองที่มากที่สุดที่หาร k ลงตัว (ซึ่งเท่ากับ k & $-k$ ในการดำเนินการบิต) ตัวอย่างเช่น:

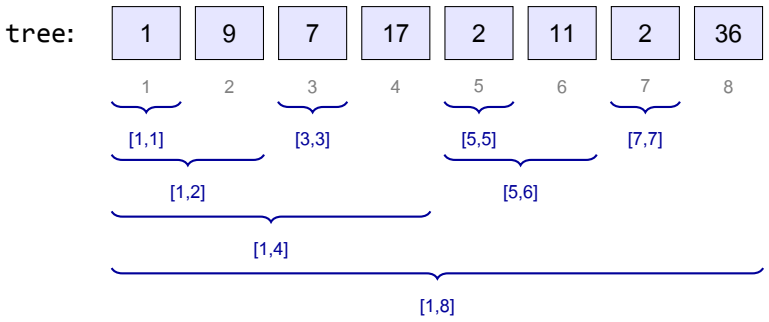
- $p(7) = 1$ เพราะ $2^0 = 1$ เป็นกำลังสองที่มากที่สุดที่หาร 7 ลงตัว
- $p(6) = 2$ เพราะ $2^1 = 2$ เป็นกำลังสองที่มากที่สุดที่หาร 6 ลงตัว
- $p(4) = 4$ เพราะ $2^2 = 4$ เป็นกำลังสองที่มากที่สุดที่หาร 4 ลงตัว

Binary Indexed Tree เก็บค่าในแถวลำดับ `tree[]` โดย:

$$\text{tree}[k] = \text{sumq}(k - p(k) + 1,\ k)$$

กล่าวคือ ช่อง k จะเก็บผลรวมของช่วงที่มีความยาว $p(k)$ และสิ้นสุดที่ตำแหน่ง k ในแถวลำดับต้นฉบับ

รูปที่ 6.3 แสดงโครงสร้างของ Binary Indexed Tree โดยแต่ละช่องเก็บผลรวมของช่วงที่แตกต่างกัน



รูปที่ 6.3 โครงสร้าง Binary Indexed Tree: แต่ละช่อง `tree[k]` เก็บผลรวมของช่วงที่มีความยาว $p(k)$ สิ้นสุดที่ตำแหน่ง k

6.3.2 การดำเนินการ

การสอบถามผลรวม $\text{sumq}(1, k)$: เราจะลดค่า k ลงทีละหนึ่ง Last Set Bit ไปเรื่อยๆ จนเป็น 0 โดยในแต่ละขั้นจะบวก `tree[k]` เข้าไปในผลรวม จำนวนขั้นคือจำนวนบิต 1 ในเลขฐานสองของ k ซึ่งไม่เกิน $\log_2 n$

```
int tree[MAXN];
int n;
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k & -k;   // Remove last set bit
    }
    return s;
}
// sumq(a, b) = sumq(1, b) - sumq(1, a-1)
```

```
int rangeSum(int a, int b) {
    return sum(b) - sum(a - 1);
}
```

ตัวอย่างโค้ดที่ 6.8: Sum Query บน Fenwick Tree

การอัปเดตค่า ที่ตำแหน่ง k เพิ่มค่า x: เราจะเพิ่ม x ให้กับทุกช่องใน `tree[]` ที่มีช่วงครอบคลุมตำแหน่ง k โดยเพิ่มตำแหน่ง k ขึ้นทีละหนึ่ง Last Set Bit แต่ละตำแหน่งใน `tree[]` ถูกครอบคลุมโดยช่องอื่นอย่างเป็นระบบ ทำให้ต้องอัปเดต $\mathcal{O}(\log n)$ ช่อง

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k & -k;  // Add last set bit
    }
}
```

ตัวอย่างโค้ดที่ 6.9: Point Update บน Fenwick Tree

ตัวอย่าง: Fenwick Tree

จากแถวลำดับ $a = [1, 8, 7, 1, 2, 9, 2, 6]$ (1-indexed) Binary Indexed Tree จะเป็น:

$$\text{tree} = [1, 9, 7, 17, 2, 11, 2, 36]$$

การสอบถาม $\text{sumq}(1, 7)$:

- $k = 7$: $s+ = \text{tree}[7] = 2$, $k = 7 - 1 = 6$
- $k = 6$: $s+ = \text{tree}[6] = 11$, $k = 6 - 2 = 4$
- $k = 4$: $s+ = \text{tree}[4] = 17$, $k = 4 - 4 = 0$
- หยุด: $s = 2 + 11 + 17 = 30$

ตรงกับ $1 + 8 + 7 + 1 + 2 + 9 + 2 = 30$ ✓

6.3.3 การสร้าง Fenwick Tree

สามารถสร้าง Fenwick Tree ได้ในเวลา $\mathcal{O}(n)$ โดยคำนวณค่าของแต่ละช่องโดยตรง:

```
void build(int arr[], int n) {
    for (int i = 1; i <= n; i++) {
        tree[i] += arr[i];
        int j = i + (i & -i);
        if (j <= n)
```

```
            tree[j] += tree[i];
        }
    }
```

ตัวอย่างโค้ดที่ 6.10: สร้าง Fenwick Tree ใน O(n)

หรืออีกวิธีคือเริ่มจาก `tree[]` ที่เป็น 0 ทั้งหมด แล้วเรียก `add(i, arr[i])` สำหรับทุกตำแหน่ง ซึ่งใช้เวลา $\mathcal{O}(n \log n)$

6.3.4 Range Update และ Point Query

Fenwick Tree สามารถดัดแปลงเพื่อรองรับ **Range Update** (เพิ่มค่าทุกตำแหน่งในช่วง $[a, b]$ ด้วยค่า x) และ **Point Query** (สอบถามค่าที่ตำแหน่ง k) ได้ โดยใช้ **Difference Array**

กำหนด `diff[i] = arr[i] - arr[i-1]` (โดย `arr[0] = 0`) เราจะเก็บ Difference Array ใน Fenwick Tree การอัปเดตช่วง $[a, b]$ ด้วยค่า x ทำได้โดย:

- `add(a, x)`: เพิ่มค่า x ที่ตำแหน่ง a
- `add(b+1, -x)`: ลดค่า x ที่ตำแหน่ง $b + 1$

และค่าที่ตำแหน่ง k คือ `sum(k)` ซึ่งเป็น Prefix Sum ของ Difference Array

```
// Add x to range [a, b]
void rangeAdd(int a, int b, int x) {
    add(a, x);
    add(b + 1, -x);
}
// Get value at position k
int pointQuery(int k) {
    return sum(k);
}
```

ตัวอย่างโค้ดที่ 6.11: Range Update และ Point Query ด้วย Fenwick Tree

6.4 Segment Tree

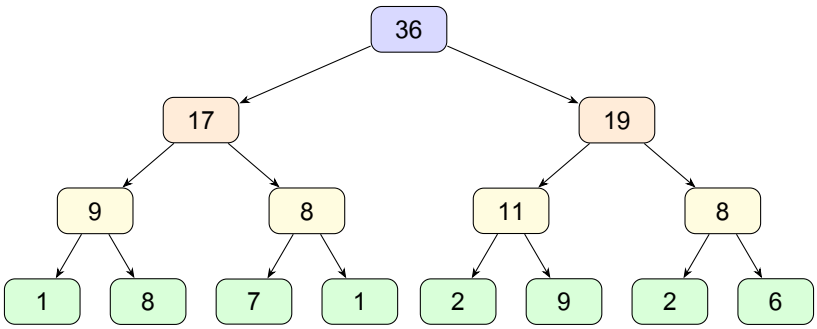
Segment Tree เป็นโครงสร้างข้อมูลที่รองรับการสอบถามเป็นช่วงและการอัปเดตค่าในแถวลำดับ เป็นโครงสร้างที่ general กว่า Binary Indexed Tree เพราะรองรับไม่เพียงแค่ Sum Queries แต่ยังรองรับ Minimum Queries, Maximum Queries, GCD, และการสอบถามแบบอื่นๆ ได้อีกด้วย[3, 11]

6.4.1 โครงสร้างของ Segment Tree

Segment Tree เป็น Binary Tree ที่:

- โหนดในชั้นล่างสุด (Leaf Node) แทนสมาชิกแต่ละตัวในแถวลำดับ
- โหนดในชั้นอื่นๆ เก็บข้อมูลที่จำเป็นสำหรับการประมวลผลแบบช่วง (เช่น ผลรวม ค่าน้อยสุด ฯลฯ)
- โหนดบนสุด (Root) แทนช่วงทั้งแถวลำดับ $[0, n-1]$
- โหนดที่แทนช่วง $[L, R]$ จะมีลูกซ้ายแทน $[L, (L+R)/2]$ และลูกขวาแทน $[(L+R)/2+1, R]$

รูปที่ 6.4 แสดง Segment Tree สำหรับ Sum Queries ของแถวลำดับ $[1, 8, 7, 1, 2, 9, 2, 6]$



รูปที่ 6.4 Segment Tree สำหรับ Sum Queries: แต่ละโหนดเก็บผลรวมของช่วงที่รับผิดชอบ

สังเกตว่าช่วง $[a, b]$ ใดๆ สามารถแบ่งออกเป็น $\mathcal{O}(\log n)$ ช่วงย่อยที่ค่าถูกเก็บในโหนดของ Segment Tree อยู่แล้ว เมื่อผลรวมถูกคำนวณโดยใช้โหนดที่อยู่ในชั้นสูงที่สุดเท่าที่จะเป็นไปได้ จะพบว่าไม่เกินสองโหนดในแต่ละชั้นที่ถูกใช้ ดังนั้นจำนวนโหนดทั้งหมดที่ใช้คือ $\mathcal{O}(\log n)$

6.4.2 การพัฒนาแบบ Recursive

วิธีหนึ่งในการพัฒนา Segment Tree คือใช้ Recursive Function โดยเก็บ Tree ในแถวลำดับขนาดประมาณ $4n$ โดย `tree[node]` เก็บค่าของโหนดที่ตำแหน่ง `node` โหนดรากอยู่ที่ตำแหน่ง 1 ลูกซ้ายและลูกขวาของโหนด `node` อยู่ที่ตำแหน่ง $2 \times$ node และ $2 \times$ node $+ 1$ ตามลำดับ

```
#define N 20
#define MAX (1 + (1 << 6))  // > 2*N
#define INF 0x7fffffff
int arr[N];
int tree[MAX];
void buildTree(int node, int a, int b) {
    // Out of range
    if (a > b) return;
    // Leaf node
    if (a == b) {
        tree[node] = arr[a];
        return;
    }
    // Build left and right child
```

```
    buildTree(node * 2, a, (a + b) / 2);
    buildTree(node * 2 + 1, 1 + (a + b) / 2, b);
    // Combine: store max of children
    tree[node] = max(tree[node * 2], tree[node * 2 + 1]);
}
void updateTree(int node, int a, int b, int i, int j, int value) {
    // Current segment is not within range [i, j]
    if (a > b || a > j || b < i) return;
    // Leaf node
    if (a == b) {
        tree[node] += value;
        return;
    }
    // Update left and right child
    updateTree(node * 2, a, (a + b) / 2, i, j, value);
    updateTree(1 + node * 2, 1 + (a + b) / 2, b, i, j, value);
    // Update current node
    tree[node] = max(tree[node * 2], tree[node * 2 + 1]);
}
int queryTree(int node, int a, int b, int i, int j) {
    // Out of range
    if (a > b || a > j || b < i) return -INF;
    // Current segment is totally within range [i, j]
    if (a >= i && b <= j) return tree[node];
    // Query left and right child
    int q1 = queryTree(node * 2, a, (a + b) / 2, i, j);
    int q2 = queryTree(1 + node * 2, 1 + (a + b) / 2, b, i, j);
    return max(q1, q2);
}
```

ตัวอย่างโค้ดที่ 6.12: Segment Tree แบบ Recursive สำหรับ Maximum Query

6.4.3 การพัฒนาแบบ Iterative (Compact Array)

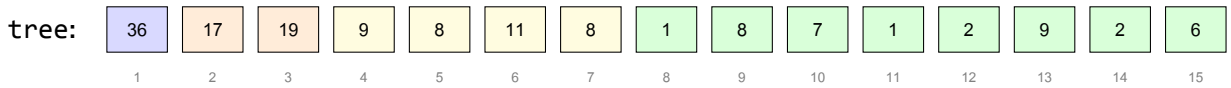
อีกวิธีหนึ่งที่กระชับกว่าคือเก็บ Segment Tree เป็นแถวลำดับขนาด $2n$ (เมื่อ n เป็นกำลังสอง) โดย:

- โหนดในชั้นล่างสุด (Leaf) อยู่ที่ตำแหน่ง n ถึง $2n-1$
- โหนดภายในอยู่ที่ตำแหน่ง 1 ถึง $n-1$
- พ่อของ `tree[k]` คือ `tree[k/2]`
- ลูกของ `tree[k]` คือ `tree[2k]` และ `tree[2k+1]`

```cpp
int tree[2 * MAXN];
int n;
void build() {
    for (int i = 0; i < n; i++)
        tree[n + i] = arr[i];
    for (int i = n - 1; i >= 1; i--)
        tree[i] = tree[2 * i] + tree[2 * i + 1];
}
int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a % 2 == 1) s += tree[a++];
        if (b % 2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}
void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2)
        tree[k] = tree[2 * k] + tree[2 * k + 1];
}
```

ตัวอย่างโค้ดที่ 6.13: Segment Tree แบบ Iterative สำหรับ Sum Query

รูปที่ 6.5 แสดงการเก็บ Segment Tree ในรูปแบบ Compact Array



รูปที่ 6.5 การเก็บ Segment Tree แบบ Compact Array: โหนดรากอยู่ที่ตำแหน่ง 1, Leaf อยู่ที่ตำแหน่ง n ถึง $2n - 1$

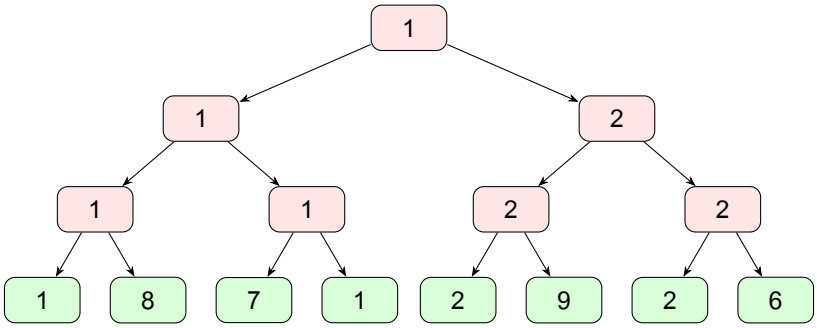
6.4.4 รองรับการสอบถามอื่น ๆ

Segment Tree สามารถรองรับการสอบถามแบบช่วงทุกรูปแบบที่สามารถแบ่งช่วงออกเป็นสองส่วน คำนวณคำตอบแยกกัน แล้วนำมารวมกันได้ ตัวอย่างเช่น:

- **Sum Query:** `tree[node] = tree[left] + tree[right]`

- **Minimum Query:** `tree[node] = min(tree[left], tree[right])`
- **Maximum Query:** `tree[node] = max(tree[left], tree[right])`
- **GCD Query:** `tree[node] = gcd(tree[left], tree[right])`
- **Bitwise AND/OR/XOR:** ใช้การดำเนินการบิตแทนการบวก

รูปที่ 6.6 แสดง Segment Tree สำหรับ Minimum Query



รูปที่ 6.6 Segment Tree สำหรับ Minimum Query: แต่ละโหนดเก็บค่าน้อยสุดของช่วงที่รับผิดชอบ

นอกจากนี้ โครงสร้างของ Segment Tree ยังอนุญาตให้ใช้ Binary Search เพื่อค้นหาตำแหน่งของโหนดที่มีค่าต้องการได้ในเวลา $\mathcal{O}(\log n)$ เช่น หาตำแหน่งของค่าน้อยสุดแรกที่น้อยกว่าค่าที่กำหนด

6.5 Lazy Propagation

เมื่อมีการอัปเดตช่วง (Range Update) บ่อยครั้งบน Segment Tree การอัปเดตทีละโหนดจะใช้เวลา $\mathcal{O}(n)$ ซึ่งช้าเกินไป **Lazy Propagation** เป็นเทคนิคที่ช่วยให้การอัปเดตช่วงทำได้ในเวลา $\mathcal{O}(\log n)$ โดยเลื่อน (propagate) การอัปเดตออกไปก่อน และทำเมื่อจำเป็นเท่านั้น[3, 11]

6.5.1 แนวคิด

สังเกตว่าใน Segment Tree โหนดที่เก็บค่าของช่วงใดช่วงหนึ่ง ถ้าช่วงของโหนดนั้นอยู่ภายในช่วงที่ต้องอัปเดตทั้งหมด แสดงว่าทุกโหนดด้านล่างของโหนดนั้นต้องถูกอัปเดตด้วย แทนที่จะอัปเดตทุกโหนดทันที เราจะ:

1. อัปเดตค่าของโหนดปัจจุบัน
2. บันทึกว่าโหนดลูกยังต้องการการอัปเดต โดยเก็บไว้ในแถวลำดับ `lazy[]` ที่เรียกว่า **Lazy Node**
3. เมื่อจำเป็นต้องเข้าถึงโหนดลูก (เช่น ตอน query หรือ update ครั้งต่อไป) จึงค่อย propagate การอัปเดตลงไป

หลักการของ Lazy Propagation

เริ่มต้นให้ทุกค่าใน `lazy[]` เป็น 0 ค่าที่ไม่เป็น 0 ของ `lazy[node]` หมายความว่าค่านี้จำเป็นที่จะต้องถูกเพิ่มให้กับโหนด node ใน Segment Tree ก่อนที่จะมีการ query อื่นกับโหนดนี้ ต้องคิดว่าถ้าเป็น sum, max, min จะอัปเดต lazy แบบไหนหรือคืนค่าแบบไหน

6.5.2 Implementation

```
int arr[N], tree[MAX], lazy[MAX];
void buildTree(int node, int a, int b) {
    lazy[node] = 0;
    if (a > b) return;
    if (a == b) {
        tree[node] = arr[a];
        return;
    }
    buildTree(node * 2, a, (a + b) / 2);
    buildTree(node * 2 + 1, 1 + (a + b) / 2, b);
    tree[node] = max(tree[node * 2], tree[node * 2 + 1]);
}
void updateTree(int node, int a, int b, int i, int j, int value) {
    // Apply pending lazy update
    if (lazy[node] != 0) {
        tree[node] += lazy[node];
        if (a != b) {
            lazy[node * 2] += lazy[node];
            lazy[node * 2 + 1] += lazy[node];
        }
        lazy[node] = 0;
    }
    // Out of range
    if (a > b || a > j || b < i) return;
    // Fully within range
    if (a >= i && b <= j) {
        tree[node] += value;
        if (a != b) {
            lazy[node * 2] += value;
            lazy[node * 2 + 1] += value;
        }
        return;
    }
```

```
34      // Partial overlap: recurse
35      updateTree(node * 2, a, (a + b) / 2, i, j, value);
36      updateTree(node * 2 + 1, 1 + (a + b) / 2, b, i, j, value);
37      tree[node] = max(tree[node * 2], tree[node * 2 + 1]);
38  }
39  int queryTree(int node, int a, int b, int i, int j) {
40      // Apply pending lazy update
41      if (lazy[node] != 0) {
42          tree[node] += lazy[node];
43          if (a != b) {
44              lazy[node * 2] += lazy[node];
45              lazy[node * 2 + 1] += lazy[node];
46          }
47          lazy[node] = 0;
48      }
49      // Out of range
50      if (a > b || a > j || b < i) return -INF;
51      // Fully within range
52      if (a >= i && b <= j) return tree[node];
53      // Partial overlap: query children
54      int q1 = queryTree(node * 2, a, (a + b) / 2, i, j);
55      int q2 = queryTree(node * 2 + 1, 1 + (a + b) / 2, b, i, j);
56      return max(q1, q2);
57  }
```

ตัวอย่างโค้ดที่ 6.14: Segment Tree พร้อม Lazy Propagation

ตัวอย่าง: Lazy Propagation

สมมติ Segment Tree สำหรับ Maximum Query บนแถวลำดับ $[1, 8, 7, 1, 2, 9, 2, 6]$ และเราต้องการอัปเดตช่วง $[0, 5]$ เพิ่มค่า 5:

1. เริ่มจาก Root ที่แทนช่วง $[0, 7]$ ไม่อยู่ในช่วง $[0, 5]$ ทั้งหมด จึงต้องลงไปที่ลูกทั้งสอง
2. ลูกซ้ายแทนช่วง $[0, 3]$ อยู่ในช่วง $[0, 5]$ ทั้งหมด ($3 \leq 5$) $\rightarrow$ อัปเดตค่าและ mark lazy ให้ลูก แล้ว return ทันที (ไม่ต้อง recurse ต่อ)
3. ลูกขวาแทนช่วง $[4, 7]$ ไม่อยู่ในช่วง $[0, 5]$ ทั้งหมด ลงไปที่ลูกต่อ
4. โหนดที่แทนช่วง $[4, 5]$ อยู่ในช่วง $[0, 5]$ ทั้งหมด $\rightarrow$ อัปเดตค่าและ mark lazy ให้ลูก แล้ว return ทันที
5. โหนดที่แทนช่วง $[6, 7]$ อยู่นอกช่วง $[0, 5]$ $\rightarrow$ ไม่ต้องอัปเดต

ด้วยวิธีนี้ เราไม่ต้อง traverse ไปทุก Leaf Node ทำให้การอัปเดตช่วงใช้เวลา $\mathcal{O}(\log n)$

6.6 เทคนิคเพิ่มเติม

6.6.1 Difference Array สำหรับ Range Update

เมื่อต้องการอัปเดตช่วงและสอบถามค่าเดียว (Range Update, Point Query) เราสามารถใช้ **Difference Array** ซึ่งเก็บค่าผลต่างระหว่างสมาชิกที่ติดกันในแถวลำดับต้นฉบับ

$$\text{diff}[i] = \text{arr}[i] - \text{arr}[i-1], \qquad \text{diff}[0] = \text{arr}[0]$$

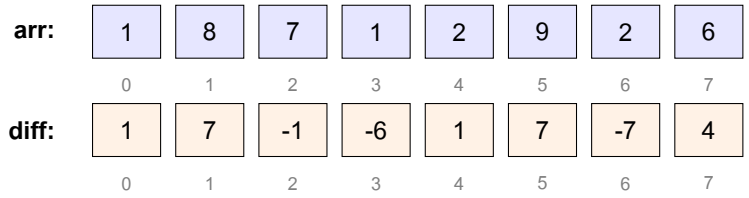
ข้อดีคือการเพิ่มค่า x ในช่วง $[a, b]$ ของแถวลำดับต้นฉบับ ทำได้โดยเปลี่ยนเพียง 2 ค่าใน Difference Array:

$$\text{diff}[a] \mathrel{+}= x, \qquad \text{diff}[b+1] \mathrel{-}= x$$

และค่าที่ตำแหน่ง k ในแถวลำดับต้นฉบับคือ Prefix Sum ของ Difference Array:

$$\text{arr}[k] = \sum_{i=0}^{k} \text{diff}[i]$$

รูปที่ 6.7 แสดงความสัมพันธ์ระหว่างแถวลำดับต้นฉบับและ Difference Array



รูปที่ 6.7 Difference Array: แถวลำดับต้นฉบับ (บน) และ Difference Array (ล่าง) ค่าที่ตำแหน่ง k ของแถวลำดับต้นฉบับคือ Prefix Sum ของ Difference Array

ตัวอย่าง: Range Update ด้วย Difference Array

จากแถวลำดับต้นฉบับ $[1, 8, 7, 1, 2, 9, 2, 6]$ ต้องการเพิ่มค่า 5 ในช่วง $[1, 4]$:

- $\text{diff}[1] \mathrel{+}= 5 \rightarrow \text{diff}[1] = 7 + 5 = 12$
- $\text{diff}[5] \mathrel{-}= 5 \rightarrow \text{diff}[5] = 7 - 5 = 2$
- Difference Array ใหม่: $[1, 12, -1, -6, 1, 2, -7, 4]$
- แถวลำดับต้นฉบับใหม่: $[1, 13, 12, 6, 7, 9, 2, 6]$

ตรวจสอบ: ตำแหน่ง 1 ถึง 4 เพิ่มขึ้น 5 ทุกตำแหน่ง ✓

หากต้องการรองรับทั้ง Range Update และ Range Query สามารถใช้ Difference Array ร่วมกับ Fenwick Tree หรือ Segment Tree เพื่อให้ได้ทั้งการอัปเดตและการสอบถามในเวลา $\mathcal{O}(\log n)$

6.6.2 เปรียบเทียบโครงสร้างข้อมูล

ตารางต่อไปนี้สรุปเปรียบเทียบโครงสร้างข้อมูลทั้งหมดที่กล่าวถึงในบทนี้:

ตารางที่ 6.1 เปรียบเทียบโครงสร้างข้อมูลสำหรับ Range Queries

โครงสร้างข้อมูล	Preprocess	Query	Update	ประเภท Query
Prefix Sum Array	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Sum
Lookup Table	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	—	Min/Max
Sq. Root Decomp.	$\mathcal{O}(n)$	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(\sqrt{n})$	ทั่วไป
Sparse Table	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	—	Min/Max
Fenwick Tree	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	Sum
Segment Tree	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	ทั่วไป
Seg. Tree + Lazy	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)^*$	ทั่วไป

*Range Update ด้วย Lazy Propagation

เลือกโครงสร้างข้อมูลอย่างไร?

- ถ้าแถวลำดับไม่เปลี่ยนแปลง (Static) และต้องการ Sum Query → **Prefix Sum Array**
- ถ้าแถวลำดับไม่เปลี่ยนแปลง (Static) และต้องการ Min/Max Query → **Sparse Table**
- ถ้าแถวลำดับเปลี่ยนแปลง (Dynamic) และต้องการ Sum Query เท่านั้น → **Fenwick Tree** (เขียนง่ายกว่า)
- ถ้าแถวลำดับเปลี่ยนแปลง (Dynamic) และต้องการ Query หลายประเภท → **Segment Tree**
- ถ้าต้องการ Range Update + Range Query → **Segment Tree พร้อม Lazy Propagation**

6.7 โจทย์แนะนำ

1. **Static Range Sum Query:** กำหนดแถวลำดับขนาด n และ q คำถาม แต่ละคำถามให้ช่วง $[a, b]$ จงหาผลรวมในช่วง (ฝึกใช้ Prefix Sum Array)
2. **Static Range Minimum Query:** กำหนดแถวลำดับขนาด n และ q คำถาม แต่ละคำถามให้ช่วง $[a, b]$ จงหาค่าน้อยสุดในช่วง (ฝึกใช้ Sparse Table)
3. **Dynamic Range Sum Query:** กำหนดแถวลำดับขนาด n มี q คำสั่ง แต่ละคำสั่งเป็นได้สองแบบ: (1) เปลี่ยนค่าตำแหน่ง k เป็น x (2) ถามผลรวมในช่วง $[a, b]$ (ฝึกใช้ Fenwick Tree หรือ Segment Tree)
4. **Range Update และ Point Query:** กำหนดแถวลำดับขนาด n มี q คำสั่ง แต่ละคำสั่งเป็นได้สองแบบ: (1) เพิ่มค่า x ในช่วง $[a, b]$ (2) ถามค่าที่ตำแหน่ง k (ฝึกใช้ Difference Array + Fenwick Tree)

5. **Range Update และ Range Query:** กำหนดแถวลำดับขนาด n มี q คำสั่ง แต่ละคำสั่งเป็นได้สองแบบ: (1) เพิ่มค่า x ในช่วง $[a, b]$ (2) ถามค่ามากสุด/ผลรวมในช่วง $[a, b]$ (ฝึกใช้ Segment Tree + Lazy Propagation)

6. **2D Range Sum Query:** กำหนดตารางขนาด $n \times m$ และ q คำถาม แต่ละคำถามให้พิกัดมุมบนซ้ายและมุมล่างขวาของพื้นที่สี่เหลี่ยม จงหาผลรวมในพื้นที่นั้น (ฝึกใช้ Prefix Sum 2 มิติ)

โจทย์แนะนำจาก UVa Online Judge

สำหรับผู้ที่ต้องการฝึกฝนเพิ่มเติม โจทย์ที่น่าสนใจจาก Competitive Programming 3 ได้แก่:

- UVa 11402 — Ahoy Pirates! (Segment Tree + Lazy Propagation สำหรับ Range Update) [20]
- UVa 108 — Maximum Sum (2D Range Sum Query)[20]
- UVa 11235 — Frequent Values (Sparse Table หรือ Segment Tree)[20]
- UVa 12299 — RMQ with Shifts (Segment Tree สำหรับ RMQ พร้อมการอัปเดต)[20]

6.8 สรุปบทที่ 6

ในบทนี้ เราได้ศึกษาโครงสร้างข้อมูลและเทคนิคสำหรับตอบคำถามช่วง (Range Queries) ตั้งแต่วิธีพื้นฐานอย่าง Prefix Sum ไปจนถึง Segment Tree พร้อม Lazy Propagation แต่ละโครงสร้างมีจุดแข็งและข้อจำกัดที่แตกต่างกัน การเลือกใช้โครงสร้างที่เหมาะสมขึ้นอยู่กับลักษณะของคำถามและการเปลี่ยนแปลงข้อมูล

สรุปสิ่งสำคัญของบทที่ 16

1. **Prefix Sum Array** ตอบคำถามผลรวมบนแถวลำดับคงที่ได้ใน $\mathcal{O}(1)$ — เร็วและเขียนง่ายที่สุด
2. **Prefix Sum 2 มิติ** ขยายแนวคิด Prefix Sum สำหรับตาราง 2 มิติ
3. **Sparse Table** ตอบคำถามค่าต่ำสุด/สูงสุดบนแถวลำดับคงที่ได้ใน $\mathcal{O}(1)$ แต่สร้างช้ากว่า $\mathcal{O}(n \log n)$
4. **Fenwick Tree (BIT)** รองรับการอัปเดตค่าและคำถามผลรวมได้ใน $\mathcal{O}(\log n)$ — เขียนง่ายและคงที่ตัวเล็ก
5. **Segment Tree** ยืดหยุ่นที่สุด รองรับคำถามได้หลายประเภท (ผลรวม, ค่าต่ำสุด, ค่าสูงสุด, ฯลฯ)
6. **Lazy Propagation** ช่วยให้ Segment Tree รองรับ Range Update ได้อย่างมีประสิทธิภาพ
7. **กฎเลือกใช้:** แถวลำดับคงที่ $\rightarrow$ Prefix Sum / Sparse Table; แถวลำดับเปลี่ยนแปลง $\rightarrow$ Fenwick / Segment Tree; Range Update $\rightarrow$ Segment Tree + Lazy

Chapter 7

Complete Search

ในการโปรแกรมเชิงแข่งขัน หนึ่งในกลยุทธ์ที่ทรงพลังและตรงไปตรงมาที่สุดคือ **Complete Search** หรือการค้นหาแบบครบถ้วน[3, 7] แนวคิดหลักคือลองตรวจสอบทุกคำตอบที่เป็นไปได้ แล้วเลือกคำตอบที่ดีที่สุด แม้ว่าวิธีนี้อาจดู “ช้า” ในความรู้สึกแรกเริ่ม แต่ในทางปฏิบัติ Complete Search มักเป็นจุดเริ่มต้นที่ดีที่สุดในการแก้โจทย์หลายประเภท โดยเฉพาะเมื่อขนาดข้อมูลนำเข้ามีข้อจำกัดไม่ใหญ่เกินไป ในบทนี้ เราจะศึกษาเทคนิค Complete Search ตั้งแต่พื้นฐาน ไปจนถึงเทคนิคขั้นสูงอย่าง Backtracking, Pruning และ Meet in the Middle ที่ช่วยให้เราแก้โจทย์ได้เร็วขึ้นอย่างมาก

7.1 แนะนำ Complete Search

Complete Search คือกลยุทธ์การแก้ปัญหาที่ตรวจสอบทุกความเป็นไปได้ของคำตอบ แล้วเลือกคำตอบที่ตรงตามเงื่อนไขหรือให้ค่าดีที่สุด วิธีนี้มีข้อดีคือ **ง่ายต่อการเขียนและเข้าใจ** และ **ให้คำตอบที่ถูกต้องแน่นอน** เพราะเราไม่ได้ข้ามความเป็นไปได้ใดๆ เลย

อย่างไรก็ตาม Complete Search ไม่ได้เหมาะกับทุกโจทย์ ถ้าจำนวนความเป็นไปได้มากเกินไป (เช่น 2^{100} หรือ $100!$) โปรแกรมจะทำงานช้าเกินไปและไม่ทันเวลา ในกรณีเช่นนี้ เราต้องใช้กลยุทธ์อื่นแทน เช่น ขั้นตอนวิธี Greedy หรือ Dynamic Programming

เมื่อไหร่ควรใช้ Complete Search

ใช้ Complete Search เมื่อ:

- จำนวนความเป็นไปได้มีขนาด “พอรับได้” (เช่น $n \leq 20$ สำหรับ Subset หรือ $n \leq 10$ สำหรับ Permutation)
- ไม่สามารถหารูปแบบหรือสูตรที่ดีกว่าได้
- ต้องการคำตอบที่ถูกต้องแน่นอน (Optimal Solution)
- ต้องการเขียนโค้ดเร็วและไม่ซับซ้อน

ถ้า Complete Search ช้าเกินไป ให้ลอง Greedy หรือ Dynamic Programming แทน

7.1.1 Complete Search คืออะไร

นิยาม 7.1 (Complete Search)**.** Complete Search คือวิธีการแก้ปัญหาโดยลองตรวจสอบ **ทุก** คำตอบที่เป็นไปได้ แล้วเลือกคำตอบที่ดีที่สุด หรือนับจำนวนคำตอบที่ถูกต้อง

ตัวอย่างเช่น ถ้าโจทย์ถามว่า “มีสมาชิกตัวไหนในแถวลำดับที่มีค่าเท่ากับ x หรือไม่” เราสามารถลองตรวจสอบสมาชิกทุกตัวได้ นี่คือ Complete Search ในรูปแบบที่ง่ายที่สุด

ข้อดีหลักของ Complete Search:

1. **ง่ายต่อการนำไปใช้** — โค้ดมักสั้นและตรงไปตรงมา

2. **ถูกต้องแน่นอน** — เพราะตรวจสอบทุกกรณี จึงไม่มีโอกาสพลาดคำตอบ

3. **เป็นจุดเริ่มต้นที่ดี** — แม้จะช้าเกินไปสำหรับโจทย์บางข้อ แต่ Complete Search ช่วยให้เราเข้าใจโจทย์ได้ดีขึ้น และอาจนำไปสู่ขั้นตอนวิธีที่มีประสิทธิภาพกว่า

7.1.2 ค้นหา 1 ธาตุจาก n

รูปแบบที่ง่ายที่สุดของ Complete Search คือการค้นหาธาตุหนึ่งตัวจากทั้งหมด n ตัว ตัวอย่างเช่น การหาว่ามีตัวเลข x อยู่ในแถวลำดับหรือไม่ เราแค่วนลูปตรวจสอบสมาชิกทุกตัว:

```cpp
// Check if x exists in the array
bool found = false;
for (int i = 0; i < n; i++) {
    if (arr[i] == x) {
        found = true;
        break;
    }
}
```

ตัวอย่างโค้ดที่ 7.1: ค้นหา 1 ธาตุจาก n

ความซับซ้อนของเวลาคือ $\mathcal{O}(n)$ เพราะเราอาจต้องตรวจสอบสมาชิกทุกตัวในกรณีที่เลวร้ายที่สุด นี่คือ Linear Search ที่เราได้เรียนรู้มาแล้วในบทก่อนหน้า

ตัวอย่าง: หาค่ามากสุดในแถวลำดับ

```cpp
int maxVal = arr[0];
for (int i = 1; i < n; i++) {
    if (arr[i] > maxVal) {
        maxVal = arr[i];
    }
}
// maxVal stores the maximum value
```

วิธีนี้ตรวจสอบสมาชิกทุกตัว จึงมั่นใจได้ว่าได้ค่ามากสุดที่ถูกต้องเสมอ

7.1.3 ค้นหา 2 ธาตุจาก n

เมื่อโจทย์ต้องการหาคู่ของธาตุที่ตอบเงื่อนไขบางอย่าง เราต้องใช้ลูปซ้อน 2 ชั้น ยกตัวอย่างเช่น การหาคู่ตัวเลขที่ผลบวกเท่ากับ x:

```
// Find pair where arr[i] + arr[j] == x (i < j)
bool found = false;
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arr[i] + arr[j] == x) {
            found = true;
            break;
        }
    }
    if (found) break;
}
```

ตัวอย่างโค้ดที่ 7.2: ค้นหา 2 ธาตุจาก n

ความซับซ้อนของเวลาคือ $\mathcal{O}(n^2)$ เพราะเราตรวจสอบคู่ทั้งหมด $\frac{n(n-1)}{2}$ คู่ สังเกตว่าเราใช้ `j = i + 1` เพื่อไม่ให้ตรวจสอบคู่ซ้ำ (เช่น ทั้งคู่ (i, j) และ (j, i)) และไม่ให้ตรวจสอบคู่ที่ $i = j$

แนวคิดนี้สามารถขยายไปยังการค้นหา k ธาตุจาก n ได้ โดยใช้ลูปซ้อน k ชั้น ซึ่งจะมีความซับซ้อน $\mathcal{O}(n^k)$ ยกตัวอย่างเช่น การหา 3 จำนวนที่ผลรวมเท่ากับ 0 (3-Sum Problem):

ตัวอย่าง: นับจำนวนคู่ที่ผลบวกเป็นจำนวนคู่

```
int cnt = 0;
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if ((arr[i] + arr[j]) % 2 == 0) {
            cnt++;
        }
    }
}
cout << cnt << "\n";
```

ถ้า $n = 4$ และ `arr = {1, 2, 3, 4}` คู่ที่ผลบวกเป็นคู่คือ $(1, 3), (2, 4)$ รวม 2 คู่

```
// Find three elements where arr[i] + arr[j] + arr[k] == 0
bool found = false;
for (int i = 0; i < n - 2; i++) {
    for (int j = i + 1; j < n - 1; j++) {
        for (int k = j + 1; k < n; k++) {
            if (arr[i] + arr[j] + arr[k] == 0) {
```

```
                    found = true;
                }
            }
        }
    }
    // Complexity: O(n^3)
```

ตัวอย่างโค้ดที่ 7.3: ค้นหา 3 ธาตุจาก n (3-Sum)

ตารางที่ 7.1 ความซับซ้อนของการค้นหา k ธาตุจาก n

k	ความซับซ้อน	n ที่ใช้ได้
1	$\mathcal{O}(n)$	$n \leq 10^8$
2	$\mathcal{O}(n^2)$	$n \leq 10^4$
3	$\mathcal{O}(n^3)$	$n \leq 500$
4	$\mathcal{O}(n^4)$	$n \leq 100$

ระวังขอบเขตลูป

เมื่อค้นหา 2 ธาตุ ต้องระวังว่า:

- ใช้ `i < n-1` ไม่ใช่ `i < n` (เพราะต้องมี $j > i$)
- ใช้ `j = i+1` ไม่ใช่ `j = 0` (เพื่อไม่ตรวจซ้ำ)
- ถ้าโจทย์อนุญาตให้ $i = j$ ก็ต้องปรับขอบเขตให้ตรงกับเงื่อนไข

7.2 การสร้างชุดย่อย (Generating Subsets)

ในหัวข้อนี้ เราจะศึกษาวิธีสร้างชุดย่อย (Subset) ทั้งหมดจากเซตขนาด n ซึ่งมีจำนวนทั้งสิ้น 2^n ชุดย่อย การสร้างชุดย่อยเป็นเทคนิคพื้นฐานที่ใช้บ่อยในโจทย์ Complete Search หลายประเภท

7.2.1 ภาพรวม: ชุดย่อยและการเรียงสับเปลี่ยน

ก่อนจะเจาะจงเทคนิค เรามาทำความเข้าใจความแตกต่างระหว่างสองแนวคิดสำคัญ:

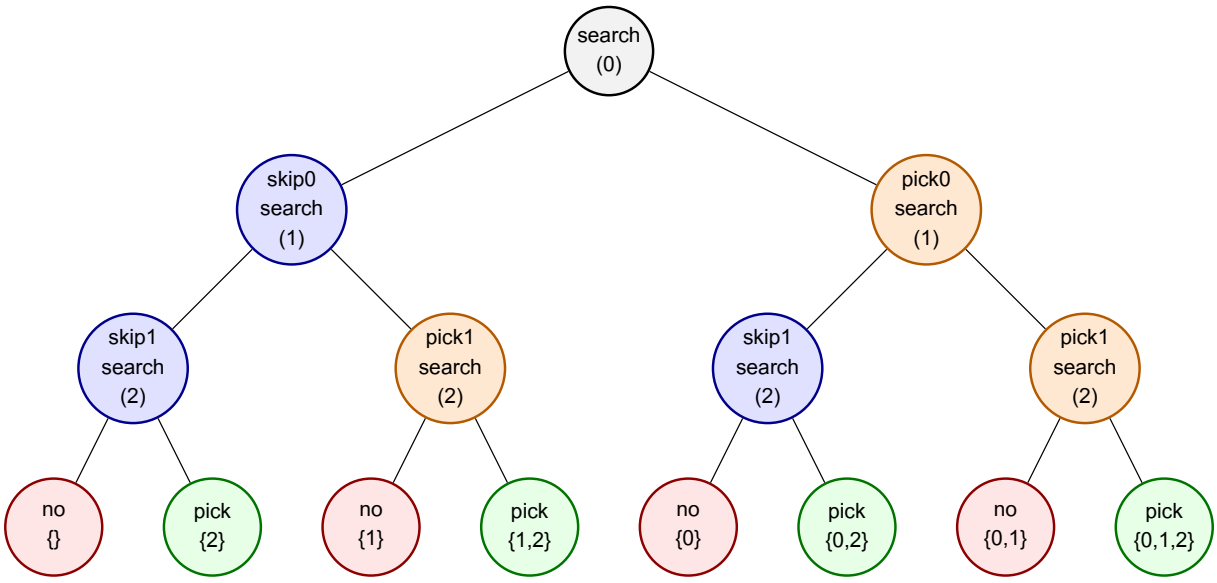
- **ชุดย่อย (Subset):** เลือกสมาชิกบางส่วนจากเซตขนาด n โดยไม่สนลำดับ เช่น จาก $\{0, 1, 2\}$ ชุดย่อยหนึ่งคือ $\{0, 2\}$ จำนวนชุดย่อยทั้งหมดคือ 2^n ชุด (รวมเซตว่าง)
- **การเรียงสับเปลี่ยน (Permutation):** เรียงสมาชิกทุกตัวในทุกลำดับที่เป็นไปได้ เช่น จาก $\{0, 1, 2\}$ การเรียงสับเปลี่ยนหนึ่งคือ $(1, 0, 2)$ จำนวนการเรียงสับเปลี่ยนทั้งหมดคือ $n!$ วิธี

ตารางที่ 7.2 เปรียบเทียบชุดย่อยกับการเรียงสับเปลี่ยน

คุณสมบัติ	ชุดย่อย (Subset)	การเรียงสับเปลี่ยน (Permutation)
สนใจลำดับ?	ไม่สน	สน
จำนวนทั้งหมด	2^n	$n!$
เลือกสมาชิก	บางส่วนหรือทั้งหมด	ทั้งหมด
ตัวอย่าง ($n = 3$)	$\{0, 2\}$, $\{1\}$, $\emptyset$	$(0, 1, 2)$, $(2, 0, 1)$

จะเห็นว่าจำนวนชุดย่อย (2^n) น้อยกว่าจำนวนการเรียงสับเปลี่ยน ($n!$) มากเมื่อ n มีค่ามาก เช่น เมื่อ $n = 20$ จะมีชุดย่อย $2^{20} \approx 10^6$ ชุด แต่มีการเรียงสับเปลี่ยน $20! \approx 2.4 \times 10^{18}$ วิธี ซึ่งต่างกันมาก

เราสามารถวาดต้นไม้เรียกซ้ำ (Recursion Tree) สำหรับการสร้างชุดย่อยของ $\{0, 1, 2\}$ ได้ดังนี้:



รูปที่ 7.1 Search tree with skip/pick decisions

จะเห็นว่าในแต่ละระดับของต้นไม้ เราตัดสินใจว่าจะเลือกหรือไม่เลือกสมาชิกตัวนั้น ทำให้เกิดกิ่ง 2 กิ่งในทุกระดับ รวมเป็น $2^3 = 8$ ใบ ซึ่งตรงกับจำนวนชุดย่อยทั้งหมด

7.2.2 วิธีที่ 1: Recursive Search

วิธีแรกในการสร้างชุดย่อยคือใช้ **การเรียกซ้ำ (Recursion)** แนวคิดคือสำหรับสมาชิกตัวที่ k เรามี 2 ทางเลือก: **ไม่เลือก** หรือ **เลือก** สมาชิกตัวนั้น เมื่อพิจารณาครบทุกสมาชิก ($k = n$) เราจะได้ชุดย่อยหนึ่งชุด

```
#include <bits/stdc++.h>
using namespace std;
int n = 3;
vector<int> subset;
void search(int k) {
    if (k == n) {
```

```
7           // Process current subset
8           cout << "{";
9           for (int i = 0; i < (int)subset.size(); i++) {
10              if (i > 0) cout << ", ";
11              cout << subset[i];
12          }
13          cout << "}\n";
14      } else {
15          // Choice 1: don't select element k
16          search(k + 1);
17          // Choice 2: select element k
18          subset.push_back(k);
19          search(k + 1);
20          subset.pop_back();      // backtrack: remove the just-selected element
21      }
22  }
23  int main() {
24      search(0);
25      return 0;
26  }
```

ตัวอย่างโค้ดที่ 7.4: สร้างชุดย่อยด้วย Recursive Search

ลำดับการเลือกมีผลต่อลำดับผลลัพธ์

ในโค้ดข้างต้น เราเลือก “ไม่เลือก” ก่อนแล้วจึง “เลือก” ทำให้ชุดย่อยถูกสร้างตามลำดับเลขฐานสอง กล่าวคือ ชุดย่อยแรกสุดจะเป็นเซตว่าง $\emptyset$ และชุดย่อยสุดท้ายจะเป็นเซตเต็ม $\{0, 1, 2\}$

ผลลัพธ์ของโปรแกรมสำหรับ $n = 3$:

```
{}
{2}
{1}
{1, 2}
{0}
{0, 2}
{0, 1}
{0, 1, 2}
```

จะเห็นว่ามีชุดย่อยทั้งหมด $2^3 = 8$ ชุด ตรงตามที่คาดไว้

7.2.3 วิธีที่ 2: การแทนด้วยบิต (Bitmask)

วิธีที่สะดวกและมีประสิทธิภาพอีกวิธีหนึ่งคือใช้ **บิต (Bitmask)** แทนชุดย่อย แนวคิดคือชุดย่อยแต่ละชุดสามารถแทนด้วยจำนวนเต็ม n บิต โดย:

- บิตที่ i เป็น 1 หมายถึง สมาชิกตัวที่ i **ถูกเลือก**
- บิตที่ i เป็น 0 หมายถึง สมาชิกตัวที่ i **ไม่ถูกเลือก**

ตัวอย่างเช่น สำหรับ $n = 3$:

- $0 = 000_2$ แทนเซตว่าง $\emptyset$
- $5 = 101_2$ แทนเซต $\{0, 2\}$
- $7 = 111_2$ แทนเซต $\{0, 1, 2\}$

ดังนั้นเราสามารถวนลูปจาก 0 ถึง $2^n - 1$ เพื่อสร้างชุดย่อยทั้งหมด:

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n = 3;
    // Loop from 0 to 2^n - 1
    for (int b = 0; b < (1 << n); b++) {
        cout << "{";
        bool first = true;
        for (int i = 0; i < n; i++) {
            if (b & (1 << i)) {    // Check if bit i is 1 or not
                if (!first) cout << ", ";
                cout << i;
                first = false;
            }
        }
        cout << "}\n";
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 7.5: สร้างชุดย่อยด้วย Bitmask

ผลลัพธ์ของโปรแกรม:

```
{}
{0}
{1}
{0, 1}
```

```
{2}
{0, 2}
{1, 2}
{0, 1, 2}
```

เข้าใจ `1 << n` และ Bitmask

- `1 << n` หมายถึง 2^n — เลื่อนบิต 1 ไปทางซ้าย n ตำแหน่ง
- `1 << i` หมายถึงจำนวนเต็มที่มีเฉพาะบิตที่ i เป็น 1
- `b & (1 << i)` ตรวจสอบว่าบิตที่ i ของ b เป็น 1 หรือไม่ — ถ้าผลลัพธ์ไม่เท่ากับ 0 แสดงว่าเป็น 1
- `b | (1 << i)` ตั้งบิตที่ i ของ b เป็น 1
- `b & ~(1 << i)` ตั้งบิตที่ i ของ b เป็น 0

ตัวอย่าง: หาชุดย่อยที่ผลรวมใกล้เคียงเป้าหมายมากที่สุด

กำหนดแถวลำดับ n จำนวน และค่าเป้าหมาย x จงหาชุดย่อยที่ผลรวมใกล้ x ที่สุด

```cpp
int best = 0;
for (int b = 0; b < (1 << n); b++) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        if (b & (1 << i)) {
            sum += arr[i];
        }
    }
    if (abs(sum - x) < abs(best - x)) {
        best = sum;
    }
}
cout << best << "\n";
```

วิธีนี้มีความซับซ้อน $\mathcal{O}(n \cdot 2^n)$ ซึ่งใช้ได้ดีเมื่อ $n \leq 20$

ตารางที่ 7.3 การดำเนินการบิตที่สำคัญสำหรับ Bitmask

การดำเนินการ	โค้ด C++	ความหมาย
ตรวจสอบบิตที่ i	`b & (1 << i)`	ไม่เท่ากับ 0 แสดงว่าบิตเป็น 1
ตั้งบิตที่ i เป็น 1	`b |= (1 << i)`	เปิดบิตที่ตำแหน่ง i
ตั้งบิตที่ i เป็น 0	`b &= ~(1 << i)`	ปิดบิตที่ตำแหน่ง i
สลับบิตที่ i	`b ^= (1 << i)`	สลับ 0/1 ที่ตำแหน่ง i
นับบิตที่เป็น 1	`__builtin_popcount(b)`	จำนวนบิตที่เป็น 1

ฟังก์ชัน `__builtin_popcount`

GCC มีฟังก์ชัน `__builtin_popcount(b)` ที่นับจำนวนบิตที่เป็น 1 ในจำนวนเต็ม `int` ได้อย่างรวดเร็ว (ใช้คำสั่งเครื่องจักรโดยตรง) สำหรับ `long long` ให้ใช้ `__builtin_popcountll(b)` แทน ฟังก์ชันนี้มีประโยชน์มากในโจทย์ที่เกี่ยวข้องกับ Bitmask

7.3 การสร้างการเรียงสับเปลี่ยน (Generating Permutations)

การเรียงสับเปลี่ยน (Permutation) คือการจัดเรียงสมาชิกทั้งหมดของเซตในทุกลำดับที่เป็นไปได้ สำหรับเซตขนาด n จะมีการเรียงสับเปลี่ยนทั้งหมด $n!$ วิธี ในหัวข้อนี้ เราจะเรียนรู้ 2 วิธีในการสร้างการเรียงสับเปลี่ยน

7.3.1 วิธีที่ 1: Recursive

เช่นเดียวกับการสร้างชุดย่อย เราสามารถใช้การเรียกซ้ำในการสร้างการเรียงสับเปลี่ยนได้ แนวคิดคือเราสร้างการเรียงสับเปลี่ยนทีละตำแหน่ง โดยเลือกสมาชิกที่ยังไม่ถูกเลือกมาใส่ในตำแหน่งปัจจุบัน

เราใช้แถวลำดับ `chosen[]` เพื่อติดตามว่าสมาชิกตัวไหนถูกเลือกไปแล้ว:

```cpp
#include <bits/stdc++.h>
using namespace std;
int n = 3;
vector<int> permutation;
bool chosen[4];    // chosen[i] = true if i has been selected
void search() {
    if ((int)permutation.size() == n) {
        // Process current permutation
        for (int x : permutation) cout << x << " ";
        cout << "\n";
    } else {
        for (int i = 0; i < n; i++) {
            if (chosen[i]) continue;    // Skip if already selected
            chosen[i] = true;
```

```
            permutation.push_back(i);
            search();
            permutation.pop_back();      // backtrack
            chosen[i] = false;
        }
    }
}
int main() {
    search();
    return 0;
}
```

ตัวอย่างโค้ดที่ 7.6: สร้างการเรียงสับเปลี่ยนด้วย Recursive

ผลลัพธ์ของโปรแกรมสำหรับ $n = 3$:

```
0 1 2
0 2 1
1 0 2
1 2 0
2 0 1
2 1 0
```

มีการเรียงสับเปลี่ยนทั้งหมด $3! = 6$ วิธี ตรงตามที่คาดไว้
สังเกตว่าโครงสร้างของโปรแกรมคล้ายกับการสร้างชุดย่อย แต่ต่างกันตรงที่:

- ชุดย่อย: แต่ละสมาชิกมี 2 ทางเลือก (เลือก/ไม่เลือก) $\rightarrow 2^n$ ชุดย่อย
- การเรียงสับเปลี่ยน: ในแต่ละตำแหน่งมีหลายทางเลือก (สมาชิกที่ยังไม่ถูกเลือก) $\rightarrow n!$ วิธี

การเรียกซ้ำสำหรับการเรียงสับเปลี่ยนจะมี “กิ่ง” ที่ไม่เท่ากันในแต่ละระดับ ในระดับแรกมี n ทางเลือก ระดับถัดไปมี $n - 1$ ทางเลือก ไปเรื่อยๆ จนเหลือ 1 ทางเลือก ทำให้จำนวนใบทั้งหมดเท่ากับ $n \times (n - 1) \times \cdots \times 1 = n!$

ความซับซ้อนของการเรียงสับเปลี่ยน

เนื่องจากมีการเรียงสับเปลี่ยน $n!$ วิธี ความซับซ้อนของเวลาจึงเป็น $\mathcal{O}(n!)$ ซึ่งเติบโตอย่างรวดเร็วมาก:

- $n = 10$: $10! = 3\,628\,800$ (ยังพอรับได้)
- $n = 12$: $12! = 479\,001\,600$ (เริ่มช้า)
- $n = 15$: $15! > 10^{12}$ (ช้าเกินไป)

ดังนั้น วิธีนี้ใช้ได้ดีเมื่อ $n \leq 10$ หรือ $n \leq 12$ เท่านั้น

7.3.2 วิธีที่ 2: next_permutation

C++ มีฟังก์ชัน `next_permutation()` ในไลบรารี `<algorithm>` ที่สร้างการเรียงสับเปลี่ยนถัดไปในลำดับพจนานุกรม (Lexicographical Order) โดยอัตโนมัติ ทำให้เราไม่ต้องเขียนฟังก์ชันเรียกซ้ำเอง

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> v = {0, 1, 2};  // Must start sorted in ascending order!
    do {
        for (int x : v) cout << x << " ";
        cout << "\n";
    } while (next_permutation(v.begin(), v.end()));
    return 0;
}
```

ตัวอย่างโค้ดที่ 7.7: สร้างการเรียงสับเปลี่ยนด้วย next_permutation

สำคัญ: ต้องเริ่มจากแถวลำดับที่เรียงลำดับแล้ว!

`next_permutation()` สร้างการเรียงสับเปลี่ยน **ถัดไป** ในลำดับพจนานุกรม ดังนั้นถ้าแถวลำดับเริ่มต้นไม่ได้เรียงจากน้อยไปมาก การเรียงสับเปลี่ยนบางส่วนจะหายไป! ต้อง `sort()` ก่อนเสมอ:

```cpp
sort(v.begin(), v.end());  // Sort first!
do {
    // Process current permutation
} while (next_permutation(v.begin(), v.end()));
```

ตัวอย่าง: หาการเรียงสับเปลี่ยนที่ให้ค่ามากสุด

กำหนดแถวลำดับ n จำนวน จงหาการเรียงสับเปลี่ยนที่ค่าของนิพจน์ $\sum_{i=0}^{n-2} |a_i - a_{i+1}|$ มากที่สุด

```cpp
sort(arr.begin(), arr.end());
int best = 0;
do {
    int sum = 0;
    for (int i = 0; i < n - 1; i++) {
        sum += abs(arr[i] - arr[i + 1]);
    }
    best = max(best, sum);
} while (next_permutation(arr.begin(), arr.end()));
cout << best << "\n";
```

วิธีนี้ลองทุกการเรียงสับเปลี่ยน จึงได้คำตอบที่ถูกต้องแน่นอน

7.4 Backtracking

Backtracking เป็นเทคนิคที่สำคัญมากในการแก้ปัญหาแบบ Complete Search[2, 7] โดยเฉพาะปัญหาที่มีข้อจำกัด (Constraint) หลายอย่าง แนวคิดหลักคือสร้างคำตอบทีละขั้นตอน และเมื่อพบว่าเส้นทางปัจจุบันไม่สามารถนำไปสู่คำตอบที่ถูกต้องได้ ก็จะ “ถอยกลับ” (Backtrack) เพื่อลองเส้นทางอื่น

7.4.1 แนวคิด Backtracking

Backtracking เป็นกลยุทธ์การแก้ปัญหาแบบ **ทดลองและถอยกลับ** (Trial and Error) โดยมีขั้นตอนดังนี้:

1. **เริ่มต้น** ด้วยคำตอบเปล่า (Empty Solution)
2. **ขยาย** คำตอบโดยเพิ่มตัวเลือกที่เป็นไปได้ทีละขั้นตอน
3. **ตรวจสอบ** ว่าคำตอบบางส่วนยังเป็นไปได้หรือไม่ (ยังไม่ขัดกับเงื่อนไข)
4. ถ้าคำตอบบางส่วน **ไม่เป็นไปได้** $\rightarrow$ **ถอยกลับ** (Backtrack) ไปลองตัวเลือกอื่น
5. ถ้าคำตอบบางส่วน **เป็นไปได้** $\rightarrow$ ขยายต่อไป
6. ถ้าคำตอบ **สมบูรณ์** $\rightarrow$ บันทึกคำตอบ

Backtracking vs Brute Force

Backtracking ต่างจาก Brute Force ตรงที่ Backtracking **ตรวจสอบระหว่างทาง** ว่าคำตอบบางส่วนยังเป็นไปได้หรือไม่ ถ้าไม่เป็นไปได้ก็ถอยกลับทันที โดยไม่ต้องสร้างคำตอบเต็มรูปแบบก่อนค่อยตรวจสอบ ทำให้ประหยัดเวลาได้มาก

7.4.2 ปัญหา N-Queens

ปัญหาคลาสสิกที่ใช้ Backtracking แก้ได้ดีที่สุดคือ **N-Queens Problem**: จงวาง Queen n ตัวบนกระดานหมากรุกขนาด $n \times n$ โดยที่ Queen ไม่ตัวใดสามารถจับตัวอื่นได้ (Queen สามารถเคลื่อนที่ในแนวนอน แนวตั้ง และแนวทแยง)

สำหรับ $n = 4$ กระดานมีขนาด 4×4 และมีคำตอบทั้งหมด 2 แบบ:

```
Type 1:          Type 2:
. Q . .           . . Q .
. . . Q          Q . . .
Q . . .           . . . Q
. . Q .          . Q . .
```

กลยุทธ์ในการแก้ปัญหาคือวาง Queen ทีละแถว โดยในแต่ละแถวเราจะลองวาง Queen ในทุกคอลัมน์ที่เป็นไปได้ ถ้าตำแหน่งนั้นปลอดภัย (ไม่ถูก Queen ตัวอื่นจับได้) เราจะวางแล้วไปแถวถัดไป ถ้าไม่ปลอดภัยเราจะข้ามตำแหน่งนั้น

มาดูขั้นตอนการทำงานของ Backtracking สำหรับ $n = 4$ อย่างละเอียด:

```
Row 0: Try placing at column 0
Row 1: Try placing at column 0 -> Captured (vertical) -> Skip
        Try placing at column 1 -> Captured (diagonal) -> Skip
        Try placing at column 2 -> Safe!
    Row 2: Try placing at column 0 -> Captured (vertical) -> Skip
            Try placing at column 1 -> Captured (diagonal) -> Skip
            Try placing at column 2 -> Captured (vertical) -> Skip
            Try placing at column 3 -> Captured (diagonal) -> Skip
            -> No safe column -> BACKTRACK!
... (Keep trying until solution found)
```

จะเห็นว่า Backtracking ช่วยให้เราไม่ต้องลองทุกวิธีที่เป็นไปไม่ได้ เมื่อพบว่าแถว 2 ไม่มีคอลัมน์ที่ปลอดภัย เราจะถอยกลับไปแถว 1 แล้วลองคอลัมน์อื่นแทน

7.4.3 โค้ดแก้ N-Queens

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 8;
int board[N][N];
int cnt = 0;
// Check if Queen can be placed at (row, col)
bool isSafe(int row, int col) {
    // Check vertical (same column) in rows above
    for (int i = 0; i < row; i++) {
        if (board[i][col]) return false;
    }
    // Check upper-left diagonal
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j]) return false;
    }
    // Check upper-right diagonal
    for (int i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++) {
        if (board[i][j]) return false;
    }
    return true;
}
```

```cpp
// Place Queen row by row
void solveNQUtil(int row) {
    if (row >= N) {
        // All rows filled = solution found
        cnt++;
        return;
    }
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;        // Place Queen
            solveNQUtil(row + 1);       // Try next row
            board[row][col] = 0;        // backtrack: remove Queen
        }
    }
}
int main() {
    memset(board, 0, sizeof(board));
    solveNQUtil(0);
    cout << "Number of solutions: " << cnt << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 7.8: แก้ปัญหา N-Queens ด้วย Backtracking

ต้องเขียน isSafe() เอง!

ฟังก์ชัน `isSafe()` เป็นส่วนสำคัญที่สุดของโปรแกรม N-Queens ต้องตรวจสอบให้ครบทั้ง 3 ทิศทาง: แนวตั้ง, แนวทแยงซ้ายบน, และแนวทแยงขวาบน ไม่ต้องตรวจแนวนอนเพราะเราวาง Queen ทีละแถวอยู่แล้ว (แถวละ 1 ตัวแน่นอน) และไม่ต้องตรวจทิศทางล่างเพราะยังไม่ได้วาง Queen ในแถวด้านล่าง

สำหรับ $n = 8$ (กระดานมาตรฐาน) มีคำตอบทั้งหมด 92 แบบ โปรแกรมข้างต้นจะพิมพ์ `Number of solutions: 92`

ตัวอย่าง: ปริ้นกระดาน N-Queens ที่พบ

หากต้องการแสดงกระดานด้วย สามารถเพิ่มโค้ดในส่วน base case ได้:

```cpp
if (row >= N) {
    cnt++;
    // Display board
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```

```
            cout << (board[i][j] ? "Q" : ".") << " ";
        }
        cout << "\n";
    }
    cout << "\n";
    return;
}
```

ตารางที่ 7.4 จำนวนคำตอบของ N-Queens สำหรับ n ต่างๆ

n	จำนวนคำตอบ
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92
10	724
12	14200

7.5 การตัดแต่งกิ่ง (Pruning)

Pruning (การตัดแต่งกิ่ง) เป็นเทคนิคที่สำคัญอย่างยิ่งสำหรับการเร่งความเร็วของ Backtracking[7] แนวคิดคือตรวจจับให้เร็วที่สุดว่าคำตอบบางส่วน (Partial Solution) ไม่สามารถนำไปสู่คำตอบที่ถูกต้องได้ แล้วตัดกิ่งนั้นทิ้งทันทีโดยไม่ต้องสำรวจต่อ เทคนิคนี้สามารถลดเวลาทำงานได้อย่างมาก บางครั้งจากหลายชั่วโมงเหลือเพียงเสี้ยววินาที

7.5.1 แนวคิด Pruning

ใน Backtracking ธรรมดา เราสร้างคำตอบไปเรื่อยๆ จนกว่าจะครบหรือพบว่าขัดเงื่อนไข แต่ในหลายกรณีเราสามารถสังเกตได้ตั้งแต่ early stage ว่าเส้นทางปัจจุบันไม่มีทางนำไปสู่คำตอบที่ถูกต้อง

ยกตัวอย่างเช่น ในปัญหา N-Queens ถ้าเราวาง Queen 2 ตัวแล้วพบว่าจับกันได้ เราไม่จำเป็นต้องลองวาง Queen อีก $n-2$ ตัว เพียงแค่ถอยกลับแล้วลองตำแหน่งอื่นได้เลย นี่คือ Pruning ในรูปแบบที่ง่ายที่สุด

นิยาม 7.2 (Pruning)**.** Pruning คือเทคนิคการตรวจจับล่วงหน้าว่าคำตอบบางส่วนไม่สามารถนำไปสู่คำตอบที่สมบูรณ์ได้ แล้วตัดกิ่งนั้นทิ้งทันที เพื่อไม่เสียเวลาสำรวจเส้นทางที่ไร้ค่า

7.5.2 ตัวอย่าง: นับเส้นทางในตาราง

พิจารณาปัญหาต่อไปนี้: บนตารางขนาด $n \times n$ ให้นับจำนวนเส้นทางจากมุมซ้ายบนไปมุมขวาล่าง ที่ผ่านทุกช่องพอดีทีละครั้ง โดยเคลื่อนที่ได้เฉพาะขึ้น ลง ซ้าย ขวา เท่านั้น

สำหรับ $n = 7$ วิธี Brute Force ธรรมดาใช้เวลาประมาณ 483 วินาที (ประมาณ 8 นาที!) ซึ่งช้าเกินไป แต่ด้วยการใช้ Pruning 4 ขั้นตอน เราสามารถลดเวลาเหลือเพียง 0.6 วินาที ซึ่งเร็วขึ้นเกือบ 1,000 เท่า!

ก่อนอื่น มาดูโครงสร้างพื้นฐานของโปรแกรม Backtracking สำหรับปัญหานี้:

```
#include <bits/stdc++.h>
using namespace std;
int n;
bool visited[10][10];
int dr[] = {-1, 1, 0, 0};   // up, down, left, right
int dc[] = {0, 0, -1, 1};
int cnt = 0;
void search(int r, int c, int steps) {
    // Reached destination and visited all cells
    if (r == n-1 && c == n-1) {
        if (steps == n*n) cnt++;
        return;
    }
    // Try all directions
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
        if (visited[nr][nc]) continue;
        visited[nr][nc] = true;
        search(nr, nc, steps + 1);
        visited[nr][nc] = false;   // backtrack
    }
}
int main() {
    cin >> n;
    memset(visited, false, sizeof(visited));
    visited[0][0] = true;
    search(0, 0, 1);
    cout << cnt << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 7.9: โครงสร้างพื้นฐาน: นับเส้นทางในตาราง

โปรแกรมนี้ทำงานได้ถูกต้อง แต่ช้ามากสำหรับ $n = 7$ จึงต้องใช้ Pruning เพื่อเร่งความเร็ว

7.5.3 4 ขั้นการ Pruning

ขั้นที่ 1: สมมาตร (Symmetry)

เนื่องจากตารางมีสมมาตรแบบซ้าย-ขวา (หรือบน-ล่าง) ทุกเส้นทางที่เริ่มจากการเคลื่อนที่ไปทางขวาจะมีเส้นทางที่สมมาตรซึ่งเริ่มจากการเคลื่อนที่ลง ดังนั้นเราสามารถบังคับให้ก้าวแรกเป็นการเคลื่อนที่ไปทางขวาเท่านั้น แล้วคูณคำตอบด้วย 2 เพื่อให้ได้คำตอบที่ถูกต้อง

ในโค้ด เราสามารถทำได้โดยบังคับให้ก้าวแรกเป็นการเคลื่อนที่ไปทางขวาเท่านั้น:

```
// Instead of plain search(0, 0, 1)
// Force first step to be right
visited[0][0] = true;
visited[0][1] = true;
search(0, 1, 2);
// Final answer = cnt * 2
```

การ Pruning นี้ลดเวลาจาก 483 วินาทีเหลือ 244 วินาที (เร็วขึ้น 2 เท่า)

ขั้นที่ 2: มาถึงจุดหมายเร็วเกินไป (Early Arrival)

ถ้าเส้นทางมาถึงมุมขวาล่างก่อนจะผ่านทุกช่อง เส้นทางนั้นไม่ใช่คำตอบ เราสามารถตรวจสอบได้ทุกขั้นว่าถ้าเราอยู่ติดกับจุดหมายแล้ว แต่ยังเหลือช่องที่ยังไม่ได้ผ่าน เส้นทางนี้ไม่มีทางเป็นคำตอบได้ ให้ตัดทิ้งทันที

ในโค้ด เราสามารถตรวจสอบได้ดังนี้:

```
// Check Early Arrival
if (r == n-1 && c == n-2 && steps < n*n - 1) return;
if (r == n-2 && c == n-1 && steps < n*n - 1) return;
```

การ Pruning นี้ลดเวลาจาก 244 วินาทีเหลือ 119 วินาที (เร็วขึ้นอีก 2 เท่า)

ขั้นที่ 3: กำแพงแบ่ง (Wall Split)

สังเกตว่าถ้าเส้นทางเคลื่อนที่มาถึงขอบของตาราง และสามารถเลี้ยวได้ทั้ง 2 ทิศทาง นั่นหมายความว่าเส้นทางได้แบ่งพื้นที่ที่ยังไม่ได้สำรวจออกเป็น 2 ส่วนที่ไม่ต่อเนื่องกัน เนื่องจากเส้นทางต้องผ่านทุกช่องพอดี การแบ่งพื้นที่เป็น 2 ส่วนจะทำให้ไม่สามารถเข้าถึงส่วนหนึ่งได้ ดังนั้นเส้นทางนี้ไม่เป็นไปได้

การตรวจสอบนี้ซับซ้อนกว่าขั้นก่อนหน้า แต่หลักการคือ: ถ้าเส้นทางเคลื่อนที่มาถึงขอบตาราง และในจุดปัจจุบันสามารถเลี้ยวได้ทั้งซ้ายและขวา (2 ทิศทางขนานกับขอบ) แสดงว่าเส้นทางได้แบ่งพื้นที่ออกเป็น 2 ส่วนที่แยกกัน ซึ่งไม่เป็นไปได้

การ Pruning นี้ลดเวลาจาก 119 วินาทีเหลือ 1.8 วินาที (เร็วขึ้นกว่า 60 เท่า!)

ขั้นที่ 4: กำแพงแบ่งแบบทั่วไป (Generalized Split)

ขั้นที่ 3 ตรวจจับเฉพาะกรณีที่เส้นทางชนขอบตาราง เราสามารถขยายแนวคิดนี้ได้: ถ้าในจุดปัจจุบันเราไม่สามารถเดินตรงได้ (ช่องด้านหน้าถูกใช้แล้ว) แต่สามารถเลี้ยวได้ทั้งซ้ายและขวา นั่นหมายความว่าเส้นทางจะแบ่งพื้นที่ที่ยังไม่ได้สำรวจออกเป็น 2 ส่วน ซึ่งทำให้ไม่สามารถผ่านทุกช่องได้ เราจึงตัดกิ่งนี้ทิ้ง

ในโค้ด เราตรวจสอบว่าในจุดปัจจุบัน ถ้าไม่สามารถเดินตรงได้ (ช่องด้านหน้าถูกใช้แล้วหรือเป็นขอบ) แต่สามารถเลี้ยวได้ทั้งซ้ายและขวา ก็ให้ตัดกิ่งทิ้งทันที เงื่อนไขนี้ครอบคลุมกรณีของขั้นที่ 3 ด้วย

การ Pruning นี้ลดเวลาจาก 1.8 วินาทีเหลือ 0.6 วินาที (เร็วขึ้นอีก 3 เท่า)

7.5.4 ตารางสรุปผลการ Pruning

ตารางที่ 7.5 ผลการ Pruning สำหรับปัญหานับเส้นทางในตาราง 7×7

การ Optimize	เวลา (วินาที)	การเรียกซ้ำ (ครั้ง)
ไม่มี (Brute Force)	483	$> 10^{10}$
+ ขั้น 1: สมมาตร	244	$\approx 5 \times 10^9$
+ ขั้น 2: Early Arrival	119	$\approx 2.5 \times 10^9$
+ ขั้น 3: Wall Split	1.8	$\approx 3 \times 10^7$
+ ขั้น 4: Generalized Split	0.6	$\approx 10^7$

จะเห็นว่า Pruning ขั้นที่ 3 ให้ผลดีที่สุด (ลดเวลาจาก 119 วินาทีเหลือ 1.8 วินาที) เนื่องจากมันตัดกิ่งที่ "ไร้ค่า" ออกไปจำนวนมหาศาล โดยรวมแล้ว การ Pruning ทั้ง 4 ขั้นทำให้เวลาลดลงจาก 483 วินาทีเหลือ 0.6 วินาที เร็วขึ้นเกือบ 1,000 เท่า!

เคล็ดลับ: การ Pruning ที่ดีที่สุดมักเกิดจากการสังเกตเหตุผลเชิงโครงสร้าง

การ Pruning ที่มีประสิทธิภาพสูงสุดมักมาจากการสังเกตโครงสร้างเฉพาะของปัญหา ไม่ใช่แค่การตรวจสอบเงื่อนไขทั่วไป ยิ่งเราเข้าใจโครงสร้างของปัญหามากเท่าไหร่ ก็ยิ่ง Pruning ได้มากเท่านั้น ขั้นตอนแรกๆ ของการ Pruning (เช่น สมมาตร) มักให้ผลดีที่สุดเมื่อเทียบกับความยากในการนำไปใช้

7.6 Meet in the Middle

Meet in the Middle เป็นเทคนิคที่ทรงพลังมากสำหรับลดเวลาทำงานของ Complete Search[3, 11] โดยแบ่งช่องการค้นหา (Search Space) ออกเป็น 2 ส่วนเท่าๆ กัน ค้นหาแต่ละส่วนแยกกัน แล้วจึงรวมผลลัพธ์ เทคนิคนี้สามารถลดความซับซ้อนจาก $\mathcal{O}(2^n)$ เหลือ $\mathcal{O}\left(2^{n/2}\right)$ ซึ่งเร็วขึ้นอย่างมาก

7.6.1 แนวคิด

สมมติว่าเราต้องค้นหาคำตอบในช่องการค้นหาขนาด 2^n แทนที่จะค้นหาทั้งหมดในครั้งเดียว (ซึ่งใช้เวลา $\mathcal{O}(2^n)$) เราแบ่งปัญหาออกเป็น 2 ส่วน:

1. แบ่งข้อมูลนำเข้าเป็นสองกลุ่ม A และ B ขนาดเท่าๆ กัน (ประมาณ $n/2$ ตัว)
2. สร้างชุดคำตอบที่เป็นไปได้ทั้งหมดจากกลุ่ม A (ขนาด $2^{n/2}$)
3. สร้างชุดคำตอบที่เป็นไปได้ทั้งหมดจากกลุ่ม B (ขนาด $2^{n/2}$)

4. รวมคำตอบจากทั้งสองกลุ่มเพื่อหาคำตอบสุดท้าย

เนื่องจากแต่ละกลุ่มมีขนาด $2^{n/2}$ เวลารวมจึงเป็น $\mathcal{O}\left(2^{n/2}\right)$ ซึ่งเร็วกว่า $\mathcal{O}(2^n)$ มาก เช่น ถ้า $n = 40$:

- Brute Force: $2^{40} \approx 10^{12}$ (ช้าเกินไป)
- Meet in the Middle: $2^{20} \approx 10^6$ (ใช้ได้!)

7.6.2 ตัวอย่าง: Subset Sum

ปัญหา Subset Sum: กำหนดจำนวนเต็ม n จำนวนและค่าเป้าหมาย x จงตรวจสอบว่ามีชุดย่อยใดของจำนวนทั้งหมดที่ผลรวมเท่ากับ x หรือไม่

วิธี Brute Force: ลองทุกชุดย่อย $\rightarrow$ ความซับซ้อน $\mathcal{O}(2^n)$

วิธี Meet in the Middle: แบ่งรายการเป็น 2 ส่วน แล้วหาผลรวมของชุดย่อยในแต่ละส่วน

7.6.3 ขั้นตอน Meet in the Middle

กำหนดรายการตัวเลข $[2, 4, 5, 9]$ และค่าเป้าหมาย $x = 15$

ขั้นที่ 1: แบ่งรายการเป็นสองกลุ่ม:

- $A = [2, 4]$
- $B = [5, 9]$

ขั้นที่ 2: สร้างชุดผลรวมของชุดย่อยทั้งหมดจาก A:

$$S_A = \{0, 2, 4, 6\}$$

(เซตว่าง=0, {2}=2, {4}=4, {2,4}=6)

ขั้นที่ 3: สร้างชุดผลรวมของชุดย่อยทั้งหมดจาก B:

$$S_B = \{0, 5, 9, 14\}$$

(เซตว่าง=0, {5}=5, {9}=9, {5,9}=14)

ขั้นที่ 4: ตรวจสอบว่ามี $a \in S_A$ และ $b \in S_B$ ที่ $a + b = 15$ หรือไม่:

- $0 + 15 = 15$? ไม่มี 15 ใน S_B
- $2 + 13 = 15$? ไม่มี 13 ใน S_B
- $4 + 11 = 15$? ไม่มี 11 ใน S_B
- $6 + 9 = 15$? มี 9 ใน S_B! $\rightarrow$ พบ!

ดังนั้นมีชุดย่อยที่ผลรวมเท่ากับ 15 ได้แก่ $\{2, 4\}$ จาก A (ผลรวม 6) และ $\{9\}$ จาก B (ผลรวม 9) รวม $6 + 9 = 15$

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, x;
    cin >> n >> x;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];
    // Split into 2 groups
    int mid = n / 2;
    vector<int> left(arr.begin(), arr.begin() + mid);
    vector<int> right(arr.begin() + mid, arr.end());
    // Generate subset sums of left group
    vector<long long> SA;
    for (int b = 0; b < (1 << (int)left.size()); b++) {
        long long sum = 0;
        for (int i = 0; i < (int)left.size(); i++) {
            if (b & (1 << i)) sum += left[i];
        }
        SA.push_back(sum);
    }
    // Generate subset sums of right group
    vector<long long> SB;
    for (int b = 0; b < (1 << (int)right.size()); b++) {
        long long sum = 0;
        for (int i = 0; i < (int)right.size(); i++) {
            if (b & (1 << i)) sum += right[i];
        }
        SB.push_back(sum);
    }
    // Sort SB for binary search
    sort(SB.begin(), SB.end());
    // Check if a + b = x exists
    bool found = false;
    for (long long a : SA) {
        long long need = x - a;
        if (binary_search(SB.begin(), SB.end(), need)) {
            found = true;
            break;
        }
    }
    cout << (found ? "YES" : "NO") << "\n";
```

```
42      return 0;
43  }
```

ตัวอย่างโค้ดที่ 7.10: Subset Sum ด้วย Meet in the Middle

7.6.4 การวิเคราะห์ความซับซ้อน

วิเคราะห์ความซับซ้อนของวิธี Meet in the Middle:

1. **สร้าง** S_A**:** มี $2^{n/2}$ ชุดย่อย แต่ละชุดใช้เวลา $\mathcal{O}(n/2)$ ในการคำนวณผลรวม $\rightarrow \mathcal{O}\big((n/2) \cdot 2^{n/2}\big)$
2. **สร้าง** S_B**:** เช่นเดียวกัน $\rightarrow \mathcal{O}\big((n/2) \cdot 2^{n/2}\big)$
3. **เรียง** S_B**:** ขนาด $2^{n/2} \rightarrow \mathcal{O}\big(2^{n/2} \log(2^{n/2})\big) = \mathcal{O}\big((n/2) \cdot 2^{n/2}\big)$
4. **ค้นหา:** สำหรับแต่ละ $a \in S_A$ (มี $2^{n/2}$ ตัว) ใช้ Binary Search $\mathcal{O}\big(\log(2^{n/2})\big) = \mathcal{O}(n/2) \rightarrow \mathcal{O}\big((n/2) \cdot 2^{n/2}\big)$

รวมทั้งหมด: $\mathcal{O}\big(n \cdot 2^{n/2}\big)$
เปรียบเทียบกับ Brute Force ที่ใช้เวลา $\mathcal{O}(n \cdot 2^n)$:

ตารางที่ 7.6 เปรียบเทียบเวลา: Brute Force vs Meet in the Middle

n	**Brute Force (2^n)**	**Meet in the Middle ($2^{n/2}$)**
20	$\approx 10^6$	$\approx 10^3$
30	$\approx 10^9$	$\approx 3 \times 10^4$
40	$\approx 10^{12}$	$\approx 10^6$
50	$\approx 10^{15}$	$\approx 3 \times 10^7$

จะเห็นว่า Meet in the Middle ทำให้เราสามารถแก้ปัญหาที่ n มีค่ามากขึ้นได้อย่างมาก จาก $n \approx 20$ (Brute Force) เป็น $n \approx 40$ (Meet in the Middle)

ข้อจำกัดของ Meet in the Middle

Meet in the Middle ใช้ได้ดีเฉพาะเมื่อ:

- ช่องการค้นหาสามารถแบ่งออกเป็น 2 ส่วนได้อย่างเท่าๆ กัน
- สามารถรวมผลลัพธ์จาก 2 ส่วนได้อย่างมีประสิทธิภาพ (เช่น ด้วย Binary Search)
- มีหน่วยความจำเพียงพอที่จะเก็บผลลัพธ์จากแต่ละส่วน (ขนาด $2^{n/2}$ ตัว)

ถ้า $n = 40$ เราต้องเก็บ $2^{20} \approx 10^6$ ค่าในหน่วยความจำ ซึ่งยังพอรับได้ แต่ถ้า $n = 60$ ต้องเก็บ $2^{30} \approx 10^9$ ค่า ซึ่งอาจใช้หน่วยความจำเกินขีดจำกัด

ตัวอย่าง: หาชุดย่อยที่ผลรวมใกล้เคียง x ที่สุดด้วย Meet in the Middle

กำหนด $n = 34$ ตัวเลข และค่าเป้าหมาย x หาชุดย่อยที่ผลรวมใกล้ x ที่สุด

Brute Force: $2^{34} \approx 1.7 \times 10^{10}$ ซึ่งช้าเกินไป

Meet in the Middle: แบ่งเป็นกลุ่มละ 17 ตัว $\rightarrow 2^{17} = 131072$ ต่อกลุ่ม ซึ่งทำได้สบาย!

```cpp
// Build SA and SB, then find pair where a + b is closest to x
// Sort SB and use lower_bound to find complement
for (long long a : SA) {
    long long need = x - a;
    auto it = lower_bound(SB.begin(), SB.end(), need);
    // Check *it and *(it-1) to see which is closer to x
    if (it != SB.end()) {
        long long sum = a + *it;
        if (abs(sum - x) < abs(best - x)) best = sum;
    }
    if (it != SB.begin()) {
        long long sum = a + *(it - 1);
        if (abs(sum - x) < abs(best - x)) best = sum;
    }
}
```

เทคนิคสำคัญคือต้องตรวจสอบทั้ง `*it` (ค่าแรกที่ $\geq$ need) และ `*(it-1)` (ค่าสุดท้ายที่ $<$ need) เพราะคู่ที่ใกล้ที่สุดอาจมาจากทิศทางใดก็ได้

7.7 Generating vs. Filtering และ Pre-Computation

ในหัวข้อนี้ เราจะศึกษาเทคนิคเชิงกลยุทธ์ที่ช่วยเพิ่มประสิทธิภาพให้กับการแก้ปัญหาแบบ Complete Search ได้อย่างมาก ไม่ว่าจะเป็นการเลือกวิธีสร้างคำตอบที่เหมาะสม การใช้พื้นที่แลกเวลา หรือการมองปัญหาในมุมที่ต่างออกไป

7.7.1 Generating vs. Filtering

เมื่อต้องการหาคำตอบที่ตรงตามเงื่อนไข เรามี 2 แนวทางหลัก:

1. **Filtering (การกรอง):** สร้างคำตอบที่เป็นไปได้ทั้งหมด แล้วกรองเอาเฉพาะคำตอบที่ถูกต้อง
2. **Generating (การสร้าง):** สร้างเฉพาะคำตอบที่ถูกต้องโดยตรง โดยไม่สร้างคำตอบที่ผิดเงื่อนไขเลย

Generating vs. Filtering

Filtering เขียนง่ายกว่าแต่ช้ากว่า เพราะต้องสร้างและตรวจสอบทุกคำตอบ **Generating** เขียนยากกว่าแต่เร็วกว่า เพราะสร้างเฉพาะคำตอบที่ถูกต้อง ลดขนาดพื้นที่ค้นหาได้มาก

ยกตัวอย่างเช่น สมมติเราต้องการสร้าง Permutation ที่สมาชิกตัวแรกต้องเป็นเลขคู่:

```
// Approach 1: Filtering (generate all, then keep valid)
vector<vector<int>> ans;
vector<int> perm = {0, 1, 2, 3};
sort(perm.begin(), perm.end());
do {
    if (perm[0] % 2 == 0) {    // Filter: first element must be even
        ans.push_back(perm);
    }
} while (next_permutation(perm.begin(), perm.end()));
// Generates 4! = 24 permutations, keeps only 12 valid ones
```

ตัวอย่างโค้ดที่ 7.11: Filtering: สร้างทั้งหมดแล้วกรอง

```
// Approach 2: Generating (construct only valid candidates)
vector<vector<int>> ans;
vector<int> perm;
bool chosen[4] = {};
void search(int pos) {
    if (pos == 4) {
        ans.push_back(perm);
        return;
    }
    for (int i = 0; i < 4; i++) {
        if (chosen[i]) continue;
        // Prune: first position must be even
        if (pos == 0 && i % 2 != 0) continue;
        chosen[i] = true;
        perm.push_back(i);
        search(pos + 1);
        perm.pop_back();
        chosen[i] = false;
    }
}
// Only generates 12 valid permutations directly
```

ตัวอย่างโค้ดที่ 7.12: Generating: สร้างเฉพาะคำตอบที่ถูกต้อง

วิธี Generating ตัดกิ่งที่ไม่จำเป็นออกตั้งแต่ต้น ทำให้ไม่เสียเวลาสร้างและตรวจสอบคำตอบที่ผิดเงื่อนไข

7.7.2 Pre-Computation / Pre-Calculation

Pre-Computation คือการคำนวณคำตอบบางส่วนไว้ล่วงหน้า แล้วเก็บไว้ในตาราง (Lookup Table) เพื่อดึงมาใช้ได้ทันทีในภายหลัง แนวคิดหลักคือ **แลกพื้นที่ (Space) เพื่อเวลา (Time)**

แลก Space แลก Time

Pre-Computation ใช้หน่วยความจำเพิ่มขึ้น แต่ลดเวลาค้นหา/คำนวณได้มาก ในการแข่งขัน ขีดจำกัดหน่วยความจำมักอยู่ที่ 256 MB ซึ่งเพียงพอสำหรับตารางขนาดกลาง ดังนั้นการแลกพื้นที่เพื่อเวลามักคุ้มค่าเสมอ

ตัวอย่างที่ 1: **Sieve of Eratosthenes** — สร้างตารางจำนวนเฉพาะไว้ล่วงหน้า แล้วตอบคำถาม "x เป็นจำนวนเฉพาะหรือไม่" ได้ในเวลา $\mathcal{O}(1)$

```
const int MAXN = 1000001;
bool isPrime[MAXN];
void sieve() {
    memset(isPrime, true, sizeof(isPrime));
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i < MAXN; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j < MAXN; j += i) {
                isPrime[j] = false;
            }
        }
    }
    // Now: isPrime[x] answers "is x prime?" in O(1)
}
```

ตัวอย่างโค้ดที่ 7.13: Pre-Computation: Sieve of Eratosthenes

ตัวอย่างที่ 2: **Factorial Pre-Computation** — สร้างตาราง Factorial ไว้ล่วงหน้าเพื่อตอบคำถาม nCr ได้อย่างรวดเร็ว

```
const int MOD = 1e9 + 7;
const int MAXN = 1000001;
long long fac[MAXN], invFac[MAXN];
long long power(long long base, long long exp, long long mod) {
    long long ans = 1;
    while (exp > 0) {
        if (exp % 2 == 1) ans = ans * base % mod;
        base = base * base % mod;
        exp /= 2;
    }
```

```
    return ans;
}
void precompute() {
    fac[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        fac[i] = fac[i - 1] * i % MOD;
    }
    invFac[MAXN - 1] = power(fac[MAXN - 1], MOD - 2, MOD);
    for (int i = MAXN - 2; i >= 0; i--) {
        invFac[i] = invFac[i + 1] * (i + 1) % MOD;
    }
    // Now: nCr = fac[n] * invFac[r] % MOD * invFac[n-r] % MOD
}
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fac[n] * invFac[r] % MOD * invFac[n - r] % MOD;
}
```

ตัวอย่างโค้ดที่ 7.14: Pre-Computation: Factorial สำหรับ nCr

ตัวอย่างที่ 3: **Lookup Table สำหรับ Input เล็ก** — บางครั้งค่าที่เป็นไปได้มีจำนวนจำกัด เราสามารถคำนวณคำตอบทั้งหมดไว้ล่วงหน้าแล้วเก็บในแถวลำดับ

```
// Pre-compute number of set bits for all 8-bit values (0..255)
int popcountTable[256];
void precompute() {
    for (int i = 0; i < 256; i++) {
        popcountTable[i] = __builtin_popcount(i);
    }
    // Now: popcountTable[x] gives number of 1-bits in x instantly
}
// Count set bits in a 32-bit integer using the memo table
int countBits(int x) {
    return popcountTable[x & 0xFF]
        + popcountTable[(x >> 8) & 0xFF]
        + popcountTable[(x >> 16) & 0xFF]
        + popcountTable[(x >> 24) & 0xFF];
}
```

ตัวอย่างโค้ดที่ 7.15: Pre-Computation: Lookup Table

7.7.3 Decomposition (การแยกส่วน)

Decomposition คือการแบ่งปัญหาออกเป็นส่วนย่อยที่เป็นอิสระต่อกัน แก้แต่ละส่วนด้วยขั้นตอนวิธีที่เหมาะสมที่สุด แล้วนำผลลัพธ์มารวมกัน

ตัวอย่างเช่น โจทย์อาจถามหาจำนวนคู่ (i, j) ที่ $a_i + a_j = x$ เราอาจแบ่งเป็น 2 ส่วน:

1. ส่วนที่ 1: เรียงแถวลำดับด้วย $\mathcal{O}(n \log n)$
2. ส่วนที่ 2: ใช้ Two Pointers นับจำนวนคู่ในเวลา $\mathcal{O}(n)$

ผลรวมของเวลาทั้งสองส่วนคือ $\mathcal{O}(n \log n)$ ซึ่งดีกว่าการใช้ Brute Force $\mathcal{O}(n^2)$

Decomposition ในโลกจริง

Decomposition พบได้บ่อยในโจทย์ที่ซับซ้อน เช่น โจทย์ที่ต้องหาค่าบางอย่างบน Tree อาจแบ่งเป็น (1) สร้าง Tree (2) คำนวณบน Tree หรือโจทย์ที่ต้องประมวลผลข้อความ อาจแบ่งเป็น (1) แยกคำ (2) นับความถี่ การแบ่งปัญหาให้เป็นส่วนย่อยช่วยให้เลือกขั้นตอนวิธีที่ดีที่สุดสำหรับแต่ละส่วนได้

7.7.4 Symmetry (สมมาตร)

Symmetry คือการใช้สมมาตรของปัญหาเพื่อลดขนาดพื้นที่ค้นหา ถ้าปัญหามีสมมาตร เราสามารถคำนวณเพียงส่วนหนึ่งแล้วคูณด้วยจำนวนสมมาตรได้

ประเภทของสมมาตรที่พบบ่อย:

- **2-way symmetry:** สะท้อนซ้าย-ขวา ลดพื้นที่ค้นหาลดครึ่งหนึ่ง (คูณด้วย 2)
- **4-way symmetry:** สมมาตร 4 ทิศทาง (บน-ล่าง-ซ้าย-ขวา) ลดพื้นที่ค้นหาเหลือ 1/4 (คูณด้วย 4)
- **8-way symmetry:** สมมาตร 8 ทิศทาง (รวมการหมุนและสะท้อน) ลดพื้นที่ค้นหาเหลือ 1/8 (คูณด้วย 8)

ตัวอย่าง: ใช้ Symmetry ลดการค้นหา

สมมติต้องนับจำนวนวิธีวางวัตถุบนกระดาน $n \times n$ ที่มีสมมาตร 4 ทิศทาง แทนที่จะลองวางบนทุกช่อง เราสามารถลองวางเฉพาะช่องในส่วน 1/4 ของกระดาน แล้วคูณคำตอบด้วย 4 ได้เลย (ต้องระวังกรณีช่องที่อยู่บนเส้นสมมาตร)
เช่น สำหรับกระดาน 4×4 แทนที่จะพิจารณา 16 ช่อง อาจพิจารณาเพียง 4 ช่องในจตุภาคเดียว แล้วคูณด้วย 4

7.7.5 Solving Forward vs. Backward (แก้ไปข้างหน้า vs. ย้อนกลับ)

บางครั้งการแก้ปัญหาจากท้ายกลับไปหน้า (Backward) ง่ายกว่าการแก้จากหน้าไปท้าย (Forward) ตัวอย่างเช่น:

- โจทย์ที่ถาม “ต้องทำอะไรบ้างเพื่อไปถึงเป้าหมาย” อาจง่ายกว่าถ้าเริ่มจากเป้าหมายแล้วย้อนกลับ
- โจทย์ที่มีเงื่อนไขที่ชัดเจนที่จุดสิ้นสุด แต่เงื่อนไขที่จุดเริ่มต้นไม่ชัดเจน
- โจทย์ Dynamic Programming หลายข้อสามารถคิดย้อนกลับได้ง่ายกว่า

ลองคิดย้อนกลับ!

เมื่อติดขัดกับปัญหา ลองถามตัวเองว่า “ถ้าเริ่มจากคำตอบแล้วย้อนกลับ จะง่ายขึ้นไหม?” บ่อยครั้งที่มุมมองย้อนกลับทำให้เห็นรูปแบบหรือสูตรที่ซ่อนอยู่

ตัวอย่าง: หาจำนวนน้อยสุดที่ต้องหารเพื่อให้ได้ 1

กำหนดจำนวนเต็ม n ต้องการหาจำนวนขั้นต่ำสุดที่ต้องดำเนินการ (หารด้วย 2, 3, หรือลบ 1) เพื่อให้ได้ 1

วิธี Forward: ลองทุกเส้นทางจาก n ไป 1 — อาจมีหลายทางเลือก

วิธี Backward: เริ่มจาก 1 แล้วถามว่า “จะถึง n ได้อย่างไร?” — โครงสร้างการค้นหาจะชัดเจนขึ้น เพราะจาก 1 เราสามารถคูณ 2, คูณ 3, หรือบวก 1 ซึ่งเป็นการดำเนินการที่ชัดเจน

```
// Bottom-up DP (from 1 up to n)
int minSteps(int n) {
    vector<int> dp(n + 1, 0);
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + 1;    // Option: subtract 1
        if (i % 2 == 0) dp[i] = min(dp[i], dp[i / 2] + 1);
        if (i % 3 == 0) dp[i] = min(dp[i], dp[i / 3] + 1);
    }
    return dp[n];
}
```

วิธีนี้ใช้หลักคิดจาก 1 ขึ้นไปหา n (Forward ในมุมของ DP) ซึ่งง่ายกว่าการคิดจาก n ย้อนลงไปหา 1 แบบ Recursive

7.8 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับการฝึกฝนเทคนิค Complete Search จากแพลตฟอร์มต่างๆ:

โจทย์แนะนำสำหรับฝึก Complete Search

Codeforces[10]:

- 1167A - Telephone Number (ลูปง่าย)
- 1097B - Petr and a Combination Lock (Bitmask)

- 1178D - Prime Graph (วนลูปหาจำนวนเฉพาะ)
- 888C - Palindromic Matrix (ลูปซ้อน)

AtCoder:

- ABC100 C - *3 or /2 (Complete Search)
- ABC104 C - All Green (Bitmask + Greedy)
- ABC180 E - Traveling Salesman among Aerial Cities (Bitmask DP)

CSES:

- 1623 - Apple Division (ชุดย่อย)
- 1642 - Sum of Two Values (Meet in the Middle)
- 1652 - Creating Strings (การเรียงสับเปลี่ยน)
- 2205 - Gray Code (Bitmask)
- 1070 - Permutations (การเรียงสับเปลี่ยน)
- 1622 - Creating Strings II (การเรียงสับเปลี่ยน)

UVA[20]:

- 524 - Prime Ring Problem (Backtracking)
- 750 - 8 Queens Chess Problem (Backtracking)
- 129 - Krypton Factor (Backtracking)

7.9 สรุปบทที่ 7

ในบทนี้ เราได้เรียนรู้เทคนิค Complete Search ตั้งแต่พื้นฐานไปจนถึงขั้นสูง ตั้งแต่การค้นหาแบบง่ายๆ ด้วยลูป ไปจนถึงเทคนิค Backtracking, Pruning และ Meet in the Middle ที่ช่วยให้เราแก้โจทย์ที่ซับซ้อนได้อย่างมีประสิทธิภาพ

7.9.1 ตารางสรุปเทคนิค Complete Search

ตารางที่ 7.7 สรุปเทคนิค Complete Search

เทคนิค	ความซับซ้อน	เมื่อใดควรใช้
ลูป 1 ชั้น	$\mathcal{O}(n)$	ค้นหาธาตุ 1 ตัวจาก n
ลูป 2 ชั้น	$\mathcal{O}(n^2)$	ค้นหาคู่ธาตุจาก n
ชุดย่อย (Bitmask)	$\mathcal{O}(n \cdot 2^n)$	เลือกสมาชิกบางส่วน, $n \leq 20$
ชุดย่อย (Recursive)	$\mathcal{O}(n \cdot 2^n)$	เช่นเดียวกับ Bitmask แต่ยืดหยุ่นกว่า
การเรียงสับเปลี่ยน	$\mathcal{O}(n!)$	ลองทุกลำดับ, $n \leq 10$
Backtracking	ขึ้นกับปัญหา	มีข้อจำกัดหลายเงื่อนไข
Pruning	ขึ้นกับปัญหา	เร่ง Backtracking ให้เร็วขึ้น
Meet in the Middle	$\mathcal{O}\left(n \cdot 2^{n/2}\right)$	แบ่งซ่องค้นหาเป็น 2 ส่วน, $n \leq 40$

กฎง่าย ๆ: Complete Search ก่อน!

เมื่อเจอโจทย์ใหม่ ให้ลองคิด Complete Search ก่อนเสมอ แม้ว่ามันอาจจะช้าเกินไป แต่มันช่วยให้เราเข้าใจโครงสร้างของปัญหาได้ดีขึ้น จากนั้นค่อยพิจารณาว่าจะปรับปรุงด้วย Pruning หรือใช้เทคนิคอื่น เช่น Greedy หรือ Dynamic Programming แทน หลายครั้ง Complete Search กับ Pruning ดี ๆ ก็เพียงพอต่อการแก้โจทย์ได้

สรุปสิ่งสำคัญของบทที่ 7

1. Complete Search ลองทุกความเป็นไปได้ — ง่ายแต่ถูกต้องแน่นอน
2. ใช้ Bitmask สร้างชุดย่อยเมื่อ $n \leq 20$
3. ใช้ `next_permutation` สร้างการเรียงสับเปลี่ยนเมื่อ $n \leq 10$
4. Backtracking ช่วยลดการค้นหาโดยถอยกลับเมื่อเจอทางตัน
5. Pruning ช่วยเร่ง Backtracking ได้มาก — บางครั้งเร็วขึ้น 1,000 เท่า
6. Meet in the Middle ลด $\mathcal{O}(2^n)$ เหลือ $\mathcal{O}\left(2^{n/2}\right)$ — แก้ได้เมื่อ $n \approx 40$

3

ชุดปัญหาด้านคณิตศาสตร์

ส่วนนี้ครอบคลุมคณิตศาสตร์เชิงคำนวณที่ใช้ในการแข่งขันเขียนโปรแกรม ตั้งแต่พื้นฐานทฤษฎีจำนวน คอมบินาทอริกซ์ ลอการิทึม การคำนวณเชิงโมดุลาร์ ไปจนถึงเทคนิคทางคณิตศาสตร์ที่จำเป็นสำหรับการแก้ปัญหาในการแข่งขัน เนื้อหาในส่วนนี้เป็นพื้นฐานสำคัญที่นักศึกษาจะนำไปใช้ต่อในหัวข้อขั้นสูงเช่นกำหนดการพลวัตและเรขาคณิตเชิงคำนวณ

Chapter 8

คณิตศาสตร์พื้นฐาน

คณิตศาสตร์เป็นรากฐานสำคัญของการโปรแกรมเชิงแข่งขัน โจทย์จำนวนมากต้องใช้ความรู้ทางคณิตศาสตร์ตั้งแต่พื้นฐานจนถึงขั้นสูง ในบทนี้ เราจะทบทวนความรู้ทางคณิตศาสตร์ที่จำเป็นสำหรับการแข่งขัน ตั้งแต่สัญลักษณ์และสัญกรณ์ ผลบวกและผลคูณ อนุกรม ทฤษฎีเซต ตรรกะ ไปจนถึงขั้นตอนวิธีพื้นฐานอย่าง GCD และ LCM รวมถึงทฤษฎีจำนวนเบื้องต้นและคอมบินาทอริกซ์ที่สำคัญสำหรับการแข่งขัน[2]

8.1 สัญลักษณ์และสัญกรณ์คณิตศาสตร์

ในการโปรแกรมเชิงแข่งขัน เรามักพบสัญลักษณ์และสัญกรณ์ทางคณิตศาสตร์ในโจทย์และเอกสารอ้างอิง ความเข้าใจในสัญลักษณ์เหล่านี้จะช่วยให้อ่านโจทย์และเขียนเฉลยได้อย่างมีประสิทธิภาพ สัญลักษณ์ที่พบบ่อยแบ่งได้เป็นสองกลุ่มหลัก ได้แก่ อักษรกรีกที่ใช้แทนตัวแปรหรือค่าคงที่ทางคณิตศาสตร์ และสัญลักษณ์ตรรกะและเซตที่ใช้ในการเขียนนิพจน์อย่างกระชับ

8.1.1 อักษรกรีก

อักษรกรีกถูกใช้อย่างกว้างขวางในคณิตศาสตร์และวิทยาศาสตร์ ตารางต่อไปนี้แสดงอักษรกรีกที่พบบ่อยในการแข่งขันพร้อมความหมายทั่วไป:

ตารางที่ 8.1 อักษรกรีกที่พบบ่อยในการโปรแกรมเชิงแข่งขัน

สัญลักษณ์	ชื่อ	ความหมายทั่วไป
Σ	Sigma	ผลบวก (Summation)
Π	Pi	ผลคูณ (Product)
α	Alpha	มุม, ค่าคงที่ทั่วไป
β	Beta	มุม, ค่าคงที่ทั่วไป
θ	Theta	มุม, ขอบเขตเวลา Θ
π	Pi	ค่าคงที่ $\pi \approx 3.14159$
φ	Phi	ฟังก์ชันออยเลอร์โทเทียนท์
λ	Lambda	ค่าคงที่, ฟังก์ชันนิรนาม
ω	Omega	ขอบเขตล่าง Ω
Ω	Omega	ขอบเขตล่าง (Big-Omega)

8.1.2 สัญลักษณ์ตรรกะและเซต

สัญลักษณ์ตรรกะและเซตใช้สำหรับเขียนนิพจน์ทางคณิตศาสตร์ให้กระชับและแม่นยำ โดยเฉพาะในโจทย์ที่เกี่ยวข้องกับทฤษฎีจำนวนและคอมบินาทอริกซ์:

ตารางที่ 8.2 สัญลักษณ์ตรรกะและเซตที่สำคัญ

สัญลักษณ์	ชื่อ	ความหมาย
$\exists$	Existential quantifier	มีอยู่ (There exists)
$\forall$	Universal quantifier	สำหรับทุก (For all)
$\in$	Element of	เป็นสมาชิกของ
$\notin$	Not element of	ไม่เป็นสมาชิกของ
$\subset$	Subset	เป็นสับเซตแท้
$\subseteq$	Subset or equal	เป็นสับเซต
$\cup$	Union	ยูเนียน
$\cap$	Intersection	อินเตอร์เซกชัน
$\emptyset$	Empty set	เซตว่าง
$\Rightarrow$	Implies	แสดงว่า
$\Leftrightarrow$	If and only if	ก็ต่อเมื่อ

การใช้สัญลักษณ์ในการแข่งขัน

ในการแข่งขัน ไม่จำเป็นต้องจำสัญลักษณ์ทุกตัว แต่ควรจำสัญลักษณ์ที่พบบ่อย เช่น Σ (ผลบวก), Π (ผลคูณ), $\in$ (เป็นสมาชิก), $\forall$ (สำหรับทุก), และ $\exists$ (มีอยู่) เพราะสัญลักษณ์เหล่านี้ปรากฏในโจทย์เป็นประจำ

8.2 ผลบวกและผลคูณ (Summation and Product)

ผลบวก (Summation) และผลคูณ (Product) เป็นสัญกรณ์ที่ใช้แทนการบวกหรือคูณจำนวนหลายจำนวนเข้าด้วยกันอย่างกระชับ สัญกรณ์เหล่านี้พบได้บ่อยมากในโจทย์การแข่งขัน ทั้งในรูปแบบของสูตรคณิตศาสตร์และการวิเคราะห์ความซับซ้อนของขั้นตอนวิธี

8.2.1 สัญกรณ์ผลบวก (Summation Notation)

สัญกรณ์ผลบวก Σ ใช้เขียนแทนการบวกจำนวนหลายจำนวน:

$$\sum_{k=1}^{n} f(k) = f(1) + f(2) + f(3) + \cdots + f(n)$$

สูตรผลบวกที่สำคัญและใช้บ่อยที่สุดมีดังนี้:

ทฤษฎีบท 8.1 (สูตรผลบวกพื้นฐาน).

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \tag{8.1}$$

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} \tag{8.2}$$

$$\sum_{k=1}^{n} k^3 = \left(\frac{n(n+1)}{2}\right)^2 \tag{8.3}$$

ตัวอย่าง: คำนวณผลบวก k^2+5k-4 จาก k

เราสามารถแยกผลบวกได้ดังนี้:

$$\sum_{k=1}^{100}(k^2+5k-4) = \sum_{k=1}^{100} k^2 + 5\sum_{k=1}^{100} k - \sum_{k=1}^{100} 4$$

คำนวณทีละพจน์:

$$\sum_{k=1}^{100} k^2 = \frac{100 \cdot 101 \cdot 201}{6} = 338350$$

$$5\sum_{k=1}^{100} k = 5 \cdot \frac{100 \cdot 101}{2} = 25250$$

$$\sum_{k=1}^{100} 4 = 400$$

ดังนั้น $338350 + 25250 - 400 = 363200$

สมบัติของผลบวก

ผลบวกมีสมบัติเชิงเส้น (Linearity) ที่สำคัญ:

- $\sum_{k=m}^{n}(a \cdot f(k) + b \cdot g(k)) = a\sum_{k=m}^{n} f(k) + b\sum_{k=m}^{n} g(k)$
- $\sum_{k=m}^{n} c = c \cdot (n-m+1)$ สำหรับค่าคงที่ c

สมบัติเหล่านี้ช่วยให้เราแยกผลบวกที่ซับซ้อนเป็นส่วนที่ง่ายกว่าได้

8.2.2 สัญกรณีผลคูณ (Product Notation)

สัญกรณีผลคูณ Π ใช้เขียนแทนการคูณจำนวนหลายจำนวน:

$$\prod_{k=1}^{n} f(k) = f(1) \times f(2) \times f(3) \times \cdots \times f(n)$$

8.2.3 แฟกทอเรียล (Factorial)

แฟกทอเรียลเป็นกรณีพิเศษของผลคูณที่สำคัญมากในการแข่งขัน:

นิยาม 8.1 (แฟกทอเรียล)**.** สำหรับจำนวนเต็มไม่เป็นลบ n แฟกทอเรียลของ n นิยามว่า:

$$n! = \prod_{k=1}^{n} k = 1 \times 2 \times 3 \times \cdots \times n, \qquad 0! = 1$$

ตารางต่อไปนี้แสดงค่าแฟกทอเรียลของจำนวนเต็มบางค่า:

ตารางที่ 8.3 ค่าแฟกทอเรียลของจำนวนเต็ม $0 \le n \le 20$

n	$n!$
0	1
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
12	479001600
13	6227020800
15	1307674368000
20	2432902008176640000

กับดัก: Overflow ของ Factorial

ค่า $n!$ เติบโตอย่างรวดเร็วมาก:

- $12! = 479001600$ ยังพอเก็บใน `int` ได้ (ค่าสูงสุด $\approx 2 \times 10^9$)

- $13! = 6227020800$ เกินขอบเขต `int` แล้ว — ต้องใช้ `long long`
- $21! = 51090942171709440000$ เกินขอบเขต `long long` (ค่าสูงสุด $\approx 9 \times 10^{18}$) แล้ว

ดังนั้นหากโจทย์ต้องการ $n!$ เมื่อ $n \geq 21$ ต้องใช้เลขคณิตมอดูลาร์หรือ Big Integer

8.3 การหารลงตัว (Divisibility)

การหารลงตัวเป็นแนวคิดพื้นฐานที่สำคัญที่สุดในทฤษฎีจำนวน และเป็นรากฐานของหัวข้อหลายๆ เรื่องในบทนี้ เช่น จำนวนเฉพาะ การแยกตัวประกอบ และ GCD/LCM ความเข้าใจในการหารลงตัวจะช่วยให้แก้โจทย์ทฤษฎีจำนวนได้อย่างมีประสิทธิภาพ

นิยาม 8.2 (การหารลงตัว)**.** กำหนดให้ d และ n เป็นจำนวนเต็ม โดย $d \neq 0$ เรากล่าวว่า d หาร n ลงตัว (เขียนแทนด้วย $d \mid n$) ก็ต่อเมื่อมีจำนวนเต็ม k ที่ทำให้ $n = dk$ กล่าวอีกนัยหนึ่ง $n \bmod d = 0$

8.3.1 สมบัติของการหารลงตัว

การหารลงตัวมีสมบัติสำคัญดังต่อไปนี้:

ทฤษฎีบท 8.2 (สมบัติการหารลงตัว)**.**

1. **การถ่ายทอด (Transitivity):** ถ้า $d \mid m$ และ $m \mid n$ แล้ว $d \mid n$
2. **ความเป็นเชิงเส้น (Linearity):** ถ้า $d \mid m$ และ $d \mid n$ แล้ว $d \mid (am + bn)$ สำหรับทุกจำนวนเต็ม a, b
3. **การหารด้วยผลคูณ:** ถ้า $d \mid a$ แล้ว $d \mid ab$ สำหรับทุกจำนวนเต็ม b

ตัวอย่างการหารลงตัว

- $3 \mid 12$ เพราะ $12 = 3 \times 4$
- $5 \mid 100$ เพราะ $100 = 5 \times 20$
- $7 \nmid 15$ เพราะ $15 \bmod 7 = 1 \neq 0$
- จากสมบัติเชิงเส้น: $3 \mid 12$ และ $3 \mid 21$ ดังนั้น $3 \mid (5 \times 12 - 2 \times 21) = 3 \mid 18$ (จริง)

8.4 อนุกรม (Series)

อนุกรม (Series) คือผลบวกของลำดับจำนวน ในการแข่งขัน เรามักพบอนุกรม 3 ประเภทหลัก ได้แก่ อนุกรมเลขคณิต (Arithmetic Progression), อนุกรมเรขาคณิต (Geometric Progression) และอนุกรมฮาร์โมนิก (Harmonic Series) แต่ละประเภทมีสูตรและสมบัติเฉพาะที่สำคัญสำหรับการแก้โจทย์

8.4.1 อนุกรมเลขคณิต (Arithmetic Progression)

อนุกรมเลขคณิตคือลำดับที่ผลต่างระหว่างพจน์ติดกันเป็นค่าคงที่ เรียกว่าผลต่างร่วม (Common Difference) d

นิยาม 8.3 (อนุกรมเลขคณิต)**.** พจน์ที่ n ของอนุกรมเลขคณิตที่มีพจน์แรก a_1 และผลต่างร่วม d คือ:

$$a_n = a_1 + (n-1)d$$

ผลบวกของอนุกรมเลขคณิต n พจน์แรกคือ:

$$S_n = \frac{n(a_1 + a_n)}{2} = \frac{n}{2}\big(2a_1 + (n-1)d\big)$$

ตัวอย่าง: ผลบวก 1+2+3+...+100

เป็นอนุกรมเลขคณิตที่ $a_1 = 1$, $a_n = 100$, $n = 100$:

$$S_{100} = \frac{100(1+100)}{2} = \frac{100 \times 101}{2} = 5050$$

8.4.2 อนุกรมเรขาคณิต (Geometric Progression)

อนุกรมเรขาคณิตคือลำดับที่อัตราส่วนระหว่างพจน์ติดกันเป็นค่าคงที่ เรียกว่าอัตราส่วนร่วม (Common Ratio) r

นิยาม 8.4 (อนุกรมเรขาคณิต)**.** พจน์ที่ n ของอนุกรมเรขาคณิตที่มีพจน์แรก a_1 และอัตราส่วนร่วม r คือ:

$$a_n = a_1 \cdot r^{n-1}$$

ผลบวกของอนุกรมเรขาคณิต n พจน์แรก (เมื่อ $r \neq 1$) คือ:

$$S_n = a_1 \cdot \frac{r^n - 1}{r - 1}$$

ตัวอย่าง: 2+6+18+54+... (10 พจน์)

เป็นอนุกรมเรขาคณิตที่ $a_1 = 2$, $r = 3$, $n = 10$:

$$S_{10} = 2 \cdot \frac{3^{10} - 1}{3 - 1} = \frac{2(59049 - 1)}{2} = 59048$$

8.4.3 อนุกรมฮาร์โมนิก (Harmonic Series)

อนุกรมฮาร์โมนิกคือผลบวกของส่วนกลับของจำนวนเต็มบวก:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^{n} \frac{1}{k}$$

อนุกรมฮาร์โมนิกมีสมบัติสำคัญคือเติบโตอย่างช้าๆ (Logarithmic Growth):

$$H_n \leq \log_2 n + 1$$

ความสำคัญของอนุกรมฮาร์โมนิก

อนุกรมฮาร์โมนิกมีบทบาทสำคัญในการวิเคราะห์ความซับซ้อนของขั้นตอนวิธี เช่น:

- ตะแกรงเอราทอสเธเนสมีความซับซ้อน $\mathcal{O}(n \log \log n)$ ซึ่งเกี่ยวข้องกับ H_n
- การวิเคราะห์ความซับซ้อนของลูป `for(i=1; i<=n; i++) for(j=i; j<=n; j+=i)` ให้ผลรวมเป็น $\sum_{i=1}^{n} \lfloor n/i \rfloor \approx n \cdot H_n = \mathcal{O}(n \log n)$

8.5 ทฤษฎีเซตและตรรกะ

ทฤษฎีเซตและตรรกะเป็นรากฐานของคณิตศาสตร์สมัยใหม่ และมีประโยชน์อย่างยิ่งในการโปรแกรมเชิงแข่งขัน เพราะช่วยให้เราคิดวิเคราะห์อย่างเป็นระบบ ตรวจสอบความถูกต้องของเงื่อนไข และเข้าใจความสัมพันธ์ระหว่างกลุ่มข้อมูลต่างๆ

8.5.1 การดำเนินการบนเซต

การดำเนินการพื้นฐานบนเซตมีดังนี้:

- **ยูเนียน (Union):** $A \cup B = \{x \mid x \in A \text{ หรือ } x \in B\}$
- **อินเตอร์เซกชัน (Intersection):** $A \cap B = \{x \mid x \in A \text{ และ } x \in B\}$
- **ผลต่าง (Difference):** $A \setminus B = \{x \mid x \in A \text{ และ } x \notin B\}$
- **ส่วนเติมเต็ม (Complement):** $A^c = U \setminus A$ โดยที่ U เป็นเอกภพสัมพัทธ์

ทฤษฎีบท 8.3 (กฎเดอมอร์แกน (De Morgan's Laws))**.**

$$(A \cup B)^c = A^c \cap B^c \tag{8.4}$$

$$(A \cap B)^c = A^c \cup B^c \tag{8.5}$$

กฎเดอมอร์แกนเป็นสมบัติที่มีประโยชน์มากในการแปลงนิพจน์เชิงตรรกะและเซต ช่วยให้สลับระหว่างยูเนียนกับอินเตอร์เซกชันได้

8.5.2 เพาเวอร์เซต (Power Set)

เพาเวอร์เซตของเซต A คือเซตของเซตย่อยทั้งหมดของ A ถ้า $|A| = n$ แล้วเพาเวอร์เซตมีจำนวนสมาชิก 2^n ตัว นี่เป็นเหตุผลว่าทำไมการวนลูปผ่านเซตย่อยทั้งหมดของเซตขนาด n จึงมีความซับซ้อน $\mathcal{O}(2^n)$

8.5.3 เซตของจำนวนที่สำคัญ

เซตของจำนวนที่สำคัญในคณิตศาสตร์มีดังนี้:

ตารางที่ 8.4 เซตของจำนวนที่สำคัญ

สัญลักษณ์	ชื่อ	ตัวอย่าง
$\mathbb{N}$	จำนวนธรรมชาติ	$0, 1, 2, 3, \ldots$
$\mathbb{Z}$	จำนวนเต็ม	$\ldots, -2, -1, 0, 1, 2, \ldots$
$\mathbb{Q}$	จำนวนตรรกยะ	$\frac{1}{2}, -\frac{3}{7}, 0.75$
$\mathbb{R}$	จำนวนจริง	$\pi, e, \sqrt{2}, -1.5$

สังเกตว่า $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$

8.5.4 ตรรกะ (Logic)

ตรรกะเป็นเครื่องมือพื้นฐานในการให้เหตุผลและพิสูจน์ ในการแข่งขัน ความเข้าใจในตรรกะช่วยในการวิเคราะห์เงื่อนไขของโจทย์และตรวจสอบความถูกต้องของโปรแกรม

ตารางค่าความจริง (Truth Table):

ตารางที่ 8.5 ตารางค่าความจริงสำหรับ $\neg$, $\wedge$, $\vee$, $\Rightarrow$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$
T	T	F	T	T	T
T	F	F	F	T	F
F	T	T	F	T	T
F	F	T	F	F	T

ตัวปริมาณ (Quantifiers):

- $\forall x\, P(x)$: สำหรับทุก x, $P(x)$ เป็นจริง
- $\exists x\, P(x)$: มี x อย่างน้อยหนึ่งตัวที่ทำให้ $P(x)$ เป็นจริง

การปฏิเสธตัวปริมาณ

กฎสำคัญที่ใช้บ่อย:

- $\neg(\forall x\, P(x)) \equiv \exists x\, \neg P(x)$
- $\neg(\exists x\, P(x)) \equiv \forall x\, \neg P(x)$

คิดง่ายๆ: ปฏิเสธ "ทุก" กลายเป็น "มีอย่างน้อยหนึ่งที่ไม่" และปฏิเสธ "มีอยู่" กลายเป็น "ทุกอันไม่"

8.6 ฟังก์ชันพื้นฐาน

ฟังก์ชันพื้นฐานทางคณิตศาสตร์ที่ใช้บ่อยในการแข่งขัน ได้แก่ ฟังก์ชันปัดลง (Floor), ปัดขึ้น (Ceiling), ค่าต่ำสุด (min), และค่าสูงสุด (max) แม้จะดูเรียบง่าย แต่การใช้ฟังก์ชันเหล่านี้อย่างถูกต้องมีความสำคัญอย่างยิ่งต่อความถูกต้องของโปรแกรม

8.6.1 ฟังก์ชันปัดลงและปัดขึ้น (Floor and Ceiling)

นิยาม 8.5 (Floor and Ceiling).

- **Floor:** $\lfloor x \rfloor$ คือจำนวนเต็มที่มากที่สุดที่ไม่เกิน x (ปัดลง)
- **Ceiling:** $\lceil x \rceil$ คือจำนวนเต็มที่น้อยที่สุดที่ไม่น้อยกว่า x (ปัดขึ้น)

ตัวอย่าง: $\lfloor 3.7 \rfloor = 3$, $\lceil 3.2 \rceil = 4$, $\lfloor -2.3 \rfloor = -3$, $\lceil -2.3 \rceil = -2$

ระวังการใช้ Floor กับจำนวนลบ

ใน C++ การหารจำนวนเต็มจะปัดเศษ **ไปทางศูนย์** (Truncation) ไม่ใช่ปัดลง (Floor) ดังนั้น:

- `-7 / 2` ได้ `-3` (ปัดเศษไปทางศูนย์)
- แต่ $\lfloor -7/2 \rfloor = \lfloor -3.5 \rfloor = -4$

หากต้องการ Floor ที่ถูกต้อง ให้ใช้ฟังก์ชัน `floor()` จาก `<cmath>`

8.6.2 ฟังก์ชัน min และ max

ฟังก์ชัน `min` และ `max` เป็นฟังก์ชันที่ใช้บ่อยมากในการแข่งขัน C++ มีฟังก์ชันเหล่านี้ใน STL:

```
#include <algorithm>
int a = min(3, 7);          // a = 3
int b = max(3, 7);          // b = 7
// min/max of multiple values
int c = min({3, 1, 4, 1, 5});  // c = 1 (C++11)
int d = max({3, 1, 4, 1, 5});  // d = 5 (C++11)
```

```cpp
// min/max in array
int arr[] = {3, 1, 4, 1, 5, 9};
int mn = *min_element(arr, arr+6);  // mn = 1
int mx = *max_element(arr, arr+6);  // mx = 9
```

ตัวอย่างโค้ดที่ 8.1: การใช้ min และ max ใน C++

8.7 จำนวนฟีโบนักชี (Fibonacci Numbers)

จำนวนฟีโบนักชีเป็นลำดับจำนวนที่มีชื่อเสียงและปรากฏในโจทย์การแข่งขันบ่อยครั้ง ทั้งในรูปแบบของโจทย์โดยตรงและเป็นส่วนหนึ่งของขั้นตอนวิธีที่ซับซ้อนกว่า ความเข้าใจในจำนวนฟีโบนักชีและสมบัติของมันจึงเป็นทักษะพื้นฐานที่สำคัญ

นิยาม 8.6 (จำนวนฟีโบนักชี)**.** ลำดับฟีโบนักชีนิยามโดย:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad \text{สำหรับ } n \geq 2$$

พจน์แรกๆ ของลำดับคือ: $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots$

8.7.1 สูตรของบิเน (Binet's Formula)

จำนวนฟีโบนักชีสามารถคำนวณได้โดยตรงจากสูตรปิด (Closed-form Formula):

ทฤษฎีบท 8.4 (สูตรของบิเน)**.**

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$$

โดยที่ $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (อัตราส่วนทองคำ) และ $\psi = \frac{1-\sqrt{5}}{2} \approx -0.618$

เนื่องจาก $|\psi| < 1$ เมื่อ n มีค่ามาก ψ^n จะมีค่าใกล้ศูนย์มาก ดังนั้น:

$$F_n \approx \frac{\varphi^n}{\sqrt{5}}$$

8.7.2 อัตราส่วนทองคำ (Golden Ratio)

อัตราส่วนทองคำ $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$ เป็นค่าคงที่ทางคณิตศาสตร์ที่มีสมบัติพิเศษหลายประการ:

- $\varphi^2 = \varphi + 1$
- $\frac{1}{\varphi} = \varphi - 1$
- $\lim_{n\to\infty} \frac{F_{n+1}}{F_n} = \varphi$

8.7.3 เทคนิคสำหรับการแข่งขัน

ในการแข่งขัน การคำนวณจำนวนฟีโบนักชีมักทำด้วยวิธีวนลูปธรรมดา ซึ่งมีความซับซ้อน $\mathcal{O}(n)$:

```cpp
// Compute F(n) mod m iteratively
long long fibonacci(long long n, long long m) {
    if (n <= 1) return n % m;
    long long a = 0, b = 1;
    for (long long i = 2; i <= n; i++) {
        long long c = (a + b) % m;
        a = b;
        b = c;
    }
    return b;
}
```

ตัวอย่างโค้ดที่ 8.2: คำนวณจำนวนฟีโบนักชีแบบวนลูป

ค่า Fibonacci เติบโตเร็วมาก

- $F_{46} = 1836311903$ พอเก็บใน `int` ได้
- $F_{47} = 2971215073$ เกินขอบเขต `int` แล้ว
- $F_{92} = 7540113804746346429$ พอเก็บใน `long long` ได้
- F_{93} เกินขอบเขต `long long` แล้ว

หากโจทย์ต้องการ F_n เมื่อ $n \geq 93$ มักจะให้ตอบ mod m เพื่อหลีกเลี่ยงปัญหา overflow

เทคนิค Matrix Exponentiation

สำหรับ n ที่มีค่ามากๆ (เช่น $n \leq 10^{18}$) เราสามารถคำนวณ F_n mod m ได้ในเวลา $\mathcal{O}(\log n)$ โดยใช้ Matrix Exponentiation:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

รายละเอียดของเทคนิค Matrix Exponentiation ผู้อ่านสามารถศึกษาเพิ่มเติมได้จากเอกสารอ้างอิง[2]

8.8 ลอการิทึม (Logarithm)

ลอการิทึมเป็นฟังก์ชันผกผันของการยกกำลัง และเป็นเครื่องมือที่ขาดไม่ได้ในการโปรแกรมเชิงแข่งขัน ทั้งในด้านการวิเคราะห์ความซับซ้อนของขั้นตอนวิธี การแก้โจทย์คณิตศาสตร์ และการประยุกต์ใช้ในขั้นตอนวิธีต่างๆ เช่น Binary Search

8.8.1 นิยามและสมบัติ

นิยาม 8.7 (ลอการิทึม). $\log_b x = y$ ก็ต่อเมื่อ $b^y = x$ โดยที่ $b > 0$, $b \neq 1$, $x > 0$

ลอการิทึมมีสมบัติสำคัญดังต่อไปนี้:

ทฤษฎีบท 8.5 (สมบัติของลอการิทึม).

$$\log_b(xy) = \log_b x + \log_b y \tag{8.6}$$

$$\log_b\left(\frac{x}{y}\right) = \log_b x - \log_b y \tag{8.7}$$

$$\log_b(x^k) = k \cdot \log_b x \tag{8.8}$$

$$\log_b x = \frac{\log_k x}{\log_k b} \tag{8.9}$$

สมการ (8.9) เรียกว่าสูตรเปลี่ยนฐาน (Change of Base Formula) ซึ่งมีประโยชน์มากเพราะช่วยให้เราแปลงลอการิทึมฐานใดๆ เป็นฐานอื่นได้

8.8.2 ลอการิทึมธรรมชาติ (Natural Logarithm)

ลอการิทึมธรรมชาติใช้ฐาน $e \approx 2.71828$:

$$\ln x = \log_e x$$

ใน C++ สามารถคำนวณลอการิทึมได้ดังนี้:

```cpp
#include <cmath>
log(x);       // ln(x) -- natural logarithm
log2(x);      // log_2(x)
log10(x);     // log_10(x)
log(x)/log(b);  // log_b(x) -- change of base
```

ตัวอย่างโค้ดที่ 8.3: ฟังก์ชันลอการิทึมใน C++

8.8.3 การนับหลัก (Digit Counting)

ลอการิทึมมีประโยชน์ในการนับจำนวนหลักของจำนวนเต็มบวก:

ทฤษฎีบท 8.6 (จำนวนหลักของจำนวนเต็ม). จำนวนหลักของจำนวนเต็มบวก n ในฐาน b คือ:

$$\text{จำนวนหลัก} = \lfloor \log_b n \rfloor + 1$$

ตัวอย่าง: จำนวนหลักของ 10^9

$\lfloor \log_{10} 10^9 \rfloor + 1 = 9 + 1 = 10$ หลัก (จริง เพราะ $10^9 = 1000000000$)

8.8.4 การใช้ลอการิทึมในการวิเคราะห์ขั้นตอนวิธี

ลอการิทึมปรากฏในความซับซ้อนของขั้นตอนวิธีหลายตัว:

- Binary Search: $\mathcal{O}(\log n)$
- Merge Sort: $\mathcal{O}(n \log n)$
- การยกกำลังแบบ Binary: $\mathcal{O}(\log n)$
- Segment Tree: $\mathcal{O}(\log n)$ ต่อการสืบค้น

ลอการิทึมคือ “การนับว่าหารได้กี่ครั้ง”

วิธีเข้าใจลอการิทึมอย่างเป็นสังหรณ์: $\log_2 n$ บอกว่าต้องหาร n ด้วย 2 กี่ครั้งจึงจะได้ 1 ตัวอย่างเช่น $\log_2 1024 = 10$ เพราะต้องหาร 1024 ด้วย 2 จำนวน 10 ครั้งจึงจะได้ 1

8.9 ตัวหารร่วมมากและตัวคูณร่วมน้อย (GCD and LCM)

ตัวหารร่วมมาก (Greatest Common Divisor — GCD) และตัวคูณร่วมน้อย (Least Common Multiple — LCM) เป็นแนวคิดพื้นฐานในทฤษฎีจำนวนที่มีประโยชน์อย่างยิ่งในการแข่งขัน GCD และ LCM ปรากฏในโจทย์หลายประเภท ตั้งแต่โจทย์คำนวณพื้นฐานไปจนถึงโจทย์ทฤษฎีจำนวนขั้นสูง[2, 3]

8.9.1 ตัวหารร่วมมาก (GCD)

นิยาม 8.8 (ตัวหารร่วมมาก)**.** ตัวหารร่วมมากของจำนวนเต็ม a และ b (เขียนแทนด้วย $\gcd(a, b)$) คือจำนวนเต็มบวกที่มากที่สุดที่หารทั้ง a และ b ลงตัว

8.9.2 ขั้นตอนวิธีของยูคลิด (Euclidean Algorithm)

ขั้นตอนวิธีของยูคลิดเป็นวิธีคำนวณ GCD ที่มีประสิทธิภาพสูง โดยอาศัยสมบัติ:

$$\gcd(a, b) = \gcd(b, a \bmod b), \qquad \gcd(a, 0) = a$$

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

ตัวอย่างโค้ดที่ 8.4: ขั้นตอนวิธีของยูคลิดแบบเรียกซ้ำ

```cpp
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

ตัวอย่างโค้ดที่ 8.5: ขั้นตอนวิธีของยูคลิดแบบวนลูป

ขั้นตอนวิธีของยูคลิดมีความซับซ้อน $\mathcal{O}(\log(\min(a,b)))$ ซึ่งรวดเร็วมาก แม้สำหรับค่า a, b ที่มีขนาดใหญ่

ใช้ __gcd ใน C++

C++ มีฟังก์ชัน `__gcd(a, b)` ใน `<algorithm>` (GCC) หรือ `std::gcd(a, b)` ใน C++17 (ใน `<numeric>`) อยู่แล้ว สามารถใช้ได้ทันทีโดยไม่ต้องเขียนเอง:

```cpp
#include <algorithm>
int g = __gcd(12, 18);  // g = 6 (GCC)
```

8.9.3 ตัวคูณร่วมน้อย (LCM)

นิยาม 8.9 (ตัวคูณร่วมน้อย). ตัวคูณร่วมน้อยของจำนวนเต็ม a และ b (เขียนแทนด้วย $\text{lcm}(a,b)$) คือจำนวนเต็มบวกที่น้อยที่สุดที่หารด้วยทั้ง a และ b ลงตัว

ความสัมพันธ์ระหว่าง GCD และ LCM คือ:

$$\text{lcm}(a,b) = \frac{a \times b}{\gcd(a,b)}$$

```cpp
long long lcm(long long a, long long b) {
    return a / __gcd(a, b) * b;  // Divide before multiply to reduce overflow risk
}
```

ตัวอย่างโค้ดที่ 8.6: คำนวณ LCM อย่างปลอดภัยจาก Overflow

ระวัง Overflow ในการคำนวณ LCM

ห้ามเขียน `a * b / __gcd(a, b)` เพราะ `a * b` อาจเกินขอบเขตของ `long long` ก่อนจะหาร วิธีที่ถูกต้องคือหารด้วย GCD ก่อนแล้วค่อยคูณ: `a / __gcd(a, b) * b`

8.10 กลยุทธ์การแก้ปัญหาทางคณิตศาสตร์

ในการแข่งขัน โจทย์คณิตศาสตร์มักต้องใช้ความคิดสร้างสรรค์และกลยุทธ์ในการแก้ปัญหา ไม่ใช่แค่ความรู้สูตรอย่างเดียว หัวข้อนี้นำเสนอกลยุทธ์พื้นฐาน 4 ข้อที่ช่วยให้แก้โจทย์คณิตศาสตร์ได้อย่างมีประสิทธิภาพ

8.10.1 หารูปแบบ (Find Patterns)

การหารูปแบบเป็นกลยุทธ์ที่ทรงพลังที่สุดอย่างหนึ่ง เมื่อเจอโจทย์ที่ดูซับซ้อน ให้ลองคำนวณค่าสำหรับกรณีเล็กๆ ก่อน แล้วดูว่ามีรูปแบบ (Pattern) ใดๆ หรือไม่ รูปแบบที่พบมักนำไปสู่สูตรหรือขั้นตอนวิธีที่มีประสิทธิภาพ

ตัวอย่าง: หารูปแบบ

โจทย์: มีวิธีวางกระเบื้องขนาด 1×2 บนพื้นที่ขนาด $2 \times n$ กี่วิธี?
ลองคำนวณสำหรับ n เล็กๆ:

- $n = 1$: 1 วิธี
- $n = 2$: 2 วิธี
- $n = 3$: 3 วิธี
- $n = 4$: 5 วิธี
- $n = 5$: 8 วิธี

จะเห็นรูปแบบชัดเจน: $1, 2, 3, 5, 8, \ldots$ นี่คือลำดับฟีโบนักชี! คำตอบคือ F_{n+1}

8.10.2 ทำตาราง (Make a List/Table)

การทำตารางค่าสำหรับกรณีเล็กๆ ช่วยให้เห็นรูปแบบและตรวจสอบสมมติฐานได้ ตารางยังช่วยในการตรวจสอบว่าสูตรที่คิดไว้ถูกต้องหรือไม่ โดยการเปรียบเทียบค่าจากสูตรกับค่าที่คำนวณด้วยมือ

8.10.3 วาดรูป (Draw a Picture)

การวาดรูปช่วยให้เข้าใจโจทย์ได้ดีขึ้น โดยเฉพาะโจทย์ที่เกี่ยวข้องกับเรขาคณิต กราฟ หรือความสัมพันธ์เชิงพื้นที่ การวาดรูปช่วยให้มองเห็นความสัมพันธ์ที่อาจมองข้ามได้หากคิดในหัวอย่างเดียว

8.10.4 ทำย้อนกลับ (Work Backwards)

บางครั้งการเริ่มจากเป้าหมายแล้วทำย้อนกลับไปหาจุดเริ่มต้นจะง่ายกว่าการเริ่มจากจุดเริ่มต้นไปหาเป้าหมาย กลยุทธ์นี้มีประโยชน์มากในโจทย์ที่มีเงื่อนไขขาออกที่ชัดเจนแต่เงื่อนไขขาเข้าซับซ้อน

สรุปกลยุทธ์การแก้ปัญหา

1. **หารูปแบบ** — คำนวณกรณีเล็กๆ แล้วมองหา Pattern
2. **ทำตาราง** — จัดระเบียบข้อมูลให้เห็นความสัมพันธ์
3. **วาดรูป** — ใช้ภาพช่วยเข้าใจโจทย์
4. **ทำย้อนกลับ** — เริ่มจากเป้าหมายแล้วย้อนกลับ

กลยุทธ์เหล่านี้ไม่ได้ใช้แยกกัน แต่มักใช้ร่วมกัน เช่น อาจเริ่มจากวาดรูป แล้วทำตาราง แล้วจึงหารูปแบบจากตาราง

8.11 จำนวนเฉพาะและตะแกรงเอราทอสเธเนส (Prime Numbers and Sieve of Eratosthenes)

จำนวนเฉพาะ (Prime Number) เป็นหัวใจสำคัญของทฤษฎีจำนวน และเป็นแนวคิดที่ปรากฏในโจทย์การแข่งขันบ่อยครั้ง ทั้งในรูปแบบของการตรวจสอบจำนวนเฉพาะ การนับจำนวนเฉพาะ และการประยุกต์ใช้ในขั้นตอนวิธีอื่นๆ เช่น การแยกตัวประกอบจำนวนเฉพาะและการเข้ารหัส

นิยาม 8.10 (จำนวนเฉพาะ)**.** จำนวนเฉพาะคือจำนวนเต็ม $n > 1$ ที่มีตัวหารบวกเพียง 2 ตัว คือ 1 และ n เท่านั้น จำนวนเต็มที่มากกว่า 1 และไม่ใช่จำนวนเฉพาะเรียกว่าจำนวนประกอบ (Composite Number)

จำนวนเฉพาะตัวแรกๆ ได้แก่: $2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, \ldots$
สังเกตว่า 2 เป็นจำนวนเฉพาะคู่เพียงตัวเดียว จำนวนเฉพาะทุกตัวที่เหลือเป็นเลขคี่

8.11.1 การทดสอบจำนวนเฉพาะแบบหารลอง (Trial Division)

วิธีง่ายที่สุดในการตรวจสอบว่า n เป็นจำนวนเฉพาะหรือไม่คือการลองหารด้วยจำนวนเต็มตั้งแต่ 2 ถึง $\sqrt{n}$ หากมีตัวใดหารลงตัว แสดงว่า n ไม่ใช่จำนวนเฉพาะ

ทฤษฎีบท 8.7 (ขอบเขตการตรวจสอบจำนวนเฉพาะ)**.** ถ้า n เป็นจำนวนประกอบ แล้ว n มีตัวหารที่เป็นจำนวนเฉพาะไม่เกิน $\sqrt{n}$

Proof. สมมติว่า $n = ab$ โดยที่ $a \leq b$ และ $a > \sqrt{n}$ แล้ว $b \geq a > \sqrt{n}$ ดังนั้น $ab > \sqrt{n} \cdot \sqrt{n} = n$ ซึ่งขัดแย้งกับ $ab = n$ ดังนั้นต้องมีอย่างน้อยหนึ่งตัวหารที่ $\leq \sqrt{n}$ □

```cpp
bool isPrime(long long n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return false;
```

```
    }
    return true;
}
```

ตัวอย่างโค้ดที่ 8.7: ทดสอบจำนวนเฉพาะแบบ Trial Division

ฟังก์ชันนี้มีความซับซ้อน $\mathcal{O}(\sqrt{n})$ ซึ่งเพียงพอสำหรับการตรวจสอบจำนวนเดี่ยวๆ แต่ไม่เหมาะสำหรับการตรวจสอบจำนวนเฉพาะหลายจำนวนพร้อมกัน

8.11.2 ตะแกรงเอราทอสเธเนส (Sieve of Eratosthenes)

ตะแกรงเอราทอสเธเนสเป็นขั้นตอนวิธีที่มีประสิทธิภาพสูงสำหรับหาจำนวนเฉพาะทั้งหมดที่ไม่เกิน N หลักการคือเริ่มจากจำนวนเต็ม 2 ถึง N แล้วตัด (Sieve) พหุคูณของแต่ละจำนวนเฉพาะออกทีละลำดับ

ขั้นตอนวิธี:

1. สร้างแถวลำดับ `is_prime[0..N]` กำหนดค่าเป็น `true` ทั้งหมด (ยกเว้น `is_prime[0]` และ `is_prime[1]` เป็น `false`)
2. สำหรับ $i = 2, 3, \ldots, \sqrt{N}$: ถ้า `is_prime[i]` เป็น `true` ให้ตัดพหุคูณทั้งหมดของ i (คือ $i^2, i^2+i, i^2+2i, \ldots$) โดยกำหนด `is_prime` ให้เป็น `false`
3. จำนวนเฉพาะทั้งหมดคือ i ที่ `is_prime[i]` ยังเป็น `true`

```
const int N = 10000000;  // 10 million
bool isPrime[N+1];
void sieve() {
    fill(isPrime, isPrime + N + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (long long i = 2; i * i <= N; i++) {
        if (isPrime[i]) {
            for (long long j = i * i; j <= N; j += i) {
                isPrime[j] = false;
            }
        }
    }
}
```

ตัวอย่างโค้ดที่ 8.8: ตะแกรงเอราทอสเธเนส

ทำไมเริ่มตัดจาก i^2?

เมื่อเราพิจารณาจำนวนเฉพาะ i พหุคูณที่น้อยกว่า i^2 อย่างเช่น $2i, 3i, \ldots, (i-1)i$ ได้ถูกตัดไปแล้วโดยจำนวนเฉพาะที่น้อยกว่า i ดังนั้นเราเริ่มตัดจาก i^2 ได้โดยตรง ซึ่งช่วยประหยัดเวลาได้พอสมควร

ตะแกรงเอราทอสเธเนสมีความซับซ้อน $\mathcal{O}(n \log \log n)$ ซึ่งใกล้เคียงกับ $\mathcal{O}(n)$ มาก ทำให้สามารถหาจำนวนเฉพาะถึง 10^7 หรือมากกว่าได้อย่างรวดเร็ว

8.11.3 การเพิ่มประสิทธิภาพตะแกรง (Sieve Optimization)

เราสามารถเพิ่มประสิทธิภาพของตะแกรงเอราทอสเธเนสได้หลายวิธี:

1. **ข้ามเลขคู่:** เนื่องจาก 2 เป็นจำนวนเฉพาะคู่เพียงตัวเดียว เราสามารถจัดการเลขคู่แยกต่างหาก แล้วตรวจสอบเฉพาะเลขคี่ ซึ่งช่วยลดเวลาและหน่วยความจำลงครึ่งหนึ่ง

2. **เริ่มตัดจาก i^2:** ดังที่กล่าวแล้ว พหุคูณที่น้อยกว่า i^2 ถูกตัดไปแล้วโดยจำนวนเฉพาะที่น้อยกว่า

3. **หยุดที่ $\sqrt{N}$:** ไม่จำเป็นต้องวนลูปเกิน $\sqrt{N}$ เพราะพหุคูณของจำนวนเฉพาะที่มากกว่า $\sqrt{N}$ จะมีค่าเกิน N อยู่แล้ว

```cpp
const int N = 10000000;
bool isPrime[N+1];
void sieveOptimized() {
    fill(isPrime, isPrime + N + 1, true);
    isPrime[0] = isPrime[1] = false;
    // Handle even numbers
    for (int i = 4; i <= N; i += 2)
        isPrime[i] = false;
    // Check odd numbers only
    for (long long i = 3; i * i <= N; i += 2) {
        if (isPrime[i]) {
            for (long long j = i * i; j <= N; j += 2 * i) {
                isPrime[j] = false;
            }
        }
    }
}
```

ตัวอย่างโค้ดที่ 8.9: ตะแกรงเอราทอสเธเนสแบบปรับปรุง (ข้ามเลขคู่)

เทคนิคข้ามเลขคู่ในลูปตัดพหุคูณ

ในโค้ดข้างต้น ลูปตัดพหุคูณใช้ `j += 2 * i` แทน `j += i` เพราะถ้า i เป็นเลขคี่ แล้ว i^2 เป็นเลขคี่ ดังนั้น $i^2 + i$ เป็นเลขคู่ (ซึ่งถูกตัดไปแล้ว) เราจึงข้ามไป $i^2 + 2i$ ซึ่งเป็นเลขคี่ถัดไป ทำให้ลดการทำงานลงครึ่งหนึ่งในลูปตัดพหุคูณ

ตัวอย่าง: นับจำนวนเฉพาะไม่เกิน N

หลังจากรันตะแกรงแล้ว สามารถนับจำนวนเฉพาะได้ง่ายๆ:

```
int countPrimes(int N) {
    sieve();
    int cnt = 0;
    for (int i = 2; i <= N; i++) {
        if (isPrime[i]) cnt++;
    }
    return cnt;
}
```

หรือถ้าต้องการตอบคำถามหลายครั้ง สามารถสร้างแถวลำดับ prefix sum ไว้ล่วงหน้าได้

หน่วยความจำของตะแกรง

ตะแกรงเอราทอสเธเนสใช้แถวลำดับขนาด $N+1$ ตัว ดังนั้น:

- ถ้าใช้ `bool` (1 ไบต์ต่อช่อง): $N = 10^8$ ใช้หน่วยความจำประมาณ 100 MB
- ถ้าใช้ `vector<bool>` (บีบเก็บ 1 บิตต่อช่อง): ประหยัดหน่วยความจำได้ 8 เท่า
- ควรระวัง Memory Limit ของโจทย์ โดยทั่วไป $N \leq 10^7$ จะปลอดภัยสำหรับข้อจำกัด 256 MB

8.12 การแยกตัวประกอบจำนวนเฉพาะ (Prime Factorization)

การแยกตัวประกอบจำนวนเฉพาะ (Prime Factorization) คือกระบวนการเขียนจำนวนเต็มบวกในรูปผลคูณของจำนวนเฉพาะ ซึ่งเป็นแนวคิดพื้นฐานที่สำคัญที่สุดในทฤษฎีจำนวน และมีประโยชน์อย่างยิ่งในการแก้โจทย์ที่เกี่ยวข้องกับจำนวนตัวหาร ผลรวมตัวหาร และฟังก์ชันทฤษฎีจำนวนอื่นๆ

ทฤษฎีบท 8.8 (ทฤษฎีบทมูลฐานของเลขคณิต (Fundamental Theorem of Arithmetic))**.** ทุกจำนวนเต็ม $n > 1$ สามารถเขียนได้ในรูปผลคูณของจำนวนเฉพาะได้วิธีเดียว ยกเว้นลำดับของตัวประกอบ:

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdot p_3^{e_3} \cdots p_k^{e_k}$$

โดยที่ $p_1 < p_2 < \cdots < p_k$ เป็นจำนวนเฉพาะ และ $e_i \geq 1$ เป็นเลขชี้กำลัง

8.12.1 ขั้นตอนวิธีการแยกตัวประกอบ

วิธีง่ายที่สุดคือการลองหารด้วยจำนวนเต็มตั้งแต่ 2 ขึ้นไป หากหารลงตัว ให้หารซ้ำจนกว่าจะหารไม่ลงตัว แล้วจึงขยับไปตัวถัดไป:

```
#include <vector>
using namespace std;
```

```cpp
// Returns vector of (prime, exponent) pairs
vector<pair<long long, int>> primeFactorize(long long n) {
    vector<pair<long long, int>> factors;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            int exp = 0;
            while (n % i == 0) {
                n /= i;
                exp++;
            }
            factors.push_back({i, exp});
        }
    }
    if (n > 1) {
        factors.push_back({n, 1});
    }
    return factors;
}
```

ตัวอย่างโค้ดที่ 8.10: การแยกตัวประกอบจำนวนเฉพาะ

ขั้นตอนวิธีนี้มีความซับซ้อน $\mathcal{O}(\sqrt{n})$ เช่นเดียวกับการทดสอบจำนวนเฉพาะ

ตัวอย่าง: แยกตัวประกอบ 360

$$360 = 2^3 \times 3^2 \times 5^1$$

ขั้นตอน:

1. $360 \div 2 = 180 \div 2 = 90 \div 2 = 45$ (หาร 2 ได้ 3 ครั้ง)
2. $45 \div 3 = 15 \div 3 = 5$ (หาร 3 ได้ 2 ครั้ง)
3. $5 \div 5 = 1$ (หาร 5 ได้ 1 ครั้ง)

ดังนั้น $360 = 2^3 \cdot 3^2 \cdot 5^1$

8.12.2 จำนวนตัวหาร

จากการแยกตัวประกอบ $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$ จำนวนตัวหารบวกทั้งหมดของ n คือ:

ทฤษฎีบท 8.9 (จำนวนตัวหาร).

$$d(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1) = \prod_{i=1}^{k} (e_i + 1)$$

ตัวอย่าง: จำนวนตัวหารของ 360

$360 = 2^3 \cdot 3^2 \cdot 5^1$

$$d(360) = (3+1)(2+1)(1+1) = 4 \times 3 \times 2 = 24$$

ตัวหารทั้งหมด 24 ตัวคือ: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18, 20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180, 360

8.12.3 ผลรวมตัวหาร

ผลรวมของตัวหารบวกทั้งหมดของ n คือ:

ทฤษฎีบท 8.10 (ผลรวมตัวหาร)**.**

$$\sigma(n) = \prod_{i=1}^{k} \frac{p_i^{e_i+1} - 1}{p_i - 1}$$

ตัวอย่าง: ผลรวมตัวหารของ 360

$$\begin{aligned}\sigma(360) &= \frac{2^4-1}{2-1} \cdot \frac{3^3-1}{3-1} \cdot \frac{5^2-1}{5-1} \\ &= \frac{15}{1} \cdot \frac{26}{2} \cdot \frac{24}{4} \\ &= 15 \cdot 13 \cdot 6 = 1170\end{aligned}$$

สูตรผลรวมตัวหารมาจากไหน?

สูตร $\sigma(n) = \prod \frac{p_i^{e_i+1}-1}{p_i-1}$ มาจากการกระจายผลคูณ:

$$\sigma(p^e) = 1 + p + p^2 + \cdots + p^e = \frac{p^{e+1}-1}{p-1}$$

ซึ่งเป็นอนุกรมเรขาคณิต และเนื่องจาก σ เป็นฟังก์ชันคูณ (Multiplicative) เราจึงสามารถคูณผลของแต่ละตัวประกอบจำนวนเฉพาะเข้าด้วยกันได้

8.12.4 การแยกตัวประกอบของแฟกทอเรียล (Factorization of Factorials)

ในการแข่งขัน เรามักต้องหาเลขชี้กำลังของจำนวนเฉพาะ p ใน $n!$ กล่าวคือ หาค่า e ที่ทำให้ p^e หาร $n!$ ลงตัวมากที่สุด สูตรของเลอฌ็องดร์ (Legendre's Formula) ช่วยคำนวณค่านี้ได้อย่างมีประสิทธิภาพ:

ทฤษฎีบท 8.11 (สูตรของเลอฌ็องดร์ (Legendre's Formula))**.** เลขชี้กำลังของจำนวนเฉพาะ p ใน $n!$ คือ:

$$v_p(n!) = \left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \cdots$$

ซึ่งมีจำนวนพจน์จำกัด เพราะเมื่อ $p^k > n$ แล้ว $\lfloor n/p^k \rfloor = 0$

หลักการของสูตรนี้คือ: ในจำนวน $1, 2, \ldots, n$ มี $\lfloor n/p \rfloor$ ตัวที่หารด้วย p ลงตัว, $\lfloor n/p^2 \rfloor$ ตัวที่หารด้วย p^2 ลงตัว (ซึ่งนับเพิ่มเติมอีก 1 เท่าจากที่นับไปแล้ว), $\lfloor n/p^3 \rfloor$ ตัวที่หารด้วย p^3 ลงตัว (นับเพิ่มอีก 1), ฯลฯ

```cpp
// Returns exponent of p in n! (v_p(n!))
long long legendre(long long n, long long p) {
    long long exp = 0;
    long long pk = p;
    while (pk <= n) {
        exp += n / pk;
        pk *= p;
    }
    return exp;
}
```

ตัวอย่างโค้ดที่ 8.11: หาเลขชี้กำลังของ p ใน $n!$

ตัวอย่าง: เลขชี้กำลังของ 2 ใน 10!

$$v_2(10!) = \left\lfloor \frac{10}{2} \right\rfloor + \left\lfloor \frac{10}{4} \right\rfloor + \left\lfloor \frac{10}{8} \right\rfloor + \left\lfloor \frac{10}{16} \right\rfloor + \cdots = 5 + 2 + 1 + 0 = 8$$

ตรวจสอบ: $10! = 3628800 = 2^8 \times 3^4 \times 5^2 \times 7$ ✓

การแยกตัวประกอบของแฟกทอเรียลมีประโยชน์อย่างยิ่งในการคำนวณคอมบินาทอริกซ์ภายใต้มอดูลาร์ เช่น การหาจำนวนศูนย์ท้ายของ $n!$ (ซึ่งเท่ากับ $v_5(n!)$) หรือการตรวจสอบว่า $\binom{n}{r}$ หารด้วย p ลงตัวหรือไม่

การแยกตัวประกอบจำนวนมากเป็นปัญหายาก

แม้การทดสอบจำนวนเฉพาะจะทำได้ในเวลาพหุนาม (ด้วย AKS Primality Test ซึ่งเป็น deterministic หรือ Miller–Rabin ซึ่งเป็น randomized) แต่การแยกตัวประกอบจำนวนเต็มขนาดใหญ่ยังเป็นปัญหาที่ยังไม่พบขั้นตอนวิธีพหุนาม (polynomial-time) ที่มีประสิทธิภาพ ความยากนี้เป็นรากฐานของระบบเข้ารหัส RSA ในการแข่งขัน หากโจทย์ให้ n มีขนาดใหญ่มาก (เช่น $n > 10^{18}$) และต้องการแยกตัวประกอบ มักจะมีเงื่อนไขพิเศษ เช่น n เป็นผลคูณของจำนวนเฉพาะสองตัว หรือ n มีตัวหารจำนวนเฉพาะขนาดเล็ก

8.13 ฟังก์ชันทฤษฎีจำนวน (Number Theory Functions)

ฟังก์ชันทฤษฎีจำนวนคือฟังก์ชันที่นิยามบนจำนวนเต็มบวก และมีสมบัติพิเศษที่เกี่ยวข้องกับการหารลงตัวและจำนวนเฉพาะ ในหัวข้อนี้เราจะศึกษาฟังก์ชันที่สำคัญที่สุด ได้แก่ ฟังก์ชันออยเลอร์โทเทียนท์ (Euler's Totient), ฟังก์ชันจำนวนตัวหาร (Tau), และฟังก์ชันผลรวมตัวหาร (Sigma) รวมถึงแนวคิดเรื่องฟังก์ชันคูณ (Multiplicative Function) ที่เชื่อมโยงฟังก์ชันเหล่านี้เข้าด้วยกัน

8.13.1 ฟังก์ชันออยเลอร์โทเทียนท์ (Euler's Totient Function)

นิยาม 8.11 (ฟังก์ชันออยเลอร์โทเทียนท์). $\varphi(n)$ คือจำนวนของจำนวนเต็ม k ที่ $1 \le k \le n$ และ $\gcd(k, n) = 1$ กล่าวคือ จำนวนจำนวนเต็มบวกที่ไม่เกิน n และเป็นจำนวนเฉพาะสัมพัทธ์กับ n

ตัวอย่าง: $\varphi(12) = 4$ เพราะจำนวนที่เป็นจำนวนเฉพาะสัมพัทธ์กับ 12 ในช่วง 1–12 คือ 1, 5, 7, 11

ทฤษฎีบท 8.12 (สูตรคำนวณ $\varphi(n)$). ถ้า $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$ แล้ว:

$$\varphi(n) = n \cdot \prod_{p \mid n} \left(1 - \frac{1}{p}\right) = n \cdot \left(1 - \frac{1}{p_1}\right)\left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right)$$

8.13.2 สมบัติสำคัญของ $\varphi(n)$

ฟังก์ชันออยเลอร์โทเทียนท์มีสมบัติสำคัญดังนี้:

ทฤษฎีบท 8.13 (สมบัติของ $\varphi(n)$).

1. $\varphi(p) = p - 1$ สำหรับจำนวนเฉพาะ p
2. $\varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p-1)$ สำหรับจำนวนเฉพาะ p และ $k \ge 1$
3. $\varphi(ab) = \varphi(a) \cdot \varphi(b)$ เมื่อ $\gcd(a, b) = 1$ (ฟังก์ชันคูณ)
4. $\sum_{d \mid n} \varphi(d) = n$ (สมเอกลักษณ์ของออยเลอร์)

ตัวอย่าง: คำนวณ phi(360)

$360 = 2^3 \cdot 3^2 \cdot 5^1$

$$\varphi(360) = 360 \cdot \left(1 - \frac{1}{2}\right)\left(1 - \frac{1}{3}\right)\left(1 - \frac{1}{5}\right) = 360 \cdot \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{4}{5} = 96$$

8.13.3 โค้ดคำนวณ $\varphi(n)$

```cpp
long long eulerTotient(long long n) {
    long long ans = n;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            ans -= ans / i;  // ans *= (1 - 1/i)
        }
    }
    if (n > 1) ans -= ans / n;
    return ans;
}
```

ตัวอย่างโค้ดที่ 8.12: คำนวณ $\varphi(n)$ จากการแยกตัวประกอบ

เทคนิคในโค้ด

ในโค้ดข้างต้น เราใช้ `ans -= ans / i` ซึ่งเทียบเท่ากับ `ans *= (1 - 1/i)` แต่ทำงานในเลขคณิตจำนวนเต็มโดยไม่ต้องใช้ทศนิยม ทำให้หลีกเลี่ยงปัญหาความคลาดเคลื่อนของจุดทศนิยม

8.13.4 ทฤษฎีบทของออยเลอร์ (Euler's Theorem)

ทฤษฎีบทของออยเลอร์เป็นบททั่วไป (Generalization) ของทฤษฎีบทเล็กของแฟร์มา ซึ่งใช้ได้กับมอดูลัสใดๆ ไม่จำเป็นต้องเป็นจำนวนเฉพาะ:

ทฤษฎีบท 8.14 (ทฤษฎีบทของออยเลอร์ (Euler's Theorem))**.** ถ้า $\gcd(a, n) = 1$ แล้ว:

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

สังเกตว่าทฤษฎีบทเล็กของแฟร์มาเป็นกรณีพิเศษของทฤษฎีบทของออยเลอร์ เพราะถ้า $n = p$ เป็นจำนวนเฉพาะ แล้ว $\varphi(p) = p - 1$ ทำให้ได้ $a^{p-1} \equiv 1 \pmod{p}$

บทแทรก 8.1. ถ้า $\gcd(a, n) = 1$ แล้ว:

$$a^b \equiv a^{b \bmod \varphi(n)} \pmod{n}$$

เมื่อ $b \geq \varphi(n)$

บทแทรกนี้มีประโยชน์มากในการแข่งขัน เพราะช่วยลดเลขชี้กำลังที่มีค่ามากๆ ให้อยู่ในช่วงที่จัดการได้ โดยเฉพาะเมื่อมอดูลัสไม่ใช่จำนวนเฉพาะ

ตัวอย่าง: $7^{222} \bmod 12$

$\varphi(12) = 4$ (จาก $12 = 2^2 \cdot 3$, $\varphi(12) = 12 \cdot \frac{1}{2} \cdot \frac{2}{3} = 4$)
$\gcd(7, 12) = 1$ ดังนั้นใช้ทฤษฎีบทของออยเลอร์ได้:
$7^{222} \equiv 7^{222 \bmod 4} = 7^2 = 49 \equiv 1 \pmod{12}$
ตรวจสอบ: $7^4 = 2401 = 200 \times 12 + 1 \equiv 1 \pmod{12}$ ✓

ระวัง: เมื่อ a และ n ไม่เป็นจำนวนเฉพาะสัมพัทธ์กัน

ทฤษฎีบทของออยเลอร์ใช้ได้ก็ต่อเมื่อ $\gcd(a, n) = 1$ เท่านั้น ถ้า $\gcd(a, n) > 1$ แล้ว $a^{\varphi(n)}$ จะไม่เท่ากับ $1 \pmod{n}$ โดยทั่วไป ในกรณีนี้ต้องใช้วิธีอื่น เช่น การยกกำลังแบบไบนารีโดยตรง

8.13.5 จำนวนตัวหาร $\tau(n)$ และผลรวมตัวหาร $\sigma(n)$

เราได้ศึกษาฟังก์ชัน $\tau(n)$ (จำนวนตัวหาร) และ $\sigma(n)$ (ผลรวมตัวหาร) ไปแล้วในหัวข้อ 8.12 ขอสรุปสั้นๆ อีกครั้ง:

- $\tau(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$ — จำนวนตัวหารบวกทั้งหมดของ n

- $\sigma(n) = \prod_{i=1}^{k} \frac{p_i^{e_i+1}-1}{p_i-1}$ — ผลรวมตัวหารบวกทั้งหมดของ n

8.13.6 ฟังก์ชันคูณ (Multiplicative Functions)

นิยาม 8.12 (ฟังก์ชันคูณ). ฟังก์ชัน f เรียกว่า **ฟังก์ชันคูณ** (Multiplicative Function) ถ้า $f(ab) = f(a) \cdot f(b)$ สำหรับทุก a, b ที่ $\gcd(a, b) = 1$

ฟังก์ชันคูณมีสมบัติสำคัญคือ ถ้าเรารู้ค่า $f(p^k)$ สำหรับทุกจำนวนเฉพาะ p และทุก $k \geq 1$ เราสามารถคำนวณ $f(n)$ สำหรับ n ใดๆ ได้จากการแยกตัวประกอบ:

$$f(n) = f(p_1^{e_1}) \cdot f(p_2^{e_2}) \cdots f(p_k^{e_k})$$

ฟังก์ชันที่เป็นฟังก์ชันคูณได้แก่ $\varphi(n)$, $\tau(n)$, $\sigma(n)$, และฟังก์ชันเมอบิอุส $\mu(n)$ (ซึ่งจะกล่าวถึงในบทถัดไป) ความสำคัญของฟังก์ชันคูณคือเราสามารถคำนวณค่าได้อย่างมีประสิทธิภาพด้วยตะแกรงเอราทอสเธเนส

8.14 การยกกำลังแบบมอดูลาร์ (Modular Exponentiation)

ในการแข่งขันจำนวนมาก เราจำเป็นต้องคำนวณค่า $a^b \bmod m$ สำหรับ b ที่มีค่ามากๆ เช่น $b = 10^9$ หรือมากกว่า วิธีการบ้านๆ ที่คูณ a ซ้ำๆ b ครั้งใช้เวลา $\mathcal{O}(b)$ ซึ่งช้าเกินไป การยกกำลังแบบไบนารี (Binary Exponentiation) หรือเรียกอีกอย่างว่าการยกกำลังโดยการยกกำลังสองซ้ำ (Exponentiation by Squaring) ช่วยลดเวลาลงเหลือ $\mathcal{O}(\log b)$ ได้

8.14.1 ปัญหา: คำนวณ $a^b \bmod m$ อย่างมีประสิทธิภาพ

วิธีแบบง่าย (Naive) คือการคูณ a ซ้ำๆ b ครั้ง:

```cpp
long long powerNaive(long long a, long long b, long long m) {
    long long ans = 1;
    for (long long i = 0; i < b; i++) {
        ans = ans * a % m;
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 8.13: วิธี Naive: O(b)

วิธีนี้ช้าเกินไปเมื่อ b มีค่ามาก เช่น $b = 10^{18}$

8.14.2 การยกกำลังแบบไบนารี (Binary Exponentiation)

หลักการของการยกกำลังแบบไบนารีคือการแยกเลขชี้กำลัง b ในรูปฐานสอง แล้วใช้สมบัติ:

$$a^b = \begin{cases} 1 & \text{ถ้า } b = 0 \\ (a^{b/2})^2 & \text{ถ้า } b \text{ เป็นคู่} \\ (a^{(b-1)/2})^2 \cdot a & \text{ถ้า } b \text{ เป็นคี่} \end{cases}$$

```cpp
long long modPow(long long a, long long b, long long m) {
    if (b == 0) return 1 % m;
    long long half = modPow(a, b / 2, m);
    half = (half * half) % m;
    if (b % 2 == 1) half = (half * a) % m;
    return half;
}
```

ตัวอย่างโค้ดที่ 8.14: การยกกำลังแบบไบนารี (Recursive)

```cpp
long long modPow(long long a, long long b, long long m) {
    long long ans = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 8.15: การยกกำลังแบบไบนารี (Iterative)

ทำความเข้าใจเวอร์ชัน Iterative

เวอร์ชัน Iterative ทำงานโดยพิจารณาบิตของ b ทีละบิตจากน้อยไปมาก:

- ถ้าบิตปัจจุบันเป็น 1 ให้คูณ a เข้ากับผลลัพธ์
- ในทุกรอบ ให้ยกกำลังสอง a (เพราะบิตถัดไปแทนกำลังที่เพิ่มขึ้น 2 เท่า)
- เลื่อน b ไปทางขวา 1 บิต

ตัวอย่าง: $a^{13} = a^{1101_2} = a^8 \cdot a^4 \cdot a^1$

8.14.3 ทฤษฎีบทเล็กของแฟร์มา (Fermat's Little Theorem)

ทฤษฎีบท 8.15 (ทฤษฎีบทเล็กของแฟร์มา)**.** ถ้า p เป็นจำนวนเฉพาะ และ $\gcd(a, p) = 1$ แล้ว:

$$a^{p-1} \equiv 1 \pmod{p}$$

บทแทรก 8.2. ถ้า p เป็นจำนวนเฉพาะ และ $\gcd(a, p) = 1$ แล้ว:

$$a^b \equiv a^{b \bmod (p-1)} \pmod{p}$$

สูตรนี้มีประโยชน์เมื่อ b มีค่ามาก (เช่น $b \geq p - 1$)

ตัวอย่าง: $3^{100} \bmod 7$

จากทฤษฎีบทเล็กของแฟร์มา: $3^6 \equiv 1 \pmod{7}$

$100 = 16 \times 6 + 4$

$3^{100} = (3^6)^{16} \cdot 3^4 \equiv 1^{16} \cdot 3^4 \equiv 81 \equiv 4 \pmod{7}$

8.14.4 ตัวผกผันมอดูลาร์ (Modular Inverse)

นิยาม 8.13 (ตัวผกผันมอดูลาร์)**.** ตัวผกผันมอดูลาร์ของ a ภายใต้มอดูลัส m คือจำนวนเต็ม x ที่ทำให้:

$$ax \equiv 1 \pmod{m}$$

เขียนแทนด้วย $x = a^{-1} \bmod m$

ตัวผกผันมอดูลาร์มีอยู่ก็ต่อเมื่อ $\gcd(a, m) = 1$ เท่านั้น

คำนวณด้วยทฤษฎีบทเล็กของแฟร์มา

ถ้า $m = p$ เป็นจำนวนเฉพาะ และ $\gcd(a, p) = 1$ จากทฤษฎีบทเล็กของแฟร์มา:

$$a \cdot a^{p-2} \equiv a^{p-1} \equiv 1 \pmod{p}$$

ดังนั้น $a^{-1} \equiv a^{p-2} \pmod{p}$

```cpp
long long modInverse(long long a, long long p) {
    return modPow(a, p - 2, p);
}
// Valid when p is prime and gcd(a, p) = 1
```

ตัวอย่างโค้ดที่ 8.16: ตัวผกผันมอดูลาร์ด้วยทฤษฎีบทเล็กของแฟร์มา

ตัวอย่าง: $3^{-1} \bmod 7$

$3^{-1} \equiv 3^{7-2} = 3^5 = 243 \equiv 5 \pmod 7$

ตรวจสอบ: $3 \times 5 = 15 \equiv 1 \pmod 7$ ✓

การประยุกต์ใช้: การหารในเลขคณิตมอดูลาร์

ในเลขคณิตมอดูลาร์ เราไม่สามารถหารได้โดยตรง แต่สามารถคูณด้วยตัวผกผันแทนได้:

$$\frac{a}{b} \bmod p = a \cdot b^{-1} \bmod p$$

เทคนิคนี้จำเป็นอย่างยิ่งสำหรับการคำนวณ $\binom{n}{r} \bmod p$ ซึ่งจะกล่าวถึงในหัวข้อ 8.16

8.15 ขั้นตอนวิธีของยูคลิดแบบขยาย (Extended Euclidean Algorithm)

ขั้นตอนวิธีของยูคลิดแบบขยาย (Extended Euclidean Algorithm) เป็นส่วนขยายของขั้นตอนวิธีของยูคลิด ซึ่งนอกจากจะหา $\gcd(a, b)$ แล้ว ยังหาค่า x และ y ที่ทำให้:

$$ax + by = \gcd(a, b)$$

ฟังก์ชันนี้มีประโยชน์อย่างยิ่งในการหาตัวผกผันมอดูลาร์ (Modular Inverse) และการแก้สมการไดโอแฟนไทน์เชิงเส้น (Linear Diophantine Equations)

8.15.1 หลักการ

จากขั้นตอนวิธีของยูคลิด เราทราบว่า $\gcd(a, b) = \gcd(b, a \bmod b)$ สมมติว่าเราทราบค่า x' และ y' ที่ทำให้:

$$bx' + (a \bmod b)y' = \gcd(b, a \bmod b) = \gcd(a, b)$$

เนื่องจาก $a \bmod b = a - \lfloor a/b \rfloor \cdot b$ แทนค่ากลับได้:

$$bx' + (a - \lfloor a/b \rfloor \cdot b)y' = \gcd(a, b)$$
$$ay' + b(x' - \lfloor a/b \rfloor \cdot y') = \gcd(a, b)$$

ดังนั้น $x = y'$ และ $y = x' - \lfloor a/b \rfloor \cdot y'$

8.15.2 โค้ดขั้นตอนวิธีของยูคลิดแบบขยาย

```
1  // Returns gcd(a, b) and finds x, y such that ax + by = gcd(a, b)
```

```cpp
long long extGCD(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

ตัวอย่างโค้ดที่ 8.17: Extended Euclidean Algorithm (Recursive)

```cpp
// Returns gcd(a, b) and finds x, y such that ax + by = gcd(a, b)
long long extGCD_iter(long long a, long long b, long long &x, long long &y)
    {
    long long x0 = 1, y0 = 0, x1 = 0, y1 = 1;
    while (b != 0) {
        long long q = a / b;
        long long temp = b;
        b = a % b;
        a = temp;
        // Update x and y
        long long xt = x0 - q * x1;
        long long yt = y0 - q * y1;
        x0 = x1; y0 = y1;
        x1 = xt; y1 = yt;
    }
    x = x0;
    y = y0;
    return a;
}
```

ตัวอย่างโค้ดที่ 8.18: Extended Euclidean Algorithm (Iterative)

เลือกเวอร์ชัน Recursive หรือ Iterative?

เวอร์ชัน Recursive สั้นและอ่านง่ายกว่า แต่อาจเกิด Stack Overflow ได้ถ้าการเรียกซ้ำลึกเกินไป (ซึ่งไม่น่าเกิดกับ $\mathcal{O}(\log n)$ รอบของ Euclidean Algorithm) เวอร์ชัน Iterative ปลอดภัยกว่าและเร็วกว่าเล็กน้อย ในการแข่งขัน ทั้งสองเวอร์ชันใช้ได้ดี

ตัวอย่าง: Extended GCD ของ 35 และ 15

$\gcd(35, 15) = 5$

จาก `extGCD(35, 15, x, y)`:

- $35(1) + 15(-2) = 35 - 30 = 5$ ✓

ดังนั้น $x = 1, y = -2$

8.15.3 ความสัมพันธ์กับตัวผกผันมอดูลาร์

ถ้า $\gcd(a, m) = 1$ แล้วจากสมการ $ax + my = 1$ จะได้ว่า:

$$ax \equiv 1 \pmod{m}$$

ดังนั้น x mod m คือตัวผกผันมอดูลาร์ของ a ภายใต้มอดูลัส m

ข้อได้เปรียบสำคัญของวิธีนี้เมื่อเทียบกับทฤษฎีบทเล็กของแฟร์มาคือ **ใช้ได้กับมอดูลัส m ใด ๆ** ไม่จำเป็นต้องเป็นจำนวนเฉพาะ ตราบใดที่ $\gcd(a, m) = 1$

```cpp
long long modInverse(long long a, long long m) {
    long long x, y;
    long long g = extGCD(a, m, x, y);
    if (g != 1) return -1;  // No inverse (a and m are not coprime)
    return (x % m + m) % m;  // Ensure ans is positive
}
```

ตัวอย่างโค้ดที่ 8.19: ตัวผกผันมอดูลาร์ด้วย Extended GCD

ทำไมต้อง `(x % m + m) % m`?

ค่า x ที่ได้จาก Extended GCD อาจเป็นลบได้ การใช้ `(x % m + m) % m` เป็นเทคนิคมาตรฐานในการแปลงค่าให้อยู่ในช่วง $[0, m-1]$ ซึ่งถูกต้องเพราะ x mod m และ $(x + km)$ mod m ให้ค่าเดียวกันสำหรับทุกจำนวนเต็ม k

8.15.4 การประยุกต์ใช้: สมการไดโอแฟนไทน์เชิงเส้น

สมการไดโอแฟนไทน์เชิงเส้น (Linear Diophantine Equation) คือสมการรูป $ax + by = c$ ที่ต้องการหาคำตอบเป็นจำนวนเต็ม

ทฤษฎีบท 8.16 (เงื่อนไขการมีคำตอบของสมการไดโอแฟนไทน์)**.** สมการ $ax + by = c$ มีคำตอบเป็นจำนวนเต็มก็ต่อเมื่อ $\gcd(a, b) \mid c$

ถ้า (x_0, y_0) เป็นคำตอบหนึ่งของสมการ แล้วคำตอบทั้งหมดคือ:

$$x = x_0 + \frac{b}{\gcd(a,b)} \cdot t$$
$$y = y_0 - \frac{a}{\gcd(a,b)} \cdot t$$

สำหรับทุกจำนวนเต็ม t

ตัวอย่าง: แก้สมการ Diophantine 35x+15y

$\gcd(35, 15) = 5$ ซึ่ง $5 \mid 20$ ดังนั้นมีคำตอบ
จาก Extended GCD: $35(1) + 15(-2) = 5$
คูณทั้งสองข้างด้วย 4: $35(4) + 15(-8) = 20$
คำตอบเฉพาะ: $x_0 = 4, y_0 = -8$
คำตอบทั่วไป: $x = 4 + 3t,\ y = -8 - 7t$ สำหรับทุกจำนวนเต็ม t

8.16 คอมบินาทอริกซ์ (Combinatorics)

คอมบินาทอริกซ์ (Combinatorics) เป็นสาขาคณิตศาสตร์ที่ศึกษาเรื่องการนับ การจัดเรียง และการเลือก ในการแข่งขัน โจทย์คอมบินาทอริกซ์พบได้บ่อยมาก โดยเฉพาะโจทย์ที่ต้องนับจำนวนวิธี หรือหาความน่าจะเป็น หัวข้อนี้จะครอบคลุมแนวคิดพื้นฐานตั้งแต่การเรียงสับเปลี่ยน (Permutation) การจัดหมู่ (Combination) ไปจนถึงการคำนวณภายใต้มอดูลาร์และจำนวนคาตาลัน (Catalan Numbers)[2]

8.16.1 การเรียงสับเปลี่ยน (Permutation)

นิยาม 8.14 (การเรียงสับเปลี่ยน)**.** จำนวนวิธีเลือกและจัดเรียงวัตถุ r ชิ้นจาก n ชิ้น (โดยสนใจลำดับ) คือ:

$$P(n,r) = \frac{n!}{(n-r)!}$$

ตัวอย่าง: จัดเรียง 3 คนจาก 5 คน

$P(5,3) = \frac{5!}{2!} = \frac{120}{2} = 60$ วิธี

กรณีพิเศษ: $P(n,n) = n!$ คือจำนวนวิธีจัดเรียงวัตถุ n ชิ้นทั้งหมด

8.16.2 การจัดหมู่ (Combination)

นิยาม 8.15 (การจัดหมู่)**.** จำนวนวิธีเลือกวัตถุ r ชิ้นจาก n ชิ้น (โดยไม่สนใจลำดับ) คือ:

$$C(n,r) = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

ตัวอย่าง: เลือก 3 คนจาก 5 คน

$C(5,3) = \frac{5!}{3!\cdot 2!} = \frac{120}{6\cdot 2} = 10$ วิธี

สมบัติสำคัญของ Combination:

- $\binom{n}{r} = \binom{n}{n-r}$ (สมมาตร)
- $\binom{n}{0} = \binom{n}{n} = 1$
- $\binom{n}{1} = n$

8.16.3 เอกลักษณ์ของปัสกาล (Pascal's Identity)

ทฤษฎีบท 8.17 (เอกลักษณ์ของปัสกาล)**.**

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$$

เอกลักษณ์นี้สะท้อนถึงการเลือก r ชิ้นจาก n ชิ้น โดยพิจารณาวัตถุชิ้นหนึ่ง: ถ้าเลือกชิ้นนั้น ต้องเลือกอีก $r-1$ จาก $n-1$ ที่เหลือ ($\binom{n-1}{r-1}$); ถ้าไม่เลือก ต้องเลือก r จาก $n-1$ ที่เหลือ ($\binom{n-1}{r}$)

เอกลักษณ์ของปัสกาลเป็นรากฐานของ **สามเหลี่ยมปัสกาล** (Pascal's Triangle) ซึ่งจัดเรียงค่า $\binom{n}{r}$ ในรูปสามเหลี่ยม:

$$\begin{array}{ccccccccc}
 & & & & 1 & & & & \\
 & & & 1 & & 1 & & & \\
 & & 1 & & 2 & & 1 & & \\
 & 1 & & 3 & & 3 & & 1 & \\
1 & & 4 & & 6 & & 4 & & 1
\end{array}$$

แต่ละค่าในสามเหลี่ยมเป็นผลบวกของสองค่าที่อยู่ด้านบนติดกันพอดี

8.16.4 การคำนวณ $\binom{n}{r} \bmod p$

ในการแข่งขัน เรามักต้องคำนวณ $\binom{n}{r} \bmod p$ สำหรับ p เป็นจำนวนเฉพาะ (โดยทั่วไป $p = 10^9 + 7$) วิธีที่มีประสิทธิภาพคือการคำนวณล่วงหน้า (Precompute) ค่าแฟกทอเรียลและตัวผกผันมอดูลาร์ของแฟกทอเรียล:

$$\binom{n}{r} \bmod p = \frac{n!}{r!(n-r)!} \bmod p = n! \cdot (r!)^{-1} \cdot ((n-r)!)^{-1} \bmod p$$

```
const int MAXN = 1000005;
const long long MOD = 1000000007;
long long fact[MAXN], invFact[MAXN];
void precompute() {
    fact[0] = 1;
```

```
    for (int i = 1; i < MAXN; i++) {
        fact[i] = fact[i-1] * i % MOD;
    }
    invFact[MAXN-1] = modPow(fact[MAXN-1], MOD-2, MOD);
    for (int i = MAXN-2; i >= 0; i--) {
        invFact[i] = invFact[i+1] * (i+1) % MOD;
    }
}
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n-r] % MOD;
}
```

ตัวอย่างโค้ดที่ 8.20: คำนวณ nCr mod p ด้วย Precomputation

วิธีคำนวณ `invFact` ย้อนกลับ

สังเกตว่าเราคำนวณ `invFact[MAXN-1]` ก่อน (ใช้ทฤษฎีบทเล็กของแฟร์มา) แล้วจึงคำนวณ `invFact[i]` จาก `invFact[i+1]` โดยอาศัย:

$$(i!)^{-1} = ((i+1)!)^{-1} \cdot (i+1) \pmod{p}$$

วิธีนี้มีประสิทธิภาพกว่าการคำนวณ `modPow` สำหรับแต่ละ `i` แยกกัน เพราะใช้เวลา $\mathcal{O}(n)$ แทน $\mathcal{O}(n \log p)$

8.16.5 เอกลักษณ์คอมบินาทอริกซ์ที่สำคัญ (Common Combinatorial Identities)

ในการแข่งขัน มีเอกลักษณ์ (Identity) ทางคอมบินาทอริกซ์หลายอย่างที่ปรากฏบ่อยครั้ง การจดจำและเข้าใจเอกลักษณ์เหล่านี้จะช่วยให้แก้โจทย์ได้รวดเร็วยิ่งขึ้น:

ทฤษฎีบท 8.18 (เอกลักษณ์คอมบินาทอริกซ์สำคัญ).

$$\binom{n}{r} = \binom{n}{n-r} \tag{8.10}$$

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r} \tag{8.11}$$

$$\sum_{r=0}^{n} \binom{n}{r} = 2^n \tag{8.12}$$

$$\sum_{r=0}^{n} (-1)^r \binom{n}{r} = 0 \quad (n \geq 1) \tag{8.13}$$

$$\sum_{r=0}^{n} \binom{n}{r}^2 = \binom{2n}{n} \tag{8.14}$$

$$\sum_{r=0}^{k} \binom{m}{r}\binom{n}{k-r} = \binom{m+n}{k} \tag{8.15}$$

ขออธิบายแต่ละเอกลักษณ์:

- **สมการ** (8.10) **(สมมาตร):** การเลือก r จาก n มีวิธีเท่ากับการเลือก $n-r$ จาก n เพราะการเลือกสิ่งที่ต้องการเทียบเท่ากับการเลือกสิ่งที่ปฏิเสธ
- **สมการ** (8.11) **(ปัสกาล):** ได้กล่าวไปแล้วข้างต้น เป็นรากฐานของสามเหลี่ยมปัสกาล
- **สมการ** (8.12) **(ผลบวกแถว):** จำนวนวิธีเลือกสับเซตใดๆ จากเซตขนาด n เท่ากับ 2^n เพราะแต่ละสมาชิกมี 2 ทางเลือก (เลือกหรือไม่เลือก)
- **สมการ** (8.13) **(ผลบวกสลับ):** มาจากการแทนค่า $x = -1$ ในทฤษฎีบททวินาม $(1+x)^n$
- **สมการ** (8.14) **และ** (8.15) **(แวนเดอร์มอนด์):** กรณีพิเศษเมื่อ $m = n = k$ ให้ $\sum \binom{n}{r}^2 = \binom{2n}{n}$ ซึ่งตีความได้ว่า การเลือก n คนจากกลุ่ม n คนซ้ายและ n คนขวา เทียบเท่ากับการเลือก n จาก $2n$

ตัวอย่าง: ใช้เอกลักษณ์คอมบินาทอริกซ์

โจทย์: มีวิธีเลือกสับเซตจากเซต $\{1, 2, \ldots, 10\}$ กี่วิธี?
คำตอบ: จากสมการ (8.12), จำนวนสับเซตทั้งหมดคือ $2^{10} = 1024$ วิธี
อีกโจทย์: $\binom{20}{0}^2 + \binom{20}{1}^2 + \cdots + \binom{20}{20}^2 = ?$
คำตอบ: จากสมการ (8.14), ผลลัพธ์คือ $\binom{40}{20}$

8.16.6 หลักการดาวและแท่ง (Stars and Bars)

หลักการดาวและแท่ง (Stars and Bars) เป็นเทคนิคสำคัญสำหรับนับจำนวนวิธีแจกวัตถุเหมือนกันให้กล่องต่างๆ ปัญหาพื้นฐานคือ: มีวัตถุเหมือนกัน n ชิ้น ต้องแจกใส่กล่อง k ใบ แต่ละใบอาจได้ 0 ชิ้นขึ้นไป มีกี่วิธี?

ทฤษฎีบท 8.19 (หลักการดาวและแท่ง — แจกได้ 0 ชิ้น)**.** จำนวนวิธีแจกวัตถุเหมือนกัน n ชิ้นใส่กล่อง k ใบ (แต่ละกล่องอาจได้ 0 ชิ้นขึ้นไป) คือ:

$$\binom{n+k-1}{k-1} = \binom{n+k-1}{n}$$

หลักการ: จินตนาการว่าเราเขียนดาว n ดวงแทนวัตถุ และใช้แท่ง $k-1$ อันแยกกล่อง แต่ละการจัดเรียงดาวและแท่งจะสอดคล้องกับวิธีแจกหนึ่งวิธี จำนวนตำแหน่งทั้งหมดคือ $n+k-1$ และเราเลือก $k-1$ ตำแหน่งสำหรับแท่ง (หรือเลือก n ตำแหน่งสำหรับดาว)

ทฤษฎีบท 8.20 (หลักการดาวและแท่ง — แจกอย่างน้อย 1 ชิ้น)**.** จำนวนวิธีแจกวัตถุเหมือนกัน n ชิ้นใส่กล่อง k ใบ (แต่ละกล่องต้องได้อย่างน้อย 1 ชิ้น) คือ:

$$\binom{n-1}{k-1}$$

เมื่อ $n \geq k$

หลักการ: แจกวัตถุให้แต่ละกล่อง 1 ชิ้นก่อน แล้วจึงแจกวัตถุที่เหลือ $n-k$ ชิ้นให้กล่อง k ใบ (แจกได้ 0 ชิ้น) = $\binom{(n-k)+k-1}{k-1} = \binom{n-1}{k-1}$

ตัวอย่าง: แจกลูกอม 10 ลูกให้ 3 คน

ถ้าให้แต่ละคนอาจได้ 0 ลูก:

$$\binom{10+3-1}{3-1} = \binom{12}{2} = 66 \text{ วิธี}$$

ถ้าให้แต่ละคนต้องได้อย่างน้อย 1 ลูก:

$$\binom{10-1}{3-1} = \binom{9}{2} = 36 \text{ วิธี}$$

การประยุกต์ใช้ Stars and Bars

หลักการดาวและแท่งปรากฏในโจทย์หลายรูปแบบ:

- หาจำนวนคำตอบของสมการ $x_1 + x_2 + \cdots + x_k = n$ ที่ $x_i \geq 0$ ทุกตัว — คำตอบคือ $\binom{n+k-1}{k-1}$
- หาจำนวนคำตอบเมื่อ $x_i \geq m$ ทุกตัว — แทนค่า $y_i = x_i - m$ แล้วใช้สูตรดาวและแท่งกับ $y_i \geq 0$
- หาจำนวนพจน์ในพหุนามหลายตัวแปร — จำนวนมอนอมใน $x_1, x_2, \ldots, x_k$ ดีกรีรวม n คือ $\binom{n+k-1}{k-1}$

8.16.7 หลักการนับเพิ่มลบ (Inclusion-Exclusion Principle)

หลักการนับเพิ่มลบ (Inclusion-Exclusion Principle) เป็นเทคนิคพื้นฐานในการนับจำนวนสมาชิกของยูเนียนของเซตหลายเซต โดยพิจารณาการซ้อนทับ (Overlap) ระหว่างเซตอย่างถูกต้อง

ทฤษฎีบท 8.21 (หลักการนับเพิ่มลบ (Inclusion-Exclusion Principle))**.** สำหรับเซตจำกัด $A_1, A_2, \ldots, A_n$:

$$|A_1 \cup A_2 \cup \cdots \cup A_n| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n+1} |A_1 \cap A_2 \cap \cdots \cap A_n|$$

สำหรับกรณี 2 เซต สูตรคือ:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

สำหรับกรณี 3 เซต:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

หลักการนับเพิ่มลบมีประโยชน์มากในการนับจำนวนเต็มที่ “ไม่มีสมบัติใดๆ เลย” โดยอาศัยสูตร:

$$|A_1^c \cap A_2^c \cap \cdots \cap A_n^c| = |U| - |A_1 \cup A_2 \cup \cdots \cup A_n|$$

ตัวอย่าง: นับจำนวนที่หารด้วย 2 หรือ 3 หรือ 5 ลงตัวไม่เกิน 100

กำหนด: $A_2 = \{$จำนวนไม่เกิน 100 ที่หารด้วย 2 ลงตัว$\}$, $A_3 = \{$จำนวนไม่เกิน 100 ที่หารด้วย 3 ลงตัว$\}$, $A_5 = \{$จำนวนไม่เกิน 100 ที่หารด้วย 5 ลงตัว$\}$

$$\begin{aligned}
|A_2| &= \lfloor 100/2 \rfloor = 50 \\
|A_3| &= \lfloor 100/3 \rfloor = 33 \\
|A_5| &= \lfloor 100/5 \rfloor = 20 \\
|A_2 \cap A_3| &= \lfloor 100/6 \rfloor = 16 \\
|A_2 \cap A_5| &= \lfloor 100/10 \rfloor = 10 \\
|A_3 \cap A_5| &= \lfloor 100/15 \rfloor = 6 \\
|A_2 \cap A_3 \cap A_5| &= \lfloor 100/30 \rfloor = 3
\end{aligned}$$

ดังนั้น:

$$|A_2 \cup A_3 \cup A_5| = 50 + 33 + 20 - 16 - 10 - 6 + 3 = 74$$

คำตอบ: มี 74 จำนวนที่หารด้วย 2, 3, หรือ 5 ลงตัว และมี $100 - 74 = 26$ จำนวนที่ไม่หารลงตัว

ด้วยทั้ง 2, 3, และ 5

หลักการนับเพิ่มลบกับตัวผกผันมอดูลาร์

ในการแข่งขัน มักต้องคำนวณหลักการนับเพิ่มลบภายใต้มอดูลาร์ เราสามารถคำนวณผลบวกสลับแทนค่าเข้าไปได้โดยตรง โดยจำไว้ว่าการลบภายใต้มอดูลัส m คือการบวกด้วย $m - x$:

$$(a - b) \bmod m = (a + m - b \bmod m) \bmod m$$

8.16.8 จำนวนคาตาลัน (Catalan Numbers)

จำนวนคาตาลันเป็นลำดับจำนวนที่ปรากฏในโจทย์คอมบินาทอริกซ์หลายประเภท และเป็นหนึ่งในลำดับที่สำคัญที่สุดในการโปรแกรมเชิงแข่งขัน

นิยาม 8.16 (จำนวนคาตาลัน)**.** จำนวนคาตาลันตัวที่ n นิยามว่า:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)! \cdot n!}$$

จำนวนคาตาลันตัวแรกๆ ได้แก่:

ตารางที่ 8.6 จำนวนคาตาลันตัวแรก

n	C_n
0	1
1	1
2	2
3	5
4	14
5	42
6	132
7	429
8	1430
9	4862
10	16796

8.16.9 สมบัติเรียกซ้ำของจำนวนคาตาลัน

จำนวนคาตาลันสามารถคำนวณได้จากสมการเรียกซ้ำ:

$$C_0 = 1, \quad C_{n+1} = \sum_{i=0}^{n} C_i \cdot C_{n-i} = \frac{2(2n+1)}{n+2} C_n$$

สมการเรียกซ้ำรูปที่สอง $C_{n+1} = \frac{2(2n+1)}{n+2} C_n$ มีประโยชน์มากสำหรับการคำนวณแบบวนลูป:

```cpp
long long catalan(long long n, long long p) {
    if (n == 0) return 1 % p;
    // C_n = C(2n, n) / (n+1) = C(2n, n) * (n+1)^(-1) mod p
    // Or use recurrence: C_n = 2*(2n-1)/(n+1) * C_{n-1}
    long long c = 1;
    for (long long i = 1; i <= n; i++) {
        c = c * 2 % p * (2*i - 1) % p;
        c = c * modInverse(i + 1, p) % p;
    }
    return c;
}
```

ตัวอย่างโค้ดที่ 8.21: คำนวณจำนวนคาตาลัน mod$p$

8.16.10 การประยุกต์ใช้จำนวนคาตาลัน

จำนวนคาตาลันปรากฏในโจทย์หลายประเภท:

1. **วงเล็บที่ถูกต้อง:** จำนวนวิธีวางวงเล็บเปิด n อันและวงเล็บปิด n อันที่ถูกต้อง (Balanced Parentheses) คือ C_n

   ตัวอย่าง: $n = 3$ มี $C_3 = 5$ วิธี: `((()))`, `(()())`, `(())()`, `()(())`, `()()()`

2. **Dyck Path:** จำนวนเส้นทางจากจุด $(0, 0)$ ถึง $(2n, 0)$ บนกริดที่ใช้ก้าวขึ้น $(1, +1)$ และก้าวลง $(1, -1)$ โดยไม่ตกต่ำกว่าเส้นแกนนอน คือ C_n

3. **Binary Search Tree:** จำนวนวิธีสร้าง BST ที่มี n โหนด คือ C_n

4. **รูปหลายเหลี่ยม:** จำนวนวิธีแบ่งรูปหลายเหลี่ยมนูน $(n + 2)$ ด้าน เป็นสามเหลี่ยมโดยใช้เส้นทแยงมุมที่ไม่ตัดกัน คือ C_n

จำจำนวนคาตาลัน 5 ค่าแรก

จำ C_0 ถึง C_4: **1, 1, 2, 5, 14** — ค่าเหล่านี้ปรากฏในโจทย์บ่อยมาก หากเจอโจทย์ที่คำตอบสำหรับกรณีเล็กๆ ตรงกับ 1, 1, 2, 5, 14 ให้ลองตรวจสอบว่าคำตอบอาจเป็นจำนวนคาตาลัน

ตัวอย่าง: วงเล็บที่ถูกต้อง

โจทย์: มีกี่วิธีที่จะวางวงเล็บเปิดและปิดอย่างละ 4 อันให้ถูกต้อง?
คำตอบ: $C_4 = 14$ วิธี
เราสามารถคำนวณได้:

$$C_4 = \frac{1}{5}\binom{8}{4} = \frac{1}{5} \cdot 70 = 14$$

สรุปบทที่

บทนี้ครอบคลุมพื้นฐานคณิตศาสตร์ที่จำเป็นสำหรับการแข่งขัน:

- **สัญลักษณ์และสัญกรณ์** — อักษรกรีก, สัญลักษณ์เซตและตรรกะ
- **ผลบวกและผลคูณ** — สูตร $\sum k$, $\sum k^2$, $\sum k^3$, แฟกทอเรียล
- **อนุกรม** — เลขคณิต, เรขาคณิต, ฮาร์โมนิก
- **เซตและตรรกะ** — การดำเนินการบนเซต, กฎเดอมอร์แกน, ตรรกะ
- **ฟังก์ชันพื้นฐาน** — Floor, Ceiling, min, max
- **ฟีโบนักชี** — นิยาม, สูตรบิเน, อัตราส่วนทองคำ
- **ลอการิทึม** — สมบัติ, การนับหลัก, การวิเคราะห์ขั้นตอนวิธี
- **GCD & LCM** — ขั้นตอนวิธีของยูคลิด, สูตร LCM
- **จำนวนเฉพาะ** — ทดสอบจำนวนเฉพาะ, ตะแกรงเอราทอสเธเนส, การเพิ่มประสิทธิภาพตะแกรง
- **การแยกตัวประกอบ** — ทฤษฎีบทมูลฐาน, จำนวนตัวหาร, ผลรวมตัวหาร, การแยกตัวประกอบแฟกทอเรียล
- **ฟังก์ชันทฤษฎีจำนวน** — $\varphi(n)$, ทฤษฎีบทของออยเลอร์, $\tau(n)$, $\sigma(n)$, ฟังก์ชันคูณ
- **การยกกำลังมอดูลาร์** — Binary Exponentiation, ทฤษฎีบทเล็กของแฟร์มา, ตัวผกผันมอดูลาร์
- **Extended GCD** — สมการ $ax + by = \gcd(a, b)$, ตัวผกผันมอดูลาร์ทั่วไป, สมการไดโอแฟนไทน์
- **คอมบินาทอริกซ์** — Permutation, Combination, เอกลักษณ์คอมบินาทอริกซ์, หลักการนับเพิ่มลบ, nCr mod p, จำนวนคาตาลัน

8.17 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนคณิตศาสตร์เชิงการแข่งขัน:

โจทย์แนะนำสำหรับบทที่ 8

เลขคณิตมอดูลาร์และการยกกำลัง:

- CSES Bit Strings — การยกกำลังแบบมอดูลาร์
- Codeforces 1114C — Trailing Loves (number theory)
- UVA 10127 — Ones (modular arithmetic)

จำนวนเฉพาะและตะแกรง:

- CSES Counting Divisors — ตะแกรง + การแยกตัวประกอบ
- Codeforces 230B — T-primes (จำนวนเฉพาะ)
- SPOJ PRIME1 — Prime Generator (Segmented Sieve)

คอมบินาทอริกซ์:

- CSES Binomial Coefficients — nCr mod p
- Codeforces 300C — Beautiful Numbers (Combinatorics)
- UVA 10375 — Choose and Divide (Log trick)

GCD และตัวผกผันมอดูลาร์:

- CSES Stick Game (Game Theory + GCD)
- Codeforces 1105C — Ayoub and Lost Array (DP + Modular)
- UVA 10104 — Euclid Problem (Extended GCD)

8.18 สรุปบทที่ 8

ในบทนี้ เราได้ศึกษาพื้นฐานคณิตศาสตร์ที่จำเป็นสำหรับการโปรแกรมเชิงแข่งขัน ตั้งแต่สัญลักษณ์และสัญกรณ์ ผลบวกและผลคูณ ไปจนถึงทฤษฎีจำนวนขั้นสูงและคอมบินาทอริกซ์ แต่ละหัวข้อมีความสำคัญต่อการแก้โจทย์ในการแข่งขัน โดยเฉพาะเลขคณิตมอดูลาร์ จำนวนเฉพาะ และคอมบินาทอริกซ์ ซึ่งปรากฏในโจทย์บ่อยครั้ง

สรุปสิ่งสำคัญของบทที่ 8

1. **เลขคณิตมอดูลาร์** เป็นพื้นฐานสำคัญที่สุด — ระวัง overflow และค่าติดลบ
2. **Binary Exponentiation** ช่วยคำนวณ $a^n \bmod p$ ได้ใน $\mathcal{O}(\log n)$
3. **ตะแกรงเอราทอสเธเนส** หาจำนวนเฉพาะทั้งหมดถึง n ได้ใน $\mathcal{O}(n \log \log n)$
4. **การแยกตัวประกอบ** ช่วยหาจำนวนตัวหารและผลรวมตัวหาร
5. **Extended GCD** แก้สมการ $ax + by = \gcd(a, b)$ และหาตัวผกผันมอดูลาร์
6. **คอมบินาทอริกซ์** $\binom{n}{r} \bmod p$ ใช้ Precompute Factorial + Inverse Factorial
7. **จำนวนคาตาลัน** ปรากฏในโจทย์วงเล็บ, Dyck Path, BST บ่อยมาก

4

ชุดปัญหาด้านการโปรแกรมแบบพลวัต

ส่วนนี้นำเสนอเทคนิค Dynamic Programming (DP) ซึ่งเป็นหนึ่งในเทคนิคที่ทรงพลังและพบบ่อยที่สุดในการแข่งขัน ครอบคลุมหลักการพื้นฐาน Memoization และ Tabulation รวมถึงปัญหาคลาสสิกอย่าง Coin Change, Knapsack, TSP, Edit Distance, LIS และ LCS เนื้อหาในส่วนนี้เป็นหัวข้อที่ใช้เวลาเรียนมากและมีความสำคัญสูงในการประเมินผลการเรียนรู้

Chapter 9

Dynamic Programming

Dynamic Programming (DP) เป็นหนึ่งในเทคนิคที่ทรงพลังและใช้กันแพร่หลายที่สุดในการโปรแกรมเชิงแข่งขัน[2, 3, 5] หลายคนมองว่า DP เป็นหัวข้อที่ยากที่สุดสำหรับผู้เริ่มต้น เพราะต้องอาศัยทั้งความคิดสร้างสรรค์ในการหาสมการเรคัวเรนซ์ และทักษะการเขียนโปรแกรมในการนำไปใช้งาน อย่างไรก็ตาม เมื่อเข้าใจหลักการพื้นฐานแล้ว DP จะกลายเป็นเครื่องมือที่แข็งแกร่งสำหรับแก้โจทย์ปัญหาได้หลากหลาย ในบทนี้ เราจะศึกษาตั้งแต่แนวคิดพื้นฐานของ DP ไปจนถึงปัญหาคลาสสิกที่สำคัญ เช่น Coin Change, 0-1 Knapsack, TSP, และ Edit Distance

9.1 แนะนำ Dynamic Programming

9.1.1 Dynamic Programming คืออะไร

Dynamic Programming คือเทคนิคการแก้ปัญหาโดย **แบ่งปัญหาใหญ่ออกเป็นปัญหาย่อย** (Subproblems) แก้ปัญหาย่อยแต่ละอัน แล้ว **นำคำตอบของปัญหาย่อยมาประกอบเป็นคำตอบของปัญหาใหญ่** จุดสำคัญที่ทำให้ DP แตกต่างจากการแบ่งแยกและเอาชนะ (Divide and Conquer) ทั่วไปคือ DP **จำคำตอบของปัญหาย่อยที่เคยคำนวณแล้ว** เพื่อไม่ให้ต้องคำนวณซ้ำอีก

นิยาม 9.1. Dynamic Programming คือเทคนิคการแก้ปัญหาที่แบ่งปัญหาออกเป็นปัญหาย่อยที่ซ้อนทับกัน (Overlapping Subproblems) และจัดเก็บคำตอบของปัญหาย่อยไว้ในตาราง เพื่อหลีกเลี่ยงการคำนวณซ้ำ

อาจกล่าวได้ว่า **“DP คือ Brute Force แบบฉลาด”** — เรายังตรวจสอบทุกความเป็นไปได้เหมือน Brute Force แต่เก็บคำตอบที่เคยคำนวณแล้วไว้ใน **ตาราง** (Table) ทำให้ไม่ต้องคำนวณซ้ำเมื่อเจอปัญหาย่อยเดิมอีกครั้ง

DP กับตาราง

สิ่งที่ทำให้ DP ทำงานได้คือ **ตาราง** — เราใช้ตาราง (แถวลำดับหรือแผนที่) เก็บคำตอบของปัญหาย่อยแต่ละอัน เมื่อต้องการคำตอบของปัญหาย่อยที่เคยคำนวณแล้ว เราเพียงค้นหาในตารางแทนที่จะคำนวณใหม่ นี่คือเหตุผลที่ DP มักถูกเรียกว่า “การโปรแกรมแบบพลวัต” เพราะตารางถูกสร้างขึ้นทีละส่วนจนครบ

9.1.2 Overlapping Subproblems

สมบัติแรกที่ปัญหาต้องมีจึงจะใช้ DP ได้คือ **ปัญหาย่อยซ้อนทับกัน** (Overlapping Subproblems) หมายความว่าปัญหาย่อยเดียวกันถูกคำนวณซ้ำหลายครั้ง หากปัญหาไม่มีสมบัตินี้ DP จะไม่มีประโยชน์ เพราะไม่มีอะไรให้เก็บไว้ใช้ซ้ำ

ตัวอย่างเช่น ในการคำนวณ Fibonacci ฟังก์ชัน `fibo(3)` ถูกเรียกหลายครั้งในต้นไม้ recursive แสดงว่ามีปัญหาย่อยซ้อนทับกัน การเก็บค่า `fibo(3)` ไว้จึงช่วยประหยัดเวลาได้มาก

9.1.3 Optimal Substructure

สมบัติที่สองที่ปัญหาต้องมีคือ **โครงสร้างย่อยที่เหมาะที่สุด** (Optimal Substructure) หมายความว่า **คำตอบที่เหมาะที่สุดของปัญหาใหญ่ สามารถสร้างได้จากคำตอบที่เหมาะที่สุดของปัญหาย่อย** เช่น เส้นทางสั้นที่สุดจาก A ถึง C ผ่าน B ต้องประกอบด้วยเส้นทางสั้นที่สุดจาก A ถึง B และจาก B ถึง C

ถ้าปัญหามีทั้ง Overlapping Subproblems และ Optimal Substructure เราสามารถใช้ DP แก้ปัญหาได้อย่างมีประสิทธิภาพ

9.2 Overlapping Subproblems

ในหัวข้อนี้เราจะศึกษา Overlapping Subproblems อย่างละเอียดผ่านตัวอย่าง Fibonacci ซึ่งเป็นตัวอย่างคลาสสิกที่แสดงให้เห็นว่าปัญหาย่อยซ้อนทับกันอย่างไร และการจำคำตอบช่วยประหยัดเวลาได้มากแค่ไหน

9.2.1 ตัวอย่าง: Fibonacci

ลำดับ Fibonacci นิยามว่า $F(0) = 0$, $F(1) = 1$ และ $F(n) = F(n-1) + F(n-2)$ สำหรับ $n \geq 2$ หากเขียนฟังก์ชัน Fibonacci แบบ recursive ธรรมดาจะได้ดังนี้:

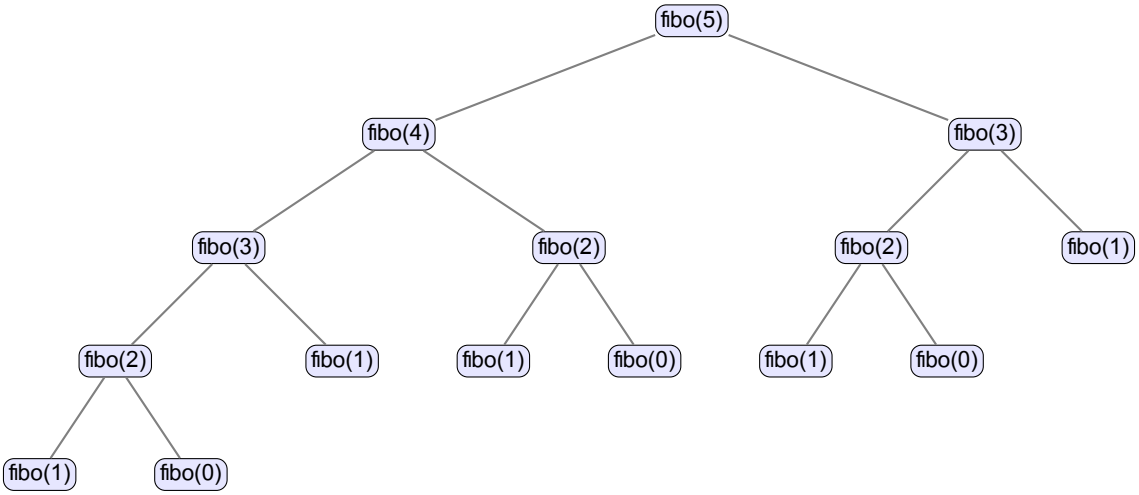
```
int fibo(int n) {
    if (n <= 1) return n;
    return fibo(n - 1) + fibo(n - 2);
}
```

ตัวอย่างโค้ดที่ 9.1: Fibonacci แบบ recursive ธรรมดา

โค้ดนี้ถูกต้องแต่ **ช้ามาก** เพราะมีการคำนวณซ้ำจำนวนมาก ลองพิจารณาการเรียกฟังก์ชันเมื่อคำนวณ `fibo(5)`:

9.2.2 ต้นไม้ recursive ของ Fibonacci

ต้นไม้ recursive ของ `fibo(5)` แสดงได้ดังนี้:



รูปที่ 9.1 ต้นไม้ recursive ของ `fibo(5)` แสดงการคำนวณซ้ำของปัญหาย่อย (Overlapping Subproblems)

สังเกตว่า `fibo(3)` ถูกเรียก **2 ครั้ง** (ที่โหนดซ้ายและโหนดขวาของระดับแรก) และ `fibo(2)` ถูกเรียก **3 ครั้ง** นี่คือ Overlapping Subproblems — ปัญหาย่อยเดียวกันถูกแก้ซ้ำหลายครั้ง

ยิ่ง n ใหญ่ขึ้น การคำนวณซ้ำยิ่งมากขึ้นแบบทวีคูณ ความซับซ้อนของเวลาเป็น $\mathcal{O}(2^n)$ ซึ่งไม่สามารถรับค่า n ที่มากๆ ได้ ตัวอย่างเช่น `fibo(50)` จะใช้เวลานานมากจนรอไม่ไหว

เราสามารถวิเคราะห์จำนวนการเรียกฟังก์ชันได้ดังนี้: สำหรับ `fibo(n)` จำนวนการเรียกฟังก์ชันทั้งหมดคือ $2F(n+1)-1$ เมื่อ $F(n)$ คือเลข Fibonacci ตัวที่ n เอง ซึ่งเติบโตแบบ Exponential ตารางต่อไปนี้แสดงจำนวนการเรียกฟังก์ชันสำหรับ n เล็กๆ:

ตารางที่ 9.1 จำนวนการเรียก fibo(n) แบบ recursive ธรรมดา

n	จำนวนการเรียกฟังก์ชัน
5	15
10	177
20	21,891
30	2,692,573
40	331,160,281
50	**40,730,022,147**

จะเห็นว่าจำนวนการเรียกเพิ่มขึ้นอย่างรวดเร็วมาก — สำหรับ $n = 50$ มีการเรียกฟังก์ชันกว่า 40 พันล้านครั้ง! นี่คือเหตุผลที่เราต้องใช้ DP

ตัวอย่าง: จำนวนการเรียกซ้ำ

จากต้นไม้ของ `fibo(5)`:

- `fibo(3)` ถูกเรียก 2 ครั้ง
- `fibo(2)` ถูกเรียก 3 ครั้ง

- `fibo(1)` ถูกเรียก 5 ครั้ง
- `fibo(0)` ถูกเรียก 3 ครั้ง

รวมการเรียกฟังก์ชันทั้งหมด 15 ครั้ง ทั้งที่คำตอบที่ต้องการมีเพียง 6 ค่า (fibo(0) ถึง fibo(5))

9.2.3 เมื่อไหร่ DP ไม่มีประโยชน์

DP มีประโยชน์ก็ต่อเมื่อมี Overlapping Subproblems หากปัญหาย่อยไม่ซ้อนทับกัน DP จะไม่ช่วยอะไร ตัวอย่างเช่น:

Binary Search — แต่ละขั้นตอนของ Binary Search แบ่งช่วงการค้นหาเป็นครึ่ง และ **ไม่มี** การค้นหาช่วงเดิมซ้ำ ดังนั้น Binary Search ไม่มี Overlapping Subproblems และ DP ไม่มีประโยชน์

Merge Sort — แบ่งแถวลำดับเป็นครึ่งแล้วเรียงแต่ละครึ่ง แต่ละครึ่งถูกเรียงเพียงครั้งเดียว ไม่มีการเรียงซ้ำ จึงไม่มี Overlapping Subproblems

สรุป

DP มีประโยชน์เฉพาะเมื่อปัญหามี **Overlapping Subproblems** ก่อนจะใช้ DP ควรตรวจสอบก่อนว่าปัญหาย่อยซ้อนทับกันจริงหรือไม่

วิธีตรวจสอบ Overlapping Subproblems

วาดต้นไม้ recursive สำหรับกรณีเล็กๆ (เช่น $n = 4$ หรือ 5) แล้วดูว่ามีโหนดไหนซ้ำกันหรือไม่ ถ้ามีโหนดซ้ำ = Overlapping Subproblems = DP มีประโยชน์

9.3 Memoization (Top-Down)

Memoization คือเทคนิคการแก้ปัญหา DP แบบ **บนลงล่าง** (Top-Down)[2] โดยเริ่มจากปัญหาใหญ่ แบ่งเป็นปัญหาย่อย และ **จำคำตอบของปัญหาย่อยไว้ในตาราง** ก่อนที่จะคืนค่า เมื่อเจอปัญหาย่อยเดิมอีกครั้ง จะดึงคำตอบจากตารางมาใช้ทันทีโดยไม่ต้องคำนวณใหม่

9.3.1 แนวคิด

ขั้นตอนของ Memoization มีดังนี้:

1. ก่อนจะคำนวณปัญหาย่อย ให้ **ตรวจสอบในตาราง** ก่อนว่าเคยคำนวณแล้วหรือยัง
2. หากเคยคำนวณแล้ว (ค่าในตารางไม่ใช่ค่าว่าง) ให้ **ใช้คำตอบจากตาราง** โดยตรง
3. หากยังไม่เคยคำนวณ ให้ **คำนวณตามปกติ** แล้ว **เก็บคำตอบลงตาราง** ก่อนคืนค่า

จำไว้

Memoization = **จำ** (Memo) คำตอบที่เคยคำนวณแล้วไว้ เพื่อไม่ต้องคำนวณซ้ำ

9.3.2 โค้ด Fibonacci แบบ Memoization

เราใช้แถวลำดับ `memo[]` เป็นตารางเก็บคำตอบ กำหนดค่าเริ่มต้นเป็น -1 (แทนค่าว่าง) เพื่อบอกว่ายังไม่เคยคำนวณ:

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1005;
long long memo[MAXN]; // Table to store answers
long long fibo(int n) {
    if (n <= 1) return n;
    // Check if already computed
    if (memo[n] != -1) return memo[n];
    // Not yet computed -> compute and store in table
    memo[n] = fibo(n - 1) + fibo(n - 2);
    return memo[n];
}
int main() {
    int n = 50;
    // Initialize all slots to -1 (not yet computed)
    memset(memo, -1, sizeof(memo));
    cout << "fibo(" << n << ") = " << fibo(n) << "\n";
    // fibo(50) = 12586269025
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.2: Fibonacci แบบ Memoization (Top-Down)

ด้วย Memoization แต่ละค่า $F(n)$ จะถูกคำนวณเพียง **ครั้งเดียว** ครั้งต่อไปที่เรียก `fibo(n)` จะดึงค่าจาก `memo[n]` ทันที ความซับซ้อนของเวลาลดลงจาก $\mathcal{O}(2^n)$ เป็น $\mathcal{O}(n)$ — เร็วขึ้นมาก!

ตัวอย่าง: ลำดับการคำนวณ

เมื่อเรียก `fibo(5)` แบบ Memoization:

1. `fibo(5)` ☒ ตารางว่าง ☒ คำนวณ `fibo(4) + fibo(3)`
2. `fibo(4)` ☒ ตารางว่าง ☒ คำนวณ `fibo(3) + fibo(2)`
3. `fibo(3)` ☒ ตารางว่าง ☒ คำนวณ `fibo(2) + fibo(1)`

4. `fibo(2)` ⊠ ตารางว่าง ⊠ คำนวณ `fibo(1) + fibo(0)`
5. `fibo(1)` ⊠ คืนค่า 1 (base case)
6. `fibo(0)` ⊠ คืนค่า 0 (base case)
7. เก็บ `memo[2] = 1`, `memo[3] = 2`, `memo[4] = 3`
8. `fibo(3)` (เรียกจาก fibo(5)) ⊠ **ตารางมีค่าแล้ว** ⊠ คืน 2 ทันที!
9. เก็บ `memo[5] = 5`

9.4 Tabulation (Bottom-Up)

Tabulation คือเทคนิคการแก้ปัญหา DP แบบ **ล่างขึ้นบน** (Bottom-Up)[2] โดยเริ่มจากปัญหาย่อยที่เล็กที่สุด (base case) แล้วค่อยๆ สร้างตารางขึ้นไปจนถึงคำตอบของปัญหาใหญ่

9.4.1 แนวคิด

ขั้นตอนของ Tabulation มีดังนี้:

1. สร้างตาราง (แถวลำดับ) ขนาดเท่ากับจำนวนปัญหาย่อย
2. กำหนดค่าเริ่มต้น (base case) ให้ตาราง
3. วนลูปจากปัญหาเล็กไปหาปัญหาใหญ่ เติมค่าลงตารางทีละช่อง
4. คำตอบของปัญหาใหญ่อยู่ที่ช่องสุดท้ายของตาราง

จำไว้

Tabulation = สร้าง **ตาราง** (Table) จากล่างขึ้นบน เติมค่าทีละช่องจนครบ

9.4.2 โค้ด Fibonacci แบบ Tabulation

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n = 50;
    // Create table
    long long f[n + 1];
    // Set base case
    f[0] = 0;
```

```
    f[1] = 1;
    // Loop to fill table bottom-up
    for (int i = 2; i <= n; i++) {
        f[i] = f[i - 1] + f[i - 2];
    }
    cout << "fibo(" << n << ") = " << f[n] << "\n";
    // fibo(50) = 12586269025
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.3: Fibonacci แบบ Tabulation (Bottom-Up)

โค้ดนี้ง่ายและสั้นกว่า Memoization อย่างเห็นได้ชัด ไม่ต้องใช้ recursion และไม่ต้องกังวลเรื่อง stack overflow ความซับซ้อนของเวลาเป็น $\mathcal{O}(n)$ เช่นเดียวกับ Memoization

เวอร์ชันประหยัดหน่วยความจำ: สังเกตว่าการคำนวณ $f[i]$ ต้องใช้เพียง $f[i-1]$ และ $f[i-2]$ เท่านั้น เราไม่จำเป็นต้องเก็บแถวลำดับทั้งหมด:

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n = 50;
    // Store only 2 previous values
    long long prev2 = 0; // f[i-2]
    long long prev1 = 1; // f[i-1]
    // Handle n = 0 or n = 1 explicitly
    long long current = (n <= 1) ? (long long)n : 0;
    for (int i = 2; i <= n; i++) {
        current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    cout << "fibo(" << n << ") = " << current << "\n";
    // fibo(50) = 12586269025
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.4: Fibonacci แบบ Tabulation — ประหยัดหน่วยความจำ

เวอร์ชันนี้ใช้หน่วยความจำเพียง $\mathcal{O}(1)$ แทนที่จะเป็น $\mathcal{O}(n)$ แต่ยังคงใช้เวลา $\mathcal{O}(n)$ เท่าเดิม เทคนิคนี้เรียกว่า **Space Optimization** และสามารถนำไปใช้กับ DP หลายปัญหา

ตาราง DP ของ Fibonacci แบบ Tabulation

สำหรับ $n = 10$:

i	0	1	2	3	4	5	6	7	8	9	10
$f[i]$	0	1	1	2	3	5	8	13	21	34	55

เราเติมตารางจากซ้ายไปขวา แต่ละช่องใช้เวลาคงที่ $\mathcal{O}(1)$ รวมทั้งหมด $\mathcal{O}(n)$

9.4.3 เปรียบเทียบ Top-Down vs Bottom-Up

ตารางที่ 9.2 เปรียบเทียบ Memoization (Top-Down) กับ Tabulation (Bottom-Up)

คุณสมบัติ	**Top-Down (Memoization)**	**Bottom-Up (Tabulation)**
แนวคิด	เริ่มจากปัญหาใหญ่ ลงไปหาปัญหาย่อย	เริ่มจากปัญหาย่อย ขึ้นไปหาปัญหาใหญ่
การแปลงโค้ด	ง่าย — เพิ่มตารางลงในโค้ด recursive	ต้องคิดลำดับการเติมตาราง
การคำนวณ	คำนวณเฉพาะปัญหาย่อยที่จำเป็น	คำนวณทุกปัญหาย่อย
ความเร็ว	ช้ากว่าเล็กน้อย (มี overhead จาก function call)	เร็วกว่า (ไม่มี function call overhead)
หน่วยความจำ	ใช้ stack เพิ่มจาก recursion	ประหยัดกว่า (ไม่ใช้ stack)
ความเสี่ยง	อาจเกิด Stack Overflow ถ้า n ใหญ่	ไม่มีปัญหา Stack Overflow

คำแนะนำ

- หากเริ่มจากโค้ด recursive อยู่แล้ว ใช้ **Memoization** สะดวกกว่า เพราะเพียงเพิ่มตารางไม่กี่บรรทัด
- หากต้องการประสิทธิภาพสูงสุด ใช้ **Tabulation** เพราะไม่มี overhead จากการเรียกฟังก์ชัน
- ทั้งสองวิธีเร็วกว่า pure recursion **มาก** — จาก $\mathcal{O}(2^n)$ เป็น $\mathcal{O}(n)$

9.5 Optimal Substructure

สมบัติที่สองของ DP คือ Optimal Substructure ซึ่งหมายความว่าคำตอบที่เหมาะที่สุดของปัญหาใหญ่สามารถสร้างได้จากคำตอบที่เหมาะที่สุดของปัญหาย่อย อย่างไรก็ตาม ไม่ใช่ทุกปัญหาที่มีสมบัตินี้

9.5.1 ตัวอย่าง: Shortest Path มี Optimal Substructure

พิจารณาปัญหาหาเส้นทางสั้นที่สุด (Shortest Path) บนกราฟที่มีน้ำหนักไม่ติดลบ สมมติเส้นทางสั้นที่สุดจากจุด u ไป v ผ่านจุดกลาง x จะได้ว่า:

$$\text{shortest}(u, v) = \text{shortest}(u, x) + \text{shortest}(x, v) \tag{9.1}$$

กล่าวคือ เส้นทางสั้นที่สุดจาก u ไป v ผ่าน x จะต้องประกอบด้วยเส้นทางสั้นที่สุดจาก u ไป x และเส้นทางสั้นที่สุดจาก x ไป v หากเส้นทางใดเส้นทางหนึ่งไม่ใช่เส้นทางสั้นที่สุด เราสามารถแทนที่ด้วยเส้นทางที่สั้นกว่าได้ ทำให้เส้นทางรวมสั้นลง — ขัดแย้งกับสมมติฐาน

นี่คือ Optimal Substructure: คำตอบที่เหมาะที่สุดของปัญหาใหญ่สร้างได้จากคำตอบที่เหมาะที่สุดของปัญหาย่อย ขั้นตอนวิธี Dijkstra และ Floyd-Warshall ใช้ประโยชน์จากสมบัตินี้

9.5.2 ตัวอย่าง: Longest Path ไม่มี Optimal Substructure

ตรงกันข้ามกับ Shortest Path ปัญหา **Longest Path** (หาเส้นทางยาวที่สุดแบบไม่ซ้ำจุด) บนกราฟทั่วไป **ไม่มี** Optimal Substructure

พิจารณากราฟ: $a \to b \to c$ และ $a \to c$ (เส้นเชื่อมตรง)

เส้นทางยาวที่สุดจาก a ไป c คือ $a \to b \to c$ (ความยาว 2) แต่:

- เส้นทางยาวที่สุดจาก a ไป b คือ $a \to b$ (ความยาว 1)
- เส้นทางยาวที่สุดจาก b ไป c คือ $b \to c$ (ความยาว 1)

ดูเหมือนจะใช้ได้ แต่ลองพิจารณากราฟที่ซับซ้อนขึ้น: $a \to b \to d$, $a \to c \to d$, $b \to c$, $c \to b$

เส้นทางยาวที่สุดจาก a ไป d อาจเป็น $a \to b \to c \to d$ (ความยาว 3) แต่เส้นทางยาวที่สุดจาก a ไป b คือ $a \to c \to b$ (ความยาว 2) ซึ่งผ่าน c และถ้าเรานำเส้นทางนี้มาต่อกับเส้นทางยาวที่สุดจาก b ไป d จะเกิดการซ้ำจุด (เพราะ c ถูกใช้แล้ว) ดังนั้น Longest Path จาก a ไป d **ไม่ได้มาจาก** Longest Path จาก a ไป b บวก Longest Path จาก b ไป d

ระวัง

ไม่ใช่ทุกปัญหาที่มี Optimal Substructure! ก่อนจะใช้ DP ต้องพิสูจน์ก่อนว่าปัญหามีสมบัตินี้จริง Longest Path บนกราฟทั่วไปเป็นปัญหา NP-Hard ซึ่งไม่มีขั้นตอนวิธีที่มีประสิทธิภาพ (เว้นแต่กราฟเป็น DAG)

9.6 4 ขั้นตอนแก้ปัญหา DP

การแก้ปัญหา DP สามารถแบ่งเป็น 4 ขั้นตอนอย่างเป็นระบบ ดังนี้:

9.6.1 ขั้นที่ 1: ระบุว่าปัญหาเป็น DP

ก่อนอื่นต้องตัดสินใจว่าปัญหานี้ควรใช้ DP หรือไม่ โดยดูจากลักษณะของปัญหา:

- ปัญหา **การหาค่าที่เหมาะที่สุด** (Optimization) เช่น หาค่ามากสุด น้อยสุด มักเป็น DP

- ปัญหา **การนับจำนวนวิธี** (Counting) เช่น นับจำนวนวิธีจัดเรียง มักเป็น DP
- มองหา **Overlapping Subproblems** — ถ้ามีการคำนวณซ้ำ ใช้ DP
- ถ้า Greedy ใช้ไม่ได้ (ได้คำตอบผิด) DP มักเป็นทางเลือกถัดไป

สังเกต

หากโจทยามีข้อจำกัดที่ค่อนข้างเล็ก (เช่น $n \leq 1000$ หรือ $n \leq 20$) และดูเหมือนจะต้องลองทุกความเป็นไปได้ แต่ Brute Force ช้าเกินไป DP มักเป็นคำตอบ

9.6.2 ขั้นที่ 2: กำหนด State

State (สถานะ) คือเซตของพารามิเตอร์ที่ระบุปัญหาย่อยแต่ละอันอย่างไม่ซ้ำซ้อน การกำหนด State ที่ดีเป็นกุญแจสำคัญของการแก้ปัญหา DP

นิยาม 9.2. State คือเซตของพารามิเตอร์ที่ระบุตำแหน่งในตาราง DP แต่ละชุดของค่าพารามิเตอร์จะตรงกับปัญหาย่อยหนึ่งๆ

หลักการสำคัญ: พยายาม **ลดจำนวนพารามิเตอร์** ให้น้อยที่สุด เพราะจำนวนพารามิเตอร์ยิ่งมาก ตาราง DP ยิ่งใหญ่ ใช้เวลาและหน่วยความจำมากขึ้น

ตัวอย่างเช่น:

- Fibonacci: state = n (1 พารามิเตอร์) ⊠ ตารางขนาด n
- 0-1 Knapsack: state = (i, w) (2 พารามิเตอร์) ⊠ ตารางขนาด $n \times W$
- TSP: state = $(pos, mask)$ (2 พารามิเตอร์) ⊠ ตารางขนาด $n \times 2^n$

9.6.3 ขั้นที่ 3: สร้างสมการเรคัวเรนซ์

นี่คือขั้นตอนที่ **ยากที่สุด** ของ DP — ต้องหาความสัมพันธ์ระหว่าง state ปัจจุบันกับ state ก่อนหน้า กล่าวคือ **คำตอบของปัญหาย่อยปัจจุบันหาได้อย่างไรจากคำตอบของปัญหาย่อยที่เล็กกว่า?**

สมการเรคัวเรนซ์ (Recurrence Relation) เป็นสูตรที่บอกว่า:

$$\text{dp[state}_i\text{]} = f(\text{dp[state}_1\text{]}, \text{dp[state}_2\text{]}, \ldots) \tag{9.2}$$

การหาสมการนี้ต้องอาศัยความคิดสร้างสรรค์และ **การฝึกฝน** — ยิ่งทำโจทย์ DP มาก ยิ่งเห็นแพทเทิร์นของสมการเรคัวเรนซ์ได้เร็วขึ้น

9.6.4 ขั้นที่ 4: เลือกวิธีเก็บคำตอบ

นี่คือขั้นตอนที่ **ง่ายที่สุด** — เลือกว่าจะใช้ Memoization หรือ Tabulation ในการเก็บคำตอบของปัญหาย่อยดังที่ได้อธิบายไปแล้วในหัวข้อ 9.3 และ 9.4

9.6.5 ตัวอย่าง: Coin Combination

ให้เหรียญมีมูลค่า $\{1, 3, 5\}$ จงนับจำนวนวิธีที่จะสร้างมูลค่ารวมเท่ากับ N บาท (ลำดับการหย่อนเหรียญไม่สำคัญ)

ขั้นที่ 1: ระบุว่าเป็น DP — เป็นปัญหานับจำนวนวิธี (Counting) มี Overlapping Subproblems

ขั้นที่ 2: กำหนด State — state(n) = จำนวนวิธีสร้างมูลค่า n บาท

ขั้นที 3: สร้างสมการเรคัวเรนซ์ — หากเราหย่อนเหรียญมูลค่า c เป็นเหรียญสุดท้าย จะเหลือมูลค่า $n - c$ ที่ต้องสร้าง:

$$\text{state}(n) = \text{state}(n-1) + \text{state}(n-3) + \text{state}(n-5) \tag{9.3}$$

Base case: state$(0) = 1$ (มี 1 วิธีที่จะสร้างมูลค่า 0 คือไม่หย่อนเหรียญเลย) และ state$(n < 0) = 0$

ขั้นที่ 4: เลือกวิธีเก็บคำตอบ — แสดง 3 เวอร์ชัน:

```
int ways(int n) {
    if (n < 0) return 0;
    if (n == 0) return 1;
    return ways(n - 1) + ways(n - 3) + ways(n - 5);
}
```

ตัวอย่างโค้ดที่ 9.5: Coin Combination — แบบ Recursive ธรรมดา

```
const int MAXN = 10005;
int memo[MAXN];
int ways(int n) {
    if (n < 0) return 0;
    if (n == 0) return 1;
    if (memo[n] != -1) return memo[n];
    memo[n] = ways(n - 1) + ways(n - 3) + ways(n - 5);
    return memo[n];
}
int main() {
    memset(memo, -1, sizeof(memo));
    int N = 8;
    cout << ways(N) << "\n"; // ways(8) = 5
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.6: Coin Combination — แบบ Memoization

```
int main() {
    int N = 8;
    int coins[] = {1, 3, 5};
```

```
    int dp[MAXN];
    memset(dp, 0, sizeof(dp));
    dp[0] = 1; // base case
    for (int i = 1; i <= N; i++) {
        for (int j = 0; j < 3; j++) {
            if (i >= coins[j]) {
                dp[i] += dp[i - coins[j]];
            }
        }
    }
    cout << dp[N] << "\n"; // dp[8] = 5
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.7: Coin Combination — แบบ Tabulation

ตาราง DP ของ Coin Combination

สำหรับ $N = 8$, เหรียญ $\{1, 3, 5\}$:

n	0	1	2	3	4	5	6	7	8
dp$[n]$	1	1	1	2	3	5	8	12	19

ตรวจสอบ: dp$[8]$ = dp$[7]$ + dp$[5]$ + dp$[3]$ = $12 + 5 + 2 = 19$ ☒

รอสักครู่ — ต้องระวังว่าเรานับแบบใด ถ้านับแบบ **combination** (ลำดับไม่สำคัญ) ต้องวนลูปเหรียญนอก และมูลค่าข้างใน ดังนี้:

```
int main() {
    int N = 8;
    int coins[] = {1, 3, 5};
    int dp[MAXN];
    memset(dp, 0, sizeof(dp));
    dp[0] = 1;
    // Loop coins outer, value inner -> combination
    for (int j = 0; j < 3; j++) {
        for (int i = coins[j]; i <= N; i++) {
            dp[i] += dp[i - coins[j]];
        }
    }
    cout << dp[N] << "\n"; // dp[8] = 5
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.8: Coin Combination — แบบ Combination (ลำดับไม่สำคัญ)

ระวัง: Permutation vs Combination

ลำดับการวนลูปมีผลต่อคำตอบ!

- **วนมูลค่านอก เหรียญข้างใน** ⊠ นับเป็น Permutation (ลำดับสำคัญ) — 1+3 กับ 3+1 นับต่างกัน
- **วนเหรียญนอก มูลค่าข้างใน** ⊠ นับเป็น Combination (ลำดับไม่สำคัญ) — 1+3 กับ 3+1 นับเหมือนกัน

ต้องอ่านโจทย์ให้ดีว่าต้องการนับแบบใด

9.7 Max 1D Range Sum (Kadane's Algorithm)

Max 1D Range Sum คือปัญหาการหาผลรวมสูงสุดของช่วงต่อเนื่อง (Contiguous Subarray) ในลำดับตัวเลข เป็นหนึ่งในปัญหา DP ที่พบบ่อยที่สุด และมีขั้นตอนวิธีที่สวยงามมาก[3]

9.7.1 นิยามปัญหา

กำหนดลำดับตัวเลข $a[0], a[1], \ldots, a[n-1]$ ซึ่งอาจเป็นบวกหรือลบก็ได้ จงหาผลรวมสูงสุดของช่วงต่อเนื่องบางช่วง

ตัวอย่าง

- ลำดับ $\{5, -2, 3\}$ ⊠ ผลรวมสูงสุด = $5 + (-2) + 3 = 6$
- ลำดับ $\{5, -6, 3, 6\}$ ⊠ ผลรวมสูงสุด = $3 + 6 = 9$ (เลือกช่วง $\{3, 6\}$)
- ลำดับ $\{-3, -1, -5\}$ ⊠ ผลรวมสูงสุด = -1 (เลือกช่วงสั้นที่สุดคือตัวเดียว)

9.7.2 Recurrence

นิยามให้ $S(j)$ คือผลรวมสูงสุดของช่วงต่อเนื่องที่ **จบที่ตำแหน่ง** j (ไม่ใช่ผลรวมสูงสุดทั้งหมดนะ!) สมการเรคัวเรนซ์คือ:

$$S(j) = \max(S(j-1) + a[j],\ a[j]) \tag{9.4}$$

ความหมาย: ที่ตำแหน่ง j เรามี 2 ทางเลือก:

1. ต่อช่วงเดิม: เพิ่ม $a[j]$ เข้าไปในช่วงที่จบที่ $j-1$ ⊠ ได้ $S(j-1) + a[j]$
2. เริ่มช่วงใหม่: ให้ $a[j]$ เป็นจุดเริ่มต้นของช่วงใหม่ ⊠ ได้ $a[j]$

เลือกทางที่ให้ค่ามากกว่า คำตอบของปัญหาคือ $\max_{0 \le j < n} S(j)$

ระวัง

$S(j)$ คือผลรวมสูงสุดของช่วงที่ **จบที่ตำแหน่ง** j ไม่ใช่ผลรวมสูงสุดทั้งหมด คำตอบสุดท้ายต้องหา max จาก $S(0)$ ถึง $S(n-1)$

9.7.3 โค้ด 3 เวอร์ชัน

เวอร์ชันที่ 1: Recursive + ตัวแปร global

```cpp
#include <bits/stdc++.h>
using namespace std;
int a[] = {5, -6, 3, 6};
int n = 4;
int g_max; // Store global max
int maxSum(int j) {
    if (j == 0) {
        g_max = a[0];
        return a[0];
    }
    int s = max(maxSum(j - 1) + a[j], a[j]);
    g_max = max(g_max, s);
    return s;
}
int main() {
    maxSum(n - 1);
    cout << g_max << "\n"; // 9
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.9: Max 1D Range Sum — แบบ Recursive

เวอร์ชันที่ 2: Bottom-Up ด้วยแถวลำดับ

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a[] = {5, -6, 3, 6};
    int n = 4;
    int B[n]; // B[j] = max sum ending at position j
    B[0] = a[0];
    for (int j = 1; j < n; j++) {
        B[j] = max(B[j - 1] + a[j], a[j]);
```

```
    }
    // Find max from B[0] to B[n-1]
    int ans = B[0];
    for (int j = 1; j < n; j++) {
        ans = max(ans, B[j]);
    }
    cout << ans << "\n"; // 9
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.10: Max 1D Range Sum — แบบ Bottom-Up

เวอร์ชันที่ 3: Space-Optimized (Kadane's Algorithm)

สังเกตว่าในเวอร์ชันที่ 2 เราใช้เพียง $B[j-1]$ ในการคำนวณ $B[j]$ ดังนั้นไม่จำเป็นต้องเก็บแถวลำดับทั้งหมด เก็บเพียงค่าก่อนหน้าและค่า max ทั้งหมดก็พอ:

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a[] = {5, -6, 3, 6};
    int n = 4;
    int maxEndingHere = a[0]; // Current S(j)
    int maxSoFar = a[0];      // Global max
    for (int j = 1; j < n; j++) {
        maxEndingHere = max(maxEndingHere + a[j], a[j]);
        maxSoFar = max(maxSoFar, maxEndingHere);
    }
    cout << maxSoFar << "\n"; // 9
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.11: Kadane's Algorithm — ประหยัดหน่วยความจำ

Kadane's Algorithm

เวอร์ชันสุดท้ายนี้คือ **Kadane's Algorithm** — ใช้เวลา $\mathcal{O}(n)$ และหน่วยความจำเพียง $\mathcal{O}(1)$ เป็นวิธีที่มีประสิทธิภาพสูงสุดสำหรับปัญหา Max 1D Range Sum

9.8 Coin Change

Coin Change เป็นอีกหนึ่งปัญหาคลาสสิกของ DP ที่พบได้บ่อยในการแข่งขัน ปัญหานี้แตกต่างจาก Coin Combination ตรงที่เราต้องการหา **จำนวนเหรียญน้อยที่สุด** แทนที่จะนับจำนวนวิธี

9.8.1 นิยามปัญหา

กำหนดเหรียญหลายชนิดที่มีมูลค่าต่างกัน (มีเหรียญแต่ละชนิดให้ใช้ไม่จำกัด) จงหาจำนวนเหรียญ **น้อยที่สุด** ที่รวมเป็นมูลค่า V พอดี หากทำไม่ได้ ให้ตอบ -1

ตัวอย่าง

- $V = 10$, เหรียญ $\{1, 5\}$ ⊠ ตอบ 2 (ใช้เหรียญ 5 สองเหรียญ)
- $V = 7$, เหรียญ $\{1, 3, 4, 5\}$ ⊠ ตอบ 2 (ใช้เหรียญ 3+4)
- $V = 3$, เหรียญ $\{2, 4\}$ ⊠ ตอบ -1 (ทำไม่ได้)

9.8.2 Greedy ไม่สำเร็จ

สำหรับบางชุดเหรียญ วิธี Greedy (เลือกเหรียญมูลค่ามากที่สุดก่อนเสมอ) ใช้ได้ เช่น เหรียญ $\{1, 5, 10, 25\}$ (เหรียญของสหรัฐ) แต่สำหรับชุดเหรียญทั่วไป Greedy **อาจได้คำตอบผิด**:

ตัวอย่างที่ Greedy ล้มเหลว

$V = 7$, เหรียญ $\{1, 3, 4, 5\}$
วิธี Greedy: เลือก $5 + 1 + 1 = 7$ ⊠ ใช้ 3 เหรียญ
วิธี DP: เลือก $3 + 4 = 7$ ⊠ ใช้ **2 เหรียญ** (น้อยกว่า!)
Greedy เลือกเหรียญ 5 ก่อนเพราะมีมูลค่ามากสุด แต่ทำให้เหลือ 2 ซึ่งต้องใช้เหรียญ 1 สองเหรียญ ในขณะที่การเลือก 3+4 ใช้เหรียญน้อยกว่า

ดังนั้น **ต้องใช้ DP** เพื่อหาคำตอบที่เหมาะที่สุดเสมอ

เมื่อไหร่ Greedy ใช้ได้?

Greedy ใช้ได้กับเหรียญที่มีสมบัติพิเศษ เช่น เหรียญสกุลเงินมาตรฐาน {1, 5, 10, 25, 100} ของสหรัฐ หรือเหรียญที่แต่ละเหรียญมีมูลค่าเป็นเท่าของเหรียญที่เล็กกว่า แต่สำหรับเหรียญทั่วไป ต้องใช้ DP เสมอ ในการแข่งขัน ถ้าไม่แน่ใจว่า Greedy ใช้ได้ **ให้ใช้ DP** เพราะ DP ให้คำตอบที่ถูกต้องเสมอ

9.8.3 Recurrence

นิยาม change(v) = จำนวนเหรียญน้อยสุดที่รวมเป็นมูลค่า v

$$\text{change}(0) = 0 \tag{9.5}$$

$$\text{change}(v < 0) = \infty \tag{9.6}$$

$$\text{change}(v) = 1 + \min_{\text{coin} \in \text{coins}} \text{change}(v - \text{coin}) \tag{9.7}$$

ความหมาย: สำหรับมูลค่า v เราลองเลือกเหรียญทุกชนิดเป็นเหรียญแรก แล้วหาค่าที่เหลือ $v - \text{coin}$ แบบ recursive เลือกเหรียญที่ทำให้จำนวนเหรียญรวมน้อยที่สุด

9.8.4 โค้ด Recursive + Bottom-Up

เวอร์ชัน Recursive (Top-Down):

```cpp
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9;
int coins[] = {1, 3, 4, 5};
int m = 4;
int memo[10005];
int change(int v) {
    if (v < 0) return INF;
    if (v == 0) return 0;
    if (memo[v] != -1) return memo[v];
    int best = INF;
    for (int i = 0; i < m; i++) {
        best = min(best, 1 + change(v - coins[i]));
    }
    memo[v] = best;
    return best;
}
int main() {
    memset(memo, -1, sizeof(memo));
    int V = 7;
    int ans = change(V);
    if (ans >= INF) cout << -1 << "\n";
    else cout << ans << "\n"; // 2
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.12: Coin Change — แบบ Recursive + Memoization

เวอร์ชัน Bottom-Up (Tabulation):

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int coins[] = {1, 3, 4, 5};
    int m = 4;
    int V = 7;
    const int INF = 1e9;
```

```
    int dp[V + 1];
    dp[0] = 0;
    for (int v = 1; v <= V; v++) {
        dp[v] = INF;
        for (int i = 0; i < m; i++) {
            if (v >= coins[i]) {
                dp[v] = min(dp[v], 1 + dp[v - coins[i]]);
            }
        }
    }
    if (dp[V] >= INF) cout << -1 << "\n";
    else cout << dp[V] << "\n"; // 2
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.13: Coin Change — แบบ Bottom-Up

ตาราง DP ของ Coin Change

สำหรับ $V = 7$, เหรียญ $\{1, 3, 4, 5\}$:

v	0	1	2	3	4	5	6	7
dp$[v]$	0	1	2	1	1	1	2	2

ตรวจสอบ: dp$[7] = \min(1 + \text{dp}[6], 1 + \text{dp}[4], 1 + \text{dp}[3], 1 + \text{dp}[2]) = \min(3, 2, 2, 3) = 2$ ✓

วิธีได้มา: dp$[7] = 1 + \text{dp}[4] = 1 + 1 = 2$ (ใช้เหรียญ 3 แล้วเหลือ 4) หรือ dp$[7] = 1 + \text{dp}[3] = 1 + 1 = 2$ (ใช้เหรียญ 4 แล้วเหลือ 3) ทั้งสองวิธีใช้ 2 เหรียญ

การติดตามเส้นทาง (Tracing)

หากต้องการรู้ว่าใช้เหรียญอะไรบ้าง (ไม่ใช่แค่จำนวน) เราสามารถเก็บ `choice[v]` ไว้ด้วย — บอกว่าเหรียญชนิดไหนที่ถูกเลือกที่มูลค่า v:

```
int choice[V + 1]; // Store selected coins
for (int v = 1; v <= V; v++) {
    dp[v] = INF;
    for (int i = 0; i < m; i++) {
        if (v >= coins[i] && 1 + dp[v - coins[i]] < dp[v]) {
            dp[v] = 1 + dp[v - coins[i]];
            choice[v] = coins[i]; // Record selected coin
        }
    }
}
// Trace the path
cout << "Coins: ";
```

```cpp
int v = V;
while (v > 0) {
    cout << choice[v] << " ";
    v -= choice[v];
}
// For V=7, coins={1,3,4,5}: prints "4 3" or "3 4"
```

ตัวอย่างโค้ดที่ 9.14: Coin Change พร้อมการติดตามเส้นทาง

9.9 0-1 Knapsack

0-1 Knapsack เป็นหนึ่งในปัญหา DP ที่สำคัญที่สุดและเป็นพื้นฐานของปัญหา DP อีกมากมาย[2, 7] ชื่อ “0-1” มาจากการที่แต่ละไอเทมสามารถเลือก (1) หรือไม่เลือก (0) เท่านั้น ไม่สามารถเลือกซ้ำได้

9.9.1 นิยามปัญหา

กำหนดไอเทม n ชิ้น ไอเทมชิ้นที่ i มีมูลค่า val$[i]$ และน้ำหนัก wt$[i]$ และกระเป๋าที่มีความจุ S จงเลือกไอเทมบางชิ้นใส่ในกระเป๋า (แต่ละชิ้นเลือกได้มากสุด 1 ครั้ง) เพื่อให้ **มูลค่ารวมสูงสุด** โดยที่ **น้ำหนักรวมไม่เกิน** S

ตัวอย่าง

ไอเทม 3 ชิ้น, ความจุ $S = 5$:

ไอเทม	มูลค่า	น้ำหนัก	ความคุ้มค่า
0	60	5	12
1	50	3	16.67
2	70	4	17.5

คำตอบ: เลือกไอเทม 1 และ 2 ⊠ น้ำหนักรวม = $3 + 4 = 7 > 5$ (เกิน!)
ลองใหม่: เลือกไอเทม 0 ⊠ น้ำหนัก 5, มูลค่า 60
หรือเลือกไอเทม 1 ⊠ น้ำหนัก 3, มูลค่า 50
หรือเลือกไอเทม 2 ⊠ น้ำหนัก 4, มูลค่า 70
หรือเลือกไอเทม 1 + 2? ⊠ น้ำหนัก 7 > 5 เกิน!
คำตอบที่ดีที่สุด: เลือกไอเทม 2 ⊠ มูลค่า 70

9.9.2 เลือก/ไม่เลือก

สำหรับไอเทมแต่ละชิ้น เรามี 2 ทางเลือก:

1. **เลือก** ไอเทมชิ้นนี้: น้ำหนักที่เหลือลดลงเป็น $W -$ wt$[i]$ และพิจารณาไอเทมชิ้นที่ $0 \ldots i-1$ ต่อไป

2. **ไม่เลือก** ไอเทมชิ้นนี้: น้ำหนักที่เหลือยังเป็น W เหมือนเดิม และพิจารณาไอเทมชิ้นที่ $0 \dots i-1$ ต่อไป

เราจะเลือกทางที่ให้มูลค่ารวมสูงกว่า

9.9.3 Recurrence

นิยาม $K[i][w]$ = มูลค่าสูงสุดที่ได้จากไอเทมชิ้นที่ $0 \dots i-1$ เมื่อความจุที่เหลือคือ w

$$K[i][w] = \max(\text{val}[i-1] + K[i-1][w - \text{wt}[i-1]],\ K[i-1][w]) \tag{9.8}$$

$$K[0][w] = 0 \quad \text{(ไม่มีไอเทมให้เลือก)} \tag{9.9}$$

$$K[i][0] = 0 \quad \text{(ความจุเหลือ 0)} \tag{9.10}$$

ระวัง

ในสมการข้างต้น เราใช้ $\text{val}[i-1]$ และ $\text{wt}[i-1]$ (ไม่ใช่ i) เพราะ $K[i]$ หมายถึงพิจารณาไอเทม i ชิ้นแรก (ดัชนี 0 ถึง $i-1$) ดังนั้นไอเทมลำดับที่ i มีดัชนีแถวลำดับเป็น $i-1$

9.9.4 โค้ด Top-Down + Bottom-Up

เวอร์ชัน Top-Down (Memoization):

```cpp
#include <bits/stdc++.h>
using namespace std;
int val[] = {60, 50, 70};
int wt[]  = {5, 3, 4};
int n = 3, S = 5;
int memo[105][10005]; // memo[i][w]
int knapsack(int i, int w) {
    if (i == 0 || w == 0) return 0;
    if (memo[i][w] != -1) return memo[i][w];
    // Don't select item i
    int notTake = knapsack(i - 1, w);
    // Select item i (if weight is enough)
    int take = 0;
    if (wt[i - 1] <= w) {
        take = val[i - 1] + knapsack(i - 1, w - wt[i - 1]);
    }
    memo[i][w] = max(take, notTake);
    return memo[i][w];
}
```

```cpp
int main() {
    memset(memo, -1, sizeof(memo));
    cout << knapsack(n, S) << "\n"; // 70
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.15: 0-1 Knapsack — แบบ Memoization

เวอร์ชัน Bottom-Up (Tabulation):

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int val[] = {60, 50, 70};
    int wt[]  = {5, 3, 4};
    int n = 3, S = 5;
    int K[n + 1][S + 1];
    // Set base case
    for (int w = 0; w <= S; w++) K[0][w] = 0;
    for (int i = 0; i <= n; i++) K[i][0] = 0;
    // Fill the table
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= S; w++) {
            // Don't select item i
            K[i][w] = K[i - 1][w];
            // Select item i (if weight is enough)
            if (wt[i - 1] <= w) {
                K[i][w] = max(K[i][w],
                    val[i - 1] + K[i - 1][w - wt[i - 1]]);
            }
        }
    }
    cout << K[n][S] << "\n"; // 70
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.16: 0-1 Knapsack — แบบ Bottom-Up

ตาราง DP ของ 0-1 Knapsack

สำหรับ val = {60, 50, 70}, wt = {5, 3, 4}, $S = 5$:

$K[i][w]$	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (val=60, wt=5)	0	0	0	0	0	60
2 (val=50, wt=3)	0	0	0	50	50	60
3 (val=70, wt=4)	0	0	0	50	70	70

คำตอบ: $K[3][5] = 70$ (เลือกไอเทมชิ้นที่ 3 คือ val=70, wt=4)

ความซับซ้อน

0-1 Knapsack ใช้เวลา $\mathcal{O}(n \times S)$ และหน่วยความจำ $\mathcal{O}(n \times S)$ สังเกตว่าความซับซ้อนนี้ **ไม่ใช่** polynomial เทียบกับขนาดข้อมูลนำเข้า เพราะ S อาจมีค่ามากถึง 10^6 หรือมากกว่า ดังนั้น 0-1 Knapsack เป็นปัญหา **NP-Complete** ในทางทฤษฎี แต่ DP ช่วยแก้ได้เมื่อ $n \times S$ ไม่ใหญ่เกินไป (เช่น $n \leq 100$, $S \leq 10000$)

เวอร์ชันประหยัดหน่วยความจำ: สังเกตว่า `K[i][w]` ต้องใช้เพียง `K[i-1][...]` เท่านั้น เราสามารถลดหน่วยความจำจาก $\mathcal{O}(n \times S)$ เป็น $\mathcal{O}(S)$ ได้:

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int val[] = {60, 50, 70};
    int wt[]  = {5, 3, 4};
    int n = 3, S = 5;
    int dp[S + 1] = {}; // 1D array, default value 0
    for (int i = 0; i < n; i++) {
        // Loop from S down to wt[i] (very important!)
        for (int w = S; w >= wt[i]; w--) {
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }
    cout << dp[S] << "\n"; // 70
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.17: 0-1 Knapsack — ประหยัดหน่วยความจำ (1D Array)

ทำไมต้องวนลูปจาก S ลงมา?

สำคัญมาก! ต้องวนลูปจาก S ลงมาหา `wt[i]` ไม่ใช่จาก `wt[i]` ขึ้นไป **เพราะ** ถ้าวนจากน้อยไปมาก `dp[w - wt[i]]` อาจถูกอัปเดตไปแล้วในรอบปัจจุบัน ทำให้ไอเทมชิ้นเดียวกันถูกเลือกมากกว่า 1 ครั้ง (กลายเป็น Unbounded Knapsack แทน!) การวนจากมากไปน้อยรับประกันว่า `dp[w -`

`wt[i]` ยังเป็นค่าจากรอบก่อนหน้า (`i-1`) อยู่

9.10 Traveling Salesman Problem (TSP)

Traveling Salesman Problem (TSP) เป็นปัญหาคลาสสิกที่มีชื่อเสียงมากในวิทยาการคอมพิวเตอร์[2, 7] เป็นปัญหาที่ยากในทางทฤษฎี (NP-Hard) แต่ DP กับ Bitmask ช่วยแก้ได้สำหรับ n ไม่ใหญ่เกินไป (ประมาณ $n \leq 16$)

9.10.1 นิยามปัญหา

กำหนดเมือง n เมือง และเมทริกซ์ระยะทาง dist$[i][j]$ ซึ่งบอกระยะทางจากเมือง i ไปเมือง j นักเดินทางต้องเยือนทุกเมืองพอดีทีละเมือง แล้วกลับมาที่เมืองเริ่มต้น จงหาเส้นทางที่มีระยะทางรวม **น้อยที่สุด** (Hamiltonian Tour)

ตัวอย่าง

4 เมือง ด้วยเมทริกซ์ระยะทาง:

dist	0	1	2	3
0	0	20	42	35
1	20	0	30	34
2	42	30	0	12
3	35	34	12	0

คำตอบ: เส้นทาง $0 \to 1 \to 2 \to 3 \to 0$ ระยะทางรวม = $20 + 30 + 12 + 35 = 97$

9.10.2 Bitmask DP

สำหรับ TSP เราต้องติดตาม 2 สิ่ง:

1. **ตำแหน่งปัจจุบัน** (pos) — เมืองที่นักเดินทางอยู่ตอนนี้
2. **เซตของเมืองที่เยือนแล้ว** (mask) — เมืองไหนบ้างที่ผ่านมาแล้ว

เราใช้ **Bitmask** (จำนวนเต็ม n บิต) แทนเซตของเมืองที่เยือนแล้ว:

- บิตที่ i เป็น 1 = เมือง i ถูกเยือนแล้ว
- บิตที่ i เป็น 0 = เมือง i ยังไม่ถูกเยือน

การดำเนินการกับ Bitmask:

- ตรวจสอบว่าเมือง i ถูกเยือนหรือยัง: `mask & (1 << i)`
- ทำเครื่องหมายว่าเมือง i ถูกเยือนแล้ว: `mask | (1 << i)`
- เมืองทุกเมืองถูกเยือนหมดแล้ว: `mask == (1 << n) - 1`

Bitmask คืออะไร

Bitmask คือการใช้จำนวนเต็มแทนเซต โดยแต่ละบิตแทนสมาชิกหนึ่งตัวของเซต ตัวอย่างเช่น ถ้ามี 4 เมือง:

- `mask = 0` (binary: `0000`) = ยังไม่เยือนเมืองไหนเลย
- `mask = 5` (binary: `0101`) = เยือนเมือง 0 และ 2 แล้ว
- `mask = 15` (binary: `1111`) = เยือนทุกเมืองแล้ว

9.10.3 Recurrence

นิยาม tsp$(pos, mask)$ = ระยะทางน้อยสุดที่เหลือ เมื่ออยู่ที่เมือง pos และเยือนเมืองตาม $mask$ แล้ว

$$\text{tsp}(pos, 2^n - 1) = \text{dist}[pos][0] \quad \text{(เยือนครบแล้ว กลับเมือง 0)} \tag{9.11}$$

$$\text{tsp}(pos, mask) = \min_{\text{nxt} \notin \text{mask}} \left(\text{dist}[pos][\text{nxt}] + \text{tsp}(\text{nxt},\ mask \mid (1 \ll \text{nxt}))\right) \tag{9.12}$$

ความซับซ้อน: $\mathcal{O}(n^2 \times 2^n)$ (มี $n \times 2^n$ state แต่ละ state ต้องลอง n เมือง) สามารถแก้ได้สำหรับ $n \approx 16$

9.10.4 โค้ด TSP + Bitmask

```cpp
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9;
int n;
int dist[20][20];
int memo[20][1 << 20]; // memo[pos][bitmask]
int tsp(int pos, int mask) {
    // Visited all cities -> return to city 0
    if (mask == (1 << n) - 1) {
        return dist[pos][0];
    }
    // Check memo
    if (memo[pos][mask] != -1) {
        return memo[pos][mask];
    }
    int ans = INF;
    for (int nxt = 0; nxt < n; nxt++) {
        // If city nxt hasn't been visited
```

```
        if (!(mask & (1 << nxt))) {
            int cost = dist[pos][nxt]
                    + tsp(nxt, mask | (1 << nxt));
            ans = min(ans, cost);
        }
    }
    memo[pos][mask] = ans;
    return ans;
}
int main() {
    n = 4;
    // Set distance matrix
    int d[4][4] = {
        {0, 20, 42, 35},
        {20, 0, 30, 34},
        {42, 30, 0, 12},
        {35, 34, 12, 0}
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            dist[i][j] = d[i][j];
    memset(memo, -1, sizeof(memo));
    // Start from city 0, city 0 already visited (bit 0 = 1)
    cout << tsp(0, 1) << "\n"; // 97
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.18: TSP — แบบ Memoization + Bitmask

เริ่มต้นที่เมืองไหนก็ได้

เนื่องจาก TSP เป็นวงรอบ (เริ่มที่ไหน กลับมาที่นั่น) เราสามารถเริ่มที่เมือง 0 เสมอ โดยเรียก `tsp(0, 1)` (bit 0 เป็น 1 เพราะเยือนเมือง 0 แล้ว)

ตัวอย่าง: ลำดับการคำนวณ TSP

สำหรับ 4 เมือง เริ่มจาก `tsp(0, 1)` (binary `0001`):

1. `tsp(0, 1)`: ยังไม่ครบ ☒ ลองเมือง 1, 2, 3
    - ไปเมือง 1: $\text{dist}[0][1] + \text{tsp}(1, 3)$ (binary `0011`)
    - ไปเมือง 2: $\text{dist}[0][2] + \text{tsp}(2, 5)$ (binary `0101`)
    - ไปเมือง 3: $\text{dist}[0][3] + \text{tsp}(3, 9)$ (binary `1001`)

2. `tsp(1, 3)`: เยือนเมือง 0, 1 แล้ว ☒ ลองเมือง 2, 3
   - ไปเมือง 2: dist$[1][2]$ + tsp$(2, 7)$ (binary `0111`)
   - ไปเมือง 3: dist$[1][3]$ + tsp$(3, 11)$ (binary `1011`)
3. `tsp(2, 7)`: เยือนเมือง 0, 1, 2 แล้ว ☒ ลองเมือง 3
   - ไปเมือง 3: dist$[2][3]$ + tsp$(3, 15)$ (binary `1111`)
4. `tsp(3, 15)`: เยือนครบแล้ว! ☒ คืน dist$[3][0] = 35$

เส้นทาง: 0☒1☒2☒3☒0 ระยะทาง = $20 + 30 + 12 + 35 = 97$

9.10.5 การสร้างตารางระยะทาง

ในหลายโจทย์ เราไม่ได้รับเมทริกซ์ระยะทางโดยตรง แต่ได้พิกัดของแต่ละเมืองแทน เราต้องคำนวณระยะทางเอง:

ระยะทางแบบ Manhattan:

$$\text{dist}[i][j] = |x_i - x_j| + |y_i - y_j| \tag{9.13}$$

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        dist[i][j] = abs(x[i] - x[j]) + abs(y[i] - y[j]);
    }
}
```

ตัวอย่างโค้ดที่ 9.19: สร้างตารางระยะทางแบบ Manhattan

ระยะทางแบบ Euclidean:

$$\text{dist}[i][j] = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{9.14}$$

```
#include <cmath>
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        double dx = x[i] - x[j];
        double dy = y[i] - y[j];
        dist[i][j] = round(sqrt(dx * dx + dy * dy));
    }
}
```

ตัวอย่างโค้ดที่ 9.20: สร้างตารางระยะทางแบบ Euclidean

ระวัง

ถ้าโจทย์ให้พิกัดและใช้ระยะทาง Euclidean ต้องระวังเรื่องจุดทศนิยม ใน TSP แบบ DP ข้างต้น เราใช้ `int` เก็บระยะทาง หากระยะทางเป็นจุดทศนิยม ต้องปัดเศษหรือใช้ `double` แทน

9.11 Partition Problem

Partition Problem เป็นปัญหาที่ถามว่า เซตของจำนวนเต็มที่กำหนดให้ สามารถแบ่งออกเป็น 2 สับเซตที่มีผลรวมเท่ากันได้หรือไม่ ปัญหานี้มีความคล้ายคลึงกับ 0-1 Knapsack อย่างมาก

9.11.1 นิยาม

กำหนดเซตของจำนวนเต็ม $S = \{a_1, a_2, \ldots, a_n\}$ จงตรวจสอบว่าสามารถแบ่ง S ออกเป็น 2 สับเซต S_1 และ S_2 โดยที่ $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset$ และ $\sum_{x \in S_1} x = \sum_{x \in S_2} x$ ได้หรือไม่

สังเกตสำคัญ

ถ้าผลรวมของเซตทั้งหมดเป็น **จำนวนคี่** (odd) ก็ **แบ่งไม่ได้** ทันที เพราะไม่มีทางที่ผลรวมของ 2 สับเซตจะเท่ากันได้ (จำนวนคี่ไม่สามารถหาร 2 ลงตัว) ดังนั้นเราต้องตรวจสอบนี้ก่อนเป็นอันดับแรก

ดังนั้นปัญหาจึงลดรูปเป็น: หาสับเซตที่มีผลรวมเท่ากับ $\frac{\text{sum}}{2}$ (เรียกว่า **target**) ได้หรือไม่

ตัวอย่าง

$S = \{1, 5, 11, 5\}$, ผลรวม = 22 (คู่) ☒ target = 11
สับเซตที่ผลรวมเท่ากับ 11: $\{11\}$ หรือ $\{1, 5, 5\}$
ดังนั้นสามารถแบ่งได้: $S_1 = \{11\}$, $S_2 = \{1, 5, 5\}$ ☒ ผลรวมเท่ากัน = 11 ✓

9.11.2 Recurrence

ปัญหานี้เหมือนกับการถามว่า “มีสับเซตของแถวลำดับ n ตัวแรกที่ผลรวมเท่ากับ sum หรือไม่”

$$\text{isSubsetSum}(n, \text{sum}) = \text{isSubsetSum}(n-1, \text{sum}) \vee \text{isSubsetSum}(n-1, \text{sum} - \text{arr}[n-1]) \tag{9.15}$$

ความหมาย: สำหรับไอเทมตัวที่ n (ดัชนี $n-1$):

- ไม่เลือก: ตรวจสอบว่าหาผลรวม sum จากไอเทม $n-1$ ตัวแรกได้หรือไม่
- เลือก: ตรวจสอบว่าหาผลรวม $\text{sum} - \text{arr}[n-1]$ จากไอเทม $n-1$ ตัวแรกได้หรือไม่

Base case:

$$\text{isSubsetSum}(n, 0) = \text{true} \quad \text{(ผลรวม 0 ได้เสมอ โดยไม่เลือกอะไรเลย)} \tag{9.16}$$

$$\text{isSubsetSum}(0, \text{sum} \neq 0) = \text{false} \quad \text{(ไม่มีไอเทม แต่ต้องการผลรวมไม่ใช่ 0)} \tag{9.17}$$

9.11.3 โค้ด Recursive + Bottom-Up

เวอร์ชัน Recursive:

```cpp
#include <bits/stdc++.h>
using namespace std;
bool isSubsetSum(int arr[], int n, int sum) {
    if (sum == 0) return true;
    if (n == 0) return false;
    // If last element is greater than sum, don't select
    if (arr[n - 1] > sum) {
        return isSubsetSum(arr, n - 1, sum);
    }
    // Select or don't select
    return isSubsetSum(arr, n - 1, sum)
        || isSubsetSum(arr, n - 1, sum - arr[n - 1]);
}
int main() {
    int arr[] = {1, 5, 11, 5};
    int n = 4;
    int totalSum = 0;
    for (int i = 0; i < n; i++) totalSum += arr[i];
    if (totalSum % 2 != 0) {
        cout << "false\n";
    } else {
        int target = totalSum / 2;
        cout << (isSubsetSum(arr, n, target) ? "true" : "false")
            << "\n"; // true
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.21: Partition Problem — แบบ Recursive

เวอร์ชัน Bottom-Up (Tabulation):

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
```

```cpp
    int arr[] = {1, 5, 11, 5};
    int n = 4;
    int totalSum = 0;
    for (int i = 0; i < n; i++) totalSum += arr[i];
    if (totalSum % 2 != 0) {
        cout << "false\n";
        return 0;
    }
    int target = totalSum / 2;
    // part[i][j] = true if sum j can be formed from first i elements
    bool part[n + 1][target + 1];
    // Base case: sum 0 is always possible
    for (int i = 0; i <= n; i++) part[i][0] = true;
    // Base case: no items, sum != 0 is impossible
    for (int j = 1; j <= target; j++) part[0][j] = false;
    // Fill the table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= target; j++) {
            // Don't select item i
            part[i][j] = part[i - 1][j];
            // Select item i (if j is enough)
            if (j >= arr[i - 1]) {
                part[i][j] = part[i][j] || part[i - 1][j - arr[i - 1]];
            }
        }
    }
    cout << (part[n][target] ? "true" : "false") << "\n"; // true
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.22: Partition Problem — แบบ Bottom-Up

ตาราง DP ของ Partition Problem

สำหรับ arr = {1, 5, 11, 5}, target = 11:

$i \backslash j$	0	1	2	3	4	5	6	7	8	9	10	11
0	T	F	F	F	F	F	F	F	F	F	F	F
1	T	T	F	F	F	F	F	F	F	F	F	F
2	T	T	F	F	F	T	T	F	F	F	F	F
3	T	T	F	F	F	T	T	F	F	F	F	T
4	T	T	F	F	F	T	T	F	F	F	T	T

part$[4][11]$ = true ✓ (สามารถแบ่งได้)

9.12 Edit Distance

Edit Distance (หรือ Levenshtein Distance) เป็นปัญหาที่วัดระยะห่างระหว่างสองข้อความ[2, 3] โดยนับจำนวนการดำเนินการขั้นต่ำที่ต้องทำเพื่อแปลงข้อความหนึ่งไปเป็นอีกข้อความหนึ่ง มีประโยชน์มากในโจทย์เกี่ยวกับการประมวลผลข้อความ การตรวจสอบการสะกด และชีวสารสนเทศ

9.12.1 นิยาม

กำหนดข้อความ `str1` ยาว m ตัวอักษร และ `str2` ยาว n ตัวอักษร จงหาจำนวนการดำเนินการ **น้อยที่สุด** ที่ต้องทำเพื่อแปลง `str1` ให้เป็น `str2` โดยมีการดำเนินการ 3 ชนิด:

1. **Insert** (แทรก) — แทรกตัวอักษรหนึ่งตัวเข้าไป
2. **Remove** (ลบ) — ลบตัวอักษรหนึ่งตัวออก
3. **Replace** (แทนที่) — แทนที่ตัวอักษรหนึ่งตัวด้วยตัวอักษรอื่น

ตัวอย่าง

- `str1 = "sunday"`, `str2 = "saturday"` ⊠ Edit Distance = 3
  1. แทรก `'a'` และ `'t'` หลัง `'s'` ⊠ “**sa**tunday”
  2. แทนที่ `'n'` ด้วย `'r'` ⊠ “satur**r**day”
  3. ผลลัพธ์: “saturday” ⊠ ใช้ 3 การดำเนินการ
- `str1 = "cat"`, `str2 = "cut"` ⊠ Edit Distance = 1 (แทนที่ `'a'` ด้วย `'u'`)
- `str1 = "abc"`, `str2 = "abc"` ⊠ Edit Distance = 0 (เหมือนกันอยู่แล้ว)

9.12.2 3 Operations

พิจารณาอักขระตัวสุดท้ายของ `str1` และ `str2`:

กรณีที่ 1: อักขระตัวสุดท้ายเหมือนกัน — ไม่ต้องทำอะไร ลดปัญหาเป็น editDist$(m-1, n-1)$

กรณีที่ 2: อักขระตัวสุดท้ายต่างกัน — ต้องเลือกหนึ่งใน 3 การดำเนินการ:

- **Insert:** แทรกอักขระที่เหมือน `str2[n-1]` เข้าไปใน `str1` ⊠ ปัญหาลดเป็น editDist$(m, n-1)$ (ตัวที่แทรกตรงกับ `str2[n-1]` แล้ว เหลือ `str2` อีก $n-1$ ตัว)
- **Remove:** ลบอักขระตัวสุดท้ายของ `str1` ออก ⊠ ปัญหาลดเป็น editDist$(m-1, n)$ (ลบไปแล้ว เหลือ `str1` อีก $m-1$ ตัว)
- **Replace:** แทนที่อักขระตัวสุดท้ายของ `str1` ด้วย `str2[n-1]` ⊠ ปัญหาลดเป็น editDist$(m-1, n-1)$ (ทั้งสองข้างตรงกันแล้ว)

9.12.3 Recurrence

$$\text{editDist}(m,n) = \begin{cases} \text{editDist}(m-1,n-1) & \text{if str1}[m-1] = \text{str2}[n-1] \\ 1 + \min \begin{cases} \text{editDist}(m,n-1) & \text{(Insert)} \\ \text{editDist}(m-1,n) & \text{(Remove)} \\ \text{editDist}(m-1,n-1) & \text{(Replace)} \end{cases} & \text{otherwise} \end{cases} \tag{9.18}$$

Base case:

- $\text{editDist}(0,n) = n$ — ต้องแทรก n ตัวอักษร
- $\text{editDist}(m,0) = m$ — ต้องลบ m ตัวอักษร

9.12.4 โค้ด Bottom-Up

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    string str1 = "sunday";
    string str2 = "saturday";
    int m = str1.size();
    int n = str2.size();
    // dp[i][j] = edit distance of str1[0..i-1] and str2[0..j-1]
    int dp[m + 1][n + 1];
    // Base case
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    // Fill the table
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (str1[i - 1] == str2[j - 1]) {
                // Same character -> no operation needed
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                // Different characters -> choose best operation
                dp[i][j] = 1 + min({
                    dp[i][j - 1],      // Insert
                    dp[i - 1][j],      // Remove
                    dp[i - 1][j - 1]   // Replace
                });
```

```
            }
        }
    }
    cout << dp[m][n] << "\n"; // 3
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.23: Edit Distance — แบบ Bottom-Up

ตาราง DP ของ Edit Distance

สำหรับ `str1 = "sunday"`, `str2 = "saturday"`:

	∅	s	a	t	u	r	d	a	y
∅	0	1	2	3	4	5	6	7	8
s	1	0	1	2	3	4	5	6	7
u	2	1	1	2	2	3	4	5	6
n	3	2	2	2	3	3	4	5	6
d	4	3	3	3	3	4	3	4	5
a	5	4	3	4	4	4	4	3	4
y	6	5	4	4	5	5	5	4	3

$\text{dp}[6][8] = 3$ ✓

Overlapping Subproblems ใน Edit Distance

ถ้าเขียน Edit Distance แบบ recursive ธรรมดา ความซับซ้อนจะเป็น $\mathcal{O}(3^{m+n})$ เพราะทุกจุดที่อักขระต่างกันมี 3 ทางเลือก แต่ด้วย DP ความซับซ้อนลดลงเหลือ $\mathcal{O}(m \times n)$ เพราะมีเพียง $(m+1)\times(n+1)$ state ที่แตกต่างกัน

เวอร์ชันประหยัดหน่วยความจำ: เช่นเดียวกับปัญหา DP อื่นๆ เราสามารถลดหน่วยความจำได้ สังเกตว่า `dp[i][j]` ต้องใช้เพียงแถวก่อนหน้า (`dp[i-1][...]`) และแถวปัจจุบัน (`dp[i][...]`) เท่านั้น:

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string str1 = "sunday";
    string str2 = "saturday";
    int m = str1.size();
    int n = str2.size();
    // Use only 2 rows
    int prev[n + 1], curr[n + 1];
    // Base case: row 0
    for (int j = 0; j <= n; j++) prev[j] = j;
    for (int i = 1; i <= m; i++) {
```

```
        curr[0] = i; // Base case: column 0
        for (int j = 1; j <= n; j++) {
            if (str1[i - 1] == str2[j - 1]) {
                curr[j] = prev[j - 1];
            } else {
                curr[j] = 1 + min({curr[j - 1],
                                   prev[j],
                                   prev[j - 1]});
            }
        }
        // Swap rows
        for (int j = 0; j <= n; j++) prev[j] = curr[j];
    }
    cout << prev[n] << "\n"; // 3
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.24: Edit Distance — ประหยัดหน่วยความจำ (2 แถว)

เวอร์ชันนี้ใช้หน่วยความจำ $\mathcal{O}(\min(m, n))$ แทนที่จะเป็น $\mathcal{O}(m \times n)$

9.13 Longest Common Subsequence (LCS)

Longest Common Subsequence (LCS) เป็นอีกหนึ่งปัญหาคลาสสิกของ DP ที่พบบ่อยในการแข่งขัน[2, 3] ปัญหานี้ว่าด้วยการหาลำดับย่อย (Subsequence) ที่ยาวที่สุดที่ปรากฏในสายอักขระทั้งสองสายอักขระพร้อมกัน

9.13.1 นิยามปัญหา

กำหนดสายอักขระ A ความยาว n และสายอักขระ B ความยาว m จงหาลำดับย่อยที่ยาวที่สุดที่ปรากฏในทั้งสองสายอักขระ โดย **ลำดับย่อย** (Subsequence) ไม่จำเป็นต้องอยู่ติดกัน แต่ต้อง **ปรากฏตามลำดับเดิม**

ตัวอย่าง: LCS

- สายอักขระ A = `"ABCBDAB"`, สายอักขระ B = `"BDCAB"`
- LCS = `"BCAB"` (ความยาว 4)
- สังเกตว่า `'B'`, `'C'`, `'A'`, `'B'` ปรากฏตามลำดับในทั้งสองสายอักขระ แม้ไม่ได้อยู่ติดกัน
- อีกคำตอบที่เป็นไปได้: `"BDAB"` (ความยาว 4 เช่นกัน) — LCS อาจมีได้หลายคำตอบ แต่ความยาวเท่ากันเสมอ

9.13.2 สมการเรคัวเรนซ์

นิยามให้ `dp[i][j]` คือความยาวของ LCS ของ $A[0 \ldots i-1]$ และ $B[0 \ldots j-1]$ (คือ i ตัวอักษรแรกของ A และ j ตัวอักษรแรกของ B) สมการเรคัวเรนซ์คือ:

$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] + 1 & \text{ถ้า } A[i-1] = B[j-1] \\ \max(\text{dp}[i-1][j],\ \text{dp}[i][j-1]) & \text{ถ้า } A[i-1] \neq B[j-1] \end{cases} \tag{9.19}$$

ความหมาย:

- ถ้าอักษรตัวสุดท้ายของทั้งสองสายอักขระ **เหมือนกัน** ($A[i-1] = B[j-1]$) เรานำอักษรนั้นมารวมใน LCS ดังนั้นความยาวเพิ่ม 1 จาก LCS ของสายอักขระที่สั้นลงทั้งคู่
- ถ้าอักษรตัวสุดท้าย **ต่างกัน** เราต้องเลือกว่าจะละทิ้งอักษรตัวสุดท้ายของ A หรือ B — เลือกทางที่ให้ LCS ยาวกว่า

Base case: `dp[0][j] = 0` และ `dp[i][0] = 0` เพราะสายอักขระว่างกับสายอักขระใดๆ มี LCS ความยาว 0

คำตอบ: `dp[n][m]`

9.13.3 การวิเคราะห์ความซับซ้อน

- **เวลา:** $\mathcal{O}(nm)$ — ต้องเติมตารางทุกช่อง แต่ละช่องใช้เวลา $\mathcal{O}(1)$
- **หน่วยความจำ:** $\mathcal{O}(nm)$ สำหรับตาราง 2 มิติ หรือ $\mathcal{O}(\min(n, m))$ ด้วยการประหยัดหน่วยความจำ (เก็บเฉพาะแถวก่อนหน้า)

9.13.4 โค้ด C++ สำหรับ LCS

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    string A = "ABCBDAB";
    string B = "BDCAB";
    int n = A.size();
    int m = B.size();
    // dp[i][j] = LCS length of A[0..i-1] and B[0..j-1]
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    // Fill DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (A[i - 1] == B[j - 1]) {
```

```
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    cout << "LCS length = " << dp[n][m] << "\n"; // 4
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.25: Longest Common Subsequence — คำนวณความยาว

9.13.5 การสร้างสายอักขระ LCS ย้อนกลับ

หลังจากคำนวณตาราง dp เสร็จแล้ว เราสามารถย้อนกลับจาก dp[n][m] เพื่อสร้างสายอักขระ LCS ได้:

```
string reconstructLCS(const string& A, const string& B,
                      const vector<vector<int>>& dp) {
    int i = A.size();
    int j = B.size();
    string lcs = "";
    // Trace back from dp[n][m]
    while (i > 0 && j > 0) {
        if (A[i - 1] == B[j - 1]) {
            // Characters match -> part of LCS
            lcs = A[i - 1] + lcs;
            i--;
            j--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            // Move to the direction with larger value
            i--;
        } else {
            j--;
        }
    }
    return lcs; // "BDAB"
}
```

ตัวอย่างโค้ดที่ 9.26: LCS — สร้างสายอักขระย้อนกลับ (Backtracking)

หลักการย้อนกลับ:

1. เริ่มจาก dp[n][m]

2. ถ้า $A[i-1] = B[j-1]$ แสดงว่าอักษรนี้เป็นส่วนหนึ่งของ LCS ☒ เพิ่มเข้าผลลัพธ์ แล้วลดทั้ง i และ j
3. ถ้าต่างกัน ☒ เลื่อนไปทางที่ค่า dp มากกว่า (ลด i หรือ j)
4. ทำซ้ำจนกว่าจะถึงขอบตาราง

9.13.6 การประหยัดหน่วยความจำ

เนื่องจาก dp[i][j] ใช้เพียงแถว dp[i-1][*] ในการคำนวณ เราสามารถเก็บเพียง 2 แถวได้:

```cpp
int lcsLength(const string& A, const string& B) {
    int n = A.size(), m = B.size();
    // Use only 2 rows: prev and curr
    vector<int> prev(m + 1, 0), curr(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (A[i - 1] == B[j - 1]) {
                curr[j] = prev[j - 1] + 1;
            } else {
                curr[j] = max(prev[j], curr[j - 1]);
            }
        }
        swap(prev, curr); // Move to next row
    }
    return prev[m]; // Answer is in prev after last swap
}
```

ตัวอย่างโค้ดที่ 9.27: LCS — ประหยัดหน่วยความจำ (เก็บ 2 แถว)

วิธีนี้ใช้หน่วยความจำ $\mathcal{O}(\min(n, m))$ แทนที่ $\mathcal{O}(nm)$ แต่ **ไม่สามารถย้อนกลับสร้างสายอักขระ LCS ได้** เพราะไม่ได้เก็บตารางทั้งหมด

ตัวอย่าง: ตาราง DP ของ LCS

สายอักขระ A = "ABC", B = "BDC"

$i \backslash j$	0	1(B)	2(D)	3(C)
0	0	0	0	0
1(A)	0	0	0	0
2(B)	0	1	1	1
3(C)	0	1	1	2

- dp[2][1]: $A[1]$='B' $=$ $B[0]$='B' ☒ dp[1][0]+1 = 1
- dp[2][2]: $A[1]$='B' $\neq$ $B[1]$='D' ☒ max(dp[1][2], dp[2][1]) = max$(0, 1)$ = 1

- `dp[3][3]`: $A[2]$='C' $= B[2]$='C' ⊠ `dp[2][2]+1` = 2
- คำตอบ: `dp[3][3]` = 2 ซึ่งก็คือ LCS = "BC"

LCS กับ Edit Distance

LCS และ Edit Distance เป็นปัญหาที่ดูคล้ายกันแต่ **ต่างกัน**:

- **LCS** — หาลำดับย่อยร่วมที่ยาวที่สุด (เน้นหาสิ่งที่เหมือนกัน)
- **Edit Distance** — หาจำนวนการดำเนินการขั้นต่ำที่ทำให้สายอักขระหนึ่งเปลี่ยนเป็นอีกสายอักขระ (เน้นหาสิ่งที่ต่างกัน)

อย่างไร ก็ตาม ทั้ง สอง ปัญหา ใช้ ตาราง DP 2 มิติ แบบ เดียวกัน และ มี ความ สัมพันธ์: $\text{EditDistance}(A, B) = |A| + |B| - 2 \times \text{LCS}(A, B)$ (เฉพาะ กรณี ที่ อนุญาต เฉพาะ การ แทรก และลบเท่านั้น)

9.14 Longest Increasing Subsequence (LIS)

Longest Increasing Subsequence (LIS) เป็นปัญหาคลาสสิกอีกปัญหาหนึ่ง[2, 3]ที่มีทั้งวิธีแก้ $\mathcal{O}(n^2)$ และ $\mathcal{O}(n \log k)$ โดย k คือความยาวของ LIS การเข้าใจทั้งสองวิธีเป็นสิ่งสำคัญ เพราะวิธี $\mathcal{O}(n \log k)$ มักจำเป็นในการแข่งขัน

9.14.1 นิยามปัญหา

กำหนดลำดับตัวเลข $a[0], a[1], \ldots, a[n-1]$ จงหาลำดับย่อยที่ยาวที่สุดซึ่งมีสมาชิก **เรียงจากน้อยไปมากอย่างเคร่งครัด** (Strictly Increasing) โดยลำดับย่อยไม่จำเป็นต้องอยู่ติดกัน

ตัวอย่าง: LIS

ลำดับ $[10, 22, 9, 33, 21, 50, 41, 60, 80]$

- LIS = $[10, 22, 33, 50, 60, 80]$ (ความยาว 6)
- สังเกตว่าสมาชิกไม่ได้อยู่ติดกันในแถวลำดับเดิม แต่เรียงจากน้อยไปมาก
- อีกคำตอบที่เป็นไปได้: $[10, 22, 33, 41, 60, 80]$ (ความยาว 6 เช่นกัน)

9.14.2 วิธีที่ 1: DP $\mathcal{O}(n^2)$

นิยามให้ `dp[i]` คือความยาวของ LIS ที่ **จบที่ตำแหน่ง** i (กล่าวคือ $a[i]$ เป็นสมาชิกตัวสุดท้าย) สมการเรคัวเรนซ์คือ:

$$\text{dp}[i] = \max_{0 \le j < i,\ a[j] < a[i]} (\text{dp}[j] + 1) \tag{9.20}$$

Base case: `dp[i] = 1` สำหรับทุก i (LIS ที่จบที่ตำแหน่งใดๆ มีความยาวอย่างน้อย 1)
คำตอบ: $\max_{0 \le i < n} \text{dp}[i]$

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a[] = {10, 22, 9, 33, 21, 50, 41, 60, 80};
    int n = 9;
    // dp[i] = length of LIS ending at index i
    vector<int> dp(n, 1); // Base case: every element is LIS of length 1
    int ans = 1;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        ans = max(ans, dp[i]);
    }
    cout << "LIS length = " << ans << "\n"; // 6
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.28: LIS — วิธี $\mathcal{O}(n^2)$ DP

การวิเคราะห์: เวลา $\mathcal{O}(n^2)$, หน่วยความจำ $\mathcal{O}(n)$

9.14.3 การสร้างลำดับ LIS ย้อนกลับ ($\mathcal{O}(n^2)$)

เราสามารถเก็บ `parent[i]` เพื่อบันทึกว่า `dp[i]` มาจากตำแหน่งใด แล้วย้อนกลับเพื่อสร้างลำดับ LIS:

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a[] = {10, 22, 9, 33, 21, 50, 41, 60, 80};
    int n = 9;
    vector<int> dp(n, 1);
    vector<int> parent(n, -1); // parent[i] = previous index in LIS ending at i
    int bestEnd = 0;
    for (int i = 1; i < n; i++) {
```

```
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }
        if (dp[i] > dp[bestEnd]) {
            bestEnd = i;
        }
    }
    // Reconstruct LIS by following parent pointers
    vector<int> lis;
    for (int i = bestEnd; i != -1; i = parent[i]) {
        lis.push_back(a[i]);
    }
    reverse(lis.begin(), lis.end());
    cout << "LIS length = " << dp[bestEnd] << "\n";
    // LIS: 10 22 33 50 60 80
    for (int x : lis) cout << x << " ";
    cout << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.29: LIS — สร้างลำดับย้อนกลับ ($\mathcal{O}(n^2)$)

9.14.4 วิธีที่ 2: Binary Search $\mathcal{O}(n \log k)$

วิธีนี้ใช้แนวคิด **Patience Sorting** โดยดูแลแถวลำดับ `tail[]` ซึ่ง:

- `tail[i]` = สมาชิกตัวสุดท้ายที่ **เล็กที่สุด** ของ Increasing Subsequence ที่มีความยาว $i+1$
- `tail[]` จะ **เรียงจากน้อยไปมาก** เสมอ ทำให้สามารถใช้ Binary Search ได้

สำหรับแต่ละ สมาชิก $a[i]$ ให้หา ตำแหน่งใน `tail[]` ที่ $a[i]$ สามารถ แทนที่ได้ (คือ ตำแหน่ง แรก ที่ `tail[pos]` $\geq$ `a[i]`) โดยใช้ `lower_bound`:

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a[] = {10, 22, 9, 33, 21, 50, 41, 60, 80};
    int n = 9;
    // tail[i] = smallest last element of IS of length i+1
    vector<int> tail;
```

```
    for (int i = 0; i < n; i++) {
        // Find position to replace: first element >= a[i]
        auto it = lower_bound(tail.begin(), tail.end(), a[i]);
        if (it == tail.end()) {
            // a[i] is larger than all -> extend LIS
            tail.push_back(a[i]);
        } else {
            // Replace to keep tail values as small as possible
            *it = a[i];
        }
    }
    // Length of LIS = size of tail array
    cout << "LIS length = " << tail.size() << "\n"; // 6
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.30: LIS — วิธี $\mathcal{O}(n \log k)$ ด้วย Binary Search

การวิเคราะห์: เวลา $\mathcal{O}(n \log k)$ (เพราะแต่ละสมาชิกใช้ Binary Search ใน `tail[]` ซึ่งมีขนาดไม่เกิน k) หน่วยความจำ $\mathcal{O}(k)$

9.14.5 การสร้างลำดับ LIS ย้อนกลับ ($\mathcal{O}(n \log k)$)

วิธี $\mathcal{O}(n \log k)$ ต้องใช้ความพยายามมากขึ้นในการสร้างลำดับย้อนกลับ เราต้องเก็บข้อมูลเพิ่มเติมระหว่างการทำงาน:

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int a[] = {10, 22, 9, 33, 21, 50, 41, 60, 80};
    int n = 9;
    vector<int> tail;      // tail values
    vector<int> tailIdx;   // index in a[] of each tail value
    vector<int> prev(n, -1); // for reconstruction
    for (int i = 0; i < n; i++) {
        auto it = lower_bound(tail.begin(), tail.end(), a[i]);
        int pos = it - tail.begin();
        if (it == tail.end()) {
            tail.push_back(a[i]);
            tailIdx.push_back(i);
        } else {
            *it = a[i];
            tailIdx[pos] = i;
```

```
        }
        // Link to previous element in the LIS
        if (pos > 0) {
            prev[i] = tailIdx[pos - 1];
        }
    }
    // Reconstruct LIS
    vector<int> lis;
    for (int i = tailIdx.back(); i != -1; i = prev[i]) {
        lis.push_back(a[i]);
    }
    reverse(lis.begin(), lis.end());
    cout << "LIS length = " << tail.size() << "\n";
    for (int x : lis) cout << x << " ";
    cout << "\n";
    // Output: 10 22 33 41 60 80
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.31: LIS — สร้างลำดับย้อนกลับ ($\mathcal{O}(n \log k)$)

ตัวอย่าง: การทำงานของ Binary Search LIS

ลำดับ: $[10, 22, 9, 33, 21, 50, 41, 60, 80]$

ขั้น	สมาชิก	tail[]
1	10	[10]
2	22	[10, 22]
3	9	[9, 22]
4	33	[9, 22, 33]
5	21	[9, 21, 33]
6	50	[9, 21, 33, 50]
7	41	[9, 21, 33, 41]
8	60	[9, 21, 33, 41, 60]
9	80	[9, 21, 33, 41, 60, 80]

ขนาดของ `tail[]` = 6 = ความยาวของ LIS

หมายเหตุ: `tail[]` **ไม่ใช่** LIS เสมอไป (ดูตัวแรกคือ 9 ซึ่งไม่ได้อยู่ใน LIS) แต่ **ขนาด** ของ `tail[]` เท่ากับความยาวของ LIS เสมอ

เลือกวิธีไหนดี

- วิธี $\mathcal{O}(n \log k)$ เป็นที่นิยมในการแข่งขันเพราะเร็วกว่า ใช้ `std::lower_bound` เพื่อเขียนโค้ดได้สั้นและถูกต้อง

- วิธี $\mathcal{O}(n^2)$ ง่ายกว่าในการทำความเข้าใจและสร้างลำดับย้อนกลับ และเพียงพอสำหรับ $n \leq 5000$
- หากต้องการทั้งความเร็วและการสร้างลำดับย้อนกลับ ให้ใช้วิธี $\mathcal{O}(n \log k)$ พร้อมเก็บ `prev[]` และ `tailIdx[]`

9.15 Dynamic Programming บน DAG (DP on DAG)

หลายปัญหา DP สามารถมองเป็นปัญหาการหาเส้นทางบน **DAG** (Directed Acyclic Graph — กราฟระบุทิศทางไม่มีวงจร) ได้ การมองปัญหา DP เป็น DAG ช่วยให้เข้าใจโครงสร้างของปัญหาได้ชัดเจนขึ้น และเปิดมุมมองใหม่ในการแก้ปัญหา

9.15.1 DAG คืออะไร

DAG (Directed Acyclic Graph) คือกราฟระบุทิศทางที่ **ไม่มีวงจร** (Cycle) กล่าวคือ ไม่มีเส้นทางจากโหนดใดๆ กลับมาถึงตัวเอง สมบัตินี้ทำให้เราสามารถเรียงลำดับโหนดได้ (Topological Sort) ซึ่งเป็นพื้นฐานของการทำ DP บน DAG

DAG กับ DP

DAG และ DP มีความเกี่ยวข้องกันอย่างลึกซึ้ง:

- ทุกปัญหา DP สามารถมองเป็น DAG ได้ — โหนดคือ state ของ DP และเส้นเชื่อมคือสมการเรคัวเรนซ์
- ในทางกลับกัน ปัญหาบน DAG สามารถแก้ด้วย DP ได้เสมอ เพราะ Topological Sort ให้ลำดับการประมวลผลที่ถูกต้อง

9.15.2 DAG เบื้องหลังปัญหา DP คลาสสิก

ปัญหา DP หลายปัญหาที่เราศึกษาไปแล้ว สามารถมองเป็น DAG ได้:

ตารางที่ 9.3 DAG เบื้องหลังปัญหา DP คลาสสิก

ปัญหา	ปัญหาบน DAG	โครงสร้าง DAG
LIS	Longest Path	เส้นเชื่อมจาก j ไป i ถ้า $a[j] < a[i]$; หาเส้นทางยาวสุด
0-1 Knapsack	Longest Path	โหนด $= (i, w)$; เส้นเชื่อมสำหรับ “เลือก” และ “ไม่เลือก”
Coin Change	Shortest Path	โหนด $=$ มูลค่าคงเหลือ; เส้นเชื่อมสำหรับแต่ละเหรียญ
TSP	Shortest Path	โหนด $= (pos, mask)$; เส้นเชื่อมไปยังเมืองถัดไป

ตัวอย่างเช่น ในปัญหา LIS เราสามารถสร้าง DAG โดย:

1. สร้างโหนดสำหรับแต่ละตำแหน่ง i ในแถวลำดับ
2. สร้างเส้นเชื่อมจากโหนด j ไปโหนด i ถ้า $j < i$ และ $a[j] < a[i]$
3. เพิ่มโหนดเริ่มต้น s เชื่อมไปยังทุกโหนด
4. LIS = เส้นทางยาวที่สุดจาก s ไปยังโหนดใดๆ

ดังนั้น LIS เป็นเพียง **Longest Path บน DAG**!

9.15.3 เทคนิคทั่วไป: Topological Sort + DP

เทคนิคมาตรฐานสำหรับ DP บน DAG มี 2 ขั้นตอน:

1. **Topological Sort** — เรียงลำดับโหนดของ DAG ให้ทุกเส้นเชื่อมไปจากซ้ายไปขวา
2. **ประมวลผลตามลำดับ** — วนลูปตามลำดับ Topological Sort และ “คลาย” (Relax) ค่า DP ของโหนดเป้าหมาย

เนื่องจาก Topological Sort รับประกันว่าเราประมวลผลโหนดต้นทางก่อนโหนดปลายทางเสมอ จึงมั่นใจได้ว่าค่า DP ของโหนดต้นทางถูกต้องแล้วเมื่อเรานำไปใช้

9.15.4 หาเส้นทางยาวสุดบน DAG

การหา Longest Path บนกราฟทั่วไปเป็นปัญหา NP-Hard แต่บน DAG สามารถแก้ได้ในเวลา $\mathcal{O}(V + E)$ โดยใช้ Topological Sort + DP:

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int V = 6;
    vector<vector<pair<int, int>>> adj(V); // adj[u] = {(v, weight)}
    // Build DAG: edges with weights
    adj[0].push_back({1, 5});
    adj[0].push_back({2, 3});
    adj[1].push_back({3, 6});
    adj[1].push_back({2, 2});
    adj[2].push_back({4, 4});
    adj[2].push_back({5, 2});
    adj[3].push_back({4, 6});
    adj[4].push_back({5, 7});
    // Step 1: Topological Sort using Kahn's algorithm
    vector<int> inDegree(V, 0);
    for (int u = 0; u < V; u++) {
```

```cpp
        for (auto& [v, w] : adj[u]) {
            inDegree[v]++;
        }
    }
    queue<int> q;
    for (int i = 0; i < V; i++) {
        if (inDegree[i] == 0) q.push(i);
    }
    vector<int> topo;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (auto& [v, w] : adj[u]) {
            if (--inDegree[v] == 0) q.push(v);
        }
    }
    // Step 2: DP - find longest path from source (node 0)
    const int NEG_INF = -1e9;
    vector<int> dist(V, NEG_INF);
    dist[0] = 0; // Source
    for (int u : topo) {
        if (dist[u] == NEG_INF) continue; // Unreachable from source
        for (auto& [v, w] : adj[u]) {
            dist[v] = max(dist[v], dist[u] + w); // Relax with max
        }
    }
    // Print longest distance to each node
    for (int i = 0; i < V; i++) {
        cout << "Longest path to " << i << " = ";
        if (dist[i] == NEG_INF) cout << "unreachable";
        else cout << dist[i];
        cout << "\n";
    }
    // Longest path to 5 = 24 (0->1->3->4->5)
    return 0;
}
```

ตัวอย่างโค้ดที่ 9.32: Longest Path on DAG

9.15.5 หาจำนวนเส้นทางบน DAG

นอกจากหาเส้นทางยาว/สั้นที่สุดแล้ว เราสามารถนับ **จำนวนเส้นทาง** บน DAG ได้ โดยเปลี่ยนจาก `max` เป็น `+`:

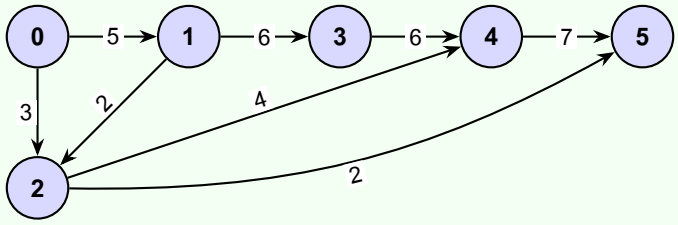
```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int V = 5;
    vector<vector<int>> adj(V); // Unweighted DAG
    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(3);
    adj[3].push_back(4);
    // Topological Sort (same as before, simplified)
    vector<int> inDegree(V, 0);
    for (int u = 0; u < V; u++) {
        for (int v : adj[u]) inDegree[v]++;
    }
    queue<int> q;
    for (int i = 0; i < V; i++) {
        if (inDegree[i] == 0) q.push(i);
    }
    vector<int> topo;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : adj[u]) {
            if (--inDegree[v] == 0) q.push(v);
        }
    }
    // Count number of paths from node 0 to each node
    vector<long long> paths(V, 0);
    paths[0] = 1; // One way to be at source
    for (int u : topo) {
        for (int v : adj[u]) {
            paths[v] += paths[u]; // Add counts instead of max/min
        }
    }
    cout << "Paths from 0 to 4 = " << paths[4] << "\n"; // 2
    // 0->1->3->4 and 0->2->3->4
```

```
38      return 0;
39  }
```

ตัวอย่างโค้ดที่ 9.33: Number of Paths on DAG

ตัวอย่าง: Longest Path บน DAG

พิจารณา DAG ต่อไปนี้:



รูปที่ 9.2 ตัวอย่าง DAG สำหรับ Longest Path

Topological Order: $[0, 1, 2, 3, 4, 5]$

โหนด	Longest Distance จาก 0
0	0
1	5 (จาก 0)
2	7 (จาก 0 ไป 1 ไป 2)
3	11 (จาก 0 ไป 1 ไป 3)
4	17 (จาก 0 ไป 1 ไป 3 ไป 4)
5	24 (จาก 0 ไป 1 ไป 3 ไป 4 ไป 5)

เส้นทางยาวสุดจาก 0 ถึง 5 คือ $0 \to 1 \to 3 \to 4 \to 5$ รวมน้ำหนัก $= 5 + 6 + 6 + 7 = 24$

DP on DAG คือเทคนิคที่ทรงพลัง

หากคุณสามารถสร้างแบบจำลองปัญหาเป็น DAG ได้ การใช้ Topological Sort + DP จะเป็นแนวทางที่ทรงพลังมาก:

- **Longest Path** ใช้ `max` แทน `min` ในการ Relax
- **Shortest Path** ใช้ `min` ตามปกติ (เหมือน DAG Shortest Path ของ Dijkstra)
- **Count Paths** ใช้ + แทน `max/min`
- **Probability** คูณความน่าจะเป็นตามเส้นเชื่อม

ข้อได้เปรียบคือทำงานได้ในเวลา $\mathcal{O}(V + E)$ ซึ่งเร็วกว่า Dijkstra ($\mathcal{O}((V + E) \log V)$) และ Bellman-Ford ($\mathcal{O}(VE)$) สำหรับกราฟที่เป็น DAG

9.16 โจทย์แนะนำ

โจทย์แนะนำสำหรับฝึก DP

ระดับเริ่มต้น:

- Codeforces[10] 189A — Cut Ribbon (Coin Change variant)
- Codeforces 507C — Guess Your Way Out! (Tree DP)
- Kattis — mileage (Kadane's Algorithm)
- UVA[20] 11450 — Wedding Shopping (0-1 Knapsack variant)
- UVA 10943 — How do you add? (Counting DP)

ระดับปานกลาง:

- Codeforces 1155D — Beautiful Array (Kadane + trick)
- Codeforces 466C — Number of Ways (Prefix Sum + DP)
- UVA 104 — Arbitrage (Floyd-Warshall variant)
- UVA 103 — Stacking Boxes (LIS variant)
- Kattis — shopping (Bitmask DP)

ระดับก้าวหน้า:

- Codeforces 1515E — Phoenix and Computers (Combinatorics + DP)
- Codeforces 1312E — Array Shrinking (Interval DP)
- UVA 108 — Maximum Sum (2D Kadane)
- UVA 10405 — Longest Common Subsequence (LCS)
- Kattis — salesmen (TSP variant)

9.17 สรุปบทที่ 9

ในบทนี้ เราได้ศึกษา Dynamic Programming ตั้งแต่แนวคิดพื้นฐานไปจนถึงปัญหาคลาสสิกที่สำคัญ DP เป็นเทคนิคที่ทรงพลังมาก แต่ต้องอาศัยการฝึกฝนเพื่อให้เห็นแพทเทิร์นของสมการเรคัวเรนซ์ได้

9.17.1 ตารางสรุปปัญหา DP ทั้งหมด

ตารางที่ 9.4 สรุปปัญหา DP ทั้งหมดในบทที่ 7

ปัญหา	State	Recurrence	เวลา	หน่วยความจำ
Fibonacci	n	$F(n) = F(n-1) + F(n-2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Coin Combination	n	$\text{state}(n) = \sum \text{state}(n-c)$	$\mathcal{O}(n \times k)$	$\mathcal{O}(n)$
Kadane	j	$S(j) = \max(S(j-1)+a[j], a[j])$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Coin Change	v	$\text{change}(v) = 1 + \min \text{change}(v-c)$	$\mathcal{O}(V \times k)$	$\mathcal{O}(V)$
0-1 Knapsack	(i, w)	$K[i][w] = \max(\text{take}, \text{not})$	$\mathcal{O}(nS)$	$\mathcal{O}(nS)$
TSP	$(pos, mask)$	$\min(\text{dist} + \text{tsp}(\text{nxt}, \ldots))$	$\mathcal{O}(n^2 2^n)$	$\mathcal{O}(n2^n)$
Partition	(n, sum)	$\text{part}[i][j] = \text{part}[i-1][j] \vee \ldots$	$\mathcal{O}(n \times S)$	$\mathcal{O}(n \times S)$
Edit Distance	(m, n)	$1 + \min(\text{ins}, \text{rem}, \text{rep})$	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$
LCS	(i, j)	$\max(\text{dp}[i-1][j], \text{dp}[i][j-1])$ หรือ $\text{dp}[i-1][j-1] + 1$	$\mathcal{O}(nm)$	$\mathcal{O}(nm)$
LIS	i	$\max(\text{dp}[j] + 1)$ หรือ Binary Search	$\mathcal{O}(n \log k)$	$\mathcal{O}(n)$
DP on DAG	โหนด u	Topological Sort + Relax	$\mathcal{O}(V + E)$	$\mathcal{O}(V)$

เทคนิคสำคัญของ DP

1. **ระบุปัญหาว่าเป็น DP** — มองหา Optimization, Counting, Overlapping Subproblems
2. **กำหนด State** — พารามิเตอร์อะไรบ้างที่ระบุปัญหาย่อย? พยายามลดจำนวนให้น้อยที่สุด
3. **หาสมการเรควิเรนซ์** — ยากที่สุด ต้องฝึกฝน! ลองเขียนตัวอย่างเล็กๆ ดูก่อน
4. **เลือก Top-Down หรือ Bottom-Up** — ง่ายที่สุด เลือกตามความถนัด
5. **อย่าลืม Base Case!** — ความผิดพลาดที่พบบ่อยที่สุดคือลืมกำหนด base case ทำให้ได้คำตอบผิด

ข้อควรจำ

1. DP ไม่ใช่เทคนิคที่เรียนแล้วเข้าใจได้ทันที — **ต้องฝึกฝน** ด้วยการทำโจทย์จำนวนมาก
2. ถ้าคิดสมการเรคัวเรนซ์ไม่ออก ให้ **ลองเขียนตัวอย่างเล็ก ๆ ด้วยมือ** ก่อน แล้วหาแพทเทิร์น
3. ระวัง **base case** และ **ขอบเขตของตาราง** — ความผิดพลาดที่พบบ่อยที่สุด
4. ถ้า DP ช้าเกินไป ลอง **ลดจำนวนพารามิเตอร์** หรือ **ใช้ Space Optimization**
5. DP กับ Bitmask เป็นเทคนิคที่ทรงพลังมากสำหรับปัญหาที่มีเซตขนาดเล็ก ($n \leq 20$)

5

ชุดปัญหาด้านเกมเชิงการจัด

ส่วนนี้ครอบคลุมปัญหาด้านเกมเชิงการจัด (Combinatorial Game) และกลยุทธ์ที่เกี่ยวข้อง โดยเน้นขั้นตอนวิธีแบบโลภ (Greedy Algorithms) ซึ่งเป็นกลยุทธ์สำคัญในการแก้ปัญหาที่สามารถหาคำตอบที่ดีที่สุดโดยรวมจากการเลือกคำตอบที่ดีที่สุดในแต่ละขั้นตอน เนื้อหาครอบคลุมหลักการพื้นฐาน โจทย์คลาสสิก และวิธีพิสูจน์ความถูกต้องของขั้นตอนวิธีแบบโลภ ซึ่งเป็นพื้นฐานสำคัญในการวิเคราะห์และแก้ปัญหาเชิงการจัดในการแข่งขัน

Chapter 10

ขั้นตอนวิธีแบบโลภ

ขั้นตอนวิธีแบบโลภ (Greedy Algorithm) เป็นหนึ่งในกลยุทธ์การออกแบบขั้นตอนวิธีที่ใช้หลักการง่ายๆ แต่ทรงพลัง นั่นคือ **ทำสิ่งที่ดีที่สุดในขณะนั้น** โดยไม่คำนึงถึงผลกระทบในอนาคต แม้แนวคิดจะดูเรียบง่าย แต่ขั้นตอนวิธีแบบโลภกลับเป็นเครื่องมือสำคัญในการแข่งขันโปรแกรมเชิงแข่งขัน เพราะหลายโจทย์สามารถแก้ได้อย่างรวดเร็วและถูกต้องด้วยวิธีนี้ อย่างไรก็ตาม สิ่งที่ท้าทายที่สุดคือการตัดสินใจว่า **เมื่อไรที่ Greedy ใช้ได้** เพราะหากใช้ในสถานการณ์ที่ไม่เหมาะสม จะได้คำตอบที่ผิดอย่างสมบูรณ์ ในบทนี้เราจะศึกษาขั้นตอนวิธีแบบโลภอย่างเป็นระบบ ตั้งแต่แนวคิดพื้นฐาน โจทย์คลาสสิก ไปจนถึงวิธีพิสูจน์ความถูกต้อง[2, 4]

10.1 แนะนำขั้นตอนวิธีแบบโลภ

10.1.1 ขั้นตอนวิธีแบบโลภคืออะไร

ขั้นตอนวิธีแบบโลภ (Greedy Algorithm) คือขั้นตอนวิธีที่ในแต่ละขั้นตอนจะ **เลือกตัวเลือกที่ดีที่สุดในเวลานั้น** (Locally Optimal Choice) โดยหวังว่าการเลือกที่ดีที่สุดในแต่ละขั้นจะนำไปสู่คำตอบที่ดีที่สุดโดยรวม (Globally Optimal Solution) กล่าวอีกนัยหนึ่ง Greedy ไม่เคยย้อนกลับไปเปลี่ยนการตัดสินใจที่ทำไปแล้ว ทุกการตัดสินใจเป็นการตัดสินใจขั้นสุดท้าย

นิยาม 10.1 (ขั้นตอนวิธีแบบโลภ)**.** ขั้นตอนวิธีแบบโลภคือขั้นตอนวิธีที่สร้างคำตอบทีละขั้นตอน โดยในแต่ละขั้นเลือกตัวเลือกที่ดีที่สุดในขณะนั้น (Local Optimum) และไม่เคยย้อนกลับเพื่อพิจารณาตัวเลือกอื่น

ตัวอย่างชีวิตประจำวัน: สมมติคุณไปซื้อของในร้านสะดวกซื้อและต้องจ่ายเงิน 47 บาท คุณมีแบงค์และเหรียญ 1, 5, 10, 20, 50, 100 บาท วิธีแบบโลภคือเลือกแบงค์ใหญ่สุดที่ใช้ได้ก่อนเสมอ: เลือก 20 บาท (เหลือ 27), เลือก 20 บาท (เหลือ 7), เลือก 5 บาท (เหลือ 2), เลือก 1 บาท (เหลือ 1), เลือก 1 บาท (เหลือ 0) รวม 5 เหรียญ/แบงค์ ซึ่งเป็นจำนวนน้อยที่สุดที่เป็นไปได้

10.1.2 คุณสมบัติสำคัญสองประการ

เพื่อให้ขั้นตอนวิธีแบบโลภให้คำตอบที่ถูกต้อง โจทย์ปัญหาจะต้องมีคุณสมบัติสำคัญสองประการ:

ทฤษฎีบท 10.1 (Greedy Choice Property)**.** ปัญหามีคุณสมบัติ Greedy Choice ก็ต่อเมื่อ **สามารถสร้างคำตอบที่เหมาะที่สุดได้จากการเลือกที่ดีที่สุดในเวลานั้น** โดยไม่จำเป็นต้องพิจารณาผลกระทบต่ออนาคต กล่าวคือ การเลือกในเวลาปัจจุบันไม่ขัดแย้งกับคำตอบที่เหมาะที่สุดในอนาคต

ทฤษฎีบท 10.2 (Optimal Substructure)**.** ปัญหามีคุณสมบัติ Optimal Substructure ก็ต่อเมื่อ **คำตอบที่เหมาะที่สุดของปัญหาหลักประกอบด้วยคำตอบที่เหมาะที่สุดของปัญหาย่อย** กล่าวคือ ถ้าแบ่งปัญหาเป็นปัญหาย่อย คำตอบที่ดีที่สุดของปัญหาย่อยทุกส่วนจะรวมกันเป็นคำตอบที่ดีที่สุดของปัญหาหลัก

คุณสมบัติ Optimal Substructure ไม่เพียงพอ!

หลายคนเข้าใจผิดว่าถ้าปัญหามี Optimal Substructure แล้ว Greedy จะใช้ได้เสมอ แต่ความจริงคือ **ทั้งสองคุณสมบัติต้องเป็นจริงพร้อมกัน** ตัวอย่างเช่น ปัญหา 0-1 Knapsack มี Optimal Substructure แต่ไม่มี Greedy Choice Property ดังนั้น Greedy ใช้ไม่ได้ (จะอธิบายเพิ่มในหัวข้อ 10.5)

10.1.3 เมื่อไร Greedy ใช้ได้และใช้ไม่ได้

การตัดสินใจว่า Greedy ใช้ได้หรือไม่เป็นสิ่งที่ยากที่สุดในการแก้โจทย์แบบ Greedy ไม่มีสูตรสำเร็จที่บอกได้ชัดเจน แต่มีแนวทางดังนี้:

- **Greedy มักใช้ได้** เมื่อปัญหาเกี่ยวกับการเลือกลำดับ (Scheduling), การเลือกช่วง (Interval Selection), หรือการหาค่าต่ำสุด/สูงสุดที่มีโครงสร้างเรียงลำดับได้
- **Greedy มักใช้ไม่ได้** เมื่อการเลือกในขั้นตอนหนึ่งส่งผลกระทบต่อตัวเลือกในอนาคตอย่างซับซ้อน (เช่น ปัญหาที่ต้องพิจารณาทุกกรณี)
- **วิธีที่ดีที่สุด** คือลองหา Counter-example (ตัวอย่างที่ทำให้ Greedy ผิด) ถ้าหาไม่ได้และมีเหตุผลสนับสนุน ก็ลองใช้ Greedy ดู

10.1.4 ตัวอย่าง: ปัญหาการทอนเงิน (Coin Change)

ปัญหาการทอนเงินเป็นตัวอย่างคลาสสิกที่แสดงให้เห็นว่า Greedy อาจใช้ได้หรือไม่ได้ขึ้นอยู่กับเหรียญที่มี

เหรียญมาตรฐานสหรัฐ {1, 5, 10, 25, 50, 100}: Greedy **ใช้ได้** เสมอ เพราะเหรียญแต่ละชนิดเป็นเท่าของเหรียญที่เล็กกว่าหรือเผื่อเหลือได้อย่างเหมาะสม

เหรียญพิเศษ {1, 3, 4}: Greedy **ใช้ไม่ได้!** ลองพิจารณาการทอนเงิน 6 บาท:

- **Greedy:** เลือก 4 (เหลือ 2) → เลือก 1 (เหลือ 1) → เลือก 1 (เหลือ 0) → รวม **3 เหรียญ**
- **คำตอบที่เหมาะที่สุด:** เลือก 3 (เหลือ 3) → เลือก 3 (เหลือ 0) → รวม **2 เหรียญ**

ตัวอย่าง: Coin Change แบบ Greedy ที่ผิด

```cpp
#include <bits/stdc++.h>
using namespace std;
// Greedy coin change (works for standard coins only!)
int greedyCoinChange(int amount, vector<int>& coins) {
    sort(coins.rbegin(), coins.rend());  // Sort in descending order
    int count = 0;
    for (int c : coins) {
        count += amount / c;
        amount %= c;
    }
```

```cpp
    if (amount != 0) return -1;   // Cannot make change
    return count;
}
int main() {
    vector<int> thai_coins = {1, 5, 10, 25, 50, 100};
    cout << greedyCoinChange(47, thai_coins) << "\n";   // 5 (20+20+5+1+1)
    // But with coins {1,3,4} it's wrong!
    vector<int> bad_coins = {1, 3, 4};
    cout << greedyCoinChange(6, bad_coins) << "\n";    // 3 (4+1+1) but answer is 2 (3+3)
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.1: Coin Change แบบ Greedy — ใช้ได้เฉพาะเหรียญมาตรฐาน

บทเรียนสำคัญ

จากตัวอย่าง Coin Change เราเรียนรู้ว่า Greedy อาจดูถูกต้องในบางกรณี แต่ผิดในกรณีอื่น ดังนั้น **ต้องพิสูจน์ความถูกต้อง** ก่อนจะใช้ Greedy ในการแข่งขันเสมอ หัวข้อ 10.8 จะกล่าวถึงวิธีพิสูจน์อย่างละเอียด

10.2 การเลือกกิจกรรม (Activity Selection)

ปัญหาการเลือกกิจกรรม (Activity Selection Problem) เป็นหนึ่งในโจทย์ Greedy คลาสสิกที่สำคัญที่สุด และเป็นต้นแบบของโจทย์ Interval หลายประเภท[2]

10.2.1 นิยามปัญหา

นิยาม 10.2 (Activity Selection Problem). กำหนดกิจกรรม n กิจกรรม แต่ละกิจกรรมมีเวลาเริ่ม s_i และเวลาจบ f_i กิจกรรมสองกิจกรรม **เข้ากันได้** (Compatible) ถ้าช่วงเวลาของทั้งสองไม่ทับซ้อนกัน (กิจกรรมหนึ่งจบก่อนอีกกิจกรรมเริ่ม) จงหาจำนวนกิจกรรมที่มากที่สุดที่สามารถเลือกได้โดยไม่มีการทับซ้อน

ตัวอย่าง

มีกิจกรรม 8 กิจกรรม:

กิจกรรม	เวลาเริ่ม s_i	เวลาจบ f_i	ความยาว
1	1	4	3
2	3	5	2
3	0	6	6
4	5	7	2
5	3	9	6
6	5	9	4
7	6	10	4
8	8	11	3

คำตอบที่เหมาะที่สุดคือเลือกกิจกรรม 1, 4, 8 (เวลา [1,4], [5,7], [8,11]) รวม 3 กิจกรรม

10.2.2 กลยุทธ์แบบโลภ: เรียงตามเวลาจบ

มีกลยุทธ์ Greedy หลายแบบที่อาจนึกได้ แต่มีเพียงแบบเดียวที่ถูกต้อง:

1. **เลือกกิจกรรมที่สั้นที่สุดก่อน** $\rightarrow$ **ผิด!** กิจกรรมสั้นอาจทับช่วงเวลาที่สำคัญ
2. **เลือกกิจกรรมที่เริ่มเร็วที่สุดก่อน** $\rightarrow$ **ผิด!** กิจกรรมที่เริ่มเร็วแต่จบช้าจะบังคับให้เลือกได้น้อย
3. **เลือกกิจกรรมที่จบเร็วที่สุดก่อน** $\rightarrow$ **ถูกต้อง!** การจบเร็วทิ้งพื้นที่ว่างให้กิจกรรมอื่นมากที่สุด

ทฤษฎีบท 10.3 (ความถูกต้องของการเรียงตามเวลาจบ)**.** การเลือกกิจกรรมที่จบเร็วที่สุดก่อนเสมอ จะได้จำนวนกิจกรรมที่มากที่สุดเสมอ

10.2.3 การพิสูจน์ความถูกต้อง (Exchange Argument)

เราจะใช้เทคนิค **Exchange Argument** (การสลับ) ซึ่งเป็นวิธีพิสูจน์ Greedy ที่ทรงพลังที่สุด:

แนวคิด: ให้คำตอบที่เหมาะที่สุด (Optimal Solution) เป็น O^* และคำตอบของ Greedy เป็น G เราจะแสดงว่าสามารถ **สลับ** กิจกรรมแรกของ O^* มาเป็นกิจกรรมแรกของ G ได้โดยไม่ทำให้คำตอบแย่ลง แล้วใช้ Induction กับปัญหาย่อยที่เหลือ

พิสูจน์:

1. สมมติ O^* เลือกกิจกรรม $a_1, a_2, \ldots, a_k$ (เรียงตามเวลาจบ) และ G เลือก $g_1, g_2, \ldots, g_m$ (เรียงตามเวลาจบ)
2. เนื่องจาก G เลือกกิจกรรมที่จบเร็วที่สุดเป็นอันดับแรก $f(g_1) \leq f(a_1)$
3. สร้างคำตอบใหม่ $O' = \{g_1, a_2, a_3, \ldots, a_k\}$ — แทนที่ a_1 ด้วย g_1
4. เนื่องจาก $f(g_1) \leq f(a_1)$ กิจกรรม g_1 จบก่อนหรือพร้อม a_1 ดังนั้น a_2 (ซึ่งเริ่มหลัง $f(a_1)$) ย่อมเริ่มหลัง $f(g_1)$ ด้วย แสดงว่า O' เป็นคำตอบที่ถูกต้อง

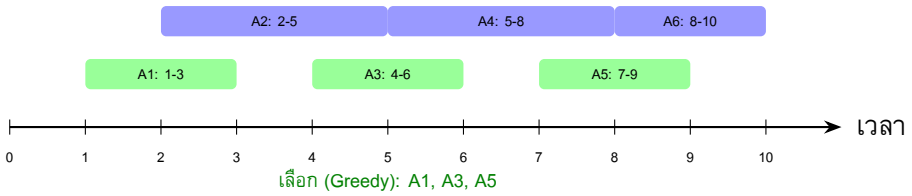
5. ดังนั้น $|O'| = |O^*|$ และ O' ตรงกับ G ในกิจกรรมแรกแล้ว

6. ใช้ Induction กับปัญหาย่อยที่เหลือ (เริ่มจากเวลา $f(g_1)$ เป็นต้นไป) จะได้ว่า $m \geq k$ ดังนั้น Greedy ไม่แย่กว่า Optimal

Exchange Argument — เทคนิคสำคัญ

Exchange Argument เป็นเทคนิคพิสูจน์ Greedy ที่ใช้บ่อยที่สุด แนวคิดคือ: (1) สมมติมีคำตอบที่ดีกว่า Greedy (2) แสดงว่าสามารถสลับตัวเลือกในคำตอบนั้นมาเป็นตัวเลือกของ Greedy ได้โดยไม่ทำให้ผลลัพธ์แย่ลง (3) สรุปว่า Greedy ไม่แย่กว่าคำตอบที่ดีที่สุด

รูปที่ 10.1 แสดงตัวอย่างปัญหา Activity Selection โดยกิจกรรมที่ถูกเลือก (ตาม Greedy: เลือกที่จบเร็วสุด) แสดงด้วยสีเข้ม



รูปที่ 10.1 Activity Selection: เลือกกิจกรรมที่จบเร็วที่สุดก่อน (สีเขียว = กิจกรรมที่เลือก)

10.2.4 โค้ด C++ สำหรับ Activity Selection

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> activities(n);  // {end, start}
    for (int i = 0; i < n; i++) {
        int s, f;
        cin >> s >> f;
        activities[i] = {f, s};  // Store end first for easier sorting
    }
    // Sort by end time in ascending order
    sort(activities.begin(), activities.end());
    int count = 0;
    int lastEnd = -1;  // End time of last selected activity
    for (auto& [f, s] : activities) {
        if (s >= lastEnd) {  // This activity starts after previous one ends
            count++;
```

```
            lastEnd = f;
        }
    }
    cout << count << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.2: Activity Selection — เลือกกิจกรรมที่มากที่สุด

ความซับซ้อน: การเรียง $\mathcal{O}(n \log n)$ + การวน $\mathcal{O}(n)$ = $\mathcal{O}(n \log n)$

กรณีขอบเขตเข้มงวด

ในบางโจทย์ กิจกรรมสองกิจกรรมที่จบพร้อมกันถือว่า **ทับซ้อน** (ต้องใช้เงื่อนไข `s > lastEnd`) แต่ในบางโจทย์ถือว่า **ไม่ทับซ้อน** (ต้องใช้ `s >= lastEnd`) อ่านโจทย์ให้ละเอียดทุกครั้ง!

10.2.5 การแสดงกิจกรรมที่เลือก

พิจารณากิจกรรม 8 กิจกรรมจากตัวอย่างข้างต้น หลังเรียงตามเวลาจบ:

ลำดับหลังเรียง	เวลาเริ่ม	เวลาจบ	เลือก?
1	1	4	✓
2	3	5	× (เริ่ม 3 < จบของอันก่อน 4)
3	0	6	× (เริ่ม 0 < จบของอันก่อน 4)
4	5	7	✓
5	3	9	× (เริ่ม 3 < จบของอันก่อน 7)
6	5	9	× (เริ่ม 5 < จบของอันก่อน 7)
7	6	10	× (เริ่ม 6 < จบของอันก่อน 7)
8	8	11	✓

ผลลัพธ์: เลือกกิจกรรมที่จบเวลา 4, 7, 11 รวม 3 กิจกรรม

10.3 ปัญหาการจัดตาราง (Scheduling Problems)

ปัญหาการจัดตาราง (Scheduling) เป็นกลุ่มโจทย์ Greedy ที่พบบ่อยมากในการแข่งขัน โดยมีเป้าหมายแตกต่างกันไปตามเงื่อนไขของโจทย์

10.3.1 การลดเวลารอคอยรวม (Minimizing Total Completion Time)

นิยาม 10.3 (Minimizing Total Completion Time)**.** กำหนดงาน n งาน งานที่ i ใช้เวลา t_i หน่วย งานทั้งหมดทำงานได้ทีละ 1 งาน จงหาลำดับการทำงานที่ทำให้ **ผลรวมของเวลาเสร็จสิ้นของทุกงาน** น้อยที่สุด

เวลาเสร็จสิ้นของงานที่อยู่ในลำดับที่ k คือ $C_k = t_1 + t_2 + \cdots + t_k$ (ผลรวมของเวลาทำงานตั้งแต่งานแรกถึงงานที่ k) เป้าหมายคือลด $\sum_{k=1}^{n} C_k$

ทฤษฎีบท 10.4 (Shortest Job First)**.** การเรียงงานจาก **เวลาน้อยไปมาก** (Shortest Job First) ให้ผลรวมของเวลาเสร็จสิ้นน้อยที่สุด

เหตุผล: งานที่ใช้เวลาน้อยควรทำก่อน เพราะทุกงานที่อยู่หลังมันจะต้องรอมันอยู่แล้ว การทำงานสั้นก่อนจะลดเวลารอของงานอื่นๆ ลง

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> t(n);
    for (int i = 0; i < n; i++) {
        cin >> t[i];
    }
    // Sort in ascending order of time
    sort(t.begin(), t.end());
    long long totalCompletion = 0;
    long long prefixSum = 0;
    for (int i = 0; i < n; i++) {
        prefixSum += t[i];
        totalCompletion += prefixSum;
    }
    cout << totalCompletion << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.3: Minimizing Total Completion Time — Shortest Job First

ตัวอย่าง

งาน 4 งาน ใช้เวลา 3, 1, 4, 2 ตามลำดับ

ลำดับ Shortest Job First: 1, 2, 3, 4

- งานเวลา 1: เสร็จเวลา 1
- งานเวลา 2: เสร็จเวลา 1+2 = 3
- งานเวลา 3: เสร็จเวลา 1+2+3 = 6
- งานเวลา 4: เสร็จเวลา 1+2+3+4 = 10

ผลรวม = 1 + 3 + 6 + 10 = 20 (น้อยที่สุด)

ลำดับอื่น เช่น 3, 4, 2, 1:

- งานเวลา 3: เสร็จเวลา 3
- งานเวลา 4: เสร็จเวลา 7
- งานเวลา 2: เสร็จเวลา 9
- งานเวลา 1: เสร็จเวลา 10

ผลรวม = 3 + 7 + 9 + 10 = 29 (มากกว่า)

10.3.2 การจัดตารางตามกำหนด (Minimize Maximum Lateness)

นิยาม 10.4 (Minimize Maximum Lateness). กำหนดงาน n งาน งานที่ i ใช้เวลา t_i และมี Deadline d_i งานทั้งหมดทำงานได้ทีละ 1 งาน เริ่มตั้งแต่เวลา 0 Lateness ของงาน i คือ $L_i = \max(0, C_i - d_i)$ โดย C_i คือเวลาเสร็จของงาน i จงหาลำดับการทำงานที่ทำให้ **Lateness สูงสุด** น้อยที่สุด

ทฤษฎีบท 10.5 (Earliest Deadline First). การเรียงงานตาม **Deadline จากน้อยไปมาก** (Earliest Deadline First) ทำให้ Maximum Lateness น้อยที่สุด

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> jobs(n);   // {deadline, time}
    for (int i = 0; i < n; i++) {
        cin >> jobs[i].second >> jobs[i].first;   // time, deadline
    }
    // Sort by deadline in ascending order
    sort(jobs.begin(), jobs.end());
    long long currentTime = 0;
    long long maxLateness = 0;
    for (auto& [deadline, time] : jobs) {
        currentTime += time;
        long long lateness = max(0LL, currentTime - deadline);
        maxLateness = max(maxLateness, lateness);
    }
    cout << maxLateness << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.4: Minimize Maximum Lateness — Earliest Deadline First

ตัวอย่าง

งาน 3 งาน:

	เวลา t_i	Deadline d_i
งาน A	3	6
งาน B	2	8
งาน C	1	9

เรียงตาม Deadline: A, B, C

- A เสร็จเวลา 3, lateness = max(0, 3-6) = 0
- B เสร็จเวลา 5, lateness = max(0, 5-8) = 0
- C เสร็จเวลา 6, lateness = max(0, 6-9) = 0

Maximum Lateness = 0 (ดีที่สุด)

สรุปกลยุทธ์ Scheduling

เป้าหมาย	กลยุทธ์ Greedy
ลดผลรวม Completion Time	Shortest Job First
ลด Maximum Lateness	Earliest Deadline First
หาจำนวนกิจกรรมสูงสุด	Earliest Finish Time First

จดจำตารางนี้ไว้ เพราะกลยุทธ์เหล่านี้ใช้บ่อยมากในการแข่งขัน

10.4 ฮัฟฟ์แมนเข้ารหัส (Huffman Coding)

ฮัฟฟ์แมนเข้ารหัส (Huffman Coding) เป็นขั้นตอนวิธีบีบอัดข้อมูลแบบ Lossless ที่ใช้หลักการ Greedy อย่างสวยงาม[2] เป็นตัวอย่างที่ดีของการใช้ Greedy ร่วมกับโครงสร้างข้อมูล Priority Queue

10.4.1 ปัญหาการเข้ารหัส

สมมติเรามีข้อความที่ประกอบด้วยอักขระหลายชนิด แต่ละชนิดปรากฏด้วยความถี่แตกต่างกัน เช่น:

อักขระ	A	B	C	D	E
ความถี่	5	9	12	13	16

ถ้าใช้รหัสความยาวคงที่ (Fixed-length) ต้องใช้ 3 บิตต่ออักขระ ($2^3 = 8 \geq 5$) รวม 55 บิต แต่ถ้าใช้รหัสความยาวผันแปร (Variable-length) โดยกำหนดรหัสสั้นให้อักขระที่พบบ่อย จะลดจำนวนบิตรวมลงได้

เงื่อนไขสำคัญ: รหัสต้องเป็น **Prefix Code** คือรหัสของอักขระใดๆ ไม่เป็น Prefix ของอักขระอื่น เพื่อให้ถอดรหัสได้โดยไม่กำกวม

นิยาม 10.5 (Huffman Coding Problem)**.** กำหนดอักขระ n ชนิด แต่ละชนิดมีความถี่ f_i จงหารหัสแบบ Prefix Code ที่ทำให้ **จำนวนบิตรวม** $\sum_{i=1}^{n} f_i \times l_i$ น้อยที่สุด โดย l_i คือความยาวรหัสของอักขระ i

10.4.2 กลยุทธ์ Greedy: รวมสองความถี่ต่ำสุด

ขั้นตอนวิธีของ Huffman ใช้กลยุทธ์ง่ายๆ:

1. สร้าง Priority Queue (Min-Heap) ของ Node แต่ละอันแทนอักขระ 1 ชนิด มีค่า weight = ความถี่
2. ในแต่ละขั้น: ดึง Node ที่มี weight ต่ำสุด 2 อันออก สร้าง Node ใหม่ที่มี weight = ผลรวมของทั้งสอง แล้วใส่กลับเข้า Priority Queue
3. ทำซ้ำจนเหลือ Node เดียวใน Queue — Node นั้นคือรากของ Huffman Tree

ความยาวรหัสของแต่ละอักขระ = ความลึกของ Leaf Node นั้นใน Huffman Tree

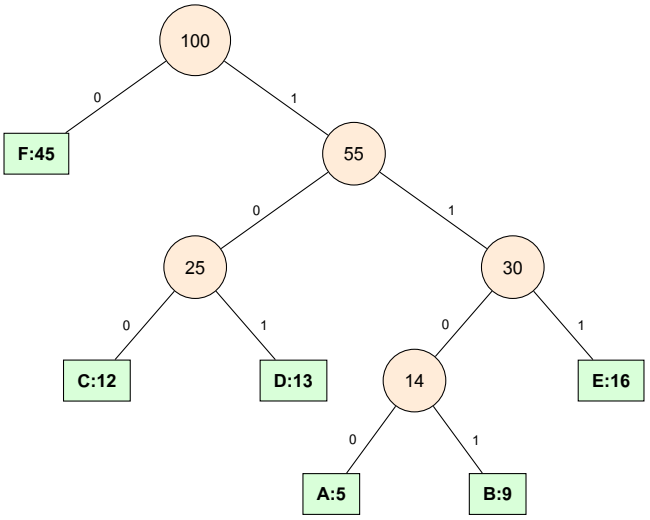
ตัวอย่าง: สร้าง Huffman Tree

จากความถี่ A=5, B=9, C=12, D=13, E=16:
ขั้นที่ 1: ดึง A(5) และ B(9) $\rightarrow$ สร้าง Node ใหม่ weight=14, ใส่กลับ
Queue: [C(12), Node(14), D(13), E(16)]
ขั้นที่ 2: ดึง C(12) และ D(13) $\rightarrow$ สร้าง Node ใหม่ weight=25, ใส่กลับ
Queue: [Node(14), E(16), Node(25)]
ขั้นที่ 3: ดึง Node(14) และ E(16) $\rightarrow$ สร้าง Node ใหม่ weight=30, ใส่กลับ
Queue: [Node(25), Node(30)]
ขั้นที่ 4: ดึง Node(25) และ Node(30) $\rightarrow$ สร้าง Root Node weight=55
Queue: [Root(55)] — เสร็จสิ้น!
รหัสที่ได้:

- C: 00 (ความยาว 2) $\rightarrow$ บิตรวม = $12 \times 2 = 24$
- D: 01 (ความยาว 2) $\rightarrow$ บิตรวม = $13 \times 2 = 26$
- A: 100 (ความยาว 3) $\rightarrow$ บิตรวม = $5 \times 3 = 15$
- B: 101 (ความยาว 3) $\rightarrow$ บิตรวม = $9 \times 3 = 27$
- E: 11 (ความยาว 2) $\rightarrow$ บิตรวม = $16 \times 2 = 32$

รวม = 124 บิต เทียบกับ 165 บิตหากใช้ Fixed-length (3 บิตทุกตัว) — ประหยัดกว่า!

รูปที่ 10.2 แสดงตัวอย่างต้นไม้ฮัฟฟ์แมนสำหรับข้อความที่มีตัวอักษร A (freq=5), B (freq=9), C (freq=12), D (freq=13), E (freq=16), F (freq=45)



รูปที่ 10.2 ต้นไม้ฮัฟฟ์แมน: ตัวอักษรที่มีความถี่สูง (F=45) ได้รหัสสั้น (0) ตัวอักษรที่มีความถี่ต่ำ (A=5 และ B=9) ได้รหัสยาว (1100 และ 1101)

10.4.3 โค้ด C++ สำหรับ Huffman Coding

```cpp
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int freq;
    char ch;
    Node *left, *right;
    Node(int f, char c, Node* l = nullptr, Node* r = nullptr)
        : freq(f), ch(c), left(l), right(r) {}
};
// Comparator for Min-Heap (smallest freq on top)
struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;   // > to make it a min-heap
    }
};
// Build codes from Huffman Tree
void buildCodes(Node* root, string code,
                map<char, string>& codes) {
    if (!root) return;
    if (!root->left && !root->right) {
        // Leaf node = character
        codes[root->ch] = code;
        return;
    }
```

```
    buildCodes(root->left, code + "0", codes);
    buildCodes(root->right, code + "1", codes);
}
int main() {
    int n;
    cin >> n;
    priority_queue<Node*, vector<Node*>, Compare> pq;
    for (int i = 0; i < n; i++) {
        char c;
        int f;
        cin >> c >> f;
        pq.push(new Node(f, c));
    }
    // Special case: only one character
    if (n == 1) {
        cout << pq.top()->ch << ": 0\n";
        cout << "Total bits: " << pq.top()->freq << "\n";
        return 0;
    }
    // Build Huffman Tree
    while (pq.size() > 1) {
        Node* left = pq.top(); pq.pop();
        Node* right = pq.top(); pq.pop();
        Node* parent = new Node(left->freq + right->freq,
                                '\0', left, right);
        pq.push(parent);
    }
    Node* root = pq.top();
    // Build codes
    map<char, string> codes;
    buildCodes(root, "", codes);
    // Display result
    int totalBits = 0;
    for (auto& [ch, code] : codes) {
        cout << ch << ": " << code << "\n";
    }
    for (auto& [ch, code] : codes) {
        // Need actual frequency to calculate total bits
        // (In real code, should store freq in map too)
    }
    return 0;
```

```
66 }
```

ตัวอย่างโค้ดที่ 10.5: Huffman Coding ด้วย Priority Queue

ความซับซ้อน: การดึงและใส่ Priority Queue แต่ละครั้งใช้เวลา $\mathcal{O}(\log n)$ ทำทั้งหมด $n-1$ ครั้ง รวม $\mathcal{O}(n \log n)$

Huffman Coding กับ Greedy

Huffman Coding เป็นตัวอย่างที่ยอดเยี่ยมของ Greedy เพราะการรวมสอง Node ที่มีความถี่ต่ำสุดในแต่ละขั้น เป็นตัวเลือกที่ดีที่สุดในเวลานั้น และนำไปสู่คำตอบที่เหมาะที่สุดโดยรวม สามารถพิสูจน์ได้ด้วย Exchange Argument คล้าย Activity Selection

10.5 ปัญหา Fractional Knapsack

ปัญหา Knapsack เป็นหนึ่งในปัญหาคลาสสิกของ Computer Science ที่แสดงให้เห็นความแตกต่างระหว่าง Greedy ที่ใช้ได้และไม่ได้อย่างชัดเจน[2]

10.5.1 นิยามปัญหา

นิยาม 10.6 (Fractional Knapsack Problem)**.** กำหนดสินค้า n ชิ้น สินค้าชิ้นที่ i มีน้ำหนัก w_i และมูลค่า v_i และกระเป๋าที่รับน้ำหนักได้ W สามารถเลือกสินค้าได้ **เป็นส่วน ๆ** (Fractional) กล่าวคือ สามารถเอาสินค้าเพียงส่วนหนึ่ง เช่น ครึ่งหนึ่งของสินค้าชิ้นที่ i จงหามูลค่ารวมสูงสุดที่สามารถบรรจุในกระเป๋าได้

10.5.2 ความแตกต่างจาก 0-1 Knapsack

0-1 Knapsack vs Fractional Knapsack

- **0-1 Knapsack:** ต้องเลือกสินค้าทั้งชิ้นหรือไม่เลือกเลย (0 หรือ 1) Greedy **ใช้ไม่ได้** ต้องใช้ Dynamic Programming
- **Fractional Knapsack:** สามารถเลือกสินค้าเป็นส่วนๆ ได้ Greedy **ใช้ได้** โดยเลือกตามอัตราส่วนมูลค่าต่อน้ำหนัก

ตัวอย่างที่แสดงให้เห็นความแตกต่าง:

ตัวอย่าง: Greedy ผิดสำหรับ 0-1 Knapsack

กระเป๋า W=10, สินค้า:

	น้ำหนัก w_i	มูลค่า v_i	อัตราส่วน v_i/w_i
สินค้า A	6	30	5.0
สินค้า B	5	28	5.6
สินค้า C	5	28	5.6

Greedy (เลือกตามอัตราส่วน): B + C = 28 + 28 = 56 (น้ำหนัก 10) ✓
แต่ถ้าเพิ่มสินค้า D: น้ำหนัก 6, มูลค่า 30, อัตราส่วน 5.0 และลด W=10:
Greedy: เลือก B (อัตราส่วน 5.6) แล้ว C (อัตราส่วน 5.6) $\rightarrow$ มูลค่า 56
Optimal (0-1): เลือก A อย่างเดียว $\rightarrow$ มูลค่า 30
แต่สำหรับ Fractional: Greedy ถูกต้องเสมอ!

10.5.3 กลยุทธ์ Greedy: เรียงตามอัตราส่วนมูลค่าต่อน้ำหนัก

ทฤษฎีบท 10.6 (Fractional Knapsack Greedy)**.** การเลือกสินค้าตามลำดับ **อัตราส่วนมูลค่าต่อน้ำหนัก** (v_i/w_i) จากมากไปน้อย จะได้มูลค่ารวมสูงสุดสำหรับ Fractional Knapsack เสมอ

แนวคิด: สินค้าที่มีอัตราส่วนสูงคือ “คุ้ม” ที่สุดต่อหน่วยน้ำหนัก ดังนั้นควรเลือกมันก่อน เมื่อเลือกทั้งชิ้นไม่ได้ (กระเป๋าเต็ม) ก็เลือกเป็นส่วนๆ ได้

10.5.4 โค้ด C++ สำหรับ Fractional Knapsack

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    double W;
    cin >> n >> W;
    vector<tuple<double, double, double>> items(n);   // {ratio, weight, value}
    for (int i = 0; i < n; i++) {
        double v, w;
        cin >> v >> w;
        items[i] = {v / w, w, v};
    }
    // Sort by ratio in descending order
    sort(items.begin(), items.end(),
        greater<tuple<double, double, double>>());
    double totalValue = 0;
    double remaining = W;
    for (auto& [ratio, weight, value] : items) {
        if (remaining <= 0) break;
```

```
        double take = min(weight, remaining);
        totalValue += take * ratio;   // = take * (value / weight)
        remaining -= take;
    }
    cout << fixed << setprecision(6) << totalValue << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.6: Fractional Knapsack — เลือกตามอัตราส่วนมูลค่าต่อน้ำหนัก

ความซับซ้อน: $\mathcal{O}(n \log n)$ จากการเรียง

ทำไม Fractional ใช้ Greedy ได้แต่ 0-1 ใช้ไม่ได้?

ความแตกต่างสำคัญอยู่ที่ **ความยืดหยุ่น** ใน Fractional Knapsack เราสามารถ “เติมช่องว่าง” ด้วยส่วนของสินค้าได้ ทำให้ไม่มีพื้นที่สูญเปล่า ดังนั้นการเลือกสินค้าคุ้มที่สุดก่อนจะไม่มีทางเสียเปรียบ แต่ใน 0-1 Knapsack การเลือกสินค้าคุ้มอาจ “เปลืองที่” ในกระเป๋า ทำให้สินค้าอื่นใส่ไม่ได้ นี่คือเหตุผลที่ Greedy Choice Property ไม่เป็นจริงสำหรับ 0-1 Knapsack

10.6 การเลือกช่วงเวลา/เส้นเชื่อม (Interval Problems)

ปัญหาเกี่ยวกับช่วงเวลา (Interval) เป็นกลุ่มโจทย์ Greedy ที่พบบ่อยที่สุดในการแข่งขัน[4, 3] เราจะศึกษา 3 ประเภทย่อยที่สำคัญ

10.6.1 ช่วงเวลาไม่ทับซ้อนสูงสุด (Maximum Non-overlapping Intervals)

ปัญหานี้เหมือนกับ Activity Selection ที่เราศึกษาไปแล้ว แต่จะขยายให้เห็นการประยุกต์ใช้หลายรูปแบบ

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> intervals(n);   // {end, start}
    for (int i = 0; i < n; i++) {
        int s, e;
        cin >> s >> e;
        intervals[i] = {e, s};
    }
    sort(intervals.begin(), intervals.end());
    int count = 0;
    int lastEnd = INT_MIN;
    for (auto& [e, s] : intervals) {
```

```
        if (s > lastEnd) {   // Non-overlapping (use > if edges can touch)
            count++;
            lastEnd = e;
        }
    }
    cout << count << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.7: Maximum Non-overlapping Intervals

10.6.2 จำนวนลูกธนูต่ำสุดเพื่อทะลุบอลลูน (Minimum Arrows to Burst Balloons)

นิยาม 10.7 (Minimum Arrows to Burst Balloons)**.** กำหนดบอลลูน n ลูก บอลลูนลูกที่ i ครอบคลุมช่วงแนวนอน $[x_{start_i}, x_{end_i}]$ ลูกธนู 1 ดอกที่ยิงในแนวนอนตำแหน่ง x จะทะลุบอลลูนทุกลูกที่ช่วงครอบคลุมตำแหน่ง x จงหาจำนวนลูกธนูต่ำสุดที่ทะลุบอลลูนทั้งหมด

กลยุทธ์ Greedy: เรียงบอลลูนตามจุดสิ้นสุด x_{end} จากน้อยไปมาก แล้วยิงธนูที่จุดสิ้นสุดของบอลลูนแรกที่ยังไม่ถูกทะลุ ธนูดอกนี้จะทะลุบอลลูนทุกลูกที่ครอบคลุมตำแหน่งนั้น

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> balloons(n);   // {end, start}
    for (int i = 0; i < n; i++) {
        int s, e;
        cin >> s >> e;
        balloons[i] = {e, s};
    }
    sort(balloons.begin(), balloons.end());
    int arrows = 0;
    int arrowPos = INT_MIN;
    for (auto& [end, start] : balloons) {
        if (start > arrowPos) {
            // This balloon not burst by current arrow -> shoot new arrow
            arrows++;
            arrowPos = end;   // Shoot at the end of this balloon
        }
    }
    cout << arrows << "\n";
    return 0;
```

```
}
```

ตัวอย่างโค้ดที่ 10.8: Minimum Arrows to Burst Balloons

ตัวอย่าง

บอลลูน 5 ลูก ครอบคลุมช่วง: [1,6], [2,8], [7,12], [10,16], [14,18]

1. เรียงตามจุดสิ้นสุด: [1,6], [2,8], [7,12], [10,16], [14,18]
2. ยิงธนูที่ $x = 6$: ทะลุ [1,6] และ [2,8] (เพราะ 6 อยู่ในช่วง [2,8] ด้วย)
3. บอลลูน [7,12]: ไม่ถูกทะลุ (7 > 6) ⊠ ยิงธนูที่ $x = 12$: ทะลุ [7,12] และ [10,16]
4. บอลลูน [14,18]: ไม่ถูกทะลุ (14 > 12) ⊠ ยิงธนูที่ $x = 18$

คำตอบ: 3 ดอก

10.6.3 การรวมช่วงเวลา (Merging Intervals)

นิยาม 10.8 (Merge Intervals). กำหนดช่วงเวลา n ช่วง ช่วงที่ทับซ้อนกันให้รวมเป็นช่วงเดียว จงหาช่วงเวลาที่ไม่ทับซ้อนกันทั้งหมดหลังรวม

กลยุทธ์ Greedy: เรียงช่วงตามจุดเริ่มต้น จากน้อยไปมาก แล้ววนตรวจสอบทีละช่วง ถ้าช่วงปัจจุบันทับซ้อนกับช่วงสุดท้ายในผลลัพธ์ ให้รวม ถ้าไม่ทับซ้อน ให้เพิ่มเข้าผลลัพธ์

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<pair<int,int>> intervals(n);
    for (int i = 0; i < n; i++) {
        cin >> intervals[i].first >> intervals[i].second;
    }
    // Sort by starting point
    sort(intervals.begin(), intervals.end());
    vector<pair<int,int>> merged;
    for (auto& [s, e] : intervals) {
        if (merged.empty() || s > merged.back().second) {
            // Non-overlapping -> add new interval
            merged.push_back({s, e});
        } else {
            // Overlapping -> merge (extend end point)
            merged.back().second = max(merged.back().second, e);
```

```
        }
    }
    cout << merged.size() << "\n";
    for (auto& [s, e] : merged) {
        cout << s << " " << e << "\n";
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.9: Merge Intervals

ตัวอย่าง

ช่วงเวลา: [1,3], [2,6], [8,10], [15,18]

1,3 : ผลลัพธ์ = [1,3]

2,6 : 2 ≤ 3 (ทับซ้อน) ⊠ รวมเป็น [1,6]

8,10 : 8 > 6 (ไม่ทับซ้อน) ⊠ เพิ่ม ผลลัพธ์ = [1,6], [8,10]

15,18 : 15 > 10 (ไม่ทับซ้อน) ⊠ เพิ่ม ผลลัพธ์ = [1,6], [8,10], [15,18]

คำตอบ: 3 ช่วง — [1,6], [8,10], [15,18]

10.6.4 การปกคลุมช่วงเวลา (Interval Covering / Point Covering)

นิยาม 10.9 (Interval Covering). กำหนดช่วงเป้าหมาย $[L, R]$ และช่วงย่อย n ช่วง ช่วงย่อยแต่ละช่วงครอบคลุม $[a_i, b_i]$ จงหาจำนวนช่วงย่อยที่น้อยที่สุดที่รวมกันครอบคลุม $[L, R]$ ทั้งหมด

กลยุทธ์ Greedy: เรียงช่วงย่อยตามจุดเริ่มต้น ในแต่ละขั้น เลือกช่วงที่เริ่มก่อนหรือเท่ากับจุดปัจจุบัน และยืดไปได้ไกลที่สุด

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    double L, R;
    cin >> n >> L >> R;
    vector<pair<double,double>> segs(n);
    for (int i = 0; i < n; i++) {
        cin >> segs[i].first >> segs[i].second;
    }
    // Sort by starting point
    sort(segs.begin(), segs.end());
```

```
    int count = 0;
    double current = L;
    int i = 0;
    while (current < R && i < n) {
        // Find interval that starts <= current and extends farthest
        double bestEnd = current;
        while (i < n && segs[i].first <= current) {
            bestEnd = max(bestEnd, segs[i].second);
            i++;
        }
        if (bestEnd == current) {
            // No interval covers current -> cannot cover all
            cout << -1 << "\n";
            return 0;
        }
        count++;
        current = bestEnd;
    }
    if (current >= R) {
        cout << count << "\n";
    } else {
        cout << -1 << "\n";   // Cannot cover to R
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.10: Interval Covering — หาจำนวนช่วงน้อยสุดที่ครอบคลุม

สรุปปัญหา Interval

ปัญหา	กลยุทธ์ Greedy	เรียงตาม
กิจกรรมไม่ทับซ้อนสูงสุด	เลือกจบเร็วสุดก่อน	เวลาจบ (end)
ธนูทะลุบอลลูนน้อยสุด	ยิงที่จุดจบ	เวลาจบ (end)
รวมช่วงทับซ้อน	รวมเมื่อทับซ้อน	เวลาเริ่ม (start)
ครอบคลุมช่วง	เลือกยืดไกลสุด	เวลาเริ่ม (start)

10.7 Greedy on Graphs

ในบทก่อนหน้า เราได้ศึกษาขั้นตอนวิธีกราฟหลายตัวที่แท้จริงแล้วเป็น Greedy ทั้งสิ้น ในหัวข้อนี้จะทบทวนและเชื่อมโยงความรู้

10.7.1 Dijkstra: Greedy บน Shortest Path

ขั้นตอนวิธี Dijkstra (บทที่ 12) เป็น Greedy Algorithm อย่างแท้จริง ในแต่ละขั้น Dijkstra เลือก **จุดยอดที่มีระยะทางน้อยที่สุด** จากจุดเริ่มต้น (ที่ยังไม่ได้ประมวลผล) ออกมาประมวลผล โดยเชื่อมั่นว่าระยะทางที่คำนวณได้ในขณะนั้นคือระยะทางที่สั้นที่สุดแล้ว (ไม่มีทางที่จะสั้นลงไปอีก) — นี่คือ Greedy Choice Property อย่างชัดเจน

Dijkstra ใช้ได้เฉพาะกราฟที่ **ไม่มีน้ำหนักติดลบ** เพราะถ้ามีน้ำหนักติดลบ การเลือกที่ดีที่สุดในขณะนั้นอาจไม่ใช่ทางที่ดีที่สุดในอนาคต (เส้นเชื่อมติดลบอาจทำให้เส้นทางอื่นสั้นกว่าที่คิด) — นี่คือตัวอย่างที่ดีของกรณีที่ Greedy ใช้ไม่ได้เมื่อเงื่อนไขไม่เหมาะสม

10.7.2 Kruskal's และ Prim's: Greedy บน MST

ขั้นตอนวิธี Kruskal's (บทที่ 13) ใช้กลยุทธ์ Greedy โดย **เลือกเส้นเชื่อมที่มีน้ำหนักน้อยที่สุด** ในแต่ละขั้นโดยไม่ทำให้เกิดวงจร (ตรวจสอบด้วย Union-Find) การเลือกเส้นน้ำหนักน้อยสุดก่อนเป็น Greedy Choice ที่ถูกต้อง เพราะถ้าเส้นน้ำหนักน้อยสุดไม่อยู่ใน MST เราสามารถแทนที่เส้นอื่นใน MST ด้วยเส้นนี้ได้ (Exchange Argument)

ขั้นตอนวิธี Prim's ก็ใช้ Greedy เช่นกัน โดยในแต่ละขั้น **เลือกเส้นเชื่อมที่น้ำหนักน้อยที่สุดที่เชื่อมระหว่างส่วนที่เลือกแล้วกับส่วนที่ยังไม่เลือก** คล้ายกับการขยายเกาะทีละเส้น

Greedy ซ่อนอยู่ในกราฟ

หลายคนไม่รู้ว่า Dijkstra, Kruskal's, Prim's เป็น Greedy Algorithm การเข้าใจว่าเป็น Greedy ช่วยให้เห็นรูปแบบร่วมกัน: **เลือกสิ่งที่ดีที่สุดในเวลานั้น แล้วพิสูจน์ว่าการเลือกนั้นไม่ทำให้เสียโอกาสในอนาคต** นี่คือหัวใจของ Greedy

10.7.3 ช่วงเวลาไม่ทับซ้อนสูงสุด (กราฟช่วงเวลา)

ปัญหาช่วงเวลาไม่ทับซ้อนสูงสุด (Maximum Disjoint Intervals) สามารถมองเป็นปัญหากราฟได้: สร้างกราฟที่แต่ละช่วงเวลาเป็นจุดยอด และมีเส้นเชื่อมระหว่างช่วงที่ทับซ้อนกัน ปัญหาจะเทียบเท่ากับการหา Independent Set ที่ใหญ่ที่สุด ซึ่งโดยทั่วไปเป็นปัญหา NP-Hard แต่เนื่องจากกราฟช่วงเวลา (Interval Graph) มีโครงสร้างพิเศษ Greedy จึงใช้ได้

นี่เป็นตัวอย่างสำคัญ: Greedy อาจใช้ไม่ได้สำหรับปัญหาทั่วไป แต่ใช้ได้เมื่อข้อมูลมีโครงสร้างพิเศษ

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    // Store {end, start, original_index}
    vector<tuple<int,int,int>> intervals(n);
    for (int i = 0; i < n; i++) {
        int s, e;
```

```
        cin >> s >> e;
        intervals[i] = {e, s, i + 1};
    }
    sort(intervals.begin(), intervals.end());
    vector<int> selected;
    int lastEnd = INT_MIN;
    for (auto& [e, s, idx] : intervals) {
        if (s > lastEnd) {
            selected.push_back(idx);
            lastEnd = e;
        }
    }
    cout << selected.size() << "\n";
    for (int x : selected) cout << x << " ";
    cout << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.11: Maximum Disjoint Intervals — เวอร์ชันคืนดัชนี

10.8 การพิสูจน์ความถูกต้อง (Proving Greedy Correctness)

หัวใจสำคัญของการใช้ Greedy คือการพิสูจน์ว่ากลยุทธ์ที่เลือกใช้นั้นถูกต้อง[4] ในการแข่งขัน ถ้าพิสูจน์ไม่ได้ ก็ต้องลองหา Counter-example ให้แน่ใจก่อน ในหัวข้อนี้เราจะศึกษาเทคนิคพิสูจน์ 2 วิธีหลัก และตัวอย่างของ Greedy ที่ผิด

10.8.1 Exchange Argument (การสลับ)

เราได้เห็น Exchange Argument ในหัวข้อ Activity Selection แล้ว มาสรุปเป็นขั้นตอนทั่วไป:

1. **สมมติ** มีคำตอบที่เหมาะที่สุด O^* ที่ต่างจากคำตอบของ Greedy G
2. **ระบุจุดแรก** ที่ O^* ต่างจาก G (เช่น ตัวเลือกแรกที่ต่างกัน)
3. **สลับ** ตัวเลือกนั้นใน O^* มาเป็นตัวเลือกของ G
4. **แสดงว่า** หลังสลับแล้ว O^* ยังเป็นคำตอบที่ถูกต้อง และไม่แย่ลง
5. **สรุป** โดย Induction ว่า G ไม่แย่กว่าคำตอบที่ดีที่สุด ดังนั้น G เป็นคำตอบที่เหมาะที่สุด

ตัวอย่าง: พิสูจน์ Shortest Job First

อ้าง: เรียงงานจากเวลาน้อยไปมาก ให้ผลรวม Completion Time น้อยที่สุด
พิสูจน์: สมมติคำตอบที่เหมาะที่สุด O^* ไม่ได้เรียงตามเวลา แสดงว่ามีงาน i อยู่ก่อนงาน j แต่ $t_i > t_j$ (งานยาวกว่ามาก่อนงานสั้น)
ผลรวม Completion Time ส่วนที่เกี่ยวข้องกับงาน i และ j: $C_i + C_j = (\text{prefix} + t_i) + (\text{prefix} + t_i + t_j)$
ถ้าสลับให้ j มาก่อน i: $C'_j + C'_i = (\text{prefix} + t_j) + (\text{prefix} + t_j + t_i)$
ผลต่าง: $(C_i + C_j) - (C'_j + C'_i) = t_i - t_j > 0$ (เพราะ $t_i > t_j$)
ดังนั้นการสลับทำให้ผลรวมลดลง ขัดแย้งกับที่ O^* เป็นคำตอบที่ดีที่สุด □

10.8.2 Stays-Ahead Method (วิธีนำหน้า)

วิธี Stays-Ahead เปรียบเทียบ Greedy กับคำตอบที่เหมาะที่สุดทีละขั้น โดยแสดงว่า Greedy **ไม่เคยตามหลัง** คำตอบที่ดีที่สุด

1. กำหนดฟังก์ชันวัดคุณภาพ $f(k)$ สำหรับขั้นที่ k
2. แสดงว่าสำหรับทุก k: $f(G_k) \geq f(O_k^*)$ (Greedy นำหน้าเสมอ)
3. สรุปว่าเมื่อจบกระบวนการ Greedy ไม่แย่กว่า Optimal

ตัวอย่าง: พิสูจน์ Activity Selection ด้วย Stays-Ahead

ให้ $G_1, G_2, \ldots, G_m$ เป็นกิจกรรมที่ Greedy เลือก (เรียงตามเวลาจบ) และ $O_1, O_2, \ldots, O_k$ เป็นกิจกรรมที่ Optimal เลือก
แสดงว่า: $f(G_i) \leq f(O_i)$ สำหรับทุก i (กิจกรรมที่ i ของ Greedy จบก่อนหรือพร้อมกับกิจกรรมที่ i ของ Optimal)
Base case: G_1 จบเร็วที่สุด ดังนั้น $f(G_1) \leq f(O_1)$
Inductive step: ถ้า $f(G_i) \leq f(O_i)$ แล้ว G_{i+1} เป็นกิจกรรมแรกที่เริ่มหลัง $f(G_i)$ ซึ่งเริ่มหลัง $f(O_i)$ ด้วย (เพราะ $f(G_i) \leq f(O_i)$) Greedy เลือกกิจกรรมที่จบเร็วที่สุดจากเซตนี้ ดังนั้น $f(G_{i+1}) \leq f(O_{i+1})$
ดังนั้น $m \geq k$ □

10.8.3 ตัวอย่าง Greedy ที่ผิด

สิ่งสำคัญที่สุดในการเรียนรู้ Greedy คือการรู้จัก **Greedy ที่ดูถูกต้องแต่ผิด** ต่อไปนี้คือตัวอย่างที่พบบ่อย:

ตัวอย่างที่ 1: 0-1 Knapsack ด้วยอัตราส่วน

Greedy ผิด: 0-1 Knapsack

กระเป๋า W=10 สินค้า:

	น้ำหนัก	มูลค่า	อัตราส่วน
สินค้า 1	10	60	6.0
สินค้า 2	4	28	7.0
สินค้า 3	4	28	7.0

Greedy (เลือกตามอัตราส่วน): เลือก 2+3 = 56 แต่ Optimal: เลือก 1 = 60

ตัวอย่างที่ 2: Activity Selection เลือกสั้นสุดก่อน

Greedy ผิด: เลือกกิจกรรมสั้นสุดก่อน

กิจกรรม:

	เริ่ม	จบ
A	0	10
B	1	3
C	4	6
D	7	9

Greedy (เลือกสั้นสุดก่อน): B, C, D = 3 กิจกรรม ✓
แต่ลองเปลี่ยนเป็น:

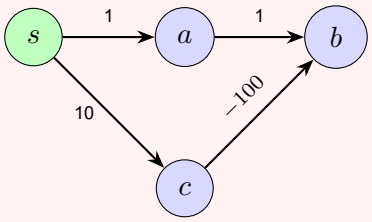
	เริ่ม	จบ
A	0	10
B	2	5
C	3	8

Greedy (สั้นสุดก่อน): B (ยาว 3) แล้วไม่เลือกอะไรได้อีก = 1 กิจกรรม
Optimal: A = 1 กิจกรรม (เท่ากัน แต่ในกรณีอื่นอาจต่างกันมาก)

ตัวอย่างที่ 3: Shortest Path บนกราฟน้ำหนักติดลบ

Greedy ผิด: Dijkstra กับน้ำหนักติดลบ

พิจารณากราฟในรูปที่ 10.3



รูปที่ 10.3 ตัวอย่างกราฟน้ำหนักติดลบที่ Dijkstra ล้มเหลว: เส้นทาง $s \to c \to b$ (รวม $10 + (-100) = -90$) สั้นกว่าเส้นทาง $s \to a \to b$ (รวม 2) ที่ Dijkstra เลือก

Dijkstra: เลือก s (dist=0) แล้ว a (dist=1) แล้ว b (dist=1+1=2) แล้ว c (dist=10)
แต่เส้นทางที่สั้นกว่า: $s \to c \to b = 10 + (-100) = -90$
Dijkstra ไม่พบเส้นทางนี้เพราะ "ปิด" b ไปแล้ว (dist=2) ก่อนจะประมวลผล c! $\to$ **ผิด!**

กฎทองของ Greedy

ถ้าไม่แน่ใจว่า Greedy ถูก ให้หา Counter-example ก่อน! ลองสร้างข้อมูลทดสอบเล็กๆ ที่อาจทำให้ Greedy ผิด ถ้าหาไม่ได้ และมีเหตุผลสนับสนุน ค่อยใช้ Greedy ในการแข่งขันจริง หากเวลาจำกัด และไม่แน่ใจ อาจลองส่ง Greedy ดูก่อน ถ้า Wrong Answer ค่อยคิดวิธีอื่น

10.9 ตัวอย่างโจทย์เพิ่มเติม

ในหัวข้อนี้เราจะศึกษาโจทย์ Greedy เพิ่มเติมที่พบบ่อยในการแข่งขัน เพื่อเสริมสร้างความเข้าใจและทักษะ

10.9.1 ปัญหาการจัดสรรทรัพยากร (Resource Allocation)

โจทย์: มีงาน n งาน งานที่ i ต้องใช้ทรัพยากร r_i หน่วย และมีทรัพยากรทั้งหมด R หน่วย จงหาจำนวนงานที่มากที่สุดที่สามารถทำได้

กลยุทธ์ Greedy: เรียงงานตามทรัพยากรที่ต้องการจากน้อยไปมาก แล้วเลือกทำงานทีละอันจนทรัพยากรไม่พอ

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    long long R;
    cin >> n >> R;
```

```
    vector<long long> r(n);
    for (int i = 0; i < n; i++) {
        cin >> r[i];
    }
    // Sort in ascending order
    sort(r.begin(), r.end());
    int count = 0;
    long long used = 0;
    for (int i = 0; i < n; i++) {
        if (used + r[i] <= R) {
            used += r[i];
            count++;
        } else {
            break;
        }
    }
    cout << count << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.12: Resource Allocation — หาจำนวนงานมากสุด

10.9.2 ปัญหา Minimizing the Sum of Products

โจทย์: มีแถวลำดับ A และ B ความยาว n ต้องการจับคู่สมาชิกของ A กับ B (Permutation ของ B) เพื่อให้ $\sum_{i=1}^{n} A_i \times B_{\pi(i)}$ มีค่าน้อยที่สุด

กลยุทธ์ Greedy: เรียง A จากน้อยไปมาก เรียง B จากมากไปน้อย แล้วคูณตำแหน่งเดียวกัน

ทฤษฎีบท 10.7 (Rearrangement Inequality). $\sum_{i=1}^{n} A_i \times B_i$ มีค่าน้อยที่สุด เมื่อ A เรียงจากน้อยไปมาก และ B เรียงจากมากไปน้อย

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<long long> A(n), B(n);
    for (int i = 0; i < n; i++) cin >> A[i];
    for (int i = 0; i < n; i++) cin >> B[i];
    // Sort A in ascending order
    sort(A.begin(), A.end());
    // Sort B in descending order
```

```
    sort(B.begin(), B.end(), greater<long long>());
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += A[i] * B[i];
    }
    cout << sum << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.13: Minimizing Sum of Products

ตัวอย่าง

$A = \{3, 1, 2\}$, $B = \{4, 5, 6\}$

เรียง A: {1, 2, 3}, เรียง B ย้อน: {6, 5, 4}

sum $= 1 \times 6 + 2 \times 5 + 3 \times 4 = 6 + 10 + 12 = 28$

ลองจับคู่อื่น: $3 \times 4 + 1 \times 5 + 2 \times 6 = 12 + 5 + 12 = 29 > 28$ ✓

10.9.3 Jump Game — กระโดดให้ถึงท้ายสุด

โจทย์: มีแถวลำดับ nums ที่ nums[i] แทนจำนวนก้าวสูงสุดที่กระโดดได้จากตำแหน่ง i จงหาจำนวนกระโดด **น้อยที่สุด** เพื่อไปถึงตำแหน่งสุดท้าย

กลยุทธ์ Greedy: ในแต่ละครั้ง กระโดดไปยังตำแหน่งที่ทำให้ไปได้ไกลที่สุดในขั้นถัดไป (Greedy BFS style)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> nums(n);
    for (int i = 0; i < n; i++) cin >> nums[i];
    if (n <= 1) {
        cout << 0 << "\n";
        return 0;
    }
    int jumps = 0;
    int currentEnd = 0;    // Current jump boundary
    int farthest = 0;      // Farthest reachable position
    for (int i = 0; i < n - 1; i++) {
        farthest = max(farthest, i + nums[i]);
        if (i == currentEnd) {
            // Reached boundary -> must jump
```

```
            jumps++;
            currentEnd = farthest;
            if (currentEnd >= n - 1) break;
        }
    }
    cout << jumps << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.14: Jump Game II — กระโดดน้อยที่สุด

ตัวอย่าง

nums = {2, 3, 1, 1, 4}

- ตำแหน่ง 0: ไปได้ไกลสุดถึง 0+2=2, currentEnd=0 → กระโดด, currentEnd=2
- ตำแหน่ง 1: ไปได้ไกลสุดถึง 1+3=4, farthest=4
- ตำแหน่ง 2: ไปได้ไกลสุดถึง 2+1=3, farthest=4, i==currentEnd → กระโดด, currentEnd=4 ≥ 4 → จบ

คำตอบ: 2 ครั้ง (0→1→4)

10.9.4 Candy Distribution — แจกลูกกวาด

โจทย์: มีเด็ก n คนเรียงแถว เด็กแต่ละคนมีคะแนน ratings[i] ต้องแจกลูกกวาดให้:

1. เด็กที่มีคะแนนมากกว่าจุดข้างเคียง ต้องได้ลูกกวาดมากกว่าจุดข้างเคียง
2. เด็กทุกคนต้องได้อย่างน้อย 1 ลูก

จงหาจำนวนลูกกวาดที่น้อยที่สุด

กลยุทธ์: วน 2 ทิศทาง (ซ้าย→ขวา และ ขวา→ซ้าย)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> ratings(n);
    for (int i = 0; i < n; i++) cin >> ratings[i];
    vector<int> candy(n, 1);   // Start everyone with 1 candy
    // Left to right: if ratings[i] > ratings[i-1]
    // Must get more than left neighbor
```

```
    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i-1]) {
            candy[i] = candy[i-1] + 1;
        }
    }
    // Right to left: if ratings[i] > ratings[i+1]
    // Must get more than right neighbor
    for (int i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i+1]) {
            candy[i] = max(candy[i], candy[i+1] + 1);
        }
    }
    long long total = 0;
    for (int c : candy) total += c;
    cout << total << "\n";
    return 0;
}
```

ตัวอย่างโค้ดที่ 10.15: Candy Distribution

ความซับซ้อน: $\mathcal{O}(n)$ (วน 2 รอบ รอบละ $\mathcal{O}(n)$)

Candy Distribution ไม่ใช่ Greedy แบบดั้งเดิม

Candy Distribution ใช้กลยุทธ์ที่เรียกว่า **Two-Pass Greedy** — วนสองทิศทางเพื่อให้เงื่อนไขทั้งสองข้างเป็นจริงพร้อมกัน เทคนิคนี้พบบ่อยในโจทย์ที่มีเงื่อนไข “จุดข้างเคียงทั้งสองข้าง” เช่น Trapping Rain Water

10.10 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ฝึกหัด Greedy จะแบ่งตามระดับความยาก:

ตารางที่ 10.1 โจทย์ฝึกหัด Greedy

โจทย์	แหล่งที่มา	ระดับ
Activity Selection	CSES Movie Festival	เริ่มต้น
Fractional Knapsack	CSES Stick Lengths (ประยุกต์)	เริ่มต้น
Merge Intervals	LeetCode 56	เริ่มต้น
Min Arrows	LeetCode 452	เริ่มต้น
Jump Game	LeetCode 55, 45	ปานกลาง
Candy	LeetCode 135	ปานกลาง
Huffman Coding	SPOJ HC	ปานกลาง
Shortest Job First	Codeforces 1191A (ประยุกต์)[10]	ปานกลาง
Interval Cover	UVa 10382[20]	ปานกลาง
Task Scheduling	CSES Tasks and Deadlines	ปานกลาง
Rearrangement	Codeforces 1092B[10]	ปานกลาง
Boat Saving	Codeforces 1385C[10]	ยาก
Greedy + Sorting	AtCoder ABC 273 D	ยาก
Min Cost Flow (Greedy)	CSES Reading Books	ยาก
Complex Greedy	Codeforces 1498C[10]	ยากมาก

คำแนะนำในการฝึก

1. เริ่มจากโจทย์เริ่มต้นก่อน ให้เข้าใจแนวคิด Greedy แบบตรงไปตรงมา
2. สำหรับทุกโจทย์ ให้ **พยายามพิสูจน์** ว่า Greedy ที่ใช้ถูกต้อง ไม่ใช่แค่ส่งแล้วผ่าน
3. โจทย์ระดับยากมักผสม Greedy กับเทคนิคอื่น เช่น Binary Search, Two Pointers, หรือ DP
4. ถ้าส่งผิด (Wrong Answer) ให้ลองหา Counter-example ก่อนเปลี่ยนวิธี — บางครั้งแค่ปรับเงื่อนไขเล็กน้อยก็ผ่าน
5. อย่าท้อ! Greedy เป็นหัวข้อที่ต้องฝึกเยอะๆ จะคุ้นเคยกับรูปแบบโจทย์

10.11 สรุปบทที่ 10

10.11.1 ตารางสรุปปัญหา Greedy

ตารางที่ 10.2 สรุปปัญหา Greedy ที่สำคัญ

ปัญหา	กลยุทธ์ Greedy	หลักการ	ความซับซ้อน
Activity Selection	จบเร็วสุดก่อน	เลือกที่เปิดโอกาสให้อื่นมากสุด	$\mathcal{O}(n \log n)$
Shortest Job First	เวลาน้อยไปมาก	ลดเวลารอของงานอื่น	$\mathcal{O}(n \log n)$
Earliest Deadline	Deadline น้อยไปมาก	ลด Lateness สูงสุด	$\mathcal{O}(n \log n)$
Huffman Coding	รวมความถี่ต่ำสุด	อักขระหายากได้รหัสยาว	$\mathcal{O}(n \log n)$
Fractional Knapsack	อัตราส่วนสูงก่อน	เลือกสินค้าคุ้มสุดก่อน	$\mathcal{O}(n \log n)$
Merge Intervals	เรียงตามจุดเริ่ม	รวมช่วงทับซ้อน	$\mathcal{O}(n \log n)$
Min Arrows	จบเร็วสุดก่อน	ยิงที่จุดจบของกลุ่ม	$\mathcal{O}(n \log n)$
Interval Cover	เริ่มน้อยไปมาก	เลือกยืดไกลสุด	$\mathcal{O}(n \log n)$
Dijkstra	ระยะน้อยสุดก่อน	ไม่มีเส้นติดลบ	$\mathcal{O}((V+E) \log V)$
Kruskal's	น้ำหนักน้อยสุดก่อน	ไม่ทำให้เกิดวงจร	$\mathcal{O}(E \log V)$

10.11.2 เทคนิคจดจำปัญหา Greedy

จะรู้ได้อย่างไรว่าโจทย์น่าจะใช้ Greedy ได้? สังเกตจากลักษณะเหล่านี้:

1. **โจทย์ขอหาค่าต่ำสุด/สูงสุด** ของปริมาณที่วัดได้ (เช่น จำนวนกิจกรรม, เวลารวม, จำนวนเหรียญ)
2. **มีลำดับที่เป็นธรรมชาติ** เช่น เวลา, น้ำหนัก, ราคา — สามารถเรียงลำดับได้
3. **การตัดสินใจในอดีตไม่กระทบอนาคต** หรือกระทบในทิศทางที่ควบคุมได้
4. **ไม่มี "trade-off" ที่ซับซ้อน** คือไม่มีกรณีที่เลือกอย่างหนึ่งแล้วเสียอย่างอื่นอย่างผูกพัน
5. **Dynamic Programming ใช้ไม่ได้หรือซับซ้อนเกินไป** เช่น ค่า n ใหญ่มาก ทำให้ DP ไม่ทันเวลา

10.11.3 ข้อผิดพลาดที่พบบ่อย

ข้อผิดพลาดยอดนิยมในการใช้ Greedy

1. **ใช้ Greedy โดยไม่พิสูจน์** — ดูคร่าวๆ แล้วคิดว่าถูก แต่จริงๆ ผิด ต้องหา Counter-example ก่อนเสมอ
2. **สับสนระหว่างเรียงตามจุดเริ่มกับจุดจบ** — Activity Selection เรียงตาม **จุดจบ** ไม่ใช่จุดเริ่ม! สับสนบ่อยมาก

3. **สับสน Fractional กับ 0-1** — Fractional Knapsack ใช้ Greedy ได้ แต่ 0-1 Knapsack ใช้ไม่ได้
4. **ลืมจัดการกรณีพิเศษ** — เช่น ช่วงเวลาที่ขอบทับซ้อนกันพอดี (s=e), อักขระเดียวใน Huffman
5. **เปรียบเทียบจุดทศนิยมด้วย ==** — ใช้ EPS เสมอในการเปรียบเทียบ
6. **Overflow** — การคำนวณผลรวมอาจเกิน `int` ใช้ `long long` เสมอ
7. **ไม่สังเกตโครงสร้างพิเศษ** — เช่น กราฟเป็น DAG, Interval Graph, ทำให้ Greedy ใช้ได้

10.11.4 วิธีคิด Greedy เป็นขั้นตอน

เมื่อเจอโจทย์ที่น่าจะเป็น Greedy ให้ทำตามขั้นตอนนี้:

1. **ทำความเข้าใจโจทย์** — อ่านให้ละเอียด เข้าใจ Input/Output และเงื่อนไข
2. **คิดกลยุทธ์ Greedy ที่เป็นไปได้** — มักมีหลายวิธี เช่น เรียงตามค่า A หรือค่า B
3. **ทดสอบด้วยตัวอย่างเล็ก** — ลองสร้างข้อมูลนำเข้าเล็กๆ แล้วตรวจสอบด้วยมือ
4. **หา Counter-example** — พยายามสร้างกรณีที่ Greedy อาจผิด
5. **พิสูจน์ความถูกต้อง** — ใช้ Exchange Argument หรือ Stays-Ahead (แม้ไม่เขียนออกมาเป็นทางการ แต่คิดในใจ)
6. **เขียนโค้ด** — ระวังรายละเอียด เช่น ประเภทข้อมูล, เงื่อนไขขอบ, การเรียงลำดับ
7. **ทดสอบ** — ลองรันกับ Test Case ที่ครอบคลุมกรณีต่างๆ

สรุปบทที่ 10

- **Greedy Algorithm** คือขั้นตอนวิธีที่เลือกตัวเลือกที่ดีที่สุดในเวลานั้น โดยหวังว่าจะนำไปสู่คำตอบที่ดีที่สุดโดยรวม
- Greedy ใช้ได้เมื่อปัญหามี **Greedy Choice Property** และ **Optimal Substructure** ทั้งคู่
- ปัญหาคลาสสิก: Activity Selection (เรียงตามเวลาจบ), Shortest Job First, Earliest Deadline First, Huffman Coding, Fractional Knapsack, Interval Problems
- วิธีพิสูจน์: **Exchange Argument** (สลับตัวเลือก) และ **Stays-Ahead** (นำหน้าเสมอ)
- ข้อผิดพลาดที่พบบ่อย: ใช้ Greedy โดยไม่พิสูจน์, สับสน 0-1 vs Fractional Knapsack, สับสนจุดเริ่ม vs จุดจบ
- กฎทอง: **ถ้าไม่แน่ใจ ให้หา Counter-example ก่อน!**

6

ชุดปัญหาด้านกราฟ

ส่วนนี้ครอบคลุมทฤษฎีกราฟและขั้นตอนวิธีกราฟ ตั้งแต่พื้นฐาน DFS/BFS การหาองค์ประกอบเชื่อมโยง การเรียงลำดับทอพอโลยี ไปจนถึงขั้นตอนวิธีเส้นทางสั้นสุด Dijkstra, Bellman-Ford, Floyd-Warshall, Union-Find, Minimum Spanning Tree และหัวข้อกราฟขั้นสูงเช่น SCC, Articulation Points และ 2-SAT

Chapter 11

ทฤษฎีกราฟ

กราฟ (Graph) เป็นหนึ่งในโครงสร้างข้อมูลที่ทรงพลังและใช้กันแพร่หลายที่สุดในการโปรแกรมเชิงแข่งขัน กราฟสามารถจำลองปัญหาได้หลากหลายรูปแบบ ตั้งแต่เครือข่ายถนน ความสัมพันธ์ระหว่างบุคคล ไปจนถึงระบบการจราจร ในบทนี้ เราจะศึกษาพื้นฐานของกราฟ วิธีการแทนกราฟในโปรแกรม และขั้นตอนวิธีการสำรวจกราฟที่สำคัญที่สุดสองตัว ได้แก่ Depth-First Search (DFS) และ Breadth-First Search (BFS) รวมถึงการประยุกต์ใช้ DFS/BFS ในการหาองค์ประกอบเชื่อมโยง Flood Fill และการเรียงลำดับทอพอโลยี ซึ่งเป็นพื้นฐานสำคัญสำหรับขั้นตอนวิธีกราฟขั้นสูงในบทถัดไป[2, 3]

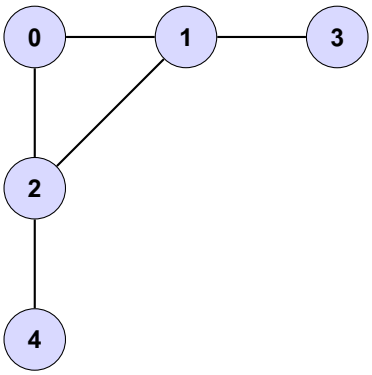
11.1 แนะนำกราฟ

กราฟเป็นโครงสร้างทางคณิตศาสตร์ที่ใช้จำลองความสัมพันธ์ระหว่างวัตถุ ในชีวิตจริง เราสามารถมองเครือข่ายถนนเป็นกราฟที่เมืองต่างๆ เป็นจุด (Vertex) และถนนเชื่อมระหว่างเมืองเป็นเส้นเชื่อม (Edge) ในการโปรแกรมเชิงแข่งขัน กราฟปรากฏในโจทย์หลากหลายรูปแบบ ทั้งที่ระบุชัดเจนว่าเป็นกราฟ และที่ต้อง “มองเห็น” เองว่าโจทย์สามารถจำลองเป็นกราฟได้

11.1.1 กราฟคืออะไร

นิยาม 11.1 (กราฟ)**.** กราฟ $G = (V, E)$ ประกอบด้วยเซตของ **จุดยอด** (Vertex/Node) V และเซตของ **เส้นเชื่อม** (Edge) E โดยแต่ละเส้นเชื่อมเชื่อมระหว่างจุดยอดสองจุด เรามักใช้ $n = |V|$ แทนจำนวนจุดยอด และ $m = |E|$ แทนจำนวนเส้นเชื่อม

ตัวอย่างเช่น กราฟที่มีจุดยอด $V = \{0, 1, 2, 3, 4\}$ และเส้นเชื่อม $E = \{(0, 1), (0, 2), (1, 2), (1, 3), (2, 4)\}$ แสดงภาพได้ดังรูปที่ 11.1



รูปที่ 11.1 ตัวอย่างกราฟไม่มีทิศทาง: $V = \{0, 1, 2, 3, 4\}$, $E = \{(0, 1), (0, 2), (1, 2), (1, 3), (2, 4)\}$

ในกราฟดังกล่าว $n = 5$ จุดยอด และ $m = 5$ เส้นเชื่อม จุดยอดมักถูกกำหนดหมายเลขตั้งแต่ 0 ถึง $n - 1$ หรือ 1 ถึง n ขึ้นอยู่กับโจทย์

11.1.2 เส้นทาง วงจร และเส้นทางอย่างง่าย

นิยาม 11.2 (เส้นทาง (Path)). เส้นทางจากจุดยอด a ไปจุดยอด b คือลำดับของจุดยอด $v_0, v_1, \ldots, v_k$ ที่ $v_0 = a$, $v_k = b$ และมีเส้นเชื่อมระหว่าง v_{i-1} กับ v_i สำหรับทุก $i = 1, \ldots, k$ ความยาวของเส้นทางคือจำนวนเส้นเชื่อมที่ผ่าน ซึ่งเท่ากับ k

ตัวอย่างเช่น ในกราฟข้างต้น เส้นทางจาก 0 ถึง 4 อาจเป็น $0 \to 2 \to 4$ (ความยาว 2) หรือ $0 \to 1 \to 2 \to 4$ (ความยาว 3)

นิยาม 11.3 (วงจร (Cycle)). วงจรคือเส้นทางที่จุดเริ่มต้นและจุดสิ้นสุดเป็นจุดเดียวกัน กล่าวคือ $v_0 = v_k$ และ $k \geq 1$

ตัวอย่างเช่น $0 \to 1 \to 2 \to 0$ เป็นวงจรในกราฟข้างต้น

นิยาม 11.4 (เส้นทางอย่างง่าย (Simple Path)). เส้นทางอย่างง่ายคือเส้นทางที่ไม่มีจุดยอดซ้ำกัน กล่าวคือจุดยอดทุกจุดในเส้นทางแตกต่างกันหมด

เส้นทาง $0 \to 2 \to 4$ เป็นเส้นทางอย่างง่าย (ไม่มีจุดซ้ำ) แต่ $0 \to 1 \to 2 \to 0 \to 2 \to 4$ ไม่ใช่เส้นทางอย่างง่าย เพราะจุด 0 และ 2 ปรากฏซ้ำ

11.1.3 กราฟเชื่อมโยง องค์ประกอบเชื่อมโยง ต้นไม้

นิยาม 11.5 (กราฟเชื่อมโยง (Connected Graph)). กราฟไม่มีทิศทางเป็นกราฟเชื่อมโยง ถ้ามีเส้นทางระหว่างจุดยอดทุกคู่ กล่าวคือสำหรับจุดยอด u และ v ใดๆ จะต้องมีเส้นทางจาก u ไป v เสมอ

นิยาม 11.6 (องค์ประกอบเชื่อมโยง (Connected Component)). องค์ประกอบเชื่อมโยงคือกราฟย่อยที่เชื่อมโยงมากที่สุด (Maximal Connected Subgraph) กล่าวคือไม่สามารถเพิ่มจุดยอดใดๆ เข้าไปได้อีกโดยยังคงความเชื่อมโยง กราฟที่ไม่เชื่อมโยงจะมีหลายองค์ประกอบเชื่อมโยง

ตัวอย่างเช่น กราฟที่มีจุดยอด $\{0, 1, 2, 3, 4, 5, 6, 7, 8\}$ และมี 3 องค์ประกอบเชื่อมโยง:

- องค์ประกอบที่ 1: $\{0, 1, 2, 3, 4\}$
- องค์ประกอบที่ 2: $\{5\}$ (จุดเดี่ยว)
- องค์ประกอบที่ 3: $\{6, 7, 8\}$

นิยาม 11.7 (ต้นไม้ (Tree)). ต้นไม้คือกราฟเชื่อมโยงที่ไม่มีวงจร ต้นไม้ที่มี n จุดยอดจะมีเส้นเชื่อม $n - 1$ เส้นเสมอ และมีเส้นทางเพียงเส้นเดียวระหว่างจุดยอดทุกคู่

สมบัติสำคัญของต้นไม้

ต้นไม้มีสมบัติที่เทียบเท่ากันหลายประการ ถ้ากราฟ G มี n จุดยอด ข้อความต่อไปนี้เทียบเท่ากัน:

1. G เป็นต้นไม้ (เชื่อมโยง + ไม่มีวงจร)
2. G เชื่อมโยงและมี $n-1$ เส้นเชื่อม
3. G ไม่มีวงจรและมี $n-1$ เส้นเชื่อม
4. มีเส้นทางเพียงเส้นเดียวระหว่างจุดยอดทุกคู่

11.1.4 กราฟมีทิศทาง

นิยาม 11.8 (กราฟมีทิศทาง (Directed Graph / Digraph))**.** กราฟมีทิศทางคือกราฟที่เส้นเชื่อมมีทิศทาง เส้นเชื่อม (u, v) หมายถึงสามารถเดินจาก u ไป v ได้ แต่ไม่จำเป็นต้องเดินจาก v ไป u ได้

ในกราฟมีทิศทาง เส้นทางต้องเดินตามทิศทางของเส้นเชื่อมเท่านั้น ตัวอย่างเช่น ถ้ามีเส้นเชื่อม $3 \to 1$, $1 \to 2$, $2 \to 5$ แล้วเส้นทาง $3 \to 1 \to 2 \to 5$ เป็นเส้นทางที่ถูกต้อง แต่ $5 \to 3$ ไม่ใช่เส้นทาง เพราะไม่มีเส้นเชื่อมในทิศทางนั้น

11.1.5 กราฟมีน้ำหนัก

นิยาม 11.9 (กราฟมีน้ำหนัก (Weighted Graph))**.** กราฟมีน้ำหนักคือกราฟที่แต่ละเส้นเชื่อมมีค่าน้ำหนัก (Weight) กำกับ ซึ่งอาจแทนระยะทาง ต้นทุน หรือเวลา ความยาวของเส้นทางในกราฟมีน้ำหนักคือผลรวมของน้ำหนักเส้นเชื่อมทั้งหมดบนเส้นทางนั้น

ตัวอย่างเช่น ในกราฟที่เส้นเชื่อม $0 \to 3$ มีน้ำหนัก 5, $0 \to 2$ มีน้ำหนัก 4, $3 \to 4$ มีน้ำหนัก 9, $2 \to 4$ มีน้ำหนัก 2 เส้นทาง $0 \to 3 \to 4$ มีความยาว $5 + 9 = 14$ ในขณะที่เส้นทาง $0 \to 2 \to 4$ มีความยาว $4 + 2 = 6$ ซึ่งสั้นกว่า

11.1.6 ดีกรีและจุดข้างเคียง

นิยาม 11.10 (จุดข้างเคียงและดีกรี)**. จุดข้างเคียง (Neighbor)** ของจุดยอด v คือจุดยอดที่เชื่อมกับ v ด้วยเส้นเชื่อม **ดีกรี (Degree)** ของ v คือจำนวนจุดข้างเคียง หรือเทียบเท่ากับจำนวนเส้นเชื่อมที่ติดกับ v

ตัวอย่างเช่น ในกราฟที่จุดยอด 2 มีจุดข้างเคียงคือ $\{0, 1, 4\}$ ดังนั้น ดีกรีของจุดยอด 2 เท่ากับ 3

ทฤษฎีบท 11.1 (ทฤษฎีบทการจับมือ (Handshaking Lemma))**.** ในกราฟไม่มีทิศทาง ผลรวมของดีกรีของจุดยอดทุกจุดเท่ากับ $2m$ เสมอ กล่าวคือ

$$\sum_{v \in V} \deg(v) = 2m \tag{11.1}$$

ทฤษฎีบทนี้เป็นจริงเพราะเส้นเชื่อมแต่ละเส้นมีสองปลาย จึงถูกนับในดีกรีของจุดยอด 2 ครั้ง ผลรวมดีกรีจึงเป็นจำนวนคู่เสมอ

สำหรับกราฟมีทิศทาง เราแบ่งดีกรีออกเป็น:

- **ดีกรีเข้า (Indegree):** จำนวนเส้นเชื่อมเข้าหา v (เส้นเชื่อมที่ปลายที่ v)
- **ดีกรีออก (Outdegree):** จำนวนเส้นเชื่อมออกจาก v (เส้นเชื่อมที่ต้นทางเป็น v)

ตัวอย่างเช่น จุดยอด 2 ที่มีเส้นเชื่อมเข้า $1 \to 2$ และ $3 \to 2$ และเส้นเชื่อมออก $2 \to 4$ จะมีดีกรีเข้า = 2 และดีกรีออก = 1

11.1.7 กราฟแบบพิเศษ

กราฟสมบูรณ์ (Complete Graph) K_n คือกราฟที่จุดยอดทุกคู่มีเส้นเชื่อมเชื่อมระหว่างกัน จุดยอดทุกจุดมีดีกรีเท่ากับ $n-1$ และมีเส้นเชื่อมทั้งหมด $\frac{n(n-1)}{2}$ เส้น ตัวอย่างเช่น K_3 (สามเหลี่ยม) มี 3 เส้นเชื่อม, K_4 มี 6 เส้นเชื่อม, K_5 มี 10 เส้นเชื่อม

กราฟปกติ (Regular Graph) คือกราฟที่จุดยอดทุกจุดมีดีกรีเท่ากันหมด เช่น 3-Regular Graph (Cubic Graph) คือกราฟที่จุดยอดทุกจุดมีดีกรี = 3

นิยาม 11.11 (กราฟทวิภาคี (Bipartite Graph))**.** กราฟทวิภาคีคือกราฟที่สามารถแบ่งจุดยอดออกเป็น 2 เซต A และ B โดยที่เส้นเชื่อมทุกเส้นเชื่อมระหว่างจุดยอดใน A กับจุดยอดใน B เท่านั้น ไม่มีเส้นเชื่อมระหว่างจุดยอดในเซตเดียวกัน

ทฤษฎีบท 11.2 (เงื่อนไขกราฟทวิภาคี)**.** กราฟเป็นกราฟทวิภาคีก็ต่อเมื่อ **ไม่มีวงจรความยาวเป็นเลขคี่** (No Odd Cycle) กล่าวคือถ้ากราฟมีวงจรความยาว 3, 5, 7, ...กราฟนั้นไม่ใช่กราฟทวิภาคี

การระบุกราฟทวิภาคี

วิธีตรวจสอบว่ากราฟเป็นสองส่วนหรือไม่ คือลองระบายสีจุดยอดด้วย 2 สี โดยเริ่มจากจุดใดจุดหนึ่งระบายสี 0 แล้วระบายสีจุดข้างเคียงเป็น 1 สลับกันไปเรื่อยๆ ถ้าเจอจุดข้างเคียงที่มีสีเดียวกัน = ไม่ใช่กราฟทวิภาคี ถ้าระบายครบทุกจุดโดยไม่ขัดแย้ง = เป็นกราฟทวิภาคี เราจะเรียนรู้วิธีทำนี้ด้วย BFS ในหัวข้อต่อไป

11.2 การแทนกราฟ

การเลือกวิธีแทนกราฟมีผลโดยตรงต่อประสิทธิภาพของโปรแกรม วิธีที่เลือกขึ้นอยู่กับลักษณะของกราฟและขั้นตอนวิธีที่จะใช้ ในหัวข้อนี้เราจะศึกษา 3 วิธีหลักในการแทนกราฟ ได้แก่ Adjacency List, Adjacency Matrix และ Edge List

11.2.1 Adjacency List

Adjacency List เป็นวิธีที่นิยมใช้มากที่สุดในการโปรแกรมเชิงแข่งขัน เพราะประหยัดหน่วยความจำและมีประสิทธิภาพสำหรับกราฟเบาบาง (Sparse Graph) ซึ่งเป็นกราฟส่วนใหญ่ที่พบในการแข่งขัน

แนวคิดคือจุดยอดแต่ละจุดเก็บ **รายการของจุดข้างเคียง** ไว้ ใน C++ เราใช้ `vector<int> adj[N]` สำหรับกราฟไม่มีน้ำหนัก และ `vector<pair<int,int>> adj[N]` สำหรับกราฟมีน้ำหนัก (เก็บคู่ (จุดข้างเคียง, น้ำหนัก))

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);    // Add v as neighbor of u
        adj[v].push_back(u);    // Undirected graph: add both directions
    }
    // Display neighbors of vertex 0
    for (int v : adj[0]) {
        printf("%d ", v);
    }
    printf("\n");
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.1: สร้าง Adjacency List สำหรับกราฟไม่มีน้ำหนัก

สำหรับกราฟมีน้ำหนัก:

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;       // (neighbor, weight)
const int N = 100010;
vector<ii> adj[N];
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        adj[u].push_back({v, w});    // Edge u -> v with weight w
        adj[v].push_back({u, w});    // Undirected: add both directions
    }
    // Display neighbors and weights of vertex 0
```

```
    for (auto [v, w] : adj[0]) {
        printf("-> %d (weight = %d)\n", v, w);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.2: สร้าง Adjacency List สำหรับกราฟมีน้ำหนัก

ข้อดีของ Adjacency List

- ใช้หน่วยความจำ $\mathcal{O}(n+m)$ — ประหยัดสำหรับกราฟเบาบาง
- สามารถวนลูปดูจุดข้างเคียงของจุดยอด v ได้รวดเร็ว: `for (int u : adj[v])`
- เป็นวิธีที่ใช้กับขั้นตอนวิธีส่วนใหญ่ เช่น DFS, BFS, Dijkstra

11.2.2 Adjacency Matrix

Adjacency Matrix ใช้แถวลำดับ 2 มิติ `adj[n][n]` เก็บข้อมูลเส้นเชื่อม โดย `adj[u][v]` = 1 ถ้ามีเส้นเชื่อมจาก u ไป v (สำหรับกราฟมีน้ำหนัก เก็บค่าน้ำหนักแทน 1)

```
#include <bits/stdc++.h>
using namespace std;
int adj[105][105];   // adj[u][v] = weight of edge u-v (0 = no edge)
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    memset(adj, 0, sizeof(adj));
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        adj[u][v] = w;
        adj[v][u] = w;   // Undirected graph
    }
    // Check if there is an edge between u and v: O(1)
    int u = 0, v = 3;
    if (adj[u][v] > 0) {
        printf("has edge, weight = %d\n", adj[u][v]);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.3: สร้าง Adjacency Matrix

ข้อจำกัดของ Adjacency Matrix

- ใช้หน่วยความจำ $\mathcal{O}(n^2)$ — ไม่เหมาะกับกราฟที่ n มากๆ (เช่น $n > 5000$)
- การวนลูปดูจุดข้างเคียงของ v ใช้เวลา $\mathcal{O}(n)$ แม้ v จะมีจุดข้างเคียงไม่กี่จุด
- กราฟส่วนใหญ่ในการแข่งขันเป็น Sparse Graph (จำนวนเส้นเชื่อมน้อยกว่า n^2 มาก) ทำให้ Adjacency Matrix สิ้นเปลืองหน่วยความจำ

Adjacency Matrix เหมาะสมเมื่อ:

- กราฟมีความหนาแน่น (Dense Graph) คือ m ใกล้เคียง n^2
- ต้องการตรวจสอบการมีอยู่ของเส้นเชื่อมบ่อยๆ (เวลา $\mathcal{O}(1)$)
- ใช้กับขั้นตอนวิธี Floyd-Warshall (ที่ต้องใช้ Matrix อยู่แล้ว)

11.2.3 Edge List

Edge List เก็บเส้นเชื่อมทั้งหมดในลิสต์เดียว โดยแต่ละเส้นเชื่อมเก็บเป็นคู่ (หรือสามช่องสำหรับกราฟมีน้ำหนัก) วิธีนี้สะดวกสำหรับขั้นตอนวิธีที่ประมวลผลเส้นเชื่อมทั้งหมด เช่น Kruskal's Algorithm และ Bellman-Ford Algorithm

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef tuple<int,int,int> Edge;     // (weight, vertex u, vertex v)
vector<Edge> edges;
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        edges.push_back(make_tuple(w, u, v));
    }
    // Sort edges by weight (used in Kruskal's Algorithm)
    sort(edges.begin(), edges.end());
    for (auto [w, u, v] : edges) {
        printf("Edge %d-%d, weight = %d\n", u, v, w);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.4: สร้าง Edge List

เลือกน้ำหนักไว้ที่ตำแหน่งแรก

สังเกตว่าเราเก็บ `(w, u, v)` ไม่ใช่ `(u, v, w)` เพราะ C++ จะเรียง `tuple` ตามลำดับช่อง ถ้าเราเก็บน้ำหนักไว้ช่องแรก การเรียงด้วย `sort()` จะเรียงตามน้ำหนักทันที ซึ่งตรงกับสิ่งที่ Kruskal's Algorithm ต้องการ

11.2.4 เปรียบเทียบวิธีแทนกราฟ

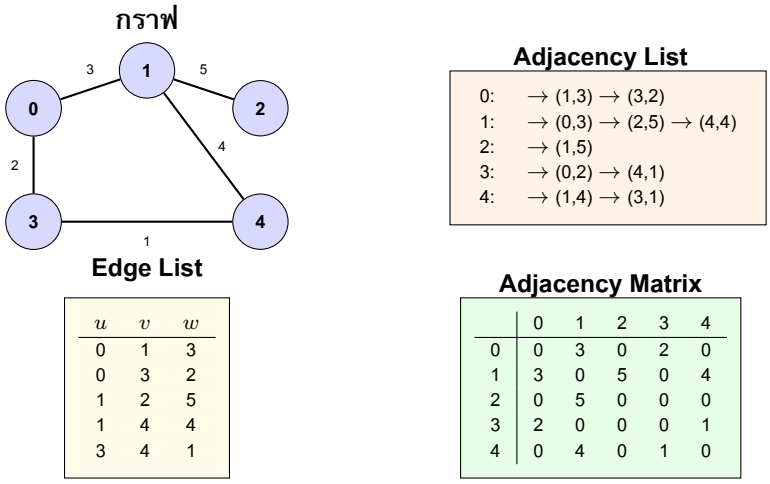
ตารางที่ 11.1 เปรียบเทียบวิธีแทนกราฟทั้ง 3 วิธี

การดำเนินการ	Adjacency List	Adjacency Matrix	Edge List
หน่วยความจำ	$\mathcal{O}(n+m)$	$\mathcal{O}(n^2)$	$\mathcal{O}(m)$
เพิ่มเส้นเชื่อม	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
ตรวจสอบเส้นเชื่อม (u,v)	$\mathcal{O}(\deg(u))$	$\mathcal{O}(1)$	$\mathcal{O}(m)$
วนลูปจุดข้างเคียงของ v	$\mathcal{O}(\deg(v))$	$\mathcal{O}(n)$	$\mathcal{O}(m)$
เหมาะกับ	กราฟเบาบาง	กราฟหนาแน่น	Kruskal's, Bellman-Ford

คำแนะนำ

ในการแข่งขัน ให้ใช้ **Adjacency List** เป็นค่าเริ่มต้น เพราะกราฟส่วนใหญ่เป็น Sparse Graph และ Adjacency List ทำงานได้ดีกับขั้นตอนวิธี DFS, BFS, Dijkstra และอื่นๆ ใช้ Adjacency Matrix เฉพาะเมื่อต้องการตรวจสอบเส้นเชื่อมบ่อยๆ หรือใช้ Floyd-Warshall และใช้ Edge List เมื่อต้องการเรียงเส้นเชื่อมตามน้ำหนัก

รูปที่ 11.2 แสดงตัวอย่างกราฟเดียวกันที่แทนด้วย 3 วิธี



รูปที่ 11.2 กราฟเดียวกันแทนด้วย 3 วิธี: Edge List, Adjacency List, Adjacency Matrix

11.3 Depth-First Search (DFS)

Depth-First Search (DFS) เป็นขั้นตอนวิธีการสำรวจกราฟพื้นฐานที่สำคัญที่สุด แนวคิดหลักคือเริ่มจากจุดยอดเริ่มต้น แล้ว “ดำดิ่ง” ไปให้ลึกที่สุดก่อน เมื่อถึงทางตัน (ไม่มีจุดข้างเคียงที่ยังไม่เคยเยี่ยม) จึงถอยกลับ (Backtrack) ไปลองเส้นทางอื่น DFS มีประโยชน์มากในการตรวจสอบโครงสร้างของกราฟ เช่น การหาองค์ประกอบเชื่อมโยง การตรวจจับวงจร และการเรียงลำดับทอพอโลยี[2]

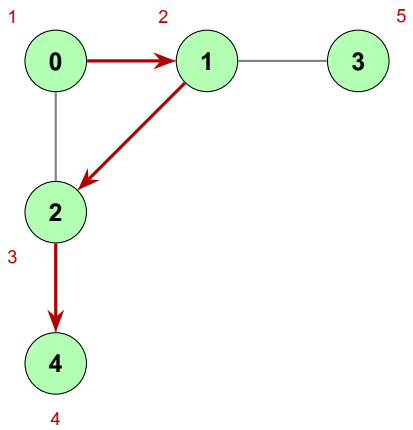
11.3.1 แนวคิด DFS

ขั้นตอนของ DFS มีดังนี้:

1. เริ่มจากจุดยอดเริ่มต้น s ทำเครื่องหมายว่า “เคยเยี่ยมแล้ว”
2. สำหรับจุดข้างเคียงแต่ละจุดของ s ที่ยังไม่เคยเยี่ยม ให้เรียก DFS ซ้ำจากจุดนั้น
3. เมื่อสำรวจจุดข้างเคียงทุกจุดแล้ว การเรียกซ้ำจะจบลง

DFS ใช้ **การเรียกซ้ำ (Recursion)** อย่างเป็นธรรมชาติ — โครงสร้างการเรียกซ้ำตรงกับการดำดิ่งลึกลงไปในกราฟ แต่ละจุดยอดถูกเยี่ยมครั้งเดียว และแต่ละเส้นเชื่อมถูกพิจารณาสองครั้ง (กราฟไม่มีทิศทาง) ดังนั้นความซับซ้อนของเวลาเป็น $\mathcal{O}(n+m)$

รูปที่ 11.3 แสดงลำดับการสำรวจกราฟด้วย DFS เริ่มจากจุดยอด 0 สังเกตว่า DFS จะดำดิ่งไปให้ลึกที่สุดก่อน แล้วจึงถอยกลับ



รูปที่ 11.3 ลำดับ DFS: $0 \to 1 \to 2 \to 4 \to 3$ (เส้นสีแดง = เส้นทาง DFS ดำดิ่ง, เลข = ลำดับเยี่ยม)

11.3.2 โค้ด DFS

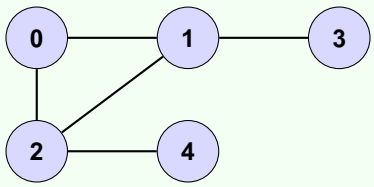
```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
```

```cpp
bool visited[N];
void dfs(int u) {
    visited[u] = true;
    printf("%d ", u);  // Process vertex u
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);      // Recurse for unvisited neighbors
        }
    }
}
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    memset(visited, false, sizeof(visited));
    dfs(0);  // Start DFS from vertex 0
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.5: DFS สำหรับกราฟไม่มีน้ำหนัก

ตัวอย่าง: ลำดับการเยี่ยมของ DFS

พิจารณากราฟในรูปที่ 11.4



รูปที่ 11.4 ตัวอย่างกราฟสำหรับ DFS

เริ่ม DFS จากจุดยอด 0 ลำดับการเยี่ยมอาจเป็น: 0, 1, 3, 2, 4 (ขึ้นอยู่กับลำดับของจุดข้างเคียงใน Adjacency List)

11.3.3 DFS สำหรับกราฟมีน้ำหนัก

สำหรับกราฟมีน้ำหนัก โครงสร้าง DFS เหมือนเดิม แต่ใช้ Adjacency List ที่เก็บคู่ (จุดข้างเคียง, น้ำหนัก):

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;
typedef vector<ii> vii;
#define UNVISITED -1
#define VISITED 1
typedef vector<int> vi;
vector<vii> adj;
vi visited;
void dfs(int u) {
    printf(" %d", u);
    visited[u] = VISITED;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        ii v = adj[u][j];
        if (visited[v.first] == UNVISITED)
            dfs(v.first);
    }
}
```

ตัวอย่างโค้ดที่ 11.6: DFS สำหรับกราฟมีน้ำหนัก

Stack Overflow

DFS ใช้การเรียกซ้ำ ดังนั้นถ้ากราฟมีความลึกมาก (เช่น กราฟที่เป็นเส้นตรงยาว n จุดยอด) การเรียกซ้ำจะมีความลึก n ชั้น ซึ่งอาจทำให้เกิด Stack Overflow ได้ ในการแข่งขัน ขีดจำกัด stack โดยทั่วไปอยู่ที่ประมาณ 10^5 ถ้ากราฟอาจลึกเกินนี้ ควรเขียน DFS แบบใช้ Stack แทนการเรียกซ้ำ

11.3.4 การประยุกต์ DFS

DFS มีการประยุกต์ใช้มากมาย ได้แก่:

1. **การหาองค์ประกอบเชื่อมโยง** — เรียก DFS จากจุดยอดที่ยังไม่เคยเยี่ยม จนกว่าจะครบทุกจุด
2. **การตรวจจับวงจร** — ถ้าระหว่าง DFS เจอเส้นเชื่อมไปยังจุดที่เคยเยี่ยมแล้ว (ยกเว้นพ่อแม่) แสดงว่ามีวงจร
3. **การเรียงลำดับทอพอโลยี** — เพิ่มจุดยอดลงในรายการหลังสำรวจจุดข้างเคียงทั้งหมดแล้ว แล้วกลับลำดับ
4. **Flood Fill** — ระบายสีกราฟหรือตาราง 2 มิติ

11.3.5 การจำแนกประเภทเส้นเชื่อม (Edge Classification)

เมื่อเรารัน DFS บนกราฟมีทิศทาง เส้นเชื่อมแต่ละเส้นสามารถจำแนกออกเป็น 4 ประเภท ได้แก่ Tree edges, Back edges, Forward edges และ Cross edges การจำแนกประเภทเส้นเชื่อมเป็นพื้นฐานสำคัญสำหรับการทำความเข้าใจขั้นตอนวิธีขั้นสูง เช่น การหา Articulation Points, Bridges และ Strongly Connected Components (SCC)

นิยาม 11.12 (ประเภทของเส้นเชื่อมในกราฟมีทิศทาง)**.** เมื่อ DFS ทำงานบนกราฟมีทิศทาง เส้นเชื่อม (u, v) สามารถจำแนกได้เป็น 4 ประเภท:

1. **Tree Edge:** เส้นเชื่อมที่เป็นส่วนหนึ่งของ DFS Spanning Tree คือเส้นเชื่อมที่ใช้เพื่อ “ค้นพบ” จุดยอดใหม่ เกิดขึ้นเมื่อ v ยังไม่ถูกเยี่ยมในขณะที่สำรวจเส้นเชื่อม (u, v)
2. **Back Edge:** เส้นเชื่อมที่ชี้จากจุดยอดไปหา **บรรพบุรุษ (Ancestor)** ของมันใน DFS Tree การมี Back Edge บ่งบอกว่ากราฟมี **วงจร (Cycle)**
3. **Forward Edge:** เส้นเชื่อมที่ชี้จากบรรพบุรุษไปหา **ลูกหลาน (Descendant)** ใน DFS Tree แต่ไม่ใช่ Tree Edge (คือ v ถูกค้นพบแล้วผ่านเส้นทางอื่น)
4. **Cross Edge:** เส้นเชื่อมที่เชื่อมระหว่างจุดยอดที่ **ไม่มีความสัมพันธ์บรรพบุรุษ-ลูกหลาน** ใน DFS Tree หรือเชื่อมไปยังจุดยอดที่สำรวจเสร็จแล้ว

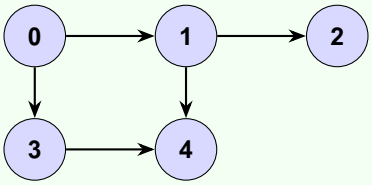
วิธีจำแนกเส้นเชื่อมด้วย Discovery/Finish Time

ระหว่าง DFS ให้บันทึก **เวลาค้นพบ** (`disc[u]`) และ **เวลาเสร็จสิ้น** (`fin[u]`) ของแต่ละจุดยอด สำหรับเส้นเชื่อม (u, v):

- **Tree Edge:** v ยังไม่ถูกค้นพบ (`disc[v] == 0`)
- **Back Edge:** v ถูกค้นพบแล้วแต่ยังไม่เสร็จสิ้น (`disc[v] > 0` และ `fin[v] == 0`) — แปลว่า v เป็นบรรพบุรุษของ u
- **Forward Edge:** v เสร็จสิ้นแล้ว และ `disc[u] < disc[v]` — v เป็นลูกหลานของ u
- **Cross Edge:** v เสร็จสิ้นแล้ว และ `disc[v] < disc[u]` — v ถูกสำรวจก่อน u

ตัวอย่าง: จำแนกเส้นเชื่อมในกราฟมีทิศทาง

พิจารณากราฟมีทิศทางต่อไปนี้ในรูปที่ 11.5



รูปที่ 11.5 ตัวอย่างกราฟมีทิศทางสำหรับจำแนกประเภทเส้นเชื่อม

เส้นเชื่อม: $(0,1),(0,3),(1,2),(1,4),(3,4)$
สมมติ DFS เริ่มจากจุด 0 และสำรวจจุดข้างเคียงตามลำดับ:

1. เยี่ยม 0: `disc[0] = 1`
2. สำรวจ $(0,1)$: 1 ยังไม่ถูกเยี่ยม $\Rightarrow$ **Tree Edge** $\Rightarrow$ เยี่ยม 1: `disc[1] = 2`
3. สำรวจ $(1,2)$: 2 ยังไม่ถูกเยี่ยม $\Rightarrow$ **Tree Edge** $\Rightarrow$ เยี่ยม 2: `disc[2] = 3`
4. 2 ไม่มีจุดข้างเคียง $\Rightarrow$ จบ 2: `fin[2] = 4`
5. สำรวจ $(1,4)$: 4 ยังไม่ถูกเยี่ยม $\Rightarrow$ **Tree Edge** $\Rightarrow$ เยี่ยม 4: `disc[4] = 5`
6. 4 ไม่มีจุดข้างเคียงใหม่ $\Rightarrow$ จบ 4: `fin[4] = 6`
7. จบ 1: `fin[1] = 7`
8. สำรวจ $(0,3)$: 3 ยังไม่ถูกเยี่ยม $\Rightarrow$ **Tree Edge** $\Rightarrow$ เยี่ยม 3: `disc[3] = 8`
9. สำรวจ $(3,4)$: 4 เสร็จสิ้นแล้ว (`fin[4] = 6`) และ `disc[3] = 8 > disc[4] = 5` $\Rightarrow$ **Cross Edge**
10. จบ 3: `fin[3] = 9`
11. จบ 0: `fin[0] = 10`

สรุป: Tree Edges = $\{(0,1),(1,2),(1,4),(0,3)\}$, Cross Edge = $\{(3,4)\}$

สำหรับ **กราฟไม่มีทิศทาง** การจำแนกจะง่ายกว่า — มีเพียง 2 ประเภทเท่านั้น:

- **Tree Edge:** เส้นเชื่อมที่ใช้ค้นพบจุดยอดใหม่
- **Back Edge:** เส้นเชื่อมที่ชี้ไปหาบรรพบุรุษ (ยกเว้นเส้นเชื่อมไปหาพ่อแม่โดยตรง)

กราฟไม่มีทิศทางไม่มี Forward Edge และ Cross Edge

ในกราฟไม่มีทิศทาง Forward Edge และ Cross Edge ไม่มีอยู่จริง เพราะเส้นเชื่อมทุกเส้นสามารถเดินได้ 2 ทิศทาง เส้นเชื่อมที่ดูเหมือน Forward Edge จะกลายเป็น Back Edge เมื่อพิจารณาทิศทางย้อนกลับ ดังนั้นในกราฟไม่มีทิศทาง มีเพียง Tree Edge และ Back Edge เท่านั้น

Back Edge บ่งบอกวงจร

ข้อสังเกตที่สำคัญที่สุด: **การมี Back Edge แปลว่ากราฟมีวงจร** ในกราฟมีทิศทาง กราฟมีวงจรก็ต่อเมื่อมี Back Edge อย่างน้อย 1 เส้น ในกราฟไม่มีทิศทาง Back Edge (ที่ไม่ใช่เส้นไปหาพ่อแม่) บ่งบอกว่ามีวงจรเช่นกัน นี่คือพื้นฐานสำคัญสำหรับขั้นตอนวิธีหา Articulation Points, Bridges และ SCC ที่เราจะเรียนในบทที่ 11

ต่อไปนี้คือโค้ด C++ ที่จำแนกประเภทเส้นเชื่อมระหว่าง DFS:

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
int disc[N], fin[N];
int timer = 0;
// Edge classification results
// 0 = Tree Edge, 1 = Back Edge, 2 = Forward Edge, 3 = Cross Edge
int classifyEdge(int u, int v) {
    if (disc[v] == 0) {
        return 0;  // Tree Edge: v not yet discovered
    } else if (fin[v] == 0) {
        return 1;  // Back Edge: v discovered but not finished (ancestor)
    } else if (disc[u] < disc[v]) {
        return 2;  // Forward Edge: v finished, v is descendant of u
    } else {
        return 3;  // Cross Edge: v finished, v is in different subtree
    }
}
void dfs(int u) {
    disc[u] = ++timer;  // Record discovery time
    for (int v : adj[u]) {
        int type = classifyEdge(u, v);
        string typeName;
        if (type == 0) typeName = "Tree Edge";
        else if (type == 1) typeName = "Back Edge";
        else if (type == 2) typeName = "Forward Edge";
        else typeName = "Cross Edge";
        printf("Edge (%d, %d) = %s\n", u, v, typeName.c_str());
        if (type == 0) {
            dfs(v);  // Only recurse on Tree Edges
        }
    }
    fin[u] = ++timer;  // Record finish time
}
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v;
```

```
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);   // Directed graph: only one direction
    }
    memset(disc, 0, sizeof(disc));
    memset(fin, 0, sizeof(fin));
    // Run DFS from all unvisited vertices
    for (int i = 0; i < n; i++) {
        if (disc[i] == 0) {
            dfs(i);
        }
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.7: จำแนกประเภทเส้นเชื่อมระหว่าง DFS

11.4 Breadth-First Search (BFS)

Breadth-First Search (BFS) เป็นขั้นตอนวิธีการสำรวจกราฟอีกตัวหนึ่งที่สำคัญ ต่างจาก DFS ตรงที่ BFS สำรวจ “กว้าง” ก่อน — เยี่ยมจุดข้างเคียงทุกจุดของจุดยอดปัจจุบันก่อน แล้วจึงลงลึกไประดับถัดไป BFS ใช้ **แถวคอย (Queue)** เป็นโครงสร้างข้อมูลหลัก และมีสมบัติพิเศษที่สำคัญคือสามารถหาระยะทางสั้นที่สุดในกราฟไม่มีน้ำหนักได้[2]

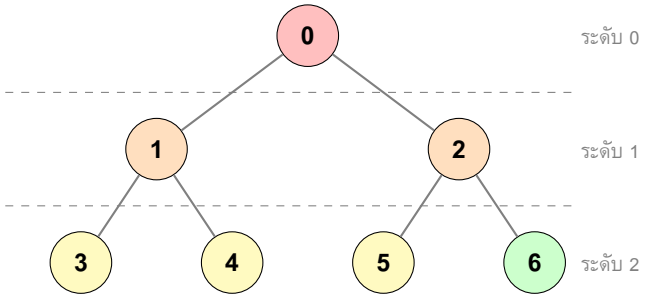
11.4.1 แนวคิด BFS

ขั้นตอนของ BFS มีดังนี้:

1. เริ่มจากจุดยอดเริ่มต้น s กำหนด `dist[s] = 0` และใส่ s ลงในแถวคอย
2. วนลูป: ดึงจุดยอด u จากด้านหน้าแถวคอย
3. สำหรับจุดข้างเคียง v แต่ละจุดของ u ที่ยังไม่เคยเยี่ยม: กำหนด `dist[v] = dist[u] + 1` และใส่ v ลงในแถวคอย
4. ทำซ้ำจนกว่าแถวคอยจะว่าง

BFS สำรวจจุดยอดตาม “ชั้น” (Layer) โดยจุดยอดที่อยู่ห่างจากจุดเริ่มต้น k เส้นเชื่อมจะถูกเยี่ยมในชั้นที่ k ทำให้ `dist[v]` เก็บระยะทางสั้นสุดจาก s ถึง v (ในกราฟไม่มีน้ำหนัก)

รูปที่ 11.6 แสดงลำดับการสำรวจกราฟด้วย BFS เริ่มจากจุดยอด 0 สังเกตว่า BFS สำรวจทีละระดับ ต่างจาก DFS ที่ดำดิ่งลึกก่อน



รูปที่ 11.6 ลำดับ BFS: สำรวจทีละระดับ (ระดับ 0 → ระดับ 1 → ระดับ 2) สีเข้ม = เยี่ยมก่อน

11.4.2 โค้ด BFS

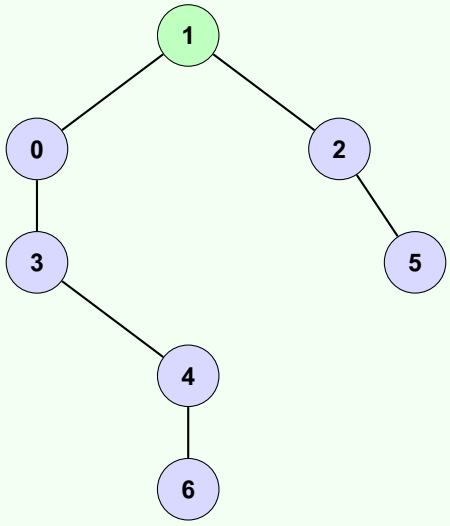
```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
int dist[N];
int parent[N];
void bfs(int s, int n) {
    // Initialize
    for (int i = 0; i < n; i++) {
        dist[i] = -1;     // -1 means not yet visited
        parent[i] = -1;
    }
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {        // Not yet visited
                dist[v] = dist[u] + 1; // Distance = parent's distance + 1
                parent[v] = u;          // Record previous vertex
                q.push(v);
            }
        }
    }
}
int main() {
    int n, m, s;
    scanf("%d %d %d", &n, &m, &s);
```

```
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    bfs(s, n);
    // Display distance from source s to all vertices
    for (int i = 0; i < n; i++) {
        printf("dist[%d] = %d\n", i, dist[i]);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.8: BFS พร้อมการหาระยะทาง

ตัวอย่าง: ระยะทางจาก BFS

พิจารณากราฟในรูปที่ 11.7



รูปที่ 11.7 ตัวอย่างกราฟสำหรับ BFS (จุดยอดเริ่มต้น = 1 สีเขียว)

เริ่ม BFS จากจุดยอด 1:

- ชั้น 0: จุด 1 (dist = 0)
- ชั้น 1: จุด 0, 2 (dist = 1)
- ชั้น 2: จุด 3, 5 (dist = 2)
- ชั้น 3: จุด 4 (dist = 3)

- ชั้น 4: จุด 6 (dist = 4)

ดังนั้น dist[6] = 4 คือระยะทางสั้นสุดจาก 1 ถึง 6

11.4.3 การสร้างเส้นทางจาก parent

จากแถวลำดับ `parent[]` เราสามารถสร้างเส้นทางจาก s ไป v ได้โดยการสืบย้อนจาก v กลับไปหา s:

```
void printPath(int v) {
    if (parent[v] == -1) {
        printf("%d", v);
        return;
    }
    printPath(parent[v]);
    printf(" -> %d", v);
}
// Usage: printPath(6) prints "1 -> 0 -> 3 -> 4 -> 6" (or another shortest path)
```

ตัวอย่างโค้ดที่ 11.9: สร้างเส้นทางจาก parent array

11.4.4 เปรียบเทียบ DFS กับ BFS

ตารางที่ 11.2 เปรียบเทียบ DFS กับ BFS

คุณสมบัติ	DFS	BFS
กลยุทธ์	ดำดิ่งลึกก่อน (Depth-First)	สำรวจกว้างก่อน (Breadth-First)
โครงสร้างข้อมูล	Stack (หรือ Recursion)	Queue
ความซับซ้อนของเวลา	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$
หน่วยความจำ	$\mathcal{O}(n)$ (stack ของ recursion)	$\mathcal{O}(n)$ (queue + dist array)
หาระยะทางสั้นสุด	ไม่ได้ (กราฟไม่มีน้ำหนัก)	ได้ (กราฟไม่มีน้ำหนัก)
การตรวจจับวงจร	ทำได้สะดวก	ทำได้แต่ไม่สะดวกเท่า
การเรียงทอพอโลยี	ทำได้โดยตรง	ใช้ Kahn's Algorithm

เลือก DFS หรือ BFS?

- ถ้าต้องการ **ระยะทางสั้นสุด** ในกราฟไม่มีน้ำหนัก ☒ ใช้ **BFS**
- ถ้าต้องการสำรวจทุกจุดยอดหรือหาองค์ประกอบเชื่อมโยง ☒ DFS หรือ BFS ก็ได้
- ถ้าต้องการ **เรียงลำดับทอพอโลยี** ☒ ใช้ DFS (ต่อท้ายหลังสำรวจเสร็จ)
- ถ้าต้องการ **ตรวจจับวงจร** ☒ ใช้ DFS (ตรวจเส้นกลับ Back Edge)

- ถ้ากราฟอาจลึกมาก (เสี่ยง Stack Overflow) ⊠ ใช้ BFS หรือ DFS แบบใช้ Stack

11.5 องค์ประกอบเชื่อมโยง

องค์ประกอบเชื่อมโยง (Connected Component) คือกราฟย่อยที่เชื่อมโยงมากที่สุด ในกราฟที่ไม่เชื่อมโยงจะมีหลายองค์ประกอบเชื่อมโยงแยกจากกัน การหาองค์ประกอบเชื่อมโยงเป็นปัญหาพื้นฐานที่ใช้ DFS หรือ BFS แก้ได้อย่างมีประสิทธิภาพ[3, 11]

11.5.1 หลักการ

หลักการสำคัญคือ **การเรียก DFS (หรือ BFS) จากจุดยอด u หนึ่งครั้ง จะเยี่ยมจุดยอดทุกจุดในองค์ประกอบเชื่อมโยงของ u** ดังนั้นวิธีหาองค์ประกอบเชื่อมโยงทั้งหมดคือ:

1. วนลูปตรวจสอบจุดยอดทุกจุดตั้งแต่ 0 ถึง $n-1$
2. ถ้าจุดยอด i ยังไม่เคยถูกเยี่ยม ⊠ เริ่ม DFS จาก i (ค้นพบองค์ประกอบใหม่)
3. นับจำนวนครั้งที่เริ่ม DFS = จำนวนองค์ประกอบเชื่อมโยง

11.5.2 โค้ดหาองค์ประกอบเชื่อมโยง

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
bool visited[N];
int component[N]; // Store component number of each vertex
int numCC = 0;
void dfs(int u, int cc) {
    visited[u] = true;
    component[u] = cc;    // Assign component number
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, cc);
        }
    }
}
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
```

```
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    memset(visited, false, sizeof(visited));
    numCC = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            numCC++;
            printf("Component %d:", numCC);
            dfs(i, numCC);
            printf("\n");
        }
    }
    printf("There are %d connected components\n", numCC);
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.10: หาองค์ประกอบเชื่อมโยงด้วย DFS

ตัวอย่างผลลัพธ์

สำหรับกราฟที่มี 9 จุดยอดและ 3 องค์ประกอบเชื่อมโยง:

```
Component 1: 0 1 3 2 4
Component 2: 5
Component 3: 6 7 8
There are 3 connected components
```

การตรวจสอบว่าสองจุดอยู่ในองค์ประกอบเดียวกันหรือไม่

หลังจากรันโค้ดข้างต้นแล้ว เราสามารถตรวจสอบได้ในเวลา $\mathcal{O}(1)$ ว่าจุดยอดสองจุดอยู่ในองค์ประกอบเดียวกันหรือไม่:

```
if (component[u] == component[v]) {
    // u and v are in the same component  connected
} else {
    // u and v are in different components  not connected
}
```

11.6 Flood Fill

Flood Fill เป็นเทคนิคการระบายสีหรือติดป้ายกำกับ (Label) บริเวณที่เชื่อมโยงกัน มักใช้กับ **กราฟโดยนัย (Implicit Graph)** เช่น ตาราง 2 มิติ ที่จุดยอดคือช่องในตาราง และเส้นเชื่อมคือความเชื่อมโยงระหว่างช่องที่อยู่ติดกัน Flood Fill มีประโยชน์มากในโจทย์ที่เกี่ยวข้องกับการหาบริเวณเชื่อมโยงบนแผนที่ ภาพ หรือตาราง[3, 11]

11.6.1 กราฟโดยนัยบนตาราง 2 มิติ

บนตาราง 2 มิติขนาด $R \times C$ แต่ละช่อง (r, c) เป็นจุดยอด เส้นเชื่อมเชื่อมระหว่างช่องที่อยู่ติดกัน โดยทั่วไปใช้ 8 ทิศทาง (S, SE, E, NE, N, NW, W, SW) หรือ 4 ทิศทาง (ขึ้น ลง ซ้าย ขวา) ขึ้นอยู่กับโจทย์

```
// 8 directions
int dr[] = {1, 1, 0, -1, -1, -1, 0, 1};
int dc[] = {0, 1, 1, 1, 0, -1, -1, -1};
// Directions: S, SE, E, NE, N, NW, W, SW
// 4 directions (up, down, left, right)
int dr4[] = {1, -1, 0, 0};
int dc4[] = {0, 0, -1, 1};
```

ตัวอย่างโค้ดที่ 11.11: แถวลำดับทิศทางสำหรับตาราง 2 มิติ

11.6.2 โค้ด Flood Fill

```
#include <bits/stdc++.h>
using namespace std;
const int MAXR = 105, MAXC = 105;
char grid[MAXR][MAXC];
int R, C;
// 8 directions
int dr[] = {1, 1, 0, -1, -1, -1, 0, 1};
int dc[] = {0, 1, 1, 1, 0, -1, -1, -1};
// Flood Fill: recolor from c1 to c2, return region size
int floodfill(int r, int c, char c1, char c2) {
    // Check bounds
    if (r < 0 || r >= R || c < 0 || c >= C) return 0;
    // Check if this cell has color c1
    if (grid[r][c] != c1) return 0;
    int ans = 1;              // Count current cell
    grid[r][c] = c2;          // Change color to prevent revisiting
    for (int d = 0; d < 8; d++) {
```

```
        ans += floodfill(r + dr[d], c + dc[d], c1, c2);
    }
    return ans;
}
int main() {
    // Read grid
    scanf("%d %d", &R, &C);
    for (int i = 0; i < R; i++) {
        scanf("%s", grid[i]);
    }
    // Find size of water region containing (3, 2)
    int size = floodfill(3, 2, 'W', '.');
    printf("Water region size: %d\n", size);
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.12: Flood Fill บนตาราง 2 มิติ

ตัวอย่าง: แผนที่น้ำท่วม

กำหนดแผนที่ขนาด 9×10:

```
LLLLLLLLL
LLWWLLWLL
LWWLLLLLL
LWWWLWWLL
LLLWWWLLL
LLLLLLLLL
LLLWWLLWL
LLWLWLLLL
LLLLLLLLL
```

`floodfill(3, 2, 'W', '.')` จะระบายสีบริเวณน้ำที่เชื่อมโยงกับช่อง (3,2) และคืนค่าขนาดของบริเวณนั้น

ระวังการเปลี่ยนสี

การเปลี่ยนสีจาก `c1` เป็น `c2` ในฟังก์ชัน `floodfill` ทำหน้าที่เป็น “ทำเครื่องหมายเยี่ยมแล้ว” ไปพร้อมกัน ทำให้ไม่ต้องใช้แถวลำดับ `visited[]` แยกต่างหาก แต่ต้องระวังว่าถ้าต้องการใช้ตารางเดิมต่อ ต้องเปลี่ยนสีกลับหรือเก็บตารางสำเนาไว้

11.7 การเรียงลำดับทอพอโลยี

การเรียงลำดับทอพอโลยี (Topological Sort) เป็นเทคนิคสำคัญสำหรับกราฟมีทิศทางที่ไม่มีวงจร (Directed Acyclic Graph หรือ DAG) โดยจัดเรียงจุดยอดให้ทุกเส้นเชื่อม $u \to v$ จุดยอด u อยู่ก่อน v เสมอ มีประโยชน์มากในโจทย์ที่เกี่ยวข้องกับการจัดลำดับกิจกรรมที่มีข้อกำหนดลำดับก่อนหน้า (Prerequisite)[2]

11.7.1 นิยามและเงื่อนไข

นิยาม 11.13 (Topological Sort)**.** การเรียงลำดับทอพอโลยีของ DAG คือลำดับเชิงเส้นของจุดยอด โดยที่สำหรับเส้นเชื่อมทุกเส้น $u \to v$ ในกราฟ จุดยอด u จะปรากฏก่อนจุดยอด v ในลำดับเสมอ

เงื่อนไขสำคัญ: **กราฟต้องเป็น DAG** (ไม่มีวงจร) ถ้ากราฟมีวงจร จะไม่มี Topological Sort เพราะจุดยอดในวงจรไม่สามารถจัดลำดับได้

ทฤษฎีบท 11.3 (การมีอยู่ของ Topological Sort)**.** กราฟมีทิศทางมี Topological Sort ก็ต่อเมื่อกราฟนั้นเป็น DAG (ไม่มีวงจร) นอกจากนี้ DAG ทุกกราฟมี Topological Sort อย่างน้อย 1 แบบ (อาจมีหลายแบบ)

11.7.2 วิธีที่ 1: DFS-based Topological Sort

วิธีนี้ใช้ DFS ปรับแต่งเล็กน้อย: หลังจากสำรวจจุดข้างเคียงทั้งหมดของจุดยอด u แล้ว ให้เพิ่ม u ลงท้ายในรายการผลลัพธ์ เมื่อ DFS เสร็จสมบูรณ์ ลำดับทอพอโลยีคือ **ลำดับย้อนกลับ** ของรายการผลลัพธ์

ทำไมต้องย้อนกลับ? เพราะจุดยอด u ถูกเพิ่มลงรายการหลังจากจุดยอดที่ u ชี้ไปทั้งหมดแล้ว ดังนั้นในรายการ u อยู่หลังจุดยอดที่ u ชี้ไป ซึ่งตรงข้ามกับที่ Topological Sort ต้องการ (u ต้องอยู่ก่อน) การย้อนกลับจึงแก้ปัญหานี้

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
bool visited[N];
vector<int> topoOrder;
void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
    topoOrder.push_back(u);   // Add u after exploring all neighbors
}
int main() {
    int n, m;
```

```
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);    // Edge u -> v (directed graph)
    }
    memset(visited, false, sizeof(visited));
    topoOrder.clear();
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    // Reverse to get Topological Sort
    reverse(topoOrder.begin(), topoOrder.end());
    printf("Topological Sort: ");
    for (int u : topoOrder) {
        printf("%d ", u);
    }
    printf("\n");
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.13: Topological Sort แบบ DFS

11.7.3 วิธีที่ 2: Kahn's Algorithm (BFS-based)

Kahn's Algorithm ใช้หลักการ: จุดยอดที่ **ดีกรีเข้า = 0** (ไม่มีเส้นเชื่อมเข้า) สามารถวางไว้ลำดับแรกได้ เมื่อนำจุดยอดออกแล้ว ให้ลดดีกรีเข้าของจุดข้างเคียง และทำซ้ำกับจุดยอดที่ดีกรีเข้าเป็น 0 ใหม่

```
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
int indegree[N];
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    memset(indegree, 0, sizeof(indegree));
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
```

```
        adj[u].push_back(v);     // u -> v
        indegree[v]++;           // Increment indegree of v
    }
    // Add vertices with indegree = 0 to queue
    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }
    vector<int> topoOrder;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        topoOrder.push_back(u);
        for (int v : adj[u]) {
            indegree[v]--;       // Remove edge u -> v
            if (indegree[v] == 0) {
                q.push(v);       // v is ready to enter the order
            }
        }
    }
    // Check if there is a cycle
    if ((int)topoOrder.size() != n) {
        printf("Graph has a cycle! No topological sort exists.\n");
    } else {
        printf("Topological Sort: ");
        for (int u : topoOrder) {
            printf("%d ", u);
        }
        printf("\n");
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 11.14: Topological Sort แบบ Kahn's Algorithm

การตรวจจับวงจรด้วย Kahn's Algorithm

หลังจากรัน Kahn's Algorithm แล้ว ถ้าจำนวนจุดยอดใน `topoOrder` ไม่เท่ากับ n แสดงว่ากราฟมีวงจร เพราะจุดยอดในวงจรจะมีดีกรีเข้าไม่เคยเป็น 0 ดังนั้นจะไม่ถูกเพิ่มลงในลำดับ วิธีนี้จึงสามารถใช้ตรวจจับวงจรในกราฟมีทิศทางได้พร้อมกับการเรียงลำดับทอพอโลยี

ตัวอย่าง: ลำดับวิชาเรียน

สมมติวิชาต่างๆ มีข้อกำหนดลำดับก่อนหน้า:

```
Course 0 -> Course 1 (must take 0 before 1)
Course 0 -> Course 2
Course 1 -> Course 3
Course 2 -> Course 3
Course 3 -> Course 4
```

Topological Sort ที่เป็นไปได้: 0, 1, 2, 3, 4 หรือ 0, 2, 1, 3, 4 (มีหลายแบบที่ถูกต้อง) ความหมายคือเราสามารถเรียนวิชาตามลำดับนี้ได้โดยไม่ขัดข้องกับข้อกำหนด

Topological Sort อาจไม่มีคำตอบเดียว

DAG หนึ่งกราฟอาจมี Topological Sort ที่ถูกต้องหลายแบบ ตัวอย่างเช่น ถ้า $0 \to 1$ และ $0 \to 2$ โดยไม่มีเส้นเชื่อมระหว่าง 1 กับ 2 แล้วทั้ง $0, 1, 2$ และ $0, 2, 1$ ล้วนเป็น Topological Sort ที่ถูกต้อง ถ้าโจทย์ต้องการ Topological Sort แบบใดแบบหนึ่งโดยเฉพาะ (เช่น ลำดับพจนานุกรมที่เล็กที่สุด) ให้ใช้ `priority_queue` แทน `queue` ใน Kahn's Algorithm

11.7.4 เปรียบเทียบสองวิธี

ตารางที่ 11.3 เปรียบเทียบ DFS-based กับ Kahn's Algorithm

คุณสมบัติ	DFS-based	Kahn's (BFS-based)
หลักการ	เพิ่มจุดยอดหลังสำรวจเสร็จ แล้วย้อนลำดับ	นำจุดยอดที่ดีกรีเข้า=0 ออกทีละจุด
ความซับซ้อนของเวลา	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$
การตรวจจับวงจร	ต้องเขียนเพิ่ม	ตรวจสอบได้โดยตรง (จากจำนวนจุดยอดในผลลัพธ์)
การใช้งาน	สะดวกถ้าใช้ DFS อยู่แล้ว	สะดวกถ้าต้องการตรวจจับวงจร
ลำดับเฉพาะเจาะจง	ขึ้นกับลำดับจุดข้างเคียง	ขึ้นกับลำดับในแถวคอย (ใช้ priority_queue ได้)

คำแนะนำ

- ถ้าต้องการ **ตรวจจับวงจร** พร้อมกับ Topological Sort ☒ ใช้ **Kahn's Algorithm**
- ถ้าต้องการ **ลำดับพจนานุกรมที่เล็กที่สุด** ☒ ใช้ Kahn's กับ `priority_queue`
- ถ้าใช้ DFS อยู่แล้วในโค้ด ☒ เพิ่ม `topoOrder.push_back(u)` หลัง DFS เพียง 1 บรรทัด

11.8 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนทฤษฎีกราฟพื้นฐาน:

โจทย์แนะนำสำหรับบทที่ 11

DFS & BFS:

- CSES Building Roads — หาองค์ประกอบเชื่อมโยง[12]
- CSES Message Route — BFS หาเส้นทางสั้นสุด[12]
- Codeforces 510A — Fox And Snake (Grid BFS)[10]
- UVA 11902 — Dominator (DFS)[20]

Flood Fill:

- UVA 469 — Wetlands of Florida (Flood Fill)[20]
- UVA 10946 — A New Island (Connected Components)[20]
- Codeforces 1592B — Hemose Shopping (Grid)[10]

Topological Sort:

- CSES Course Schedule — Topological Sort[12]
- CSES Longest Flight Route — DP on DAG[12]
- UVA 10305 — Ordering Tasks (Topological Sort)[20]
- Codeforces 510C — Fox And Names (Topological Sort + Cycle)[10]

11.9 สรุปบทที่ 11

ในบทนี้ เราได้ศึกษาพื้นฐานทฤษฎีกราฟตั้งแต่นิยามและการแทนกราฟ ไปจนถึงขั้นตอนวิธี DFS, BFS การหาองค์ประกอบเชื่อมโยง Flood Fill และการเรียงลำดับทอพอโลยี ขั้นตอนวิธีเหล่านี้เป็นรากฐานสำคัญสำหรับบทต่อๆ ไปที่เกี่ยวกับกราฟ

สรุปสิ่งสำคัญของบทที่ 11

1. **Adjacency List** เป็นวิธีแทนกราฟที่ใช้บ่อยที่สุด — ประหยัดหน่วยความจำและเร็ว
2. **DFS** ใช้สำรวจกราฟ หาองค์ประกอบเชื่อมโยง ตรวจจับวงจร และ Topological Sort

3. **BFS** ใช้หาระยะทางสั้นสุดบนกราฟไม่มีน้ำหนัก และหาเส้นทางจาก parent
4. **Flood Fill** คือ DFS/BFS บนกราฟโดยนัยจากตาราง 2 มิติ
5. **Topological Sort** ใช้ได้เฉพาะ DAG ทำได้ 2 วิธี: DFS-based และ Kahn's Algorithm
6. **Edge Classification** ช่วยจำแนกเส้นเชื่อมเป็น Tree/Back/Forward/Cross edge

Chapter 12

เส้นทางสั้นสุด

ปัญหาการหาเส้นทางสั้นสุด (Shortest Path) เป็นหนึ่งในปัญหาคลาสสิกที่สำคัญที่สุดในทฤษฎีกราฟและการโปรแกรมเชิงแข่งขัน ไม่ว่าจะเป็นการหาเส้นทางที่สั้นที่สุดระหว่างเมืองสองเมือง การหาระยะทางน้อยสุดในเครือข่ายคมนาคม หรือการหาต้นทุนต่ำสุดในการขนส่งสินค้า ทั้งหมดล้วนเป็นปัญหา Shortest Path ทั้งสิ้น ในบทนี้ เราจะศึกษาขั้นตอนวิธีหาเส้นทางสั้นสุดที่สำคัญ 4 ตัว ได้แก่ Bellman-Ford, Dijkstra, Floyd-Warshall และ Shortest Path on DAG ซึ่งแต่ละตัวมีจุดแข็งและข้อจำกัดที่แตกต่างกัน[2, 3]

12.1 แนะนำปัญหา Shortest Path

12.1.1 นิยามปัญหา

ปัญหา Shortest Path คือการหาเส้นทางระหว่างจุดยอดสองจุดในกราฟมีน้ำหนัก ที่มีผลรวมน้ำหนักต่ำสุด กล่าวคือ กำหนดกราฟมีน้ำหนัก $G = (V, E)$ และจุดยอดเริ่มต้น s เราต้องการหาระยะทางสั้นสุดจาก s ไปยังจุดยอดทุกจุด (Single-Source Shortest Path) หรือระหว่างจุดยอดทุกคู่ (All-Pairs Shortest Path)

สำหรับกราฟไม่มีน้ำหนัก เราสามารถใช้ BFS หาระยะทางสั้นสุดได้โดยตรง (เส้นทางสั้นสุด = เส้นทางที่ผ่านจำนวนเส้นเชื่อมน้อยที่สุด) แต่สำหรับกราฟมีน้ำหนัก เราต้องใช้ขั้นตอนวิธีเฉพาะ

นิยาม 12.1 (Single-Source Shortest Path — SSSP)**.** กำหนดจุดยอดเริ่มต้น s จงหาระยะทางสั้นสุดจาก s ไปยังจุดยอดทุกจุดในกราฟ

นิยาม 12.2 (All-Pairs Shortest Path — APSP)**.** จงหาระยะทางสั้นสุดระหว่างจุดยอดทุกคู่ในกราฟ

12.1.2 การผ่อนคลาย (Relaxation)

แนวคิดหลักที่ใช้ในทุกขั้นตอนวิธี Shortest Path คือ **การผ่อนคลาย (Relaxation)** กำหนดให้ `dist[v]` เก็บค่าประมาณระยะทางสั้นสุดจาก s ถึง v ในปัจจุบัน ถ้ามีเส้นเชื่อม $u \to v$ ที่มีน้ำหนัก w และ `dist[u] + w < dist[v]` เราจะ “ผ่อนคลาย” เส้นเชื่อมนี้โดยกำหนด `dist[v] = dist[u] + w` กล่าวคือเราพบเส้นทางที่สั้นกว่าผ่าน u

```
// Relax edge u -> v with weight w
if (dist[u] + w < dist[v]) {
    dist[v] = dist[u] + w;
}
```

ตัวอย่างโค้ดที่ 12.1: การผ่อนคลายเส้นเชื่อม

ขั้นตอนวิธี Shortest Path ทั้งหมดที่เราจะศึกษาใช้การผ่อนคลายเป็นกลไกหลัก แตกต่างกันที่ลำดับและจำนวนครั้งในการผ่อนคลาย

12.1.3 วงจรน้ำหนักลบ (Negative Cycle)

นิยาม 12.3 (วงจรน้ำหนักลบ)**.** วงจรน้ำหนักลบ (Negative Cycle) คือวงจรที่ผลรวมน้ำหนักของเส้นเชื่อมทั้งหมดบนวงจรเป็นค่าลบ

ถ้ากราฟมีวงจรน้ำหนักลบที่สามารถเดินทางถึงได้จากจุดเริ่มต้น s แล้วระยะทางสั้นสุดจาก s ไปยังจุดยอดบางจุด **ไม่นิยาม** เพราะเราสามารถเดินวนในวงจรนั้นได้ไม่จำกัด ทำให้ระยะทางลดลงเรื่อยๆ ไม่มีที่สิ้นสุด

วงจรน้ำหนักลบทำให้ Shortest Path ไม่นิยาม

ถ้ามีวงจรน้ำหนักลบที่สามารถเดินทางถึงได้จาก s ระยะทางจาก s ไปยังจุดยอดบางจุดจะเป็น $-\infty$ ขั้นตอนวิธีบางตัว (เช่น Bellman-Ford) สามารถตรวจจับวงจรน้ำหนักลบได้ ในขณะที่บางตัว (เช่น Dijkstra) ไม่สามารถจัดการกับน้ำหนักลบได้

12.2 Bellman-Ford Algorithm

Bellman-Ford เป็นขั้นตอนวิธี SSSP ที่สามารถจัดการกับกราฟที่มีน้ำหนักลบได้ (แต่ต้องไม่มีวงจรน้ำหนักลบ) และสามารถตรวจจับวงจรน้ำหนักลบได้ แม้จะช้ากว่า Dijkstra แต่ Bellman-Ford มีประโยชน์ในกรณีที่กราฟมีเส้นเชื่อมที่มีน้ำหนักลบ[2]

12.2.1 หลักการ

Bellman-Ford ทำงานโดยผ่อนคลายเส้นเชื่อมทั้งหมด $n-1$ รอบ ในแต่ละรอบ เราตรวจสอบเส้นเชื่อมทุกเส้นและพยายามผ่อนคลาย หลังจากรอบที่ $n-1$ ระยะทางทั้งหมดจะถูกต้องแล้ว (ถ้าไม่มีวงจรน้ำหนักลบ) เพื่อตรวจจับวงจรน้ำหนักลบ เราตรวจสอบรอบเพิ่มอีก 1 รอบ (รอบที่ n) ถ้ายังมีเส้นเชื่อมที่สามารถผ่อนคลายได้ แสดงว่ามีวงจรน้ำหนักลบ

ทำไมต้อง $n-1$ รอบ? เพราะเส้นทางสั้นสุดในกราฟที่ไม่มีวงจรน้ำหนักลบมีความยาวไม่เกิน $n-1$ เส้นเชื่อม หลังจากผ่อนคลาย $n-1$ รอบ เส้นทางทุกเส้นจะถูกพิจารณาครบแล้ว

12.2.2 โค้ด Bellman-Ford

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef tuple<int,int,int> Edge;   // (u, v, w)
#define INF 1000000000
int main() {
```

```
    int V, E, s;
    scanf("%d %d %d", &V, &E, &s);
    vector<Edge> edges;
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        edges.push_back(make_tuple(u, v, w));
    }
    // Initialize
    vector<int> dist(V, INF);
    dist[s] = 0;
    // Relax all edges V-1 times
    for (int i = 0; i < V - 1; i++) {
        for (auto [u, v, w] : edges) {
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    // Detect negative weight cycle (round V)
    bool hasNegativeCycle = false;
    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            hasNegativeCycle = true;
            break;
        }
    }
    if (hasNegativeCycle) {
        printf("Negative cycle detected!\n");
    } else {
        for (int i = 0; i < V; i++) {
            printf("SSSP(%d, %d) = %d\n", s, i, dist[i]);
        }
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.2: Bellman-Ford Algorithm

ตัวอย่าง: การทำงานของ Bellman-Ford

พิจารณากราฟ 5 จุดยอด โดยมีเส้นเชื่อม: $0 \to 1$ (น้ำหนัก 5), $0 \to 3$ (น้ำหนัก 9), $0 \to 4$ (น้ำหนัก 1), $1 \to 2$ (น้ำหนัก 2), $2 \to 3$ (น้ำหนัก 6), $4 \to 1$ (น้ำหนัก -2), $4 \to 3$ (น้ำหนัก 2)
เริ่มจากจุด 0:

- รอบที่ 1: dist[4] = 1, dist[1] = 5, dist[3] = 9, dist[1] = $1 + (-2) = -1$ (ผ่อนคลายผ่าน 4)
- รอบที่ 2: dist[2] = $-1 + 2 = 1$, dist[3] = min$(9, 1 + 2) = 3$ (ผ่อนคลายผ่าน 4)
- รอบที่ 3-4: ไม่มีการเปลี่ยนแปลงเพิ่มเติม

ผลลัพธ์: dist[0]=0, dist[1]=-1, dist[2]=1, dist[3]=3, dist[4]=1

12.2.3 สมบัติของ Bellman-Ford

ตารางที่ 12.1 สมบัติของ Bellman-Ford Algorithm

คุณสมบัติ	รายละเอียด
ประเภทปัญหา	Single-Source Shortest Path (SSSP)
ความซับซ้อนของเวลา	$\mathcal{O}(V \cdot E)$
น้ำหนักลบ	รองรับ (ต้องไม่มีวงจรน้ำหนักลบ)
ตรวจจับวงจรน้ำหนักลบ	ได้
โครงสร้างข้อมูล	Edge List
เหมาะกับ	กราฟที่มีน้ำหนักลบ, ต้องการตรวจจับวงจร

การเพิ่มประสิทธิภาพ Bellman-Ford

ในแต่ละรอบ ถ้าไม่มีการผ่อนคลายใดๆ เกิดขึ้นเลย แสดงว่าระยะทางทั้งหมดถูกต้องแล้ว เราสามารถหยุดได้ทันทีโดยไม่ต้องรอครบ $V - 1$ รอบ การเพิ่มเงื่อนไขนี้ช่วยเร่งความเร็วได้มากในกราฟหลายกรณี:

```
bool changed = false;
for (auto [u, v, w] : edges) {
    if (dist[u] != INF && dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        changed = true;
    }
}
if (!changed) break;  // Stop if no changes
```

12.3 Dijkstra's Algorithm

Dijkstra's Algorithm เป็นขั้นตอนวิธี SSSP ที่มีประสิทธิภาพสูงกว่า Bellman-Ford แต่มีข้อจำกัดคือ **ไม่สามารถจัดการกับน้ำหนักลบได้** สำหรับกราฟที่ทุกเส้นเชื่อมมีน้ำหนักไม่ติดลบ Dijkstra จะทำงานได้เร็วกว่า Bellman-Ford อย่างมาก จึงเป็นขั้นตอนวิธีที่ใช้บ่อยที่สุดในการแข่งขัน[2]

12.3.1 หลักการ

Dijkstra ใช้กลยุทธ์ **Greedy** — ในแต่ละรอบ เลือกจุดยอดที่ยังไม่ได้ประมวลผลและมี `dist` น้อยที่สุดมาประมวลผล เมื่อจุดยอดถูกประมวลผลแล้ว ระยะทางของมันจะ **ถูกต้องตามที่เป็น** (Final) และจะไม่เปลี่ยนแปลงอีก

ทำไม Dijkstra ถูกต้อง? เพราะเมื่อทุกเส้นเชื่อมมีน้ำหนักไม่ติดลบ จุดยอดที่มี `dist` น้อยที่สุดในขณะนั้น **ไม่มีทาง** จะมีเส้นทางที่สั้นกว่าผ่านจุดยอดอื่นที่ยังไม่ได้ประมวลผล เพราะจุดยอดเหล่านั้นมี `dist` มากกว่าอยู่แล้ว และเส้นเชื่อมมีน้ำหนักไม่ติดลบ จึงไม่สามารถลดระยะทางได้

12.3.2 ทำไม Dijkstra ใช้ไม่ได้กับน้ำหนักลบ

พิจารณากราฟ: $1 \to 2$ น้ำหนัก 2, $1 \to 3$ น้ำหนัก 5, $2 \to 3$ น้ำหนัก -10

Dijkstra จะประมวลผลจุดยอด 2 ก่อน (dist=2 < dist[3]=5) และกำหนด dist[2]=2 เป็นค่าถาวร แต่ในความเป็นจริง เส้นทางสั้นสุดจาก 1 ถึง 3 คือ $1 \to 2 \to 3 = 2 + (-10) = -8$ ซึ่งน้อยกว่า dist[3]=5 มาก Dijkstra จะได้คำตอบผิดเพราะมัน "ปิด" จุดยอด 2 เร็วเกินไป โดยไม่รู้ว่ามีเส้นเชื่อมน้ำหนักลบที่จะทำให้เส้นทางผ่าน 2 สั้นขึ้น

Dijkstra ไม่ทำงานกับน้ำหนักลบ!

ถ้ากราฟมีเส้นเชื่อมที่มีน้ำหนักลบ **ห้าม** ใช้ Dijkstra ให้ใช้ Bellman-Ford แทน ถ้าไม่แน่ใจว่ากราฟมีน้ำหนักลบหรือไม่ ให้ใช้ Bellman-Ford เพื่อความปลอดภัย

12.3.3 โค้ด Dijkstra ด้วย Priority Queue

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;    // (dist, node)
typedef vector<ii> vii;
#define INF 1000000000
int main() {
    int V, E, s;
    scanf("%d %d %d", &V, &E, &s);
    vector<vii> adj(V);
    for (int i = 0; i < E; i++) {
```

```
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        adj[u].push_back({v, w});     // Edge u -> v with weight w
        // Undirected: adj[v].push_back({u, w});
    }
    vector<int> dist(V, INF);
    dist[s] = 0;
    // min-heap: sorted by dist ascending
    priority_queue<ii, vector<ii>, greater<ii>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        // Lazy Deletion: skip stale entries
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    for (int i = 0; i < V; i++) {
        printf("SSSP(%d, %d) = %d\n", s, i, dist[i]);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.3: Dijkstra's Algorithm ด้วย Priority Queue

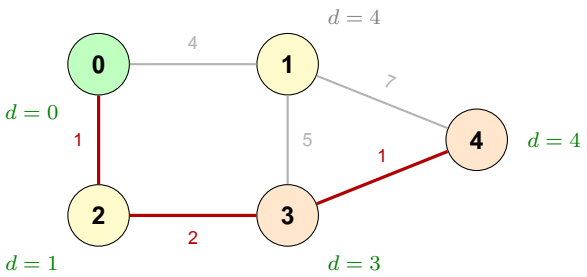
Lazy Deletion

ในโค้ดข้างต้น เราใช้เทคนิค **Lazy Deletion** — แทนที่จะลบรายการเก่าออกจาก Priority Queue เมื่อ dist[v] เปลี่ยนแปลง เราเพียงเพิ่มรายการใหม่เข้าไป เมื่อดึงรายการออกมา ถ้า `d > dist[u]` แสดงว่าเป็นรายการเก่าที่ล้าสมัยแล้ว ให้ข้ามไป เทคนิคนี้ง่ายกว่าการลบรายการเก่าและไม่ทำให้ความซับซ้อนแย่ลง

รูปที่ 12.1 แสดงตัวอย่างการทำงานของ Dijkstra's Algorithm บนกราฟมีน้ำหนัก โดยเริ่มจากจุดยอด 0 ตัวเลขในวงกลมคือระยะทางสั้นสุดจากจุดยอด 0

12.3.4 ตัวอย่างการทำงานของ Dijkstra

พิจารณากราฟ 5 จุดยอด:



รูปที่ 12.1 Dijkstra's Algorithm: เส้นทางสั้นสุดจาก 0 ไปทุกจุด (เส้นสีแดง = เส้นทางสั้นสุด) ระยะทาง: 0→2=1, 0→3=3, 0→4=4, 0→1=4

```
0 --5--> 1 --2--> 2
|        |        |
9        -2       6
|        |        |
v        v        v
3 <--2--- 4
```

เส้นเชื่อม: $0 \to 1$ (5), $0 \to 3$ (9), $1 \to 2$ (2), $1 \to 4$ (-2), $2 \to 3$ (6), $4 \to 3$ (2)

ตัวอย่างนี้มีน้ำหนักลบ!

ตัวอย่างนี้มีเส้นเชื่อม $1 \to 4$ ที่มีน้ำหนัก -2 ดังนั้น **ไม่ควรใช้ Dijkstra** กับกราฟนี้ แต่เพื่อแสดงให้เห็นว่า Dijkstra ทำงานผิดพลาดอย่างไร เราจะลองรันดู

สำหรับกราฟที่ไม่มีน้ำหนักลบ (เช่น เปลี่ยน -2 เป็น 2):

1. รอบที่ 1: ประมวลผลจุด 0 (dist=0) ⊠ dist[1]=5, dist[3]=9
2. รอบที่ 2: ประมวลผลจุด 1 (dist=5) ⊠ dist[2]=7, dist[4]=7
3. รอบที่ 3: ประมวลผลจุด 2 (dist=7) ⊠ dist[3]=9 (ไม่ดีขึ้น)
4. รอบที่ 4: ประมวลผลจุด 4 (dist=7) ⊠ dist[3]=min$(9, 7+2) = 9$ (ไม่ดีขึ้น)
5. รอบที่ 5: ประมวลผลจุด 3 (dist=9)

ผลลัพธ์: dist[0]=0, dist[1]=5, dist[2]=7, dist[3]=9, dist[4]=7

12.4 Floyd-Warshall Algorithm

Floyd-Warshall เป็นขั้นตอนวิธี All-Pairs Shortest Path (APSP) ที่หาระยะทางสั้นสุดระหว่างจุดยอด **ทุกคู่** ในกราฟ ใช้ Adjacency Matrix เป็นโครงสร้างข้อมูลหลัก และมีโค้ดที่สั้นและง่ายมาก — เพียงลูปซ้อน 3 ชั้น[2]

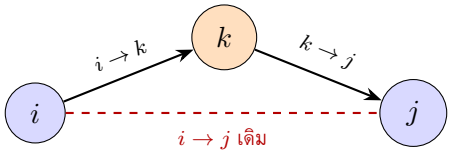
12.4.1 หลักการ

Floyd-Warshall ทำงานโดยพิจารณาจุดยอดกลาง (Intermediate Node) k ทีละจุด สำหรับแต่ละคู่ (i, j) เราตรวจสอบว่าการเดินผ่าน k สั้นกว่าเส้นทางปัจจุบันหรือไม่:

$$\text{dist}[i][j] = \min(\text{dist}[i][j],\ \text{dist}[i][k] + \text{dist}[k][j]) \tag{12.1}$$

หลังจากพิจารณาจุดยอดกลางครบทุกจุด ($k = 0, 1, \ldots, n-1$) `dist[i][j]` จะเก็บระยะทางสั้นสุดจาก i ถึง j

รูปที่ 12.2 แสดงแนวคิดหลักของ Floyd-Warshall: พิจารณาว่าการผ่านจุดกลาง k ทำให้เส้นทางจาก i ไป j สั้นลงหรือไม่



รูปที่ 12.2 Floyd-Warshall: เปรียบเทียบเส้นทางตรง $i \to j$ กับเส้นทางผ่าน k ว่าอันไหนสั้นกว่า

12.4.2 โค้ด Floyd-Warshall

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
#define INF 1000000000
#define V 200
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    int dist[V][V];
    // Initialize
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = (i == j) ? 0 : INF;
        }
    }
    // Read edges
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        dist[u][v] = min(dist[u][v], w);  // Store minimum weight
```

```
            // Undirected: dist[v][u] = min(dist[v][u], w);
    }
    // Floyd-Warshall: 3 nested loops
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    // Display result
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("APSP(%d,%d) = INF\n", i, j);
            else
                printf("APSP(%d,%d) = %d\n", i, j, dist[i][j]);
        }
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.4: Floyd-Warshall Algorithm

Floyd-Warshall สั้นและง่าย

ข้อดีหลักของ Floyd-Warshall คือโค้ดสั้นมาก (เพียงลูป 3 ชั้น) และหาระยะทางระหว่างจุดยอดทุกคู่ในครั้งเดียว ในการแข่งขัน ถ้า n ไม่ใหญ่เกินไป (เช่น $n \leq 400$) Floyd-Warshall มักเป็นตัวเลือกแรกสำหรับปัญหา APSP เพราะเขียนง่ายและตรวจสอบได้ง่าย

ต้องระวังลำดับลูป

ลำดับลูป **ต้อง** เป็น k นอกสุด, ตามด้วย i และ j ถ้าสลับลำดับ ขั้นตอนวิธีจะให้คำตอบผิด! จำง่ายๆ ว่า: “จุดกลาง k ต้องอยู่นอกสุด”

```
// Correct
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
```

```
// Wrong! Do not swap the order
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++)   // k innermost = wrong!
            ...
```

12.4.3 ตัวอย่างการทำงานของ Floyd-Warshall

พิจารณากราฟมีทิศทาง 5 จุดยอด ที่มีเส้นเชื่อม: $0 \to 1$ (5), $0 \to 3$ (9), $1 \to 2$ (2), $2 \to 3$ (6), $1 \to 4$ (1), $4 \to 3$ (2)

เมทริกซ์เริ่มต้น:

	0	1	2	3	4
0	0	5	INF	9	INF
1	INF	0	2	INF	1
2	INF	INF	0	6	INF
3	INF	INF	INF	0	INF
4	INF	INF	INF	2	0

รอบที่ $k = 1$ (ใช้จุด 1 เป็นจุดกลาง):

- dist[0][2] = min(INF, 5+2) = 7
- dist[0][4] = min(INF, 5+1) = 6

รอบที่ $k = 4$ (ใช้จุด 4 เป็นจุดกลาง):

- dist[1][3] = min(INF, 1+2) = 3 (ผ่าน 4)
- dist[0][3] = min(9, 6+2) = 8 (ผ่าน 4)

หลังจากรันครบทุกรอบ เราจะได้ระยะทางสั้นสุดระหว่างจุดยอดทุกคู่ เช่น dist[0][3] = 8 (เส้นทาง 0→1→4→3)

12.5 Shortest Path บน DAG

สำหรับกราฟมีทิศทางที่ไม่มีวงจร (Directed Acyclic Graph — DAG) เราสามารถหาเส้นทางสั้นสุดได้เร็วกว่า Dijkstra โดยใช้ **การเรียงลำดับทอพอโลยี (Topological Sort)** ร่วมกับการผ่อนคลาย ทำให้ได้ความซับซ้อนเพียง $\mathcal{O}(V + E)$ เท่านั้น[2]

12.5.1 หลักการ

แนวคิดหลักคือ เมื่อเราเรียงจุดยอดตามลำดับทอพอโลยีแล้ว เส้นเชื่อมทุกเส้นจะพุ่งจากจุดยอดที่มีลำดับก่อนไปยังจุดยอดที่มีลำดับหลังเสมอ ดังนั้นถ้าเราประมวลผลจุดยอดตามลำดับทอพอโลยีและผ่อนคลายเส้นเชื่อมออกทั้งหมดในแต่ละจุดยอด เราจะพิจารณาเส้นทางที่เป็นไปได้ทั้งหมดโดยไม่ต้องย้อนกลับมาประมวลผลซ้ำ

ขั้นตอนมีดังนี้:

1. ทำ Topological Sort บน DAG ได้ลำดับจุดยอด $v_0, v_1, \ldots, v_{n-1}$
2. กำหนดค่าเริ่มต้น `dist[s] = 0` และ `dist[v] = INF` สำหรับจุดยอดอื่นๆ
3. ประมวลผลจุดยอดตามลำดับทอพอโลยี สำหรับแต่ละจุดยอด u ให้ผ่อนคลายเส้นเชื่อมออกทั้งหมดของ u

เนื่องจากเราประมวลผลตามลำดับทอพอโลยี เมื่อถึงตาประมวลผลจุดยอด u ค่า `dist[u]` จะถูกต้องเป็นที่เรียบร้อยแล้ว (เพราะจุดยอดทุกจุดที่สามารถเดินทางมาถึง u ได้ถูกประมวลผลไปแล้ว) จึงไม่จำเป็นต้องใช้ Priority Queue ในการเลือกจุดยอดถัดไป ทำให้ประหยัดเวลาได้มาก

12.5.2 โค้ด Shortest Path บน DAG

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;    // (neighbor, weight)
typedef vector<ii> vii;
#define INF 1000000000
int V, E;
vector<vii> adj;
vector<bool> visited;
vector<int> topo;
// DFS-based topological sort
void topoSort(int u) {
    visited[u] = true;
    for (auto [v, w] : adj[u]) {
        if (!visited[v]) topoSort(v);
    }
    topo.push_back(u);   // Push after processing all descendants
}
int main() {
    scanf("%d %d", &V, &E);
    adj.assign(V, vii());
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
```

```
        adj[u].push_back({v, w});   // Directed edge u -> v
    }
    // Step 1: Topological Sort
    visited.assign(V, false);
    topo.clear();
    for (int i = 0; i < V; i++) {
        if (!visited[i]) topoSort(i);
    }
    reverse(topo.begin(), topo.end());   // Reverse to get correct order
    // Step 2: Initialize distances from source s
    int s;
    scanf("%d", &s);
    vector<int> dist(V, INF);
    dist[s] = 0;
    // Step 3: Process vertices in topological order
    for (int i = 0; i < V; i++) {
        int u = topo[i];
        if (dist[u] == INF) continue;   // Unreachable from source
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    // Output shortest distances
    for (int i = 0; i < V; i++) {
        if (dist[i] == INF)
            printf("dist(%d,%d) = INF\n", s, i);
        else
            printf("dist(%d,%d) = %d\n", s, i, dist[i]);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.5: Shortest Path on DAG using Topological Sort

ตัวอย่าง: Shortest Path บน DAG

พิจารณา DAG 5 จุดยอด ที่มีเส้นเชื่อม: $0 \to 1$ (น้ำหนัก 3), $0 \to 2$ (น้ำหนัก 6), $1 \to 2$ (น้ำหนัก 4), $1 \to 3$ (น้ำหนัก 4), $1 \to 4$ (น้ำหนัก 11), $2 \to 3$ (น้ำหนัก 8), $2 \to 4$ (น้ำหนัก 5), $3 \to 4$ (น้ำหนัก 2)

ลำดับทอพอโลยี: 0, 1, 2, 3, 4

เริ่มจากจุด $s = 0$:

- ประมวลผลจุด 0: dist[1] = 3, dist[2] = 6
- ประมวลผลจุด 1 (dist=3): dist[2] = min$(6, 3 + 4) = 6$, dist[3] = $3 + 4 = 7$, dist[4] = $3 + 11 = 14$
- ประมวลผลจุด 2 (dist=6): dist[3] = min$(7, 6 + 8) = 7$, dist[4] = min$(14, 6 + 5) = 11$
- ประมวลผลจุด 3 (dist=7): dist[4] = min$(11, 7 + 2) = 9$
- ประมวลผลจุด 4 (dist=9): ไม่มีเส้นเชื่อมออก

ผลลัพธ์: dist[0]=0, dist[1]=3, dist[2]=6, dist[3]=7, dist[4]=9
เส้นทางสั้นสุดจาก 0 ถึง 4 คือ $0 \to 1 \to 3 \to 4$ (น้ำหนักรวม = 3+4+2 = 9)

12.5.3 Longest Path บน DAG

ข้อดีที่สำคัญอย่างหนึ่งของการหาเส้นทางบน DAG คือ เราสามารถหา **Longest Path** ได้ด้วย! ในกราฟทั่วไป ปัญหา Longest Path เป็น **NP-hard** แต่บน DAG เราสามารถหาได้ในเวลา $\mathcal{O}(V + E)$ โดย:

- **วิธีที่ 1**: สลับเครื่องหมายน้ำหนักทุกเส้นเชื่อมเป็นค่าลบ แล้วใช้ Shortest Path บน DAG ปกติ (เนื่องจากไม่มีวงจร จะไม่เกิดวงจรน้ำหนักลบ)
- **วิธีที่ 2**: เปลี่ยนการผ่อนคลายจาก `min` เป็น `max` — กล่าวคือ `dist[v] = max(dist[v], dist[u] + w)` และกำหนดค่าเริ่มต้น `dist[s] = 0`, ค่าอื่น = $-\infty$

```
// After topological sort and initialization with -INF
// dist[s] = 0; others = -INF
for (int i = 0; i < V; i++) {
    int u = topo[i];
    if (dist[u] == -INF) continue;   // Unreachable from source
    for (auto [v, w] : adj[u]) {
        if (dist[u] + w > dist[v]) {   // Use max instead of min
            dist[v] = dist[u] + w;
        }
    }
}
```

ตัวอย่างโค้ดที่ 12.6: Longest Path on DAG (using max relaxation)

ปัญหา DP หลายข้อคือ Shortest/Longest Path บน DAG

ปัญหา Dynamic Programming หลายข้อสามารถมองเป็น “การหาเส้นทางสั้นสุดหรือยาวสุดบน DAG” ได้ — เช่นปัญหา Longest Increasing Subsequence (LIS), การหาเส้นทางที่มีค่ามากสุดในกราฟไม่มีวงจร ฯลฯ แนวคิดนี้เชื่อมโยงกับบทที่ 7 (DP on DAG) ที่กล่าวถึงการมองกราฟสถานะเป็น DAG และใช้การเรียงลำดับทอพอโลยีเพื่อคำนวณค่า DP ได้อย่างมีประสิทธิภาพ

ใช้ได้เฉพาะกับ DAG เท่านั้น!

ขั้นตอนวิธีนี้ใช้ได้เฉพาะกับกราฟที่ **ไม่มีวงจร** (DAG) เท่านั้น ถ้ากราฟมีวงจร ลำดับทอพอโลยีจะไม่มีอยู่ และขั้นตอนวิธีจะให้คำตอบผิด ในกรณีที่กราฟมีวงจร ให้ใช้ Dijkstra (ถ้าไม่มีน้ำหนักลบ) หรือ Bellman-Ford แทน

12.6 การประยุกต์ Floyd-Warshall เพิ่มเติม

นอกจากการหาระยะทางสั้นสุดแล้ว Floyd-Warshall ยังสามารถประยุกต์ใช้ได้อย่างหลากหลาย โดยการเปลี่ยนสูตรการอัปเดตเมทริกซ์ให้เหมาะกับปัญหาที่ต้องการ ในหัวข้อนี้เราจะศึกษาการประยุกต์ใช้ Floyd-Warshall 4 รูปแบบที่พบบ่อยในการแข่งขัน

12.6.1 Transitive Closure

ปัญหา **Transitive Closure** คือการตรวจสอบว่ามีเส้นทางจากจุดยอด i ไปยังจุดยอด j หรือไม่ (Reachability) โดยไม่สนใจว่าระยะทางเท่าไหร่ เราสามารถดัดแปลง Floyd-Warshall โดยใช้เมทริกซ์แบบ `bool` แทน `int` และเปลี่ยนสูตรการอัปเดตดังนี้:

$$d[i][j] = d[i][j] \vee (d[i][k] \wedge d[k][j]) \tag{12.2}$$

กล่าวคือ มีเส้นทางจาก i ถึง j ได้ ถ้ามีเส้นทางตรงจาก i ถึง j อยู่แล้ว **หรือ** มีเส้นทางจาก i ไป k และจาก k ไป j

```cpp
#include <cstdio>
using namespace std;
#define V 400
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    bool reach[V][V] = {};
    // Initialize: each vertex can reach itself
    for (int i = 0; i < n; i++) reach[i][i] = true;
    // Read edges
```

```
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        reach[u][v] = true;   // Directed edge u -> v
    }
    // Floyd-Warshall variant for Transitive Closure
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
            }
        }
    }
    // Query: can we reach from u to v?
    int q;
    scanf("%d", &q);
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        printf("%s\n", reach[u][v] ? "Yes" : "No");
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.7: Transitive Closure using Floyd-Warshall

การประยุกต์นี้มีประโยชน์มากสำหรับคำถามประเภท Reachability Query เช่น “จากเมือง A สามารถเดินทางไปเมือง B ได้หรือไม่” เมื่อต้องตอบคำถามหลายคำถาม การคำนวณ Transitive Closure ครั้งเดียวใน $\mathcal{O}(V^3)$ จะคุ้มค่ากว่าการรัน BFS/DFS ทีละคำถาม

12.6.2 Arbitrage Detection

ปัญหา **Arbitrage** เกิดขึ้นในกราฟอัตราแลกเปลี่ยนสกุลเงิน กำหนดให้มีสกุลเงิน n สกุล และอัตราแลกเปลี่ยน r_{ij} แทนจำนวนหน่วยของสกุลเงิน j ที่ได้จากการแลก 1 หน่วยของสกุลเงิน i ปัญหาคือ: เริ่มจากสกุลเงินหนึ่ง 1 หน่วย สามารถแลกวนกลับมาได้มากกว่า 1 หน่วยหรือไม่?

แนวคิด: แปลงอัตราแลกเปลี่ยนเป็นน้ำหนักเส้นเชื่อมโดยใช้ $w_{ij} = -\log(r_{ij})$ จากนั้นใช้ Floyd-Warshall ตรวจจับวงจรน้ำหนักลบ — ถ้ามีวงจรน้ำหนักลบ แสดงว่ามี Arbitrage

ทำไม? เพราะถ้าเราแลกเงินตามวงจร $i_1 \to i_2 \to \cdots \to i_k \to i_1$ ผลคูณอัตราแลกเปลี่ยนคือ:

$$r_{i_1 i_2} \times r_{i_2 i_3} \times \cdots \times r_{i_k i_1} > 1 \iff -\log(r_{i_1 i_2}) - \log(r_{i_2 i_3}) - \cdots - \log(r_{i_k i_1}) < 0 \qquad (12.3)$$

กล่าวคือ ผลคูณอัตราแลกเปลี่ยนมากกว่า 1 ก็ต่อเมื่อผลรวมของ $-\log$ มีค่าติดลบ ซึ่งก็คือวงจรน้ำหนักลบ

นั่นเอง

```cpp
#include <cstdio>
#include <cmath>
#include <algorithm>
using namespace std;
#define V 100
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    double dist[V][V];
    // Initialize
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = (i == j) ? 0.0 : 1e18;
        }
    }
    // Read exchange rates and convert to -log(weight)
    for (int i = 0; i < m; i++) {
        int u, v;
        double rate;
        scanf("%d %d %lf", &u, &v, &rate);
        dist[u][v] = -log(rate);  // Convert rate to edge weight
    }
    // Floyd-Warshall
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
    // Check for negative cycle (Arbitrage exists)
    bool arbitrage = false;
    for (int i = 0; i < n; i++) {
        if (dist[i][i] < -1e-9) {  // dist[i][i] < 0 means negative cycle
            arbitrage = true;
            break;
        }
    }
    printf("%s\n", arbitrage ? "Arbitrage detected!" : "No arbitrage");
    return 0;
```

```
}
```

ตัวอย่างโค้ดที่ 12.8: Arbitrage Detection using Floyd-Warshall

12.6.3 Minimax Path

ปัญหา **Minimax Path** คือการหาเส้นทางจาก i ถึง j ที่ทำให้ **น้ำหนักของเส้นเชื่อมที่หนักที่สุดบนเส้นทางมีค่าน้อยที่สุด** กล่าวคือ เราต้องการ “เส้นทางที่ปลอดภัยที่สุด” — เส้นทางที่เส้นเชื่อมที่แย่ที่สุดยังดีที่สุดเมื่อเทียบกับเส้นทางอื่น

เราสามารถดัดแปลง Floyd-Warshall โดยเปลี่ยนสูตรการอัปเดตเป็น:

$$d[i][j] = \min\big(d[i][j],\ \max(d[i][k],\ d[k][j])\big) \tag{12.4}$$

สูตรนี้มีความหมายว่า: ถ้าเราเดินจาก i ไป j ผ่าน k น้ำหนักสูงสุดบนเส้นทางนี้คือ `max(d[i][k], d[k][j])` และเราเลือกเส้นทางที่ทำให้ค่านี้น้อยที่สุด

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
#define V 400
#define INF 1000000000
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    int d[V][V];
    // Initialize
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            d[i][j] = (i == j) ? 0 : INF;
        }
    }
    // Read edges
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        d[u][v] = min(d[u][v], w);   // Store minimum weight edge
        d[v][u] = min(d[v][u], w);   // Undirected
    }
    // Floyd-Warshall variant for Minimax Path
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
```

```
                d[i][j] = min(d[i][j], max(d[i][k], d[k][j]));
            }
        }
    }
    // Query: minimax distance from u to v
    int q;
    scanf("%d", &q);
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        printf("Minimax(%d,%d) = %d\n", u, v, d[u][v]);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.9: Minimax Path using Floyd-Warshall

ตัวอย่าง: Minimax Path

พิจารณากราฟไม่มีทิศทาง 4 จุดยอด ที่มีเส้นเชื่อม: $0-1$ (น้ำหนัก 7), $0-2$ (น้ำหนัก 3), $1-2$ (น้ำหนัก 4), $1-3$ (น้ำหนัก 5), $2-3$ (น้ำหนัก 8)
ต้องการหา Minimax Path จาก 0 ถึง 3:

- เส้นทาง $0 \to 1 \to 3$: น้ำหนักสูงสุด = $\max(7,5) = 7$
- เส้นทาง $0 \to 2 \to 3$: น้ำหนักสูงสุด = $\max(3,8) = 8$
- เส้นทาง $0 \to 2 \to 1 \to 3$: น้ำหนักสูงสุด = $\max(3,4,5) = 5$

Minimax Path = $0 \to 2 \to 1 \to 3$ โดยมีน้ำหนักสูงสุดบนเส้นทางเท่ากับ 5 ซึ่งน้อยที่สุดเมื่อเทียบกับเส้นทางอื่น

12.6.4 Maximin Path

ปัญหา **Maximin Path** คือสวนทางของ Minimax Path — หาเส้นทางจาก i ถึง j ที่ทำให้ **น้ำหนักของเส้นเชื่อมที่เบาที่สุดบนเส้นทางมีค่ามากที่สุด** กล่าวคือ เราต้องการ “เส้นทางที่ไม่มีจุดอ่อน” — เส้นเชื่อมที่แย่ที่สุดบนเส้นทางยังดีที่สุดเมื่อเทียบกับเส้นทางอื่น มักเรียกปัญหานี้ว่า “Bottleneck Path”

สูตร Floyd-Warshall สำหรับ Maximin Path:

$$d[i][j] = \max\big(d[i][j],\ \min(d[i][k],\ d[k][j])\big) \tag{12.5}$$

```
#include <cstdio>
#include <algorithm>
```

```cpp
using namespace std;
#define V 400
#define NEG_INF -1
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    int d[V][V];
    // Initialize
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            d[i][j] = (i == j) ? 1e9 : NEG_INF;   // Self = large, others = -INF
        }
    }
    // Read edges
    for (int i = 0; i < m; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        d[u][v] = max(d[u][v], w);   // Store maximum weight edge
        d[v][u] = max(d[v][u], w);   // Undirected
    }
    // Floyd-Warshall variant for Maximin Path
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                d[i][j] = max(d[i][j], min(d[i][k], d[k][j]));
            }
        }
    }
    // Query: maximin distance from u to v
    int q;
    scanf("%d", &q);
    while (q--) {
        int u, v;
        scanf("%d %d", &u, &v);
        if (d[u][v] == NEG_INF)
            printf("No path from %d to %d\n", u, v);
        else
            printf("Maximin(%d,%d) = %d\n", u, v, d[u][v]);
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 12.10: Maximin Path using Floyd-Warshall

ตัวอย่างปัญหา Maximin Path ที่พบบ่อย: หาเส้นทางขนส่งที่ทำให้สะพานที่รับน้ำหนักได้น้อยที่สุดบนเส้นทางยังรับน้ำหนักได้มากที่สุด (เช่น ต้องการขนส่งสินค้าหนักที่สุดเท่าที่เป็นไปได้ โดยสินค้าจะผ่านสะพานทุกสะพานบนเส้นทาง)

จำสูตร Minimax และ Maximin ง่าย ๆ

เวลาเจอโจทย์ที่บอกว่า “ทำให้ค่ามากสุดน้อยที่สุด” (Minimize the Maximum) ให้นึกถึง **Minimax Path**: `min(max(...))` และถ้าบอกว่า “ทำให้ค่าน้อยสุดมากที่สุด” (Maximize the Minimum) ให้นึกถึง **Maximin Path**: `max(min(...))` — ทั้งสองอย่างสามารถแก้ได้ด้วย Floyd-Warshall ดัดแปลง!

12.7 เปรียบเทียบขั้นตอนวิธี Shortest Path

ตารางที่ 12.2 เปรียบเทียบขั้นตอนวิธี Shortest Path ทั้ง 4 ตัว

คุณสมบัติ	Bellman-Ford	Dijkstra	Floyd-Warshall
ประเภทปัญหา	SSSP	SSSP	APSP
ความซับซ้อน	$\mathcal{O}(VE)$	$\mathcal{O}((V+E)\log V)$	$\mathcal{O}(V^3)$
น้ำหนักลบ	รองรับ	**ไม่รองรับ**	รองรับ
ตรวจจับวงจรลบ	ได้	ไม่ได้	ได้ (เพิ่มเงื่อนไข)
โครงสร้างข้อมูล	Edge List	Adj List + PQ	Adj Matrix
เหมาะกับ	น้ำหนักลบ	กราฟใหญ่ไม่มีน้ำหนักลบ	กราฟเล็ก
ความยาวโค้ด	ปานกลาง	ปานกลาง	สั้นมาก

คำแนะนำการเลือกขั้นตอนวิธี

- ถ้าโจทย์เป็น SSSP และกราฟ **ไม่มีน้ำหนักลบ** ☒ ใช้ **Dijkstra** (เร็วสุด)
- ถ้าโจทย์เป็น SSSP และกราฟ **มีน้ำหนักลบ** หรือต้องการ **ตรวจจับวงจรลบ** ☒ ใช้ **Bellman-Ford**
- ถ้าโจทย์เป็น APSP หรือต้องการระยะทางระหว่างจุดยอด **หลายคู่** และ n ไม่ใหญ่ ($n \leq 400$) ☒ ใช้ **Floyd-Warshall**
- ถ้ากราฟไม่มีน้ำหนัก ☒ ใช้ **BFS** (เร็วสุด $\mathcal{O}(V+E)$)

ตัวอย่าง: เลือกขั้นตอนวิธีอย่างไร

- โจทย์: หาระยะทางจากกรุงเทพฯ ไปจังหวัดอื่นๆ ทั้งหมด, $n = 10^5$, $m = 10^5$, น้ำหนัก ≥ 0 ☒ **Dijkstra**
- โจทย์: หาระยะทางระหว่างเมืองทุกคู่, $n = 200$, มีน้ำหนักลบ ☒ **Floyd-Warshall**
- โจทย์: ตรวจสอบว่ามี "ช่องโหว่อัตราแลกเปลี่ยน" (Negative Cycle) หรือไม่ ☒ **Bellman-Ford**
- โจทย์: หาจำนวนเส้นเชื่อมน้อยสุดจากจุด A ไป B (กราฟไม่มีน้ำหนัก) ☒ **BFS**

12.8 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนขั้นตอนวิธี Shortest Path:

โจทย์แนะนำสำหรับบทที่ 12

Bellman-Ford:

- CSES High Score — ตรวจจับ Positive Cycle[12]
- UVA 558 — Wormholes (Negative Cycle Detection)[20]
- Codeforces 20C — Dijkstra? (Bellman-Ford for small graphs)[10]

Dijkstra:

- CSES Shortest Routes I — Dijkstra พื้นฐาน[12]
- CSES Shortest Routes II — Floyd-Warshall[12]
- Codeforces 20C — Dijkstra?[10]
- UVA 10986 — Sending email (Dijkstra)[20]

Floyd-Warshall:

- UVA 104 — Arbitrage (FW variant)[20]
- UVA 10724 — Road Construction (FW + check)[20]
- CSES Shortest Routes II — APSP[12]

Shortest Path on DAG:

- CSES Longest Flight Route — DP on DAG[12]
- CSES Game Routes — DP on DAG[12]

12.9 สรุปบทที่ 12

ในบทนี้ เราได้ศึกษาขั้นตอนวิธีหาเส้นทางสั้นสุดทั้ง 4 ตัวหลัก ได้แก่ Bellman-Ford, Dijkstra, Floyd-Warshall และ Shortest Path on DAG แต่ละขั้นตอนวิธีมีข้อจำกัดและจุดแข็งที่แตกต่างกัน การเลือกใช้ขั้นตอนวิธีที่เหมาะสมขึ้นอยู่กับลักษณะของกราฟและข้อจำกัดของโจทย์

สรุปสิ่งสำคัญของบทที่ 12

1. **Bellman-Ford** รองรับน้ำหนักลบ ตรวจจับ Negative Cycle ได้ แต่ช้า $\mathcal{O}(VE)$
2. **Dijkstra** เร็ว $\mathcal{O}((V+E)\log V)$ แต่ไม่รองรับน้ำหนักลบ
3. **Floyd-Warshall** หาทุกคู่เส้นทางสั้นสุด $\mathcal{O}(V^3)$ เหมาะกับกราฟเล็ก
4. **Shortest Path on DAG** ทำได้ใน $\mathcal{O}(V+E)$ เมื่อกราฟเป็น DAG
5. **เลือกขั้นตอนวิธีจาก constraints** — ดูขนาด V, E และว่ามีน้ำหนักลบหรือไม่

Chapter 13

Union-Find และต้นไม้แผ่คลุมต่ำสุด

ในบทนี้เราจะศึกษาโครงสร้างข้อมูล Union-Find (หรือ Disjoint Set Union) ซึ่งเป็นโครงสร้างที่มีประสิทธิภาพสูงในการจัดการกับเซตที่แยกจากกัน[2, 11] และนำไปประยุกต์ใช้กับปัญหา Minimum Spanning Tree (MST) ซึ่งเป็นหนึ่งในปัญหาคลาสสิกที่สำคัญที่สุดในทฤษฎีกราฟ Union-Find มีประโยชน์มากในการแข่งขัน เพราะใช้แก้ปัญหาได้หลากหลาย ตั้งแต่การหาองค์ประกอบเชื่อมโยง การตรวจจับวงจร ไปจนถึงการจัดกลุ่มวัตถุ

13.1 Union-Find (Disjoint Set Union)

Union-Find เป็นโครงสร้างข้อมูลที่จัดการกับ **เซตไม่มีส่วนร่วม (Disjoint Sets)** — เซตหลายเซตที่ไม่มีสมาชิกร่วมกันเลย โครงสร้างนี้รองรับการดำเนินการสำคัญ 2 อย่าง:

1. **Union:** รวมเซตสองเซตเข้าด้วยกัน
2. **Find:** หาว่าสมาชิกตัวหนึ่งอยู่ในเซตใด (หรือหาตัวแทนของเซต)

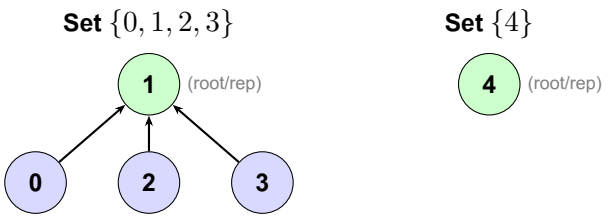
13.1.1 แรงบันดาลใจ

พิจารณาปัญหาต่อไปนี้: มีเมือง n เมือง และถนนที่สร้างขึ้นทีละสาย หลังจากสร้างถนนแต่ละสาย เราต้องการตอบคำถามว่า “เมือง a และเมือง b เชื่อมโยงกันหรือไม่?” นี่คือปัญหาที่ Union-Find แก้ได้อย่างมีประสิทธิภาพ

แนวคิดคือเริ่มต้นด้วยเซต n เซต (เมืองละเซต) เมื่อสร้างถนนเชื่อมเมือง a และ b เรารวมเซตของ a กับเซตของ b เข้าด้วยกัน (Union) เพื่อตรวจสอบว่า a และ b เชื่อมโยงกันหรือไม่ เราตรวจสอบว่า a และ b อยู่ในเซตเดียวกันหรือไม่ (Find)

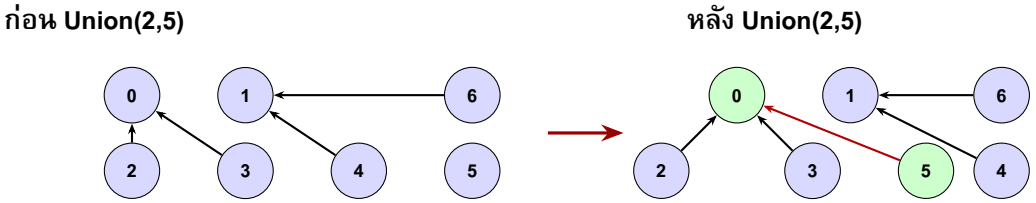
13.1.2 การแทนเซตด้วยต้นไม้

Union-Find แทนเซตแต่ละเซตด้วย **ต้นไม้** โดยรากของต้นไม้คือ **ตัวแทน (Representative)** ของเซต เราใช้แถวลำดับ `p[i]` เก็บพ่อแม่ของจุดยอด i ถ้า `p[i] = i` แสดงว่า i เป็นรากของต้นไม้ (ตัวแทนของเซต)



รูปที่ 13.1 ตัวอย่างโครงสร้าง Union-Find: เซต $\{0, 1, 2, 3\}$ มีรากเป็น 1 และเซต $\{4\}$ เป็นเซตเดี่ยว

รูปที่ 13.2 แสดงตัวอย่างโครงสร้าง Union-Find ก่อนและหลังการรวมกลุ่ม Union(2, 5)



รูปที่ 13.2 Union-Find: ก่อนและหลัง Union(2,5) — จุดยอด 5 เปลี่ยนไปชี้รากของจุดยอด 2

13.1.3 การดำเนินการพื้นฐาน

1. การเริ่มต้น (Init): กำหนดให้สมาชิกทุกตัวเป็นรากของต้นไม้ตัวเอง:

```
vi p, ranks;
void initSet(int N) {
    p.assign(N, 0);
    ranks.assign(N, 0);
    for (int i = 0; i < N; i++)
        p[i] = i;   // Everyone is its own root
}
```

ตัวอย่างโค้ดที่ 13.1: การเริ่มต้น Union-Find

2. การหาตัวแทน (Find): ติดตามพ่อแม่ขึ้นไปจนถึงราก:

```
int findSet(int i) {
    return (p[i] == i) ? i : (p[i] = findSet(p[i]));
}
```

ตัวอย่างโค้ดที่ 13.2: ฟังก์ชัน Find

3. การรวมเซต (Union): รวมเซตของ i กับเซตของ j:

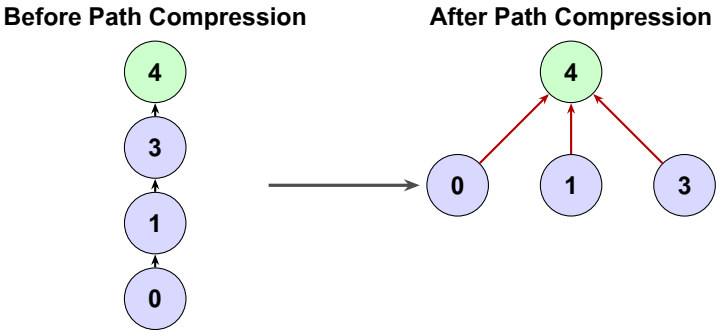
```
bool isSameSet(int i, int j) {
    return findSet(i) == findSet(j);
```

```
}
void unionSet(int i, int j) {
    if (!isSameSet(i, j)) {
        int x = findSet(i), y = findSet(j);
        if (ranks[x] > ranks[y]) {
            p[y] = x;
        } else {
            p[x] = y;
            if (ranks[x] == ranks[y])
                ranks[y]++;
        }
    }
}
```

ตัวอย่างโค้ดที่ 13.3: ฟังก์ชัน Union

13.1.4 การเพิ่มประสิทธิภาพ: Path Compression

Path Compression คือการตั้งค่าพ่อแม่ของทุกจุดยอดที่ผ่านระหว่าง Find ให้ชี้ตรงไปที่ราก ทำให้การเรียก Find ครั้งต่อไปเร็วขึ้นมาก เพราะไม่ต้องไล่ขึ้นทีละขั้น



รูปที่ 13.3 Path Compression: จุดยอด 0, 1, 3 ที่เคยเป็น chain ลึก ถูกแบนให้ชี้ตรงไปราก (4) ทันที (เส้นสีแดง = เส้นใหม่หลังบีบอัด)

สังเกตในโค้ด: `p[i] = findSet(p[i])` คือ Path Compression — เมื่อเรียก `findSet(i)` มันจะหาราก แล้วตั้ง `p[i]` ให้ชี้ตรงไปที่รากทันที

13.1.5 การเพิ่มประสิทธิภาพ: Union by Rank

Union by Rank คือการเลือกว่าจะต่อต้นไม้ต้นไหนเข้าใส่ต้นไหน โดยต่อต้นไม้ที่เตี้ยกว่าเข้าใส่ต้นไม้ที่สูงกว่าเสมอ ทำให้ต้นไม้ไม่สูงเกินไป

ทำไมต้องใช้ทั้งสองเทคนิค?

เมื่อใช้ทั้ง **Path Compression** และ **Union by Rank** ร่วมกัน ความซับซ้อนของแต่ละการดำเนินการ (Find/Union) จะเป็น $\mathcal{O}(\alpha(n))$ โดย α คือฟังก์ชัน Ackermann ผกผัน ซึ่งเติบโตช้ามากจนสำหรับ n ทุกค่าในทางปฏิบัติ $\alpha(n) \leq 4$ ดังนั้นเราสามารถมองว่าแต่ละการดำเนินการใช้เวลา $\mathcal{O}(1)$ ได้

13.1.6 โค้ด Union-Find ฉบับสมบูรณ์

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef vector<int> vi;
vi p, ranks;
void initSet(int N) {
    ranks.assign(N, 0);
    p.assign(N, 0);
    for (int i = 0; i < N; i++) p[i] = i;
}
int findSet(int i) {
    return (p[i] == i) ? i : (p[i] = findSet(p[i]));
}
bool isSameSet(int i, int j) {
    return findSet(i) == findSet(j);
}
void unionSet(int i, int j) {
    if (!isSameSet(i, j)) {
        int x = findSet(i), y = findSet(j);
        if (ranks[x] > ranks[y]) {
            p[y] = x;
        } else {
            p[x] = y;
            if (ranks[x] == ranks[y])
                ranks[y]++;
        }
    }
}
int main() {
    int n = 5;
    initSet(n);
    // Start: {0}, {1}, {2}, {3}, {4}
    unionSet(0, 1);  // Union set 0 and 1: {0,1}, {2}, {3}, {4}
    unionSet(2, 3);  // Union set 2 and 3: {0,1}, {2,3}, {4}
```

```
    unionSet(0, 2);  // Union set 0 and 2: {0,1,2,3}, {4}
    printf("isSameSet(0, 3) = %d\n", isSameSet(0, 3));  // 1 (true)
    printf("isSameSet(0, 4) = %d\n", isSameSet(0, 4));  // 0 (false)
    printf("findSet(0) = %d\n", findSet(0));            // Root of the set
    printf("findSet(4) = %d\n", findSet(4));            // 4 (alone)
    return 0;
}
```

ตัวอย่างโค้ดที่ 13.4: Union-Find ฉบับสมบูรณ์

ตัวอย่าง: การทำงานของ Union-Find ทีละขั้นตอน

เริ่มต้น: $p = [0, 1, 2, 3, 4]$, $ranks = [0, 0, 0, 0, 0]$

1. `unionSet(0, 1)`: ranks เท่ากัน ⊠ $p[0] = 1$, $ranks[1] = 1$
2. `unionSet(2, 3)`: ranks เท่ากัน ⊠ $p[2] = 3$, $ranks[3] = 1$
3. `unionSet(0, 2)`: findSet(0)=1, findSet(2)=3, ranks[1]=1=ranks[3] ⊠ $p[1] = 3$, $ranks[3] = 2$

ผลลัพธ์: จุดยอด 0,1,2,3 อยู่ในเซตเดียวกัน (รากคือ 3) ส่วนจุดยอด 4 ยังอยู่คนเดียว (รากคือ 4) — ดังนั้น `isSameSet(0, 4)` จะคืนค่า false

13.2 Minimum Spanning Tree (MST)

ปัญหา Minimum Spanning Tree (MST) เป็นหนึ่งในปัญหาสำคัญที่สุดในทฤษฎีกราฟ[2, 3] มีประยุกต์ใช้ในหลายสาขา เช่น การออกแบบเครือข่าย การวางท่อ และการจัดเส้นทางขนส่ง

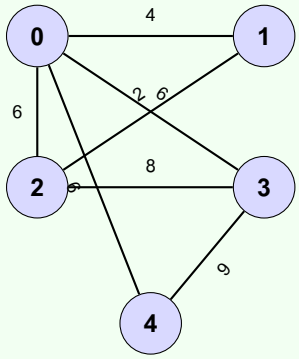
13.2.1 นิยาม MST

นิยาม 13.1 (Spanning Tree). Spanning Tree ของกราฟเชื่อมโยง $G = (V, E)$ คือกราฟย่อย $T = (V, E')$ ที่เป็นต้นไม้และครอบคลุมจุดยอดทุกจุด กล่าวคือ T มีจุดยอดทุกจุดของ G และมีเส้นเชื่อมเพียง $|V| - 1$ เส้น โดยไม่มีวงจร

นิยาม 13.2 (Minimum Spanning Tree — MST). MST ของกราฟเชื่อมโยงมีน้ำหนัก G คือ Spanning Tree ที่มีผลรวมน้ำหนักของเส้นเชื่อมต่ำสุด กล่าวคือ $\sum_{e \in E'} w(e)$ มีค่าน้อยที่สุด

ตัวอย่าง: MST

กำหนดกราฟ 5 จุดยอด 7 เส้นเชื่อม ดังรูปที่ 13.4



รูปที่ 13.4 ตัวอย่างกราฟ 5 จุดยอด 7 เส้นเชื่อม สำหรับคำนวณ MST

เส้นเชื่อม: (0,1)=4, (0,2)=6, (1,2)=2, (2,3)=8, (0,3)=6, (0,4)=6, (3,4)=9
MST เลือกเส้นเชื่อม: (1,2)=2, (0,1)=4, (0,3)=6, (0,4)=6 ⊠ น้ำหนักรวม = 18

MST อาจไม่มีเพียงคำตอบเดียว

กราฟหนึ่งกราฟอาจมี MST หลายต้นที่มีน้ำหนักรวมเท่ากัน แต่ถ้าน้ำหนักของเส้นเชื่อมทุกเส้นแตกต่างกันหมด MST จะมีเพียงคำตอบเดียว ในการแข่งขัน โจทย์มักถามน้ำหนักรวมของ MST ซึ่งไม่ขึ้นกับว่าเลือก MST ต้นไหน

13.3 Kruskal's Algorithm

Kruskal's Algorithm เป็นขั้นตอนวิธีหา MST ที่ใช้ **Union-Find** เป็นโครงสร้างข้อมูลหลัก[2] แนวคิดง่ายและสวยงาม: เรียงเส้นเชื่อมตามน้ำหนักจากน้อยไปมาก แล้วเลือกเส้นเชื่อมทีละเส้น โดยเลือกเฉพาะเส้นที่ **ไม่ทำให้เกิดวงจร**

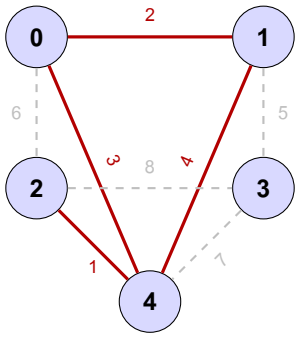
13.3.1 หลักการ

ขั้นตอนของ Kruskal's Algorithm มีดังนี้:

1. เรียงเส้นเชื่อมทั้งหมดตามน้ำหนักจากน้อยไปมาก
2. วนลูปพิจารณาเส้นเชื่อมแต่ละเส้นตามลำดับ:
   - ถ้าเส้นเชื่อม (u, v) ยังไม่ทำให้เกิดวงจร (คือ u และ v ยังไม่อยู่ในเซตเดียวกัน) ⊠ เลือกเส้นเชื่อมนี้และรวมเซตของ u กับ v
   - ถ้าเส้นเชื่อม (u, v) ทำให้เกิดวงจร (คือ u และ v อยู่ในเซตเดียวกันแล้ว) ⊠ ข้าม
3. หยุดเมื่อเลือกเส้นเชื่อมครบ $n - 1$ เส้น

การตรวจสอบว่าเส้นเชื่อมทำให้เกิดวงจรหรือไม่ ทำได้โดยใช้ Union-Find: ถ้า `findSet(u) == findSet(v)` แสดงว่า u และ v อยู่ในเซตเดียวกันแล้ว การเพิ่มเส้นเชื่อม (u, v) จะทำให้เกิดวงจร

รูปที่ 13.5 แสดงตัวอย่างกราฟและ Minimum Spanning Tree ที่ได้จาก Kruskal's Algorithm (เส้นหนาสีแดง = เส้นเชื่อมใน MST)



รูปที่ 13.5 Kruskal's Algorithm: เส้นเชื่อมใน MST (สีแดง) เลือกตามน้ำหนักน้อยสุด: 2–4 (1), 0–1 (2), 0–4 (3), 1–4 (4) น้ำหนักรวม = 10

13.3.2 โค้ด Kruskal's Algorithm

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;
typedef vector<int> vi;
typedef vector<ii> vii;
vi p, ranks;
void initSet(int N) {
    ranks.assign(N, 0);
    p.assign(N, 0);
    for (int i = 0; i < N; i++) p[i] = i;
}
int findSet(int i) {
    return (p[i] == i) ? i : (p[i] = findSet(p[i]));
}
bool isSameSet(int i, int j) {
    return findSet(i) == findSet(j);
}
void unionSet(int i, int j) {
    if (!isSameSet(i, j)) {
        int x = findSet(i), y = findSet(j);
        if (ranks[x] > ranks[y]) p[y] = x;
        else {
            p[x] = y;
            if (ranks[x] == ranks[y]) ranks[y]++;
        }
```

```
    }
}
int main() {
    int V, E;
    scanf("%d %d", &V, &E);
    // Edge List: (weight, (u, v))
    vector<pair<int, ii>> EdgeList;
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        EdgeList.push_back({w, {u, v}});
    }
    // Sort edges by weight
    sort(EdgeList.begin(), EdgeList.end());
    initSet(V);
    int mst_cost = 0;
    int numEdges = 0;
    for (int i = 0; i < E; i++) {
        auto [w, uv] = EdgeList[i];
        auto [u, v] = uv;
        if (!isSameSet(u, v)) {
            mst_cost += w;          // Add weight
            unionSet(u, v);         // Union sets
            numEdges++;
            if (numEdges == V - 1) break;   // Complete
        }
    }
    printf("MST cost = %d (Kruskal's)\n", mst_cost);
    return 0;
}
```

ตัวอย่างโค้ดที่ 13.5: Kruskal's Algorithm

ตัวอย่าง: การทำงานของ Kruskal's Algorithm

กำหนดกราฟ 5 จุดยอด 7 เส้นเชื่อม:

```
Edge    Weight
(1,2)      2      <- pick (different sets)
(0,1)      4      <- pick
(0,2)      6      x skip (0,1,2 in same set -> cycle)
(0,3)      6      <- pick
```

```
(0,4)      6      <- pick (4 edges = V-1)
(2,3)      8      x no need to consider
(3,4)      9      x no need to consider
```

MST cost = 2 + 4 + 6 + 6 = 18

13.3.3 ความซับซ้อนของ Kruskal's

ตารางที่ 13.1 ความซับซ้อนของ Kruskal's Algorithm

ขั้นตอน	ความซับซ้อน
เรียงเส้นเชื่อม	$\mathcal{O}(E \log E)$
วนลูปพิจารณาเส้นเชื่อม + Union-Find	$\mathcal{O}(E \cdot \alpha(V)) \approx \mathcal{O}(E)$
รวม	$\mathcal{O}(E \log E) = \mathcal{O}(E \log V)$

13.4 Prim's Algorithm

Prim's Algorithm เป็นขั้นตอนวิธีหา MST อีกวิธีหนึ่ง[2] ที่ใช้ **Priority Queue** เป็นโครงสร้างข้อมูลหลัก แนวคิดคือเริ่มจากจุดยอดใดจุดหนึ่ง แล้ว "ขยาย" MST ออกไปทีละเส้นเชื่อม โดยเลือกเส้นเชื่อมที่มีน้ำหนักน้อยที่สุดที่เชื่อมจากจุดยอดที่อยู่ใน MST แล้วไปยังจุดยอดที่ยังไม่อยู่ใน MST

13.4.1 หลักการ

ขั้นตอนของ Prim's Algorithm มีดังนี้:

1. เริ่มจากจุดยอดใดจุดหนึ่ง (เช่น จุด 0) ทำเครื่องหมายว่า "อยู่ใน MST แล้ว"
2. ใส่เส้นเชื่อมทั้งหมดที่ออกจากจุดยอดนั้นลงใน Priority Queue (เรียงตามน้ำหนัก)
3. วนลูป: ดึงเส้นเชื่อมที่มีน้ำหนักน้อยสุดจาก Priority Queue
   - ถ้าปลายทางของเส้นเชื่อมยังไม่อยู่ใน MST ☒ เลือกเส้นเชื่อมนี้ เพิ่มน้ำหนักลง MST cost ทำเครื่องหมายปลายทางว่าอยู่ใน MST แล้ว ใส่เส้นเชื่อมจากปลายทางนั้นลง Priority Queue
   - ถ้าปลายทางอยู่ใน MST แล้ว ☒ ข้าม
4. ทำซ้ำจนกว่า Priority Queue จะว่าง

13.4.2 โค้ด Prim's Algorithm

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;
typedef vector<int> vi;
typedef vector<ii> vii;
vector<vii> adj;
vi taken;
priority_queue<ii> pq;    // max-heap (use negative values for min-heap)
void process(int vtx) {
    taken[vtx] = 1;
    for (int j = 0; j < (int)adj[vtx].size(); j++) {
        ii v = adj[vtx][j];
        if (!taken[v.first])
            pq.push(ii(-v.second, -v.first));    // Negative values for min-heap
    }
}
int main() {
    int V, E;
    scanf("%d %d", &V, &E);
    adj.assign(V, vii());
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});    // Undirected graph
    }
    int mst_cost = 0;
    taken.assign(V, 0);
    process(0);  // Start from vertex 0
    while (!pq.empty()) {
        ii front = pq.top(); pq.pop();
        int u = -front.second;
        int w = -front.first;     // Convert back from negative
        if (!taken[u]) {
            mst_cost += w;
            process(u);
        }
    }
    printf("MST cost = %d (Prim's)\n", mst_cost);
    return 0;
}
```

ตัวอย่างโค้ดที่ 13.6: Prim's Algorithm

เทคนิคค่าลบสำหรับ min-heap

C++ `priority_queue` เรียงจากมากไปน้อยโดยค่าเริ่มต้น (max-heap) เพื่อให้ได้ min-heap เราเก็บ **ค่าลบ** ของน้ำหนักและจุดยอด ดึงออกมาแล้วคูณ -1 กลับ หรืออีกวิธีคือประกาศ:

```
priority_queue<ii, vector<ii>, greater<ii>> pq;   // min-heap directly
```

13.4.3 ความซับซ้อนของ Prim's

ตารางที่ 13.2 ความซับซ้อนของ Prim's Algorithm

ขั้นตอน	ความซับซ้อน
วนลูปเส้นเชื่อม + Priority Queue	$\mathcal{O}(E \log E) = \mathcal{O}(E \log V)$

13.5 เปรียบเทียบ Kruskal's กับ Prim's

ตารางที่ 13.3 เปรียบเทียบ Kruskal's กับ Prim's

คุณสมบัติ	Kruskal's	Prim's
กลยุทธ์	เรียงเส้นเชื่อม เลือกเส้นน้ำหนักน้อยที่ไม่ทำให้เกิดวงจร	ขยายจากจุดเริ่มต้น เลือกเส้นน้ำหนักน้อยที่ออกจาก MST
โครงสร้างข้อมูล	Edge List + Union-Find	Adjacency List + Priority Queue
การตรวจวงจร	Union-Find (`isSameSet`)	แถวลำดับ `taken[]`
ความซับซ้อน	$\mathcal{O}(E \log V)$	$\mathcal{O}(E \log V)$
เหมาะกับ	กราฟเบาบาง (Sparse)	กราฟหนาแน่น (Dense)

เลือก Kruskal's หรือ Prim's?

- ทั้งสองวิธีมีความซับซ้อนเท่ากัน ($\mathcal{O}(E \log V)$) ในกราฟทั่วไป
- **Kruskal's** ง่ายกว่าถ้ามี Union-Find อยู่แล้ว และเหมาะกับ Edge List
- **Prim's** เหมาะกว่าถ้ากราฟเก็บเป็น Adjacency List อยู่แล้ว
- ในการแข่งขัน Kruskal's นิยมใช้มากกว่าเพราะโค้ดสั้นและใช้ Union-Find ซึ่งมีประโยชน์หลายอย่าง

13.6 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝน Union-Find และ Minimum Spanning Tree:

โจทย์แนะนำสำหรับบทที่ 10

Union-Find:

- CSES Building Roads — เชื่อมกราฟด้วย Union-Find[12]
- CSES Road Construction — MST เบื้องต้น[12]
- Codeforces 1213F — Unstable String Sort (UF application)[10]

Kruskal's MST:

- CSES Road Construction — Kruskal's พื้นฐาน[12]
- UVA 908 — Re-connecting Computer Sites (MST)[20]
- UVA 10034 — Freckles (MST on points)[20]
- Codeforces 1513D — GCD and MST (MST variant)[10]

Prim's MST:

- UVA 10034 — Freckles (Prim's on dense graph)[20]
- UVA 534 — Frogger (Minimax path = MST edge)[20]
- Codeforces 472D — Design Tutorial: Inverse the Problem (MST verification)[10]

13.7 สรุปบทที่ 13

ในบทนี้ เราได้ศึกษาโครงสร้างข้อมูล Union-Find และขั้นตอนวิธี Minimum Spanning Tree ทั้งสองวิธีหลัก ได้แก่ Kruskal's Algorithm และ Prim's Algorithm การเลือกใช้ขั้นตอนวิธีขึ้นอยู่กับลักษณะของกราฟและข้อจำกัดของโจทย์

สรุปสิ่งสำคัญของบทที่ 13

1. **Union-Find** จัดการเซตไม่มีส่วนร่วมได้เร็วมาก $\mathcal{O}(\alpha(n)) \approx \mathcal{O}(1)$ พร้อม Path Compression และ Union by Rank
2. **Kruskal's** เรียงเส้นเชื่อมตามน้ำหนักแล้วเลือกทีละเส้น $\mathcal{O}(E \log E)$
3. **Prim's** เริ่มจากจุดใดจุดหนึ่งแล้วขยายต้นไม้ $\mathcal{O}(E \log V)$ ด้วย Priority Queue

4. **Kruskal's เหมาะกับกราฟเบาบาง** (Sparse) ส่วน Prim's เหมาะกับกราฟหนาแน่น (Dense)
5. **Union-Find ใช้ตรวจสอบวงจร** ใน Kruskal's และใช้ในโจทย์หลายประเภท

Chapter 14

หัวข้อกราฟเพิ่มเติม

ในบทก่อนหน้า เราได้ศึกษาขั้นตอนวิธีกราฟพื้นฐานเช่น DFS, BFS, Shortest Path และ MST แล้ว ในบทนี้ เราจะเจาะลึกลงไปในหัวข้อกราฟขั้นสูงที่สำคัญอีก 5 หัวข้อ ได้แก่ การตรวจสอบกราฟทวิภาคี (Bipartite Check) การจำแนกประเภทของเส้นเชื่อมและจุดตัด/สะพาน (Articulation Points & Bridges) การหาส่วนประกอบเชื่อมโยงเข้มด้วยขั้นตอนวิธี Tarjan การหาส่วนประกอบเชื่อมโยงเข้มด้วยขั้นตอนวิธี Kosaraju และปัญหา 2-SAT หัวข้อเหล่านี้เป็นพื้นฐานสำคัญสำหรับโจทย์กราฟระดับกลางถึงยากในการแข่งขัน

14.1 การตรวจสอบกราฟทวิภาคี (Bipartite Check)

ในบทที่ 8 เราได้นิยามกราฟทวิภาคี (Bipartite Graph) ไปแล้ว[2] กราฟทวิภาคีคือกราฟที่สามารถแบ่งจุดยอดออกเป็น 2 เซตโดยไม่มีเส้นเชื่อมระหว่างจุดยอดในเซตเดียวกัน ในหัวข้อนี้ เราจะเรียนรู้วิธีตรวจสอบว่ากราฟเป็นกราฟทวิภาคีหรือไม่ โดยใช้ BFS หรือ DFS ก็ได้

14.1.1 หลักการ: ระบายสี 2 สี

วิธีตรวจสอบกราฟทวิภาคีคือพยายามระบายสีจุดยอดด้วย 2 สี (สมมติสี 0 และสี 1) โดยเริ่มจากจุดยอดใดจุดหนึ่ง ระบายสี 0 แล้วระบายสีจุดข้างเคียงทุกจุดเป็นสี 1 สลับกันไปเรื่อยๆ ถ้าทุกจุดสามารถระบายได้โดยไม่ขัดแย้ง (จุดข้างเคียงทุกคู่มีสีต่างกัน) กราฟเป็นกราฟทวิภาคี แต่ถ้าเจอจุดข้างเคียงที่มีสีเดียวกัน กราฟไม่ใช่กราฟทวิภาคี

ทฤษฎีบท 14.1 (เงื่อนไขกราฟทวิภาคี). กราฟเป็นกราฟทวิภาคีก็ต่อเมื่อ **ไม่มีวงจรความยาวเป็นเลขคี่** ดังนั้นการระบายสี 2 สีจะล้มเหลวก็ต่อเมื่อมีวงจรเลขคี่

14.1.2 โค้ดตรวจสอบกราฟทวิภาคีด้วย BFS

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100010;
vector<int> adj[N];
int color[N];  // -1 = uncolored, 0 = color 0, 1 = color 1
bool isBipartite(int n) {
    memset(color, -1, sizeof(color));
    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;  // Already colored, skip
```

```
            // BFS from vertex s
            queue<int> q;
            color[s] = 0;
            q.push(s);
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                for (int v : adj[u]) {
                    if (color[v] == -1) {
                        // Uncolored □ paint opposite color
                        color[v] = 1 - color[u];
                        q.push(v);
                    } else if (color[v] == color[u]) {
                        // Neighbor has same color □ not bipartite
                        return false;
                    }
                }
            }
        }
        return true;
}
int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    if (isBipartite(n)) {
        printf("Graph is bipartite\n");
    } else {
        printf("Graph is NOT bipartite\n");
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 14.1: ตรวจสอบกราฟทวิภาคีด้วย BFS

ตัวอย่าง

- กราฟรูปสี่เหลี่ยม (วงจรความยาว 4): สามารถระบาย 2 สีได้ ⊠ **เป็นกราฟทวิภาคี**
- กราฟรูปสามเหลี่ยม (วงจรความยาว 3): ระบาย 2 สีไม่ได้ ⊠ **ไม่ใช่กราฟทวิภาคี**
- ต้นไม้: ไม่มีวงจรเลย ⊠ **เป็นกราฟทวิภาคี** เสมอ

การประยุกต์กราฟทวิภาคี

กราฟทวิภาคีมีประโยชน์ในโจทย์หลายประเภท:

- **Matching Problem:** การจับคู่ระหว่างสองกลุ่ม (เช่น คนกับงาน)
- **การตรวจสอบความขัดแย้ง:** โจทย์ที่ต้องแบ่งวัตถุเป็น 2 กลุ่มโดยไม่ขัดแย้งกัน
- **Graph Coloring:** การระบายสีกราฟด้วยจำนวนสีน้อยที่สุด (กราฟทวิภาคีใช้เพียง 2 สี)

14.2 จุดตัดและสะพาน (Articulation Points & Bridges)

จุดตัด (Articulation Point) และสะพาน (Bridge) เป็นแนวคิดที่สำคัญในการวิเคราะห์ความเสี่ยงของเครือข่าย[3, 11] จุดตัดคือจุดยอดที่ถ้าลบออกจะทำให้กราฟแตกออกเป็นหลายส่วน สะพานคือเส้นเชื่อมที่ถ้าลบออกจะทำให้กราฟแตก การหาจุดตัดและสะพานสามารถทำได้อย่างมีประสิทธิภาพด้วยขั้นตอนวิธีของ Tarjan ที่ใช้ DFS เพียงครั้งเดียว

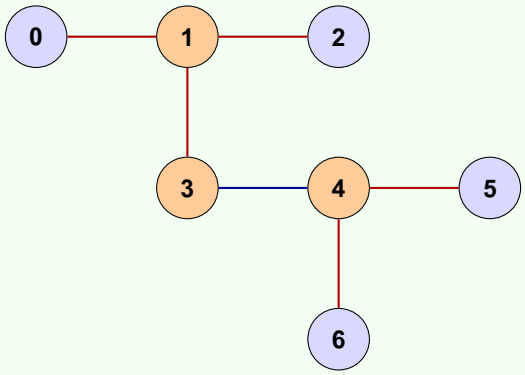
14.2.1 นิยาม

นิยาม 14.1 (จุดตัด (Articulation Point))**.** จุดยอด v เป็นจุดตัดของกราฟถ้าการลบ v และเส้นเชื่อมทั้งหมดที่ติดกับ v ออก ทำให้จำนวนองค์ประกอบเชื่อมโยงเพิ่มขึ้น

นิยาม 14.2 (สะพาน (Bridge))**.** เส้นเชื่อม e เป็นสะพานของกราฟถ้าการลบ e ออกทำให้จำนวนองค์ประกอบเชื่อมโยงเพิ่มขึ้น

ตัวอย่าง

พิจารณากราฟในรูปที่ 14.1



รูปที่ 14.1 ตัวอย่างกราฟ: จุดตัด (สีส้ม) = 1, 3, 4; สะพาน (เส้นแดง) = (0,1), (1,2), (1,3), (4,5), (4,6); ไม่ใช่สะพาน (เส้นน้ำเงิน) = (3,4)

- จุดตัด: 1 (ถ้าลบ 1 กราฟจะแตกเป็น 0, 2, 3,4,5,6), 3 (ถ้าลบ 3 จะแตกเป็น 0,1,2, 4,5,6), 4 (ถ้าลบ 4 จะแตกเป็น 0,1,2,3, 5, 6)
- สะพาน: เส้น (0,1), (1,2), (1,3), (4,5), (4,6) — ถ้าลบเส้นใดเส้นหนึ่ง กราฟจะแตก
- ไม่ใช่สะพาน: เส้น (3,4) — ถ้าลบออก กราฟยังเชื่อมโยงผ่านเส้นทางอื่น

14.2.2 ขั้นตอนวิธีของ Tarjan

ขั้นตอนวิธีของ Tarjan หาจุดตัดและสะพานได้ในเวลา $\mathcal{O}(V+E)$ โดยใช้ DFS เพียงครั้งเดียว แนวคิดหลักคือใช้ค่า 2 ค่า:

- `dfs_num[u]`: ลำดับการเยี่ยมของ u (เพิ่มขึ้นทีละ 1 เมื่อเยี่ยมจุดยอดใหม่)
- `dfs_low[u]`: ค่า `dfs_num` ต่ำสุดที่สามารถเข้าถึงได้จาก subtree ของ u ผ่านเส้นเชื่อมประเภทใดก็ได้ (ยกเว้นเส้นเชื่อมไปหาพ่อแม่)

เงื่อนไขจุดตัด: จุดยอด u เป็นจุดตัด ถ้ามีลูก v ใน DFS tree ที่ทำให้ `dfs_low[v] >= dfs_num[u]` (สำหรับ root ของ DFS tree ต้องมีลูกมากกว่า 1 คนจึงเป็นจุดตัด)

เงื่อนไขสะพาน: เส้นเชื่อม (u, v) เป็นสะพาน ถ้า `dfs_low[v] > dfs_num[u]` (สังเกต: เครื่องหมาย $>$ แบบเข้มงวด ไม่ใช่ $\geq$)

ความแตกต่างระหว่างจุดตัดกับสะพาน

จุดตัดใช้เงื่อนไข $\geq$ (มากกว่าหรือเท่ากับ) สะพานใช้เงื่อนไข $>$ (มากกว่าเท่านั้น) ความแตกต่างนี้สำคัญมาก! ถ้า `dfs_low[v] == dfs_num[u]` เส้นเชื่อม (u, v) **ไม่ใช่สะพาน** (เพราะมีเส้นทางกลับ) แต่ u **อาจเป็นจุดตัด**

14.2.3 โค้ดหาจุดตัดและสะพาน

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;
typedef vector<ii> vii;
typedef vector<int> vi;
#define UNVISITED -1
vector<vii> adj;
vi visited, dfs_low, dfs_parent;
bool articulation[100010];
int dfsNumberCounter, dfsRoot, rootChildren;
void articulationPointAndBridge(int u) {
    visited[u] = dfs_low[u] = dfsNumberCounter++;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        ii v = adj[u][j];
        if (visited[v.first] == UNVISITED) {
            dfs_parent[v.first] = u;
            if (u == dfsRoot) rootChildren++;  // Count children of root
            articulationPointAndBridge(v.first);
            // Update dfs_low from child
            dfs_low[u] = min(dfs_low[u], dfs_low[v.first]);
            // Check articulation point
            if (dfs_low[v.first] >= visited[u]) {
                if (u != dfsRoot || rootChildren > 1) {
                    articulation[u] = true;
                }
            }
            // Check bridge
            if (dfs_low[v.first] > visited[u]) {
                printf("Bridge: (%d, %d)\n", u, v.first);
            }
        } else if (v.first != dfs_parent[u]) {
            // Back Edge — update dfs_low
            dfs_low[u] = min(dfs_low[u], visited[v.first]);
        }
    }
}
int main() {
    int V, E;
    scanf("%d %d", &V, &E);
    adj.assign(V, vii());
    for (int i = 0; i < E; i++) {
```

```
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back({v, 0});
        adj[v].push_back({u, 0});
    }
    visited.assign(V, UNVISITED);
    dfs_low.assign(V, 0);
    dfs_parent.assign(V, -1);
    memset(articulation, false, sizeof(articulation));
    dfsNumberCounter = 0;
    printf("Bridges:\n");
    for (int i = 0; i < V; i++) {
        if (visited[i] == UNVISITED) {
            dfsRoot = i;
            rootChildren = 0;
            articulationPointAndBridge(i);
        }
    }
    printf("Articulation Points:\n");
    for (int i = 0; i < V; i++) {
        if (articulation[i]) {
            printf("%d ", i);
        }
    }
    printf("\n");
    return 0;
}
```

ตัวอย่างโค้ดที่ 14.2: หาจุดตัดและสะพานด้วย Tarjan's Algorithm

กรณีพิเศษของ Root

จุดยอดที่เป็นรากของ DFS tree เป็นจุดตัดก็ต่อเมื่อมี **ลูกมากกว่า 1 คน** ใน DFS tree ถ้า root มีลูกเพียง 1 คน การลบ root ออกจะไม่ทำให้กราฟแตก เพราะลูกทุกคนยังเชื่อมโยงกันผ่านเส้นเชื่อมอื่น

14.3 Strongly Connected Components (SCC)

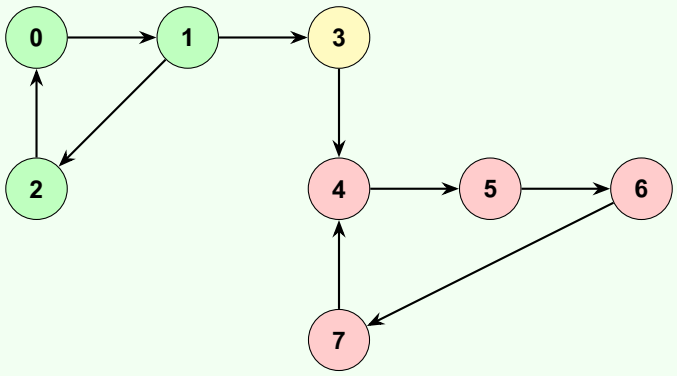
ในกราฟมีทิศทาง ส่วนประกอบเชื่อมโยงเข้ม (Strongly Connected Component — SCC) เป็นแนวคิดที่ขยายจาก Connected Component ในกราฟไม่มีทิศทาง[2, 3] โดย SCC พิจารณาทิศทางของเส้นเชื่อมด้วย

14.3.1 นิยาม SCC

นิยาม 14.3 (Strongly Connected Component — SCC)**.** SCC ของกราฟมีทิศทางคือเซตย่อยที่มากที่สุดของจุดยอด S ที่สำหรับจุดยอดทุกคู่ $u, v \in S$ มีเส้นทางจาก u ไป v **และ** จาก v ไป u

ตัวอย่าง: SCC

พิจารณากราฟมีทิศทางในรูปที่ 14.2



รูปที่ 14.2 ตัวอย่างกราฟมีทิศทาง: SCC $\{0, 1, 2\}$ (เขียว), $\{3\}$ (เหลือง), $\{4, 5, 6, 7\}$ (แดง)

- SCC 1: $\{0, 1, 2\}$ — ทุกคู่สามารถไปหากันได้ (จากวงจร $0 \to 1 \to 2 \to 0$)
- SCC 2: $\{3\}$ — จุดยอด 3 มีเฉพาะเส้นออก $3 \to 4$ และไม่มีเส้นทางกลับจาก $\{4, 5, 6, 7\}$ มาหา 3
- SCC 3: $\{4, 5, 6, 7\}$ — ทุกคู่สามารถไปหากันได้ (จากวงจร $4 \to 5 \to 6 \to 7 \to 4$)

เมื่อยุบแต่ละ SCC เป็นจุดยอดเดียว จะได้ DAG (กราฟมีทิศทางไม่มีวงจร)

14.3.2 ขั้นตอนวิธีของ Tarjan สำหรับ SCC

ขั้นตอนวิธีของ Tarjan หา SCC ทั้งหมดได้ในเวลา $\mathcal{O}(V + E)$ โดยใช้ DFS เพียงครั้งเดียว แนวคิดคล้ายกับการหาจุดตัด/สะพาน แต่เพิ่มการใช้ **กองซ้อน (Stack)** เพื่อเก็บจุดยอดที่อยู่ระหว่างการสำรวจ

ขั้นตอนสำคัญ:

1. เก็บ `dfs_num[u]` และ `dfs_low[u]` เช่นเดียวกับการหาจุดตัด
2. ใช้ **กองซ้อน (Stack)** S เก็บจุดยอดที่ถูกเยี่ยมตามลำดับ และแถวลำดับ `visited[]` ระบุว่าจุดยอดยังอยู่ในกองซ้อนหรือไม่
3. เมื่อ `dfs_low[u] == dfs_num[u]` จุดยอด u เป็น **รากของ SCC** — ดึงจุดยอดออกจากกองซ้อนจนถึง u จุดยอดที่ดึงออกทั้งหมดคือ SCC หนึ่งกลุ่ม

ความแตกต่างสำคัญจากการหาจุดตัด: เมื่ออัปเดต `dfs_low[u]` จากจุดข้างเคียง v เราจะอัปเดตเฉพาะเมื่อ v **ยังอยู่ในกองซ้อน** (`visited[v] == true`) ไม่ใช่เพียงแค่ไม่ใช่พ่อแม่

14.3.3 โค้ด Tarjan's SCC

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef pair<int,int> ii;
typedef vector<ii> vii;
typedef vector<int> vi;
#define UNVISITED -1
vector<vii> adj;
vi dfs_num, dfs_low;
vi S;            // Stack to store vertices
bool onStack[100010];
int dfsNumberCounter;
int numSCC;
void tarjanSCC(int u) {
    dfs_num[u] = dfs_low[u] = dfsNumberCounter++;
    S.push_back(u);
    onStack[u] = true;
    for (int j = 0; j < (int)adj[u].size(); j++) {
        ii v = adj[u][j];
        if (dfs_num[v.first] == UNVISITED) {
            tarjanSCC(v.first);
        }
        if (onStack[v.first]) {
            dfs_low[u] = min(dfs_low[u], dfs_low[v.first]);
        }
    }
    // If u is the root of an SCC
    if (dfs_low[u] == dfs_num[u]) {
        printf("SCC %d:", ++numSCC);
        while (true) {
            int v = S.back();
            S.pop_back();
            onStack[v] = false;
            printf(" %d", v);
            if (u == v) break;
        }
        printf("\n");
    }
}
int main() {
    int V, E;
```

```
    scanf("%d %d", &V, &E);
    adj.assign(V, vii());
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back({v, 0});    // Directed graph
    }
    dfs_num.assign(V, UNVISITED);
    dfs_low.assign(V, 0);
    S.clear();
    memset(onStack, false, sizeof(onStack));
    dfsNumberCounter = 0;
    numSCC = 0;
    for (int i = 0; i < V; i++) {
        if (dfs_num[i] == UNVISITED) {
            tarjanSCC(i);
        }
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 14.3: หา SCC ด้วย Tarjan's Algorithm

ตัวอย่าง: ผลลัพธ์ของ Tarjan's SCC

สำหรับกราฟ 8 จุดยอด:

```
SCC 1: 6 7 5 4
SCC 2: 3
SCC 3: 2 1 0
```

จะเห็นว่า Tarjan ค้นพบ SCC ตามลำดับทอพอโลยีย้อนกลับ — SCC ที่ไม่มีเส้นเชื่อมออกไปหา SCC อื่นจะถูกค้นพบก่อน

14.3.4 การประยุกต์ใช้ SCC

SCC มีประโยชน์มากในการแข่งขัน:

1. **2-SAT Problem:** ปัญหาการหาค่าตรรกะที่ทำให้สูตร CNF เป็นจริง สามารถแปลงเป็นกราฟและใช้ SCC แก้ได้
2. **การยุบกราฟ (Condensation):** ยุบแต่ละ SCC เป็นจุดยอดเดียว จะได้ DAG ที่ง่ายกว่าเดิม สามารถใช้ Topological Sort หรือ DP บน DAG ได้
3. **การตรวจจับวงจร:** กราฟมีทิศทางมีวงจรก็ต่อเมื่อมี SCC ที่มีขนาดมากกว่า 1 จุดยอด

ลำดับทอพอโลยีของ SCC

เมื่อ Tarjan ค้นพบ SCC แต่ละกลุ่ม กลุ่มที่ค้นพบก่อนจะเป็น SCC ที่อยู่ "ปลาย" ของกราฟ (ไม่มีเส้นเชื่อมออกไปหา SCC อื่น) ดังนั้นลำดับที่ SCC ถูกค้นพบคือ **ลำดับทอพอโลยีย้อนกลับ** ของ DAG ที่เกิดจากการยุบ SCC ถ้าต้องการลำดับทอพอโลยีที่ถูกต้อง ให้กลับลำดับ

14.4 Kosaraju's Algorithm — ขั้นตอนวิธี SCC ทางเลือก

ในหัวข้อก่อนหน้า เราได้ศึกษาขั้นตอนวิธีของ Tarjan สำหรับหา SCC แล้ว ในหัวข้อนี้เราจะเรียนรู้ขั้นตอนวิธีอีกตัวหนึ่งที่ใช้หา SCC คือ **Kosaraju's Algorithm** ซึ่งมีแนวคิดที่เข้าใจง่ายกว่า Tarjan แต่ต้องใช้ DFS 2 รอบ

14.4.1 หลักการ: DFS 2 รอบ + พลิกกราฟ

Kosaraju's Algorithm ทำงานใน 3 ขั้นตอน:

1. **รอบที่ 1 — DFS บนกราฟเดิม:** เรียก DFS บนกราฟเดิม เมื่อจบการสำรวจจุดยอดใด ให้นำจุดยอดนั้นใส่ลงใน **กองซ้อน** ตามลำดับเวลาที่จบ (finish time) จุดยอดที่จบล่าสุดจะอยู่บนสุดของกองซ้อน
2. **สร้างกราฟสลับทิศทาง (Transpose Graph):** พลิกเส้นเชื่อมทุกเส้น — ถ้าเดิมมีเส้น $u \to v$ ให้เปลี่ยนเป็น $v \to u$
3. **รอบที่ 2 — DFS บนกราฟสลับทิศทาง:** ดึงจุดยอดจากกองซ้อนจากบนลงล่าง (จุดยอดที่จบล่าสุดในรอบที่ 1 จะถูกประมวลผลก่อน) แล้วเรียก DFS บนกราฟสลับทิศทาง — ต้นไม้ DFS แต่ละต้นที่ได้คือ SCC หนึ่งกลุ่ม

ทฤษฎีบท 14.2 (สาระสำคัญของ Kosaraju)**.** ในกราฟสลับทิศทาง SCC ที่จบช้าที่สุดใน DFS รอบที่ 1 จะ **ไม่สามารถไปถึง SCC อื่นได้** ดังนั้นเมื่อเริ่ม DFS จากจุดยอดของ SCC นั้นบนกราฟสลับทิศทาง จะสำรวจเจอเฉพาะจุดยอดใน SCC เดียวกันเท่านั้น

เหตุผล: พิจารณา DAG ของ SCC ถ้า SCC A สามารถไปถึง SCC B ได้ ($A \to B$) แสดงว่ามีเส้นเชื่อมจาก A ไป B ในกราฟเดิม ดังนั้นจุดยอดใน A จะถูกสำรวจก่อนและจบ **หลัง** จุดยอดใน B ในทางกลับกัน ในกราฟสลับทิศทาง เส้นเชื่อมกลายเป็น $B \to A$ ทำให้จาก A ไม่สามารถไปถึง B ได้ จึงสำรวจเจอเฉพาะ A เท่านั้น

14.4.2 โค้ด Kosaraju's Algorithm

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100010;
vector<int> adj[MAXN];      // Original graph
vector<int> radj[MAXN];     // Transposed graph (all edges reversed)
```

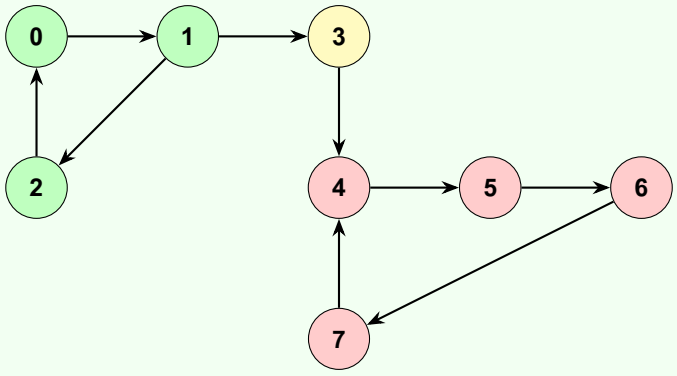
```
bool visited[MAXN];
stack<int> finishStack;       // Stack ordered by finish time
vector<vector<int>> sccs;     // Result: list of SCCs
// Pass 1: DFS on original graph, push to stack by finish time
void dfs1(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs1(v);
        }
    }
    finishStack.push(u);   // Push after all children are processed
}
// Pass 2: DFS on transposed graph to collect one SCC
void dfs2(int u, vector<int>& component) {
    visited[u] = true;
    component.push_back(u);
    for (int v : radj[u]) {
        if (!visited[v]) {
            dfs2(v, component);
        }
    }
}
int main() {
    int V, E;
    scanf("%d %d", &V, &E);
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);      // Original edge u -> v
        radj[v].push_back(u);     // Reversed edge v -> u
    }
    // Pass 1: DFS on original graph
    memset(visited, false, sizeof(visited));
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs1(i);
        }
    }
    // Pass 2: Process vertices from stack on transposed graph
    memset(visited, false, sizeof(visited));
```

```
    while (!finishStack.empty()) {
        int u = finishStack.top();
        finishStack.pop();
        if (!visited[u]) {
            vector<int> component;
            dfs2(u, component);
            sccs.push_back(component);
        }
    }
    // Output
    for (int i = 0; i < (int)sccs.size(); i++) {
        printf("SCC %d:", i + 1);
        for (int v : sccs[i]) {
            printf(" %d", v);
        }
        printf("\n");
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 14.4: หา SCC ด้วย Kosaraju's Algorithm

ตัวอย่าง: Kosaraju's Algorithm

พิจารณากราฟในรูปที่ 14.3



รูปที่ 14.3 กราฟตัวอย่างสำหรับ Kosaraju's Algorithm (กราฟเดียวกับรูปที่ 14.2)

- **รอบที่ 1 (DFS กราฟเดิม):** สมมติเริ่มจาก 0 ลำดับการจบอาจเป็น 2, 7, 6, 5, 4, 3, 1, 0 (จุดยอดที่อยู่ลึกที่สุดจบก่อน และ 1 จะจบหลังจาก subtree ของ 1+3+4+5+6+7 จบหมดแล้ว) กองซ้อนเก็บจุดยอดตามลำดับจบจากล่างขึ้นบน — จุดยอด 0 จบล่าสุดจะอยู่บนสุดของกองซ้อน
- **กราฟสลับทิศทาง:** พลิกทุกเส้นเชื่อม เช่น $0 \to 1$ กลายเป็น $1 \to 0$, $4 \to 5$ กลายเป็น $5 \to 4$ เป็นต้น

- **รอบที่ 2 (DFS กราฟสลับ):** ดึงจากกองซ้อนจุดยอดบนสุด (จบล่าสุดในรอบที่ 1) เรียก DFS บนกราฟสลับ — ต้นไม้ DFS แต่ละต้นคือ SCC หนึ่งกลุ่ม
- **ผลลัพธ์:** SCC 1 = $\{0, 1, 2\}$, SCC 2 = $\{3\}$, SCC 3 = $\{4, 5, 6, 7\}$ — ตรงกับผลของ Tarjan's SCC

14.4.3 เปรียบเทียบ Kosaraju กับ Tarjan

	Kosaraju	**Tarjan**
จำนวน DFS	2 รอบ	1 รอบ
ต้องสร้างกราฟสลับทิศทาง	ใช่	ไม่ใช่
ความเข้าใจง่าย	ง่ายกว่า	ยากกว่า
ขนาดโค้ด	ยาวกว่าเล็กน้อย	กระชับกว่า
ความซับซ้อน	$\mathcal{O}(V + E)$	$\mathcal{O}(V + E)$

เลือกขั้นตอนวิธี SCC อย่างไร?

ทั้ง Kosaraju และ Tarjan มีความซับซ้อน $\mathcal{O}(V + E)$ เท่ากัน ให้เลือกจากสิ่งที่คุณ **เขียนได้ถูกต้องภายใต้แรงกดดันของการแข่งขัน** ถ้าคุณมั่นใจกับ Tarjan ก็ใช้ Tarjan แต่ถ้ารู้สึกว่า Kosaraju เข้าใจง่ายกว่าและเขียนได้ถูกต้องมากกว่า ก็ใช้ Kosaraju ได้เช่นกัน

14.5 ปัญหา 2-SAT

ปัญหา 2-SAT เป็นหนึ่งในปัญหาตรรกะที่สำคัญที่สุดในการแข่งขันเขียนโปรแกรม[3, 11] และเป็นประยุกต์ใช้ที่สำคัญที่สุดอย่างหนึ่งของ SCC

14.5.1 นิยาม 2-SAT

นิยาม 14.4 (2-SAT)**.** กำหนดสูตรบูลีนในรูปแบบ 2-CNF (Conjunctive Normal Form) คือ **การเชื่อมกันด้วย AND** ของ clause หลาย clause โดยแต่ละ clause เป็น **การเลือกด้วย OR** ของ literal **Exactly 2 ตัว** ปัญหา 2-SAT ถามว่ามีการกำหนดค่าตรรกะให้ตัวแปรแต่ละตัวเป็น `true` หรือ `false` ที่ทำให้สูตรทั้งหมดเป็นจริงหรือไม่

ตัวอย่างสูตร 2-CNF:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2)$$

แต่ละ clause มี literal เพียง 2 ตัว เช่น clause แรก $x_1 \vee \neg x_2$ จะเป็นจริงถ้า x_1 เป็นจริง **หรือ** x_2 เป็นเท็จ

14.5.2 สร้าง Implication Graph

ข้อมูลสำคัญ: clause $(a \vee b)$ สมมูลกับ $(\neg a \to b) \wedge (\neg b \to a)$

เราสามารถอธิบายได้ว่า: ถ้า a เป็นเท็จ แล้ว b **ต้อง** เป็นจริง (เพื่อให้ clause เป็นจริง) และในทางกลับกัน ถ้า b เป็นเท็จ แล้ว a **ต้อง** เป็นจริง

จากข้อมูลนี้ เราสร้าง **Implication Graph** ได้ดังนี้:

- สำหรับตัวแปรแต่ละตัว x_i สร้างจุดยอด 2 จุด: x_i และ $\neg x_i$
- สำหรับแต่ละ clause $(a \vee b)$ เพิ่มเส้นเชื่อม $\neg a \to b$ และ $\neg b \to a$

14.5.3 เงื่อนไขความเป็นได้ของ 2-SAT

ทฤษฎีบท 14.3 (เงื่อนไข 2-SAT)**.** สูตร 2-CNF เป็น **satisfiable** ก็ต่อเมื่อ **ไม่มีตัวแปร x ใดที่ x และ $\neg x$ อยู่ใน SCC เดียวกัน**

เหตุผล: ถ้า x และ $\neg x$ อยู่ใน SCC เดียวกัน แปลว่ามีเส้นทางจาก x ไป $\neg x$ และจาก $\neg x$ ไป x นั่นคือ $x \Rightarrow \neg x$ และ $\neg x \Rightarrow x$ ซึ่งขัดแย้งกัน — ไม่ว่าจะกำหนด x เป็นค่าใดก็จะขัดแย้ง

14.5.4 การหาค่าตัวแปรเมื่อ Satisfiable

ถ้าสูตรเป็น satisfiable เราสามารถหาค่าตัวแปรได้ดังนี้:

1. หา SCC ทั้งหมดของ Implication Graph ด้วย Kosaraju's Algorithm แล้วเก็บ **หมายเลข SCC** ของแต่ละจุดยอด
2. สำหรับตัวแปร x_i: ถ้าหมายเลข SCC ของ x_i **มากกว่า** หมายเลข SCC ของ $\neg x_i$ ให้กำหนด $x_i =$ `true` มิฉะนั้นกำหนด $x_i =$ `false`

เหตุผล: ใน Implication Graph ถ้ามีเส้นเชื่อมจาก SCC หนึ่งไปอีก SCC หนึ่ง ค่าตรรกะจะถูก "ส่งผ่าน" ไปตามเส้นเชื่อม ใน Kosaraju's Algorithm หมายเลข SCC ที่มากกว่าคือ SCC ที่อยู่ปลายกราฟ (sink) การกำหนดให้ตัวแปรใน sink SCC เป็น `true` จะไม่บังคับให้ตัวแปรอื่นตามมา จึงปลอดภัยกว่า

14.5.5 โค้ด 2-SAT ด้วย Kosaraju's SCC

ในโค้ดต่อไปนี้ เราใช้การเข้ารหัสจุดยอดดังนี้:

- ตัวแปร x_i (0-indexed): จุดยอด $2i$ แทน x_i, จุดยอด $2i+1$ แทน $\neg x_i$
- ฟังก์ชัน `NOT(x) = x ^ 1` สลับระหว่าง x_i กับ $\neg x_i$

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 200010;     // 2 * number of variables
```

```
vector<int> adj[MAXN];      // Implication graph
vector<int> radj[MAXN];     // Transposed implication graph
bool visited[MAXN];
stack<int> finishStack;
int sccId[MAXN];            // SCC id of each vertex
int sccCount;
// Negate a literal: x_i <-> ¬x_i
inline int NOT(int x) { return x ^ 1; }
// Add a clause (a OR b) to the implication graph
// Equivalent to: (¬a -> b) AND (¬b -> a)
void addClause(int a, int b) {
    adj[NOT(a)].push_back(b);
    adj[NOT(b)].push_back(a);
    radj[b].push_back(NOT(a));
    radj[a].push_back(NOT(b));
}
// Pass 1: DFS on original graph
void dfs1(int u) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs1(v);
    }
    finishStack.push(u);
}
// Pass 2: DFS on transposed graph
void dfs2(int u, int id) {
    sccId[u] = id;
    for (int v : radj[u]) {
        if (sccId[v] == -1) dfs2(v, id);
    }
}
int main() {
    int n, m;    // n = number of variables, m = number of clauses
    scanf("%d %d", &n, &m);
    for (int i = 0; i < m; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        // Input: a, b encoded as 2*i for x_i, 2*i+1 for ¬x_i
        addClause(a, b);
    }
    // Kosaraju Pass 1
```

```
    memset(visited, false, sizeof(visited));
    for (int i = 0; i < 2 * n; i++) {
        if (!visited[i]) dfs1(i);
    }
    // Kosaraju Pass 2
    memset(sccId, -1, sizeof(sccId));
    sccCount = 0;
    while (!finishStack.empty()) {
        int u = finishStack.top();
        finishStack.pop();
        if (sccId[u] == -1) {
            dfs2(u, sccCount++);
        }
    }
    // Check satisfiability: x_i and ¬x_i must NOT be in the same SCC
    bool satisfiable = true;
    for (int i = 0; i < n; i++) {
        if (sccId[2 * i] == sccId[2 * i + 1]) {
            satisfiable = false;
            break;
        }
    }
    if (!satisfiable) {
        printf("UNSATISFIABLE\n");
        return 0;
    }
    // Construct assignment:
    // If sccId[x_i] > sccId[¬x_i], then x_i = true
    // (Kosaraju assigns higher id to SCCs found later = sink of condensation DAG)
    printf("SATISFIABLE\n");
    for (int i = 0; i < n; i++) {
        bool value = (sccId[2 * i] > sccId[2 * i + 1]);
        printf("x_%d = %s\n", i + 1, value ? "true" : "false");
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 14.5: 2-SAT ด้วย Kosaraju's SCC

ตัวอย่าง: 2-SAT

พิจารณาสูตร: $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2)$

สร้าง Implication Graph:

- Clause $(x_1 \vee \neg x_2)$: เพิ่ม $\neg x_1 \to \neg x_2$ และ $x_2 \to x_1$
- Clause $(\neg x_1 \vee x_3)$: เพิ่ม $x_1 \to x_3$ และ $\neg x_3 \to \neg x_1$
- Clause $(x_2 \vee \neg x_3)$: เพิ่ม $\neg x_2 \to \neg x_3$ และ $x_3 \to x_2$
- Clause $(\neg x_1 \vee \neg x_2)$: เพิ่ม $x_1 \to \neg x_2$ และ $x_2 \to \neg x_1$

หา SCC ด้วย Kosaraju:

- SCC 0 = $\{x_1, x_2, x_3\}$ — ทุกคู่สามารถไปหากันได้ผ่านวงจร $x_1 \to x_3 \to x_2 \to x_1$
- SCC 1 = $\{\neg x_1, \neg x_2, \neg x_3\}$ — ทุกคู่สามารถไปหากันได้ผ่านวงจร $\neg x_1 \to \neg x_2 \to \neg x_3 \to \neg x_1$

ตรวจสอบความเป็นได้: ไม่มีตัวแปรใดที่ x_i และ $\neg x_i$ อยู่ใน SCC เดียวกัน $\Rightarrow$ **Satisfiable!**

หาค่าตัวแปร:

- x_1: `sccId[`x_1`]` = 0, `sccId[`$\neg x_1$`]` = 1 $\Rightarrow 0 > 1$ เป็นเท็จ $\Rightarrow x_1 =$ `false`
- x_2: `sccId[`x_2`]` = 0, `sccId[`$\neg x_2$`]` = 1 $\Rightarrow 0 > 1$ เป็นเท็จ $\Rightarrow x_2 =$ `false`
- x_3: `sccId[`x_3`]` = 0, `sccId[`$\neg x_3$`]` = 1 $\Rightarrow 0 > 1$ เป็นเท็จ $\Rightarrow x_3 =$ `false`

ตรวจสอบ $x_1 = F, x_2 = F, x_3 = F$:

- $(F \vee \neg F) = (F \vee T) = T$ ✓
- $(\neg F \vee F) = (T \vee F) = T$ ✓
- $(F \vee \neg F) = (F \vee T) = T$ ✓
- $(\neg F \vee \neg F) = (T \vee T) = T$ ✓

สูตรเป็นจริงทุก clause!

2-SAT กับ SCC

2-SAT เป็นหนึ่งในประยุกต์ใช้ที่สำคัญที่สุดของ SCC ในการแข่งขันเขียนโปรแกรม เมื่อเจอโจทย์ที่มีเงื่อนไขแบบ “ถ้า A แล้วต้อง B” หรือ “A หรือ B ต้องเลือกอย่างน้อยหนึ่งอย่าง” ให้นึกถึง 2-SAT ทันที

3-SAT เป็น NP-complete!

2-SAT สามารถแก้ได้ในเวลาพหุนามเพราะแต่ละ clause มี literal **เพียง 2 ตัว** ทำให้สามารถสร้าง Implication Graph ได้ แต่ถ้า clause มี literal **3 ตัวขึ้นไป** (3-SAT) ปัญหาจะกลายเป็น **NP-complete** ไม่มีขั้นตอนวิธีที่มีประสิทธิภาพที่รู้จัก! ดังนั้นต้องแน่ใจว่าแต่ละ clause มี literal เพียง 2 ตัวจึงจะใช้ 2-SAT ได้

14.6 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนขั้นตอนวิธีกราฟขั้นสูง :

โจทย์แนะนำสำหรับขั้นตอนวิธีกราฟขั้นสูง

Bipartite Check:

- Codeforces 687A — NP-Hard Problem[10]
- CSES Planet Queries II (Bipartite Graph)[12]

Articulation Points & Bridges:

- UVA 10765 — Doves and Bombs (Articulation Points)[20]
- CSES Flight Routes (Bridges application)[12]
- Codeforces 118E — Bertown Roads (Bridges)[10]

SCC & 2-SAT:

- CSES Course Schedule (Topological Sort / SCC)[12]
- CSES Longest Flight Route (DP on DAG)[12]
- UVA 11838 — Come and Go (SCC)[20]
- Codeforces 776D — The Door Problem (2-SAT)[10]
- UVA 10319 — Manhattan (2-SAT)[20]

14.7 สรุปบทที่ 14

ตารางที่ 14.1 สรุปขั้นตอนวิธีกราฟทั้งหมดในบทที่ 8–11

ขั้นตอนวิธี	ปัญหา	ความซับซ้อน
DFS	สำรวจกราฟ, หา CC, Topological Sort	$\mathcal{O}(V+E)$
BFS	สำรวจกราฟ, หาระยะทาง (ไม่มีน้ำหนัก)	$\mathcal{O}(V+E)$
Bellman-Ford	SSSP (น้ำหนักลบได้)	$\mathcal{O}(V \cdot E)$
Dijkstra	SSSP (น้ำหนักไม่ติดลบ)	$\mathcal{O}((V+E)\log V)$
Floyd-Warshall	APSP	$\mathcal{O}(V^3)$
Union-Find	จัดการเซตไม่มีส่วนร่วม	$\mathcal{O}(\alpha(n)) \approx \mathcal{O}(1)$
Kruskal's	MST	$\mathcal{O}(E \log V)$
Prim's	MST	$\mathcal{O}(E \log V)$
Bipartite Check	ตรวจสอบกราฟทวิภาคี	$\mathcal{O}(V+E)$
Tarjan AP/Bridge	หาจุดตัดและสะพาน	$\mathcal{O}(V+E)$
Tarjan SCC	หา SCC	$\mathcal{O}(V+E)$

กลยุทธ์แก้โจทย์กราฟ

เมื่อเจอโจทย์กราฟ ให้ถามตัวเองตามลำดับ:

1. กราฟเป็นแบบใด? มีทิศทาง/ไม่มีทิศทาง? มีน้ำหนัก/ไม่มีน้ำหนัก?
2. ต้องการอะไร? เส้นทางสั้นสุด? ต้นไม้แผ่คลุม? องค์ประกอบเชื่อมโยง? วงจร?
3. ขนาดข้อมูลเท่าไร? n และ m เป็นเท่าไร? ข้อจำกัดนี้กำหนดว่าขั้นตอนวิธีไหนใช้ได้
4. มีสมบัติพิเศษหรือไม่? DAG? กราฟทวิภาคี? ต้นไม้?

การตอบคำถามเหล่านี้จะช่วยให้เลือกขั้นตอนวิธีได้ถูกต้อง

7

ชุดปัญหาด้านเครือข่ายการไหล

ส่วนนี้นำเสนอทฤษฎีเครือข่ายการไหล (Flow Network) ซึ่งเป็นหัวข้อขั้นสูงในการแข่งขันเขียนโปรแกรม ครอบคลุมแนวคิดพื้นฐานของกราฟการไหล ทฤษฎีบท Max-Flow Min-Cut และขั้นตอนวิธีสำคัญได้แก่ Ford-Fulkerson, Edmonds-Karp และ Dinic's Algorithm รวมถึงการประยุกต์ใช้ในปัญหาการจับคู่แบบทวิภาคี (Bipartite Matching) และปัญหาที่เกี่ยวข้อง

Chapter 15

การไหลสูงสุด

ปัญหาการไหลสูงสุด (Maximum Flow) เป็นหนึ่งในปัญหาคลาสสิกที่สำคัญที่สุดในทฤษฎีกราฟและการโปรแกรมเชิงแข่งขัน[2, 3] มีประยุกต์ใช้อย่างกว้างขวาง ทั้งในด้านเครือข่ายคมนาคม ระบบท่อส่ง การจัดตารางการผลิต การจับคู่แบบกราฟทวิภาคี และอื่นๆ อีกมากมาย ในบทนี้ เราจะศึกษาแนวคิดของเครือข่ายการไหล (Flow Network) นิยามของการไหลที่ถูกต้อง ขั้นตอนวิธี Ford-Fulkerson, Edmonds-Karp และ Dinic สำหรับหาค่าการไหลสูงสุด ทฤษฎีบท Max-Flow Min-Cut ที่สวยงาม รวมถึงการประยุกต์ใช้ Maximum Flow ในการแก้ปัญหาหลากหลายรูปแบบ เช่น Bipartite Matching, Edge-Disjoint Path และ Min-Cost Max-Flow

15.1 แนะนำเครือข่ายการไหล

15.1.1 แรงบันดาลใจ

จินตนาการว่าเรามีเครือข่ายท่อน้ำ โดยจุดยอดแต่ละจุดคือจุดต่อของท่อ และเส้นเชื่อมแต่ละเส้นคือท่อที่มีความจุ (Capacity) จำกัด เราต้องการส่งน้ำจากจุดกำเนิด (Source) s ไปยังจุดรับ (Sink) t ให้ได้มากที่สุด คำถามคือ: ปริมาณน้ำมากที่สุดที่สามารถไหลผ่านเครือข่ายนี้ได้คือเท่าไร?

นี่คือสาระสำคัญของปัญหา Maximum Flow ซึ่งสามารถจำลองได้หลายสถานการณ์จริง เช่น การส่งสินค้าผ่านเครือข่ายขนส่ง การจัดสรรทรัพยากรในระบบผลิต หรือการหาเส้นทางที่ไม่ทับซ้อนในเครือข่ายสื่อสาร ในการโปรแกรมเชิงแข่งขัน Maximum Flow เป็นหัวข้อที่ปรากฏบ่อยครั้ง ทั้งในรูปแบบตรงไปตรงมา (หา flow สูงสุด) และในรูปแบบที่ต้องจำลองปัญหาให้เป็น flow network ก่อน (เช่น โจทย์จับคู่ โจทย์วางตาราง โจทย์ path ไม่ซ้ำ)

15.1.2 นิยามเครือข่ายการไหล

นิยาม 15.1 (เครือข่ายการไหล (Flow Network))**.** เครือข่ายการไหลคือกราฟมีทิศทาง $G = (V, E)$ ที่มีคุณสมบัติดังนี้:

1. แต่ละเส้นเชื่อม $(u, v) \in E$ มี **ความจุ** (Capacity) $c(u, v) \geq 0$ ซึ่งแทนปริมาณสูงสุดที่สามารถไหลผ่านเส้นเชื่อมนั้นได้

2. มีจุดยอดพิเศษ 2 จุด ได้แก่ **ต้นทาง** (Source) s และ **ปลายทาง** (Sink) t

3. ถ้า $(u, v) \notin E$ แล้ว $c(u, v) = 0$

15.1.3 นิยามการไหล

นิยาม 15.2 (การไหล (Flow)). การไหลในเครือข่าย G คือฟังก์ชัน $f : V \times V \to \mathbb{R}$ ที่มีคุณสมบัติ 3 ประการ:

1. ข้อจำกัดความจุ (Capacity Constraint): สำหรับทุก $u, v \in V$:

$$0 \leq f(u, v) \leq c(u, v) \tag{15.1}$$

ปริมาณการไหลในเส้นเชื่อมใดๆ ต้องไม่เกินความจุของเส้นเชื่อมนั้น และต้องไม่ติดลบ

2. การคงรูปการไหล (Flow Conservation): สำหรับทุก $u \in V \setminus \{s, t\}$:

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) \tag{15.2}$$

ปริมาณการไหลเข้าของจุดยอดใดๆ (ยกเว้น s และ t) ต้องเท่ากับปริมาณการไหลออก กล่าวคือน้ำที่เข้ามาต้องออกไปหมด ไม่มีการสะสม

3. การไหลแบบผลัดกัน (Skew Symmetry): สำหรับทุก $u, v \in V$:

$$f(u, v) = -f(v, u) \tag{15.3}$$

นิยาม 15.3 (ค่าการไหล (Value of Flow)). ค่าการไหล $|f|$ คือปริมาณสุทธิที่ไหลออกจากต้นทาง s:

$$|f| = \sum_{v \in V} f(s, v) = \sum_{v \in V} f(v, t) \tag{15.4}$$

เนื่องจากการคงรูปการไหล ปริมาณที่ออกจาก s ต้องเท่ากับปริมาณที่เข้า t

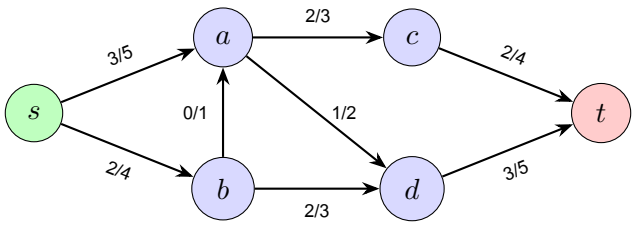
ตัวอย่าง: เครือข่ายการไหล

พิจารณาเครือข่ายในรูปที่ 15.1 ที่มี 6 จุดยอด (s, a, b, c, d, t):
เส้นเชื่อมและความจุ: $s \to a$ (5), $s \to b$ (4), $a \to c$ (3), $a \to d$ (2), $b \to d$ (3), $b \to a$ (1), $c \to t$ (4), $d \to t$ (5)
ตัวเลขบนเส้นเชื่อมในรูปแสดง (flow/capacity) ตัวอย่างการไหลที่ทำให้การไหลรวมจาก s ไป t เท่ากับ 5

รูปที่ 15.1 แสดงตัวอย่างเครือข่ายการไหล โดยเส้นเชื่อมแต่ละเส้นระบุ (ปริมาณการไหล/ความจุ)

15.2 วิธีแบบโลภและข้อจำกัด

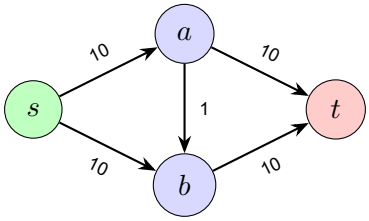
แนวคิดแรกที่นึกได้สำหรับการหา Maximum Flow คือวิธีแบบโลภ (Greedy): เริ่มต้นให้ทุกเส้นเชื่อมมีการไหลเป็น 0 แล้วหาเส้นทาง s-t เส้นทางใดเส้นทางหนึ่ง เพิ่ม (Augment) การไหลไปตามเส้นทางนั้นให้มากที่สุดเท่าที่เป็นไปได้ แล้วทำซ้ำจนกว่าจะไม่สามารถหาเส้นทางเพิ่มเติมได้



รูปที่ 15.1 เครือข่ายการไหล: ตัวเลขบนเส้นเชื่อมแสดง (flow/capacity) การไหลรวม = 5

15.2.1 ตัวอย่างที่ Greedy ล้มเหลว

พิจารณาเครือข่ายในรูปที่ 15.2

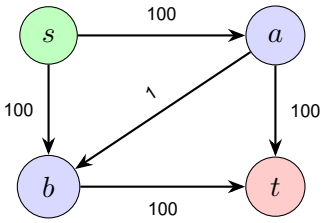


รูปที่ 15.2 เครือข่ายตัวอย่างสำหรับ Greedy: เส้นเชื่อม $a \to b$ มีความจุเพียง 1

เส้นเชื่อม: $s \to a$ (10), $s \to b$ (10), $a \to t$ (10), $a \to b$ (1), $b \to t$ (10)

สมมติ Greedy เลือกเส้นทาง $s \to a \to b \to t$ ก่อน ส่ง flow = 1 หน่วย (คอขวดคือเส้น $a \to b$ ที่มีความจุเพียง 1) หลังจากนั้น เราสามารถส่ง $s \to a \to t$ ได้อีก 9 และ $s \to b \to t$ ได้อีก 10 รวม = 20 (ซึ่งยังถูกต้องในกรณีนี้)

แต่ลองพิจารณาเครือข่ายในรูปที่ 15.3



รูปที่ 15.3 เครือข่ายตัวอย่างที่ Greedy อาจล้มเหลว: คอขวด $a \to b$ ความจุ 1 อยู่กลางกราฟ

เส้นเชื่อม: $s \to a$ (100), $s \to b$ (100), $a \to b$ (1), $a \to t$ (100), $b \to t$ (100)

Greedy เลือกเส้นทาง $s \to a \to b \to t$ ก่อน ส่ง flow = 1 (คอขวด $= 1$) จากนั้นเลือก $s \to a \to t$ ส่ง 99 และ $s \to b \to t$ ส่ง 100 รวม = 200

แต่ถ้า Greedy เลือกเส้นทาง $s \to a \to b \to t$ ก่อน และครั้งต่อมาเลือกไม่เหมาะ อาจได้ค่าต่ำกว่า 200 ปัญหาหลักคือ Greedy **ไม่สามารถยกเลิก** (Undo) การตัดสินใจที่ทำไปแล้วได้ — เมื่อส่ง flow ไปตามเส้นทางหนึ่งแล้ว เราไม่มีกลไกในการถอยกลับ

Greedy ไม่ทำงานสำหรับ Maximum Flow!

วิธี Greedy ล้มเหลวเพราะเมื่อเราเลือกเส้นทางและเติม flow แล้ว เราไม่สามารถย้อนกลับหรือเปลี่ยนแปลงได้ ถ้าเลือกเส้นทางผิด เราอาจติดกับดักและไม่สามารถไปถึงค่าการไหลสูงสุดได้ วิธีแก้คือสร้าง **กราฟคงเหลือ (Residual Graph)** ที่อนุญาตให้เรา "ยกเลิก" การไหลที่ส่งไปแล้วบางส่วนได้

15.3 กราฟคงเหลือ (Residual Graph)

กราฟคงเหลือ (Residual Graph) เป็นกุญแจสำคัญที่ทำให้เราสามารถแก้ปัญหา Maximum Flow ได้อย่างถูกต้อง โดยอนุญาตให้เรา "ยกเลิก" การไหลที่ส่งไปแล้วได้ ซึ่งเป็นสิ่งที่ Greedy ทำไม่ได้

15.3.1 ความจุคงเหลือ (Residual Capacity)

นิยาม 15.4 (ความจุคงเหลือ)**.** กำหนดการไหล f ในเครือข่าย G ความจุคงเหลือ (Residual Capacity) $c_f(u, v)$ คือปริมาณการไหลเพิ่มเติมสูงสุดที่สามารถส่งจาก u ไป v ได้:

$$c_f(u,v) = \begin{cases} c(u,v) - f(u,v) & \text{ถ้า } (u,v) \in E \\ f(v,u) & \text{ถ้า } (v,u) \in E \\ 0 & \text{อื่นๆ} \end{cases} \tag{15.5}$$

แนวคิดสำคัญ: สำหรับเส้นเชื่อมเดิม (u, v) ที่มีความจุ $c(u, v)$ และการไหล $f(u, v)$ ปริมาณที่ยังสามารถส่งเพิ่มได้คือ $c(u, v) - f(u, v)$ ในขณะเดียวกัน เราสามารถ "ยกเลิก" การไหล $f(u, v)$ ได้โดยการส่งย้อนกลับจาก v ไป u ได้สูงสุด $f(u, v)$ หน่วย นี่คือเส้นเชื่อมย้อนกลับในกราฟคงเหลือ

ตัวอย่าง: เส้นเชื่อมในกราฟคงเหลือ

ถ้าเส้นเชื่อมเดิม (u, v) มีความจุ $c(u, v) = 4$ และมีการไหล $f(u, v) = 3$:

- เส้นเชื่อมขาไป (u, v): ความจุคงเหลือ $= 4 - 3 = 1$ (ยังสามารถส่งเพิ่มได้อีก 1 หน่วย)
- เส้นเชื่อมขากลับ (v, u): ความจุคงเหลือ $= 3$ (สามารถยกเลิกการไหลย้อนกลับได้สูงสุด 3 หน่วย)

15.3.2 กราฟคงเหลือ

นิยาม 15.5 (กราฟคงเหลือ (Residual Graph))**.** กราฟคงเหลือ G_f ของเครือข่าย G ที่มีการไหล f คือกราฟที่:

- มีจุดยอดเหมือน G
- มีเส้นเชื่อม (u, v) ที่มีความจุ $c_f(u, v)$ ถ้า $c_f(u, v) > 0$

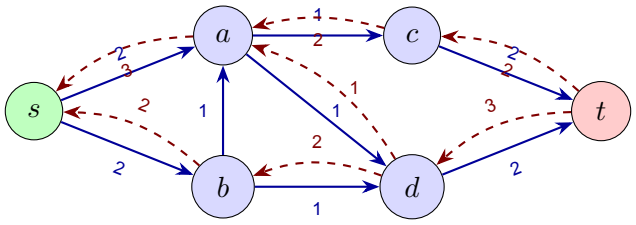
เส้นเชื่อมที่มีความจุคงเหลือเป็น 0 จะไม่ปรากฏในกราฟคงเหลือ เพราะไม่สามารถส่ง flow เพิ่มได้แล้ว

เส้นเชื่อมย้อนกลับคือกลไกการ “ยกเลิก”

เส้นเชื่อมย้อนกลับ (v, u) ในกราฟคงเหลือมีความจุเท่ากับ $f(u, v)$ หมายความว่าเราสามารถส่ง flow ย้อนจาก v ไป u ได้ ซึ่งเมื่อทำเช่นนั้นจะเท่ากับการลดการไหลบนเส้นเชื่อมเดิม (u, v) ลง กลไกนี้ช่วยให้เรา “แก้ไข” การตัดสินใจที่ผิดพลาดในอดีตได้ และเป็นสาเหตุที่ทำให้ขั้นตอนวิธี Ford-Fulkerson ทำงานได้อย่างถูกต้อง

ตัวอย่างเช่น ถ้าเราส่ง flow ไปตาม $s \to a \to b \to t$ แล้ว แต่ภายหลังพบว่าเส้นทาง $s \to a \to t$ และ $s \to b \to t$ ดีกว่า เราสามารถส่ง flow ย้อนจาก b ไป a บนเส้นเชื่อมย้อนกลับ เพื่อ “ยกเลิก” การไหลบนเส้นเชื่อม $a \to b$ ได้

รูปที่ 15.4 แสดงกราฟคงเหลือ (Residual Graph) ที่ได้จากเครือข่ายการไหลในรูปที่ 15.1 เส้นประ = เส้นย้อนกลับ (Back Edge) แสดงปริมาณที่สามารถลดการไหลได้



รูปที่ 15.4 กราฟคงเหลือ: เส้นทึบสีน้ำเงิน = ความจุที่เหลือ, เส้นประสีแดง = ปริมาณที่สามารถลดการไหลย้อนกลับได้

15.4 เส้นทางเสริม (Augmenting Path)

นิยาม 15.6 (เส้นทางเสริม (Augmenting Path))**.** เส้นทางเสริมคือเส้นทางอย่างง่าย (Simple Path) จาก s ไป t ในกราฟคงเหลือ G_f กล่าวคือ เป็นเส้นทางที่ทุกเส้นเชื่อมมีความจุคงเหลือเป็นบวก

นิยาม 15.7 (ค่าคอขวด (Bottleneck Capacity))**.** ค่าคอขวดของเส้นทางเสริม P คือความจุคงเหลือต่ำสุดของเส้นเชื่อมใดๆ บนเส้นทาง P:

$$b = \min_{(u,v) \in P} c_f(u, v) \tag{15.6}$$

เมื่อพบเส้นทางเสริม P เราสามารถเพิ่มการไหลได้ b หน่วย โดย:

- สำหรับเส้นเชื่อมขาไป $(u, v) \in E$: เพิ่ม $f(u, v)$ ขึ้น b
- สำหรับเส้นเชื่อมขากลับ (v, u): ลด $f(u, v)$ ลง b (เท่ากับการเพิ่ม $f(v, u)$ ขึ้น b)

คุณสมบัติสำคัญ: ถ้า f เป็นการไหลที่ถูกต้อง และ P เป็นเส้นทางเสริมใน G_f แล้ว f' ที่ได้จากการเสริมการไหลตาม P ด้วยค่าคอขวด b จะเป็นการไหลที่ถูกต้อง และ $|f'| = |f| + b$

ตัวอย่าง: การเสริมการไหล

พิจารณาเครือข่ายที่มีเส้นเชื่อม $s \to a$ (10), $s \to b$ (10), $a \to b$ (1), $a \to t$ (10), $b \to t$ (10)
สมมติเราส่ง flow ไปแล้ว: $s \to a \to b \to t$ ส่ง 1 หน่วย
กราฟคงเหลือจะมีเส้นเชื่อม:

- $s \to a$ (คงเหลือ 9), $s \to b$ (คงเหลือ 10)
- $a \to t$ (คงเหลือ 10), $a \to b$ (คงเหลือ 0 — เต็มแล้ว!)
- $b \to a$ (คงเหลือ 1 — เส้นเชื่อมย้อนกลับ สามารถยกเลิกการไหลได้)
- $b \to t$ (คงเหลือ 9)

ตอนนี้เราสามารถหาเส้นทางเสริมใหม่: $s \to a \to t$ (ส่ง 9 เพราะ $s \to a$ เหลือ 9) และ $s \to b \to t$ (ส่ง 10) รวมการไหล = $1 + 9 + 10 = 20$
คำตอบที่ถูกต้องคือ: ส่ง $s \to a \to t$ (10), $s \to b \to t$ (10) รวม = 20 โดยไม่ต้องใช้เส้น $a \to b$ เลย

15.5 ขั้นตอนวิธี Ford-Fulkerson

ขั้นตอนวิธี Ford-Fulkerson เป็นวิธีแก้ปัญหา Maximum Flow[2] ที่คิดค้นโดย Lester Randolph Ford และ Delbert Ray Fulkerson แนวคิดหลักคือการหาเส้นทางเสริมซ้ำๆ จนกว่าจะไม่มีเส้นทางเสริมเหลืออีกต่อไป

15.5.1 หลักการ

ขั้นตอนของ Ford-Fulkerson มีดังนี้:

1. เริ่มต้นให้ $f(u, v) = 0$ สำหรับทุกเส้นเชื่อม
2. สร้างกราฟคงเหลือ G_f
3. วนซ้ำ:
   - หาเส้นทางเสริม P จาก s ไป t ใน G_f
   - ถ้าไม่พบเส้นทางเสริม $\to$ หยุด (ได้ Maximum Flow แล้ว)
   - หาค่าคอขวด b ของ P
   - เสริมการไหลตาม P ด้วยค่า b:
     - ลดความจุคงเหลือของเส้นเชื่อมขาไป ลง b
     - เพิ่มความจุคงเหลือของเส้นเชื่อมขากลับ ขึ้น b
   - $|f| \mathrel{+}= b$

15.5.2 การวิเคราะห์ความซับซ้อน

ถ้าใช้ DFS ในการหาเส้นทางเสริม ความซับซ้อนของ Ford-Fulkerson คือ $\mathcal{O}(|f^*| \cdot E)$ เมื่อ $|f^*|$ คือค่า Maximum Flow ที่ถูกต้อง เพราะในแต่ละรอบอาจเพิ่ม flow เพียง 1 หน่วย และการหาเส้นทางด้วย DFS ใช้เวลา $\mathcal{O}(E)$

Ford-Fulkerson กับ DFS อาจช้ามาก!

ถ้าค่า Maximum Flow มีค่ามาก (เช่น 10^9) Ford-Fulkerson ที่ใช้ DFS อาจต้องทำงานถึง 10^9 รอบ ซึ่งช้าเกินไป นอกจากนี้ มีกรณีพิเศษที่ DFS สลับเลือกเส้นทางผ่านเส้นเชื่อมกลางไปมา ทำให้แต่ละรอบเพิ่ม flow เพียง 1 หน่วย ทำให้จำนวนรอบเท่ากับค่า Maximum Flow เลย
ตัวอย่างเช่น ถ้ามีเครือข่ายที่ DFS สลับเลือกเส้นทางผ่านเส้นเชื่อมกลาง $a \to b$ และ $b \to a$ (เส้นเชื่อมย้อนกลับ) ทำให้แต่ละรอบเพิ่ม flow เพียง 1 หน่วย ถ้าค่า Max Flow = 10^9 ก็ต้องทำ 10^9 รอบ!

15.6 ขั้นตอนวิธี Edmonds-Karp

ขั้นตอนวิธี Edmonds-Karp เป็นการปรับปรุง Ford-Fulkerson[2] โดยใช้ **BFS** แทน DFS ในการหาเส้นทางเสริม ซึ่งรับประกันว่าจะเลือกเส้นทางที่สั้นที่สุด (ในแง่จำนวนเส้นเชื่อม) เสมอ ค้นพบโดย Jack Edmonds และ Richard Manning Karp

15.6.1 ทำไม BFS ดีกว่า DFS

เมื่อใช้ BFS ในการหาเส้นทางเสริม เส้นทางที่ได้จะเป็นเส้นทางสั้นสุดเสมอ (Shortest Augmenting Path) ซึ่งมีคุณสมบัติสำคัญ: หลังจาก BFS รอบหนึ่ง ระยะทางสั้นสุดจาก s ถึง t ในกราฟคงเหลือ **จะไม่ลดลง** ในรอบต่อๆ ไป คุณสมบัตินี้รับประกันว่าจำนวนรอบทั้งหมดไม่เกิน $\mathcal{O}(V \cdot E)$ รอบ และแต่ละรอบ BFS ใช้เวลา $\mathcal{O}(E)$ ดังนั้นความซับซ้อนรวมคือ $\mathcal{O}(V \cdot E^2)$

Edmonds-Karp vs Ford-Fulkerson

Edmonds-Karp ดีกว่า Ford-Fulkerson (ที่ใช้ DFS) ตรงที่ความซับซ้อนไม่ขึ้นกับค่า Maximum Flow ทำให้ทำงานได้เสถียรไม่ว่าค่า flow จะมากแค่ไหน อย่างไรก็ตาม $\mathcal{O}(V \cdot E^2)$ ยังช้าเกินไปสำหรับกราฟขนาดใหญ่ (เช่น $V = 1000, E = 10000$) ดังนั้นในการแข่งขัน เรามักใช้ขั้นตอนวิธีที่เร็วกว่าอย่าง Dinic's Algorithm

15.6.2 โค้ด Edmonds-Karp (Adjacency Matrix)

```
#include <bits/stdc++.h>
using namespace std;
typedef vector<int> vi;
#define MAX_V 110
#define INF 1000000000
```

```
int res[MAX_V][MAX_V];   // Residual capacities
int mf, f, s, t;         // maxFlow, flow, source, sink
vi p;                      // parent array for BFS spanning tree
void augment(int v, int minEdge) {
    // Traverse BFS spanning tree from s to t
    if (v == s) {
        f = minEdge;
        return;
    } else if (p[v] != -1) {
        augment(p[v], min(minEdge, res[p[v]][v]));
        res[p[v]][v] -= f;    // Reduce forward capacity
        res[v][p[v]] += f;    // Increase backward capacity
    }
}
int main() {
    int V;
    scanf("%d %d %d", &V, &s, &t);
    memset(res, 0, sizeof(res));
    int E;
    scanf("%d", &E);
    for (int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        res[u][v] += w;    // If multiple parallel edges, add capacity
    }
    mf = 0;
    while (true) {
        f = 0;
        // BFS from s to t in residual graph
        vi dist(MAX_V, INF);
        dist[s] = 0;
        queue<int> q;
        q.push(s);
        p.assign(MAX_V, -1);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (u == t) break;   // Reached sink, stop BFS
            for (int v = 0; v < V; v++) {
                if (res[u][v] > 0 && dist[v] == INF) {
                    dist[v] = dist[u] + 1;
```

```
                    q.push(v);
                    p[v] = u;
                }
            }
        }
        augment(t, INF);    // Find bottleneck and augment flow
        if (f == 0) break;    // No augmenting path left -> stop
        mf += f;
    }
    printf("Maximum Flow = %d\n", mf);
    return 0;
}
```

ตัวอย่างโค้ดที่ 15.1: Edmonds-Karp Algorithm โดยใช้ Adjacency Matrix

ข้อจำกัดของ Adjacency Matrix

โค้ดข้างต้นใช้ Adjacency Matrix จึงเหมาะสำหรับกราฟขนาดเล็ก ($V \leq 100$) สำหรับกราฟที่ใหญ่กว่านั้น การใช้ Adjacency Matrix จะสิ้นเปลืองหน่วยความจำและเวลามากเกินไป ในหัวข้อถัดไป เราจะเรียนรู้ Dinic's Algorithm ที่ใช้ Adjacency List และทำงานได้เร็วกว่ามาก

15.6.3 ตัวอย่างการทำงานของ Edmonds-Karp

พิจารณาเครือข่าย 4 จุดยอด: $s = 0$, $t = 1$, เส้นเชื่อม: $0 \to 2$ (70), $0 \to 3$ (30), $2 \to 3$ (5), $2 \to 1$ (25), $3 \to 1$ (70)

รอบที่ 1: BFS หาเส้นทาง $0 \to 2 \to 1$ → คอขวด = $\min(70, 25) = 25$ → $mf = 25$

Residual: $0 \to 2$ (45), $2 \to 1$ (0), $1 \to 2$ (25), $0 \to 3$ (30), $3 \to 1$ (70)

รอบที่ 2: BFS หาเส้นทาง $0 \to 3 \to 1$ → คอขวด = $\min(30, 70) = 30$ → $mf = 55$

Residual: $0 \to 3$ (0), $3 \to 1$ (40), $1 \to 3$ (30), $0 \to 2$ (45), $2 \to 1$ (0)

รอบที่ 3: BFS หาเส้นทาง $0 \to 2 \to 3 \to 1$ → คอขวด = $\min(45, 5, 40) = 5$ → $mf = 60$

Residual: $0 \to 2$ (40), $2 \to 3$ (0), $3 \to 1$ (35), $3 \to 2$ (5)

รอบที่ 4: BFS หาเส้นทาง $0 \to 2 \to 1$ → ไม่ได้ เพราะ $2 \to 1$ = 0 แล้ว

BFS หาเส้นทางอื่น: $0 \to 2 \to 3 \to 1$ → ไม่ได้ เพราะ $2 \to 3$ = 0 แล้ว

ไม่มีเส้นทางเสริมเหลืออีก → **Maximum Flow = 60**

15.7 ขั้นตอนวิธี Dinic

ขั้นตอนวิธี Dinic (หรือ Dinitz)[11, 3] คิดค้นโดย Yefim Dinitz ในปี 1970 เป็นขั้นตอนวิธี Maximum Flow ที่**ใช้กันแพร่หลายที่สุดในการโปรแกรมเชิงแข่งขัน** เพราะมีความซับซ้อน $\mathcal{O}(V^2E)$ ซึ่งเร็วกว่า Edmonds-

Karp อย่างมาก และยิ่งไปกว่านั้น สำหรับกราฟทวิภาคี Dinic ทำงานในเวลา $\mathcal{O}(E\sqrt{V})$ ซึ่งเทียบเท่ากับขั้นตอนวิธี Hopcroft-Karp สำหรับ Bipartite Matching

15.7.1 แนวคิดหลัก

Dinic ปรับปรุง Edmonds-Karp ด้วย 2 แนวคิดสำคัญ:

1. Level Graph (กราฟระดับ): แทนที่จะหาเส้นทางเสริมทีละเส้นทาง Dinic สร้าง "กราฟระดับ" จาก BFS ก่อน โดยกำหนด `level[v]` = ระยะทางสั้นสุดจาก s ถึง v ในกราฟคงเหลือ จากนั้นพิจารณาเฉพาะเส้นเชื่อมที่ "ไปข้างหน้า" คือเส้นเชื่อม (u, v) ที่ `level[v] = level[u] + 1` เท่านั้น เส้นเชื่อมที่ไม่อยู่ใน Level Graph จะถูกข้าม

2. Blocking Flow (การไหลบล็อก): ในแต่ละ Level Graph Dinic หา **Blocking Flow** ซึ่งคือการไหลที่ทำให้ไม่มีเส้นทาง s-t เหลือใน Level Graph นั้นอีก (เส้นทางถูก "บล็อก" หมด) โดยใช้ DFS หลายเส้นทางพร้อมกัน แทนที่จะหาทีละเส้นทาง

ทำไม Dinic เร็วกว่า Edmonds-Karp

Edmonds-Karp หาเส้นทางเสริมทีละเส้นทาง ทำให้ต้องรัน BFS ใหม่ทุกรอบ ในขณะที่ Dinic สร้าง Level Graph หนึ่งครั้ง แล้วหา Blocking Flow หลายเส้นทางพร้อมกันด้วย DFS จนกว่า Level Graph จะถูก "บล็อก" หมด จึงสร้าง Level Graph ใหม่ จำนวนครั้งที่ต้องสร้าง Level Graph ใหม่ไม่เกิน $\mathcal{O}(V)$ ครั้ง เพราะระดับของ t จะเพิ่มขึ้นอย่างน้อย 1 ทุกครั้ง

15.7.2 ขั้นตอนของ Dinic

1. เริ่มต้น flow = 0
2. วนซ้ำ:
   (a) สร้าง Level Graph ด้วย BFS จาก s ในกราฟคงเหลือ
   (b) ถ้า t ไม่ถูกค้นพบ (ไม่มีเส้นทางจาก s ไป t) $\rightarrow$ หยุด คืนค่า flow
   (c) หา Blocking Flow ด้วย DFS ใน Level Graph
   (d) เพิ่ม Blocking Flow เข้ากับ flow ทั้งหมด

15.7.3 การใช้ Adjacency List สำหรับ Flow Graph

ในการเขียน Dinic เราต้องใช้ Adjacency List ที่สามารถจัดการเส้นเชื่อมย้อนกลับได้อย่างมีประสิทธิภาพ เทคนิคมาตรฐานคือเก็บเส้นเชื่อมเป็นคู่: เส้นเชื่อมขาไปและเส้นเชื่อมขากลับจะถูกเก็บไว้ติดกันใน array ดัชนี i และ $i \oplus 1$ (XOR กับ 1) เสมอ วิธีนี้ทำให้เราสามารถเข้าถึงเส้นเชื่อมย้อนกลับได้ในเวลา $\mathcal{O}(1)$

```
struct Edge {
    int to, rev;      // Destination, index of reverse edge in adj[to]
    long long cap;    // Residual capacity
```

```
};
vector<Edge> adj[MAX_V];     // Adjacency List
// Add edge u -> v with capacity cap
void addEdge(int u, int v, long long cap) {
    adj[u].push_back({v, (int)adj[v].size(), cap});
    adj[v].push_back({u, (int)adj[u].size() - 1, 0});
}
```

ตัวอย่างโค้ดที่ 15.2: โครงสร้าง Edge สำหรับ Dinic

เทคนิค XOR ดัชนี

เมื่อเราเพิ่มเส้นเชื่อม $u \to v$ ด้วยความจุ `cap` เราจะเพิ่ม 2 เส้นเชื่อมพร้อมกัน:

- เส้นเชื่อมขาไป: $u \to v$ ความจุ `cap` (ดัชนี i ใน `adj[u]`)
- เส้นเชื่อมขากลับ: $v \to u$ ความจุ 0 (ดัชนี j ใน `adj[v]`)

และเก็บ `rev` เพื่อให้สามารถหาเส้นเชื่อมคู่ได้: `adj[u][i].rev` = j และ `adj[v][j].rev` = i เวลาต้องการอัปเดตเส้นเชื่อมขาไปและขากลับพร้อมกัน: `adj[u][i].cap -= flow;` และ `adj[v][adj[u][i].rev].cap += flow;`

15.7.4 โค้ด Dinic's Algorithm

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;
struct Edge {
    int to, rev;
    ll cap;
};
class Dinic {
public:
    int n;
    vector<vector<Edge>> adj;
    vector<int> level, ptr;
    Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}
    void addEdge(int u, int v, ll cap) {
        adj[u].push_back({v, (int)adj[v].size(), cap});
        adj[v].push_back({u, (int)adj[u].size() - 1, 0});
    }
    // BFS: Build Level Graph
```

```
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto& e : adj[u]) {
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;  // If t is discovered = augmenting path exists
    }
    // DFS: Find Blocking Flow
    ll dfs(int u, int t, ll pushed) {
        if (u == t) return pushed;
        for (int& i = ptr[u]; i < (int)adj[u].size(); i++) {
            Edge& e = adj[u][i];
            if (e.cap > 0 && level[e.to] == level[u] + 1) {
                ll flow = dfs(e.to, t, min(pushed, e.cap));
                if (flow > 0) {
                    e.cap -= flow;
                    adj[e.to][e.rev].cap += flow;
                    return flow;
                }
            }
        }
        return 0;
    }
    // Find Maximum Flow from s to t
    ll maxFlow(int s, int t) {
        ll total = 0;
        while (bfs(s, t)) {
            fill(ptr.begin(), ptr.end(), 0);
            while (ll flow = dfs(s, t, INF)) {
                total += flow;
            }
        }
```

```
        return total;
    }
};
int main() {
    int n, m, s, t;
    scanf("%d %d %d %d", &n, &m, &s, &t);
    Dinic dinic(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        ll cap;
        scanf("%d %d %lld", &u, &v, &cap);
        dinic.addEdge(u, v, cap);
    }
    printf("Maximum Flow = %lld\n", dinic.maxFlow(s, t));
    return 0;
}
```

ตัวอย่างโค้ดที่ 15.3: Dinic's Algorithm (Adjacency List)

15.7.5 อธิบายโค้ด Dinic

ฟังก์ชัน `bfs(s, t)`: สร้าง Level Graph โดยกำหนด `level[v]` เป็นระยะทางสั้นสุดจาก s ถึง v ในกราฟคงเหลือ คืนค่า `true` ถ้า t ยังไปถึงได้ (มีเส้นทางเสริม) และคืนค่า `false` ถ้า t ไปไม่ถึง (ได้ Maximum Flow แล้ว)

ฟังก์ชัน `dfs(u, t, pushed)`: พยายามส่ง flow จาก u ไป t ได้มากสุด `pushed` หน่วย โดยเดินตาม Level Graph เท่านั้น (เส้นเชื่อมที่ `level[v] = level[u] + 1`) ตัวแปร `ptr[u]` เป็น **Current Arc Optimization** — เก็บตำแหน่งเส้นเชื่อมถัดไปที่จะพิจารณา ทำให้ไม่ต้องเริ่มตรวจสอบเส้นเชื่อมตั้งแต่แรกทุกรอบ

ฟังก์ชัน `maxFlow(s, t)`: วนซ้ำสร้าง Level Graph และหา Blocking Flow สลับกัน จนกว่าจะไม่มีเส้นทางเสริมเหลือ

Current Arc Optimization

ตัวแปร `ptr[u]` (บางครั้งเรียก Current Arc หรือ Iterator) เป็นการปรับปรุงสำคัญที่ทำให้ DFS ใน Dinic ไม่ต้องเริ่มตรวจสอบเส้นเชื่อมตั้งแต่ต้นทุกรอบ เมื่อเส้นเชื่อมถูกใช้จนความจุหมด (`cap = 0`) แล้ว เราจะข้ามมันไปในรอบต่อไปทันที โดยไม่ต้องพิจารณาซ้ำ การปรับปรุงนี้ทำให้ความซับซ้อนของ DFS ทั้งหมดในแต่ละ Phase เป็น $\mathcal{O}(V \cdot E)$ แทนที่จะเป็น $\mathcal{O}(E^2)$

15.7.6 การวิเคราะห์ความซับซ้อน

ตารางที่ 15.1 เปรียบเทียบความซับซ้อนของขั้นตอนวิธี Maximum Flow

ขั้นตอนวิธี	ความซับซ้อน
Ford-Fulkerson (DFS)	$\mathcal{O}(\lvert f^* \rvert \cdot E)$
Edmonds-Karp (BFS)	$\mathcal{O}(V \cdot E^2)$
Dinic	$\mathcal{O}(V^2 \cdot E)$
Dinic (กราฟทวิภาคี)	$\mathcal{O}(E\sqrt{V})$

สำหรับกราฟทั่วไปในการแข่งขัน ($V \leq 500$, $E \leq 5000$) Dinic ทำงานได้เร็วพอสมควร และสำหรับกราฟทวิภาคี Dinic ทำงานได้เร็วมาก ($\mathcal{O}(E\sqrt{V})$) ทำให้เหมาะสำหรับโจทย์ Bipartite Matching ที่มีขนาดใหญ่

ตัวอย่าง: Dinic กับกราฟทวิภาคี

สำหรับโจทย์ Maximum Bipartite Matching ที่กราฟทวิภาคีมี $|L| = |R| = 500$ และ $E = 50000$:

- Dinic ทำงานใน $\mathcal{O}(E\sqrt{V}) = \mathcal{O}(50000 \cdot \sqrt{1000}) \approx 1.6 \times 10^6$ การดำเนินการ
- Edmonds-Karp ทำงานใน $\mathcal{O}(V \cdot E^2) = \mathcal{O}(1000 \cdot 2.5 \times 10^9)$ ซึ่งช้าเกินไป

ดังนั้น Dinic เป็นตัวเลือกที่ดีกว่าอย่างเห็นได้ชัด

15.8 ทฤษฎีบท Max-Flow Min-Cut

ทฤษฎีบท Max-Flow Min-Cut เป็นหนึ่งในทฤษฎีบทที่สวยงามและสำคัญที่สุดในทฤษฎีกราฟ[2] มันเชื่อมโยงสองปัญหาที่ดูเหมือนไม่เกี่ยวข้องกัน: การหาการไหลสูงสุด และการหาตัดขั้นต่ำ

15.8.1 นิยาม s-t Cut

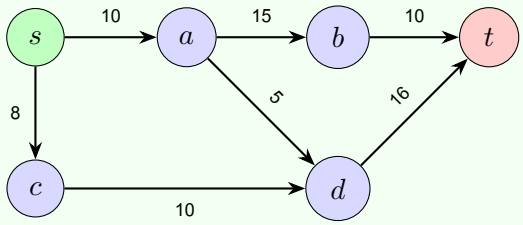
นิยาม 15.8 (s-t Cut). s-t Cut คือการแบ่งจุดยอด V ออกเป็นสองเซต A และ B โดยที่ $s \in A$ และ $t \in B$

นิยาม 15.9 (ความจุของ Cut). ความจุของ s-t cut (A, B) คือผลรวมความจุของเส้นเชื่อมที่ออกจาก A ไป B:

$$\text{cap}(A, B) = \sum_{\substack{u \in A, v \in B \\ (u,v) \in E}} c(u, v) \tag{15.7}$$

ตัวอย่าง: ความจุของ Cut

พิจารณาเครือข่าย:



ถ้า $A = \{s, a, c\}$ และ $B = \{b, d, t\}$:

- เส้นเชื่อมจาก A ไป B: $a \to b$ (15), $a \to d$ (5), $c \to d$ (10)
- $\text{cap}(A, B) = 15 + 5 + 10 = 30$

สังเกต: **ไม่นับ** เส้นเชื่อมจาก B ไป A (เช่น $d \to a$ ถ้ามี) นับเฉพาะเส้นเชื่อมที่ออกจาก A ไป B เท่านั้น

15.8.2 ทฤษฎีบท Max-Flow Min-Cut

ทฤษฎีบท 15.1 (Max-Flow Min-Cut Theorem)**.** ในเครือข่ายการไหลใดๆ ค่าการไหลสูงสุดเท่ากับความจุของ s-t cut ที่ต่ำที่สุด:

$$\max_{f} |f| = \min_{(A,B)} \text{cap}(A, B) \tag{15.8}$$

ทฤษฎีบทนี้บอกว่า ถ้าหา Maximum Flow ได้แล้ว ค่านั้นจะเท่ากับ Minimum Cut พอดี และในทางกลับกัน ถ้าหา Minimum Cut ได้ ค่านั้นก็คือ Maximum Flow

ผลพลอยได้จาก Max-Flow: หา Min-Cut

หลังจาก Dinic หยุดทำงานแล้ว เราสามารถหา Minimum Cut ได้โดย:

1. รัน BFS (หรือ DFS) จาก s ในกราฟคงเหลือตัวสุดท้าย
2. จุดยอดทุกจุดที่ไปถึงได้จาก s จะอยู่ในเซต A (S-component)
3. จุดยอดที่เหลืออยู่ในเซต B (T-component)
4. เส้นเชื่อมที่ออกจาก A ไป B ในกราฟเดิม คือ cut-set ของ Minimum Cut

ในโค้ด Dinic ข้างต้น เราสามารถใช้ `level[]` array หลังจาก `maxFlow()` ทำงานเสร็จแล้ว: จุดยอดที่ `level[v] != -1` อยู่ในเซต A และจุดยอดที่ `level[v] == -1` อยู่ในเซต B

15.8.3 ข้อสังเกตสำคัญ

จากทฤษฎีบท Max-Flow Min-Cut เราสามารถสรุปได้ว่า:

1. การไหล f เป็น Maximum Flow ก็ต่อเมื่อ **ไม่มีเส้นทางเสริม** ในกราฟคงเหลือ G_f นี่คือเงื่อนไขหยุดของ Ford-Fulkerson, Edmonds-Karp และ Dinic
2. เมื่อขั้นตอนวิธีหยุด เซตของจุดยอดที่ไปถึงได้จาก s ในกราฟคงเหลือ กับเซตที่ไปไม่ถึง เป็น Minimum Cut
3. ทุกเส้นเชื่อมใน Minimum Cut จะมีการไหลเต็มความจุ ($f(u,v) = c(u,v)$) ในขณะที่ทุกเส้นเชื่อมจาก B ไป A จะมีการไหลเป็น 0

15.9 การประยุกต์ใช้ Maximum Flow

Maximum Flow มีการประยุกต์ใช้หลากหลายมากในการโปรแกรมเชิงแข่งขัน ในหัวข้อนี้เราจะศึกษาการประยุกต์ใช้ที่สำคัญที่สุด

15.9.1 หลายต้นทาง / หลายปลายทาง (Multi-Source / Multi-Sink)

ในบางโจทย์ เราอาจมี source หลายจุด หรือ sink หลายจุด วิธีจัดการคือสร้าง **Super Source** SS และ **Super Sink** ST:

- เชื่อม SS กับ source ทุกจุดด้วยเส้นเชื่อมความจุอนันต์ (ใช้ค่ามากๆ แทน เช่น 10^9 หรือผลรวมของ capacity ทั้งหมด)
- เชื่อม sink ทุกจุดกับ ST ด้วยเส้นเชื่อมความจุอนันต์
- รัน Dinic จาก SS ไป ST ตามปกติ

ตัวอย่าง: Multi-Source Multi-Sink

สมมติมี source 3 จุด (s_1, s_2, s_3) และ sink 2 จุด (t_1, t_2):

- สร้าง SS เชื่อมไป s_1, s_2, s_3 ด้วยความจุอนันต์
- สร้าง ST รับจาก t_1, t_2 ด้วยความจุอนันต์
- รัน Max Flow จาก SS ไป ST

15.9.2 ความจุที่จุดยอด (Node Capacities)

บางโจทย์กำหนดความจุที่จุดยอดด้วย (ไม่ใช่แค่เส้นเชื่อม) วิธีจัดการคือใช้ **Node Splitting**: แบ่งจุดยอด v ออกเป็นสองจุดยอด v_{in} และ v_{out}:

- เชื่อม $v_{in} \to v_{out}$ ด้วยความจุเท่ากับความจุของจุดยอด v
- เส้นเชื่อมขาเข้าของ v เดิม เปลี่ยนมาเชื่อมเข้า v_{in}
- เส้นเชื่อมขาออกของ v เดิม เปลี่ยนมาออกจาก v_{out}

ด้วยวิธีนี้ การไหลที่ผ่านจุดยอด v จะถูกจำกัดโดยเส้นเชื่อม $v_{in} \to v_{out}$ ซึ่งมีความจุเท่ากับความจุของจุดยอด

ตัวอย่าง: Node Splitting

ถ้าจุดยอด v มีความจุ 5 และมีเส้นเชื่อมเข้าจาก a และ b และเส้นเชื่อมออกไป c และ d:

- แบ่ง v เป็น v_{in} และ v_{out}
- $a \to v_{in}$, $b \to v_{in}$ (ความจุเดิม)
- $v_{in} \to v_{out}$ (ความจุ 5 — จำกัดการไหลผ่าน v)
- $v_{out} \to c$, $v_{out} \to d$ (ความจุเดิม)

15.9.3 Bipartite Matching

ปัญหา Bipartite Matching สามารถแก้ได้ด้วย Maximum Flow โดยการสร้างเครือข่ายการไหลดังนี้:

กำหนดกราฟทวิภาคี $G = (L \cup R, E)$ ที่ต้องการหา Maximum Matching:

1. สร้าง source s และเชื่อมไปยังทุกจุดยอดใน L ด้วยความจุ 1
2. สร้าง sink t และเชื่อมทุกจุดยอดใน R ไปหา t ด้วยความจุ 1
3. กำหนดทิศทางของเส้นเชื่อมเดิมจาก L ไป R ด้วยความจุ 1
4. รัน Maximum Flow จาก s ไป t

ค่า Maximum Flow ที่ได้จะเท่ากับขนาดของ Maximum Matching ในกราฟทวิภาคีเดิม เส้นเชื่อมระหว่าง L กับ R ที่มีการไหลผ่าน 1 หน่วย คือเส้นเชื่อมใน Matching

ตัวอย่าง: Bipartite Matching

กราฟทวิภาคี: $L = \{1, 2, 3, 4, 5\}$, $R = \{a, b, c, d, e\}$

```
edges: 1-a, 1-b, 2-b, 3-a, 3-c, 4-d, 4-e, 5-c, 5-e
```

สร้าง flow graph:

- $s \to 1, s \to 2, s \to 3, s \to 4, s \to 5$ (ความจุ 1 ทุกเส้น)
- $1 \to a, 1 \to b, 2 \to b, 3 \to a, 3 \to c, 4 \to d, 4 \to e, 5 \to c, 5 \to e$ (ความจุ 1)
- $a \to t, b \to t, c \to t, d \to t, e \to t$ (ความจุ 1 ทุกเส้น)

รัน Max Flow ด้วย Dinic ได้ค่า = 5 (เช่น matching: 1-a, 2-b, 3-c, 4-d, 5-e)

Bipartite Matching ด้วย Dinic

การแปลง Bipartite Matching เป็น Max Flow เป็นเทคนิคที่ทรงพลังมาก เพราะ:

- ด้วย Dinic สามารถหา Maximum Matching ได้ในเวลา $\mathcal{O}(E\sqrt{V})$ ซึ่งเทียบเท่ากับขั้นตอนวิธี Hopcroft-Karp ที่เป็นมาตรฐานสำหรับ Bipartite Matching
- สามารถขยายเป็น Weighted Bipartite Matching ได้ (โดยใช้ Min-Cost Max-Flow ในหัวข้อถัดไป)
- โค้ดเพียงชุดเดียวสามารถใช้แก้ทั้ง Max Flow และ Bipartite Matching ได้

15.10 Hopcroft-Karp Algorithm — MCBM ที่เร็วกว่า

ในหัวข้อก่อนหน้า เราได้เห็นว่าสามารถแก้ปัญหา Maximum Cardinality Bipartite Matching (MCBM) ได้โดยการแปลงเป็นปัญหา Maximum Flow แล้วใช้ขั้นตอนวิธี Dinic หรือ Edmonds-Karp แม้ว่าวิธีนี้จะถูกต้อง แต่มีขั้นตอนวิธีเฉพาะทางที่ออกแบบมาสำหรับ Bipartite Matching โดยเฉพาะ ซึ่งมีประสิทธิภาพสูงกว่า — นั่นคือ **Hopcroft-Karp Algorithm**[2, 11]

15.10.1 ทำไมต้องมีขั้นตอนวิธีเฉพาะทาง

เมื่อเราแปลง Bipartite Matching เป็น Max Flow และใช้ขั้นตอนวิธี Edmonds-Karp ความซับซ้อนจะเป็น $\mathcal{O}(V \cdot E^2)$ ซึ่งช้าเกินไปสำหรับกราฟขนาดใหญ่ แม้จะใช้ Dinic ที่มีความซับซ้อน $\mathcal{O}(V^2E)$ สำหรับกราฟทั่วไป แต่สำหรับกราฟทวิภาคี Dinic จะทำงานในเวลา $\mathcal{O}(E\sqrt{V})$ ซึ่งเทียบเท่ากับ Hopcroft-Karp อยู่แล้ว

อย่างไรก็ตาม Hopcroft-Karp เป็นขั้นตอนวิธีที่ออกแบบมาเฉพาะสำหรับ Bipartite Matching โดยตรง จึงมีโค้ดที่กระชับกว่า ไม่ต้องสร้าง flow network ให้ยุ่งยาก และมีค่าคงที่ (Constant Factor) ที่ดีกว่าในทางปฏิบัติ ค้นพบโดย John E. Hopcroft และ Richard M. Karp ในปี 1973

15.10.2 แนวคิดหลัก

แนวคิดสำคัญของ Hopcroft-Karp คือ แทนที่จะหา **เส้นทางเสริม (Augmenting Path)** ทีละเส้นทางเหมือนในวิธีปกติ เราจะหา **หลายเส้นทางเสริมที่สั้นที่สุดพร้อมกัน** ในแต่ละรอบ กล่าวคือ:

1. **BFS Phase:** เริ่มจากจุดยอดทั้งหมดที่ยังไม่ถูกจับคู่บนฝั่งซ้าย L ค้นหาไปพร้อมกันเพื่อสร้าง “ชั้น” (Layer) ของระยะทางสั้นสุดไปยังจุดยอดแต่ละจุดบนฝั่งขวา R โดยเฉพาะจุดยอดที่ยังไม่ถูกจับคู่บน R
2. **DFS Phase:** จากจุดยอดที่ยังไม่ถูกจับคู่แต่ละจุดบน L ใช้ DFS เดินตามเส้นทางที่สั้นที่สุดเท่านั้น (ตาม Layer ที่ BFS สร้างไว้) เพื่อหาเส้นทางเสริมและเพิ่มขนาด Matching
3. **ทำซ้ำ** จนกว่าจะไม่มีเส้นทางเสริมเหลือ

เนื่องจากในแต่ละรอบเราเพิ่มหลายเส้นทางเสริมพร้อมกัน จำนวนรอบทั้งหมดจึงน้อยลงอย่างมาก — สามารถพิสูจน์ได้ว่ามีเพียง $\mathcal{O}(\sqrt{V})$ รอบ และแต่ละรอบใช้เวลา $\mathcal{O}(E)$ ดังนั้นความซับซ้อนรวมจึงเป็น $\mathcal{O}(E\sqrt{V})$

ทำมักจะมีเพียง $\mathcal{O}(\sqrt{V})$ รอบ

หลักการสำคัญ: หลังจาก $\sqrt{V}$ รอบแรก ความยาวของเส้นทางเสริมสั้นสุดจะมากกว่า $\sqrt{V}$ ซึ่งหมายความว่า Matching ปัจจุบันมีขนาดอย่างน้อย $|M| \geq |M^*| - \sqrt{V}$ เมื่อ M^* คือ Maximum Matching ดังนั้นจึงเหลือเส้นทางเสริมอีกไม่เกิน $\sqrt{V}$ เส้นทาง รวมทั้งหมดจึงไม่เกิน $\mathcal{O}(\sqrt{V})$ รอบ

15.10.3 ขั้นตอนของ Hopcroft-Karp

กำหนดกราฟทวิภาคี $G = (L \cup R, E)$ โดยเส้นเชื่อมเป็นเซต $adj[u]$ สำหรับ $u \in L$ และ `matchL[u]` = จุดยอดคู่ของ u ใน R (หรือ -1 ถ้ายังไม่มีคู่), `matchR[v]` = จุดยอดคู่ของ v ใน L (หรือ -1 ถ้ายังไม่มีคู่)

ขั้นตอน:

1. เริ่มต้นให้ `matchL[u]` = `matchR[v]` = -1 ทุกจุด, `matching` = 0
2. วนซ้ำ:
   (a) **BFS:** จากจุดยอดทุกจุดบน L ที่ `matchL[u]` $= -1$ ค้นหาเพื่อสร้างแถวลำดับ `dist[u]` แทนระยะทางสั้นสุดจากจุดเริ่มต้นที่ยังไม่มีคู่ ไปยังแต่ละจุดบน L (ผ่านเส้นเชื่อมและเส้นจับคู่สลับกัน) ถ้าไม่มีจุดยอดบน R ที่ยังไม่มีคู่ถูกค้นพบ $\rightarrow$ หยุด (ได้ Maximum Matching แล้ว)
   (b) **DFS:** สำหรับจุดยอด $u \in L$ ที่ `matchL[u]` $= -1$ ทุกจุด รัน DFS เพื่อหาเส้นทางเสริมไปตามเส้นทางที่สั้นที่สุดเท่านั้น (ใช้ `dist` เป็นเงื่อนไข) และอัปเดต `matchL`, `matchR`
   (c) เพิ่มจำนวนเส้นทางเสริมที่พบเข้า `matching`

15.10.4 โค้ด Hopcroft-Karp Algorithm

```cpp
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9;
const int MAXN = 50010;  // Maximum number of vertices on each side
vector<int> adj[MAXN];   // adj[u] = neighbors of u in L
int matchL[MAXN];        // matchL[u] = vertex in R matched with u, -1 if free
int matchR[MAXN];        // matchR[v] = vertex in L matched with v, -1 if free
int dist[MAXN];          // Distance array for BFS layering
int nL, nR;              // Number of vertices in L and R
// BFS: Build layers from all free vertices in L
// Returns true if there exists at least one augmenting path
bool bfs() {
```

```
    queue<int> q;
    for (int u = 0; u < nL; u++) {
        if (matchL[u] == -1) {
            dist[u] = 0;
            q.push(u);
        } else {
            dist[u] = INF;
        }
    }
    bool found = false;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            // If v is matched, explore through its matched vertex in L
            if (matchR[v] != -1 && dist[matchR[v]] == INF) {
                dist[matchR[v]] = dist[u] + 1;
                q.push(matchR[v]);
            } else if (matchR[v] == -1) {
                // Found a free vertex in R -> augmenting path exists
                found = true;
            }
        }
    }
    return found;
}
// DFS: Find augmenting path from u using shortest paths only
bool dfs(int u) {
    for (int v : adj[u]) {
        // Case 1: v is free -> direct match
        if (matchR[v] == -1) {
            matchL[u] = v;
            matchR[v] = u;
            return true;
        }
        // Case 2: v is matched, try to reassign through matchR[v]
        // Only follow if matchR[v] is in the next layer
        if (dist[matchR[v]] == dist[u] + 1 && dfs(matchR[v])) {
            matchL[u] = v;
            matchR[v] = u;
            return true;
        }
```

```
    }
    dist[u] = INF;  // Mark u as unusable for this BFS phase
    return false;
}
// Main function: Find Maximum Cardinality Bipartite Matching
int hopcroftKarp() {
    memset(matchL, -1, sizeof(matchL));
    memset(matchR, -1, sizeof(matchR));
    int matching = 0;
    while (bfs()) {
        for (int u = 0; u < nL; u++) {
            if (matchL[u] == -1) {
                if (dfs(u)) matching++;
            }
        }
    }
    return matching;
}
int main() {
    int E;
    scanf("%d %d %d", &nL, &nR, &E);
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u].push_back(v);  // u in L -> v in R
    }
    printf("Maximum Matching = %d\n", hopcroftKarp());
    // Print the matching pairs
    for (int u = 0; u < nL; u++) {
        if (matchL[u] != -1) {
            printf("L%d -- R%d\n", u, matchL[u]);
        }
    }
    return 0;
}
```

ตัวอย่างโค้ดที่ 15.4: Hopcroft-Karp Algorithm สำหรับ MCBM

15.10.5 อธิบายโค้ด Hopcroft-Karp

ฟังก์ชัน `bfs()`: เริ่มจากจุดยอดทุกจุดบน L ที่ยังไม่มีคู่ ค้นหาเพื่อสร้างแถวลำดับ `dist[]` โดยเดินผ่านเส้นเชื่อมจาก L ไป R แล้วตามเส้นจับคู่จาก R กลับไป L สลับกัน ถ้าพบจุดยอดบน R ที่ยังไม่มีคู่ แสดงว่า

มีเส้นทางเสริมอยู่ คืนค่า `true`

ฟังก์ชัน `dfs(u)`: พยายามหาเส้นทางเสริมจากจุดยอด $u \in L$ โดยเดินตามเส้นทางสั้นสุดเท่านั้น (ตรวจสอบด้วย `dist[matchR[v]] == dist[u] + 1`) ถ้าหาเส้นทางเสริมได้ จะอัปเดต `matchL` และ `matchR` แล้วคืนค่า `true` ถ้าหาไม่ได้ จะกำหนด `dist[u] = INF` เพื่อไม่ให้ DFS รอบต่อไปใน Phase เดียวกันมาพิจารณา u ซ้ำอีก

ฟังก์ชัน `hopcroftKarp()`: วนซ้ำสลับ BFS และ DFS จนกว่าจะไม่มีเส้นทางเสริมเหลือ ในแต่ละรอบของ BFS จะพบเส้นทางเสริมสั้นสุดหลายเส้นทางพร้อมกัน แล้วใช้ DFS เพิ่มเข้า Matching ทั้งหมด

ตัวอย่าง: Hopcroft-Karp

กราฟทวิภาคี: $L = \{0, 1, 2, 3, 4\}$, $R = \{0, 1, 2, 3, 4\}$

```
edges: 0-0, 0-1, 1-1, 2-0, 2-2, 3-3, 3-4, 4-2, 4-4
```

Phase 1: BFS พบเส้นทางเสริมสั้นสุด (ความยาว 1) จากทุกจุดยอดอิสระบน L:

- DFS จาก L_0: จับคู่ L_0-R_0
- DFS จาก L_1: จับคู่ L_1-R_1
- DFS จาก L_2: R_0 ถูกจับคู่แล้ว $\rightarrow$ ลอง $R_2 \rightarrow$ จับคู่ L_2-R_2
- DFS จาก L_3: จับคู่ L_3-R_3
- DFS จาก L_4: R_2 ถูกจับคู่แล้ว $\rightarrow$ ลอง $R_4 \rightarrow$ จับคู่ L_4-R_4

Matching = 5 (สมบูรณ์!)
ในกรณีนี้ Hopcroft-Karp ทำเพียง 1 Phase ก็ได้ Maximum Matching แล้ว ในขณะที่วิธีหาเส้นทางเสริมทีละเส้นทางอาจต้องทำหลายรอบกว่าจะเสร็จ

15.10.6 การวิเคราะห์ความซับซ้อน

ตารางที่ 15.2 เปรียบเทียบ Max Flow vs Hopcroft-Karp สำหรับ MCBM

	Max Flow (Edmonds-Karp)	Max Flow (Dinic)	Hopcroft-Karp
วัตถุประสงค์	ใช้ได้ทั่วไป	ใช้ได้ทั่วไป	เฉพาะ Bipartite
ความซับซ้อน	$\mathcal{O}(V \cdot E^2)$	$\mathcal{O}(V^2E)^*$	$\mathcal{O}(E\sqrt{V})$
ประเภทกราฟ	กราฟใดๆ	กราฟใดๆ	กราฟทวิภาคีเท่านั้น
ความยาวโค้ด	กลาง	กลาง	สั้น

*สำหรับกราฟทวิภาคี Dinic ทำงานในเวลา $\mathcal{O}(E\sqrt{V})$ เช่นเดียวกับ Hopcroft-Karp

เลือกใช้อะไรในการแข่งขัน

ถ้าโจทย์เป็น Bipartite Matching เฉยๆ โดยไม่มีเงื่อนไขอื่น Hopcroft-Karp มักเป็นตัวเลือกที่ดีกว่า เพราะโค้ดสั้นกว่าและไม่ต้องสร้าง flow network แต่ถ้าโจทย์มีเงื่อนไขเพิ่มเติม เช่น ต้องจำกัดความจุบางเส้นเชื่อม หรือต้องใช้ร่วมกับ flow อื่น ให้ใช้ Max Flow ด้วย Dinic แทน เพราะมีความยืดหยุ่นมากกว่า

ระวังการกำหนดค่าเริ่มต้นของ `dist`

ในฟังก์ชัน BFS ต้องกำหนด `dist[u] = INF` สำหรับจุดยอด $u \in L$ ที่ถูกจับคู่แล้ว และ `dist[u] = 0` สำหรับจุดยอดที่ยังไม่มีคู่ ใน DFS เมื่อหาเส้นทางเสริมไม่สำเร็จ ต้องกำหนด `dist[u] = INF` เพื่อป้องกันไม่ให้ DFS รอบต่อไปใน Phase เดียวกันมาพิจารณา u ซ้ำ ถ้าลืมทำ ขั้นตอนวิธีจะทำงานผิดพลาดหรือช้าลงอย่างมาก

15.10.7 Edge-Disjoint Path

นิยาม 15.10 (Edge-Disjoint Path)**.** เส้นทางสองเส้นทางเป็น Edge-Disjoint ถ้าไม่มีเส้นเชื่อมเหมือนกันเลย (สามารถผ่านจุดยอดเดียวกันได้ แต่ต้องไม่ใช้เส้นเชื่อมเดียวกัน)

ปัญหา: หาจำนวน Edge-Disjoint Path จาก s ไป t ที่มากที่สุด

วิธีแก้: ใช้ Maximum Flow โดยกำหนดความจุทุกเส้นเชื่อมเป็น 1 ค่า Maximum Flow ที่ได้จะเท่ากับจำนวน Edge-Disjoint Path สูงสุด

ทฤษฎีบท 15.2 (Menger's Theorem — Edge Version)**.** ในกราฟมีทิศทาง จำนวน Edge-Disjoint Path จาก s ไป t ที่มากที่สุด เท่ากับจำนวนเส้นเชื่อมน้อยที่สุดที่ต้องลบออกเพื่อตัดการเชื่อมระหว่าง s กับ t (นั่นคือ Minimum Cut)

ตัวอย่าง: Edge-Disjoint Path

กำหนดความจุทุกเส้นเชื่อมเป็น 1 รัน Maximum Flow:

- ถ้า Max Flow = 3 $\rightarrow$ มี 3 เส้นทางที่ไม่ใช้เส้นเชื่อมร่วมกัน
- ถ้า Min Cut = 3 $\rightarrow$ ต้องลบอย่างน้อย 3 เส้นเชื่อมเพื่อตัด s จาก t

นี่เป็นอีกตัวอย่างหนึ่งของทฤษฎีบท Max-Flow Min-Cut ในทางปฏิบัติ

15.10.8 Minimum Path Cover

ปัญหา Minimum Path Cover คือการหาจำนวนเส้นทางที่น้อยที่สุดที่ครอบคลุมจุดยอดทุกจุดใน DAG (Directed Acyclic Graph) โดยเส้นทางแต่ละเส้นไม่ต้องซ้ำจุดยอด

วิธีแก้ด้วย Maximum Flow:

1. สร้างกราฟทวิภาคี: แบ่งจุดยอด v ออกเป็น v_L (ฝั่งซ้าย) และ v_R (ฝั่งขวา)
2. สำหรับเส้นเชื่อมเดิม $u \rightarrow v$ ใน DAG สร้างเส้นเชื่อม $u_L \rightarrow v_R$ ในกราฟทวิภาคี
3. หา Maximum Bipartite Matching
4. คำตอบ = n − Maximum Matching

ตัวอย่าง: Minimum Path Cover

DAG ที่มี 5 จุดยอด: เส้นเชื่อม $1 \to 2$, $1 \to 3$, $2 \to 4$, $3 \to 4$, $4 \to 5$
สร้างกราฟทวิภาคี: $1_L \to 2_R$, $1_L \to 3_R$, $2_L \to 4_R$, $3_L \to 4_R$, $4_L \to 5_R$
Maximum Matching = 3 (เช่น 1_L-2_R, 3_L-4_R, 4_L-5_R หรือ 1_L-3_R, 2_L-4_R, 4_L-5_R)
ดังนั้น Minimum Path Cover = $5 - 3 = 2$ เส้นทาง (เช่น $1 \to 2 \to 4 \to 5$ และ 3 หรือ $1 \to 3 \to 4 \to 5$ และ 2)

15.11 Min-Cost Max-Flow

Min-Cost Max-Flow เป็นส่วนขยายของ Maximum Flow[3, 11] ที่เพิ่มแนวคิดของ “ต้นทุน” (Cost) เข้าไป นอกเหนือจากความจุ ในหลายโจทย์การแข่งขัน เราไม่เพียงต้องการส่ง flow ให้มากที่สุด แต่ยังต้องการให้ต้นทุนรวมต่ำที่สุดด้วย

15.11.1 นิยาม

นิยาม 15.11 (Min-Cost Max-Flow)**.** ในเครือข่ายการไหลที่แต่ละเส้นเชื่อม (u, v) มีทั้ง **ความจุ** $c(u, v)$ และ **ต้นทุน** $w(u, v)$ (ต้นทุนต่อหน่วยการไหล) ปัญหา Min-Cost Max-Flow คือการหาการไหล f ที่:

1. $|f|$ เป็น Maximum Flow (ส่งได้มากที่สุด)
2. ในบรรดา flow ที่มีค่าเท่ากับ Maximum Flow เลือก flow ที่ทำให้ $\sum_{(u,v)} w(u, v) \cdot f(u, v)$ ต่ำที่สุด

บางครั้งเราอาจต้องการส่ง flow เพียง k หน่วย (ไม่จำเป็นต้องส่งให้มากที่สุด) และต้องการให้ต้นทุนต่ำที่สุด ในกรณีนี้เราเพียงเชื่อม $s \to t$ ด้วยความจุอนันต์และต้นทุน 0 หรือหยุดเมื่อส่งได้ครบ k หน่วย

15.11.2 ขั้นตอนวิธี Successive Shortest Path

ขั้นตอนวิธีมาตรฐานสำหรับ Min-Cost Max-Flow คือ **Successive Shortest Path** (SSP) ซึ่งคล้ายกับ Ford-Fulkerson แต่แทนที่จะหาเส้นทางเสริมใดๆ เราจะหา **เส้นทางที่ถูกที่สุด** (Shortest Path ในแง่ต้นทุน) โดยใช้ Bellman-Ford หรือ SPFA ในการหาเส้นทาง

1. เริ่มต้นให้ flow = 0, cost = 0
2. วนซ้ำ:
   - หาเส้นทางสั้นสุด (ในแง่ต้นทุน) จาก s ไป t ในกราฟคงเหลือ ด้วย Bellman-Ford / SPFA (เพราะมีเส้นเชื่อมที่มีต้นทุนเป็นลบ — เส้นเชื่อมย้อนกลับ)
   - ถ้าไม่มีเส้นทาง $\to$ หยุด
   - หาค่าคอขวด b และเสริมการไหล
   - เพิ่มต้นทุนรวม: cost $+= b \times \text{dist}[t]$

เส้นเชื่อมย้อนกลับมีต้นทุนเป็นลบ

ใน Min-Cost Max-Flow เส้นเชื่อมย้อนกลับ (v, u) มีต้นทุนเท่ากับ $-w(u, v)$ (ตรงข้ามกับเส้นเชื่อมเดิม) เพราะการส่ง flow ย้อนกลับเท่ากับการ "คืน" ต้นทุนที่เสียไป ด้วยเหตุนี้ เราจึงไม่สามารถใช้ Dijkstra ได้โดยตรง (เพราะมีเส้นเชื่อมที่มีต้นทุนเป็นลบ) แต่สามารถใช้ **Johnson's Technique** (Potential) เพื่อแปลงต้นทุนให้เป็นบวกทั้งหมด แล้วใช้ Dijkstra ได้

15.11.3 โค้ด Min-Cost Max-Flow

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;
struct Edge {
    int to, rev;
    ll cap, cost;
};
class MinCostMaxFlow {
public:
    int n;
    vector<vector<Edge>> adj;
    vector<ll> dist;
    vector<int> prenode, preedge;
    MinCostMaxFlow(int n) : n(n), adj(n), dist(n),
                            prenode(n), preedge(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        adj[u].push_back({v, (int)adj[v].size(), cap, cost});
        adj[v].push_back({u, (int)adj[u].size() - 1, 0, -cost});
    }
    // SPFA: find shortest path (in terms of cost) from s to t
    bool spfa(int s, int t) {
        fill(dist.begin(), dist.end(), INF);
        vector<bool> inqueue(n, false);
        queue<int> q;
        dist[s] = 0;
        q.push(s);
        inqueue[s] = true;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            inqueue[u] = false;
            for (int i = 0; i < (int)adj[u].size(); i++) {
```

```
                Edge& e = adj[u][i];
                if (e.cap > 0 && dist[e.to] > dist[u] + e.cost) {
                    dist[e.to] = dist[u] + e.cost;
                    prenode[e.to] = u;
                    preedge[e.to] = i;
                    if (!inqueue[e.to]) {
                        q.push(e.to);
                        inqueue[e.to] = true;
                    }
                }
            }
        }
        return dist[t] != INF;
    }
    // Find Min-Cost Max-Flow from s to t
    // Returns (maxFlow, min_cost)
    pair<ll, ll> solve(int s, int t) {
        ll maxflow = 0, mincost = 0;
        while (spfa(s, t)) {
            // Find bottleneck value
            ll flow = INF;
            for (int v = t; v != s; v = prenode[v]) {
                int u = prenode[v];
                flow = min(flow, adj[u][preedge[v]].cap);
            }
            // Augment flow
            for (int v = t; v != s; v = prenode[v]) {
                int u = prenode[v];
                adj[u][preedge[v]].cap -= flow;
                adj[v][adj[u][preedge[v]].rev].cap += flow;
            }
            maxflow += flow;
            mincost += flow * dist[t];
        }
        return {maxflow, mincost};
    }
};
int main() {
    int n, m, s, t;
    scanf("%d %d %d %d", &n, &m, &s, &t);
    MinCostMaxFlow mcmf(n);
```

```
    for (int i = 0; i < m; i++) {
        int u, v;
        ll cap, cost;
        scanf("%d %d %lld %lld", &u, &v, &cap, &cost);
        mcmf.addEdge(u, v, cap, cost);
    }
    auto [maxflow, mincost] = mcmf.solve(s, t);
    printf("Maximum Flow = %lld\n", maxflow);
    printf("Minimum Cost = %lld\n", mincost);
    return 0;
}
```

ตัวอย่างโค้ดที่ 15.5: Min-Cost Max-Flow ด้วย SPFA (Successive Shortest Path)

Min-Cost Max-Flow กับ Bipartite Matching ถ่วงน้ำหนัก

Min-Cost Max-Flow สามารถใช้แก้ปัญหา **Weighted Bipartite Matching** (Assignment Problem) ได้ โดยสร้าง flow graph เหมือน Bipartite Matching ทั่วไป แต่กำหนดต้นทุนบนเส้นเชื่อมระหว่าง L กับ R เท่ากับน้ำหนักของการจับคู่ และต้นทุนบนเส้นเชื่อมจาก s และไป t เท่ากับ 0 จากนั้นรัน Min-Cost Max-Flow ผลลัพธ์ที่ได้จะเป็นการจับคู่ที่ให้ต้นทุนรวมต่ำที่สุด (หรือผลรวมน้ำหนักสูงสุด ถ้าใช้ต้นทุนเป็นลบของน้ำหนัก)

15.12 Flow Graph Modeling

ในการแข่งขัน สิ่งที่ท้าทายที่สุดไม่ใช่การเขียนโค้ด Dinic แต่เป็นการ **จำลองปัญหาให้เป็นเครือข่ายการไหล** (Flow Graph Modeling) เมื่อเรามีโค้ด Dinic แล้ว สิ่งที่เราต้องทำเมื่อเจอโจทย์คือ:

1. **มองก่อน** ว่าปัญหานี้เป็น Network Flow Problem หรือไม่ (จะคล่องขึ้นถ้าแก้ปัญหา Network Flow Problem บ่อยๆ)
2. **สร้าง Flow Graph** ที่เหมาะสม — กำหนดจุดยอด เส้นเชื่อม ความจุ source และ sink
3. **รัน Dinic** ตามปกติ

สัญญาณว่าโจทย์เป็น Network Flow

โจทย์มักเป็น Network Flow เมื่อ:

- มีการ “ส่ง” บางอย่างจากจุดหนึ่งไปอีกจุดหนึ่ง (สินค้า น้ำ ข้อมูล ฯลฯ)
- มีข้อจำกัดความจุ (Capacity) บนเส้นทาง
- ต้องการหาค่าสูงสุดหรือต่ำสุดบางอย่าง

- มีการจับคู่ (Matching) ระหว่างสองกลุ่ม
- มีการแบ่งกลุ่ม (Cut) ของจุดยอด
- มีการจัดตาราง (Scheduling) ที่มีข้อจำกัด
- มีการหาเส้นทางไม่ซ้ำ (Disjoint Path)

15.12.1 ตัวอย่างโจทย์: การจัดตารางงาน

โจทย์: มีพนักงาน N คน และงาน M งาน พนักงานคนที่ i สามารถทำงานในช่วงเวลา $[a_i, b_i]$ ได้ และแต่ละงานต้องการพนักงาน 1 คน แต่ละคนทำงานได้สูงสุด 1 งาน จงหาจำนวนงานที่ทำได้มากที่สุด

การสร้าง Flow Graph:

1. สร้าง source s และ sink t
2. สร้างจุดยอดสำหรับพนักงานแต่ละคน ($p_1, \ldots, p_N$) และงานแต่ละงาน ($j_1, \ldots, j_M$)
3. $s \to p_i$ ความจุ 1 (พนักงานทำได้สูงสุด 1 งาน)
4. $j_k \to t$ ความจุ 1 (งานต้องการพนักงาน 1 คน)
5. $p_i \to j_k$ ความจุ 1 ถ้าพนักงาน i สามารถทำงาน k ได้
6. รัน Max Flow จาก s ไป t

นี่เป็นเพียง Bipartite Matching ที่ซ่อนอยู่ในรูปแบบของโจทย์จัดตาราง

15.12.2 ตัวอย่างโจทย์: การส่งสินค้า

โจทย์: มีโรงงาน A แห่ง คลังสินค้า B แห่ง และร้านค้า C แห่ง โรงงาน i ผลิตสินค้าได้ $prod_i$ ชิ้นต่อวัน ร้านค้า k ต้องการสินค้า $need_k$ ชิ้นต่อวัน คลังสินค้า j สามารถเก็บสินค้าได้สูงสุด cap_j ชิ้น และมีเส้นทางขนส่งระหว่างจุดต่างๆ ที่มีความจุจำกัด จงหาจำนวนสินค้ามากที่สุดที่สามารถส่งจากโรงงานถึงร้านค้าได้

การสร้าง Flow Graph:

1. สร้าง Super Source SS เชื่อมไปโรงงานทุกแห่งด้วยความจุ $prod_i$
2. สร้าง Super Sink ST รับจากร้านค้าทุกแห่งด้วยความจุ $need_k$
3. คลังสินค้าใช้ Node Splitting: $j_{in} \to j_{out}$ ความจุ cap_j
4. เส้นทางขนส่งเป็นเส้นเชื่อมตามความจุที่กำหนด
5. รัน Max Flow จาก SS ไป ST

ตัวอย่างนี้ผสมผสานหลายเทคนิค: Multi-Source/Sink, Node Splitting และ Capacity Constraint

15.13 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนขั้นตอนวิธี Maximum Flow:

โจทย์แนะนำสำหรับ Maximum Flow

ระดับเริ่มต้น:

- CSES Download Speed — Maximum Flow พื้นฐาน
- UVA 820 — Internet Bandwidth (Max Flow)[20]

ระดับปานกลาง:

- CSES Police Chase — Min Cut
- CSES School Dance — Maximum Bipartite Matching
- Codeforces 1139E — Maximize Mex (Bipartite Matching)[10]
- UVA 10480 — Sabotage (Min Cut)[20]

ระดับก้าวหน้า:

- CSES Distinct Routes — Edge-Disjoint Paths
- Codeforces 1082G — Petya and Graph (Max Weight Closure)[10]
- UVA 11167 — Monkeys (Flow with Node Capacities)[20]
- SPOJ MTOTALF — Total Flow

15.14 สรุปบทที่ 15

ตารางที่ 15.3 สรุปขั้นตอนวิธีและการประยุกต์ใช้ Maximum Flow

หัวข้อ	รายละเอียด	ความซับซ้อน
Ford-Fulkerson (DFS)	หาเส้นทางเสริมด้วย DFS	$\mathcal{O}(\lvert f^* \rvert \cdot E)$
Edmonds-Karp (BFS)	หาเส้นทางเสริมด้วย BFS (shortest path)	$\mathcal{O}(V \cdot E^2)$
Dinic	Level Graph + Blocking Flow + Current Arc	$\mathcal{O}(V^2 \cdot E)$
Dinic (กราฟทวิภาคี)	Bipartite Matching ผ่าน Max Flow	$\mathcal{O}(E\sqrt{V})$
Max-Flow Min-Cut	ค่า Max Flow = ค่า Min Cut	—
Bipartite Matching	แปลงเป็น Flow Graph, ความจุเส้นเชื่อม = 1	$\mathcal{O}(E\sqrt{V})$
Edge-Disjoint Path	ความจุทุกเส้น = 1, Max Flow = จำนวนเส้นทาง	$\mathcal{O}(V^2 \cdot E)$
Min Path Cover	n - Max Bipartite Matching	$\mathcal{O}(E\sqrt{V})$
Multi-Source/Sink	สร้าง Super Source/Sink ด้วยความจุอนันต์	$\mathcal{O}(V^2 \cdot E)$
Node Capacities	Node Splitting: $v \to v_{in} \to v_{out}$	$\mathcal{O}(V^2 \cdot E)$
Min-Cost Max-Flow	Successive Shortest Path + SPFA	$\mathcal{O}(V \cdot E^2)$

กลยุทธ์แก้โจทย์ Maximum Flow

เมื่อเจอโจทย์ที่น่าจะเป็น Network Flow ให้ทำตามขั้นตอน:

1. ระบุว่าอะไรคือ source (s) และ sink (t)
2. ระบุว่าอะไรคือ “การไหล” ที่ต้องการส่ง
3. ระบุข้อจำกัดความจุบนเส้นเชื่อม (และจุดยอดถ้ามี)
4. สร้าง Flow Graph โดยใช้เทคนิคที่เหมาะสม (Multi-Source, Node Splitting, ฯลฯ)
5. รัน Dinic และตีความผลลัพธ์
6. ถ้ามีต้นทุนบนเส้นเชื่อม ใช้ Min-Cost Max-Flow แทน

การฝึกฝนสร้าง Flow Graph บ่อยๆ จะทำให้เร็วขึ้นมาก ขอแนะนำให้ลองแก้โจทย์ Network Flow จากเว็บฝึกฝนต่างๆ เช่น Kattis[15], Codeforces[10] และ CSES

โค้ด Dinic ที่ควรจดจำ

ในการแข่งขัน โค้ด Dinic (Listing 15.3) เป็นโค้ดที่คุณควรจดจำและพิมพ์ได้อย่างรวดเร็ว มันสามารถใช้แก้โจทย์ Maximum Flow ได้ทุกประเภท รวมถึง Bipartite Matching (โดยสร้าง flow graph จากกราฟทวิภาคี) และ Edge-Disjoint Path (โดยกำหนดความจุเป็น 1) สำหรับโจทย์ที่มีต้นทุน ให้ใช้โค้ด

Min-Cost Max-Flow (Listing 15.5) แทน

8

ชุดปัญหาด้านสายอักขระ

ส่วนนี้ครอบคลุมขั้นตอนวิธีประมวลผลสายอักขระ ตั้งแต่พื้นฐานสายอักขระใน C++ การค้นหารูปแบบ ไปจนถึงขั้นตอนวิธีขั้นสูงอย่าง KMP, Rabin-Karp, Z-Function และ Suffix Array พร้อมเทคนิคในการแก้ปัญหาสายอักขระที่พบบ่อยในการแข่งขัน

Chapter 16

ขั้นตอนวิธีสายอักขระ

สายอักขระ (String) คือลำดับของอักขระที่เรียงต่อกัน เช่น `"HELLO"`, `"ABCABC"`, `"algorithm"` ในการโปรแกรมเชิงแข่งขัน โจทย์ที่เกี่ยวกับสายอักขระพบได้บ่อยมาก ตั้งแต่การค้นหารูปแบบ (Pattern Matching) การตรวจสอบ Palindrome ไปจนถึงการหาคำต่อท้าย (Suffix) ที่ซ้ำกัน บทนี้จะนำเสนอขั้นตอนวิธีสายอักขระที่สำคัญที่สุดสำหรับการแข่งขัน ได้แก่ KMP, Rabin-Karp, Z-Function และ Suffix Array รวมถึงเทคนิคพื้นฐานในการจัดการสายอักขระด้วย C++[2, 3]

การประมวลผลสายอักขระมีความสำคัญอย่างยิ่งในหลายสาขา เช่น การวิเคราะห์ข้อความ (Text Analysis) การบีบอัดข้อมูล (Data Compression) ชีวสารสนเทศ (Bioinformatics) ซึ่งต้องจัดการกับสาย DNA ที่เป็นสายอักขระยาวๆ และระบบค้นหาข้อความ (Search Engine) ในการแข่งขัน โจทย์สายอักขระมักต้องใช้ขั้นตอนวิธีที่มีประสิทธิภาพสูง เพราะสายอักขระมักมีความยาวมาก (ถึง 10^5 หรือ 10^6 อักขระ) ซึ่งขั้นตอนวิธีแบบง่าย (Naive) จะทำงานช้าเกินไป

16.1 แนะนำสายอักขระใน C++

ก่อนที่จะศึกษาขั้นตอนวิธีสายอักขระขั้นสูง เรามาทบทวนพื้นฐานการใช้งานสายอักขระใน C++ กันก่อน C++ มีคลาส `std::string` ที่มีฟังก์ชันมากมายสำหรับจัดการสายอักขระ ซึ่งสะดวกและปลอดภัยกว่าการใช้แถวลำดับของ `char` แบบภาษา C

16.1.1 การสร้างและการเข้าถึงสายอักขระ

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    // Create strings in multiple ways
    string s1 = "Hello";            // From string literal
    string s2(5, 'A');              // "AAAAA" -- repeat 5 times
    string s3(s1);                  // Copy from s1
    string s4(s1, 1, 3);            // "ell" -- substring from pos 1, len 3
    string s5 = "World";
    // Access each character (0-indexed)
    cout << s1[0] << "\n";          // 'H'
    cout << s1.at(1) << "\n";       // 'e' (bounds checked)
    // String size
    cout << s1.size() << "\n";      // 5
```

```
    cout << s1.length() << "\n";   // 5 (same as size())
    // String empty?
    cout << s1.empty() << "\n";    // 0 (false)
    return 0;
}
```

ตัวอย่างโค้ดที่ 16.1: การสร้างและเข้าถึง std::string

[] vs .at()

ใช้ [] เพื่อความเร็วในการแข่งขัน (ไม่มีการตรวจสอบขอบเขต) แต่ถ้าต้องการความปลอดภัยในการดีบั๊ก ให้ใช้ .at() ซึ่งจะโยนข้อยกเว้น out_of_range ถ้าเข้าถึงตำแหน่งที่ไม่มีอยู่

16.1.2 การต่อสายอักขระและการเปรียบเทียบ

```
string a = "Hello";
string b = "World";
// Concatenate strings
string c = a + " " + b;        // "Hello World"
a += "!";                      // a = "Hello!"
// Compare strings (Lexicographic order)
string x = "apple";
string y = "banana";
cout << (x < y) << "\n";       // 1 (true) -- 'a' < 'b'
cout << (x == y) << "\n";      // 0 (false)
// Compare using compare()
cout << x.compare(y) << "\n"; // Negative value (x < y)
```

ตัวอย่างโค้ดที่ 16.2: การต่อสายอักขระและเปรียบเทียบ

การเปรียบเทียบสายอักขระใช้ **ลำดับพจนานุกรม** (Lexicographic Order) ซึ่งเปรียบเทียบอักขระทีละตัวจากซ้ายไปขวา โดยอิงจากรหัส ASCII ดังนั้นอักษรตัวใหญ่ (A–Z รหัส 65–90) จะมีค่าน้อยกว่าอักษรตัวเล็ก (a–z รหัส 97–122)

16.1.3 ฟังก์ชันสำคัญของ std::string

```
string s = "Hello World";
// substr(pos, len) -- extract substring
cout << s.substr(0, 5) << "\n";    // "Hello"
cout << s.substr(6) << "\n";       // "World" (from position 6 to end)
// find(str, pos) -- search for substring
cout << (int)s.find("World") << "\n";  // 6
```

```
cout << (int)s.find("xyz") << "\n";       // -1 (not found, actually string::npos)
// insert(pos, str) -- insert string
string t = "Helo";
t.insert(2, "l");                         // t = "Hello"
// erase(pos, len) -- erase characters
string u = "Helloo";
u.erase(5, 1);                            // u = "Hello"
// replace(pos, len, str) -- replace
string v = "Hello World";
v.replace(6, 5, "CP");                    // v = "Hello CP"
```

ตัวอย่างโค้ดที่ 16.3: ฟังก์ชันสายอักขระที่ใช้บ่อย

ความซับซ้อนของฟังก์ชันสายอักขระ

ฟังก์ชันหลายตัวของ `std::string` มีความซับซ้อนที่ซ่อนอยู่:

- `substr()` — $\mathcal{O}(k)$ เมื่อ k คือความยาวสายอักขระย่อย
- `find()` — $\mathcal{O}(nm)$ เมื่อ n คือความยาวสายอักขระ และ m คือความยาวรูปแบบที่ค้นหา
- `insert()`, `erase()`, `replace()` — $\mathcal{O}(n)$ ในกรณีทั่วไป
- การต่อสายอักขระด้วย `+` — $\mathcal{O}(n)$ เมื่อ n คือความยาวสายอักขระผลลัพธ์

ระวังการใช้ฟังก์ชันเหล่านี้ในลูป เพราะอาจทำให้โปรแกรมช้าเกินไป!

16.1.4 การรับค่าสายอักขระ

```
// Read a single word (stops at whitespace)
string word;
cin >> word;
// Read entire line
string line;
getline(cin, line);
// Read entire line after cin >> (must ignore first)
int n;
cin >> n;
cin.ignore();
string text;
getline(cin, text);
// Read multi-line strings with spaces
int numLines;
cin >> numLines;
```

```
cin.ignore();
vector<string> lines(numLines);
for (int i = 0; i < numLines; i++) {
    getline(cin, lines[i]);
}
```

ตัวอย่างโค้ดที่ 16.4: การรับค่าสายอักขระหลายรูปแบบ

16.1.5 สายอักขระเป็นแถวลำดับของอักขระ

ในการแข่งขัน เรามักมองสายอักขระเป็นแถวลำดับของอักขระ เพื่อให้สามารถประมวลผลทีละตัวได้สะดวก อักขระแต่ละตัวเก็บเป็นรหัส ASCII ซึ่งสามารถแปลงไปมาระหว่างตัวอักษรกับตัวเลขได้

```
string s = "ABCabc";
// Convert character to ASCII number
for (char c : s) {
    cout << (int)c << " ";   // 65 66 67 97 98 99
}
cout << "\n";
// Convert number to character
char ch = 65;
cout << ch << "\n";   // 'A'
// Convert lowercase/uppercase
cout << (char)tolower('A') << "\n";   // 'a'
cout << (char)toupper('a') << "\n";   // 'A'
// Check character type
cout << isdigit('5') << "\n";    // Non-zero (true)
cout << isalpha('A') << "\n";    // Non-zero (true)
cout << isalnum('5') << "\n";    // Non-zero (true)
```

ตัวอย่างโค้ดที่ 16.5: การทำงานกับอักขระ

เทคนิค: นับความถี่อักขระ

การนับความถี่ของอักขระแต่ละตัวในสายอักขระเป็นเทคนิคพื้นฐานที่ใช้บ่อย:

```
string s = "abracadabra";
int freq[256] = {0};
for (char c : s) {
    freq[(int)c]++;
}
// freq['a'] = 5, freq['b'] = 2, freq['r'] = 2, ...
```

ถ้าสายอักขระมีเฉพาะอักษรตัวเล็ก สามารถใช้ `freq[c - 'a']++` เพื่อประหยัดหน่วยความจำ (ใช้

แถวลำดับขนาด 26 แทน 256)

16.2 การค้นหารูปแบบ (Pattern Matching)

ปัญหาการค้นหารูปแบบ (Pattern Matching) เป็นหนึ่งในปัญหาพื้นฐานที่สำคัญที่สุดของสายอักขระ กำหนดสายอักขระข้อความ (Text) T ความยาว n และสายอักขระรูปแบบ (Pattern) P ความยาว m เราต้องการหาตำแหน่งทั้งหมดที่ P ปรากฏใน T

ตัวอย่างเช่น ถ้า $T =$ "AABAACAADAABAABA" และ $P =$ "AABA" แล้ว P ปรากฏที่ตำแหน่ง 0, 9, 12 (0-indexed) ใน T

16.2.1 วิธีการง่าย (Naive Pattern Matching)

วิธีที่ตรงที่สุดคือลองจับคู่ P กับทุกตำแหน่งของ T ตั้งแต่ตำแหน่ง 0 ถึง $n - m$ สำหรับแต่ละตำแหน่ง เปรียบเทียบอักขระทีละตัวจนกว่าจะไม่ตรงหรือตรงทั้งหมด

```cpp
vector<int> naiveSearch(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();
    vector<int> ans;
    for (int i = 0; i <= n - m; i++) {
        bool match = true;
        for (int j = 0; j < m; j++) {
            if (text[i + j] != pattern[j]) {
                match = false;
                break;
            }
        }
        if (match) {
            ans.push_back(i);
        }
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 16.6: Naive Pattern Matching

ตัวอย่าง: Naive Pattern Matching

$T =$ "AABAACAADAABAABA", $P =$ "AABA"

- ตำแหน่ง 0: **AABA**ACAADAABAABA ☒ ตรง! ($i = 0$)

- ตำแหน่ง 1: A**ABAA**CAADAABAABA ☒ ตรง 1 ตัวแรก ตัวที่ 2 'B' $\neq$ 'A' ☒ ไม่ตรง
- ตำแหน่ง 2: AA**BAAC**AADAABAABA ☒ 'B' $\neq$ 'A' ☒ ไม่ตรง
- ตำแหน่ง 3–8: ไม่ตรง
- ตำแหน่ง 9: AABAACAAD**AABA**ABA ☒ ตรง! ($i = 9$)
- ตำแหน่ง 12: AABAACAADAAB**AABA**BA ☒ ตรง! ($i = 12$)

ผลลัพธ์: `[0, 9, 12]`

ความซับซ้อนของ Naive Matching

วิธี Naive มีความซับซ้อน $\mathcal{O}(nm)$ ในกรณีที่เลวร้ายที่สุด เช่น T = "AAAAA...A" (ตัว A ซ้ำ n ครั้ง) และ P = "AAA...AB" (ตัว A ซ้ำ $m-1$ ครั้งตามด้วย B) จะเสียเวลาเปรียบเทียบเกือบครบทุกตำแหน่ง ถ้า $n = 10^6$ และ $m = 10^5$ วิธีนี้จะช้าเกินไป

16.2.2 ทำไมต้องมีขั้นตอนวิธีที่เร็วกว่า

ในการแข่งขัน เมื่อขนาดสายอักขระใหญ่ขึ้น ขั้นตอนวิธี $\mathcal{O}(nm)$ จะไม่สามารถทำงานได้ทันเวลา ตัวอย่างเช่น ถ้า $n = 10^6$ และ $m = 10^5$ จะมีการเปรียบเทียบถึง 10^{11} ครั้ง ซึ่งช้าเกินไปสำหรับ Time Limit ทั่วไป (1–2 วินาที) เราจึงต้องการขั้นตอนวิธีที่ทำงานได้ใน $\mathcal{O}(n+m)$ ซึ่งจะนำเสนอในหัวข้อถัดไป

16.3 ขั้นตอนวิธี KMP (Knuth-Morris-Pratt)

ขั้นตอนวิธี KMP (Knuth-Morris-Pratt) เป็นขั้นตอนวิธีค้นหารูปแบบที่มีประสิทธิภาพ $\mathcal{O}(n+m)$[2] คิดค้นโดย Donald Knuth, James Morris และ Vaughan Pratt ในปี 1977 แนวคิดหลักของ KMP คือการ "จำ" ข้อมูลที่เปรียบเทียบไปแล้ว เพื่อไม่ต้องเปรียบเทียบซ้ำ เมื่อเกิดความไม่ตรง (Mismatch) KMP จะเลื่อนรูปแบบไปยังตำแหน่งที่เหมาะสมที่สุด โดยอาศัยข้อมูลจาก **ฟังก์ชันความล้มเหลว** (Failure Function) หรือที่เรียกว่า **LPS Array**

16.3.1 ฟังก์ชันความล้มเหลว (LPS Array)

นิยาม 16.1 (LPS Array — Longest Proper Prefix which is also Suffix)**.** สำหรับสายอักขระ P ความยาว m นิยาม LPS$[i]$ คือความยาวของ **prefix ที่ยาวที่สุดที่เหมาะสม** (Proper Prefix) ของ $P[0..i]$ ที่เป็น **suffix** ของ $P[0..i]$ ไปด้วยในเวลาเดียวกัน

"เหมาะสม" (Proper) หมายถึง prefix ที่สั้นกว่าสายอักขระทั้งหมด (คือ LPS$[i] < i+1$)

ตัวอย่าง: LPS Array ของ "AABAABAAA"

i	0	1	2	3	4	5	6	7	8
$P[i]$	A	A	B	A	A	B	A	A	A
LPS$[i]$	0	1	0	1	2	3	4	5	2

อธิบายบางค่า:

- LPS$[0] = 0$: สายอักขระ "A" ไม่มี proper prefix ที่เป็น suffix
- LPS$[1] = 1$: สายอักขระ "AA" — prefix "A" = suffix "A" (ความยาว 1)
- LPS$[4] = 2$: สายอักขระ "AABAA" — prefix "AA" = suffix "AA" (ความยาว 2)
- LPS$[5] = 3$: สายอักขระ "AABAAB" — prefix "AAB" = suffix "AAB" (ความยาว 3)
- LPS$[6] = 4$: สายอักขระ "AABAABA" — prefix "AABA" = suffix "AABA" (ความยาว 4)
- LPS$[8] = 2$: สายอักขระ "AABAABAAA" — prefix "AA" = suffix "AA" (ความยาว 2)

16.3.2 การคำนวณ LPS Array

การคำนวณ LPS Array ทำได้ในเวลา $\mathcal{O}(m)$ โดยใช้สองตัวชี้ `len` และ `i`:

- `len` = ความยาวของ prefix-suffix ที่ยาวที่สุดก่อนหน้า
- `i` = ตำแหน่งปัจจุบันที่กำลังคำนวณ

```
vector<int> computeLPS(const string& pattern) {
    int m = pattern.size();
    vector<int> lps(m, 0);
    int len = 0;   // Length of previous prefix-suffix
    int i = 1;
    while (i < m) {
        if (pattern[i] == pattern[len]) {
            // Characters match -> increase length
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                // Mismatch, but there is a previous prefix-suffix
                // Try a shorter prefix-suffix
                len = lps[len - 1];
```

```
            } else {
                // No prefix-suffix at all
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}
```

ตัวอย่างโค้ดที่ 16.7: การคำนวณ LPS Array

16.3.3 การค้นหารูปแบบด้วย KMP

เมื่อได้ LPS Array แล้ว การค้นหาทำได้โดยเปรียบเทียบ T กับ P เมื่อเกิด Mismatch ที่ตำแหน่ง j ของ P แทนที่จะเริ่มเปรียบเทียบใหม่จากตำแหน่ง 0 ของ P เราจะใช้ค่า LPS$[j-1]$ เพื่อเลื่อน P ไปยังตำแหน่งที่เหมาะสม

```
vector<int> KMPSearch(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();
    vector<int> ans;
    // Compute LPS Array
    vector<int> lps = computeLPS(pattern);
    int i = 0;  // Index in text
    int j = 0;  // Index in pattern
    while (i < n) {
        if (text[i] == pattern[j]) {
            i++;
            j++;
        }
        if (j == m) {
            // Pattern found at position i - j
            ans.push_back(i - j);
            // Use LPS to continue searching (overlapping allowed)
            j = lps[j - 1];
        } else if (i < n && text[i] != pattern[j]) {
            // Mismatch
            if (j != 0) {
                j = lps[j - 1];  // Shift pattern using LPS
            } else {
                i++;  // No LPS available -> advance text
```

```
            }
        }
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 16.8: ขั้นตอนวิธี KMP สมบูรณ์

16.3.4 ตัวอย่างการทำงานของ KMP ทีละขั้นตอน

ตัวอย่าง: KMP Step-by-Step

$T =$ "ABABDABACDABABCABAB", $P =$ "ABABCABAB"

ก่อนอื่น คำนวณ LPS Array ของ P:

i	0	1	2	3	4	5	6	7	8
$P[i]$	A	B	A	B	C	A	B	A	B
LPS$[i]$	0	0	1	2	0	1	2	3	4

การค้นหา:

1. เริ่มเปรียบเทียบ $T[0..4]$ กับ $P[0..4]$: ABAB**D** vs ABAB**C** ☒ ไม่ตรงที่ $j = 4$
2. ใช้ LPS: $j =$ LPS$[3] = 2$ ☒ เปรียบเทียบต่อจาก $P[2]$ คือ 'A' กับ $T[4]$ คือ 'D' ☒ ไม่ตรง
3. ใช้ LPS: $j =$ LPS$[1] = 0$ ☒ เปรียบเทียบ $P[0]$ กับ $T[4]$ คือ 'A' vs 'D' ☒ ไม่ตรง
4. $j = 0$ ☒ เลื่อน i: $i = 5$
5. เปรียบเทียบ $T[5..13]$ กับ $P[0..8]$: ABACDABAB vs ABABCABAB ☒ ไม่ตรงที่ $j = 3$ ('C' $\neq$ 'B')
6. ... ดำเนินการต่อจนพบ P ที่ตำแหน่ง 10

ผลลัพธ์: พบ P ที่ตำแหน่ง 10

ทฤษฎีบท 16.1 (ความซับซ้อนของ KMP)**.** ขั้นตอนวิธี KMP มีความซับซ้อนเวลา $\mathcal{O}(n + m)$:

- การคำนวณ LPS Array: $\mathcal{O}(m)$
- การค้นหา: $\mathcal{O}(n)$

เนื่องจากตัวชี้ i ใน T เคลื่อนไปข้างหน้าเท่านั้น (ไม่เคยถอยหลัง) และตัวชี้ j ใน P ลดลงได้แต่ไม่เกินจำนวนครั้งที่เพิ่มขึ้น จึงรับประกันว่าการเปรียบเทียบทั้งหมดไม่เกิน $2(n + m)$ ครั้ง

การประยุกต์ใช้ KMP

- หาตำแหน่งทั้งหมดที่ Pattern ปรากฏใน Text (รวมกรณีทับซ้อน)
- นับจำนวนครั้งที่ Pattern ปรากฏ
- หาความยาวสายอักขระย่อยที่ซ้ำกัน (Period of String)
- หา shortest repeating substring: ถ้า $m \bmod (m - \text{LPS}[m-1]) = 0$ แล้วสายอักขระเป็นการซ้ำของสายอักขระย่อยความยาว $m - \text{LPS}[m-1]$

16.4 ขั้นตอนวิธี Rabin-Karp

ขั้นตอนวิธี Rabin-Karp เป็นอีกวิธีหนึ่งในการค้นหารูปแบบ[2, 11] ที่ใช้แนวคิด **Hash Function** เพื่อเปรียบเทียบสายอักขระอย่างรวดเร็ว แทนที่จะเปรียบเทียบอักขระทีละตัวเหมือน KMP ขั้นตอนวิธีนี้คิดค้นโดย Michael Rabin และ Richard Karp ในปี 1987

16.4.1 แนวคิด: Rolling Hash

แนวคิดหลักของ Rabin-Karp คือการแปลงสายอักขระเป็นค่าตัวเลข (Hash Value) แล้วเปรียบเทียบค่า Hash แทนการเปรียบเทียบสายอักขระโดยตรง หาก Hash ของสายอักขระย่อยใน T เท่ากับ Hash ของ P ก็มีโอกาสสูงที่สายอักขระจะตรงกัน

สิ่งที่ทำให้ Rabin-Karp มีประสิทธิภาพคือ **Rolling Hash** — เทคนิคที่คำนวณ Hash ของสายอักขระย่อยถัดไปจาก Hash ของสายอักขระย่อยก่อนหน้าได้ในเวลา $\mathcal{O}(1)$ โดยไม่ต้องคำนวณใหม่ทั้งหมด

16.4.2 Polynomial Hash Function

Hash Function ที่ใช้บ่อยที่สุดคือ **Polynomial Hash**:

$$H(s) = (s[0] \cdot b^{m-1} + s[1] \cdot b^{m-2} + \cdots + s[m-1] \cdot b^0) \bmod M$$

เมื่อ b คือ **ฐาน** (Base) ที่เลือก (มักใช้ 31, 137, 257) และ M คือ **มอดูลัส** (Modulus) ที่เลือก (มักใช้จำนวนเฉพาะขนาดใหญ่ เช่น $10^9 + 7$ หรือ $10^9 + 9$)

Rolling Hash: เมื่อเลื่อนหน้าต่างจาก $T[i..i+m-1]$ เป็น $T[i+1..i+m]$ สามารถคำนวณ Hash ใหม่ได้ดังนี้:

$$H(T[i+1..i+m]) = ((H(T[i..i+m-1]) - T[i] \cdot b^{m-1}) \cdot b + T[i+m]) \bmod M$$

```
typedef long long ll;
const ll BASE = 31;
```

```cpp
const ll MOD = 1e9 + 7;
vector<int> rabinKarp(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();
    vector<int> ans;
    if (m > n) return ans;
    // Compute BASE^(m-1) mod MOD
    ll h = 1;
    for (int i = 0; i < m - 1; i++) {
        h = (h * BASE) % MOD;
    }
    // Compute hash of pattern and first window of text
    ll hashP = 0, hashT = 0;
    for (int i = 0; i < m; i++) {
        hashP = (hashP * BASE + pattern[i]) % MOD;
        hashT = (hashT * BASE + text[i]) % MOD;
    }
    // Slide window by 1 each time
    for (int i = 0; i <= n - m; i++) {
        // Check hash first
        if (hashP == hashT) {
            // Verify by comparing characters (prevent collision)
            bool match = true;
            for (int j = 0; j < m; j++) {
                if (text[i + j] != pattern[j]) {
                    match = false;
                    break;
                }
            }
            if (match) {
                ans.push_back(i);
            }
        }
        // Compute rolling hash for the next window
        if (i < n - m) {
            hashT = ((hashT - text[i] * h) % MOD + MOD) % MOD;
            hashT = (hashT * BASE + text[i + m]) % MOD;
        }
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 16.9: Rabin-Karp Algorithm

Hash Collision

เนื่องจาก Hash Function เป็นการแมปจากสายอักขระ (ซึ่งมีได้ไม่จำกัด) ไปเป็นตัวเลข (ซึ่งจำกัด) จึงมีโอกาสเกิด **Collision** — สายอักขระต่างกันแต่ Hash เท่ากัน วิธีแก้:

1. **ยืนยันด้วยการเปรียบเทียบอักขระ** เมื่อ Hash เท่ากัน (ดังในโค้ดข้างบน) — ปลอดภัยแต่ช้าลงเล็กน้อย
2. **ใช้ Double Hashing** — คำนวณ Hash 2 ชุดด้วยฐานหรือมอดูลัสต่างกัน ลดโอกาส Collision อย่างมาก

16.4.3 Double Hashing

Double Hashing คือการคำนวณ Hash สองชุดด้วยพารามิเตอร์ต่างกัน ลดโอกาส Collision จากประมาณ $1/M$ เป็นประมาณ $1/M^2$

```
typedef long long ll;
typedef pair<ll,ll> pll;
const pll BASE = {31, 37};
const pll MOD = {1000000007LL, 1000000009LL};
pll operator%(pll a, pll b) {
    return {a.first % b.first, a.second % b.second};
}
pll operator*(pll a, pll b) {
    return {a.first * b.first % MOD.first,
            a.second * b.second % MOD.second};
}
pll operator+(pll a, pll b) {
    return {(a.first + b.first) % MOD.first,
            (a.second + b.second) % MOD.second};
}
pll operator-(pll a, pll b) {
    return {(a.first - b.first + MOD.first) % MOD.first,
            (a.second - b.second + MOD.second) % MOD.second};
}
// Use pair (hash1, hash2) as fingerprint of the string
// Collision probability ≈ 1/(10^9 * 10^9) ≈ 1/10^18 -- virtually impossible
```

ตัวอย่างโค้ดที่ 16.10: Rabin-Karp แบบ Double Hashing

ตัวอย่าง: Rolling Hash

$T =$ "ABABC", $P =$ "ABC", BASE = 31, MOD = $10^9 + 7$
คำนวณ Hash ของ P:

$$H(\text{"ABC"}) = (65 \cdot 31^2 + 66 \cdot 31 + 67) \bmod M = 62465 + 2046 + 67 = 64578$$

คำนวณ Hash ของหน้าต่างใน T:

- $H(\text{"ABA"}) = 65 \cdot 31^2 + 66 \cdot 31 + 65 = 62465 + 2046 + 65 = 64576 \neq 64578$
- Rolling: $H(\text{"BAB"}) = ((64576 - 65 \cdot 961) \cdot 31 + 66) \bmod M$
- ดำเนินการต่อ... จนพบ $H(\text{"ABC"}) = 64578$ ☒ พบที่ตำแหน่ง 2

16.4.4 เปรียบเทียบ Rabin-Karp กับ KMP

ตารางที่ 16.1 เปรียบเทียบ Rabin-Karp กับ KMP

คุณสมบัติ	Rabin-Karp	KMP
ความซับซ้อนเฉลี่ย	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$
ความซับซ้อนกรณีเลวร้าย	$\mathcal{O}(nm)$	$\mathcal{O}(n+m)$
การันตีไม่มี False Positive	ไม่ (ต้องยืนยัน)	ใช่
ค้นหาหลาย Pattern พร้อมกัน	ทำได้ง่าย	ทำได้ยากกว่า
การ Implement	ง่ายกว่า	ซับซ้อนกว่า
ใช้กับ 2D Pattern	สะดวกมาก	ไม่สะดวก

เลือกขั้นตอนวิธีเมื่อไร?

- ใช้ **KMP** เมื่อต้องการความแม่นยำ 100% โดยไม่ต้องยืนยัน หรือเมื่อต้องใช้ LPS Array เพื่อวัตถุประสงค์อื่น
- ใช้ **Rabin-Karp** เมื่อต้องค้นหาหลาย Pattern พร้อมกัน หรือเมื่อต้องเปรียบเทียบ Hash ของสายอักขระย่อยหลายคู่ (เช่น หาสายอักขระย่อยที่ซ้ำกัน)
- ใช้ **Rabin-Karp** เมื่อต้องทำ 2D String Matching

16.5 ฟังก์ชัน Z (Z-Function)

ฟังก์ชัน Z (Z-Function) เป็นอีกเครื่องมือหนึ่งที่ทรงพลังสำหรับการประมวลผลสายอักขระ[11, 3] มีประโยชน์หลายอย่างเช่นเดียวกับ KMP แต่มีแนวคิดที่ต่างออกไป ฟังก์ชัน Z นิยามบนสายอักขระเดียว และให้ข้อมูล

เกี่ยวกับสายอักขระย่อยที่ตรงกับ prefix ของสายอักขระนั้นเอง

16.5.1 นิยามฟังก์ชัน Z

นิยาม 16.2 (Z-Function). สำหรับสายอักขระ S ความยาว n นิยาม $Z[i]$ คือความยาวของสายอักขระย่อยที่ยาวที่สุดที่เริ่มจากตำแหน่ง i ซึ่งเป็น prefix ของ S ด้วย

โดยนิยาม $Z[0] = 0$ (หรือบางครั้งกำหนดเป็น n แล้วเริ่มคำนวณจาก $i = 1$)

ตัวอย่าง: Z-Function ของ "AABAABCAA"

i	0	1	2	3	4	5	6	7	8
$S[i]$	A	A	B	A	A	B	C	A	A
$Z[i]$	0	1	0	3	1	0	0	2	1

อธิบาย:

- $Z[1] = 1$: $S[1..] =$ "ABAABCAA" ตรงกับ prefix "A" ยาว 1 (อักขระตัวที่ 2 คือ 'B' $\neq$ 'A')
- $Z[3] = 3$: $S[3..] =$ "AABCAA" ตรงกับ prefix "AAB" ยาว 3 (อักขระตัวที่ 4 คือ 'C' $\neq$ S[3]='A')
- $Z[7] = 2$: $S[7..] =$ "AA" ตรงกับ prefix "AA" ยาว 2 (ครบสายอักขระ)
- $Z[8] = 1$: $S[8..] =$ "A" ตรงกับ prefix "A" ยาว 1

16.5.2 การคำนวณ Z-Function ใน $\mathcal{O}(n)$

การคำนวณ Z-Function อย่างมีประสิทธิภาพใช้เทคนิค "Z-Box" — เมื่อเรารู้ว่าสายอักขระย่อย $S[l..r]$ ตรงกับ prefix $S[0..r-l]$ เราสามารถใช้ข้อมูลนี้เพื่อคำนวณ $Z[i]$ สำหรับ $l \leq i \leq r$ ได้โดยไม่ต้องเปรียบเทียบทุกอักขระ

```cpp
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;   // Z-Box: [l, r) is the range matching prefix
    for (int i = 1; i < n; i++) {
        if (i < r) {
            // i is inside Z-Box -> use previously computed value
            z[i] = min(r - i, z[i - l]);
        }
        // Extend comparison
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
```

```
            // Update Z-Box
            if (i + z[i] > r) {
                l = i;
                r = i + z[i];
            }
        }
        return z;
    }
```

ตัวอย่างโค้ดที่ 16.11: การคำนวณ Z-Function

ทฤษฎีบท 16.2 (ความซับซ้อนของ Z-Function). การคำนวณ Z-Function มีความซับซ้อน $\mathcal{O}(n)$ เพราะขอบเขตด้านขวา r ของ Z-Box เคลื่อนไปทางขวาเท่านั้น และการเปรียบเทียบอักขระในลูป `while` จะเลื่อน r ไปข้างหน้าเท่านั้น ดังนั้นจำนวนครั้งทั้งหมดที่เปรียบเทียบไม่เกิน $2n$

16.5.3 การใช้ Z-Function ค้นหารูปแบบ

เราสามารถใช้ Z-Function เพื่อค้นหารูปแบบได้ โดยสร้างสายอักขระใหม่ $S = P + \texttt{"\$"} + T$ เมื่อ `$` คืออักขระพิเศษที่ไม่ปรากฏใน P หรือ T แล้วคำนวณ Z-Function ของ S ตำแหน่ง i ใดที่ $Z[i] = m$ (ความยาวของ P) จะหมายถึงพบ P ที่ตำแหน่ง $i - m - 1$ ใน T

```
vector<int> zSearch(const string& text, const string& pattern) {
    string combined = pattern + "$" + text;
    vector<int> z = zFunction(combined);
    int m = pattern.size();
    vector<int> ans;
    for (int i = m + 1; i < (int)combined.size(); i++) {
        if (z[i] == m) {
            ans.push_back(i - m - 1);
        }
    }
    return ans;
}
```

ตัวอย่างโค้ดที่ 16.12: ใช้ Z-Function ค้นหารูปแบบ

ตัวอย่าง: Z-Function Search

$T =$ "AABCAA", $P =$ "AA"

$S =$ "AA$AABCAA"

i	0	1	2	3	4	5	6	7	8
$S[i]$	A	A	$	A	A	B	C	A	A
$Z[i]$	0	1	0	2	1	0	0	2	1

$Z[3] = 2 = m \rightarrow$ พบที่ตำแหน่ง $3 - 2 - 1 = 0$ ใน T
$Z[7] = 2 = m \rightarrow$ พบที่ตำแหน่ง $7 - 2 - 1 = 4$ ใน T
$Z[8] = 1 \neq m \rightarrow$ ไม่พบ (เพราะตรงเพียง "A" ไม่ใช่ "AA")
ผลลัพธ์: `[0, 4]` (พบ "AA" ที่ตำแหน่ง 0 และ 4)
หมายเหตุ: ในตัวอย่างนี้ $Z[4] = 1$ แทนที่จะเป็น 2 เพราะ $S[4+1] = B \neq S[1] = A$ ดังนั้นไม่พบ "AA" ที่ตำแหน่ง 1 ใน T (จริงๆ แล้ว $T[1..2] =$ "AB" $\neq$ "AA") — ถูกต้อง!

การประยุกต์ใช้ Z-Function

- **ค้นหารูปแบบ:** สร้างสายอักขระ $P + \$ + T$ แล้วคำนวณ Z-Function
- **หาจำนวนครั้งที่ prefix ปรากฏ:** นับจำนวน i ที่ $Z[i] \geq$ ความยาวที่ต้องการ
- **หา Period ของสายอักขระ:** สายอักขระมี period p ก็ต่อเมื่อ $Z[n-p] = p$ สำหรับทุก p ที่หาร n ลงตัว
- **หาสายอักขระย่อยที่ซ้ำกันมากที่สุด:** หาค่า $Z[i]$ ที่มากที่สุด
- **String compression:** หาสายอักขระย่อยสั้นสุดที่ซ้ำเป็นสายอักขระเต็ม

16.6 แถวลำดับคำต่อท้าย (Suffix Array)

แถวลำดับคำต่อท้าย (Suffix Array) เป็นโครงสร้างข้อมูลที่ทรงพลังมากสำหรับการประมวลผลสายอักขระ[3, 11] มันเก็บลำดับของคำต่อท้ายทั้งหมดของสายอักขระเรียงตามลำดับพจนานุกรม ช่วยให้สามารถแก้ปัญหาสายอักขระที่ซับซ้อนได้หลายอย่าง เช่น การค้นหารูปแบบ การหาคำต่อท้ายที่ยาวที่สุดที่ซ้ำกัน การนับสายอักขระย่อยที่แตกต่างกัน และอื่นๆ

16.6.1 นิยามและแรงบันดาลใจ

นิยาม 16.3 (Suffix Array)**.** สำหรับสายอักขระ S ความยาว n **Suffix Array** SA$[0..n-1]$ เป็นแถวลำดับของดัชนีเริ่มต้นของคำต่อท้าย ทั้ง n คำ เรียงตามลำดับพจนานุกรม

ตัวอย่าง: Suffix Array ของ "BANANA"

สายอักขระ $S =$ "BANANA" ($n = 6$) คำต่อท้ายทั้งหมดคือ:

ดัชนี	คำต่อท้าย
0	BANANA
1	ANANA
2	NANA
3	ANA
4	NA
5	A

ลำดับ	SA[i]	คำต่อท้าย
0	5	A
1	3	ANA
2	1	ANANA
3	0	BANANA
4	4	NA
5	2	NANA

ดังนั้น SA $= [5, 3, 1, 0, 4, 2]$

16.6.2 การสร้าง Suffix Array แบบง่าย — $\mathcal{O}(n^2 \log n)$

วิธีง่ายที่สุดคือสร้างคำต่อท้ายทั้งหมด แล้วเรียงลำดับด้วยฟังก์ชันเปรียบเทียบสายอักขระ:

```cpp
vector<int> suffixArrayNaive(const string& s) {
    int n = s.size();
    vector<int> sa(n);
    iota(sa.begin(), sa.end(), 0);  // sa = {0, 1, 2, ..., n-1}
    sort(sa.begin(), sa.end(), [&](int a, int b) {
        return s.substr(a) < s.substr(b);
    });
    return sa;
}
```

ตัวอย่างโค้ดที่ 16.13: สร้าง Suffix Array แบบง่าย — $\mathcal{O}(n^2 \log n)$

วิธีนี้ใช้ได้กับสายอักขระสั้นๆ (เช่น $n \leq 5000$) แต่ช้าเกินไปสำหรับสายอักขระยาว เพราะการเปรียบเทียบสายอักขระสองคำต่อท้ายใช้เวลา $\mathcal{O}(n)$ และการ Sort ใช้ $\mathcal{O}(n \log n)$ ครั้งเปรียบเทียบ รวมเป็น $\mathcal{O}(n^2 \log n)$

16.6.3 การสร้าง Suffix Array ด้วย Doubling — $\mathcal{O}(n \log^2 n)$

ขั้นตอนวิธี Doubling (หรือ Prefix Doubling) เป็นวิธีสร้าง Suffix Array ที่มีประสิทธิภาพกว่า แนวคิดคือเรียงลำดับคำต่อท้ายโดยพิจารณาความยาวเพิ่มขึ้นทีละเท่า $(1, 2, 4, 8, \ldots)$ จนกว่าจะครบความยาว n

```cpp
vector<int> buildSuffixArray(const string& s) {
    int n = s.size();
    vector<int> sa(n), rank_(n), tmp(n);
    // Initialize: sort by first character
    for (int i = 0; i < n; i++) {
        sa[i] = i;
        rank_[i] = s[i];
    }
```

```
    for (int k = 1; k < n; k *= 2) {
        // Comparison function: compare (rank[i], rank[i+k])
        auto cmp = [&](int a, int b) {
            if (rank_[a] != rank_[b]) return rank_[a] < rank_[b];
            int ra = (a + k < n) ? rank_[a + k] : -1;
            int rb = (b + k < n) ? rank_[b + k] : -1;
            return ra < rb;
        };
        sort(sa.begin(), sa.end(), cmp);
        // Update rank
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; i++) {
            tmp[sa[i]] = tmp[sa[i - 1]];
            if (cmp(sa[i - 1], sa[i])) {
                tmp[sa[i]]++;
            }
        }
        rank_ = tmp;
        // If all suffixes have distinct ranks -> sorting complete
        if (rank_[sa[n - 1]] == n - 1) break;
    }
    return sa;
}
```

ตัวอย่างโค้ดที่ 16.14: สร้าง Suffix Array ด้วย Doubling — $\mathcal{O}(n \log^2 n)$

ทฤษฎีบท 16.3 (ความซับซ้อนของ Doubling). ขั้นตอนวิธี Doubling ทำการ Sort อย่างมาก $\log n$ รอบ แต่ละรอบใช้เวลา $\mathcal{O}(n \log n)$ รวมเป็น $\mathcal{O}(n \log^2 n)$ หากใช้ Radix Sort แทน Comparison Sort สามารถลดเหลือ $\mathcal{O}(n \log n)$ ได้

16.6.4 LCP Array (Longest Common Prefix)

LCP Array เป็นข้อมูลเสริมที่สำคัญมากสำหรับ Suffix Array มันเก็บความยาวของ Longest Common Prefix ระหว่างคำต่อท้ายที่อยู่ติดกันใน Suffix Array

นิยาม 16.4 (LCP Array). LCP$[i]$ = ความยาวของ Longest Common Prefix ระหว่าง SA$[i-1]$ และ SA$[i]$ สำหรับ $i = 1, 2, \ldots, n-1$ โดยนิยาม LCP$[0] = 0$

ตัวอย่าง: LCP Array ของ "BANANA"

i	SA$[i]$	คำต่อท้าย	คำต่อท้ายก่อนหน้า	LCP$[i]$
0	5	A	—	0
1	3	ANA	A	1
2	1	ANANA	ANA	3
3	0	BANANA	ANANA	0
4	4	NA	BANANA	0
5	2	NANA	NA	2

16.6.5 ขั้นตอนวิธีของ Kasai — สร้าง LCP Array ใน $\mathcal{O}(n)$

ขั้นตอนวิธีของ Kasai สร้าง LCP Array จาก Suffix Array ได้ในเวลา $\mathcal{O}(n)$ แนวคิดหลักคือถ้าเรารู้ LCP$[k]$ สำหรับคำต่อท้ายที่เริ่มจากตำแหน่ง i เมื่อเลื่อนไปตำแหน่ง $i+1$ ค่า LCP จะลดลงไม่เกิน 1

```cpp
vector<int> buildLCP(const string& s, const vector<int>& sa) {
    int n = s.size();
    vector<int> lcp(n, 0);
    vector<int> rank_(n);
    // rank[i] = position of suffix starting at i in SA
    for (int i = 0; i < n; i++) {
        rank_[sa[i]] = i;
    }
    int k = 0;
    for (int i = 0; i < n; i++) {
        if (rank_[i] == 0) {
            k = 0;
            continue;
        }
        int j = sa[rank_[i] - 1];  // Previous suffix in SA
        // Count matching characters
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
            k++;
        }
        lcp[rank_[i]] = k;
        // For the next suffix, k decreases by at most 1
        if (k > 0) k--;
    }
    return lcp;
}
```

ตัวอย่างโค้ดที่ 16.15: ขั้นตอนวิธี Kasai สร้าง LCP Array

ทฤษฎีบท 16.4 (ความซับซ้อนของ Kasai's Algorithm)**.** ขั้นตอนวิธีของ Kasai มีความซับซ้อน $\mathcal{O}(n)$ เพราะตัวแปร k เพิ่มขึ้นได้ไม่เกิน n ครั้ง (เพิ่มทีละ 1 ในลูป while) และลดลงได้ไม่เกิน n ครั้ง (ลดทีละ 1 เมื่อสิ้นสุดแต่ละรอบ) ดังนั้นจำนวนครั้งทั้งหมดของลูป while ไม่เกิน $2n$

16.6.6 การประยุกต์ใช้ Suffix Array และ LCP Array

การค้นหารูปแบบด้วย Suffix Array

เนื่องจาก Suffix Array เรียงลำดับพจนานุกรม เราสามารถใช้ **Binary Search** เพื่อค้นหารูปแบบ P ความยาว m ในเวลา $\mathcal{O}(m \log n)$:

```
// Find first position in SA where suffix >= P
int lowerBound(const string& s, const vector<int>& sa,
               const string& pattern) {
    int n = s.size();
    int m = pattern.size();
    int lo = 0, hi = n - 1, ans = n;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (s.compare(sa[mid], min(n - sa[mid], m), pattern) >= 0) {
            ans = mid;
            hi = mid - 1;
        } else {
            lo = mid + 1;
        }
    }
    return ans;
}
// Count occurrences of P
int countOccurrences(const string& s, const vector<int>& sa,
                     const string& pattern) {
    int n = s.size();
    int m = pattern.size();
    int lo = lowerBound(s, sa, pattern);
    // Find upper bound
    int hi2 = lo, ans2 = lo - 1;
    int lo2 = lo, hi22 = n - 1;
    while (lo2 <= hi22) {
        int mid = (lo2 + hi22) / 2;
```

```
        if (s.compare(sa[mid], min(n - sa[mid], m), pattern) > 0) {
            ans2 = mid - 1;
            hi22 = mid - 1;
        } else {
            lo2 = mid + 1;
            ans2 = mid;
        }
    }
    return ans2 - lo + 1;
}
```

ตัวอย่างโค้ดที่ 16.16: ค้นหารูปแบบด้วย Binary Search บน Suffix Array

หาคำต่อท้ายที่ซ้ำกันยาวที่สุด (Longest Repeated Substring)

คำต่อท้ายที่ซ้ำกันยาวที่สุดคือค่ามากที่สุดใน LCP Array:

```
int longestRepeatedSubstring(const vector<int>& lcp) {
    return *max_element(lcp.begin(), lcp.end());
}
```

ตัวอย่างโค้ดที่ 16.17: หา Longest Repeated Substring

นับจำนวนสายอักขระย่อยที่แตกต่างกัน (Number of Distinct Substrings)

จำนวนสายอักขระย่อยที่แตกต่างกันทั้งหมดของสายอักขระ S ความยาว n คือ:

$$\text{Distinct Substrings} = \frac{n(n+1)}{2} - \sum_{i=0}^{n-1} \text{LCP}[i]$$

```
long long countDistinctSubstrings(const string& s) {
    int n = s.size();
    vector<int> sa = buildSuffixArray(s);
    vector<int> lcp = buildLCP(s, sa);
    long long total = (long long)n * (n + 1) / 2;
    for (int i = 0; i < n; i++) {
        total -= lcp[i];
    }
    return total;
}
```

ตัวอย่างโค้ดที่ 16.18: นับจำนวนสายอักขระย่อยที่แตกต่างกัน

หลักการนับ Distinct Substrings

สายอักขระย่อยทั้งหมดของ S มี $\frac{n(n+1)}{2}$ สายอักขระ แต่มีการซ้ำกัน สำหรับแต่ละคำต่อท้าย $S[\text{SA}[i]..]$ มีสายอักขระย่อย $n - \text{SA}[i]$ สายอักขระ แต่ $\text{LCP}[i]$ สายอักขระแรกซ้ำกับคำต่อท้ายก่อนหน้า ดังนั้นสายอักขระย่อยใหม่ที่เพิ่มมา = $(n - \text{SA}[i]) - \text{LCP}[i]$ รวมทั้งหมดจึงเท่ากับ $\sum_i (n - \text{SA}[i] - \text{LCP}[i]) = \frac{n(n+1)}{2} - \sum \text{LCP}[i]$

16.7 ปัญหาสายอักขระที่พบบ่อย

นอกจากขั้นตอนวิธีค้นหารูปแบบแล้ว ยังมีปัญหาสายอักขระที่พบบ่อยในการแข่งขันหลายประเภท ในหัวข้อนี้จะนำเสนอวิธีแก้ปัญหาสายอักขระยอดนิยมพร้อมโค้ด C++

16.7.1 Palindrome Checking

Palindrome คือ สายอักขระที่อ่านจากซ้ายไปขวาและจากขวาไปซ้ายได้ผลเหมือนกัน เช่น "MADAM", "RACECAR", "A"

```cpp
bool isPalindrome(const string& s) {
    int n = s.size();
    for (int i = 0; i < n / 2; i++) {
        if (s[i] != s[n - 1 - i]) {
            return false;
        }
    }
    return true;
}
// Short method using C++ STL
bool isPalindromeSTL(string s) {
    return s == string(s.rbegin(), s.rend());
}
```

ตัวอย่างโค้ดที่ 16.19: ตรวจสอบ Palindrome

ตัวอย่าง

- "MADAM" ☒ Palindrome (M=M, A=A, D ตรงกลาง)
- "HELLO" ☒ ไม่ใช่ Palindrome (H $\neq$ O)
- "A" ☒ Palindrome (สายอักขระยาว 1 ตัวอักษรเป็น Palindrome เสมอ)
- "" ☒ Palindrome (สายอักขระว่างเป็น Palindrome เสมอ)

16.7.2 Longest Palindromic Substring (แบบ $\mathcal{O}(n^2)$)

ปัญหาหาสายอักขระย่อยที่เป็น Palindrome ที่ยาวที่สุด เป็นปัญหาคลาสสิก วิธี $\mathcal{O}(n^2)$ ใช้ “ขยายจากกลาง” (Expand Around Center) — ลองทุกตำแหน่งเป็นจุดศูนย์กลางของ Palindrome แล้วขยายออกไป

```
string longestPalindromicSubstring(const string& s) {
    int n = s.size();
    if (n == 0) return "";
    int start = 0, maxLen = 1;
    // Expand from center
    auto expand = [&](int left, int right) {
        while (left >= 0 && right < n && s[left] == s[right]) {
            left--;
            right++;
        }
        // Now s[left+1..right-1] is a palindrome
        int len = right - left - 1;
        if (len > maxLen) {
            maxLen = len;
            start = left + 1;
        }
    };
    for (int i = 0; i < n; i++) {
        // Odd-length palindrome (center = i)
        expand(i, i);
        // Even-length palindrome (center = between i and i+1)
        expand(i, i + 1);
    }
    return s.substr(start, maxLen);
}
```

ตัวอย่างโค้ดที่ 16.20: Longest Palindromic Substring แบบ $\mathcal{O}(n^2)$

ตัวอย่าง: Longest Palindromic Substring

$s =$ ”BANANA”

- ลองจุดกลาง 0 (B): Palindrome ”B” ยาว 1
- ลองจุดกลาง 1 (A): Palindrome ”ANA” ยาว 3 (ขยาย A□NA□AN□BANAN □)
- ลองจุดกลาง 3 (A): Palindrome ”ANANA” ยาว 5
- ผลลัพธ์: **”ANANA”** ยาว 5

Manacher's Algorithm

มีขั้นตอนวิธี $\mathcal{O}(n)$ สำหรับหา Longest Palindromic Substring ชื่อว่า **Manacher's Algorithm** ซึ่งคล้ายกับการคำนวณ Z-Function แต่ประยุกต์กับ Palindrome อย่างไรก็ตาม วิธี $\mathcal{O}(n^2)$ มักเพียงพอสำหรับการแข่งขัน เว้นแต่โจทย์จะกำหนด n ถึง 10^6

16.7.3 Anagram Detection

Anagram คือสายอักขระสองสายอักขระที่มีอักขระเหมือนกันทุกประการแต่เรียงลำดับต่างกัน เช่น `"LISTEN"` และ `"SILENT"` เป็น Anagram ของกันและกัน

```
bool isAnagram(string a, string b) {
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    return a == b;
}
// Faster method: count character frequencies -- O(n)
bool isAnagramFast(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    int freq[256] = {0};
    for (char c : a) freq[(int)c]++;
    for (char c : b) {
        freq[(int)c]--;
        if (freq[(int)c] < 0) return false;
    }
    return true;
}
```

ตัวอย่างโค้ดที่ 16.21: ตรวจสอบ Anagram

ตัวอย่าง: หา Anagram ทั้งหมดในกลุ่มคำ

กลุ่มคำ `["eat", "tea", "tan", "ate", "nat", "bat"]`
หลังจาก Sort แต่ละคำ:

- ”eat” ⊠ ”aet”, ”tea” ⊠ ”aet”, ”ate” ⊠ ”aet” ⊠ กลุ่มเดียวกัน
- ”tan” ⊠ ”ant”, ”nat” ⊠ ”ant” ⊠ กลุ่มเดียวกัน
- ”bat” ⊠ ”abt” ⊠ กลุ่มของตัวเอง

วิธี: ใช้ `map<string, vector<string>>` โดย key คือสายอักขระที่ sort แล้ว

16.7.4 String Matching with Wildcards

ปัญหา String Matching ที่มี Wildcard เช่น `'?'` แทนอักขระใดๆ ก็ได้ หรือ `'*'` แทนสายอักขระใดๆ ก็ได้ (รวมถึงสายอักขระว่าง) วิธีแก้ที่พบบ่อยคือใช้ **Dynamic Programming**

```cpp
// pattern has '?' (any single character) and '*' (any string)
bool wildcardMatch(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();
    // dp[i][j] = does text[0..i-1] match pattern[0..j-1]
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    // Pattern starting with consecutive * matches empty string
    for (int j = 1; j <= m; j++) {
        if (pattern[j - 1] == '*') {
            dp[0][j] = dp[0][j - 1];
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (pattern[j - 1] == '*') {
                // * represents empty string or at least 1 character
                dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
            } else if (pattern[j - 1] == '?' || text[i - 1] == pattern[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            }
        }
    }
    return dp[n][m];
}
```

ตัวอย่างโค้ดที่ 16.22: Wildcard Matching ด้วย DP

ความซับซ้อนของ Wildcard DP

วิธี DP มีความซับซ้อน $\mathcal{O}(nm)$ ซึ่งเพียงพอสำหรับ $n, m \leq 5000$ แต่ถ้าสายอักขระยาวกว่านั้น ต้องใช้เทคนิคอื่น เช่น ใช้ NFA (Nondeterministic Finite Automaton) หรือขั้นตอนวิธีเฉพาะทาง

16.7.5 Longest Common Substring

ปัญหาหาสายอักขระย่อยที่ยาวที่สุดที่ปรากฏในสายอักขระสองสายอักขระ เป็นปัญหาที่สามารถแก้ได้ด้วยหลายวิธี

วิธีที่ 1: ใช้ Suffix Array — สร้างสายอักขระรวม $S = s_1 + \text{'\#'} + s_2$ สร้าง Suffix Array และ LCP Array แล้วหาค่า LCP มากที่สุดที่มาจากสองสายอักขระต่างกัน

```cpp
string longestCommonSubstring(const string& a, const string& b) {
    int n = a.size();
    int m = b.size();
    // dp[i][j] = LCSubstr length ending at a[i-1] and b[j-1]
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    int maxLen = 0, endPos = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
                if (dp[i][j] > maxLen) {
                    maxLen = dp[i][j];
                    endPos = i;
                }
            }
        }
    }
    return a.substr(endPos - maxLen, maxLen);
}
```

ตัวอย่างโค้ดที่ 16.23: Longest Common Substring ด้วย DP — $\mathcal{O}(n \cdot m)$

ตัวอย่าง: Longest Common Substring

$a =$ "ABABC", $b =$ "BABCA"

	B	A	B	C	A
A	0	1	0	0	1
B	1	0	2	0	0
A	0	2	0	0	1
B	1	0	3	0	0
C	0	0	0	4	0

ค่ามากสุด = 4 ที่ $dp[5][4]$ ⊠ Longest Common Substring = **"BABC"**

16.7.6 การนับสายอักขระย่อยที่เป็น Palindrome

```cpp
int countPalindromicSubstrings(const string& s) {
    int n = s.size();
    int count = 0;
```

```
    for (int center = 0; center < n; center++) {
        // Odd-length palindrome
        int left = center, right = center;
        while (left >= 0 && right < n && s[left] == s[right]) {
            count++;
            left--;
            right++;
        }
        // Even-length palindrome
        left = center;
        right = center + 1;
        while (left >= 0 && right < n && s[left] == s[right]) {
            count++;
            left--;
            right++;
        }
    }
    return count;
}
```

ตัวอย่างโค้ดที่ 16.24: นับจำนวน Palindromic Substring

ตัวอย่าง

$s =$ "AAA"

- จุดกลาง 0: "A" (คี่), (คู่) "AA" → 2
- จุดกลาง 1: "A" (คี่), "AAA" (คี่ขยาย), (คู่) "AA" → 3
- จุดกลาง 2: "A" (คี่) → 1

รวม = 6 (ได้แก่ "A" ที่ตำแหน่ง 0, 1, 2 + "AA" ที่ตำแหน่ง 01, 12 + "AAA")

16.8 โจทย์แนะนำ

ต่อไปนี้เป็นรายการโจทย์ฝึกหัดสำหรับแต่ละหัวข้อ:

ตารางที่ 16.2 โจทย์ฝึกหัดสายอักขระแนะนำ

หัวข้อ	โจทย์	แพลตฟอร์ม
KMP	NHAY - A Needle in the Haystack	SPOJ
KMP	Find Pattern in String	AtCoder
Rabin-Karp	Pattern Matching	Codeforces[10]
Z-Function	Z-Function	Codeforces Edu[10]
Z-Function	Password	Codeforces 126B[10]
Suffix Array	SARRAY - Suffix Array	SPOJ
Suffix Array	Distinct Substrings	SPOJ SUBST1
Suffix Array	LCS - Longest Common Substring	SPOJ
Palindrome	Manacher	Codeforces Edu[10]
Anagram	Group Anagrams	LeetCode 49
LCS	Longest Common Substring	Various
Wildcard	Wildcard Matching	LeetCode 44

คำแนะนำในการฝึก

1. เริ่มจากโจทย์ง่ายก่อน (Naive Matching, Palindrome Checking) แล้วค่อยขยับไปโจทย์ที่ต้องใช้ขั้นตอนวิธีเฉพาะ
2. ฝึก Implement KMP, Z-Function, และ Rabin-Karp จนคล่อง โดยเฉพาะการคำนวณ LPS Array และ Z-Function ซึ่งเป็นส่วนที่ผิดพลาดได้ง่าย
3. สำหรับ Suffix Array ควรเข้าใจแนวคิด Doubling และ Kasai's Algorithm อย่างลึกซึ้ง และฝึก Implement จนได้โค้ดที่ถูกต้อง
4. ลองใช้ Hash Prefix เปรียบเทียบสายอักขระย่อยในโจทย์ที่ต้องเปรียบเทียบหลายครั้ง เพื่อให้คุ้นเคยกับเทคนิคนี้
5. อ่าน Editorials ของโจทย์ที่ทำไม่ได้ และลอง Implement วิธีแก้เองก่อนดูโค้ดตัวอย่าง

16.9 สรุปบทที่ 16

ในบทนี้ เราได้ศึกษาขั้นตอนวิธีสายอักขระที่สำคัญหลายตัว ตั้งแต่พื้นฐานการใช้ `std::string` ไปจนถึงขั้นตอนวิธีขั้นสูงอย่าง KMP, Rabin-Karp, Z-Function และ Suffix Array แต่ละขั้นตอนวิธีมีจุดเด่นและเหมาะกับปัญหาที่ต่างกัน

16.9.1 ตารางเปรียบเทียบขั้นตอนวิธีสายอักขระ

ตารางที่ 16.3 เปรียบเทียบขั้นตอนวิธีสายอักขระทั้งหมด

ขั้นตอนวิธี	ปัญหาหลัก	ความซับซ้อน	หมายเหตุ
Naive Matching	Pattern Matching	$\mathcal{O}(nm)$	ง่าย แต่ช้า
KMP	Pattern Matching	$\mathcal{O}(n+m)$	ใช้ LPS Array
Rabin-Karp	Pattern Matching	$\mathcal{O}(n+m)$ เฉลี่ย	ใช้ Hash
Z-Function	Prefix Matching	$\mathcal{O}(n)$	ใช้ Z-Box
Suffix Array	หลายปัญหา	$\mathcal{O}(n \log^2 n)$	ทรงพลัง
LCP Array	Common Prefix	$\mathcal{O}(n)$	ใช้ร่วมกับ SA

16.9.2 เลือกขั้นตอนวิธีเมื่อไร

คู่มือเลือกขั้นตอนวิธีสายอักขระ

1. **ค้นหา Pattern ใน Text** — ใช้ KMP ($\mathcal{O}(n+m)$) หรือ Z-Function ($\mathcal{O}(n+m)$)
2. **ค้นหาหลาย Pattern พร้อมกัน** — ใช้ Rabin-Karp (เก็บ Hash ของ Pattern ทั้งหมดใน Set) หรือ Aho-Corasick (ขั้นสูงกว่า แต่มีประสิทธิภาพ)
3. **เปรียบเทียบสายอักขระย่อยหลายคู่** — ใช้ Rabin-Karp (เตรียม Hash Prefix แล้วคำนวณ Hash สายอักขระย่อยใดๆ ได้ใน $\mathcal{O}(1)$)
4. **หาคำต่อท้ายที่ซ้ำกัน / Distinct Substrings** — ใช้ Suffix Array + LCP Array
5. **หา Palindrome** — ใช้ Expand Around Center ($\mathcal{O}(n^2)$) หรือ Manacher ($\mathcal{O}(n)$)
6. **Longest Common Substring** — ใช้ Suffix Array + LCP ($\mathcal{O}(n \log n)$) หรือ DP ($\mathcal{O}(n \cdot m)$)
7. **Wildcard Matching** — ใช้ DP ($\mathcal{O}(n \cdot m)$)

16.9.3 เทคนิคสำหรับโจทย์สายอักขระในการแข่งขัน

เทคนิคสำคัญ

1. **ใช้ Hash Prefix สำหรับเปรียบเทียบสายอักขระย่อย** — เตรียมแถวลำดับ Hash Prefix และแถวลำดับกำลังของ BASE จากนั้นสามารถคำนวณ Hash ของสายอักขระย่อย $S[l..r]$ ได้ใน $\mathcal{O}(1)$:

```
// Prepare data
const ll BASE = 31, MOD = 1e9 + 7;
```

```
vector<ll> h(n+1), pw(n+1);
pw[0] = 1;
for (int i = 0; i < n; i++) {
        h[i+1] = (h[i] * BASE + s[i]) % MOD;
        pw[i+1] = (pw[i] * BASE) % MOD;
}
// Hash of s[l..r] (0-indexed, inclusive)
auto getHash = [&](int l, int r) -> ll {
        return (h[r+1] - h[l] * pw[r-l+1] % MOD + MOD) % MOD;
};
```

2. **ระวัง 0-indexed vs 1-indexed** — การทำงานกับสายอักขระใน C++ ใช้ 0-indexed แต่บางครั้งการใช้ 1-indexed (เติมอักขระ dummy ข้างหน้า) จะทำให้โค้ดง่ายขึ้น
3. **สายอักขระย่อยของสายอักขระย่อย** — ถ้าต้องเปรียบเทียบสายอักขระย่อยหลายคู่ อย่าใช้ `substr()` เพราะช้า ($\mathcal{O}(k)$ ต่อครั้ง) ใช้ Hash เปรียบเทียบแทน
4. **เทคนิค $P + \$ + T$** — เทคนิคนี้ใช้ได้กับทั้ง Z-Function และ KMP โดยเชื่อม Pattern กับ Text ด้วยอักขระพิเศษ ทำให้สามารถค้นหาได้ในครั้งเดียว
5. **Reverse String** — หลายปัญหาสายอักขระสามารถแก้ได้ง่ายขึ้นเมื่อพิจารณาสายอักขระย้อนกลับ เช่น การหา Longest Palindromic Substring สามารถทำได้โดยหา LCS ระหว่าง S กับ S^R (สายอักขระย้อนกลับ)

16.9.4 ข้อผิดพลาดที่พบบ่อย (Common Pitfalls)

ข้อผิดพลาดที่ต้องระวัง

1. **ลืมกรณีขอบเขต** — เช่น สายอักขระว่าง, Pattern ยาวกว่า Text, สายอักขระที่มีอักขระเพียงตัวเดียว ควรทดสอบกับกรณีเหล่านี้เสมอ
2. **Overflow ใน Hash** — เมื่อคูณ BASE หลายครั้ง ค่า Hash อาจเกิน `long long` ได้ ต้อง `%` MOD ทุกขั้นตอน
3. **Hash ติดลบ** — การลบ Hash อาจให้ค่าติดลบ ต้องบวก MOD ก่อน mod อีกครั้ง: `((a - b) % MOD + MOD) % MOD`
4. **สับสน `size_t` กับ `int`** — `s.size()` คืนค่า `size_t` (unsigned) ซึ่ง `s.size() - 1` จะเป็นค่าบวกขนาดใหญ่มากเมื่อสายอักขระว่าง ควรแปลงเป็น `int` ก่อน: `(int)s.size() - 1`

5. **อักขระพิเศษใน Suffix Array** — เมื่อใช้เทคนิค $P + \$ + T$ ต้องแน่ใจว่า $ ไม่ปรากฏใน P หรือ T มิฉะนั้นผลลัพธ์จะผิด

6. **KMP กับกรณีทับซ้อน** — KMP สามารถหา Pattern ที่ทับซ้อนกันได้ (เช่น หา "AA" ใน "AAA" จะพบที่ตำแหน่ง 0 และ 1) ในขณะที่บางขั้นตอนวิธีอื่นอาจไม่สามารถทำได้

16.9.5 หัวข้อขั้นสูงเพิ่มเติม

นอกเหนือจากขั้นตอนวิธีที่กล่าวมาในบทนี้ ยังมีหัวข้อสายอักขระขั้นสูงอีกหลายหัวข้อที่น่าสนใจ:

- **Aho-Corasick Algorithm** — ค้นหาหลาย Pattern พร้อมกันใน $\mathcal{O}\big(n + m_1 + m_2 + \cdots + m_k +$ จำนวนผลลัพธ์$\big)$ โดยสร้าง Trie พร้อมลิงก์ความล้มเหลว (Failure Links) คล้าย KMP

- **Suffix Automaton** — โครงสร้างข้อมูลที่รองรับการเพิ่มอักขระทีละตัว (Online) สามารถหา Distinct Substrings, Longest Common Substring และอื่นๆ ได้ใน $\mathcal{O}(n)$

- **Suffix Tree** — โครงสร้างต้นไม้ที่เข้ารหัสคำต่อท้ายทั้งหมดของสายอักขระ สามารถสร้างได้ใน $\mathcal{O}(n)$ ด้วย Ukkonen's Algorithm

- **Manacher's Algorithm** — หา Longest Palindromic Substring ใน $\mathcal{O}(n)$

- **String Hashing ขั้นสูง** — รวมถึง 2D Hashing สำหรับการค้นหารูปแบบ 2 มิติ

- **Burrows-Wheeler Transform** — ใช้ในการบีบอัดข้อมูล (เช่น bzip2) และการจัดเก็บอ้างอิง (FM-Index)

หัวข้อเหล่านี้มีประโยชน์มากสำหรับการแข่งขันระดับสูง และผู้อ่านที่สนใจสามารถศึกษาเพิ่มเติมได้จากแหล่งอ้างอิงต่อไปนี้:

- Competitive Programming 3 โดย Steven Halim และ Felix Halim[3]

- CP-Algorithms (cp-algorithms.com)[11] — เว็บไซต์อ้างอิงสำหรับขั้นตอนวิธีเชิงแข่งขัน

- USACO Guide — คู่มือฝึกฝนโดยเฉพาะสำหรับนักเรียนมัธยม

สรุปสูตรสำคัญ

Hash Prefix:

$$H(S[l..r]) = (h[r+1] - h[l] \cdot b^{r-l+1}) \bmod M$$

Distinct Substrings:

$$\text{จำนวน} = \frac{n(n+1)}{2} - \sum_{i=1}^{n-1} \text{LCP}[i]$$

Period ของสายอักขระ (KMP):

$$\text{Period} = n - \text{LPS}[n-1] \quad \text{(ถ้า } n \bmod (n - \text{LPS}[n-1]) = 0\text{)}$$

Rolling Hash:

$$H_{\text{new}} = ((H_{\text{old}} - S[i] \cdot b^{m-1}) \cdot b + S[i+m]) \bmod M$$

9

ชุดปัญหาด้านเรขาคณิต

ส่วนนี้ครอบคลุมเรขาคณิตเชิงคำนวณ (Computational Geometry) ตั้งแต่การแทนจุดและเวกเตอร์ เส้นตรงและส่วนของเส้นตรง ผลคูณเชิงเวกเตอร์ (Cross Product) และ CCW Test ไปจนถึงเทคนิคขั้นสูงในการแก้ปัญหาเรขาคณิตในการแข่งขัน

Chapter 17

เรขาคณิตเชิงคำนวณ (Computational Geometry)

เรขาคณิตเชิงคำนวณ (Computational Geometry) เป็นหัวข้อที่พบบ่อยในการแข่งขันโปรแกรมเชิงแข่งขัน โดยครอบคลุมปัญหาที่เกี่ยวข้องกับรูปทรงเรขาคณิต เช่น จุด เส้นตรง รูปสามเหลี่ยม วงกลม และรูปหลายเหลี่ยมบนระนาบสองมิติ การแก้โจทย์ประเภทนี้ต้องอาศัยความรู้ทางคณิตศาสตร์เชิงเรขาคณิตร่วมกับทักษะการแปลงสูตรและแนวคิดให้เป็นโค้ดที่ถูกต้องและมีประสิทธิภาพ อย่างไรก็ตาม โจทย์เรขาคณิตมักมีอัตราการผ่าน (accept rate) ที่ต่ำกว่าโจทย์ประเภทอื่น เช่น Complete Search หรือ Dynamic Programming ทำให้ผู้แข่งขันหลายคนเลือกที่จะใช้เวลากับหัวข้ออื่นแทน ในบทนี้ เราจะศึกษาตั้งแต่พื้นฐานของจุด เส้นตรง เวกเตอร์ ผลคูณเชิงเวกเตอร์ (CCW Test) ไปจนถึงวัตถุ 2 มิติ (วงกลม สามเหลี่ยม สี่เหลี่ยม รูปหลายเหลี่ยม) และขั้นตอนวิธีขั้นสูงบนรูปหลายเหลี่ยม เช่น Convex Hull ด้วย Graham's Scan[3, 11]

17.1 บทนำสู่เรขาคณิตเชิงคำนวณ

17.1.1 ความท้าทายของโจทย์เรขาคณิต

ปัญหาเรขาคณิตในการแข่งขันมีความท้าทายที่แตกต่างจากปัญหาขั้นตอนวิธีทั่วไป โดยมีลักษณะเฉพาะ 4 ประการ ได้แก่

1. **กรณีพิเศษ (Special Cases) ที่ซ่อนอยู่** — โจทย์เรขาคณิตมักมี test case ที่ล่อลวงผู้แข่งขัน เช่น เส้นตรงแนวดิ่ง (vertical line) จุดหลายจุดบนเส้นตรงเดียวกัน (collinear points) หรือรูปหลายเหลี่ยมเว้า (concave polygon) หากไม่ตรวจสอบกรณีพิเศษเหล่านี้ โปรแกรมจะให้ผลลัพธ์ผิดพลาดได้

2. **ปัญหาความแม่นยำของทศนิยม (Floating-Point Precision)** — การดำเนินการทางคณิตศาสตร์ เช่น การหาร การหารากที่สอง หรือการคำนวณฟังก์ชันตรีโกณมิติ อาจทำให้เกิดข้อผิดพลาดจากการปัดทศนิยมสะสม ส่งผลให้คำตอบผิดเพียงเล็กน้อยแต่ถูกตัดสินว่าผิด

3. **โค้ดที่ยาวและซับซ้อน** — โจทย์เรขาคณิตส่วนใหญ่ต้องใช้โค้ดที่ยาวและมีความเร็วในการทำงานต่ำกว่าโจทย์ประเภทอื่น การเขียนโค้ดภายใต้ความกดดันของเวลาทำให้เกิด bug ได้ง่าย

4. **การลืมสูตรพื้นฐาน** — ผู้แข่งขันมักลืมสูตรทางเรขาคณิตที่สำคัญ หรือไม่สามารถแปลงสูตรเพื่อให้ได้ค่าที่ต้องการ ดังนั้นในการแข่งขัน ACM ICPC ทีมส่วนใหญ่จะนำ hard copy ที่รวบรวมสูตรและ library function ทางเรขาคณิตเข้าไปในห้องสอบด้วย[13]

17.1.2 หลักการพื้นฐานในการเขียนโค้ดเรขาคณิต

เพื่อลดปัญหาที่กล่าวมา ผู้เขียนโค้ดเรขาคณิตควรยึดหลักการต่อไปนี้

หลีกเลี่ยง floating-point เมื่อเป็นไปได้ — หากโจทย์กำหนดพิกัดเป็นจำนวนเต็ม ให้ใช้การดำเนินการกับจำนวนเต็ม (integer addition, subtraction, multiplication) เพื่อหลีกเลี่ยงข้อผิดพลาดจากทศนิยม ตัวอย่างเช่น การตรวจสอบว่าจุดอยู่ภายในวงกลมควรเปรียบเทียบ `dx*dx + dy*dy` กับ `r*r` แทนการใช้ `sqrt(dx*dx + dy*dy) < r`

ใช้ EPS สำหรับการเปรียบเทียบทศนิยม — เมื่อจำเป็นต้องใช้ floating-point การเปรียบเทียบความเท่ากันด้วย == ไม่สามารถทำได้เนื่องจากข้อผิดพลาดในการปัดทศนิยม ให้กำหนดค่าคงที่ `EPS = 1e-9` และใช้สูตรต่อไปนี้แทน

```cpp
#define EPS 1e-9
// Check if two floating-point numbers are equal
bool isEqual(double a, double b) {
    return fabs(a - b) < EPS;
}
// Check if x >= 0.0
bool isNonNegative(double x) {
    return x > -EPS;
}
// Check if x <= 0.0
bool isNonPositive(double x) {
    return x < EPS;
}
```

ตัวอย่างโค้ดที่ 17.1: การเปรียบเทียบ floating-point ด้วย EPS

เตรียม library function ล่วงหน้า — ก่อนการแข่งขัน ผู้แข่งควรเตรียมโค้ดสำหรับฟังก์ชันทางเรขาคณิตที่ใช้บ่อย เช่น struct `point`, struct `line`, การคำนวณระยะทาง การตรวจสอบ ccw และ Convex Hull เพื่อลดเวลาในการเขียนและลดโอกาสเกิด bug ในห้องสอบ

ข้อควรระวัง

บทนี้จะเน้นกรณีพิเศษที่อาจเกิดขึ้น และเลือกการพัฒนาที่ลดจำนวนกรณีพิเศษเหล่านั้น อย่างไรก็ตาม ผู้อ่านควรฝึกฝนกับโจทย์จริงเพื่อสร้างความคุ้นเคยกับรูปแบบปัญหาที่หลากหลาย

17.1.3 ขอบเขตของบทนี้

บทนี้ครอบคลุมเนื้อหา 2 ส่วนหลัก ดังนี้

1. **คำศัพท์และสูตรพื้นฐานของวัตถุในมิติต่าง ๆ** — ครอบคลุมวัตถุ 0 มิติ (Point) 1 มิติ (Line, Line Segment, Vector) และ 2 มิติ (Circle, Triangle, Quadrilateral, Polygon) พร้อมสูตรพื้นฐานที่ใช้เป็นข้อมูลอ้างอิงเมื่อพบโจทย์เรขาคณิต

2. **ขั้นตอนวิธีบนรูปหลายเหลี่ยม 2 มิติ** — วางรากฐานสำหรับขั้นตอนวิธีที่ใช้บ่อยในการแข่งขัน เช่น การตรวจสอบว่า polygon เป็น convex หรือ concave การตัดสินว่าจุดอยู่ภายในหรือภายนอก polygon การตัด polygon ด้วยเส้นตรง และการหา Convex Hull จากเซตของจุด โดยในบทนี้จะเน้นที่พื้นฐาน CCW Test ซึ่งเป็นรากฐานสำคัญของขั้นตอนวิธีเหล่านี้ รวมถึงนำเสนอ Graham's Scan Algorithm สำหรับหา Convex Hull ในเวลา $\mathcal{O}(n \log n)$

17.1.4 การเตรียม environment

ตัวอย่างโค้ดในบทนี้ใช้ภาษา C++ และสมมติว่ามีการ include ไฟล์ header และ define ค่าคงที่พื้นฐานดังต่อไปนี้

```cpp
#include <bits/stdc++.h>
using namespace std;
#define INF 1e9
#define EPS 1e-9
#define PI acos(-1.0)
double degToRad(double d) { return d * PI / 180.0; }
double radToDeg(double r) { return r * 180.0 / PI; }
```

ตัวอย่างโค้ดที่ 17.2: Header และค่าคงที่พื้นฐานสำหรับบทเรขาคณิต

ฟังก์ชัน `degToRad` และ `radToDeg` ใช้สำหรับแปลงหน่วยมุมระหว่างองศา (degrees) และเรเดียน (radians) ซึ่งจำเป็นเนื่องจากฟังก์ชันตรีโกณมิติใน C++ (`sin`, `cos`, `tan`, `acos` ฯลฯ) รับค่าในหน่วยเรเดียน ในขณะที่โจทย์ส่วนใหญ่กำหนดมุมเป็นองศา

17.2 จุด (Point) และเวกเตอร์ (Vector)

จุด (Point) เป็นวัตถุพื้นฐาน 0 มิติที่ใช้สร้างวัตถุในมิติที่สูงขึ้น ในระนาบยูคลิดสองมิติ (2D Euclidean space) จุดหนึ่งจุดถูกกำหนดโดยพิกัดคู่อันดับ (x, y) โดยมีจุดกำเนิด (origin) อยู่ที่ $(0, 0)$ ส่วนเวกเตอร์ (Vector) เป็นวัตถุที่มีทิศทางและขนาด ใช้สำหรับอธิบายการเคลื่อนที่หรือการแปลงตำแหน่งของจุด เวกเตอร์ในระนาบสองมิติประกอบด้วยสมาชิกสองตัวคือองค์ประกอบ x และ y ในบทนี้ เราจะใช้ struct ใน C++ สำหรับเก็บจุดและเวกเตอร์ พร้อมนิยามการดำเนินการพื้นฐานที่จำเป็นต่อการแก้โจทย์เรขาคณิต[3]

17.2.1 นิยาม struct point

เนื่องจากโจทย์เรขาคณิตอาจกำหนดพิกัดเป็นจำนวนเต็มหรือจำนวนจริง ควรเตรียม struct ทั้งสองแบบไว้ใช้งาน โดย `pointI` สำหรับจำนวนเต็ม (integer) และ `point` สำหรับจำนวนจริง (floating-point) ที่ต้องการความแม่นยำ ทั้งคู่มี default constructor ที่กำหนดค่าเริ่มต้นเป็น $(0, 0)$ และ user-defined constructor สำหรับกำหนดค่าเริ่มต้นตามต้องการ

```cpp
struct pointI {
    int x, y;
```

```
    pointI() { x = y = 0; }
    pointI(int _x, int _y) : x(_x), y(_y) {}
};
struct point {
    double x, y;
    point() { x = y = 0.0; }
    point(double _x, double _y) : x(_x), y(_y) {}
};
```

ตัวอย่างโค้ดที่ 17.3: struct point สำหรับจำนวนเต็มและจำนวนจริง

คำแนะนำ

หากโจทย์กำหนดพิกัดเป็นจำนวนเต็มและไม่จำเป็นต้องคำนวณทศนิยม ให้ใช้ `pointI` เพื่อหลีกเลี่ยงปัญหา floating-point precision แต่ในบทนี้จะใช้ `point` (floating-point) เป็นค่าเริ่มต้นเพื่อรองรับการดำเนินการทุกประเภท

17.2.2 การเปรียบเทียบและการตรวจสอบความเท่ากัน

การเรียงจุด (Sorting Points). ในบางครั้งเราจำเป็นต้องเรียงจุดใน `vector<point>` เพื่อเตรียมข้อมูลให้ขั้นตอนวิธี เช่น Convex Hull สามารถทำได้โดย overload less-than operator ภายใน struct `point` แล้วใช้ `sort()` จาก STL การเปรียบเทียบจะเรียงตามพิกัด x ก่อน และหาก x เท่ากันจะเรียงตามพิกัด y โดยใช้ EPS ในการตรวจสอบความเท่ากันของค่า floating-point

```
bool operator < (point other) const {
    if (fabs(x - other.x) > EPS)
        return x < other.x;
    return y < other.y;
}
```

ตัวอย่างโค้ดที่ 17.4: Overload less-than operator สำหรับเรียงจุด

การตรวจสอบจุดเท่ากัน (Equality Test). การทดสอบว่าจุดสองจุดเป็นจุดเดียวกันใช้ overload equality operator โดยตรวจสอบว่าผลต่างของพิกัด x และ y มีค่าน้อยกว่า EPS ทั้งคู่

```
bool operator == (point other) const {
    return (fabs(x - other.x) < EPS && fabs(y - other.y) < EPS);
}
```

ตัวอย่างโค้ดที่ 17.5: Overload equality operator สำหรับตรวจสอบจุดเท่ากัน

17.2.3 ระยะทางยูคลิเดียน (Euclidean Distance)

ระยะทางระหว่างจุดสองจุด p1 และ p2 คำนวณได้จากสูตรระยะทางยูคลิเดียน $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ ใน C++ สามารถใช้ฟังก์ชัน `hypot(dx, dy)` จาก `<cmath>` ซึ่งคืนค่า $\sqrt{dx^2 + dy^2}$ และมีการจัดการ

overflow ที่ดีกว่าการคำนวณด้วยตนเอง

```
double dist(point p1, point p2) {
    return hypot(p1.x - p2.x, p1.y - p2.y);
}
```

ตัวอย่างโค้ดที่ 17.6: ฟังก์ชัน dist คำนวณระยะทางยูคลิเดียน

ตัวอย่าง: ระยะทางระหว่างจุด

ระยะ ทาง ระหว่าง `P[0] = (2, 2)` และ `P[5] = (6, 5)` เท่ากับ `hypot(6-2, 5-2) = hypot(4, 3) = 5.000`

17.2.4 การหมุนจุด (Rotation)

การหมุนจุด p รอบจุดกำเนิดไปตามเข็มนาฬิกาหรือทวนเข็มนาฬิกาใช้ rotation matrix ในบทนี้จะใช้ทิศทางทวนเข็มนาฬิกา (Counter-Clockwise, CCW) เป็นค่าเริ่มต้น โดยรับมุมเป็นองศา แล้วแปลงเป็นเรเดียนก่อนคำนวณ สูตรหมุนจุด (x, y) ไป θ เรเดียนทวนเข็มนาฬิกาคือ

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta$$

```
point rotate(point p, double theta) {
    double rad = degToRad(theta);
    return point(p.x * cos(rad) - p.y * sin(rad),
                 p.x * sin(rad) + p.y * cos(rad));
}
```

ตัวอย่างโค้ดที่ 17.7: ฟังก์ชัน rotate หมุนจุดรอบจุดกำเนิด

ข้อควรระวัง

ฟังก์ชัน `rotate` นี้หมุนรอบจุดกำเนิด $(0, 0)$ เท่านั้น หากต้องการหมุนรอบจุดอื่น เช่นจุด (a, b) ต้องแปลงระบบพิกัดโดยย้ายจุด (a, b) ไปที่จุดกำเนิดก่อน (translate) แล้วหมุน จากนั้นย้ายกลับ

17.2.5 การดำเนินการของเวกเตอร์ (Vector Operations)

เวกเตอร์ (Vector) ในทางเรขาคณิตเชิงคำนวณคือส่วนของเส้นตรงที่มีทิศทาง โดยทั่วไปจะแทนด้วย struct ที่มีสมาชิก x และ y คล้ายกับจุด แต่ใช้ชื่อ `vec` เพื่อหลีกเลี่ยงความสับสนกับ `std::vector` ใน STL การดำเนินการพื้นฐานของเวกเตอร์ที่ใช้บ่อยมี 3 อย่าง ได้แก่ การแปลงจุด 2 จุดเป็นเวกเตอร์ การปรับขนาดเวกเตอร์ และการย้ายจุดตามเวกเตอร์

```
struct vec {
    double x, y;
```

```
    vec(double _x, double _y) : x(_x), y(_y) {}
};
vec toVec(point a, point b) {
    return vec(b.x - a.x, b.y - a.y);
}
vec scale(vec v, double s) {
    return vec(v.x * s, v.y * s);
}
point translate(point p, vec v) {
    return point(p.x + v.x, p.y + v.y);
}
```

ตัวอย่างโค้ดที่ 17.8: struct vec และฟังก์ชันพื้นฐานของเวกเตอร์

- `toVec(a, b)` แปลงจุด a และ b เป็นเวกเตอร์ที่ชี้จาก a ไป b คือ $\vec{ab} = (b_x - a_x, b_y - a_y)$
- `scale(v, s)` ปรับขนาดเวกเตอร์ v ด้วยสเกลาร์ s โดยที่ $s < 1$ ทำให้สั้นลง $s = 1$ คงขนาดเดิม และ $s > 1$ ทำให้ยาวขึ้น
- `translate(p, v)` ย้ายจุด p ไปตามทิศทางและขนาดของเวกเตอร์ v คือ $p' = (p_x + v_x, p_y + v_y)$

17.2.6 โค้ดรวมตัวอย่างการใช้งาน

ตัวอย่างต่อไปนี้สาธิตการใช้ struct `point` และ `vec` ร่วมกับฟังก์ชันที่กำหนดข้างต้น โดยสร้างจุดหลายจุดใน `vector<point>` เรียงลำดับด้วย `sort` และคำนวณระยะทางระหว่างจุดคู่หนึ่ง

```
int main() {
    point P1, P2, P3(0, 1);
    printf("%d\n", P1 == P2);   // 1 (true)
    printf("%d\n", P1 == P3);   // 0 (false)
    vector<point> P;
    P.push_back(point(2, 2));
    P.push_back(point(4, 3));
    P.push_back(point(2, 4));
    P.push_back(point(6, 6));
    P.push_back(point(2, 6));
    P.push_back(point(6, 5));
    sort(P.begin(), P.end());   // sort by (x, y)
    double d = dist(P[0], P[5]);
    printf("Euclidean distance: %.3f\n", d);   // 5.657
    return 0;
}
```

ตัวอย่างโค้ดที่ 17.9: ตัวอย่างการใช้งาน struct point และ vec ใน main

17.3 เส้นตรง (Line) และส่วนของเส้นตรง (Line Segment)

เส้นตรง (Line) ในระนาบยูคลิดสองมิติคือเซตของจุดที่มีพิกัดสอดคล้องกับสมการเส้นตรง $ax + by + c = 0$ โดยที่ a, b, c เป็นสัมประสิทธิ์ของสมการ ส่วนของเส้นตรง (Line Segment) คือเส้นตรงที่มีจุดปลายสองจุดและมีความยาวจำกัด ในการแก้โจทย์เรขาคณิต เรามักต้องแปลงจุด 2 จุดเป็นสมการเส้นตรง ตรวจสอบความขนานหรือความตัดกันของเส้นตรง และคำนวณระยะทางจากจุดถึงเส้นตรงหรือส่วนของเส้นตรง[3]

17.3.1 นิยามและสมการเส้นตรง

เพื่อความสะดวกในการพัฒนา เราจะลดความซับซ้อนของสมการเส้นตรงโดยกำหนดให้สัมประสิทธิ์ $b = 1$ สำหรับเส้นตรงที่ไม่ใช่แนวดิ่ง (non-vertical line) และ $b = 0$ สำหรับเส้นตรงแนวดิ่ง (vertical line) การกำหนดเช่นนี้ทำให้เรามีรูปแบบสมการเพียง 2 กรณี คือ $ax + y + c = 0$ สำหรับเส้นทั่วไป และ $x + c = 0$ สำหรับเส้นแนวดิ่ง ซึ่งช่วยลดจำนวนกรณีพิเศษในการพัฒนา

```
struct line {
    double a, b, c;
};
```

ตัวอย่างโค้ดที่ 17.10: struct line สำหรับเก็บสมการเส้นตรง

ข้อควรระวัง

การกำหนด $b = 1$ หรือ $b = 0$ เป็นข้อตกลง (convention) ในบทนี้ การใช้รูปแบบสมการอื่น เช่น $y = mx + c$ (slope-intercept form) ไม่สามารถแทนเส้นตรงแนวดิ่งได้เนื่องจากความชัน (slope) เป็นอนันต์ จึงไม่เหมาะกับการเขียนโค้ดเรขาคณิตทั่วไป

17.3.2 การแปลงจุดเป็นเส้นตรง (pointsToLine)

เมื่อกำหนดจุด 2 จุด (x_1, y_1) และ (x_2, y_2) ที่อยู่บนเส้นตรงเดียวกัน สามารถคำนวณสัมประสิทธิ์ a, b, c ได้จากความสัมพันธ์ $ax_1 + y_1 + c = 0$ และ $ax_2 + y_2 + c = 0$ ซึ่งให้ $a = -(y_1 - y_2)/(x_1 - x_2)$ และ $c = -ax_1 - y_1$ อย่างไรก็ตาม ต้องตรวจสอบกรณีพิเศษคือเส้นตรงแนวดิ่ง ซึ่ง $x_1 = x_2$ ทำให้ตัวหารเป็นศูนย์

```
void pointsToLine(point p1, point p2, line &l) {
    if (fabs(p1.x - p2.x) < EPS) {   // vertical line
        l.a = 1.0; l.b = 0.0; l.c = -p1.x;
    } else {
        l.a = -(double)(p1.y - p2.y) / (p1.x - p2.x);
        l.b = 1.0;   // IMPORTANT: fix b = 1.0
        l.c = -(double)(l.a * p1.x) - p1.y;
    }
}
```

ตัวอย่างโค้ดที่ 17.11: ฟังก์ชัน pointsToLine แปลงจุด 2 จุดเป็นสมการเส้นตรง

17.3.3 การตรวจสอบเส้นตรงขนานและเส้นตรงเดียวกัน

เส้นตรง 2 เส้นขนานกัน (parallel) ถ้าค่าสัมประสิทธิ์ a และ b ของทั้งสองเส้นเท่ากัน เส้นตรง 2 เส้นเป็นเส้นเดียวกัน (same) ถ้าเป็นเส้นขนานและค่า c เท่ากันด้วย การตรวจสอบใช้ EPS สำหรับเปรียบเทียบค่า floating-point

```
bool areParallel(line l1, line l2) {
    return (fabs(l1.a - l2.a) < EPS) && (fabs(l1.b - l2.b) < EPS);
}
bool areSame(line l1, line l2) {
    return areParallel(l1, l2) && (fabs(l1.c - l2.c) < EPS);
}
```

ตัวอย่างโค้ดที่ 17.12: ฟังก์ชัน areParallel และ areSame

17.3.4 การหาจุดตัดของเส้นตรงสองเส้น (areIntersect)

ถ้าเส้นตรง 2 เส้นไม่ขนานกัน (และไม่ใช่เส้นเดียวกัน) จะตัดกันที่จุดจุดหนึ่ง จุดตัด (x, y) หาได้จากการแก้ระบบสมการเชิงเส้น 2 ตัวแปร

$$\begin{cases} a_1x + b_1y + c_1 = 0 \\ a_2x + b_2y + c_2 = 0 \end{cases} \implies x = \frac{b_2c_1 - b_1c_2}{a_2b_1 - a_1b_2}, \quad y = -\frac{a_1x + c_1}{b_1}$$

```
bool areIntersect(line l1, line l2, point &p) {
    if (areParallel(l1, l2)) return false;  // no intersection
    p.x = (l2.b * l1.c - l1.b * l2.c) / (l2.a * l1.b - l1.a * l2.b);
    if (fabs(l1.b) > EPS)
        p.y = -(l1.a * p.x + l1.c);
    else
        p.y = -(l2.a * p.x + l2.c);
    return true;
}
```

ตัวอย่างโค้ดที่ 17.13: ฟังก์ชัน areIntersect หาจุดตัดของเส้นตรง 2 เส้น

ข้อควรระวัง

ต้องตรวจสอบ `fabs(l1.b) > EPS` ก่อนคำนวณ y เพื่อหลีกเลี่ยงการหารด้วยศูนย์ในกรณีที่ `l1` เป็นเส้นแนวดิ่ง ($b = 0$) ในกรณีนั้นให้ใช้ `l2` ในการคำนวณแทน

17.3.5 ระยะทางจากจุดถึงเส้นตรง (distToLine)

เมื่อกำหนดจุด p และเส้นตรงที่อธิบายด้วยจุด 2 จุด a และ b ระยะทางสั้นที่สุดจาก p ไปยังเส้นตรงหาได้จากการคำนวณตำแหน่งของจุด c บนเส้นตรงที่ใกล้กับ p มากที่สุด จากนั้นหาระยะทางระหว่าง p และ c จุด c สามารถมองเป็นจุด a ที่ถูกแปลงโดยการย่อ/ขยายขนาดของเวกเตอร์ $\vec{ab}$ ด้วยสเกลาร์ u นั่นคือ $c = a + u \cdot \vec{ab}$ โดยค่า u คำนวณจาก scalar projection ของ $\vec{ap}$ ลงบน $\vec{ab}$ โดยใช้ dot product

$$u = \frac{\vec{ap} \cdot \vec{ab}}{|\vec{ab}|^2}$$

```
double dot(vec a, vec b) { return a.x * b.x + a.y * b.y; }
double normSq(vec v) { return v.x * v.x + v.y * v.y; }
double distToLine(point p, point a, point b, point &c) {
    vec ap = toVec(a, p), ab = toVec(a, b);
    double u = dot(ap, ab) / normSq(ab);
    c = translate(a, scale(ab, u));
    return dist(p, c);
}
```

ตัวอย่างโค้ดที่ 17.14: ฟังก์ชัน dot, normSq และ distToLine

17.3.6 ระยะทางจากจุดถึงส่วนของเส้นตรง (distToLineSegment)

ระยะทางสั้นที่สุดจากจุด p ไปยังส่วนของเส้นตรง ab ต้องพิจารณา 2 กรณีพิเศษเพิ่มเติมจาก `distToLine` คือ ถ้าจุด c ที่คำนวณได้มีพารามิเตอร์ $u < 0$ แสดงว่าจุดที่ใกล้ที่สุดคือจุดปลาย a และถ้า $u > 1$ จุดที่ใกล้ที่สุดคือจุดปลาย b ในกรณีอื่นให้ใช้ `distToLine` ตามปกติ

```
double distToLineSegment(point p, point a, point b, point &c) {
    vec ap = toVec(a, p), ab = toVec(a, b);
    double u = dot(ap, ab) / normSq(ab);
    if (u < 0.0) {
        c = point(a.x, a.y);
        return dist(p, a);
    }
    if (u > 1.0) {
        c = point(b.x, b.y);
        return dist(p, b);
    }
    return distToLine(p, a, b, c);
}
```

ตัวอย่างโค้ดที่ 17.15: ฟังก์ชัน distToLineSegment คำนวณระยะทางถึงส่วนของเส้นตรง

ตัวอย่าง: ระยะทางจากจุดถึงส่วนของเส้นตรง

กำหนดจุด `a(2, 2)`, `b(4, 3)`, `p(3, 2)` ค่า $u = \frac{(1,0)\cdot(2,1)}{5} = \frac{2}{5} = 0.4$ ซึ่งอยู่ในช่วง $[0, 1]$ ดังนั้นจุดที่ใกล้ที่สุด $c = a + 0.4 \cdot \vec{ab} = (2.8, 2.4)$ และระยะทาง `dist(p, c) ≈ 0.447`

17.3.7 สรุปฟังก์ชันเส้นตรง

ตารางต่อไปนี้สรุปฟังก์ชันที่เกี่ยวกับเส้นตรงและส่วนของเส้นตรงที่กำหนดข้างต้น

ตารางที่ 17.1 สรุปฟังก์ชันเส้นตรงและส่วนของเส้นตรง

ฟังก์ชัน	คืนค่า	คำอธิบาย
`pointsToLine(p1, p2, l)`	void	แปลงจุด 2 จุดเป็นสมการเส้นตรง (เก็บใน `l`)
`areParallel(l1, l2)`	bool	ตรวจสอบว่าเส้นตรง 2 เส้นขนานกัน
`areSame(l1, l2)`	bool	ตรวจสอบว่าเส้นตรง 2 เส้นเป็นเส้นเดียวกัน
`areIntersect(l1, l2, p)`	bool	หาจุดตัดของเส้นตรง 2 เส้น (เก็บจุดตัดใน `p`)
`distToLine(p, a, b, c)`	double	ระยะทางจากจุดถึงเส้นตรงที่ลากผ่าน `a`, `b` (เก็บจุดใกล้สุดใน `c`)
`distToLineSegment(p, a, b, c)`	double	ระยะทางจากจุดถึงส่วนของเส้นตรง `ab` (เก็บจุดใกล้สุดใน `c`)

17.4 ผลคูณเชิงเวกเตอร์ (Cross Product) และ CCW Test

ผลคูณเชิงเวกเตอร์ (Cross Product) เป็นการดำเนินการที่มีประโยชน์อย่างยิ่งในเรขาคณิตเชิงคำนวณ 2 มิติ โดยใช้สำหรับตัดสินว่าจุด 3 จุดหมุนซ้าย หมุนขวา หรืออยู่บนเส้นตรงเดียวกัน การทดสอบนี้เรียกว่า CCW Test (Counter-Clockwise Test) และเป็นพื้นฐานของขั้นตอนวิธีหลายตัว เช่น การตรวจสอบว่า polygon เป็น convex การตัด polygon ด้วยเส้นตรง และ Convex Hull ส่วนผลคูณจุด (Dot Product) ใช้สำหรับคำนวณมุมระหว่างเวกเตอร์และการ project เวกเตอร์ ซึ่งได้นำเสนอไปแล้วในส่วนของ `distToLine`[3, 2]

17.4.1 ผลคูณจุด (Dot Product) และมุมระหว่างเวกเตอร์

กำหนดเวกเตอร์ $\vec{a}$ และ $\vec{b}$ ผลคูณจุดนิยามโดย $\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$ ซึ่งเท่ากับ $|\vec{a}||\vec{b}| \cos\theta$ เมื่อ θ คือมุมระหว่างเวกเตอร์ทั้งสอง ดังนั้นสามารถคำนวณมุม θ ระหว่างเวกเตอร์ $\vec{oa}$ และ $\vec{ob}$ ได้จาก

$$\theta = \arccos\left(\frac{\vec{oa} \cdot \vec{ob}}{|\vec{oa}||\vec{ob}|}\right)$$

```
double angle(point a, point o, point b) {
    vec oa = toVec(o, a), ob = toVec(o, b);
```

```
    return acos(dot(oa, ob) / sqrt(normSq(oa) * normSq(ob)));
}
```

ตัวอย่างโค้ดที่ 17.16: ฟังก์ชัน angle คำนวณมุม aob ในหน่วยเรเดียน

หน่วยของมุม

ฟังก์ชัน `angle` คืนค่ามุมในหน่วยเรเดียน หากต้องการแสดงผลเป็นองศา ให้คูณด้วย `180.0 / PI` หรือเรียกฟังก์ชัน `radToDeg`

17.4.2 ผลคูณเชิงเวกเตอร์ (Cross Product)

กำหนดเวกเตอร์ $\vec{a}$ และ $\vec{b}$ ในระนาบ 2 มิติ ผลคูณเชิงเวกเตอร์นิยามโดย $\vec{a} \times \vec{b} = a_x b_y - a_y b_x$ ซึ่งให้ค่าเป็นจำนวนจริง (ไม่ใช่เวกเตอร์ใน 2D) ค่านี้มีนัยสำคัญทางเรขาคณิต 3 ประการ ได้แก่

- **เครื่องหมาย** บอกทิศทางการหมุน: ค่าบวกหมายถึง $\vec{b}$ อยู่ทางซ้ายของ $\vec{a}$ (หมุนซ้าย/CCW) ค่าลบหมายถึง $\vec{b}$ อยู่ทางขวา (หมุนขวา/CW) และค่าศูนย์หมายถึง $\vec{a}$ และ $\vec{b}$ อยู่บนเส้นตรงเดียวกัน (collinear)
- **ขนาด** เท่ากับพื้นที่ของสี่เหลี่ยมด้านขนานที่เกิดจาก $\vec{a}$ และ $\vec{b}$ นั่นคือ $|\vec{a} \times \vec{b}| = |\vec{a}||\vec{b}||\sin\theta|$
- **พื้นที่สามเหลี่ยม** ที่เกิดจากจุด 3 จุด p, q, r เท่ากับ $\frac{1}{2}|\vec{pq} \times \vec{pr}|$

```
double cross(vec a, vec b) {
    return a.x * b.y - a.y * b.x;
}
```

ตัวอย่างโค้ดที่ 17.17: ฟังก์ชัน cross คำนวณผลคูณเชิงเวกเตอร์

17.4.3 CCW Test (Counter-Clockwise Test)

เมื่อกำหนดเส้นตรงที่นิยามด้วยจุด 2 จุด `p` และ `q` และต้องการตัดสินว่าจุด `r` อยู่ทางซ้าย ทางขวา หรืออยู่บนเส้นตรงเดียวกัน สามารถทำได้โดยคำนวณ cross product ของ $\vec{pq}$ และ $\vec{pr}$ ถ้าค่าที่ได้เป็นบวก/ศูนย์/ลบ แสดงว่า $p \to q \to r$ หมุนซ้าย/อยู่บนเส้นเดียวกัน/หมุนขวา ตามลำดับ การทดสอบหมุนซ้ายมักเรียกว่า CCW Test (Counter-Clockwise Test)

```
// returns true if point r is on the left side of line pq
bool ccw(point p, point q, point r) {
    return cross(toVec(p, q), toVec(p, r)) > 0;
}
// returns true if point r is on the same line as the line pq
bool collinear(point p, point q, point r) {
    return fabs(cross(toVec(p, q), toVec(p, r))) < EPS;
}
```

ตัวอย่างโค้ดที่ 17.18: ฟังก์ชัน ccw และ collinear

การรับจุด collinear

โดยค่าเริ่มต้น `ccw` ใช้ `> 0` ทำให้จุดที่อยู่บนเส้นตรงเดียวกัน (collinear) ถือว่าไม่ใช่การหมุนซ้าย หากต้องการรับจุด collinear ให้เปลี่ยนเป็น `>= 0` แทน การเลือกนี้มีผลต่อขั้นตอนวิธีบางตัว เช่น Convex Hull ที่อาจต้องการเก็บหรือตัดจุด collinear ออกจากผลลัพธ์

17.4.4 ตัวอย่างการใช้งาน CCW

ตัวอย่าง: ตรวจสอบทิศทางการหมุน

กำหนดจุด `p(3, 7)`, `q(11, 13)`, `r(35, 30)` เราต้องการตรวจสอบว่า `r` อยู่ทางซ้ายหรือทางขวาของเส้นตรง `pq`

- $\vec{pq} = (11-3, 13-7) = (8, 6)$
- $\vec{pr} = (35-3, 30-7) = (32, 23)$
- $\vec{pq} \times \vec{pr} = 8 \times 23 - 6 \times 32 = 184 - 192 = -8$

เนื่องจากผลคูณเชิงเวกเตอร์เป็นลบ แสดงว่า `r` อยู่ทางขวาของเส้นตรง `pq` และ `ccw(p, q, r)` คืนค่า `false`

17.4.5 ความสัมพันธ์ระหว่าง dot, cross และฟังก์ชันอื่น

ตารางต่อไปนี้สรุปความสัมพันธ์ระหว่างการดำเนินการของเวกเตอร์และฟังก์ชันที่ใช้ในบทนี้

ตารางที่ 17.2 สรุปการดำเนินการของเวกเตอร์และฟังก์ชันที่เกี่ยวข้อง

ฟังก์ชัน	สูตร	การใช้งาน
`dot(a, b)`	$a_x b_x + a_y b_y$	คำนวณมุมระหว่างเวกเตอร์, scalar projection
`cross(a, b)`	$a_x b_y - a_y b_x$	ตรวจสอบทิศทางการหมุน (CCW), พื้นที่สามเหลี่ยม
`normSq(v)`	$v_x^2 + v_y^2$	คำนวณ $\lvert\vec{v}\rvert^2$ โดยไม่ใช้ `sqrt`
`angle(a, o, b)`	$\arccos\left(\frac{\vec{oa}\cdot\vec{ob}}{\lvert\vec{oa}\rvert\lvert\vec{ob}\rvert}\right)$	มุม $\angle aob$ ในหน่วยเรเดียน
`ccw(p, q, r)`	$\vec{pq} \times \vec{pr} > 0$	ตรวจสอบว่า `r` อยู่ทางซ้ายของเส้น `pq`
`collinear(p, q, r)`	$\lvert\vec{pq} \times \vec{pr}\rvert < \text{EPS}$	ตรวจสอบว่า `p, q, r` อยู่บนเส้นเดียวกัน

เคล็ดลับ

การดำเนินการ dot และ cross ใช้เฉพาะการคูณและการบวกของ floating-point โดยไม่มี sqrt หรือ acos จึงมีความเสถียรทางตัวเลข (numerical stability) ที่ดีกว่าฟังก์ชันที่ใช้ sqrt ควรใช้ cross สำหรับการตัดสินทิศทางแทนการคำนวณมุมโดยตรงเมื่อเป็นไปได้

17.5 วัตถุ 2 มิติ: วงกลม (Circle)

วงกลม (Circle) เป็นวัตถุ 2 มิติที่พบบ่อยในโจทย์เรขาคณิตเชิงคำนวณ โดยเฉพาะปัญหาที่เกี่ยวข้องกับการตรวจสอบการตัดกันของวงกลม การหาวงกลมที่ล้อมรอบหรือจารึกในรูปหลายเหลี่ยม และการคำนวณพื้นที่ส่วนต่างๆ ของวงกลม ในส่วนนี้ เราจะทบทวนนิยามและสูตรพื้นฐานของวงกลม พร้อมตัวอย่างโค้ดที่ใช้ในการแข่งขัน[3]

17.5.1 นิยามและการตรวจสอบจุดภายในวงกลม

วงกลมที่มีจุดศูนย์กลางที่คู่อันดับ (a, b) ในระนาบยูคลิด 2 มิติ ด้วยรัศมี r คือเซตของทุกจุด (x, y) ที่สอดคล้องกับสมการ

$$(x - a)^2 + (y - b)^2 = r^2$$

จุดที่อยู่ภายในวงกลมจะมีผลรวมกำลังสองน้อยกว่า r^2 จุดที่อยู่บนเส้นรอบวงจะมีค่าเท่ากับ r^2 และจุดที่อยู่ภายนอกจะมีค่ามากกว่า r^2 การตรวจสอบนี้สามารถทำได้ด้วยจำนวนเต็มทั้งหมดเพื่อหลีกเลี่ยงปัญหาทศนิยม โดยเปรียบเทียบ dx*dx + dy*dy กับ r*r โดยตรง

```
int insideCircle(pointI p, pointI c, int r) { // all integer version
    int dx = p.x - c.x, dy = p.y - c.y;
    int Euc = dx * dx + dy * dy, rSq = r * r; // all integer
    return Euc < rSq ? 0 : Euc == rSq ? 1 : 2;
} // 0 = inside, 1 = border, 2 = outside
```

ตัวอย่างโค้ดที่ 17.19: ฟังก์ชัน insideCircle ตรวจสอบจุดภายใน/บน/ภายนอกวงกลม

17.5.2 ค่าคงที่และสูตรพื้นฐาน

ค่าคงที่ π (Pi) เป็นอัตราส่วนของเส้นรอบวงกับเส้นผ่านศูนย์กลางของวงกลม เพื่อป้องกัน error จากการปัดทศนิยม ค่าที่ปลอดภัยสำหรับการแข่งขันคือ PI = acos(-1.0) หรือ PI = 2 * acos(0.0) วงกลมที่มีรัศมี r จะมีคุณสมบัติดังตารางต่อไปนี้

ตารางที่ 17.3 สูตรพื้นฐานของวงกลม

คุณสมบัติ	สูตร	หมายเหตุ
เส้นผ่านศูนย์กลาง	$d = 2r$	diameter
เส้นรอบวง	$c = 2\pi r$	circumference/perimeter
พื้นที่	$A = \pi r^2$	area
ความยาว Arc	arc $= \frac{\alpha}{360} \times c$	α = มุมที่จุดศูนย์กลาง (องศา)
ความยาว Chord	$2r \sin(\alpha/2)$	เทียบเท่า $\sqrt{2r^2(1-\cos\alpha)}$
พื้นที่ Sector	$\frac{\alpha}{360} \times A$	บริเวณปิดด้วยรัศมี 2 เส้นและ arc
พื้นที่ Segment	พื้นที่ Sector $-$ พื้นที่สามเหลี่ยม	บริเวณปิดด้วย chord และ arc

Arc (ส่วนโค้ง) คือส่วนที่เชื่อมต่อกันของเส้นรอบวง c เมื่อกำหนดมุมที่จุดศูนย์กลาง (central angle) α องศา สามารถคำนวณความยาว arc ได้จากสัดส่วน $\alpha/360$ ของเส้นรอบวงทั้งหมด ในทำนองเดียวกัน Sector (เซกเตอร์) คือบริเวณของวงกลมที่ถูกปิดด้วยรัศมี 2 เส้นและ arc จึงมีพื้นที่เป็นสัดส่วน $\alpha/360$ ของพื้นที่วงกลมทั้งหมด ส่วน Segment (เซกเมนต์) คือบริเวณที่ถูกปิดด้วย chord และ arc หาได้จากการลบพื้นที่สามเหลี่ยมหน้าจั่ว (ที่มีด้าน r, r และความยาว chord) ออกจากพื้นที่ sector ที่สอดคล้องกัน

17.5.3 การหาจุดศูนย์กลางวงกลมจาก 2 จุดและรัศมี

เมื่อกำหนดจุด 2 จุดบนวงกลม p1 และ p2 และรัศมี r ของวงกลมที่สอดคล้อง เราสามารถหาตำแหน่งของจุดศูนย์กลางได้ 2 ตำแหน่งที่เป็นไปได้ (เนื่องจากวงกลมสามารถอยู่ทั้งสองด้านของเส้นตรง p1p2) ฟังก์ชันต่อไปนี้คืนค่า `false` หากรัศมีเล็กเกินไปที่จะเชื่อมจุดทั้งสอง

```
bool circle2PtsRad(point p1, point p2, double r, point &c) {
    double d2 = (p1.x - p2.x) * (p1.x - p2.x) +
                (p1.y - p2.y) * (p1.y - p2.y);
    double det = r * r / d2 - 0.25;
    if (det < 0.0) return false; // r is not sufficient
    double h = sqrt(det);
    c.x = (p1.x + p2.x) * 0.5 + (p1.y - p2.y) * h;
    c.y = (p1.y + p2.y) * 0.5 + (p2.x - p1.x) * h;
    return true;
} // swap p1 and p2 to get the other center
```

ตัวอย่างโค้ดที่ 17.20: ฟังก์ชัน circle2PtsRad หาจุดศูนย์กลางวงกลมจาก 2 จุดและรัศมี

เคล็ดลับ

ฟังก์ชัน `circle2PtsRad` ใช้สูตรทางเรขาคณิตที่อิงกับจุดกึ่งกลางของ p1p2 และระยะตั้งฉากจากจุดกึ่งกลางไปยังจุดศูนย์กลาง เพื่อให้ได้จุดศูนย์กลางอีกด้านหนึ่ง ให้สลับ p1 และ p2 แล้วเรียกฟังก์ชันอีกครั้ง

17.6 วัตถุ 2 มิติ: สามเหลี่ยม (Triangle)

สามเหลี่ยม (Triangle) เป็นรูปหลายเหลี่ยมที่มีจุดยอด 3 จุดและด้าน 3 ด้าน ถือเป็นรูปหลายเหลี่ยมที่ง่ายที่สุดและเป็นรากฐานของรูปหลายเหลี่ยมอื่นๆ เนื่องจากรูปหลายเหลี่ยมใดๆ สามารถแบ่งออกเป็นสามเหลี่ยมได้ (triangulation) ความรู้เกี่ยวกับสมบัติของสามเหลี่ยมจึงจำเป็นสำหรับการแก้โจทย์เรขาคณิตส่วนใหญ่[3]

17.6.1 ชนิดของสามเหลี่ยม

สามเหลี่ยมแบ่งออกเป็นหลายชนิดตามความยาวด้านและมุมภายใน การจำแนกชนิดของสามเหลี่ยมเป็นจุดเริ่มต้นสำคัญในการเลือกสูตรหรือขั้นตอนวิธีที่เหมาะสม สามเหลี่ยมแต่ละชนิดมีสมบัติเฉพาะที่ส่งผลต่อการคำนวณ

- **สามเหลี่ยมด้านเท่า (Equilateral):** ด้านทุกด้านยาวเท่ากันและมุมภายในทุกมุมเท่ากับ 60°
- **สามเหลี่ยมหน้าจั่ว (Isosceles):** มีสองด้านที่ยาวเท่ากัน และมุมภายในสองมุมที่อยู่ตรงข้ามด้านเท่ากันนั้นเท่ากัน
- **สามเหลี่ยมด้านไม่เท่า (Scalene):** ทุกด้านมีความยาวไม่เท่ากัน
- **สามเหลี่ยมมุมฉาก (Right):** มีมุมภายในมุมหนึ่งเป็นมุม 90° (มุมฉาก)

17.6.2 พื้นที่และเส้นรอบรูป

สามเหลี่ยมที่มีฐานยาว b และความสูง h จะมีพื้นที่ $A = 0.5 \times b \times h$ อย่างไรก็ตาม ในการแข่งขันเรามักได้รับความยาวด้านทั้งสามแทนความสูง ในกรณีนั้นใช้สูตรของ Heron (Heron's Formula) ซึ่งอ้างอิง semi-perimeter $s = (a + b + c)/2$

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

สำหรับเส้นรอบรูป เราสามารถคำนวณได้โดยตรงจากผลรวมความยาวด้านทั้งสาม $p = a + b + c$

```
double perimeter(double ab, double bc, double ca) {
    return ab + bc + ca;
}
double area(double ab, double bc, double ca) {
    double s = 0.5 * perimeter(ab, bc, ca);
    return sqrt(s * (s - ab) * (s - bc) * (s - ca)); // Heron's Formula
}
```

ตัวอย่างโค้ดที่ 17.21: ฟังก์ชัน area และ perimeter ของสามเหลี่ยมจากความยาวด้าน

17.6.3 วงกลมจารึก (Incircle) และวงกลมล้อมรอบ (Circumcircle)

วงกลมจารึก (Incircle) คือวงกลมที่ใหญ่ที่สุดที่อยู่ภายในสามเหลี่ยมและสัมผัสด้านทั้งสาม รัศมีของ incircle หาได้จาก $r = A/s$ เมื่อ A คือพื้นที่และ s คือ semi-perimeter จุดศูนย์กลางของ incircle คือจุดบรรจบกันของเส้นแบ่งครึ่งมุม (angle bisectors) ของสามเหลี่ยม การหาจุดศูนย์กลางนี้ต้องหาจุดตัดของเส้นแบ่งครึ่งมุมอย่างน้อย 2 เส้น

```
double rInCircle(double ab, double bc, double ca) {
    return area(ab, bc, ca) / (0.5 * perimeter(ab, bc, ca));
}
double rInCircle(point a, point b, point c) {
    return rInCircle(dist(a, b), dist(b, c), dist(c, a));
}
```

ตัวอย่างโค้ดที่ 17.22: ฟังก์ชัน rInCircle คำนวณรัศมีวงกลมจารึก

สำหรับการหาจุดศูนย์กลางของ incircle ฟังก์ชัน `inCircle` ใช้หลักการ angle bisector: สร้างเส้นแบ่งครึ่งมุม 2 เส้นจากจุดยอด 2 จุด แล้วหาจุดตัดของทั้งสองเส้น โดยใช้ ratio ของระยะทางเพื่อกำหนดจุดบนเส้นแบ่งครึ่งมุม

```
// returns 1 if there is an inCircle center, 0 otherwise
int inCircle(point p1, point p2, point p3, point &ctr, double &r) {
    r = rInCircle(p1, p2, p3);
    if (fabs(r) < EPS) return 0; // no inCircle center
    line l1, l2; // compute two angle bisectors
    double ratio = dist(p1, p2) / dist(p1, p3);
    point p = translate(p2, scale(toVec(p2, p3), ratio / (1 + ratio)));
    pointsToLine(p1, p, l1);
    ratio = dist(p2, p1) / dist(p2, p3);
    p = translate(p1, scale(toVec(p1, p3), ratio / (1 + ratio)));
    pointsToLine(p2, p, l2);
    areIntersect(l1, l2, ctr); // get their intersection
    return 1;
}
```

ตัวอย่างโค้ดที่ 17.23: ฟังก์ชัน inCircle หาจุดศูนย์กลางและรัศมีของ incircle

วงกลมล้อมรอบ (Circumcircle) คือวงกลมที่เล็กที่สุดที่ล้อมรอบสามเหลี่ยมและผ่านจุดยอดทั้งสาม รัศมีของ circumcircle หาได้จาก

$$R = \frac{abc}{4A}$$

เมื่อ a, b, c คือความยาวด้านทั้งสาม และ A คือพื้นที่ของสามเหลี่ยม จุดศูนย์กลางของ circumcircle คือจุดตัดกันของเส้นแบ่งครึ่งด้านที่ตั้งฉาก (perpendicular bisectors) ของด้านทั้งสาม

```
double rCircumCircle(double ab, double bc, double ca) {
```

```
    return ab * bc * ca / (4.0 * area(ab, bc, ca));
}
double rCircumCircle(point a, point b, point c) {
    return rCircumCircle(dist(a, b), dist(b, c), dist(c, a));
}
```

ตัวอย่างโค้ดที่ 17.24: ฟังก์ชัน rCircumCircle คำนวณรัศมีวงกลมล้อมรอบ

17.6.4 อสมการสามเหลี่ยม (Triangle Inequality)

ในการตรวจสอบว่าส่วนของเส้นตรง 3 ส่วน a, b, c สามารถเป็นด้านของสามเหลี่ยมได้หรือไม่ เราใช้อสมการสามเหลี่ยม (Triangle Inequalities) คือ $(a + b > c) \wedge (a + c > b) \wedge (b + c > a)$ ถ้าผลลัพธ์เป็นเท็จ ส่วนของเส้นตรงทั้งสามไม่สามารถสร้างสามเหลี่ยมได้ ถ้าความยาวทั้งสามถูกเรียงโดยให้ a เป็นเส้นที่สั้นที่สุดและ c เป็นเส้นที่ยาวที่สุด สามารถตรวจสอบเพียง $a + b > c$ ได้ เนื่องจากอสมการอีกสองเส้นจะเป็นจริงโดยปริยาย

17.6.5 กฎของ Cosine และกฎของ Sines

กฎของ Cosine (Law of Cosines) เป็นบทบัญญัติเกี่ยวกับสามเหลี่ยมทั่วไปที่สัมพันธ์ความยาวด้านกับ cos ของมุม พิจารณาสามเหลี่ยมที่มีด้าน a, b, c และมุม γ ที่อยู่ตรงข้ามด้าน c จะได้

$$c^2 = a^2 + b^2 - 2ab\cos\gamma, \qquad \gamma = \arccos\left(\frac{a^2 + b^2 - c^2}{2ab}\right)$$

สูตรสำหรับมุมอื่นอีกสองมุม (α และ β) มีรูปแบบคล้ายกัน เพียงหมุนเปลี่ยนตัวแปร

กฎของ Sines (Law of Sines) เป็นสมการที่สัมพันธ์ความยาวด้านของสามเหลี่ยมใดๆ กับ sin ของมุมที่อยู่ตรงข้าม เมื่อ R แทนรัศมีของวงกลมล้อมรอบสามเหลี่ยม จะได้

$$\frac{a}{\sin\alpha} = \frac{b}{\sin\beta} = \frac{c}{\sin\gamma} = 2R$$

ทฤษฎีบทปิทากอรัส (Pythagorean Theorem) เป็นกรณีพิเศษของกฎของ Cosines ที่ใช้กับสามเหลี่ยมมุมฉาก ถ้ามุม γ เป็นมุมฉาก (90° หรือ $\pi/2$ เรเดียน) แล้ว $\cos\gamma = 0$ ทำให้ได้ $c^2 = a^2 + b^2$ Pythagorean Triple คือ triple ของจำนวนเต็มบวก (a, b, c) ที่สอดคล้องกับ $a^2 + b^2 = c^2$ ตัวอย่างที่คลาสสิกคือ $(3, 4, 5)$ ถ้า (a, b, c) เป็น Pythagorean triple แล้ว (ka, kb, kc) สำหรับจำนวนเต็มบวก k ใดๆ ก็เป็น Pythagorean triple เช่นกัน

แบบฝึกหัดท้ายหัวข้อ

1. จงเขียนโปรแกรมเพื่อหาจุดศูนย์กลางของวงกลมล้อมรอบสามเหลี่ยมที่ผ่านจุดสามจุด a, b, c โครงสร้างฟังก์ชันคล้ายกับฟังก์ชัน `inCircle`

2. จงเขียนโปรแกรมเพื่อตรวจสอบว่าจุด d อยู่ภายในวงกลมล้อมรอบสามเหลี่ยมของจุดสามจุด a, b, c หรือไม่

17.7 วัตถุ 2 มิติ: สี่เหลี่ยม (Quadrilateral)

สี่เหลี่ยม (Quadrilateral หรือ Quadrangle) เป็นรูปหลายเหลี่ยมที่มี 4 จุดยอดและ 4 ด้าน สี่เหลี่ยมแบ่งออกเป็นหลายประเภทตามสมบัติของด้านและมุม ความเข้าใจสมบัติของแต่ละประเภทช่วยให้เลือกสูตรที่ถูกต้องสำหรับการคำนวณพื้นที่และเส้นรอบรูป[3]

ตารางที่ 17.4 สรุปประเภทของสี่เหลี่ยมและสูตรพื้นที่

ประเภท	สมบัติ/สูตร	พื้นที่ A
Rectangle (สี่เหลี่ยมมุมฉาก)	มี 4 มุมฉาก กว้าง w สูง h พื้นที่ $A = wh$ เส้นรอบรูป $p = 2(w + h)$	$w \times h$
Square (สี่เหลี่ยมจัตุรัส)	กรณีพิเศษของ Rectangle ที่ $w = h$	w^2
Trapezium (สี่เหลี่ยมคางหมู)	มีด้านขนาน 1 คู่ ความยาว w_1, w_2 สูง h ถ้าด้านที่ไม่ขนานเท่ากัน เรียก Isosceles Trapezium	$0.5(w_1 + w_2)h$
Parallelogram (สี่เหลี่ยมด้านขนาน)	ด้านตรงข้ามขนานกัน	ฐาน $\times$ สูง
Kite (สี่เหลี่ยมรูปว่าว)	ด้านที่ติดกันยาวเท่ากัน 2 คู่	$\frac{d_1 \times d_2}{2}$
Rhombus (สี่เหลี่ยมขนมเปียกปูน)	กรณีพิเศษของ Parallelogram ที่ทุกด้านยาวเท่ากัน (เป็น Kite ด้วย)	$\frac{d_1 \times d_2}{2}$

สี่เหลี่ยมมุมฉาก (Rectangle) เป็นรูปหลายเหลี่ยมที่มี 4 จุดยอดเชื่อมด้วยเส้น 4 เส้น และมี 4 มุมฉาก มีความกว้าง w และความสูง h จะมีพื้นที่ $A = w \times h$ และเส้นรอบรูป $p = 2 \times (w + h)$ สี่เหลี่ยมจัตุรัส (Square) เป็นกรณีพิเศษของสี่เหลี่ยมมุมฉากที่ $w = h$

สี่เหลี่ยมคางหมู (Trapezium) เป็นรูปหลายเหลี่ยมที่มีด้านคู่หนึ่งขนานกัน ถ้าด้านที่ไม่ขนานกันสองด้านมีความยาวเท่ากัน เราเรียกว่า Isosceles Trapezium สี่เหลี่ยมคางหมูที่มีด้านขนานยาว w_1 และ w_2 และมีความสูง h ระหว่างเส้นขนาน จะมีพื้นที่ $A = 0.5 \times (w_1 + w_2) \times h$

สี่เหลี่ยมด้านขนาน (Parallelogram) มีสมบัติด้านตรงข้ามขนานกัน ส่วนสี่เหลี่ยมรูปว่าว (Kite) มีด้านที่ติดกันยาวเท่ากัน 2 คู่ พื้นที่ของสี่เหลี่ยมรูปว่าวคือ $\frac{d_1 \times d_2}{2}$ เมื่อ d_1, d_2 คือความยาวเส้นทแยงมุม สี่เหลี่ยมขนมเปียกปูน (Rhombus) เป็นกรณีพิเศษของทั้ง Parallelogram และ Kite ที่ทุกด้านยาวเท่ากัน

17.8 รูปหลายเหลี่ยม (Polygon)

รูปหลายเหลี่ยม (Polygon) คือรูปบนระนาบที่ถูกล้อมรอบด้วยเส้นทางปิด (เส้นทางที่เริ่มต้นและจบที่จุดยอดเดียวกัน) ประกอบด้วยลำดับจำกัดของส่วนของเส้นตรง ซึ่งส่วนของเส้นตรงนี้ถูกเรียกว่าขอบหรือด้าน จุดที่

ด้านสองด้านมาบรรจบกันคือจุดยอดของ polygon หรือมุม วิธีมาตรฐานในการเก็บ polygon คือการแทนค่าจุดยอดในลำดับวนตามเข็มหรือทวนเข็ม แล้วเพิ่มจุดยอดแรกเป็นจุดยอดสุดท้ายเพื่อปิดวง ในที่นี้เราจะใช้ลำดับวนทวนเข็มนาฬิกา (counter-clockwise) เป็นหลัก[3, 11]

17.8.1 การเก็บจุดยอดของ Polygon

ตัวอย่างต่อไปนี้สร้าง polygon ที่มี 6 จุดยอดในลำดับทวนเข็มนาฬิกา โดยใช้ `vector<point>` สังเกตว่าจุดยอดแรกถูกเพิ่มเป็นจุดสุดท้ายด้วย เพื่อให้ลูปการคำนวณสามารถวนจาก `P[i]` ไป `P[i+1]` ได้โดยไม่ต้องจัดการกรณีพิเศษที่จุดสุดท้าย

```
vector<point> P;
P.push_back(point(1, 1));    // P0
P.push_back(point(3, 3));    // P1
P.push_back(point(9, 1));    // P2
P.push_back(point(12, 4));   // P3
P.push_back(point(9, 7));    // P4
P.push_back(point(1, 7));    // P5
P.push_back(P[0]);           // important: loop back
```

ตัวอย่างโค้ดที่ 17.25: การสร้าง polygon 6 จุดยอดในลำดับทวนเข็มนาฬิกา

17.8.2 เส้นรอบรูปของ Polygon

เส้นรอบรูป (Perimeter) ของ polygon ที่มี n จุดยอดสามารถคำนวณได้จากผลรวมของระยะทางยูคลิเดียนระหว่างจุดยอดคู่ข้างเคียง เนื่องจากเราเก็บจุดยอดแรกเป็นจุดสุดท้ายด้วย การวนซ้ำจาก `i = 0` ถึง `P.size()-2` จะครอบคลุมด้านทั้งหมด

```
// returns the perimeter, which is the sum of Euclidean
// distances of consecutive line segments (polygon edges)
double perimeter(const vector<point> &P) {
    double result = 0.0;
    for (int i = 0; i < (int)P.size()-1; i++)
        // remember that P[0] = P[n-1]
        result += dist(P[i], P[i+1]);
    return result;
}
```

ตัวอย่างโค้ดที่ 17.26: ฟังก์ชัน perimeter คำนวณเส้นรอบรูปของ polygon

17.8.3 พื้นที่ของ Polygon (Shoelace Formula)

พื้นที่ A ของ polygon ที่มี n จุดยอดสามารถคำนวณได้จาก determinant ของ matrix ที่เกิดจากจุดยอดคู่ข้างเคียง สูตรนี้เรียกว่า Shoelace Formula มีรูปแบบดังนี้

$$A = \frac{1}{2}\left|\sum_{i=0}^{n-2}(x_i y_{i+1} - x_{i+1} y_i)\right|$$

ค่า absolute ทำให้ไม่ต้องกังวลว่าจุดยอดถูกเก็บในลำดับตามเข็มหรือทวนเข็มนาฬิกา ผลลัพธ์จะเป็นพื้นที่ที่เป็นบวกเสมอ

```
// returns the area, which is half the determinant
double area(const vector<point> &P) {
    double result = 0.0, x1, y1, x2, y2;
    for (int i = 0; i < (int)P.size()-1; i++) {
        x1 = P[i].x;    x2 = P[i+1].x;
        y1 = P[i].y;    y2 = P[i+1].y;
        result += (x1 * y2 - x2 * y1);
    }
    return fabs(result) / 2.0;
}
```

ตัวอย่างโค้ดที่ 17.27: ฟังก์ชัน area คำนวณพื้นที่ของ polygon ด้วย Shoelace Formula

17.8.4 การตรวจสอบ Convex/Concave

รูปหลายเหลี่ยมนูน (Convex Polygon) คือรูปหลายเหลี่ยมที่พื้นที่ภายในเป็นเซตนูน (convex set) สมบัติต่อไปนี้ล้วนสมมูลกับ convex polygon: มุมภายในทุกมุมมีขนาดน้อยกว่า $180°$, ส่วนของเส้นตรงทุกเส้นที่เชื่อมระหว่างจุดยอดสองจุดใดๆ วางตัวอยู่ภายในขอบเขตของรูปหลายเหลี่ยม และเส้นตรงใดๆ ที่วาดภายในรูปหลายเหลี่ยมไม่ตัดกับขอบใดๆ รูปที่ไม่ได้เป็น convex จะถูกเรียกว่ารูปหลายเหลี่ยมเว้า (Concave Polygon)

ในการทดสอบว่า polygon เป็น convex มีวิธีที่ง่ายกว่าการลองตรวจสอบทุกส่วนของเส้นตรงว่าอยู่ภายในหรือไม่ คือตรวจสอบว่าทุกจุดยอดสามจุดที่ติดกันหมุนไปทางเดียวกัน (หมุนซ้าย/ทวนเข็ม ถ้าจุดยอดถูกเรียงในลำดับทวนเข็ม หรือหมุนขวาในทางกลับกัน) ถ้าพบว่ามีอย่างน้อยสามจุดที่ไม่สอดคล้องกับทิศทางเดียวกัน แสดงว่า polygon เป็น concave

```
bool isConvex(const vector<point> &P) {
    int sz = (int)P.size();
    if (sz <= 3) return false; // a point/sz=2 or a line/sz=3
    bool isLeft = ccw(P[0], P[1], P[2]); // remember one result
    for (int i = 1; i < sz-1; i++) // then compare with the others
        if (ccw(P[i], P[i+1], P[(i+2) == sz ? 1 : i+2]) != isLeft)
            return false; // different sign -> concave
```

```
    return true;
}
```

ตัวอย่างโค้ดที่ 17.28: ฟังก์ชัน isConvex ตรวจสอบว่า polygon เป็น convex หรือไม่

17.8.5 การตรวจสอบจุดภายใน Polygon (inPolygon)

การทดสอบอีกอย่างที่ใช้บ่อยคือการตรวจสอบว่าจุด pt อยู่ภายในหรือภายนอก polygon P ฟังก์ชันต่อไปนี้ใช้ Winding Number Algorithm ซึ่งทำงานโดยการคำนวณผลรวมของมุมระหว่างจุด 3 จุด $\{P[i], pt, P[i+1]\}$ เมื่อ $P[i]$-$P[i+1]$ คือด้านที่ติดกันของ polygon ต้องระวังว่าหากหมุนซ้ายจะเพิ่มมุม หากหมุนขวาจะลดมุม ถ้าผลรวมได้ 2π (360 องศา) แสดงว่า pt อยู่ภายใน polygon P

```
// returns true if point p is in either convex/concave polygon P
bool inPolygon(point pt, const vector<point> &P) {
    if ((int)P.size() == 0) return false;
    double sum = 0;
    // assume the first vertex is equal to the last vertex
    for (int i = 0; i < (int)P.size()-1; i++) {
        if (ccw(pt, P[i], P[i+1]))
            sum += angle(P[i], pt, P[i+1]); // left turn/ccw
        else
            sum -= angle(P[i], pt, P[i+1]); // right turn/cw
    }
    return fabs(fabs(sum) - 2*PI) < EPS;
}
```

ตัวอย่างโค้ดที่ 17.29: ฟังก์ชัน inPolygon ตรวจสอบว่าจุดอยู่ภายใน polygon หรือไม่

ข้อควรระวัง

Winding Number Algorithm ใช้ acos ทำให้มีความเสี่ยงด้าน numerical precision สำหรับ convex polygon สามารถใช้ Ray Casting Algorithm หรือการตรวจสอบ half-plane แต่ละด้านแทนได้ ซึ่งเร็วและเสถียรกว่า

17.8.6 การตัด Polygon ด้วยเส้นตรง (cutPolygon)

สิ่งที่น่าสนใจอีกอย่างที่ทำกับ convex polygon คือการตัดมันเป็นสอง convex sub-polygons ด้วยเส้นตรงที่นิยามด้วยจุดสองจุด a และ b แนวคิดเบื้องต้นของ cutPolygon คือการวนรอบทีละจุดยอดของ original polygon Q ถ้าเส้นตรง ab และจุดยอด v ของ polygon ทำให้เกิดหมุนซ้าย (นั่นหมายความว่า v อยู่ทางซ้ายของเส้น ab) เราจะเก็บ v ไว้ใน new polygon P เมื่อหาด้านของ polygon edge ที่ตัดกับเส้น ab เราจะใช้จุดตัดนั้นเป็นส่วนหนึ่งของ new polygon P จากนั้นข้ามจุดยอดของ Q ที่อยู่ทางด้านขวาของเส้น ab จนกระทั่งกลับมายังอีกด้านของ polygon ที่ตัดกับเส้น ab อีกครั้ง

```
// line segment p-q intersect with line A-B
point lineIntersectSeg(point p, point q, point A, point B) {
    double a = B.y - A.y;
    double b = A.x - B.x;
    double c = B.x * A.y - A.x * B.y;
    double u = fabs(a * p.x + b * p.y + c);
    double v = fabs(a * q.x + b * q.y + c);
    return point((p.x * v + q.x * u) / (u+v),
                 (p.y * v + q.y * u) / (u+v));
}
```

ตัวอย่างโค้ดที่ 17.30: ฟังก์ชัน lineIntersectSeg หาจุดตัดระหว่าง segment และเส้นตรง

```
// cuts polygon Q along the line formed by point a -> point b
// (note: the last point must be the same as the first point)
vector<point> cutPolygon(point a, point b, const vector<point> &Q) {
    vector<point> P;
    for (int i = 0; i < (int)Q.size(); i++) {
        double left1 = cross(toVec(a, b), toVec(a, Q[i])), left2 = 0;
        if (i != (int)Q.size()-1) left2 = cross(toVec(a, b), toVec(a, Q[i
    +1]));
        if (left1 > -EPS) P.push_back(Q[i]); // Q[i] is on the left of ab
        if (left1 * left2 < -EPS) // edge (Q[i], Q[i+1]) crosses line ab
            P.push_back(lineIntersectSeg(Q[i], Q[i+1], a, b));
    }
    if (!P.empty() && !(P.back() == P.front()))
        P.push_back(P.front()); // make P's first point = P's last point
    return P;
}
```

ตัวอย่างโค้ดที่ 17.31: ฟังก์ชัน cutPolygon ตัด polygon ด้วยเส้นตรง a-b

17.9 เปลือกนูน (Convex Hull)

Convex Hull ของเซตของจุด P คือ convex polygon ที่มีขนาดเล็กที่สุด CH(P) ที่ครอบคลุมจุดทุกจุดใน P สำหรับแต่ละจุดใน P นั้นเป็นขอบของ CH(P) หรือไม่ก็อยู่ภายใน จินตนาการว่ามีจุดอยู่บนกระดาน แล้วเราตอกตะปูบนจุด จากนั้นนำหนังยางมาครอบจุด ถ้าปล่อยหนังยาง หนังยางจะหุ้มเป็นบริเวณขนาดเล็กที่สุดเท่าที่จะทำได้ บริเวณที่หุ้มได้นั่นคือ convex hull ของเซตของจุด ในชีวิตจริง convex hull มี application ในปัญหา packing problems และการประมวลผลภาพ[3, 2]

17.9.1 แนวคิดของขั้นตอนวิธีหา Convex Hull

เนื่องจากทุกจุดยอดใน CH(P) คือจุดยอดในเซต P ขั้นตอนวิธีในการหา convex hull คือขั้นตอนวิธีที่ตัดสินใจว่าจุดใดใน P ที่ควรเลือกเป็นส่วนหนึ่งของ convex hull มีขั้นตอนวิธีในการหา convex hull อยู่หลายตัว ในเรื่องนี้เราเลือก Graham's Scan Algorithm ซึ่งมีความซับซ้อนเวลา $\mathcal{O}(n \log n)$

17.9.2 Graham's Scan Algorithm

Graham's Scan เริ่มต้นจากการเรียงทั้ง n จุดของ P ตามมุมเมื่อเทียบกับจุด pivot ในที่นี้เราจะหยิบจุดล่างสุดเป็น pivot (จุดที่มีค่า y ต่ำที่สุด ถ้าเท่ากันให้เลือกจุดที่ x มากที่สุด เพื่อหลีกเลี่ยงกรณีพิเศษ) หลังจากเรียงตามมุมแล้ว เราจะเห็นเส้น 0-1, 0-2, 0-3, ..., 0-n-1 ในลำดับทวนเข็มนาฬิกา

จากนั้น algorithm จะเก็บ stack S ของจุด แต่ละจุดของ P จะถูก push เพียงครั้งเดียวเข้าไปใน S และจุดที่ไม่เป็นส่วนหนึ่งของ CH(P) จะถูก pop ออกจาก S Graham's Scan รักษาคุณสมบัติต่อไปนี้: สมาชิก 3 ตัวบนของ stack S จะต้องหมุนซ้ายตลอด (ซึ่งเป็นคุณสมบัติพื้นฐานของ convex polygon)

การดำเนินการทีละขั้น: เริ่มต้นเราใส่จุด 3 จุด คือจุด $n-1$, 0, และ 1 ในตัวอย่าง stack เริ่มต้นจะเก็บจุด 11-0-1 ซึ่งจะเป็นหมุนซ้ายเสมอ ต่อไปลองใส่จุด 2 และ 0-1-2 เป็นหมุนซ้าย ดังนั้นเราจะเก็บจุด 2 เข้าไปใน Stack ทำให้ S ขณะนี้เก็บ (bottom) 11-0-1-2 (top)

ต่อไปลองใส่จุด 3 และ 1-2-3 เป็นการหมุนขวา ถ้าเราเก็บจุดที่มาก่อนก่อนจุด 3 (จุด 2) จะทำให้ไม่ได้ convex polygon ดังนั้นเราจะ pop จุด 2 ออกจาก Stack ทำให้ S ปัจจุบันเป็น (bottom) 11-0-1 (top) อีกครั้ง จากนั้นลองใส่จุด 3 อีกครั้ง ครั้งนี้ 0-1-3 เป็นหมุนซ้าย ดังนั้นเราจะเก็บจุด 3 ทำให้ Stack S ปัจจุบันเป็น (bottom) 11-0-1-3 (top) เราทำเช่นนี้ไปเรื่อยๆ กับจุดถัดไป

เมื่อ Graham's Scan หยุด อะไรก็ตามที่เหลืออยู่ใน S จะเป็นจุดใน CH(P) ในตัวอย่างคือ (bottom) 11-0-1-4-7-10-11 (top) Graham's Scan จะกำจัดทุกจุดที่หมุนขวา เมื่อจุดยอดสามจุดที่ติดกันใน S จะหมุนซ้ายเสมอ ทำให้สุดท้ายเราได้ convex polygon

17.9.3 โค้ด Graham's Scan

การเรียงจุดตามมุมใช้ฟังก์ชัน comparator `angleCmp` ที่อ้างอิง pivot ส่วนกลาง ถ้าสองจุด collinear กับ pivot ให้เรียงตามระยะทางใกล้ก่อน มิฉะนั้นเรียงตามมุม `atan2` ของเวกเตอร์จาก pivot ไปยังจุด

```
point pivot(0, 0);
bool angleCmp(point a, point b) { // angle-sorting function
    if (collinear(pivot, a, b)) // special case
        return dist(pivot, a) < dist(pivot, b); // closer first
    double d1x = a.x - pivot.x, d1y = a.y - pivot.y;
    double d2x = b.x - pivot.x, d2y = b.y - pivot.y;
    return (atan2(d1y, d1x) - atan2(d2y, d2x)) < 0;
}
```

ตัวอย่างโค้ดที่ 17.32: ฟังก์ชัน angleCmp สำหรับเรียงจุดตามมุม

```
vector<point> CH(vector<point> P) {  // content of P may be reshuffled
    int i, j, n = (int)P.size();
    if (n <= 3) {
        if (!(P[0] == P[n-1])) P.push_back(P[0]);  // safeguard
        return P;  // special case, the CH is P itself
    }
    // first, find P0 = point with lowest Y and if tie: rightmost X
    int P0 = 0;
    for (i = 1; i < n; i++)
        if (P[i].y < P[P0].y || (P[i].y == P[P0].y && P[i].x > P[P0].x))
            P0 = i;
    point temp = P[0]; P[0] = P[P0]; P[P0] = temp;  // swap P[P0] with P[0]
    // second, sort points by angle w.r.t. pivot P0
    pivot = P[0];  // use this global variable as reference
    sort(++P.begin(), P.end(), angleCmp);  // we do not sort P[0]
    // third, the ccw tests
    vector<point> S;
    S.push_back(P[n-1]); S.push_back(P[0]); S.push_back(P[1]);  // initial S
    i = 2;  // then, we check the rest
    while (i < n) {  // note: N must be >= 3 for this method to work
        j = (int)S.size()-1;
        if (ccw(S[j-1], S[j], P[i])) S.push_back(P[i++]);  // left turn, accept
        else S.pop_back();  // or pop the top of S until we have a left turn
    }
    return S;  // return the result
}
```

ตัวอย่างโค้ดที่ 17.33: ฟังก์ชัน CH หา Convex Hull ด้วย Graham's Scan

```
int main() {
    vector<point> P;
    // ... add points to P ...
    P = CH(P);  // replace P with its Convex Hull
    // P now holds vertices of the Convex Hull in CCW order
    // (P[0] == P[P.size()-1] to close the loop)
    return 0;
}
```

ตัวอย่างโค้ดที่ 17.34: การเรียกใช้งานฟังก์ชัน CH ใน main

ความซับซ้อนของ Graham's Scan

- **เวลา:** $\mathcal{O}(n \log n)$ สำหรับการ sort ตามมุม และ $\mathcal{O}(n)$ สำหรับการ scan (แต่ละจุดถูก push และ pop ไม่เกินครั้งละ 1 ครั้ง) รวม $\mathcal{O}(n \log n)$
- **พื้นที่:** $\mathcal{O}(n)$ สำหรับ stack
- **ข้อควรระวัง:** กรณี collinear points ต้องระวังเป็นพิเศษ การตั้งค่า `ccw` ให้ใช้ `> 0` (ไม่รับ collinear) ทำให้ collinear points ที่อยู่ระหว่างจุดยอดของ hull ถูกตัดออก

17.10 โจทย์แนะนำ

ต่อไปนี้คือโจทย์ที่แนะนำสำหรับฝึกฝนเรขาคณิตเชิงคำนวณ:

โจทย์แนะนำสำหรับบทที่ 17

จุด เส้นตรง และเวกเตอร์:

- UVA 10432 — Polygon Inside A Circle (พื้นที่รูปหลายเหลี่ยม)[20]
- UVA 10585 — Center of symmetry? (จุด)[20]
- Codeforces 514B — Han Solo and Lazer Gun (เส้นตรง)[10]

Cross Product และ CCW:

- UVA 10927 — Bright Light (CCW)[20]
- UVA 10078 — Art Gallery (Convex Hull concept)[20]
- Codeforces 166B — Polygons (CCW application)[10]

เรขาคณิตเชิงคำนวณทั่วไป:

- CSES Point Location Test — จุดอยู่ในรูปหลายเหลี่ยมหรือไม่
- CSES Polygon Area — พื้นที่รูปหลายเหลี่ยม
- UVA 11479 — Is this the easiest problem? (สามเหลี่ยม)[20]

17.11 สรุปบทที่ 17

ในบทนี้ เราได้ศึกษาพื้นฐานเรขาคณิตเชิงคำนวณตั้งแต่การแทนจุดและเวกเตอร์ การดำเนินการพื้นฐาน เส้นตรงและส่วนของเส้นตรง ผลคูณเชิงเวกเตอร์และ CCW Test วัตถุ 2 มิติ (วงกลม สามเหลี่ยม สี่เหลี่ยม และรูปหลายเหลี่ยม) ไปจนถึงขั้นตอนวิธีเรขาคณิตขั้นสูง เช่น การตรวจสอบ convex/concave การตัด polygon

และการหา Convex Hull ด้วย Graham's Scan ซึ่งเป็นรากฐานสำคัญสำหรับการแก้โจทย์เรขาคณิตในการแข่งขัน

สรุปสิ่งสำคัญของบทที่ 17

1. **จุดและเวกเตอร์** แทนด้วย `struct point` และ `struct vec` พร้อมฟังก์ชันพื้นฐาน เช่น การเปรียบเทียบ การคำนวณระยะทาง และการหมุนจุด
2. **ผลคูณจุด (Dot Product)** ใช้คำนวณมุมระหว่างเวกเตอร์และ scalar projection
3. **เส้นตรง** แทนด้วยสมการ $ax + by + c = 0$ พร้อมฟังก์ชันตรวจสอบขนาน ตัดกัน และระยะทาง
4. **ผลคูณเชิงเวกเตอร์ (Cross Product)** ใช้ตรวจสอบทิศทางการหมุน (CCW) และคำนวณพื้นที่
5. **CCW Test** เป็นพื้นฐานของ Convex Hull, Polygon Operations และขั้นตอนวิธีเรขาคณิตหลายตัว
6. **วัตถุ 2 มิติ** ครอบคลุมวงกลม (สูตร arc/chord/sector/segment, circle2PtsRad), สามเหลี่ยม (Heron, incircle, circumcircle, กฎ Cosine/Sines), สี่เหลี่ยม (Rectangle, Trapezium, Parallelogram, Kite, Rhombus) และรูปหลายเหลี่ยม (Shoelace Formula, isConvex, inPolygon, cutPolygon)
7. **Convex Hull** ใช้ Graham's Scan Algorithm ในเวลา $\mathcal{O}(n \log n)$ โดยเรียงจุดตามมุมแล้วใช้ stack กับ CCW test เพื่อคัดเลือกจุดยอดของ hull
8. **ระวัง Floating Point** — ใช้ `EPS` สำหรับการเปรียบเทียบ และหลีกเลี่ยง `sqrt` เมื่อเป็นไปได้

อภิธานศัพท์

อภิธานศัพท์นี้รวบรวมคำศัพท์สำคัญที่ปรากฏในหนังสือเล่มนี้ จัดเรียงตามหมวดหมู่เพื่อให้สะดวกต่อการค้นหา แต่ละรายการแสดงคำภาษาไทย คำภาษาอังกฤษในวงเล็บ และคำนิยามแบบสั้น

ทั่วไป

การโปรแกรมเชิงแข่งขัน (Competitive Programming)
: กีฬาทางปัญญาที่ผสมผสานการออกแบบขั้นตอนวิธีและการเขียนโปรแกรมเพื่อแก้โจทย์ปัญหาภายในข้อจำกัดเวลาและทรัพยากร

ความซับซ้อนของเวลา (Time Complexity)
: การประเมินว่าขั้นตอนวิธีใช้เวลาเพิ่มขึ้นเร็วแค่ไหนเมื่อขนาดข้อมูลนำเข้าเพิ่มขึ้น วัดเป็นอัตราการเติบโต ไม่ใช่เวลาจริงเป็นวินาที

สัญกรณ์โอใหญ่ (Big-O Notation)
: สัญกรณ์ที่บอกขอบเขตบนของการเติบโตของเวลาทำงานของขั้นตอนวิธี เช่น $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n^2)$

สัญกรณ์โอมEGA (Omega Notation)
: สัญกรณ์ที่บอกขอบเขตล่างของการเติบโตของเวลาทำงานของขั้นตอนวิธี

สัญกรณ์ธีตา (Theta Notation)
: สัญกรณ์ที่บอกขอบเขตทั้งบนและล่างของการเติบโตของเวลาทำงานของขั้นตอนวิธี คือขอบเขตแน่นหนา

ความซับซ้อนของปริภูมิ (Space Complexity)
: ปริมาณหน่วยความจำที่ขั้นตอนวิธีใช้เป็นฟังก์ชันของขนาดข้อมูลนำเข้า

กรณีแย่สุด (Worst Case)
: กรณีที่ขั้นตอนวิธีใช้เวลาทำงานมากที่สุด มักใช้เป็นเกณฑ์ในการวิเคราะห์ประสิทธิภาพ

การล้ม (Overflow)
: ข้อผิดพลาดเมื่อค่าเกินขอบเขตที่ประเภทข้อมูลเก็บได้ เช่น `int` เก็บได้สูงสุดประมาณ 2×10^9

เลขคณิตมอดูลาร์ (Modular Arithmetic)
: การคำนวณเลขคณิตบนระบบเศษเหลือ mod m มีคุณสมบัติกระจายได้สำหรับการบวก ลบ และคูณ

ค่าเฝ้า (Sentinel)
: ค่าพิเศษที่ใช้บอกจุดสิ้นสุดของข้อมูลนำเข้า เช่น `0 0` หรือ -1

จุดทศนิยม (Floating Point)
การแทนจำนวนจริงในคอมพิวเตอร์ซึ่งมีข้อจำกัดเรื่องความแม่นยำ ต้องใช้ EPS ในการเปรียบเทียบ

เอพซิลอน (EPS)
ค่าคงที่ขนาดเล็ก (เช่น `1e-9`) ใช้เปรียบเทียบจุดทศนิยมแทนการใช้ `==`

STL (Standard Template Library)
ไลบรารีมาตรฐานของ C++ ที่มีโครงสร้างข้อมูลและขั้นตอนวิธีพร้อมใช้ เช่น `vector`, `set`, `map`, `sort`

มาโคร (Macro)
การใช้ `#define` เพื่อสร้างตัวย่อสำหรับคำสั่งหรือรูปแบบที่ใช้บ่อยในการแข่งขัน

typedef
คำสั่ง C++ สำหรับสร้างชื่อย่อของประเภทข้อมูล เช่น `typedef long long ll;`

ทฤษฎีบทมาสเตอร์ (Master Theorem)
ทฤษฎีบทสำหรับหาความซับซ้อนของขั้นตอนวิธีแบบแบ่งแยกและเอาชนะจากสมการเรียกซ้ำ

การวิเคราะห์เฉลี่ย (Amortized Analysis)
การวิเคราะห์เวลาเฉลี่ยของลำดับการดำเนินการ แทนที่จะวิเคราะห์การดำเนินการเดี่ยว

โครงสร้างข้อมูล

โครงสร้างข้อมูล (Data Structure)
วิธีการจัดเก็บและจัดระเบียบข้อมูลเพื่อให้เข้าถึงและดำเนินการได้อย่างมีประสิทธิภาพ

แถวลำดับ (Array)
โครงสร้างข้อมูลเก็บสมาชิกชนิดเดียวกันต่อเนื่องกัน เข้าถึงด้วยดัชนีได้ในเวลา $\mathcal{O}(1)$

เวกเตอร์ (Vector)
แถวลำดับพลวัตของ C++ STL ที่ขยายขนาดอัตโนมัติเมื่อเพิ่มสมาชิก

กองซ้อน (Stack)
โครงสร้างข้อมูลแบบเข้าทีหลังออกก่อน (LIFO) รองรับ `push` และ `pop`

แถวคอย (Queue)
โครงสร้างข้อมูลแบบเข้าก่อนออกก่อน (FIFO) รองรับ `push` และ `pop`

แถวคอยลำดับความสำคัญ (Priority Queue)
โครงสร้างข้อมูลที่ดึงสมาชิกที่มีลำดับความสำคัญสูงสุดออกก่อน ใช้ heap ในการ implement

เซต (Set)
โครงสร้างข้อมูลเก็บสมาชิกไม่ซ้ำ เรียงลำดับอัตโนมัติ ค้นหา แทรก ลบได้ใน $\mathcal{O}(\log n)$

แมป (Map)

โครงสร้างข้อมูลแมปคีย์ไปค่า เรียงตามคีย์ เข้าถึงได้ใน $\mathcal{O}(\log n)$

ตารางแฮช (Hash Table)

โครงสร้างข้อมูลที่ใช้ฟังก์ชันแฮชจัดเก็บข้อมูล เข้าถึงได้ในเฉลี่ย $\mathcal{O}(1)$

ผลรวมสะสม (Prefix Sum)

แถวลำดับที่เก็บผลรวมสะสม ช่วยตอบคำถามผลรวมช่วงได้ใน $\mathcal{O}(1)$

ต้นไม้ช่วง (Segment Tree)

ต้นไม้ทวิภาคสำหรับตอบคำถามแบบช่วงและอัปเดตค่าได้ใน $\mathcal{O}(\log n)$

ต้นไม้ทวิภาคจากดัชนี (Binary Indexed Tree / Fenwick Tree)

โครงสร้างข้อมูลกระชับสำหรับผลรวมสะสมแบบช่วง อัปเดตและสอบถามได้ใน $\mathcal{O}(\log n)$

Sparse Table

โครงสร้างข้อมูลสำหรับสอบถามค่าช่วงแบบ idempotent (เช่น ค่าน้อยสุด) บนแถวลำดับคงที่ได้ใน $\mathcal{O}(1)$

Union-Find / DSU (Disjoint Set Union)

โครงสร้างข้อมูลจัดการเซตแยกส่วน รองรับการรวมเซตและหาตัวแทนเซตได้รวดเร็ว

การบีบอัดเส้นทาง (Path Compression)

เทคนิคเพิ่มประสิทธิภาพ Union-Find โดยตั้งค่าพ่อแม่ทุกจุดระหว่าง Find ให้ชี้ตรงไปราก

Union by Rank

เทคนิคเพิ่มประสิทธิภาพ Union-Find โดยต่อต้นไม้เตี้ยเข้าต้นไม้สูงเสมอ

ขั้นตอนวิธี

การเรียงลำดับ (Sorting)

การจัดเรียงข้อมูลตามลำดับที่กำหนด ขั้นตอนวิธีมาตรฐานทำงานใน $\mathcal{O}(n \log n)$

การเรียงลำดับแบบผสาน (Merge Sort)

ขั้นตอนวิธีเรียงลำดับแบบแบ่งแยกและเอาชนะ ทำงานใน $\mathcal{O}(n \log n)$ แบบเสถียร

การเรียงลำดับแบบเร็ว (Quick Sort)

ขั้นตอนวิธีเรียงลำดับแบบแบ่งแยกและเอาชนะ โดยเลือก pivot เฉลี่ย $\mathcal{O}(n \log n)$

การค้นหาแบบทวิภาค (Binary Search)

ค้นหาค่าในแถวลำดับเรียงลำดับโดยแบ่งครึ่งทำซ้ำ ทำงานใน $\mathcal{O}(\log n)$

การค้นหาแบบทวิภาคบนคำตอบ (Binary Search on Answer)

ใช้ binary search หาคำตอบที่เป็นไปได้ของโจทย์ optimization

การค้นหาแบบครบถ้วน (Complete Search / Brute Force)

กลยุทธ์ตรวจสอบทุกความเป็นไปได้ ให้คำตอบถูกต้องแน่นอนแต่อาจช้า

การย้อนกลับ (Backtracking)

เทคนิคค้นหาแบบครบถ้วนที่ย้อนกลับเมื่อทางปัจจุบันไม่นำไปสู่คำตอบ

การตัดแต่งกิ่ง (Pruning)

เทคนิคตัดทิ้งสาขาที่ไม่นำไปสู่คำตอบในการค้นหาแบบย้อนกลับ

พบกันกลางทาง (Meet in the Middle)

เทคนิคแบ่งปัญหาเป็นสองส่วนแก้แยกกันแล้วรวมผล ลดเวลาจาก $\mathcal{O}(2^n)$ เป็น $\mathcal{O}\left(2^{n/2}\right)$

การโปรแกรมแบบพลวัต (Dynamic Programming / DP)

เทคนิคแก้ปัญหาโดยแบ่งเป็นปัญหาย่อยซ้อนทับและจดจำคำตอบไม่ให้คำนวณซ้ำ

ปัญหาย่อยซ้อนทับ (Overlapping Subproblems)

คุณสมบัติที่ปัญหาย่อยเดียวกันถูกแก้ซ้ำหลายครั้ง เป็นเงื่อนไขจำเป็นของ DP

โครงสร้างย่อยเหมาะสมที่สุด (Optimal Substructure)

คุณสมบัติที่คำตอบที่ดีสุดของปัญหาสร้างจากคำตอบที่ดีสุดของปัญหาย่อย

การจดจำ (Memoization)

เทคนิคเก็บคำตอบของปัญหาย่อยที่เคยคำนวณแล้ว ป้องกันการคำนวณซ้ำ (Top-Down)

การเขียนตาราง (Tabulation)

เทคนิคเติมตารางจากกรณีฐานขึ้นไป (Bottom-Up) อีกรูปแบบของ DP

ขั้นตอนวิธีแบบโลภ (Greedy Algorithm)

ขั้นตอนวิธีที่เลือกตัวเลือกที่ดีที่สุดในขณะนั้นโดยไม่ย้อนกลับ

แบ่งแยกและเอาชนะ (Divide and Conquer)

กลยุทธ์แบ่งปัญหาใหญ่เป็นปัญหาย่อย แก้แยกกัน แล้วรวมคำตอบ

การเรียกตัวเองซ้ำ (Recursion)

การที่ฟังก์ชันเรียกตัวเองเพื่อแก้ปัญหาย่อย

คณิตศาสตร์

ผลบวก (Summation / Σ)

ผลรวมของลำดับจำนวน เช่น $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$

ผลคูณ (Product / Π)

ผลคูณของลำดับจำนวน

ตัวหารร่วมมาก (GCD / Greatest Common Divisor)
จำนวนเต็มบวกที่มากที่สุดที่หารทั้งสองจำนวนลงตัว คำนวณด้วยขั้นตอนวิธียูคลิด

ตัวคูณร่วมน้อย (LCM / Least Common Multiple)
จำนวนเต็มบวกที่น้อยที่สุดที่หารด้วยทั้งสองจำนวนลงตัด คำนวณจาก $\mathrm{LCM}(a,b) = a \times b / \mathrm{GCD}(a,b)$

จำนวนเฉพาะ (Prime Number)
จำนวนเต็มมากกว่า 1 ที่มีตัวหารเป็น 1 และตัวมันเองเท่านั้น

ตะแกรงเอราทอสเธเนส (Sieve of Eratosthenes)
ขั้นตอนวิธีหาจำนวนเฉพาะทั้งหมดถึง n ทำงานใน $\mathcal{O}(n \log \log n)$

การแยกตัวประกอบ (Prime Factorization)
การแทนจำนวนเต็มเป็นผลคูณของจำนวนเฉพาะ

ตัวผกผันมอดูลาร์ (Modular Inverse)
จำนวน x ที่ทำให้ $(a \times x) \bmod m = 1$ ใช้แทนการหารในเลขคณิตมอดูลาร์

ฟังก์ชันออยเลอร์โทเทียนท์ (Euler's Totient Function / φ)
นับจำนวนจำนวนเต็มตั้งแต่ 1 ถึง n ที่เป็นจำนวนเฉพาะสัมพัทธ์กับ n

การเรียงสับเปลี่ยน (Permutation)
การจัดเรียงสมาชิกทั้งหมดของเซต จำนวนวิธี $= n!$

การจัดหมู่ (Combination)
การเลือกสมาชิก k ตัวจากเซต n ตัวโดยไม่สนลำดับ จำนวนวิธี $= C(n,k)$

การยกกำลังเร็ว (Fast Exponentiation)
คำนวณ $a^n \bmod m$ ได้ใน $\mathcal{O}(\log n)$ โดยใช้การยกกำลังสองซ้ำ

ทฤษฎีบทเล็กของแฟร์มา (Fermat's Little Theorem)
$a^{p-1} \equiv 1 \pmod{p}$ เมื่อ p เป็นจำนวนเฉพาะและ $\gcd(a,p) = 1$ ใช้หาตัวผกผันมอดูลาร์

กราฟ

กราฟ (Graph)
โครงสร้าง $G = (V, E)$ ประกอบด้วยจุดยอด V และเส้นเชื่อม E ใช้จำลองความสัมพันธ์

จุดยอด (Vertex / Node)
จุดในกราฟที่แทนวัตถุหรือสถานะ

เส้นเชื่อม (Edge)
เส้นเชื่อมระหว่างจุดยอดสองจุด แทนความสัมพันธ์

กราฟมีทิศทาง (Directed Graph / Digraph)
กราฟที่เส้นเชื่อมมีทิศทาง จากจุดยอดหนึ่งไปอีกจุดหนึ่ง

กราฟมีน้ำหนัก (Weighted Graph)
กราฟที่เส้นเชื่อมมีค่าน้ำหนักกำกับ

รายการเชื่อมโยง (Adjacency List)
วิธีเก็บกราฟโดยเก็บเพื่อนบ้านของแต่ละจุดยอด ประหยัดหน่วยความจำสำหรับกราฟเบาบาง

เมทริกซ์เชื่อมโยง (Adjacency Matrix)
วิธีเก็บกราฟด้วยเมทริกซ์ 2 มิติ เข้าถึงได้ $\mathcal{O}(1)$ แต่ใช้พื้นที่ $\mathcal{O}(n^2)$

การค้นหาตามความลึก (DFS / Depth-First Search)
ขั้นตอนวิธีสำรวจกราฟโดยเดินลึกที่สุดก่อนแล้วย้อนกลับ ทำงานใน $\mathcal{O}(V + E)$

การค้นหาตามความกว้าง (BFS / Breadth-First Search)
ขั้นตอนวิธีสำรวจกราฟโดยสำรวจทีละระดับ หาระยะทางสั้นสุดในกราฟไม่มีน้ำหนักได้

องค์ประกอบเชื่อมโยง (Connected Component)
เซตย่อยของจุดยอดที่เชื่อมโยงกันทั้งหมด

การเรียงลำดับทอพอโลยี (Topological Sort)
การเรียงจุดยอดของกราฟมีทิศทางแบบไม่มีวงจร ให้เส้นเชื่อมชี้จากซ้ายไปขวาเท่านั้น

เส้นทางสั้นสุด (Shortest Path)
เส้นทางระหว่างจุดยอดสองจุดที่มีผลรวมน้ำหนักต่ำสุด

ขั้นตอนวิธีไดค์สตรา (Dijkstra's Algorithm)
หาระยะทางสั้นสุดจากจุดเริ่มต้นเดียวบนกราฟน้ำหนักไม่ติดลบ ทำงานใน $\mathcal{O}((V + E) \log V)$

ขั้นตอนวิธีเบลแมน-ฟอร์ด (Bellman-Ford Algorithm)
หาระยะทางสั้นสุดจากจุดเริ่มต้นเดียว รองรับน้ำหนักติดลบและตรวจจับวงจรเชิงลบได้ ทำงานใน $\mathcal{O}(VE)$

ขั้นตอนวิธีฟลอยด์-วอร์แชล (Floyd-Warshall Algorithm)
หาระยะทางสั้นสุดทุกคู่บนกราฟ ทำงานใน $\mathcal{O}(V^3)$

วงจรเชิงลบ (Negative Cycle)
วงจรที่ผลรวมน้ำหนักเป็นลบ ทำให้ไม่มีเส้นทางสั้นสุด

ต้นไม้แผ่ซ้อนต่ำสุด (Minimum Spanning Tree / MST)
กราฟย่อยเป็นต้นไม้ที่ครอบคลุมทุกจุดยอดและมีผลรวมน้ำหนักต่ำสุด

ขั้นตอนวิธีครัสคาล (Kruskal's Algorithm)
หา MST โดยเรียงเส้นเชื่อมตามน้ำหนักแล้วเลือกเส้นที่ไม่ทำให้เกิดวงจร ใช้ Union-Find

ขั้นตอนวิธีพริม (Prim's Algorithm)
หา MST โดยขยายจากจุดเริ่มต้นเลือกเส้นน้ำหนักน้อยสุดที่ออกจาก MST ใช้ Priority Queue

กราฟทวิภาคี (Bipartite Graph)
กราฟที่สามารถแบ่งจุดยอดเป็น 2 เซตโดยไม่มีเส้นเชื่อมในเซตเดียวกัน

จุดตัด (Articulation Point / Cut Vertex)
จุดยอดที่เมื่อลบออกทำให้กราฟไม่เชื่อมโยง

สะพาน (Bridge)
เส้นเชื่อมที่เมื่อลบออกทำให้กราฟไม่เชื่อมโยง

ส่วนประกอบเชื่อมโยงเข้ม (Strongly Connected Components / SCC)
เซตย่อยของจุดยอดที่เชื่อมโยงกันได้ทั้งสองทิศทาง

การไหลสูงสุด (Maximum Flow)
ปัญหาหาปริมาณการไหลสูงสุดจากจุดกำเนิดไปจุดรับในเครือข่ายการไหล

เครือข่ายการไหล (Flow Network)
กราฟมีทิศทางที่เส้นเชื่อมมีความจุ มีจุดกำเนิดและจุดรับ

ทฤษฎีบทการไหลสูงสุด-ตัดขั้นต่ำ (Max-Flow Min-Cut Theorem)
ค่าการไหลสูงสุดเท่ากับค่าตัดขั้นต่ำ

ขั้นตอนวิธีไดนิก (Dinic's Algorithm)
ขั้นตอนวิธีหาการไหลสูงสุดที่มีประสิทธิภาพ ทำงานใน $\mathcal{O}(V^2E)$

การจับคู่ (Matching)
เซตของเส้นเชื่อมที่ไม่มีจุดยอดร่วมกัน

การจับคู่สองส่วน (Bipartite Matching)
การจับคู่บนกราฟทวิภาคี หาได้ด้วย Hopcroft-Karp หรือ Max-Flow

สายอักขระ

สายอักขระ (String)
ลำดับของอักขระที่เรียงต่อกัน

การจับคู่รูปแบบ (Pattern Matching)
ปัญหาค้นหาตำแหน่งที่สายอักขระย่อยปรากฏในสายอักขระหลัก

ขั้นตอนวิธีเคเอ็มพี (KMP Algorithm)
ขั้นตอนวิธีจับคู่รูปแบบทำงานใน $\mathcal{O}(n+m)$ โดยใช้ฟังก์ชันล้มเหลว

ฟังก์ชันล้มเหลว (Failure Function)
ตารางใน KMP ที่เก็บความยาวนำหน้าย่อยที่ยาวที่สุดที่เป็นทั้งคำนำหน้าและคำต่อท้าย

ขั้นตอนวิธีราบิน-คาร์ป (Rabin-Karp Algorithm)
ขั้นตอนวิธีจับคู่รูปแบบโดยใช้การแฮชเลื่อนได้ ทำงานเฉลี่ย $\mathcal{O}(n+m)$

ฟังก์ชันแซด (Z-Function)
แถวลำดับที่ $z[i]$ เก็บความยาวสายอักขระย่อยที่ยาวที่สุดที่ตรงกับนำหน้า เริ่มจากตำแหน่ง i

แถวลำดับคำต่อท้าย (Suffix Array)
แถวลำดับเก็บดัชนีของคำต่อท้ายทั้งหมดของสายอักขระ เรียงตามพจนานุกรม

พาลินโดรม (Palindrome)
สายอักขระที่อ่านจากหน้าไปหลังและหลังไปหน้าได้ค่าเดียวกัน

คำนำหน้า (Prefix)
สายอักขระย่อยที่เริ่มจากตำแหน่งแรกของสายอักขระ

คำต่อท้าย (Suffix)
สายอักขระย่อยที่สิ้นสุดที่ตำแหน่งสุดท้ายของสายอักขระ

การแฮช (Hashing)
การแปลงข้อมูลเป็นค่าจำนวนเต็มขนาดคงที่เพื่อการเปรียบเทียบที่รวดเร็ว

การแฮชเลื่อนได้ (Rolling Hash)
เทคนิคแฮชที่คำนวณค่าแฮชของสายอักขระย่อยถัดไปจากค่าแฮชก่อนหน้าได้ใน $\mathcal{O}(1)$

เรขาคณิต

เรขาคณิตเชิงคำนวณ (Computational Geometry)
การแก้ปัญหาเรขาคณิตด้วยขั้นตอนวิธีและการเขียนโปรแกรม

จุด (Point)
วัตถุ 0 มิติบนระนาบ กำหนดด้วยพิกัด (x, y)

เวกเตอร์ (Vector)
วัตถุที่มีทิศทางและขนาด ใช้อธิบายการเคลื่อนที่หรือการแปลงตำแหน่ง

ผลคูณจุด (Dot Product)
การดำเนินการเวกเตอร์ $\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y$ ใช้คำนวณมุมและการฉาย

ผลคูณเชิงเวกเตอร์ (Cross Product)
การดำเนินการเวกเตอร์ $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x$ ใช้ตรวจสอบทิศทางการหมุนและพื้นที่

การทดสอบทวนเข็มนาฬิกา (CCW Test)

ใช้ cross product ตัดสินว่าจุด r อยู่ทางซ้ายของเส้นตรง pq หรือไม่

ระยะทางยูคลิเดียน (Euclidean Distance)

ระยะทางระหว่างจุดสองจุด คำนวณจาก $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

เส้นตรง (Line)

เซตของจุดที่สอดคล้องกับสมการ $ax + by + c = 0$

ส่วนของเส้นตรง (Line Segment)

เส้นตรงที่มีจุดปลายสองจุดและมีความยาวจำกัด

จุดร่วมเส้น (Collinear)

จุดสามจุดขึ้นไปที่อยู่บนเส้นตรงเดียวกัน ตรวจสอบด้วย cross product $= 0$

รูปหลายเหลี่ยม (Polygon)

รูปปิดบนระนาบที่ประกอบด้วยส่วนของเส้นตรงเชื่อมต่อกัน

เปลือกนูน (Convex Hull)

รูปหลายเหลี่ยมนูนที่เล็กที่สุดที่ครอบคลุมจุดทั้งหมด

รูปหลายเหลี่ยมนูน (Convex Polygon)

รูปหลายเหลี่ยมที่มุมภายในทุกมุมน้อยกว่า 180 องศา

รูปหลายเหลี่ยมเว้า (Concave Polygon)

รูปหลายเหลี่ยมที่มีมุมภายในบางมุมมากกว่า 180 องศา

บรรณานุกรม

[1] Arefin, A. S. (2006). *Art of Programming Contest*. Gyankosh Prakashani.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

[3] Halim, S., & Halim, F. (2020). *Competitive programming 4: The lower bound of programming contests in the 2020s*. CreateSpace Independent Publishing Platform.

[4] Kleinberg, J., & Tardos, É. (2005). *Algorithm design*. Pearson.

[5] Laaksonen, A. (2018). *Competitive programmer's handbook*. CP Book.

[6] Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

[7] Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer.

[8] AtCoder. (2024). *Competitive programming platform, Retrieved July 9, 2024*. https://atcoder.jp/

[9] CodeChef. (2024). *CodeChef — Learn and practice coding with problems, Retrieved July 9, 2024*. https://www.codechef.com/

[10] Codeforces. (2024). *Competitive programming platform, Retrieved July 9, 2024*. https://codeforces.com/

[11] CP-Algorithms. (2024). *Algorithms and data structures for competitive programming, Retrieved July 9, 2024*. https://cp-algorithms.com/

[12] CSES Problem Set. (2024). *Code submission evaluation system, Retrieved July 9, 2024*. https://cses.fi/problemset/

[13] ICPC. (2024). *International Collegiate Programming Contest, Retrieved July 9, 2024*. https://icpc.global/

[14] IOI. (2024). *International Olympiad in Informatics, Retrieved July 9, 2024*. https://ioinformatics.org/

[15] Kattis. (2024). *Open automated programming contest system, Retrieved July 9, 2024*. https://open.kattis.com/

[16] LeetCode. (2024). *The world's leading online programming learning platform, Retrieved July 9, 2024*. https://leetcode.com/

[17] Programming.in.th. (2567). *โปรแกรม มิ่ง อิน ที เอซ: ศูนย์ รวม ของ โจทย์ และ เนื้อหา สำหรับ การเขียน โปรแกรม เพื่อ การ แข่งขัน และ วิทยาการ คอมพิวเตอร์, สืบค้น เมื่อ 9 กรกฎาคม 2567*. https://programming.in.th/

[18] Project Euler. (2024). *Project Euler, Retrieved July 9, 2024*. https://projecteuler.net/

[19] Topcoder. (2024). *Competitive programming and crowdsourcing platform, Retrieved July 9, 2024*. https://www.topcoder.com/

[20] UVa Online Judge. (2024). *Universidad de Valladolid Online Judge, Retrieved July 9, 2024*. https://onlinejudge.org/

[21] USACO Guide. (2024). *A free, comprehensive, modern resource for learning competitive programming, Retrieved July 9, 2024*. https://usaco.guide/

ดัชนี

A

Adjacency List . . . . . *see* รายการเชื่อมโยง
Adjacency Matrix . . . . . *see* เมทริกซ์เชื่อมโยง
Amortized Analysis*see* การวิเคราะห์ Amortized
Arc . . . . . *see* ส่วนโค้ง
Array . . . . . *see* แถวลำดับ
Articulation Point . . . . . *see* จุดตัด

B

Backtracking . . . . . *see* การย้อนกลับ
Bellman-Ford Algorithm *see* ขั้นตอนวิธีของเบลแมน-ฟอร์ด
Best Case . . . . . *see* กรณีดีสุด
BFS . . . . . *see* ค้นหาตามความกว้าง
Big-O Notation . . . . . *see* สัญกรณ์โอใหญ่
Binary Indexed Tree *see* ต้นไม้ทวิภาคจากดัชนี
Binary Search . . . . *see* การค้นหาแบบทวิภาค
Binary Search on Answer . *see* การค้นหาแบบทวิภาคบนคำตอบ
Bipartite Graph . . . . . *see* กราฟทวิภาคี
Bipartite Matching . . . . . *see* การจับคู่สองส่วน
Bottom-Up DP*see* การเขียนตาราง (Tabulation)
Bridge . . . . . *see* สะพาน
Brute Force . . . . . *see* การค้นหาแบบครบถ้วน
Bubble Sort . . . . . *see* การเรียงลำดับแบบฟอง

C

CCW Test . . . *see* การทดสอบทวนเข็มนาฬิกา
Chord . . . . . *see* คอร์ด
Circle . . . . . *see* วงกลม
Circumcircle . . . . . *see* วงกลมล้อมรอบ
Collinear . . . . . *see* จุดร่วมเส้น
Combination . . . . . *see* การจัดหมู่
Competitive Programming*see* การโปรแกรมเชิงแข่งขัน
Complete Search . *see* การค้นหาแบบครบถ้วน
Computational Geometry . . *see* เรขาคณิตเชิงคำนวณ
Concave Polygon . . . . *see* รูปหลายเหลี่ยมเว้า
Connected Component . *see* องค์ประกอบเชื่อมโยง
Convex Hull . . . . . *see* เปลือกนูน
Convex Polygon . . . . . *see* รูปหลายเหลี่ยมนูน
Counting Sort . . . . *see* การเรียงลำดับแบบนับ
Cross Product . . . . . *see* ผลคูณเชิงเวกเตอร์
Custom Comparator . *see* การเปรียบเทียบแบบกำหนดเอง

D

Data Structure . . . . . *see* โครงสร้างข้อมูล
DFS . . . . . *see* ค้นหาตามความลึก
Dijkstra's Algorithm*see* ขั้นตอนวิธีของไดค์สตรา
Divide and Conquer . *see* แบ่งแยกและเอาชนะ
Dot Product . . . . . *see* ผลคูณจุด
DSU *see* Union-Find/DSU (Disjoint Set Union)
Dynamic Programming . *see* การโปรแกรมแบบพลวัต

E

Edge . . . . . *see* เส้นเชื่อม
Edmonds-Karp Algorithm . *see* ขั้นตอนวิธีเอดมอนด์-คาร์ป
EPS . . . . . *see* เอพซิลอน
Euclidean Distance . . . *see* ระยะทางยูคลิเดียน
Euler's Totient Function . *see* ฟังก์ชันออยเลอร์โทเทียนท์

F

Failure Function . . . . . *see* ฟังก์ชันล้มเหลว
Fenwick Tree . . . . *see* ต้นไม้ทวิภาคจากดัชนี
Fermat's Little Theorem *see* ทฤษฎีบทเล็กของแฟร์มา
Floating Point . . . . . *see* จุดทศนิยม

Flow Network . . . . . *see* เครือข่ายการไหล
Floyd-Warshall Algorithm *see* ขั้นตอนวิธีของฟลอยด์-วอร์แชล
Ford-Fulkerson Algorithm . . . *see* ขั้นตอนวิธีฟอร์ด-ฟัลเกอร์สัน

G

GCD . . . . . *see* ตัวหารร่วมมาก
Graham's Scan 510
Graph . . . . . *see* กราฟ
Greedy Algorithm . . . . *see* ขั้นตอนวิธีแบบโลภ

H

Hash Table . . . . . *see* ตารางแฮช
Hashing . . . . . *see* การแฮช
Heron's Formula . . . . . *see* สูตรของ Heron

I

Incircle . . . . . *see* วงกลมจารึก
Insertion Sort . . . *see* การเรียงลำดับแบบแทรก

K

Kite . . . . . *see* สี่เหลี่ยมรูปว่าว
KMP Algorithm . . . . . *see* ขั้นตอนวิธีเคเอ็มพี
Kruskal's Algorithm *see* ขั้นตอนวิธีของครัสคาล

L

LCM . . . . . *see* ตัวคูณร่วมน้อย
Line . . . . . *see* เส้นตรง
Line Segment . . . . . *see* ส่วนของเส้นตรง

M

Map . . . . . *see* แมป
Master Theorem . . . . *see* ทฤษฎีบทมาสเตอร์
Matching . . . . . *see* การจับคู่
Max-Flow Min-Cut Theorem *see* ทฤษฎีบทการไหลสูงสุด-ตัดขั้นต่ำ
Maximum Flow . . . . . *see* การไหลสูงสุด
Meet in the Middle . . . . . *see* พบกันกลางทาง
Memoization . . . . . *see* การจดจำ
Merge Sort . . . . *see* การเรียงลำดับแบบผสาน
Minimum Spanning Tree . *see* ต้นไม้แผ่คลุมต่ำสุด
Modular Arithmetic . . . . *see* เลขคณิตมอดูลาร์
Modular Inverse . . . . . *see* ตัวผกผันมอดูลาร์

N

Negative Cycle . . . . . *see* วงจรเชิงลบ

O

Omega Notation . . . . . *see* สัญกรณ์โอเมกา
Optimal Substructure *see* โครงสร้างย่อยเหมาะสมที่สุด
Overflow . . . . . *see* การล้น
Overlapping Subproblems *see* ปัญหาย่อยซ้อนทับ

P

Palindrome . . . . . *see* พาลินโดรม
Parallelogram . . . . . *see* สี่เหลี่ยมด้านขนาน
Path Compression . . . . *see* การบีบอัดเส้นทาง
Pattern Matching . . . . . *see* การจับคู่รูปแบบ
Perimeter . . . . . *see* เส้นรอบรูป
Permutation . . . . . *see* การเรียงสับเปลี่ยน
Point . . . . . *see* จุด
Polygon . . . . . *see* รูปหลายเหลี่ยม
Polygon Area . . . . . *see* พื้นที่รูปหลายเหลี่ยม
Prefix Sum . . . . . *see* ผลรวมสะสม
Prim's Algorithm . . . . *see* ขั้นตอนวิธีของพริม
Prime Factorization . . *see* การแยกตัวประกอบ
Prime Number . . . . . *see* จำนวนเฉพาะ
Priority Queue . *see* แถวคอยลำดับความสำคัญ
Pruning . . . . . *see* การตัดแต่งกิ่ง

Q

Quadrilateral . . . . . *see* สี่เหลี่ยม
Queue . . . . . *see* แถวคอย
Quick Sort . . . . . *see* การเรียงลำดับแบบเร็ว

R

Radix Sort . . . . . *see* การเรียงลำดับแบบฐาน
Rectangle . . . . . *see* สี่เหลี่ยมมุมฉาก

Recursion . . . . . *see* การเรียกตัวเองซ้ำ
Rhombus . . . . . *see* สี่เหลี่ยมขนมเปียกปูน
Rolling Hash . . . . . *see* การแฮชเลื่อนได้

S

SCC . . . . . *see* ส่วนประกอบเชื่อมโยงเข้ม
Sector . . . . . *see* เซกเตอร์
Segment . . . . . *see* เซกเมนต์
Segment Tree . . . . . *see* ต้นไม้ช่วง
Selection Sort . . . *see* การเรียงลำดับแบบเลือก
Sentinel . . . . . *see* ค่าเฝ้า
Set . . . . . *see* เซต
Shortest Path . . . . . *see* เส้นทางสั้นสุด
Sieve of Eratosthenes . . . *see* ตะแกรงเอราทอสเธเนส
Sorting . . . . . *see* การเรียงลำดับ
Space Complexity *see* ความซับซ้อนของปริภูมิ
Sparse Table 166
Square . . . . . *see* สี่เหลี่ยมจัตุรัส
Stable Sort . . . . . *see* การเรียงลำดับเสถียร
Stack . . . . . *see* กองซ้อน
STL (Standard Template Library) 14
String . . . . . *see* สายอักขระ
Suffix Array . . . . . *see* แถวลำดับคำต่อท้าย

T

Tabulation . . . . . *see* การเขียนตาราง
Theta Notation . . . . . *see* สัญกรณ์ธีตา
Time Complexity . . *see* ความซับซ้อนของเวลา
Top-Down DP . . *see* การจดจำ (Memoization)
Topological Sort . *see* การเรียงลำดับทอพอโลยี
Trapezium . . . . . *see* สี่เหลี่ยมคางหมู
Triangle . . . . . *see* สามเหลี่ยม
typedef 20

U

Union by Rank 390
Union-Find . *see* Union-Find/DSU (Disjoint Set Union)
Union-Find/DSU (Disjoint Set Union) . . . 388

V

Vector . . . . . *see* เวกเตอร์, *see* เวกเตอร์

W

Worst Case . . . . . *see* กรณีแย่สุด

Z

Z-Function . . . . . *see* ฟังก์ชันแซด

ก

กฎการคูณ (Multiplication Rule) 32
กฎการบวก (Addition Rule) 32
กรณีดีสุด (Best Case) 28
กรณีเฉลี่ย (Average Case) 28
กรณีแย่สุด (Worst Case) 28
กราฟ (Graph) 338
กราฟทวิภาคี (Bipartite Graph) 401
กราฟมีทิศทาง (Directed Graph) 338
กราฟมีน้ำหนัก (Weighted Graph) 338
กองซ้อน (Stack) 126
การค้นหาแบบครบถ้วน (Complete Search/Brute Force) 180
การค้นหาแบบทวิภาค (Binary Search) 73
การค้นหาแบบทวิภาคบนคำตอบ (Binary Search on Answer) 79
การจดจำ (Memoization) 257
การจัดหมู่ (Combination) 241
การจับคู่ (Matching) 437
การจับคู่รูปแบบ (Pattern Matching) 459
การจับคู่สองส่วน (Bipartite Matching) 439
การตัดแต่งกิ่ง (Pruning) 194
การทดสอบทวนเข็มนาฬิกา (CCW Test) . . 498
การบีบอัดเส้นทาง (Path Compression) . . . 388
การยกกำลังเร็ว (Fast Exponentiation) . . . 108
การย้อนกลับ (Backtracking) 191
การรับค่าและแสดงผล (I/O) 15
การล้น (Overflow) 17
การวิเคราะห์ Amortized (Amortized Analysis) 48
การออกแบบขั้นตอนวิธี (Design of Algorithms) 12

การเขียนตาราง (Tabulation) 259
การเขียนโปรแกรมแสดงผล (Implementation of Algorithms) 12
การเปรียบเทียบแบบกำหนดเอง (Custom Comparator) 61
การเรียกตัวเองซ้ำ (Recursion) 36
การเรียงลำดับ (Sorting) 57
การเรียงลำดับทอพอโลยี (Topological Sort) 360
การเรียงลำดับเสถียร (Stable Sort) 57
การเรียงลำดับแบบฐาน (Radix Sort) 69
การเรียงลำดับแบบนับ (Counting Sort) 69
การเรียงลำดับแบบผสาน (Merge Sort) 69
การเรียงลำดับแบบฟอง (Bubble Sort) 66
การเรียงลำดับแบบเร็ว (Quick Sort) 69
การเรียงลำดับแบบเลือก (Selection Sort) . . 66
การเรียงลำดับแบบแทรก (Insertion Sort) . . 66
การเรียงสับเปลี่ยน (Permutation) 188
การแยกตัวประกอบ (Prime Factorization) . 229
การแฮช (Hashing) 464
การแฮชเลื่อนได้ (Rolling Hash) 464
การโปรแกรมเชิงแข่งขัน (Competitive Programming) 12
การโปรแกรมแบบพลวัต (Dynamic Programming/DP) 254
การไหลสูงสุด (Maximum Flow) 422
ขั้นตอนวิธีของครัสคาล (Kruskal's Algorithm)393
ขั้นตอนวิธีของพริม (Prim's Algorithm) . . . 396
ขั้นตอนวิธีของฟลอยด์-วอร์แชล (Floyd-Warshall Algorithm) 372
ขั้นตอนวิธีของเบลแมน-ฟอร์ด (Bellman-Ford Algorithm) 367
ขั้นตอนวิธีของไดค์สตรา (Dijkstra's Algorithm) 370
ขั้นตอนวิธีฟอร์ด-ฟัลเกอร์สัน (Ford-Fulkerson Algorithm) 427
ขั้นตอนวิธีเคเอ็มพี (KMP Algorithm) 460
ขั้นตอนวิธีเอดมอนด์-คาร์ป (Edmonds-Karp Algorithm) 428
ขั้นตอนวิธีแบบโลภ (Greedy Algorithm) . . . 305
ขั้นตอนวิธีไดนิก (Dinic's Algorithm) 430
ความซับซ้อนของปริภูมิ (Space Complexity) 51
ความซับซ้อนของเวลา (Time Complexity) . . 26
คอร์ด (Chord) 501
คำต่อท้าย (Suffix) 470
คำนำหน้า (Prefix) 460
ค่าเฝ้า (Sentinel) 21
ค้นหาตามความกว้าง (BFS/Breadth-First Search) 352
ค้นหาตามความลึก (DFS/Depth-First Search) 346
จำนวนเฉพาะ (Prime Number) 226
จุด (Point) 491
จุดตัด (Articulation Point/Cut Vertex) 403
จุดทศนิยม (Floating Point) 17
จุดยอด (Vertex/Node) 338
จุดร่วมเส้น (Collinear) 491
ตะแกรงเอราทอสเธเนส (Sieve of Eratosthenes) 226
ตัวคูณร่วมน้อย (LCM/Least Common Multiple) 223
ตัวผกผันมอดูลาร์ (Modular Inverse) 238
ตัวหารร่วมมาก (GCD/Greatest Common Divisor) 223
ตารางกระจาย (Sparse Table) 166
ตารางแฮช (Hash Table) 143
ต้นไม้ช่วง (Segment Tree) 170
ต้นไม้ทวิภาคจากดัชนี (Binary Indexed Tree/ Fenwick Tree) 167
ต้นไม้แผ่คลุมต่ำสุด (Minimum Spanning Tree/ MST) 392
ทฤษฎีบทการไหลสูงสุด-ตัดขั้นต่ำ (Max-Flow Min-Cut Theorem) 435
ทฤษฎีบทมาสเตอร์ (Master Theorem) . . . 103
ทฤษฎีบทเล็กของแฟร์มา (Fermat's Little Theorem) 235
ประเภทข้อมูลจำนวนเต็ม 18
ปัญหาย่อยซ้อนทับ (Overlapping Subproblems) 255

ผลคูณ (Product/Π) 212
ผลคูณจุด (Dot Product) 491
ผลคูณเชิงเวกเตอร์ (Cross Product) 498
ผลบวก (Summation/Σ) 212
ผลรวมสะสม (Prefix Sum) 161
พบกันกลางทาง (Meet in the Middle) 197
พาลินโดรม (Palindrome) 476
พื้นที่รูปหลายเหลี่ยม (Polygon Area) 506
ฟังก์ชันล้มเหลว (Failure Function) 460
ฟังก์ชันออยเลอร์โทเทียนท์ (Euler's Totient Function/φ) 232
ฟังก์ชันแซด (Z-Function) 467
มาโคร (Macro) 20
ระยะทางยูคลิเดียน (Euclidean Distance) . . 491
รายการเชื่อมโยง (Adjacency List) 341
รูปหลายเหลี่ยม (Polygon) 489
รูปหลายเหลี่ยมนูน (Convex Polygon) 489
รูปหลายเหลี่ยมเว้า (Concave Polygon) . . . 489
วงกลม (Circle) 501
วงกลมจารึก (Incircle) 503
วงกลมล้อมรอบ (Circumcircle) 503
วงจรเชิงลบ (Negative Cycle) 367
สะพาน (Bridge) 403
สัญกรณ์ธีตา (Theta Notation) 31
สัญกรณ์โอเมกา (Omega Notation) 31
สัญกรณ์โอใหญ่ (Big-O Notation) 28
สามเหลี่ยม (Triangle) 503
สายอักขระ (String) 455
สี่เหลี่ยม (Quadrilateral) 506
สี่เหลี่ยมขนมเปียกปูน (Rhombus) 506
สี่เหลี่ยมคางหมู (Trapezium) 506
สี่เหลี่ยมจัตุรัส (Square) 506
สี่เหลี่ยมด้านขนาน (Parallelogram) 506
สี่เหลี่ยมมุมฉาก (Rectangle) 506
สี่เหลี่ยมรูปว่าว (Kite) 506
สูตรของ Heron (Heron's Formula) 503
ส่วนของเส้นตรง (Line Segment) 495
ส่วนประกอบเชื่อมโยงเข้ม (Strongly Connected Components/SCC) 406
ส่วนโค้ง (Arc) 501
องค์ประกอบเชื่อมโยง (Connected Component) 356
เครือข่ายการไหล (Flow Network) 422
เซกเตอร์ (Sector) 501
เซกเมนต์ (Segment) 501
เซต (Set) 131
เปลือกนูน (Convex Hull) 489
เมทริกซ์เชื่อมโยง (Adjacency Matrix) 341
เรขาคณิตเชิงคำนวณ (Computational Geometry) 489
เลขคณิตมอดูลาร์ (Modular Arithmetic) 17
เวกเตอร์ (Vector) 116, 491
เส้นตรง (Line) 495
เส้นทางสั้นสุด (Shortest Path) 366
เส้นรอบรูป (Perimeter) 506
เส้นเชื่อม (Edge) 338
เอพซิลอน (EPS/Epsilon) 17
แถวคอย (Queue) 126
แถวคอยลำดับความสำคัญ (Priority Queue) 139
แถวลำดับ (Array) 116
แถวลำดับคำต่อท้าย (Suffix Array) 470
แบ่งแยกและเอาชนะ (Divide and Conquer) . 92
แมป (Map) 136
โครงสร้างข้อมูล (Data Structure) 115
โครงสร้างย่อยเหมาะสมที่สุด (Optimal Substructure) 261
โจทย์แนะนำ
  Complete Search 206
  Divide and Conquer 111
  Dynamic Programming 300
  Greedy Algorithm 332
  Maximum Flow 450
  Range Queries 178
  Shortest Path 386
  Union-Find และ MST 399
  ขั้นตอนวิธีกราฟขั้นสูง 418
  คณิตศาสตร์ 250
  ทฤษฎีกราฟ 364

สายอักขระ 481
เรขาคณิตเชิงคำนวณ 513
โครงสร้างข้อมูล 156